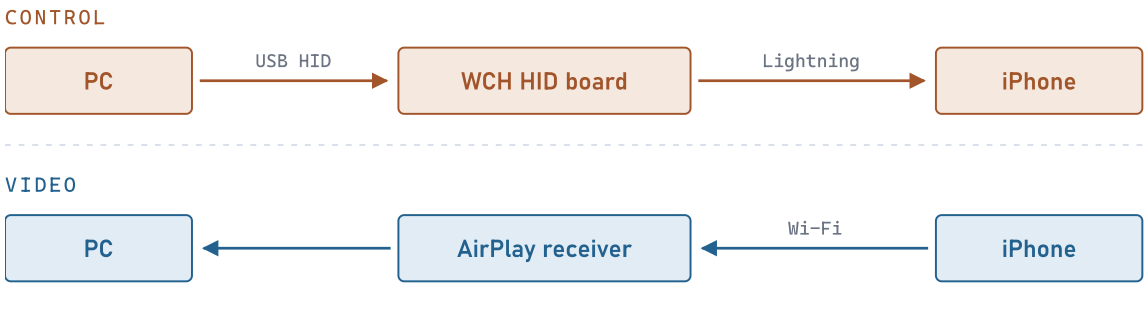
CAPABILITY HANDOFF · 10 SEPTEMBER 2026

iMouse API: what the rack can do

Everything the Phonefarm / iMouse XP kernel lets software do with a rack of real iPhones, verified against the vendor documentation and the measurements taken on this rig, organized by what it is good for: marketing automation, account management, operations, data, testing, and the ideas nobody wrote on the box.

3 use cases verified | **146** work with caveats | **16** refuted or not possible

46 API functions | **24** Shortcut actions | **2** editions mapped



Two independent channels. Control is USB HID into the phone; video is AirPlay back out. Every capability in this document is built from those two primitives plus one side channel: an iOS Shortcut the kernel can trigger with a hotkey.

CONTENTS

What is in this document

01 **Executive summary and which edition you have** 7

What the rack is
What the API fundamentally gives you
The headline: what is achievable
Which edition this rig runs, and what that means
The numbers that shape every design
How to read this document
Map of the fourteen sections
Also worth knowing
When not to use the rack

02 **The API surface, end to end** 14

Device and group management
Mouse input
Keyboard input
Wheel input
Screenshots and screen capture
Vision: image search, OCR and colour matching
Shortcut functions (id 1 to 24)
Casting and AirPlay control
Recovery
Mouse calibration
Callbacks (server-pushed events)
Implicit capabilities
Doc gaps and ambiguities
XP-only capabilities
Also worth knowing

03 **Marketing automation** 30

Publishing through the app's own composer
Launching composers by URL instead of hunting for icons
Scheduling campaigns across device groups
Direct messaging and replies
Engagement loops
Monitoring and inbox alerts
Ad account operations at agency scale
Campaign quality assurance
Coordinated publishing and guardrails
Creative testing at panel scale
Ad-verification, outside the box

04 **Platform-by-platform playbook** 60

Short-form video: TikTok, Instagram, YouTube Shorts
Text-first and native-scheduling platforms: Threads and Facebook
Niche surfaces: Pinterest and Snapchat
Commerce and seller-app forms
Messaging and community broadcast
Feature-gap workarounds on the composer surface
Cross-platform posting patterns
Also worth knowing
Domain index: where else these primitives apply

05 **Account management** 89

Login, logout and credential rotation
Knowing which slots are actually signed in
Capturing one-time codes
Account provisioning
Profile setup
Fleet app version management
Session hygiene
Navigation infrastructure
SMS and OTP retrieval

06 **Commerce, customer and business operations** 114

Listing and catalog operations
Fleet-wide storefront maintenance
Customer messaging and live commerce
Verification, compliance and monitoring
Automated video production
Also worth knowing
Content and asset production

07 **Data collection, monitoring and verification** 130

Reading on-screen text and numbers without OCR
Change detection without recognition
Native-resolution capture for evidence and QA
The clipboard as a general data-exit channel
Turning the phone into a queryable sensor
Ad and storefront verification
Network identity and provenance
Outside the box: the rack as a passive sensor
Web and app content harvesting via the clipboard
Search and pricing intelligence from real handsets
Passive telemetry: training data and health signals
Also worth knowing
Server-side cropping as the default poll shape, not a special case

08 **Testing, QA and research** 164

Campaign link, push and email QA
Visual regression and rig health
Cross-device and localisation QA
Release health: canaries, ad verification and flow recording
Outside-the-box measurement: latency, network transitions and accessibility
Long-running reliability and monitoring
Test infrastructure: mocking and replay
Concurrency and race-condition testing
Diagnostics extraction via the iOS Shortcut
Also worth knowing
Game QA and landscape apps

09 **Fleet operations and self-healing** 206

Rack registry and live health
Account and calibration watchdogs
Scheduling through partial failure
Keeping the rack connected
Self-healing input and hardware
Scheduled fleet maintenance
Proving the fleet is actually alive
Mirroring bandwidth policy
Physical fleet management
Storage housekeeping
Calibration and link-health diagnostics
Remote diagnostics and hands-on control
Guarding fleet configuration
Scaling beyond one rack
Readiness and resource tuning

10 **Outside the box** 250

Fleet-wide synchronized actuation
Turning the rack into other hardware
Driving real consumer apps end-to-end
Filling API gaps: text and vision
Also worth knowing

11 **Building on it: architecture and integration** 263

The two things worth building first
Bridging the kernel to things it cannot reach
Reaching into iOS through the Shortcut channel
Addressing: groups, fan-out and coordinate spaces
The control plane: events, and actions that prove themselves
Boundaries: an authenticated gateway and a measured rack
Further out: the rack as a render lab
Hardening the control plane
Pricing the work before you run it

Capturing operator work and replaying it
Past the boundary of one PC
Also worth knowing
Skipping straight to the vendor's own client: imouse-py

12 **Hard limits, misconceptions and risks** 312

Absent from the API in both editions
Missing on this rig, not from the product
Present, but not the shape people assume
Telemetry and phone-side configuration
Transport and frame fidelity
App state, traffic and builds
The input surface
Walls the API cannot cross
No outbound events, no caller authentication
Input primitives and their ceilings
Licensing, boards and calibration
Vision on this rig
The phone itself is a limit: locks, prompts and reboots
Exposure you cannot configure away
Physical limits: radio, heat and battery
The limits of what a response tells you
The limits of reach: files, audio and events
Also worth knowing
Audio: real at the receiver, narrower than "no audio capture"
The one avenue that could move the walls: talking to the boards directly
The Apple-ecosystem checklist

13 **Prerequisites and setup checklist** 371

Global one-time setup (per rig, not per phone)
New-phone bring-up order
Prerequisites by capability class
Verification recipes reused across capability classes
Diagnostic codes worth memorising
Also worth knowing
Phone provisioning as an automatable job
Network prerequisites for the video channel
Windows host settings that keep 19 boards enumerated
Disk budget and rotation for a frame archive
Clock sync for evidence timestamps

14 **Reference tables** 383

A. Error codes (data.code)
B. Shortcut function ids 1 to 24
C. Phonefarm to XP function mapping
D. Named keys and hotkey combos (send_key)

E. Supported iPhone models and mirror geometry

F. Ports and processes on the PC

Executive summary and which edition you have

This document maps what your iMouse rack can actually be made to do, function by function, against the API that is installed on it and against measurements taken on the rig itself on 7 to 10 September 2026. It covers 165 use cases across marketing, accounts, commerce, data collection, QA, fleet operations and integration, and it is explicit about the sixteen things people commonly expect from a phone rack that this one cannot do. Every claim below is traceable either to the vendor documentation or to a measurement on these two iPhones and nineteen boards.

What the rack is

The rack is real iPhones, not emulators and not a jailbreak. Each phone is a stock handset with no app installed on it. Two entirely separate channels reach each phone, and almost every failure mode in this document is a failure of one channel while the other is healthy.

Control runs over USB. A small WCH HID board (VID 720A, one unique PID per board) sits between the PC and the phone's Lightning port and presents itself to iOS as a mouse and keyboard. `IMService.exe` owns those boards and reports them through `/config/usb/get`. One board drives exactly one phone. Control does not use Wi-Fi at all.

Video runs over Wi-Fi. The phone casts its screen by AirPlay to a receiver advertised by this PC over Bonjour, and `AirplayMirroring.exe` decodes the stream on TCP 17000. Video is reported separately, through `/device/get` (`state` and `air_pid`).

The practical consequence is that a phone can be fully controllable with no picture, or perfectly visible and deaf to every input. Any health display you build must show both states per slot, or it will lie to you.

What the API fundamentally gives you

Stripped of detail, the kernel offers three things:

- **Pixels in.** `/pic/screenshot` returns the current decoded mirror frame for one phone, as raw BMP, JPEG or base64 JSON, optionally cropped by rectangle or written straight to disk.
- **HID events out.** `/mouse/click`, `/mouse/down`, `/mouse/move`, `/mouse/up`, `/mouse/swipe`, `/mouse/wheel`, `/mouse/reset`, `/key/sendkey`, `/key/down` and `/key/up`, each taking a comma-separated `id` list so one request fans out to many phones at once.
- **An iOS Shortcut side channel.** The `/shortcut/*` family drives an iMouse-supplied iOS Shortcut on the phone to do things a pointer cannot: push files into the camera roll or the Files tree, set and read the clipboard (the only route to non-ASCII text), open a URL, toggle some system switches, restart the handset.

Around those sit device and group management, per-device AirPlay settings, kernel configuration, account and licence state, and a WebSocket on the same port that broadcasts callbacks for device, USB and cast events. That is the whole surface. There is no element tree, no log access, no app install, no telemetry and no outbound webhook.

The headline: what is achievable

Of 165 use cases examined, 149 are achievable and 16 are refuted. Only 3 are recorded as verified without qualification, because on a rig whose only oracle is a decoded video frame, nearly everything carries a real caveat worth reading before you build on it.

Verdict	Count	What it means here
Verified	3	Confirmed on this rig or reducible to measured behavior with no open question
With caveats	146	Achievable, with a stated cost, precondition or fragility
Refuted	16	Not possible through either API edition; documented so you stop looking

By the route each one has to take:

Route	Count	Note
Native API	34	Documented XP endpoints only
Needs Shortcut	63	Requires the iMouse iOS Shortcut installed on each phone
Partial	36	Works for part of the job, or only under conditions
Plus external tooling	12	Needs a scheduler, recognizer, gateway or second machine
Needs plug-in	4	Depends on the OCR or image-search plug-in, absent here
Not possible	16	No route in either edition

The single biggest structural fact in that table is the Shortcut column. Thirty-eight percent of everything useful runs through the iOS Shortcut channel, so installing and testing that Shortcut on every phone is not an optional extra, it is the difference between a pointer toy and a working rack.

Strongest opportunities, by domain

- **Publishing and marketing.** Push one asset into every phone's camera roll with a single multi-id upload, put the caption on every clipboard with a single clipboard call, then drive each app's own composer with mirrored taps. Platform-specific sequences for TikTok, Instagram, Threads, Facebook, YouTube Shorts, Pinterest, X, LinkedIn, Telegram and Discord are in section 4. Launching a composer by URL scheme avoids home-screen icon hunting entirely.
- **QA and testing.** This is where the rack is strongest, because everything QA wants is exactly what the rack produces: golden-frame visual regression on raw frames, deep-link and universal-link matrices, push notification render and arrival latency, localization sweeps at native resolution through the Shortcut screenshot, ad slot fill timing, keyboard focus-order audits, and synchronized N-phone race testing that no simulator can reproduce.
- **Data collection and monitoring.** With no OCR plug-in, the exact-text channel is select-all, copy, then read the phone clipboard. That yields page text out of Safari, post identifiers and order numbers out of native feeds, and structured answers from a Shortcut you write yourself. Frame differencing covers the rest: grid diffs, autoplay detection, hang detection.
- **Fleet operations.** Two-channel health boards, pointer-nudge liveness probes that actually prove input is honored, USB flap watchdogs driven by the `usb_change` callback, stuck-click self-healing, cast-on-connect sweepers, Wi-Fi link scoring by delivered frames per second, and physical slot roll-call by torch.

- **Integration.** The kernel WebSocket is a usable event bus with msgid-correlated RPC. Around it you can build a per-phone job queue with one lease per board, a coordinate registry keyed by model and mirror geometry, an idempotent verify-then-escalate action wrapper, and a vision-model loop that takes a raw frame in and emits a tap out.

The clearest limits

- No SMS or iMessage send, and no way to read an incoming message, OTP or notification body. Only the `sms:` composer deep link plus a tap.
- No phone telemetry at all: no battery, temperature, storage, GPS or carrier.
- No structured app state. No accessibility tree, no DOM, no element list, no logs, no crash hooks, no network capture.
- One pointer only. No pinch, rotate, two-finger or pressure gesture exists, and the board walks the pointer at roughly 0.6 ms per pixel, so velocity-dependent flicks are out of reach.
- No Apple Pay, Face ID or side-button confirmation. Anything requiring a physical button press stops the rack.
- A comma-separated `id` list is a fan-out, not a batch: every listed phone gets the same parameters.
- No kernel-originated webhooks, and no caller authentication whatsoever.

Which edition this rig runs, and what that means

This rig runs the **iMouse XP International Edition**. That is settled by probe, not by inference.

- `IMService.exe` listens on TCP **9911** and serves both HTTP and WebSocket there. The WebSocket path is `/api`.
- Port **9912 is closed**. Nothing listens on it, so the Phonefarm document's HTTP endpoint cannot be reached at all.
- Phonefarm-style function names posted to 9911, for example `{"fun":"get_device_list"}`, return HTTP **500** with a list-index error. The XP kernel splits `fun` on a slash and expects a path, so a bare name is rejected before dispatch.
- Documented XP paths answer correctly on the same port: `/api/device/get` returns both iPhones, `/api/config/usb/get` returns nineteen boards, `/api/device/group/get` returns an empty list.

Practical rules that follow. Address everything as `GET http://127.0.0.1:9911/api/<path>?k=v`, or as a JSON POST to `/api` with `fun` set to that same path. Use `id`, not `deviceid`. Read the real result from `data.code`, where 0 is success, and treat the top-level `status` of 200 as nothing more than "the request was accepted". Match on codes, never on message text, because XP returns Chinese-language messages where the Phonefarm document shows English. A guessed path returns 404 with a Chinese "interface does not exist" string rather than a structured error.

The Phonefarm-edition document you were supplied is still worth keeping. It describes the same hardware and broadly the same capabilities, and it is a useful concept map when the XP documentation is terse. It is simply the wrong wire format for this machine, and a handful of its features do not exist here at all, notably server-pushed screenshot streaming and multiple AirPlay receivers.

> `TRAP` **Two sibling APIs, one of them not yours**
>
> Sample code found online, and most of the vendor's own examples, may be written for the Phonefarm edition. It will fail against this rig in three distinct ways: connection refused on 9912, HTTP 500 for bare function names on 9911, and 404 for paths that only exist in the other edition. Section 2 gives the mapping and section 14 the full path tables.

OPEN **The vision plug-ins are not installed**

`/api/pic/ocr` answers HTTP 200 with `data.code` 28, "plug-in not connected". Assume `/pic/find-image`, `/pic/find-image-cv`, `/pic/find-text` and their enhanced variants are in the same state until you probe them. Server-side OCR and template matching are documented capabilities of the XP edition, so this is an install question, not an architectural one, but until it is resolved every "find the button" design has to run client-side on raw frames.

The numbers that shape every design

RAW FRAME FETCH	JPEG FRAME FETCH	POINTER WALK	AIRPLAY SOURCE RATE
2 ms	**40.5 ms**	**0.6 ms/px**	**28.9 vs 4.5 fps**
p50 for BMP, 84 req/s flat out, 1.6 MB per frame	p50, capped near 21 req/s, kernel re-encodes	1000 px move measured at 592 ms	two identical iPhone XRs, same iOS build
MIRROR GEOMETRY	**CONCURRENT POLLERS**	**RACK SIZE**	**SCREEN-CHANGE FLOOR**
500 x 1080	**512 req/s**	**19 boards, 2 phones**	**0.02 vs 20+**
click space, not the 414 x 896 iOS point space	ten pollers, p50 3 ms, no shared bottleneck	three 7-port hubs, 19 of 21 slots populated	idle mean pixel diff against a real reaction

Four of those deserve a sentence each. Frame capture is effectively free on loopback, so build a wall of live thumbnails without guilt, but reuse one buffer per poller because every frame is far above the .NET large object heap threshold. Movement is not free, and a move's latency is meaningless without its distance, so coalesce drags and hovers to the newest position. The two phones deliver frames at rates that differ by a factor of six because of their Wi-Fi links, not because of contention, so never time out a screen check on a 30 fps assumption. And "did the screen change" must be a mean absolute pixel difference against an idle control, never a hash comparison, because the clock and encoder noise change every frame.

WRONG **A success response does not mean the tap landed**

HTTP 200 with `data.code` 0 means the board accepted the report. It says nothing about what iOS did. Measured on 10 September 2026: a phone reached a state where `/mouse/move` visibly moved the pointer and the Home key still worked, while `/mouse/click` produced a 0.09 mean pixel difference, indistinguishable from an untouched screen. `/mouse/up` did not clear it; `/mouse/reset` did, and it stayed cleared. The frame is the only oracle.

How to read this document

Sections 3 through 10 are built from use-case cards. Each card names the capability, carries two pills, then gives the call chain step by step, what it needs, what to watch out for, and how it scales across the rack. The first pill says what route the capability has to take. The second says how confident we are that it works.

Route pill	Meaning
`Native API`	Documented XP endpoints on 9911, nothing else required

Route pill	Meaning
`Needs plug-in`	Requires the OCR or image-search plug-in, which is not connected on this rig
`Pro edition`	Requires a feature tier this installation may not carry
`Needs Shortcut`	Requires the iMouse iOS Shortcut installed and trusted on each phone
`Plus external tooling`	Needs something outside the kernel: a scheduler, a recognizer, a gateway, a second PC
`Partial`	Part of the job is reachable, the rest is not
`Not possible`	No route exists in either edition

Verdict pill	Meaning
`Verified`	Confirmed against the docs or measured on this rig, no open question
`With caveats`	Achievable, but read the "Watch out" list before you commit to it
`Refuted`	A common expectation that does not survive contact with the API

Flagged boxes interrupt the flow deliberately. **Trap** marks something that will cost you an afternoon. **Wrong** marks a belief that measurement overturned, including beliefs held earlier in this project. **Open** marks a genuine unknown. **Tip** marks a shortcut worth taking.

> `TIP` **If you read only three sections**
>
> Read section 2 for the API surface, section 12 for the hard limits, and section 13 for the setup checklist. Section 12 in particular will save you from designing around capabilities the rack does not have, and section 13 lists the five phone-side accessibility settings that are invisible to the API and will silently break every input if any is wrong.

Map of the fourteen sections

#	Section	What it answers
1	Executive summary and which edition you have	What the rack is, what the API gives you, which edition is installed, how to read this
2	The API surface, end to end	Every endpoint family, request forms, response and error semantics, the WebSocket callbacks, the Shortcut channel
3	Marketing automation	Publishing, composers, campaign scheduling, creative splits, engagement on accounts you own
4	Platform-by-platform playbook	Concrete sequences for the major social, messaging and seller apps
5	Account management	Sign-in, session rotation, on-device 2FA, profile setup, app install and update
6	Commerce, customer and business operations	Listings, price sweeps, support desks, live commerce, payment app heartbeats
7	Data collection, monitoring and verification	Clipboard text extraction, frame diffing, screenshots as evidence, multi-geo comparison

#	Section	What it answers
8	Testing, QA and research	Visual regression, deep links, push, latency, localization, accessibility, soak and race tests
9	Fleet operations and self-healing	Health boards, watchdogs, recovery ladders, calibration, licences, maintenance windows
10	Outside the box	Uses the vendor never described: pixel walls, cameras, AirDrop, clipboard data buses
11	Building on it: architecture and integration	Job queues, event bus, coordinate registries, gateways, observability, throughput budgets
12	Hard limits, misconceptions and risks	The 16 refuted use cases, the soft limits, and the legal, Terms-of-Service and detection exposure
13	Prerequisites and setup checklist	What must be true on the PC, the boards, the network and each phone before anything runs
14	Appendix: reference tables	Path tables, error codes, callback list, hotkey table, field glossary, measured constants

One last thing to carry into every section that follows. The kernel has no authentication of any kind, and the documentation's own examples call it across a LAN. Anyone who can reach port 9911 on this PC can drive all nineteen boards. Treat that as a property of the product, firewall the port to trusted hosts, and never put this machine on an untrusted network.

Also worth knowing

When not to use the rack

Everything above holds, and none of it applies to an app whose source you control. If you own the Xcode project, XCUITest run locally, or Appium's XCUITest driver talking to WebDriverAgent, over the same USB connection this rack uses for control, gives you a real element tree, first-party logs, crash reports and app install and reinstall on demand. The rack has no route to any of that. It is not an oversight in the API: `/pic/screenshot` hands you a decoded video frame, not a view hierarchy; `/mouse/click` and `/key/sendkey` hand you a coordinate or a keycode, not an accessibility identifier to assert against; `/shortcut/*` reaches the clipboard and the camera roll, not a debugger or a log stream. Spending months turning those four primitives into element-aware, log-backed automation for your own app produces a weaker Appium, built the hard way, for a problem Appium already solved.

Use the rack for what XCUITest cannot reach at all. Other people's apps, where there is no project to attach a test target to and no way to link WebDriverAgent in. Real carrier and Wi-Fi egress through an actual SIM and antenna, which a simulator, and most cloud devices, do not reproduce. N-phone simultaneity through one comma-separated `id` list, rather than orchestrating N separate simulators or sessions by hand. And physical hardware behavior, AirPlay mirroring quality, a real Lightning HID board, real battery and thermal state under load, that a simulator run on a Mac cannot produce. A commercial device cloud sits between the two: real hardware without owning a rack, billed by the minute, with no guarantee of the same physical unit run to run, a reasonable choice for broad device coverage on a short test pass rather than a fixed rack running continuously. And when the job is really "post to this

account" or "read this account's data" rather than "operate this UI", the platform's own published API is faster, authenticated, and does not depend on a mirrored tap landing, so it belongs ahead of the rack whenever that door is open.

SECTION 2 · THE API SURFACE, END TO END

The API surface, end to end

This is the full map of what the kernel exposes: every device, input, screen, vision, shortcut, casting, recovery and calibration call, plus the events it pushes back. The installed kernel on this rig is IMService.exe, the XP edition, listening on 127.0.0.1:9911 for both HTTP and WebSocket. The Phonefarm-edition document describes a sibling API on a different port with different paths; where the two disagree, this section says so and gives the XP path to actually call. Every Phonefarm function name probed against this kernel returned HTTP 500, so build against the `/xp/...` paths below, not the bare Phonefarm names.

PHONEFARM FUNCTIONS CATALOGUED	SHORTCUT SUB-FUNCTIONS	WEBSOCKET CALLBACKS	WORKING PORT ON THIS RIG
48	**24**	**8 + 13**	**9911**
46 numbered + get_group_list + ocr_ex	ids 1-24; 9 undocumented, 12 & 14 withdrawn	Phonefarm events + XP-only events	9912 (Phonefarm HTTP) is closed

> WRONG **Phonefarm names do not work here**
>
> Posting a Phonefarm-style body such as `{"fun":"get_device_list"}` to this kernel returns status 500, "List index out of bounds (1). TStringList object range is 0..0." The XP kernel splits `fun` on "/" and expects a path with at least two segments, so a bare snake_case name is rejected before it ever reaches a handler. Every row below gives the XP path that actually answers on this rig.

Device and group management

Devices and groups are the addressing layer underneath every other call. A device only exists in the kernel after its first AirPlay connection; group ids are opaque numeric strings the kernel assigns.

Function	XP path	Purpose	Key params	Transport / edition
`set_dev`	`/device/set`	Bind a device to a USB board (vid+pid), assign a group, rename it, or point it at a calibration record by CRC.	`id` (comma list for name/gid/crc; hardware binding is one device per call), `vid` + `pid` (send both or neither), `gid`, `location_crc`, `name`	HTTP+WS, all editions
`del_dev`	`/device/del`	Remove a device record. Refuses while the phone is online.	`id` (comma list on XP)	HTTP+WS, all editions
`set_group`	`/device/group/set`	Create a new group (`gid` "0") or rename an existing one.	`gid`, `name`	HTTP+WS, all editions

Function	XP path	Purpose	Key params	Transport / edition
`del_group`	`/device/group/del`	Delete a group by id. XP refuses a non-empty group.	`gid` (comma list on XP)	HTTP+WS, all editions
`get_device_list`	`/device/get`	Return every known device, online and offline, with full properties.	none on Phonefarm; optional `id` filter on XP	HTTP+WS, all editions
`get_group_list`	`/device/group/get`	Return all device groups.	none	HTTP+WS, all editions
`get_usb_list`	`/config/usb/get`	List the connected vendor USB control boards.	none	HTTP+WS, all editions
`get_devicemodel_list`	`/config/devicemodel/get`	Return the supported-model table and the mouse-calibration library.	none	HTTP+WS, all editions

Watch out

- `location_crc` must come from a manual collection or an official library record, never invented; a wrong CRC silently mis-positions every tap.
- Verified on this rig: `/device/get` returns both XRs with `imgw` / `imgh` 500x1080, `location_crc` `B0DBD800`, and the mirror-geometry fields Phonefarm records do not carry at all.
- XP renames several fields: `username` → `user_name`, `srvname` → `srv_name`, `airhandle` → `air_handle`. Result containers also change shape: XP always returns `data.list` arrays, never an object keyed by id.
- Group id is a string that looks like a Unix timestamp; treat it as opaque, not as a sortable number.

Mouse input

Five calls cover every pointer gesture: absolute click, absolute move, press, release, and a canned swipe. Coordinates on this rig are proven to be mirror-image pixels, 500 x 1080, not the phone's logical 414 x 896 points.

Function	XP path	Purpose	Key params	Transport / edition
`click`	`/mouse/click`	Tap, or long-press when `time` > 0 (context menus, icon wiggle, haptic touch).	`id` (comma list drives N boards in parallel), `button` (Phonefarm "left"/"right"; XP numeric 1-8), `x`, `y`, `time` ms	HTTP+WS, all editions
`mouse_down`	`/mouse/down`	Press and hold a button at the current position; pair with moves and an up for an arbitrary-path drag.	`id`, `button`	HTTP+WS, all editions

Function	XP path	Purpose	Key params	Transport / edition
`mouse_move`	`/mouse/move`	Move the pointer to an absolute position (hover, or one segment of a drag).	`id`, `x`, `y`	HTTP+WS, all editions
`mouse_up`	`/mouse/up`	Release a button.	`id`, `button`	HTTP+WS, all editions
`swipe`	`/mouse/swipe`	Directional drag, either by screen percentage (kernel picks the start/end points) or between explicit coordinates, as a stepped loop.	`direction`, `length` / `len`, `sx` / `sy` / `ex` / `ey`, `for` / `stepping`, XP adds `step_sleep` and `brake`	HTTP+WS, all editions

Watch out

- Measured: the board walks the pointer at about 0.6 ms per pixel. A 1000 px move costs about 592 ms; a full-height 1080 px swipe takes roughly 580 ms, which iOS may read as a slow drag rather than a flick. Driving several phones costs the same as one, since the boards act in parallel.
- In percentage mode, `swipe` has the kernel pick the start and end coordinates itself on each call rather than using one fixed pair; the doc does not state the magnitude, nor whether it still applies when explicit `sx` / `sy` / `ex` / `ey` are given.
- Measured trap: a phone can silently stop honouring `click` while `move` and hardware keys keep working. `mouse_up` does not clear it; only the reset call in Recovery does.
- Phone-side settings invisible to the API — AssistiveTouch on, tracking sensitivity and speed at maximum, Full Keyboard Access enabled, Auto-Lock never — are the usual cause of "looks fine but ignores input."

Keyboard input

Single-key press/release and hotkey combos are ASCII-only and ship natively; Unicode text rides a separate bridge through the iMouse Shortcut.

Function	XP path	Purpose	Key params	Transport / edition
`key_down`	`/key/down`	Press and hold one key (modifiers, chords, game controls).	`id`, `key`; XP also lists `fn_key` here (leave `key` blank when using it)	HTTP+WS, all editions
`key_up`	`/key/up`	Release one key.	`id`, `key`	HTTP+WS, all editions
`send_key`	`/key/sendkey`	Type an ASCII string, or fire a modifier combination such as Home or screenshot.	`id`, `key` (text) or `fn_key` (combo, mutually exclusive)	HTTP+WS, all editions
`send_text`	`/shortcut/clipboard/set + paste hotkey`	Enter Unicode/Chinese text by handing it to the iMouse Shortcut through a bound hotkey.	`id`, `text`; XP adds `sleep`, `outtime`, then a separate `/key/sendkey` call with `fn_key` WIN+v	HTTP+WS; completion is WS-only on Phonefarm, synchronous (bounded by `outtime`) on XP

Needs

- Full Keyboard Access enabled on the phone for any key call to register.
- For `send_text`: the iMouse Shortcut installed, the kernel IP entered, run once with permissions granted, and bound to an AssistiveTouch custom key from the console; both the USB channel (to fire the hotkey) and the LAN channel (Shortcut to kernel) must be up.

Watch out

- `send_key` and `key_down` / `key_up` accept English letters, digits and ASCII punctuation only; there is no Unicode path except through `send_text` or shortcut id 10 (clipboard) plus a paste hotkey.
- Named keys on Phonefarm: BACKSPACE, SPACE, TAB, WIN, ENTER, SHIFT, CTRL, ALT, ESC, PAGEUP, PAGEDOWN, arrow keys, F1-F12, FN. XP's documented hotkey table is considerably larger — see the XP-only table further down.
- Measured: hardware keys such as Home keep working while the pointer is in the stuck-click state, making `send_key` WIN+h a useful liveness probe for the control channel.
- `send_text` is accept-only on the synchronous reply; the characters appear on screen roughly 3 seconds after the completion signal, and over plain HTTP that signal cannot be received at all.

Wheel input

Function	XP path	Purpose	Key params	Transport / edition
`mouse_wheel`	`/mouse/wheel`	Scroll content without moving the pointer, sidestepping the per-pixel walk cost of a swipe.	`id`, `direction` (up/down; XP adds left/right), `length` / `len` (max 127, an int8 HID delta), `number` of events	HTTP+WS, all editions

Screenshots and screen capture

One call captures a single frame; Phonefarm's second call turns that into a server-push stream. The XP kernel has no push stream, so a live view is built by polling.

Function	XP path	Purpose	Key params	Transport / edition
`get_device_screenshot`	`/pic/screenshot`	Capture the current mirrored frame as JPEG or BMP.	`id` (single device), `binary`, `isjpg` / `jpg`, `original` (Phonefarm) or `rect` crop + `save_path` (XP), `gzip` (Phonefarm only)	HTTP+WS; `binary` is WS-only on Phonefarm, works over plain HTTP on XP
`loop_device_screenshot`	none	Server-push stream of screenshots at a fixed interval, adjustable and stoppable live.	`id`, `time` ms interval, `isjpg`, `stop`	WS-only, Phonefarm only — no XP equivalent

RAW BMP, BINARY=TRUE	JPEG, BASE64 JSON	POLLING SUSTAIN RATE
2 ms	**40 ms**	**75–85 req/s**
1.6 MB, bottom-up header, measured	capped around 21 req/s	binary=true over loopback, measured

Watch out

- There is no push stream on this kernel. A live view is client-side polling of `/pic/screenshot?binary=true`; a still screen yields almost no new frames, and the source frame rate is per-phone and content-dependent (one XR here runs about 29 fps, its twin about 4.5 fps).
- Phonefarm's `original` flag returns a resolution different from the phone's logical size, requiring the caller to convert coordinates; XP screenshots are already in the same `imgw` x `imgh` mirror-pixel space that mouse calls use, so no conversion is needed there.
- Input traffic does not interfere with screenshot polling on this rig, measured directly.

Vision: image search, OCR and colour matching

Five recognition calls exist. Two need a picture-search plug-in, two need an OCR plug-in, and two need no plug-in at all. On this rig the OCR plug-in is confirmed not connected (probe returned code 28); the picture-search plug-in's status is unconfirmed and should be assumed unavailable until separately probed.

Function	XP path	Purpose	Key params	Transport / edition
`find_image`	`/pic/find-image`	Locate one template image on the current screen, optionally inside a region.	`id`, `rect` (4 corner points on Phonefarm; `[l,t,r,b]` on XP), `img` / `img_list`, `similarity`	HTTP+WS, all editions; needs the picture-search plug-in
`find_image_ex`	`/pic/find-image`	Locate several templates in one call, optionally every one of them and every duplicate.	`img_list`, `all`, `repeat` / `same`, `rect`, `similarity`	HTTP+WS, all editions; same plug-in requirement
`ocr`	`/pic/ocr`	Recognize all text on screen or within a region.	`id`, `rect`	HTTP+WS, all editions; needs the OCR plug-in — **confirmed not connected on this rig**
`ocr_ex`	`/pic/ocr-ex`	Enhanced, heavier OCR with the same interface as `ocr`.	`id`, `rect`	HTTP+WS, professional-only on Phonefarm, not gated on XP; same plug-in requirement
`find_multi_color`	`/pic/find-multi-color`	Multi-point colour match (anchor plus tolerance) inside a rectangle — plug-in-free.	`id`, search rect, `first_color` (RRGGBB-tolerance), offset points, `similarity`, `dir` scan order (Phonefarm only)	HTTP+WS, all editions; native, no plug-in — works on this rig today
`find_multi_color_ex`	`/pic/find-multi-color`	Run several colour-match configurations in one call.	`id`, `all`, `list` of configs	HTTP+WS, all editions; same, plug-in-free

> TRAP **OCR-dependent chain is dark on this rig**
>
> `ocr`, `ocr_ex`, and — because they need OCR internally — `save_autoscreen_point` and `save_restart_point` (and by extension the fully-automated form of `auto_connect_screen` and `restart_device`) cannot work until the OCR plug-in is installed and connected. `find_multi_color` is unaffected and is the practical plug-in-free option today.

Needs

- XP adds `/pic/find-image-cv` (OpenCV template matching) and `/pic/find-text` / `/pic/find-text-ex` (OCR plus string matching, with a `contain` flag) — same plug-in dependencies.
- XP's iMouse Shortcut version 1.3+ exposes `/shortcut/other/screenshot`, `find-image`, `find-image-cv`, `ocr`, `ocr-ex`, `find-text`, `find-text-ex` and `find-multi-color` as on-phone fallbacks that need no server-side plug-in. Their parameters are not separately documented beyond the path names — probe before relying on them, but this is the likeliest route to OCR on this rig without installing the plug-in.

Watch out

- Result shapes differ by edition: Phonefarm returns top-level `result` / `rectangle` / `confidence` / `code`; XP always returns `data.list` of `{index, centre, rect, similarity, code}`.
- Colour-match encoding differs: Phonefarm uses a relative offset-colour list with a 0-3 scan direction; XP uses a flat list of absolute `x|y|RRGGBB-tolerance` points with no scan-order option.

Shortcut functions (id 1 to 24)

Phonefarm exposes these as sub-function ids of one dispatcher call, `shortcut`, gated to the professional edition. XP splits each into its own path and does not gate by edition; licensing there runs through the account system instead. All of them depend on the iMouse Shortcut being installed on the phone, the kernel IP entered, the Shortcut run once with every permission granted, and bound to an AssistiveTouch custom key. File and album functions need iOS 15+.

id	Function	XP path	Purpose	Notes
1	Album list	`/shortcut/album/get`	List photos/videos, optionally after a date (Phonefarm) or by album name and count (XP).	Returns `{name, ext, size, time}` per item.
2	Album download	`/shortcut/album/down`	Transfer named photos/videos to the PC.	Lands in `Shortcut\Media`; XP adds a `zip` flag.
3	Album delete	`/shortcut/album/del`	Delete named photos/videos on the phone.	Needs the Shortcuts "delete without confirmation" permission.
4	File list	`/shortcut/file/get`	List files in a Files-app directory ("On My iPhone").	Empty extension in a result means a directory.
5	File download	`/shortcut/file/down`	Transfer named files to the PC.	Lands in `Shortcut\File`.
6	File delete	`/shortcut/file/del`	Delete named files on the phone.	Needs delete-without-confirmation permission.
7	Album upload	`/shortcut/album/upload`	Send photos/videos from the PC into an album (blank name = Recents).	Needs the "allow sharing large amounts of data" Shortcuts permission.

id	Function	XP path	Purpose	Notes
8	File upload	`/shortcut/file/upload`	Send files from the PC into a phone directory.	Path must start and end with "/".
9	(undocumented)	–	Skipped in the vendor table between 8 and 10.	Do not call; behaviour unknown.
10	Clipboard set	`/shortcut/clipboard/set`	Put text on the phone clipboard, Unicode-safe.	Pair with a paste hotkey; XP adds a `sleep` delay for slower phones.
11	Clipboard get	`/shortcut/clipboard/get`	Read the phone clipboard.	Returns `{text}`.
12	Send SMS	–	Struck through in the vendor doc.	Withdrawn; do not call, in either edition.
13	Open URL	`/shortcut/exec/url`	Open a URL on the phone — Safari, or any app via its URL scheme or universal link.	Lets an app be launched into a specific screen without UI navigation.
14	Read text aloud	–	Struck through in the vendor doc.	Withdrawn; do not call.
15	Set brightness	`/shortcut/switch/bril`	Set screen brightness, 0 to 1.	Useful for reducing panel load on a rack.
16	Flashlight on	`/shortcut/switch/torch`	Turn the flashlight on.	`state` =1.
17	Flashlight off	`/shortcut/switch/torch`	Turn the flashlight off.	`state` =0.
18	Airplane mode on	`/shortcut/switch/flight`	Enable airplane mode.	Drops the AirPlay video channel; USB control is unaffected (two-channel model).
19	Airplane mode off	`/shortcut/switch/flight`	Disable airplane mode.	`state` =0.
20	Cellular data on	`/shortcut/switch/cdpd`	Enable cellular data.	Requires a SIM.
21	Cellular data off	`/shortcut/switch/cdpd`	Disable cellular data.	`state` =0.
22	Wi-Fi on	`/shortcut/switch/wlan`	Enable wireless LAN.	`state` =1.
23	Wi-Fi off	`/shortcut/switch/wlan`	Disable wireless LAN.	Kills the AirPlay video channel and the Shortcut's own return path; the phone goes offline while remaining USB-controllable.
24	Get external IP	`/shortcut/device/ip`	Read the phone's public IP address.	`state` =1; XP's `state` =0 additionally returns the LAN IP.

Needs

- iMouse Shortcut installed, kernel IP entered, run once with all permissions allowed, and bound to a custom AssistiveTouch key via the console's "Send Binding Key."
- Both channels up: the USB board to fire the trigger hotkey, and the LAN to let the Shortcut talk back to the kernel.

Watch out

- Phonefarm passes `parameter` as a JSON-encoded string (not an object) and returns `retdata` as a string that must itself be parsed; `outtime` defaults to 15 seconds. XP uses native typed fields and `outtime` in milliseconds (default 15000).
- XP adds a destructive `/shortcut/album/clear` (empties an entire album) and `/shortcut/switch/device` (shutdown with `state` 0, restart with `state` 1) not present in the Phonefarm dispatcher.
- Ids 12 and 14 are struck through in the vendor doc; treat both as withdrawn and do not call them in either edition.

Casting and AirPlay control

These calls start, stop and tune the video channel — separate from the USB control channel used by mouse and keyboard calls.

Function	XP path	Purpose	Key params	Transport / edition
`auto_connect_screen`	`/device/airplay/connect`	Drive Control Center to start AirPlay mirroring to the kernel.	`id` (comma list on XP); Phonefarm's `force` flag has no XP equivalent — disconnect first instead	HTTP+WS, all editions; time-consuming, treat as async
`auto_connect_screen_all`	`/device/airplay/connect/all`	Cast every offline phone in one call.	none on XP	HTTP+WS, all editions
`discon_airplay`	`/device/airplay/disconnect`	Disconnect a device's mirroring session; USB control is unaffected.	`id` (comma list on XP)	HTTP+WS, all editions
`set_usb_autoairplay` / `get_usb_autoairplay`	`/config/imserver/set|get` (`auto_connect`)	Cast-on-connect: automatically mirror as soon as a phone is attached to its board.	Per-device on Phonefarm; a single global flag on XP, no restart needed	HTTP+WS, all editions
`set_airplay_mode` / `get_airplay_mode`	`/device/airplay/set` (or `/config/imserver/set` for the global default)	Trade picture latency against network and CPU load.	Phonefarm: three named presets, 0 energy-saving, 1 normal, 2 high-performance; XP: numeric `air_ratio`, `air_fps`, `air_refresh`, `air_img_fps`, `air_audio`, per device or globally	HTTP+WS, all editions

Function	XP path	Purpose	Key params	Transport / edition
`get_airplaysrvnum / set_airplaysrvnum`	`none`	Read or set how many AirPlay receiver services the kernel advertises.	Phonefarm concept only	HTTP+WS, Phonefarm only

Watch out

- Phonefarm's document gives rough figures for its three presets — energy-saving around 5 Mbps per landscape game and roughly 100 phones per PC; normal around 10 Mbps and 50 phones; high-performance around 20 Mbps with basically no delay — but does not define the measurement, and it is a Phonefarm-edition figure, not a measurement on this rig.
- XP runs a single AirPlay receiver (this rig advertises as `iMouseXP-5429` on port 17000); there is no multi-receiver service count to read or set. The nearest XP lever for casting reliability is `connect_failed_retry` in `/config/imserver/set`, and `/imserver/regmdns` to re-advertise the receiver if phones stop seeing it.
- Per-device `/device/airplay/set` changes disconnect the current cast and apply on the next connection.

Recovery

The rig has one recovery call per failure layer, from the pointer up to the whole kernel.

Function	XP path	Purpose	Key params	Transport / edition
`mouse_reset_pos`	`/mouse/reset`	Reset the pointer to (0,0). Measured on this rig: the only call that clears a silently stuck click state, and it stays cleared.	`id` (comma list on XP)	HTTP+WS, all editions
`key_release_all`	`/key/upall`	Release every key currently held, recovering from an interrupted chord.	`id`	HTTP+WS, all editions
`restart_usb`	`/device/usb/restart`	Reset a USB control board on both the PC and phone side, without restarting the kernel.	Phonefarm: `device id`; XP: `id` = "VID|PID" (comma list, so several boards reset in one call, even unbound ones)	HTTP+WS, all editions; needs kernel ≥1.2.0.3 and board firmware ≥3.0
`restart_device`	`/device/restart`	Restart the phone by driving its Settings UI (or, on XP, via `/shortcut/switch/device` or the hotkey CTRL+ALT+SHIFT+WIN+r).	`id`	HTTP+WS, all editions; Phonefarm needs a captured restart coordinate first, which itself needs the OCR plug-in

Function	XP path	Purpose	Key params	Transport / edition
`restart (kernel)`	`/imserver/restart`	Restart IMService. Drops every mirroring session and websocket connection; devices go offline until they re-cast.	none	HTTP+WS, all editions

> **TIP** **Five layers, five recoveries**
>
> Pointer stuck → `/mouse/reset`. Keyboard chord stuck → `/key/upall`. Board unresponsive → `/device/usb/restart`. Mirror session degraded → disconnect and re-cast. Phone unresponsive → `/device/restart` or the shortcut/hotkey alternative. Kernel itself wedged → `/imserver/restart`, the most disruptive option since it drops everything.

Mouse calibration

Calibration teaches the kernel where a phone's screen actually is under a given board, producing a "location" string and a CRC that other phones of the same model and iOS version can reuse.

Function	XP path	Purpose	Key params	Transport / edition
`mouse_collection_open`	`/device/collection/mouse (cmd=1)`	Enter collection mode for one device so its browser can open the kernel's collection page.	`id`	HTTP+WS, all editions
`mouse_collection_close`	`/device/collection/mouse (cmd=2)`	Leave collection mode. XP adds cmd=0 to query status without starting or stopping.	`id`	HTTP+WS, all editions
`mouse_collection_cfg`	`/device/collection/mouse (cmd=1, over HTTP or WS on XP)`	Run the automated collection: the kernel drives the pointer while the phone's collection page reports where it landed.	`id`	WS-only on Phonefarm; result arrives via the `collection_mouse` / `mouse_collection_cfg_ret` callback
`save_dev_location`	`/device/collection/mouse/save`	Persist a collected calibration into the shared library so same-model, same-iOS phones auto-match on connect.	`id`, `describe`	HTTP+WS, all editions
`del_dev_location`	`none`	Remove a calibration record from the library.	model, version, crc	HTTP+WS, Phonefarm only; XP has no delete, only add

Watch out

- The collection page must be launched full-screen from a home-screen icon, not a Safari tab; the `collection` callback's state field (or, on XP, the `collection_mouse` codes 8/9) is how the kernel tells the two apart.
- By default the kernel auto-matches a phone's calibration from the shared library on connect; manual collection is only needed when no match exists or taps land offset. This rig's XRs auto-match a shared library CRC, `B0DBD800`.
- The XP doc does not document the phone-side collection URL — the console shows a QR code instead of the raw `?fun=collection` link Phonefarm uses.

Callbacks (server-pushed events)

The kernel pushes state changes to every connected websocket client, unsolicited. There is no equivalent over HTTP.

Callback	Fires when	Payload highlights
`dev_connect`	A phone's AirPlay video channel comes up.	Full device record; XP wraps it in `data.list` and adds `imgw` / `imgh`, `uid`, `gname`, `uver`, the `air_*` fields.
`dev_disconnect`	The video channel drops; USB control may still be alive.	Same record, `state` 0.
`dev_rotate`	The mirrored screen changes orientation.	`rotate` 0 portrait / 1 landscape; `imgw` / `imgh` swap meaning.
`usb_change`	A control board is plugged in or unplugged.	Phonefarm: `{state, vpid}`. XP: `data.list` of `{vid,pid,uid,ver,state}` with the event type in `data.code` (12 phone attached, 19 board connected, 20 board disconnected).
`mouse_collection_cfg_ret`	Automated calibration finishes (Phonefarm name; XP calls it `collection_mouse`).	Phonefarm: `{deviceid, location}`. XP: `data.code` only — read the resulting location from `/device/get` or `dev_change`.
`collection`	The phone opened the collection page.	`{deviceid, state}` — 0 browser tab (invalid), 1 home-screen icon (valid). Folded into `collection_mouse` on XP.
`binary_frame`	Every frame of a running `loop_device_screenshot` stream, or a binary screenshot reply.	261-byte NUL-padded device id, 4-byte little-endian `isjpg`, then image bytes. Phonefarm only — no equivalent on XP.
`send_text_ok`	The Shortcut has fetched text submitted by `send_text`.	Payload undocumented. Phonefarm only.

XP-only callbacks, all wrapped in `{fun, status:200, message, msgid, data}`: `im_connect` (kernel version, on socket open), `dev_change`, `dev_delete`, `group_change`, `group_delete`, `collection_mouse`, `airplay_connect_log`, `restart_log`, `user_info` (pushed every 30 seconds), `im_log`, `error_push` (which function failed and why — the only async signal that an input call failed at the board), `im_config_change`, `logout`, and `dev_sort_change`.

Implicit capabilities

Nothing here is a separate function; each is a consequence of how the documented calls compose.

- Long-press is `click` with `time` set to 800-1000 ms rather than a dedicated call.

- A right-click primitive exists (`button` "right"/2), which under iOS pointer support maps to a secondary or context action — a second gesture alongside long-press.
- `mouse_down` , a sequence of `mouse_move` calls, and `mouse_up` compose any arbitrary-path gesture — icon rearrangement, slider drags, drawing, map panning — not just the canned `swipe` . Coalescing intermediate points costs nothing extra, since the board walks the path regardless.
- In percentage mode, `swipe` has the kernel choose the start and end coordinates itself on each call, so repeated swipes are not pixel-identical by default.
- Held modifiers via `key_down` / `key_up` support shift-selection, multi-key chords and game controls; `key_release_all` is the recovery when a chord is interrupted mid-sequence.
- A hotkey can trigger any user-authored iOS Shortcut bound to that AssistiveTouch key, not only the vendor's built-in ones — `send_text` is simply the vendor's own use of that generic channel. This opens on-phone automation (opening apps, running device actions) that has no dedicated API call.
- One `shortcut` call with a `devlist` runs the same sub-function across several phones in a single round trip; XP's comma-separated `id` lists give the same fan-out for mouse and keyboard calls.
- Shortcut ids 2/5 (download) and 7/8 (upload) plus 3/6 (delete) form a complete file-transfer pipeline between the PC and each phone's album or Files app.
- The high-resolution "original" screenshot (Phonefarm) is well suited to building template images and OCR ground truth at native quality; the caller then rescales into the coordinate space used by `click` .
- Request/response correlation via `msgid` lets many calls be in flight on one websocket connection at once, matched asynchronously as replies arrive.
- Groups plus `gid` on every device record let a controller partition the rack — by client, campaign or model — and filter `get_device_list` accordingly; XP's `/device/group/get-dev` does the reverse lookup.
- The `dev_connect` / `dev_disconnect` / `dev_rotate` / `usb_change` callbacks give a live inventory of both channels — video and control — without polling; on XP every change is broadcast to every connected client, so several consoles stay in sync automatically.
- A saved calibration (`save_dev_location`) becomes reusable across every phone of the same model and iOS version via its CRC; `get_devicemodel_list` doubles as a compatibility oracle for which models and calibrations exist.
- `set_usb_autoairplay` (or the XP global `auto_connect` flag) plus `auto_connect_screen_all` together make a rack self-healing after a power cycle: boards re-cast their phones automatically, and any still offline can be swept in one call.
- Device state (`state` / `airhandle` , the video channel) and USB binding/ `usb_change` (the control channel) are independent, so a console can distinguish "mirroring but not controllable" from "controllable but blind" rather than collapsing both into one status.

Doc gaps and ambiguities

Gap	Why it matters
No Phonefarm function name works against this kernel (all probed names return HTTP 500).	The two editions are siblings, not interchangeable; every integration on this rig must be written against the XP paths, never the Phonefarm names, regardless of which document is easier to read.

Gap	Why it matters
Coordinate space is never stated for click, move, swipe or the recognition `rect`.	Proven on this rig to be mirror-image pixels (500 x 1080), not iOS logical points (414 x 896); a Phonefarm-only client would have no way to derive this without a screenshot to compare against.
Neither edition documents any authentication.	Any host on the LAN can call every function, including device deletion, kernel restart, and Shortcut file transfer. XP's `allow_ip_list` only gates AirPlay discovery, not the HTTP/WS API itself.
Shortcut id 9 is missing from the table (the sequence runs 8, then 10).	Unclear whether it is reserved, removed, or internal; do not call it.
Ids 12 (SMS) and 14 (text-to-speech) are struck through with no documented replacement behaviour.	Treat both as withdrawn in both editions; calling them is unsupported.
Error code table has internal contradictions: code 2 means both "Device online" and, in one place, "device not online"; code 21 is used for two different messages; codes 9 and 11 share identical text.	Match on the numeric `data.code`, never on the message string, and expect at least one of these codes to be ambiguous in practice.
The recognition `rect` for `find_image` / `ocr` is four corner points in the order upper-left, lower-left, upper-right, lower-right; XP's is a flat `[left, top, right, bottom]`.	Porting a Phonefarm-shaped rectangle to an XP call without reordering the corners will crop or scan the wrong region.
`outtime` is seconds on the Phonefarm `shortcut` dispatcher and milliseconds on the equivalent XP paths.	Reusing a Phonefarm timeout value unconverted on XP will time out roughly 1000x too fast.
"Professional version only" (Phonefarm's `shortcut` and `ocr_ex`) is never mapped to an XP edition or licence tier.	There is no documented way to know, from the API alone, whether a given XP install can run these — only probing tells you.
Time-consuming calls (auto-cast, coordinate-point capture) have no documented duration or timeout, and Phonefarm gives them no progress callback.	A caller cannot distinguish "still working" from "silently failed" without XP's `airplay_connect_log` / `restart_log`, which the Phonefarm doc lacks entirely.
Device record fields `location` (a pipe-separated integer string), `type`, and `airhandle` have no documented layout or meaning.	These must be treated as opaque; do not parse or reconstruct them.
The documented model/calibration library stops at the iPhone 13 family; this rig's iPhone XRs on iOS 18.7.10 run on a shared library CRC rather than a per-device measurement.	Tap accuracy on iOS builds newer than the library's coverage is unverified by the vendor documentation and should be spot-checked, not assumed.

XP-only capabilities

These exist only in the installed edition, with no Phonefarm counterpart at all.

XP path	Purpose	Notes
`/config/user/login, /logout, /info, /switch`	Log the kernel into the licensing account and switch between up to 10 sub-accounts (`utag`).	Devices cannot come online unlicensed. The credential the console uses is stored on disk in recoverable form; never enter it into a new console without the user doing so themselves.
`/config/imserver/get, /set`	Read and write kernel configuration: receiver name, port, mDNS discovery mode, IP allow-list, default cast quality, auto-cast, thread mode.	Most fields need `/imserver/restart` to take effect; a few apply live. Broadcasts `im_config_change` to other clients.
`/imserver/regmdns`	Re-advertise the AirPlay receiver over mDNS.	Useful when the receiver disappears from a phone's Screen Mirroring list without a full kernel restart.
`/device/sort/set, /get`	Set or read the device list's sort order, mirrored in the vendor console.	Broadcasts `dev_sort_change` .
`/device/group/get-dev`	List the devices belonging to one group.	`id` =0 lists the ungrouped devices.
`/device/airplay/set`	Per-device mirroring overrides: resolution, refresh rate, video fps, image fps, audio.	0 on any field defers to the global default; changing one disconnects the current cast.
`/pic/find-image-cv`	OpenCV-based template matching, an alternative engine to the DM-style `/pic/find-image` .	Instance count is set by `opencv_num` in the kernel config.
`/pic/find-text, /pic/find-text-ex`	OCR the screen and then match specific strings, with a `contain` flag.	Same OCR plug-in dependency as `ocr` — unusable on this rig until installed.
`/pic/screenshot rect + save_path`	Crop a screenshot server-side and/or write it straight to a file.	Phonefarm's screenshot call could do neither.
`/shortcut/other/*`	On-phone, plug-in-free fallbacks for screenshot, find-image, find-image-cv, ocr, ocr-ex, find-text, find-text-ex and find-multi-color.	Needs Shortcut ≥1.3; parameters are not separately documented, so probe before use.
`/shortcut/album/clear`	Delete every item in an album in one call.	Destructive; confirm with the user before wiring it to a button.
`/shortcut/switch/device`	Shut down (`state` 0) or restart (`state` 1) the phone via the Shortcut.	Shutdown has no Phonefarm counterpart at all.
`/key/down fn_key`	Hold a combination hotkey down, not only a single key.	Phonefarm's `key_down` was single-key only.

XP path	Purpose	Notes
`/mouse/click, /up, /down buttons 3–8; /mouse/wheel left/right; /mouse/swipe step_sleep + brake`	Middle and extra mouse buttons, horizontal scroll, and finer control of swipe timing.	Phonefarm exposed left/right buttons and up/down scroll only.
`Extended hotkey vocabulary`	Home (kernel ≥2026.2.27 for iOS 26.1+), TAB+b (back), AppSwitch, ControlBar, NoticeBar, TAB+l (lock), OPENKeyboard, shift+win+4, CTRL+ALT+SHIFT+WIN+r (restart), CTRL+ALT+WIN+p (pass-through), WIN+c/v/a/x, SwitchIme, MuteSoundSwitch, VolumeUp/Down, Play/Pause.	Considerably larger than Phonefarm's documented set; covers the console's vertical control strip.
`/device/usb/restart with a VID|PID list`	Reset several boards in a single call, including ones with no phone currently bound.	Phonefarm's `restart_usb` addressed one device at a time.
`Extra device record fields`	`imgw` / `imgh` (mirror geometry), `air_pid` (live mirroring proof), `uid`, `gname`, `uver`, `srv_name`.	`imgw` / `imgh` is the coordinate space every mouse call uses; `uver` is unreliable on this build.
`imouse-py (pip install imouse-py)`	Official Python client with typed helper classes for console, device, image, keyboard, mouse and shortcut operations, plus event decorators.	Phonefarm shipped only a loose sample script; this is a useful parameter-naming reference even for a non-Python console.
`XP error-code table, 0–41`	Full `data.code` semantics, including login/licensing codes 37-40 and 41 "incompatible command."	Code 28 is what this rig returns for OCR calls; 25 is offline, 26 is screenshot failed, 36 is parameter error.

Also worth knowing

`/pic/find-image` takes two request parameters that do not appear anywhere else in this map: `delta_color` ("Color Cast") and `direction` ("Find directions"). Both are typed as plain strings, and both are documented with the same three words: "Reference DM." The doc is telling you the values follow the conventions of the third-party DM-style matching engine XP borrows for this call, not spelling them out itself, so treat the exact syntax as unverified until it is probed against this kernel rather than assumed from the parameter name. Neither parameter is optional filler: they sit in the request table alongside `img_list`, `rect` and `similarity` as first-class inputs to the same call.

Read against what `similarity` already does, the pairing makes sense. `similarity` is a single floating-point threshold on overall match confidence, greater than 0 and less than 1. `delta_color` is a different axis entirely: a color-cast tolerance, letting a template still register as a match when the live frame's colors have drifted from the frame the template was cut from, rather than forcing the caller to lower `similarity` and accept more false positives everywhere else on screen. That distinction matters on this rack specifically because `/shortcut/switch/bril` lets a console set a phone's brightness anywhere from 0 to 1 on command, and a brightness change, an iOS light/dark mode switch, True Tone's automatic warm-cool drift, and the lossy color handling of an AirPlay mirroring encoder all

move the same kind of pixel values that a color-cast tolerance exists to absorb. `direction` is documented too thinly to say what it changes with confidence; by name it reads as a search-order control rather than a tolerance, so treat it as a secondary lever and confirm its effect before depending on it.

Both knobs are specific to the DM-style engine at `/pic/find-image`. `/pic/find-image-cv`, the OpenCV alternative XP adds alongside it, has no `delta_color` or `direction` field in its own request table, only `img_list`, `rect`, `similarity`, `all` and `same` — so a color-cast tolerance is a reason to prefer the DM engine over OpenCV on this hardware, not an incidental feature of whichever one is faster. Feasibility is unchanged from the rest of this group either way: both calls need the picture-search plug-in, whose status on this rig is unconfirmed and should be assumed unavailable, NEEDS PLUG-IN WITH CAVEATS, until it is separately probed alongside the delta_color syntax itself.

Marketing automation

The rack's most obvious job is turning N phones, each signed into an account the operator owns, into posting infrastructure: push a creative and a caption once, drive each app's own composer with mirrored taps, and verify per phone. Every path below runs on the installed XP kernel with the parameters shown. The catch is verification: the OCR and image-search plug-ins are not connected on this rig (code 28), so nothing here can read a success banner or a button label, only compare raw screenshots by pixel difference. Media and Unicode text both depend on the iMouse iOS Shortcut, a per-phone manual install that has never been probed on this rig, not on anything gated by XP versus Phonefarm edition specifically. XP does not gate the shortcut dispatcher by edition; Phonefarm marks the equivalent "professional version only", but licensing on both editions is per device through the vendor account.

Publishing through the app's own composer

Album push plus native composer: publish a feed post on every phone

NEEDS SHORTCUT

WITH CAVEATS

Push the same JPEG or MP4 into the Recents album of every bound phone with one multi-id call, put the caption on every clipboard with another, then drive the social app's own composer with mirrored taps: home, app icon, '+', newest thumbnail, 'Next', caption field, paste, 'Share'. Every tap is one comma-separated request that the boards walk on all phones in parallel, so a 19-phone batch costs roughly the wall time of one phone plus the media transfer. This is the core reason to own a rack: real accounts, on real iPhones, posting through the app's own UI with no platform API access.

HOW

1. `get_device_list` XP `GET /api/device/get (optionally ?id=a,b)` — build the id list from phones that are mirroring (state != 0) and share one imgw x imgh (500x1080 on both XRs here); a coordinate on a different mirror size is a different place.
2. `get_usb_list` XP `GET /api/config/usb/get` — confirm the control channel separately: state 2 means the board is enumerated and the phone is cabled. Video state proves nothing about control.
3. `mouse_reset_pos` XP `GET /api/mouse/reset?id=<all ids>` — clears a measured silent fault where a phone stops honoring `/mouse/click` while moves and hardware keys keep working. Send it whenever the target set changes.
4. `shortcut#7` XP `POST /api/shortcut/album/upload {id, files, album_name, zip, outtime}` — pushes the creative into Recents on every listed phone in one call. `outtime` is milliseconds on XP; the Phonefarm equivalent takes seconds.
5. `shortcut#10` XP `POST /api/shortcut/clipboard/set {id, text, sleep, outtime}` — the only documented Unicode or emoji text path on XP; `sleep` is seconds. POST JSON so the caption is not mangled in a query string.
6. `send_key` XP `GET /api/key/sendkey?id=<all ids>&fn_key=WIN+h` (twice) — normalizes every phone to home page one; the first press leaves the app, the second returns home.

7. `click` XP `GET /api/mouse/click?id=<all ids>&x=&y=` — drives app icon, '+', newest album thumbnail, 'Next'. Coordinates are mirror pixels, never the 414x896 logical points.
8. `click + send_key` XP `…&fn_key=WIN+v` — focuses the caption field, then pastes.
9. `click` XP `GET /api/mouse/click?id=<all ids>&x=&y=` — Share. Irreversible: one request publishes on every phone in the id list at once. Treat as a separate, explicitly approved call.
10. `get_device_screenshot` XP `GET /api/pic/screenshot?id=<one id>&binary=true` — per-phone raw BMP, compared to the pre-Share frame by mean absolute pixel difference.
11. `find_multi_color` XP `POST /api/pic/find-multi-color` — optional stronger check by absolute anchor-point color match, single device only; the plug-in status behind this endpoint is untested here.

NEEDS

- IMService.exe plus IMConsole.exe running and logged in on TCP 9911; Phonefarm function names return HTTP 500 on this kernel.
- iMouse Shortcut installed, IP-edited to the kernel, and bound to an AssistiveTouch key on every phone — unverified on this rig; prove it first with `GET /api/shortcut/device/ip?id=<id>&state=0`.
- Board bound (vid+pid together) and at `/config/usb/get` state 2; `location_crc` non-empty or every mouse call returns code 11.
- iOS settings the API cannot see: AssistiveTouch on with max tracking, Full Keyboard Access on, Trackpad & Mouse speed max, Auto-Lock never, portrait lock, StandBy off.
- Every phone on the same owned account, app version and composer layout — the two XRs here have different home screens, so shared coordinates must be aligned by hand.
- Kernel logged into a licensed vendor account; unlicensed devices are refused.

WATCH OUT

- The Shortcut is the hard gate for both media and Unicode text; without it, only ASCII typing remains and no file can reach the phone at all.
- No OCR or image-search plug-in: verification is blind pixel diff, unable to read a success or error banner.
- Screenshots are single-id; every input step is one comma-list call, but verification is a per-phone loop.
- A multi-id call reports one result code for the whole list — a partial failure inside the fan-out is invisible except through per-phone screenshots.
- The creative stays in every phone's Recents afterward; cleanup calls are destructive and deserve their own confirmation.
- An identical caption pasted from one call is byte-identical on every account, which platforms treat as duplicate content; distinct captions cost one call per phone.
- `outtime` is milliseconds on XP and seconds on Phonefarm — a value copied across editions times out 1000x too early.
- Only 2 of 19 boards have ever mirrored simultaneously on this rig; extrapolating today's numbers to a full 19-phone run is unproven.

> TRAP **Pixel diff is a change signal, not a success signal**
>
> A permission sheet, an OTP prompt, an app crash, or a content-moderation interstitial all produce a mean pixel difference as large as a genuine post. Without OCR, treat any post-Share diff above threshold as "needs a human look," never as a checkmark.

SCALE

Album upload and clipboard set are each one request for the whole id list, but the bytes move as N separate LAN transfers — a 20 MB video to 19 phones is roughly 380 MB total, so raise `outtime` into the minutes and use `zip=1` for small images. Every tap, hotkey and reset also costs one request regardless of N at about 0.6 ms per pixel of pointer travel, so the nine-step composer sequence runs roughly 10-15 seconds of board time for the whole group. Verification is the per-phone cost: raw BMP grabs take 2 ms each, but the real wait is frame production on the slow XR's AirPlay link (about 4.5 new frames per second against its twin's 29), so budget 220 ms or more per frame before trusting a "closed" read. Because the Share tap is irreversible and belongs behind a human confirmation, realistic throughput lands around 20-40 full-rack post cycles per hour with an operator actively confirming each one, well below the sub-minute figure raw API latency alone would suggest. Keep per-account posting frequency far below that ceiling: platforms treat identical simultaneous content across many owned accounts as coordinated activity regardless of how fast the rack fires the taps.

Stories and Reels vertical video with a dragged sticker placement

NEEDS SHORTCUT WITH CAVEATS

Push a 9:16 MP4 into each phone's camera roll, confirm it is the newest album item by name and create time, then drive the Story or Reel composer: pick the clip, open the sticker tray, paste link or mention text from the clipboard, and place the sticker with a real press-hold-drag (`mouse/down` to `mouse/move` to `mouse/up`) rather than a canned swipe. Stories and Reels are mobile-only surfaces with no usable desktop composer, and the sticker drag has no web equivalent at all — it is exactly the gesture the vendor's own console uses, and mirroring it to N phones costs the same wall time as one because the boards walk in parallel.

HOW

1. `get_device_list + get_usb_list` XP `GET /api/device/get?id=<ids> and GET /api/config/usb/get` — preflight both channels and read imgw/imgh per phone (the mirror-pixel coordinate space, not 414x896).
2. `shortcut#7` XP `POST /api {"fun":"/shortcut/album/upload", …, "outtime":180000}` — pushes the clip into every phone's camera roll in one call; leave `zip=0` for a single large video.
3. `shortcut#1` XP `GET /api/shortcut/album/get?id=<single id>&num=1` — confirms the clip is genuinely the newest item (name, ext, create_time) before trusting a "tap the first thumbnail" coordinate. Single id only; loop per phone.
4. `shortcut#10` XP `GET /api/shortcut/clipboard/set?id=<ids>&text=&sleep=1` — stages the sticker's link or mention text; the only Unicode-safe text path on XP.
5. `send_key` XP `…&fn_key=WIN+h` (twice) — normalizes every phone to home page one.
6. `mouse_reset_pos` XP `GET /api/mouse/reset?id=<ids>` — clears the stuck-click state before anything tap-critical.
7. `click` — opens app, composer and gallery picker, selects the newest clip; an optional `time=` gives a long press.
8. `click` — opens the sticker tray, chooses the link or mention sticker, focuses its text field; `fn_key=OPENKeyboard` is a fallback if focus does not take.
9. `send_key` XP `…&fn_key=WIN+v` — pastes the sticker text; requires Full Keyboard Access on, with no completion signal.
10. `mouse_down → mouse_move (repeated) → mouse_up` — places the sticker with a real drag. Move onto the sticker first: `/mouse/down` takes no coordinates and presses wherever the pointer already is.
11. `click` — Publish. Irreversible; screenshot and inspect first, since one coordinate set is only valid within a single app build and experiment bucket.

12. `get_device_screenshot` XP `GET /api/pic/screenshot?id=<single id>&binary=true` — per-phone verify.
13. `shortcut#3` XP `POST /api {"fun":"/shortcut/album/del", "data":{"id", "list":[{"album_name","name","ext"}]}}` — cleans up the staged media so the next run's "newest item" coordinate stays true; `album/clear` wipes a whole album and is destructive.

NEEDS

- iMouse Shortcut installed, IP-edited and hotkey-bound on every phone, with Shortcuts > Advanced set to allow running scripts, sharing large data, and deleting (with and without confirmation).
- Phone-to-kernel LAN path in both directions; the API is unauthenticated, so firewall 9911 and 17000 to trusted hosts.
- iOS accessibility settings invisible to the API: AssistiveTouch max, Full Keyboard Access, Trackpad & Mouse speed max, Auto-Lock never, portrait lock, StandBy off.
- Board bound and physically connected (state 1 then 2) with the vendor cable; `location_crc` non-empty.
- IMConsole.exe running and logged in; every target phone on the same app build and A/B bucket if one coordinate set is shared.
- Accounts the operator owns or is authorized for, checked against each platform's automation terms.
- OCR and image-search plug-ins strongly advised, but not installed on this rig.

WATCH OUT

- Every call in the chain exists on the installed XP kernel with the stated parameters — verification, not availability, is the limiting factor.
- With no OCR or image-search, the chain runs blind on fixed coordinates plus mean pixel diff: it cannot confirm which thumbnail was tapped, that the tray opened, or where the sticker landed.
- `/pic/screenshot` and `/shortcut/album/get` are single-id, so confirm and verify are per-phone loops while input stays one comma-list call.
- No success callback exists for the Shortcut path on XP; a 15 s timeout (code 30) looks identical to a slow response.
- XP's swipe table documents no start or end randomization (that behavior appears only in the Phonefarm sibling doc); use down/move/up for the drag regardless, since it is deterministic and matches the vendor console.
- An interrupted drag leaves the button held; `/mouse/up` releases it but does not clear the separate stuck-click state — only `/mouse/reset` does.
- Portrait lock still matters on XP, but for a different reason than a borrowed Phonefarm error code suggests: rotation swaps imgw/imgh and fires `dev_rotate`, invalidating every stored coordinate.
- Uploading to N phones is N separate LAN transfers, not a broadcast, even though it is one request.
- There is no documented bandwidth figure for the album-upload channel itself; the vendor's AirPlay energy-mode Mbps figures describe outbound mirroring, not this inbound file path, and should not be reused to estimate video-upload time.
- Only 2 of 19 boards have ever mirrored simultaneously here, so the frame-rate figures behind the screenshot-gate wait logic are unproven at full rack scale.

> TRAP **A down-press has no confirmation**
>
> The only available check after a drag is a post-move pixel diff, which cannot tell "the sticker moved" from "the pointer moved and the sticker stayed put." Diff a small box around the sticker's start position, in addition to its destination, to catch a grab that missed.

SCALE

Every input step takes a comma id list and costs one request regardless of N; a 400 px sticker drag is about 250 ms of board time whether the list holds one phone or nineteen. Only the screenshot and album-listing checks are per-phone, and pollers do not bottleneck each other (measured 512 req/s across ten concurrent pollers). The real cost center is the video transfer: each phone receives its own copy of the MP4 over Wi-Fi through the Shortcut with no documented bandwidth budget for that channel, so a typical 15-30 s clip (10-40 MB) plausibly runs low tens of seconds to a few minutes per phone, and a 19-phone batch upload is plausibly several minutes even largely in parallel. Realistic throughput for the full upload-confirm-normalize-paste-drag-publish-verify cycle, human-gated at the publish step, is on the order of one full-rack cycle every 5-10 minutes, or 6-12 cycles per hour run back to back — a shared wall-clock rate, not a per-phone-independent one. Keep per-account posting frequency well below this ceiling for the same coordinated-content reasons as the feed post above.

Launching composers by URL instead of hunting for icons

Cross-post one asset by launching each app's composer via URL scheme

NEEDS SHORTCUT

WITH CAVEATS

Push one asset into every camera roll, put the caption on every clipboard, then open each target app's composer directly with a URL scheme or universal link instead of finding its home-screen icon. Finish with mirrored taps and a WIN+v paste, and confirm each stage with a screenshot. On this rig the two XRs have entirely different home screens and wallpapers, so the same mirror coordinate is Clock on one phone and Reminders on the other; a URL scheme sidesteps that entirely because it puts every phone on the same composer screen at once, after which one mirrored coordinate is legitimately fair across the whole set.

HOW

1. `get_device_list + get_usb_list + mouse_reset_pos` XP `GET /api/device/get?id=<ids> | GET /api/config/usb/get | GET /api/mouse/reset?id=<ids>` — confirms video and control are both up and clears the stuck-click state before anything tap-critical.
2. `shortcut#7` XP `POST /api {"fun":"/shortcut/album/upload", …}` — pushes the asset into every camera roll once per campaign.
3. `shortcut#10` XP `POST /api {"fun":"/shortcut/clipboard/set", …}` — pushes the caption once.
4. `shortcut#13` XP `POST /api {"fun":"/shortcut/exec/url", "data":{"id","url","outtime"}}` — launches each target app's composer by URL scheme or universal link, one call per app, all phones at once. The API places no constraint on the URL string; no scheme for any specific app is documented anywhere in this project — confirm each by hand before scripting it.
5. `get_device_screenshot` XP `GET /api/pic/screenshot?id=<one id>&binary=true` — per phone, confirms the composer (not Safari, not a permission sheet) actually opened before any tap.
6. `click / swipe / down-move-up` — drives the composer with mirrored taps, now that every phone shares one screen.
7. `send_key` XP `…&fn_key=WIN+v` — pastes the caption into the focused field.
8. `click + get_device_screenshot + send_key` XP `…&fn_key=WIN+h` (twice) — submits, verifies per phone, then normalizes before the next app's launch.

NEEDS

- Account authorization for every account involved — the precondition for the whole use case, not a footnote.

- iMouse Shortcut installed and hotkey-bound on every phone; unverified on this rig, iOS 15+ for the file-family shortcut functions.
- Board bound and at `/config/usb/get` state 2; mirroring live with a matching air_pid; IMConsole.exe running and logged in.
- iOS accessibility settings invisible to the API (AssistiveTouch, Full Keyboard Access, Trackpad & Mouse speed, Auto-Lock, portrait lock, StandBy) and `location_crc` non-empty.
- Target app already installed and signed in on every phone — nothing in this chain authenticates; a launch into a logged-out app lands on a login screen.
- The URL scheme or universal link for each target app, confirmed by hand on the installed app version — not documented anywhere in this project or by the platforms.
- A client-side frame comparator, since the server-side OCR and image-search plug-ins are unavailable here.

WATCH OUT

- The API hands iOS whatever URL string it is given; whether that string opens a composer is entirely a platform matter, undocumented and version-dependent. A retired scheme leaves the phone on the home screen or in Safari, and every mirrored tap after that lands on something unintended.
- Universal links fall back to Safari if the app is not installed or the association is broken — the verifier must distinguish "app opened" from "Safari opened."
- Opening a URL scheme from an automation context can surface a one-time "Open in [App]?" confirmation sheet distinct from the composer itself, especially the first time a scheme runs after install or an OS update. A tap sequence written only for the composer lands on this sheet instead, on an unpredictable subset of phones. Positive-match the sheet's shape, or dismiss it with a known fixed tap, rather than gating only on how much the screen changed.
- No server-side recognition (code 28): verification is client-side pixel diff or template matching on raw BMP, never an MD5 comparison, since the AirPlay encoder changes every frame regardless of phone activity.
- Mirrored taps are only fair once every phone is genuinely on the same screen; drop any phone whose launch failed from the id list for the rest of the sequence rather than tapping it blind.
- Phones with different imgw/imgh must never share a coordinate; both XRs here happen to match at 500x1080.
- Caption text with emoji or CJK must travel by clipboard, never `/key/sendkey`, which is ASCII-only on both editions.
- App updates commonly retire or alter custom URL schemes, and auto-update timing varies per phone, so a scheme validated today can succeed on seventeen phones and silently fail on two in the same run.
- Rapid app-open-close cycling across several target apps is itself a pattern some platforms' client-side heuristics can flag; keep the number of apps cross-posted per run small and paced, the same discipline as phone count.
- Licensing is per device through the vendor account; confirm `/config/user/info` shows coverage for the device count before a large run.

> TRAP **A stale URL scheme fails silently into whatever is foreground**
>
> The post-launch screenshot check is the only thing standing between a retired scheme and a blind tap storm across every phone in the set. Treat it as mandatory, and fail the whole batch closed rather than continuing with partial confirmation.

SCALE

The write-side shortcut calls plus click and sendkey all take a comma id list, so N phones cost one request per step, not N; the board walks every listed phone in parallel. Per app the cost is one exec/url call, a handful of mirrored clicks, and one paste, with `outtime` defaulting to 15000 ms on XP and needing headroom for the hotkey-to-Shortcut round trip. Verification is the per-phone cost: single-id screenshots at 2 ms each, but the real ceiling is the phone's own AirPlay source rate, roughly 220 ms per new frame on the slower of the two XRs here. Practically, one app cycle runs 5-15 seconds of enforced screenshot-gate waiting plus whatever the app itself takes to load after the URL fires, which this API does not bound and which often runs several seconds on its own. Across M target apps sharing one asset and caption push, a full cross-post run is roughly M times 10-20 seconds of mechanics — five apps in one to two minutes, a technical ceiling near 30-60 cross-app runs per hour. Mandatory per-step screenshot review and a human approval before each submit tap will realistically cut sustained throughput to a handful of full cross-post runs per hour.

Scheduling campaigns across device groups

Campaign calendar: external scheduler fires posting jobs at device groups

NEEDS SHORTCUT

WITH CAVEATS

The kernel has no timer of any kind, so the calendar lives outside it: Windows Task Scheduler, cron, or an n8n/Airflow job fires a posting run at the planned minute against a named device group. Groups give real tenant isolation, since group membership is the resolver a job trusts, so a mistyped device id resolves to nothing rather than to the wrong client's phones. A per-slot preflight on two independent channels plus a per-slot screenshot turns an unattended overnight run into an auditable evidence row per phone instead of a silent no-op — which matters, because this rig has a measured failure mode where a phone follows moves and hardware keys perfectly while ignoring every click.

HOW

1. `set_group` XP `/device/group/set?id=0&name=ClientA` — one-time: creates a group per client or campaign; `id=0` creates, any other id renames.
2. `set_dev` XP `/device/set?id=<id1,id2>&gid=<gid>&name=Slot-01` — assigns phones to the group and names each slot. `gid` / `name` take a comma list; vid/pid hardware binding is one device per call.
3. `get_group_list + get_group_dev` XP `/device/group/get then /device/group/get-dev?id=<gid>` — resolves live group membership at fire time. XP-only endpoint; single group id per call; returns full device records, so no second lookup is needed.
4. `get_device_list` XP `/device/get?id=<ids>` — preflight the video channel (state != 0, a live air_pid) and read imgw/imgh per phone.
5. `get_usb_list` XP `/config/usb/get` — preflight the control channel (state 2, vid/pid/uid bound), independently of video.
6. `mouse_reset_pos` XP `/mouse/reset?id=<members>` — clears the stuck-click state across the whole target set at the start of every run.
7. `shortcut#7` XP `/shortcut/album/upload?id=<members>&files=[…]&zip=1&outtime=60000` — pushes the campaign asset; `zip=1` for many small images, `outtime` in milliseconds.
8. `shortcut#10 + send_key` XP `/shortcut/clipboard/set?id=<members>&text=… then /key/sendkey?id=<members>&fn_key=WIN+v` — delivers a Unicode caption, then pastes it into the focused field.

9. `click` XP `/mouse/click?id=<members>&x=&y=&button=1&time=` — drives the in-app posting flow in mirror-pixel coordinates.
10. `get_device_screenshot` XP `/pic/screenshot?id=<one id>&binary=true` — evidence frame per slot; single-id fan-out.
11. `callbacks` XP `ws://127.0.0.1:9911/api – dev_connect, dev_disconnect, usb_change, error_push` — optional async watch; `error_push` carries the failed call and message for a board-level input failure.
12. `the scheduler itself` XP `none – no timer, cron, or scheduled-job endpoint exists in either doc set` — the load-bearing external dependency; Task Scheduler, cron, n8n or a LAN service does the actual firing.

NEEDS

- An external scheduler on the LAN — the kernel has no timers; this is the mechanism, not garnish.
- IMConsole.exe running and logged in; a scheduled run should gate on board count greater than zero before firing.
- Every phone bound (vid+pid) with non-empty `location_crc`.
- Auto-Lock Never plus no pending passcode prompt — a phone that rebooted overnight sits on the lock screen with no API path to unlock it.
- iMouse Shortcut installed, IP-edited and hotkey-bound on every phone, with the same Shortcuts > Advanced permissions as the other entries.
- Asset files readable by the kernel process at the given PC paths, with Windows backslashes JSON-escaped.
- Coordinates authored per phone layout, plus a known starting state (two WIN+h presses) before any shared coordinate is used.
- Cheapest proof the Shortcut path is alive before a campaign night: `GET /shortcut/device/ip` returning code 0 with a matching ip field.

WATCH OUT

- Calling this "native" is wrong on two counts: there is no scheduler anywhere in either doc set, so the calendar is entirely external tooling, and asset and caption delivery both run through the per-phone Shortcut install. Only grouping, gating, reset, tapping and screenshot are native.
- Group device listing, screenshots, album listing and clipboard reads are all single-id; input endpoints and the write-side shortcut endpoints take comma lists — do not assume every call fans out the same way.
- With no OCR or image-search plug-in, the job cannot read "Posted" or find a moved Share button; verification is pixel diff plus human review of saved frames, and UI navigation must stay coordinate-based.
- `outtime` is milliseconds on XP and seconds on Phonefarm; clipboard's `sleep` is seconds either way — mixing units silently truncates a video upload.
- `/device/set` binds vid+pid for one device per call even on a comma list, so per-slot hardware binding is one call per phone.
- Group deletion refuses while members remain (code 34); device deletion refuses online devices (code 7) — move phones to gid 0 first for teardown.
- No completion callback exists for the Shortcut path on XP; the screenshot is the only proof that pasted text actually landed.
- None of the iOS accessibility prerequisites are queryable over the API; the per-slot preflight is a proxy, not a guarantee, and a phone can show board state 2 with a stale air_pid while actually sitting on a lock screen with mirroring paused.
- No per-phone locking exists: two scheduled jobs, or a job and a manual console session, can target the same board at once and interleave taps with no error from either side.

- Binding a new phone into a group is two non-atomic calls (bind, then add to group); a crash between them leaves it bound but ungrouped and silently excluded from that night's run.
- The two Shortcut calls were never exercised on this rig — if the Shortcut is not bound, both simply time out (code 30) and the run posts nothing while every other preflight gate reports fine.
- An evidence frame only shows the screen changed near the expected coordinate, not that a post succeeded; treat it as requiring a human spot-check, not as an auditable pass or fail.
- A coordinate-based Share or Post tap has no re-verification available; a silently moved button gets tapped wrong indefinitely until a human reviews saved frames.
- The 15,000 ms default timeout for both Shortcut calls has no measured round-trip time on this rig; treat "minutes, not seconds" as an assumption pending that measurement, not a fact.

> TRAP **A silent Shortcut failure looks identical to success**
>
> If the iMouse Shortcut is not installed or bound on a phone, album upload and clipboard set both simply time out while `/mouse/reset` and every other preflight gate reports fine. Add `GET /shortcut/device/ip` as a hard canary check at the start of every scheduled run, not an optional aside.

SCALE

Group resolution and gating cost two GETs regardless of rack size, and input fans out for free: one click or sendkey call with a comma id list drives every listed board in parallel at about 0.6 ms per pixel, so a 20-phone group costs roughly what one phone costs on the input side. Evidence capture does not fan out, but phones are independent (measured 512 req/s across ten concurrent pollers), so a 20-shot evidence sweep is noise next to everything else. The real per-phone budget is the two Shortcut round trips, each capped at a 15 s timeout with no measured typical latency on this rig; budget minutes, not seconds, for a group of any size, more for video. On the slower of the two XRs (about 4.5 new AirPlay frames per second against its twin's 29), any screen-state wait must budget 220 ms or more per frame and use a pixel-difference threshold rather than a frame counter. Mechanically the rig could plausibly sustain 2-4 scheduled posting rounds per minute with shared content, or 2-4 phones per minute with per-phone content, but that ceiling is irrelevant in practice: a marketing calendar fires a handful of posts per day per account, and each platform's own posting-frequency policy is the real limit long before this API is stressed.

Direct messaging and replies

Unicode comment and DM replies pasted from the phone clipboard NEEDS SHORTCUT WITH CAVEATS

`/key/sendkey` is documented ASCII-only in both editions, and the XP kernel has no `send_text` function at all. The only documented route to emoji, accents, CJK or Arabic text anywhere in this API is to set the phone's clipboard from the PC and paste it in: `/shortcut/clipboard/set` followed by a `WIN+v` hotkey. The same route serves comment replies, DM replies, captions, bios and search queries. Identical text to a group is one clipboard call with a comma id list; per-recipient text costs one clipboard call per phone.

HOW

1. `mouse_reset_pos` XP `GET /api/mouse/reset?id=<ids>` — clear any silent stuck-click state on the target set before the tap-critical steps that follow.
2. `shortcut#10` XP `POST {"fun":"/shortcut/clipboard/set","data":{"id":"<ids>","text":"<reply>","sleep":1,"outtime":15000}}` — push the reply text to each phone's clipboard; one call per distinct

reply, one comma-id call when the text is identical.

3. `click` `XP GET /api/mouse/click?id=<ids>&x=&y=` — focus the comment or DM field (or `XP fn_key=OPENKeyboard` to raise the on-screen keyboard).
4. `send_key` `XP GET /api/key/sendkey?id=<ids>&fn_key=WIN+v` — paste. One request covers the whole target set.
5. `send_key` / `click` `XP GET /api/key/sendkey?id=<ids>&fn_key=ENTER` or `XP GET /api/mouse/click?id=<ids>&x=&y=` — submit. ENTER's behaviour is app-specific and undocumented; the Send-button tap is the safer default.
6. `shortcut#11` (canary) `XP GET /api/shortcut/clipboard/get?id=<one id>&outtime=15000` — read back the clipboard on one phone before a batch, to byte-compare the Unicode round trip and confirm the Shortcut path is alive at all.
7. `get_device_screenshot` `XP GET /api/pic/screenshot?id=<one id>&binary=true` — confirm the text landed and the field cleared after send, by mean-pixel-difference against the pre-paste frame.

NEEDS

- iMouse iOS Shortcut installed on every phone, kernel IP edited in, run once with every permission allowed, and bound to an AssistiveTouch custom key. Not exercised on this rig; prove it with the clipboard-get canary before building anything larger.
- Full Keyboard Access on (so `WIN+v` and `ENTER` are interpreted), AssistiveTouch on with tracking sensitivity maxed, Auto-Lock Never, portrait lock.
- Board bound and cabled (`/config/usb/get` state 2) with a non-empty `location_crc` , or every mouse call fails with code 11.
- Phone reachable to the kernel over Wi-Fi on 9911, since the Shortcut posts its result back that way even though the hotkey that triggers it arrives over USB.
- The target text field already focused before `WIN+v` fires — a paste into nothing returns code 0 with no error.
- Reply text authored upstream. This rig has no server-side OCR to read the inbound comment or DM (`/pic/ocr` answers code 28, plug-in not connected).

WATCH OUT

- Nothing under `/shortcut/*` was exercised in the 10 September probe. `/mouse/*` , `/key/sendkey` and `/pic/screenshot` are confirmed; the Unicode half of this chain rests on an unverified assumption.
- Unit trap: XP's `outtime` is milliseconds (default 15000), `sleep` is seconds. The Phonefarm sibling doc uses seconds for both. Passing a Phonefarm value gives a 15 ms timeout and a guaranteed code 30.
- A timed-out `clipboard/set` (code 30) leaves the clipboard holding the previous phone's text — a blind `WIN+v` afterwards pastes the wrong reply into this phone's field. Treat any non-zero `data.code` as a hard stop for that phone, not something to retry.
- With no OCR plug-in there is no server-side way to confirm the reply text rendered correctly or to catch an in-app error banner; verification is limited to a raw-BMP mean-pixel-difference check, or client-side OCR you run yourself against the frame.
- Coordinates are per phone: the two XRs on this rig have different home screens, so a comma-id click is only valid when the Send control sits at the identical mirror-pixel position on every listed phone.
- `fn_key=ENTER` submitting the reply is app-dependent and undocumented — in many apps it inserts a newline instead. Verify per app before trusting it; default to clicking Send.
- Phonefarm-style calls (bare `{"fun":"send_text"}`) return HTTP 500 on this kernel. There is no `send_text` and no `send_text_ok` callback on XP.

- The clipboard-get canary proves the round trip for one phone at that moment; it does not prove every other phone in a batch has the Shortcut installed and bound. A 19-phone batch needs 19 canary checks, or a one-time setup audit, not one canary standing in for the group.

> TRAP **A stale clipboard publishes on the wrong account**
>
> Because a timed-out `clipboard/set` leaves the old text in place with nothing visibly wrong, an orchestration wrapper that checks HTTP status instead of the JSON `data.code` field can paste and submit one phone's reply under a different phone's account. Hard-fail on any non-zero code before the paste step ever fires.

SCALE

The shortcut round trip (HID hotkey → Shortcut launches → phone calls back over Wi-Fi to 9911) has never been timed on this rig; the documented bound is the 15,000 ms `outtime` ceiling, not a measured latency. Until it is timed, plan on up to 15 s per distinct reply, sequential across phones needing different text — identical text to a whole group is a single call regardless of group size. Steps that take a comma-id list (reset, click, paste, submit) cost one request for the whole target set either way, at roughly 0.6 ms/px of pointer travel. Verification screenshots are cheap and fully parallel across phones (~2 ms raw BMP each, phones independent, ten concurrent pollers measured 512 req/s aggregate), so they add negligible time. The slower of the two XRs on this rig delivers only ~4.5 new frames/s versus ~29 on its twin, so any screen-change check on it needs a longer wait budget, not a fixed 30 fps assumption.

Prefilled messaging deep links (wa.me, tg://, sms:) opened per phone, with a batched Send tap

NEEDS SHORTCUT · WITH CAVEATS

Instead of typing into a composer, hand each phone a URL that already carries the recipient and the message body: a `wa.me` or `whatsapp://send` link, an `sms:` body, or a `tg://msg` share. `/shortcut/exec/url` opens it and the app lands in the chat with the text already in the box, including emoji and CJK, so neither the ASCII-only hotkey path nor the clipboard round trip is needed. Nothing sends silently: every scheme still requires a visible composer and one real tap, and XP has no SMS-send endpoint at all.

HOW

1. `send_key` XP `GET /api/key/sendkey?id=<ids>&fn_key=WIN+h` (twice) — normalise every target phone to home screen page one.
2. `mouse_reset_pos` XP `GET /api/mouse/reset?id=<ids>` — clear stuck-click state before the batched Send tap later in the chain.
3. `shortcut#13` XP `POST {"fun":"/shortcut/exec/url","data":{"id":"<deviceid>","url":"https://wa.me/<number>?text=<url-encoded>","outtime":15000}}` — open the prefilled chat; one call per distinct URL, since only an identical URL can share a comma id list. Use the JSON POST form so the URL's own `?` and `&` don't collide with query-string encoding.
4. alternative bodies — XP `whatsapp://send?phone=&text=` (no Safari bounce) · XP `sms:<number>&body=` (inconsistent across iOS builds) · XP `tg://msg?text=` (opens Telegram's share sheet, not a direct chat — no recipient in the scheme).
5. `get_device_screenshot` XP `GET /api/pic/screenshot?id=<deviceid>&binary=true` — confirm the chat actually opened with the text in the box before anything is tapped.
6. client-side check (preferred) or `find_multi_color` XP `POST /api/pic/find-multi-color {...}` — decide from that frame whether the Send control is present. The colour-probe endpoint exists as documented but is

unprobed on this rig.

7. `click` XP `GET /api/mouse/click?id=<ids>&x=&y=&button=1` — Send, one request for the whole batch, only once every phone in the list has independently confirmed its own composer is populated.
8. `send_key` (alternative) XP `GET /api/key/sendkey?id=<ids>&fn_key=ENTER` — app- and setting-dependent; test before trusting it over the tap.
9. `shortcut#10` + `send_key` (Unicode fallback) XP `/shortcut/clipboard/set then fn_key=WIN+v` — for a scheme that refuses to prefill.
10. `shortcut#7` (optional media) XP `POST {"fun":"/shortcut/album/upload", ...}` — stage an attachment into the phone's own album first, since a deep link cannot carry one; picking it into the chat is then ordinary tap automation.

NEEDS

- iMouse iOS Shortcut installed, IP-edited, permissioned and hotkey-bound on every phone — unverified here; cheapest proof is XP `GET /api/shortcut/device/ip?id=<id>&state=0` matched against `/device/get`'s `ip` field.
- WhatsApp, Telegram or Messages installed and registered on that phone's own SIM or Apple ID — a link to a non-user lands on an error page, which is exactly what the confirmation screenshot is for.
- Documented consent for every recipient, and a message template and volume that comply with the target platform's terms and applicable marketing law, established before any of this touches real numbers.
- Send-button coordinate captured per app version from a real screenshot, re-captured after any update.

WATCH OUT

- This is a hard, binary gate, not one caveat among several: every path to opening a URL scheme on iOS from this API goes through `/shortcut/exec/url`, and XP has no way to type a URL into Safari's address bar as a fallback. If the Shortcut isn't installed and bound on a given phone, there is no automation path to this use case on that phone at all.
- Deep-link prefill is a property of the receiving app, not of this API; the vendor's own examples pass a bare hostname, so a long query-carrying URL is untested territory. Probe the length ceiling once and keep bodies well under roughly 1,500 characters.
- `tg://msg?text=` opens a share-sheet chooser, not a one-tap send — it does not have the parity with `wa.me` an early draft implied.
- `sms:` body prefill is inconsistent across iOS builds; verify per device before trusting it.
- Server-side screen verification is weak here: `/pic/ocr` answers code 28 and image search is presumed the same, so confirm the composer client-side on the raw BMP with a mean-pixel-difference threshold, never MD5.
- The batched Send tap is only safe once every phone's own confirmation frame has independently landed. Given a measured ~29 fps vs ~4.5 fps gap between the two XRs on this rig, firing the batch on a shared timer risks tapping Send on the slow phone before its composer is actually populated, sending a blank or partial message. Gate the batch on a per-phone confirmed flag, not a shared wait.
- `wa.me` bounces through Safari before handing off (allow ~2 s); `whatsapp://` avoids the bounce.
- XP's `outtime` is milliseconds, default 15000; copying a Phonefarm example verbatim (seconds) gives a 15 ms timeout.
- The Shortcut's triggering hotkey rides the same USB/HID control channel that can silently stop honouring `/mouse/click`. There's no evidence `/mouse/reset` also un-sticks a hotkey path — verify hotkey delivery per run rather than assuming the existing reset step covers it.

OPEN **Platform limits, not hardware, set the real ceiling**

WhatsApp, Telegram and carrier SMS all throttle and review accounts sending patterned, scripted messages, well below anything this rack can mechanically produce. Treat rack capacity as headroom for reliability, never as a sending target, and stop on any block or rate-limit notice rather than retrying.

SCALE

Per-recipient personalisation makes the URL-open step one request per phone; only an identical URL can share a comma id list. The dominant cost is on-phone, not on the wire — hotkey fire, Shortcut launch, app render — budget roughly 2-4 s per phone before the confirmation frame means anything, against a 15 s default timeout ceiling. The normalise, reset and Send steps each collapse to a single request for the whole batch regardless of size. Verification is cheap and parallel (~2 ms raw BMP per phone, ten concurrent pollers measured 512 req/s aggregate). With only two phones currently attached and mirroring, a realistic round is on the order of N messages per 5-8 s of wall clock; scaled to all 19 boards the mechanical ceiling would be roughly 1,000-1,800 messages/hour, but the actual safe rate is set by per-number platform limits and by how many recipients have consented — realistically tens to low hundreds of messages per day per number, not per hour.

Engagement loops

Engagement loops (like, follow, story view) with a colour-check de-duplication probe before each toggle tap

PARTIAL WITH CAVEATS

Drive a social feed on your own signed-in accounts through the pointer channel alone: normalise home, open the app, swipe the feed, and — before tapping a like or Follow toggle — read the control's own pixels first so the loop never un-likes a post it already liked or unfollows an account it already followed. The interesting part is that read-before-write probe: a colour check against the filled heart or Follow-button pixels, done either through the documented `find_multi_color` endpoint or, since this rig's plug-ins aren't confirmed present, by sampling the raw screenshot bytes directly. The same pattern generalises to any stateful control whose state shows as colour — mute, saved, subscribed, dark-mode — with no OCR and no accessibility tree.

HOW

1. `get_device_list` XP `GET /api/device/get?id=<deviceid>` — read `imgw` / `imgh` (500x1080 on these XRs) as the coordinate space for every later call, and confirm the phone is live.
2. `mouse_reset_pos` XP `GET /api/mouse/reset?id=<ids>` — clear stuck-click state on the whole target set before any tap-critical pass; a stuck phone measured 0.09 mean pixel diff on a click, 141.96 after reset, and `/mouse/up` did not fix it.
3. `send_key` XP `GET /api/key/sendkey?id=<ids>&fn_key=WIN+h` (twice) — normalise to home screen page one.
4. `click` XP `GET /api/mouse/click?id=<ids>&x=&y=` — open the app, at a coordinate read from that phone's own screenshot.
5. `shortcut#13` (optional) XP `GET /api/shortcut/exec/url?id=<ids>&url=&outtime=` — deep-link entry instead of an icon tap; needs the Shortcut chain, not confirmed here.
6. `swipe` XP `GET /api/mouse/swipe?id=<ids>&direction=up&len=0.5` — advance the feed. XP's swipe is deterministic given the same parameters; there is no randomisation in the XP table (that claim exists only in the Phonefarm doc).

7. `get_device_screenshot` XP `GET /api/pic/screenshot?id=<one id>&binary=true` — grab the frame the probe will read, cropped with `rect` to the heart/Follow strip.
8. `find_multi_color` XP `POST /api/pic/find-multi-color {id, list:[{first_color, rect, similarity}]}` — the de-duplication probe: documented with exactly this absolute-colour syntax, but not confirmed working through the plug-in layer on this rig.
9. fallback probe (client-side) XP `GET /api/pic/screenshot?id=<id>&binary=true&rect=` + your own pixel sampling — reproduces the same check in a few lines of code at 2 ms per frame, with no plug-in dependency. Build this path first.
10. `click` XP `GET /api/mouse/click?id=<one id>&x=&y=&time=80` — tap the control only when the probe says it is un-actioned. There is no native double-click; compose one from two calls roughly 100 ms apart.
11. `click` XP `GET /api/mouse/click?id=<ids>&x=&y=` — story view: tap the frame's right edge to advance, with a dwell before the next tap.
12. `ocr` / `find_image` (not usable here) XP `/pic/find-text or /pic/find-image-cv` — the documented way to detect a rate-limit or "Action Blocked" interstitial; both are plug-in-gated and unavailable on this rig. Substitute a whole-frame mean-pixel-difference spike and halt the pass rather than retry.
13. `mouse_reset_pos` / `error_push` XP `GET /api/mouse/reset?id=<ids>`; WebSocket XP `ws://127.0.0.1:9911/api` — recovery and async failure signal; `error_push` is the only way to learn an input call failed at the board.

NEEDS

- Accounts you own or are authorised to operate, already signed in by hand on each phone. This describes driving your own logged-in sessions, not creating or disguising accounts.
- Vendor console running and logged in, board state 2 per phone, vid/pid bound, and a non-empty `location_crc`, or every mouse call fails with code 11.
- iOS accessibility set invisible to the API: AssistiveTouch on at max tracking sensitivity, Full Keyboard Access on, Trackpad & Mouse speed maxed, Auto-Lock Never, portrait lock.
- All coordinates in that device's `imgw` x `imgh` (500x1080), read per phone — the two XRs have different home screens and share nothing without measurement.
- For the server-side colour probe: the image-search plug-in installed and enabled, unverified on this rig.
- The API on 9911 has no authentication of any kind; firewall it to trusted hosts before running anything unattended.

WATCH OUT

- The colour-check de-duplication — the whole point of this entry — is not confirmed working via the API on this rig. The only recognition endpoint actually probed, `/pic/ocr`, returned code 28 (plug-in not connected). Probe `find-multi-color` before designing around it; the client-side raw-BMP fallback is the safe path to build first.
- No text recognition at all is available here — any "read the counter" or "read the dialog" step must be done client-side or not at all.
- Every `/pic/*` observation call is single-device; input mirrors to N phones in one request, observation never does. After the first personalised feed load, the realistic shape is one independent loop per phone, not one mirrored loop.
- No native double-click or double-tap: compose it from two `/mouse/click` calls, and note this is asserted, not measured, to register as an iOS double-tap gesture — if the two clicks land outside the recognition window it can silently degrade to two independent taps that toggle the like state twice, defeating the idempotency design.

- `/mouse/click` can silently stop working on a phone while move and hotkeys still succeed; only `/mouse/reset` clears it, and every call still returns success while it's stuck.
- Frame-based verification is bounded by the phone: one XR delivers ~29 new frames/s under motion, its twin ~4.5, and a still screen produces almost no new frames — any "did it fill in?" check must threshold mean pixel difference and budget hundreds of ms on the slow phone. Do not derive the poll interval from observed frame gaps; that decayed 23 fps to 16.5 in testing.
- Reference colours are coupled to the phone's mirror resolution and app/iOS version; an app update that restyles the heart icon invalidates the table silently, with no drift detection available without OCR or image search.
- The probe and the tap are two separate round trips with no atomicity — the control's state could change between the read and the write on any account with more than one active client.
- Story-view and heart coordinates must be measured per app per phone from a real screenshot; illustrative coordinates are not safe to ship.

> TIP **The idempotency pattern is the reusable part**
>
> Reading a control's pixels before writing to it is what turns a screen-automation loop from a demo into something re-runnable on a schedule. It applies to any visibly-stateful toggle, well beyond likes and follows.

> TRAP **Unattended engagement automation is a platform-policy problem, not a technical one**
>
> Unattended like, follow and story-view automation is prohibited by the terms of service of essentially every major social platform; follow/unfollow churn is separately covered by spam policies. The action rate and pattern are fully visible to the platform regardless of how the input is generated. Run this only on accounts you own or are contractually authorised to operate, and treat any rate-limit or "Action Blocked" interstitial as a hard stop for the session, not something to retry on a timer.

SCALE

Input is cheap and parallel: click, swipe, reset and hotkeys all take a comma id list and cost the same for one phone or N. Observation is the bottleneck by design, since every `/pic/*` call is single-id — two phones polled flat out measured 83 and 85 req/s at p50 3 ms, ten concurrent pollers 512 req/s, and input traffic measurably did not slow polling. Per post, budget roughly 300-600 ms of swipe or scroll travel, ~2 ms for the screenshot, a sub-millisecond client colour probe, the tap itself, and a render-settle dwell bounded by the phone's own frame rate — about 1-3 s/action on the fast phone and 2-5 s on the slow one, a mechanical ceiling on the order of 700-3,600 actions/hour/phone. That mechanical number is not the meaningful one: any real deployment must throttle far below it, since the platform's own anti-abuse thresholds, not the rig, set the sustainable rate.

Monitoring and inbox alerts

Badge-polling watcher for new DMs, offers and mentions

PARTIAL WITH CAVEATS

Park every managed account on its inbox or notifications tab, then poll each phone's raw mirror frame at a fixed 1-2 s cadence and watch a small fixed box — the tab-bar badge — for change against a per-phone baseline crop. The detection itself is a client-side mean-absolute-pixel-difference on a 2 ms raw BMP, no plug-in and no OCR

required. iOS push notifications are invisible to the PC, but the AirPlay mirror is not, so a rack that already has every account logged in and open doubles as a real-time monitor with no platform API and no scraping — a human still does all the replying.

HOW

1. `get_device_list` XP `GET /api/device/get` — enumerate targets, filter to state non-zero with a live `air_pid`, and read each phone's own `imgw` / `imgh`.
2. `send_key` XP `GET /api/key/sendkey?id=<ids>&fn_key=WIN+h` (twice) — normalise each phone to a known state.
3. `mouse_reset_pos` XP `GET /api/mouse/reset?id=<ids>` — clear stuck-click state before any tap.
4. `click` XP `GET /api/mouse/click?id=<ids>&x=&y=` — park each phone on its inbox or notifications tab. A shared coordinate is only fair when every phone in the list actually has that tab at that position.
5. `shortcut#13` (optional) XP `GET /api/shortcut/exec/url?id=<ids>&url=&outtime=` — deep-link straight into the inbox instead of tapping; needs the Shortcut chain, unverified here.
6. `get_device_screenshot` XP `GET /api/pic/screenshot?id=<single id>&binary=true` — poll the raw frame per phone at a fixed cadence, one request per phone, concurrent pollers.
7. client-side detection XP `none — your own code on the decoded BMP` — the load-bearing step: mean absolute pixel difference of the badge box against baseline. Idle noise floor measured 0.02-0.03; a real change measures 20-160. Never MD5 or whole-frame diff.
8. `find_multi_color` (optional hardening, unverified) XP `POST /api/pic/find-multi-color {...}` — confirm the iOS red-badge colour. Documented but never probed on this rig; treat as optional, not the detection mechanism.
9. `get_device_screenshot` XP `GET /api/pic/screenshot?id=<id>&binary=true&jpg=true` — on trigger, capture a full-frame JPEG thumbnail for the alert (cheap since it fires once per event, not per poll).
10. none (external) — deliver the alert via Slack, Teams, webhook or console UI. No iMouse function of either edition sends anything off-box.
11. `mouse_reset_pos` + `send_key` + `click` — re-baseline and re-park after a human clears the badge, or after any `/device/airplay/set` change (which alters mirror geometry and invalidates every stored box).
12. `shortcut#15` (optional) XP `GET /api/shortcut/switch/bril?id=<ids>&state=0.1&outtime=` — dim the phone's own panel to cut thermal/power load on a permanently-lit rack; needs the Shortcut chain.

NEEDS

- Vendor console running and logged in, every watched phone mirroring with a live `air_pid`, board state 2, and the iOS accessibility set (AssistiveTouch, Full Keyboard Access) for the parking taps only.
- Auto-Lock Never, portrait lock, brightness and Night Shift frozen — any drift decays both the pixel baseline and any colour table.
- A per-phone baseline crop captured right after the inbox is cleared, and per-app badge-box coordinates in that phone's own mirror-pixel space.
- An alerting sink entirely outside the iMouse API — Slack/Teams webhook, console UI, pager. Nothing in either edition sends a notification off-box.
- Written authorisation to automate the accounts watched, and confirmation the platform's terms permit this kind of automated, read-only access.

WATCH OUT

- The detection mechanism is entirely client-side image processing; the API's whole contribution is a 2 ms frame grab. Don't describe this as native detection.

- `find-multi-color` being plug-in-free is an inference from the doc naming no plug-in, not a documented fact — it sits on the same page as the confirmed plug-in-gated OCR call. Probe it before designing around it; if it returns code 28, drop that step.
- No OCR means you get "something changed in the badge box," never "a DM from X about Y." Triage still requires a human opening the phone.
- Badge coordinates and colour tables are brittle across app updates and are not portable between phones with different home screens — recut boxes after every update, and validate a new box for an hour against an idle phone before trusting it (a box overlapping a scrolling region, animated avatar or timestamp will false-trigger continuously).
- Only two phones are currently online; every throughput figure is measured on two, and the aggregate model at full rack scale is untested.
- Changing `air_ratio` disconnects the cast and alters mirror geometry on reconnect, invalidating every baseline, box and colour table.
- A badge dot downscales from native resolution to roughly 12 px in this rig's 500x1080 mirror — well above the measured noise floor here, but this margin should not be assumed to hold on a higher-resolution phone model.
- Continuous multi-hour foreground mirroring is only validated by short benchmark runs; a watcher meant to run for hours needs its own reconnection and staleness handling for AirPlay drops or the app being evicted by iOS memory pressure, not just a startup check.

> TRAP **A silent watcher failure looks identical to no news**
>
> A phone whose mirror has dropped (state 0, no `air_pid`) produces no diffs and therefore no alerts — indistinguishable from zero new messages. Subscribe to the `dev_disconnect` and `error_push` WebSocket callbacks and surface per-phone video liveness alongside the badge state, or the system fails closed without telling anyone.

> WRONG **Unauthenticated read of customer conversations**
>
> The API has no authentication, so a watcher pulling frames of logged-in seller and DM inboxes turns port 9911 into an unauthenticated read of private customer conversations for any host on the subnet. Screenshots saved to disk or pushed into a Slack channel are a personal-data store; set retention, restrict membership, and prefer holding frames in memory. Firewall 9911 and 17000 to trusted hosts.

SCALE

One raw BMP grab per phone at a 1 s cadence measures p50 2.0 ms and 1,620,054 bytes. Nineteen phones would be roughly 19 req/s and 31 MB/s on loopback against a measured ceiling of 84 req/s for a single flat-out poller and 512 req/s aggregate for ten concurrent pollers — about 4% of demonstrated capacity, needing no throttling. Phones are independent, and input traffic measurably does not slow polling. Each 1.6 MB frame exceeds the .NET Large Object Heap threshold: without buffer reuse, measured allocation was 488 MB/s with 496 gen-2 GCs per 8 s, fixed to 2.9 MB/s by reusing one buffer per poller. Today only two phones are actually online, so real current throughput is 2 req/s at a 1 s cadence — trivially sustainable, with headroom for a sub-500 ms cadence if faster badge detection is later wanted.

Ad account operations at agency scale

Mobile-only ad platform operations across client accounts (TikTok Ads, Meta Ads Manager, Snapchat approvals)

PARTIAL WITH CAVEATS

Give each client's ad account its own phone in the rack, permanently signed in to the mobile ads app, and run a daily supervised routine: normalise home, open the app, capture a proof frame per client, scroll the campaign list, and tap Pause or Approve only on the narrow subset that needs it. Navigation and capture are fully native on this rig; reading the spend and delivery numbers off the screen is not, since the OCR plug-in is absent, so figures must be read client-side or by a human. Several ad-platform surfaces are mobile-first or mobile-only, and a phone per client keeps each authorised session isolated with its own notifications, turning a multi-minute daily check per client into one scripted pass plus a dated proof frame.

HOW

1. `set_group` + `set_dev` XP `GET /api/device/group/set?id=0&name=` then XP `GET /api/device/set?id=&gid=&name=` — partition the rack by client: create a group, then name and assign each phone to it.
2. `get_device_list` XP `GET /api/device/get` — read each phone's coordinate space (`imgw` / `imgh`) and cross-check readiness against XP `/api/config/usb/get` (state 2 confirms the control channel, not just video).
3. `mouse_reset_pos` XP `GET /api/mouse/reset?id=<ids>` — clear stuck-click state across the whole target set.
4. `send_key` XP `GET /api/key/sendkey?id=<ids>&fn_key=WIN+h` (twice) — normalise to home screen page one.
5. `click` XP `GET /api/mouse/click?id=<ids>&x=&y=` — open the ads app on each phone; only mirror one coordinate to a group after confirming every phone in it has the app at that position.
6. `get_device_screenshot` XP `GET /api/pic/screenshot?id=<single id>&binary=true` — capture the overview proof frame, one request per phone, optionally cropped with `rect` .
7. `mouse_wheel` XP `GET /api/mouse/wheel?id=<ids>&direction=down&len=100&number=2`, then repeat capture — walk the campaign list. Wheel is preferable to swipe here since a full-height swipe costs ~580 ms of pointer travel and iOS may read it as a slow drag.
8. client-side match / OCR (both server-side paths plug-in-gated) — verify the screen against a stored golden frame by mean absolute pixel difference before any state-changing tap. This is the gate that makes the whole use case safe; only proceed once the frame matches the expected screen.
9. `click` + `get_device_screenshot` XP `GET /api/mouse/click?id=<verified subset>&x=&y=` then XP `GET /api/pic/screenshot?id=<id>&binary=true&rect=` — tap Pause/Approve only on the verified subset, then crop-diff around the control to prove the toggle actually moved.
10. `ocr` / `ocr_ex` (unavailable on this rig) XP `GET /api/pic/ocr?id=&rect=` — lift spend and delivery figures into a sheet; currently answers code 28. Options are installing the plug-in, running client-side OCR against the raw BMP, or the Shortcut-based OCR fallback.
11. `send_key` + `shortcut#1/#2` (optional) XP `fn_key=shift+win+3` then XP `/shortcut/album/get` / XP `/shortcut/album/down` — pull a native-resolution screenshot when the 500x1080 mirror is too small to read per-row figures; needs the Shortcut chain.

NEEDS

- Vendor console running and logged in, with `/config/user/info` showing an active license before any run.
- Per phone, the invisible iOS accessibility set: AssistiveTouch, Full Keyboard Access, Trackpad & Mouse speed maxed, Auto-Lock Never, portrait lock, vendor cable only.

- Each client's ad account already authenticated on its assigned phone by a person, under that client's authorisation and the platform's own terms for its mobile app — credentials are never typed by the automation.
- Identical ads-app version and icon position across any phone group driven by a shared coordinate, or per-phone coordinate maps otherwise.
- Either the vendor OCR plug-in, an external client-side OCR engine, or a human reader for figure extraction.
- The Shortcut chain (installed, IP-edited, permissioned, hotkey-bound) for the native-resolution capture path only.

WATCH OUT

- Navigation and capture are fully native and doc-verified; reading the numbers is not — `/pic/ocr` answers code 28 and image search is presumed the same until proven otherwise.
- The 500x1080 mirror is a pixel-exact downscale of the phone's own render, not a lossy copy — large headline figures are usually legible, per-row campaign metrics often are not. That's what the native-resolution capture path is for.
- Mirroring a Pause/Approve tap to a whole group is the sharpest edge here. Send state-changing taps only to an explicitly verified subset, with a per-phone screen check and an after-tap frame diff, and exclude any phone whose `imgw` / `imgh` differs from the master.
- The idle-floor mean-diff constant (0.02-0.03) assumes a static screen. Ads-manager feeds routinely contain autoplaying creative previews and live spend counters, which will sit well above that floor even with zero input — capture a per-app-screen baseline over a few idle frames, don't borrow the home-screen number.
- An immediate screenshot after opening the app can capture a splash or blank frame rather than the campaign list — the verification gate needs a wait-for-target-screen loop, not a single capture.
- A Pause/Approve toggle produces its own UI transition (switch animation, confirmation sheet), a different and unmeasured signature from a tap landing on an idle home screen — capture that app's own before/after pair rather than assuming the same numeric range confirms it.
- 2FA and identity challenges are only partly automatable: a code arriving on the same phone can be entered via a tap plus ASCII `/key/sendkey` ; a code on another device, an app-approval prompt, or a Face ID re-authentication all require a person, and the last one blocks the whole daily run for that client until unlocked in person — the API has no camera or biometric interface at all.
- Non-ASCII client or campaign names need the clipboard-plus-paste route, which needs the Shortcut chain; there is no other text-entry path.
- The claimed time savings is about request count, not wall time — the bottleneck is UI settle and app-launch time, unmeasured and app-version-dependent, not the kernel's 2 ms screenshot. Budget seconds, not milliseconds, per client for each navigate-and-verify leg.

> TRAP **Mechanically identical timing reads as automation to the platform**
>
> Mobile ad-management apps commonly run client-side anti-automation heuristics — impossible-precision tap timing, absence of the sensor noise a hand-held phone produces, identical inter-action intervals every day. A HID board driving AssistiveTouch produces exactly that signature, so expect a materially higher chance of forced re-auth or session flags than a human operator would see, independent of anything this API can observe or control. Log and escalate to a human rather than retrying — this is a platform fraud-detection outcome, not a bug in the chain.

WRONG **A mis-timed tap can pause a live campaign or approve the wrong spend**

A mirrored tap on a stale or mis-positioned screen is a money risk, not a cosmetic one. The mitigations are structural: reset before every tap-critical sequence, a golden-frame diff check per phone before the tap, per-phone coordinates unless the group is provably identical, and an after-frame diff proving the control moved. Treat Pause/Approve as needing a human in the loop until the verification gate has a track record.

SCALE

Navigation fans out for free — one comma-id request drives every listed phone in parallel, so a 20-client group costs the same wall time as one phone for the shared steps. Capture does not fan out (screenshot is single-id) but phones are fully independent: measured 83-85 req/s for two concurrent pollers and 512 req/s aggregate for ten, with input traffic not slowing polling. A daily pass over roughly 20 clients is dominated by unmeasured app behaviour — cold launch, list-scroll settle, toggle-confirmation animation — rather than by the kernel's sub-5 ms operations; a conservative 8-15 s per client, run concurrently across the rack's boards, puts a full daily pass at roughly 1-3 minutes of wall time end to end. Because this is a once-per-day-per-client design, the meaningful throughput ceiling is the client-account count — bounded by the 19 populated board slots, one phone per client — not requests per second.

Campaign quality assurance

Landing page render check against a golden frame

NEEDS SHORTCUT WITH CAVEATS

Open your own campaign URL, with UTMs, on every mirrored phone in one call, capture the mirror frame at t+0.5 s, t+3 s and t+10 s, and compare the settled frame against a golden render captured earlier on a known-good phone at the same mirror resolution. It is read-only against your own property: nothing is clicked, nothing is posted, and there is no cross-account or geo-fabrication exposure, which makes it the strongest candidate of this group to build first.

HOW

1. `get_device_list` XP `GET /api/device/get` — read imgw/imgh per phone (500×1080 on both XRs); every later rect and coordinate is in that mirror-pixel space, not the 414×896 logical points. Exclude any phone whose imgw/imgh does not match the golden-frame phone.
2. `mouse_reset_pos` XP `GET /api/mouse/reset?id=<list>` — clears the measured silent stuck-click state before the Safari-driving fallback; harmless when the Shortcut path is used.
3. `shortcut#12 open URL` XP `GET /api/shortcut/exec/url?id=<list>&url=<campaign URL>&outtime=<ms>` — one comma-id call opens the URL on the whole set; outtime is milliseconds (default 15000), not the Phonefarm seconds. Needs the iMouse Shortcut installed, IP-configured and AssistiveTouch-bound on every phone.
4. `click + send_key` XP `POST /api/mouse/click then GET /api/key/sendkey?key=<ascii url> then fn_key=ENTER` — fallback when the Shortcut is not installed: tap Safari, type the URL, hit Enter. `key=` is ASCII-only, so no Unicode or IDN URLs, and the two XRs have Safari at different home-screen coordinates, so this fallback cannot be mirrored across phones that differ.
5. `get_device_screenshot` XP `GET /api/pic/screenshot?id=<one id>&binary=true` — one capture per phone per timestamp; single-id only. Raw BMP measured at 2 ms / 1,620,054 bytes versus 40.5 ms and a ~21 req/s cap for jpg=true.

6. `none` XP `client-side diff` — compare the settled frame to the golden render and the t+3 frame to the t+10 frame by mean absolute pixel difference; no comparison endpoint exists in either edition.
7. `find_multi_color` XP `POST /api/pic/find-multi-color {id, list:[{first_color:'x|y|RRGGBB-101010,...', rect:[l,t,r,b], similarity}]}` — optional CTA-colour presence check; XP's anchor syntax is absolute points, not the Phonefarm offset/dir scheme, and plug-in gating is unverified (probe for data.code 28 first).
8. `shortcut#19` XP `GET /api/shortcut/device/ip?id=<one id>&state=1` — optional: record the egress IP each phone actually used, for a geo-verification audit trail.
9. `callback` XP `WS ws://127.0.0.1:9911/api – error_push` — the only asynchronous signal that an input call failed at the board, separate from what the screenshot shows.

NEEDS

- IMConsole.exe running and logged in — with only IMService.exe up, both phones showed state 0 and an empty board list.
- iMouse Shortcut installed on each phone, kernel IP configured, run once with prompts allowed, bound to an AssistiveTouch key — required for `/shortcut/exec/url` and `/shortcut/device/ip`; unverified on this rig.
- For the Safari fallback: board bound and state 2, non-empty location_crc, AssistiveTouch, Full Keyboard Access, Auto-Lock Never, portrait lock — none queryable over the API.
- A golden frame per page captured on a known-good phone at matching imgw×imgh, with brightness, True Tone, Night Shift and text size frozen afterward.
- Masks defined for dynamic regions (carousels, countdown timers, personalized blocks, the status-bar clock) before any diff is meaningful.
- UTMs tagged internal, or the rig's egress IP filtered out of analytics, so the pre-flight does not pollute campaign reporting.

WATCH OUT

- The Shortcut is not installed on this rig today, so the one-call open step is currently unavailable and the Safari fallback is what actually runs.
- No text assertion is possible here: `/api/pic/ocr` answers data.code 28 (plug-in not connected), so headline or price copy cannot be checked server-side; everything is pixel- or colour-based until the OCR plug-in is installed.
- `/pic/find-multi-color`'s plug-in independence is not stated in the docs and should be assumed gated, the same as OCR, until a live probe proves otherwise.
- This rig has 19 boards but only two phones mirroring, and one delivers ~4.5 new AirPlay frames/s against ~29 on its twin — a t+0.5 s capture on the slow phone may still show the previous screen (at most ~2 source frames land in that window); push its first checkpoint to at least t+1.5 s.
- A consent or cookie wall should be photographed and flagged, not auto-dismissed — clicking Accept is a consent action and also destroys the regression being tested for.
- The "air_play_ratio" field some drafts cite for matching golden and target phones does not appear in the documented `/device/get` response; match on imgw/imgh instead, which is confirmed and is the constraint that actually governs pixel-diff comparability.
- Licence state can still gate phones coming online per device (`/config/user/info`, user_state, total_license) even though the XP docs carry no per-endpoint shortcut edition note.

SCALE

One `/shortcut/exec/url` call covers the whole fleet regardless of size; capture is one request per phone per timestamp, and raw BMP at a measured p50 of 2 ms (phones proven independent up to ten concurrent pollers at 512 req/s aggregate) keeps three timestamps across the live phones to single-digit milliseconds of kernel time. The wall-clock cost is the 10-second observation window itself: a full three-shot pre-flight completes in roughly 11-15 seconds whether run against 2 phones or a hypothetical 19, because the window dominates, not the request count. Budget 1.6 MB per raw frame with a reused buffer. A monitoring cadence of one pre-flight run every few minutes per URL is easily sustainable; the ceiling is operator-chosen polling frequency, not the rig.

Coordinated publishing and guardrails

Synchronized drop: one multi-id click publishes every account at once

NEEDS SHORTCUT

WITH CAVEATS

Stage the composer on every phone up to but not including the Share button, park each pointer directly on the Share coordinate, verify per-phone readiness from a raw mirror frame, then fire a single click carrying the whole comma-separated id list at To. Because every mouse endpoint accepts a device list and the boards walk it in parallel, N phones tap together in one request instead of N sequential ones — a pure-native variant (assets pre-loaded by hand, ASCII caption) needs no Shortcut at all.

HOW

1. `get_device_list / get_usb_list` XP `GET /api/device/get and GET /api/config/usb/get` — build the drop list from records with a live state, air_pid, non-empty location_crc, and identical imgw/imgh; a phone with different mirror geometry cannot share a coordinate.
2. `mouse_reset_pos` XP `GET /api/mouse/reset?id=<list>` — clears the measured stuck-click state (0.09 mean pixel diff before reset, 141.96 after) whenever the target set changes.
3. `shortcut#7` XP `POST /api/shortcut/album/upload {id, files, album_name, zip, outtime}` — puts the media on each phone; requires the Shortcut installed and hotkey-bound, none of which is verified here.
4. `send_key / click / swipe` XP `GET /api/key/sendkey?fn_key=WIN+h then /api/mouse/click and /api/mouse/swipe` — drives the composer to the screen before Share; a mirrored coordinate is only fair when every phone has the same app build and the same element at that point.
5. `shortcut#10 + send_key` XP `POST /api/shortcut/clipboard/set then GET /api/key/sendkey?fn_key=WIN+v` — the only documented non-ASCII caption route; a pure-ASCII caption can instead go straight in with `/key/sendkey?key=`.
6. `mouse_move` XP `GET /api/mouse/move?id=<list>&x=&y=` — parks every pointer on Share ahead of To. The board walks at ~0.6 ms/px, so pre-positioning removes the dominant cost of the To tap itself.
7. `get_device_screenshot (+ find_multi_color)` XP `GET /api/pic/screenshot?id=<one id>&binary=true` — per-phone readiness check; single-id only, so this is N requests, acceptable because phones are independent.
8. `click` XP `POST /api/mouse/click {id:'<a,b,c,...>', x, y, button:'1'}` — the To tap: one HTTP request for the whole ready set. Button is numeric on XP ('1' left, '2' right), not the Phonefarm 'left'/'right' strings.
9. `get_device_screenshot (+ callbacks)` XP `GET /api/pic/screenshot per phone; WS error_push` — records what actually happened per account; error_push is the only asynchronous board-failure signal and is invisible to an HTTP-only client.

NEEDS

- Every target phone bound to its own board, state 2, non-empty location_crc, and identical imgw×imgh across the set.
- iOS accessibility settings on every phone (AssistiveTouch, Full Keyboard Access, tracking speed, Auto-Lock Never, portrait lock) — none queryable over the API.
- Identical app build and composer layout across the set, so one Share coordinate is correct everywhere.
- For media and non-ASCII captions: the iMouse Shortcut installed, IP-configured and hotkey-bound on each phone, reaching the kernel on TCP 9911.
- IMConsole.exe running and logged in; a WebSocket client on the callback path for asynchronous failure signals.
- Accounts the operator legitimately owns or is authorized to post from, on a platform whose terms permit automated posting.

WATCH OUT

- Simultaneity is at the tap, not at publication: `/mouse/click` returning code 0 only means the kernel accepted the request, and upload time, server-side queuing and per-phone network conditions afterward are outside the API's visibility. This should be described as synchronized initiation, not synchronized publication.
- Parallel board dispatch is measured for two phones only; whether 19 simultaneous HID writes serialize inside the driver stack is unverified and is the single biggest open question for this use case at rack scale.
- The rig's own AirPlay measurements show a 6× throughput gap between two identical XRs on the same Wi-Fi — direct evidence that per-phone link quality already differs enough to spread real publish times by whole seconds even given a perfectly simultaneous click.
- A readiness check via `/pic/find-multi-color` should be assumed plug-in gated, like OCR, until proven otherwise; the dependable path is a client-side pixel test on the already-fetched raw frame.
- Every `/pic/*` endpoint is single-id, so verify-before and verify-after are N requests each, unlike the input phase.
- Simultaneous multi-megabyte media uploads from many phones sharing one access point can saturate its uplink, adding queuing delay that further desynchronizes real publish times — a Wi-Fi capacity constraint, not a software bug, and it scales with fleet size.
- No Phonefarm function name works on this kernel, and the API has no authentication and listens on all interfaces — firewall 9911 to trusted hosts before wiring a scheduled drop to it.
- Coordinated, identically-timed publication across many accounts is a strong and visible signal to platform anti-abuse systems regardless of intent; restrict this to accounts the operator owns or administers, on platforms whose terms permit it, with a stop-on-rate-limit policy rather than any attempt to soften the timing signature.

SCALE

The To action is exactly one HTTP request regardless of set size; with pointers pre-parked, its own pointer travel is near zero, against a measured board-walk cost of ~0.6 ms/px that would otherwise add hundreds of milliseconds per phone. That parallelism is verified for two phones only — 19-phone parallelism is untested. Staging is the real budget: media upload via the Shortcut path is bounded by its 15-60 second outtime per phone and is serialized by the hotkey trigger, while readiness screenshots are single-id at ~2 ms each and phones poll independently (512 req/s aggregate across ten concurrent pollers, input and screenshot traffic

proven not to interfere). For the two phones actually live here, a full stage-verify-fire-confirm cycle is plausibly 10-30 seconds; for a hypothetical 19-phone drop, budget tens of seconds of staging before T0 and expect several seconds of real publish-time spread across accounts even though the tap lands in one request.

Throttle and block detector: stop the loop when the app pushes back

PARTIAL WITH CAVEATS

After every scripted action, pull a cropped raw frame for that phone and test it for a rate-limit dialog: a dimmed background plus an app-drawn card and button fill. On a hit, the controller stops issuing actions for that device, writes a full-frame evidence screenshot, dismisses the dialog, drops the id from the active list and stamps a cool-down timestamp entirely in controller-side state. This is the backstop that distinguishes a genuine platform refusal from the rig's own measured stuck-click state, and it is designed to halt on the platform's own stop signal, not to retry past it.

HOW

1. `get_device_list` XP `GET /api/device/get` — read imgw/imgh per device before the loop starts; every rect and coordinate below is in that mirror-pixel space.
2. `mouse_reset_pos` XP `GET /api/mouse/reset?id=<list>` — pre-flight on the whole target set; without it a "no dialog, no reaction" reading is ambiguous between a block and a stuck pointer.
3. `get_device_screenshot` XP `GET /api/pic/screenshot?id=<id>&binary=true&rect=[0,300,500,800]` — captures the post-action frame cropped to the alert band; single-id only.
4. `none` XP `client-side decode` — the only detection path proven available here: test fixed anchor pixels with per-channel tolerance plus mean-absolute-pixel-difference against the pre-action frame (idle floor 0.02-0.03, real reaction 20-160). Never compare by MD5.
5. `find_multi_color` XP `GET/POST /api/pic/find-multi-color` — kernel-side alternative, unverified here; treat as plug-in gated like OCR until probed.
6. `ocr / ocr_ex` XP `/api/pic/ocr, /pic/find-text` — NOT available on this rig: data.code 28, plug-in not connected. Would otherwise be the robust way to match dialog text.
7. `get_device_screenshot` XP `GET /api/pic/screenshot?id=<id>&save_path=...` — on a hit, write a full-frame evidence screenshot, timestamped and paired with the device's user_name and ip.
8. `click / send_key` XP `GET /api/mouse/click?id=<id>&x=&y= or /api/key/sendkey?fn_key=TAB+b` — dismisses the dialog; TAB+b ("Return, not supported by all apps") is a fallback, not the primary path, and needs its own before/after diff verification.
9. `none` XP `controller-side state` — remove the id from the active list and stamp a cool-down; optionally move it into a visible device group via `/device/group/set` and `/device/set`.
10. `(no equivalent)` XP `WS ws://127.0.0.1:9911/api - error_push, dev_disconnect, usb_change` — the only asynchronous board-level failure signal; not proven to fire for a purely UI-level app refusal, only hardware/link failures.
11. `send_key + mouse_reset_pos` XP `GET /api/key/sendkey?fn_key=WIN+h (twice) then /api/mouse/reset` — renormalises the active set before any resumed batch.
12. `none` XP `controller-side, human in the loop` — nothing in either edition's API says the limit has lifted; resuming a specific account is an operator decision.

NEEDS

- IMConsole.exe running and logged in, with the board bound, connected and location_crc non-empty for every monitored phone.

- Colour anchors and the crop rect captured from a real rate-limit dialog on this app version at this mirror resolution — anchors are resolution-specific and invalidated by any air_play_ratio change.
- Per-phone baselines rather than shared ones: the two XRs have different home screens, so a shared coordinate and a shared "idle" frame do not hold across phones.
- A client-side BMP decoder with a reused frame buffer — each raw frame is 1.62 MB, well over the .NET Large Object Heap threshold.
- Only for the kernel-side variant: the vendor image-search and/or OCR plug-ins, neither installed on this rig.
- Accounts the loop drives must be ones the operator owns and is permitted to automate under the target platform's terms.

WATCH OUT

- The dependable detection path is client-side pixel testing; assume `/pic/find-multi-color` is plug-in gated like OCR (data.code 28) until a live probe proves otherwise, and build the pixel-anchor test as the primary detector, not a fallback bolted on after a failed probe.
- Colour-anchor detection breaks on an app redesign, a dark/light mode switch, a different-width locale dialog, a Dynamic Type change, or a continuous server-side A/B test of dialog copy or layout — it needs a frequent (hours, not weeks) self-test against a stored reference crop.
- A missed detection is not neutral: fail closed. If a post-action frame cannot be classified with confidence, including "no new frame arrived," stop that phone and flag it for a human rather than assuming all clear.
- On the slow XR (~4.5 new frames/s versus ~29 on its twin) the post-action frame may be stale by ~220 ms or more; wait for a frame that actually differs from the pre-action frame rather than polling on a 30 fps assumption.
- A "no dialog, no reaction" reading has at least three indistinguishable causes: the app ignored the tap, the pointer is stuck, or the tap landed off target. Reset before every batch and before every dismiss tap, not only at batch boundaries — the stuck state can appear mid-session on a previously working phone.
- Automated dismissal taps a coordinate based on a colour match; a false positive taps whatever is actually there. Derive the dismiss point from the matched card's own geometry and require two consecutive positive frames before acting.
- This design catches only the app's own explicit modal. A platform that throttles silently (delayed delivery, quiet feed reordering) will not trip it, and the loop keeps running at full rate against a service that is discarding the extra actions.
- The cool-down table, timestamps, pacing and resume decision are all controller-side code; neither edition has a scheduler, a rate limiter or any notion of an account.

SCALE

The detector is not the bottleneck: a cropped raw frame is a few milliseconds, the client-side anchor test on a small crop is sub-millisecond, and input traffic and screenshot traffic measured no interference with each other. But `/pic/screenshot`, `/pic/find-multi-color` and `/pic/ocr` all reject multiple ids, so capture and detection are strictly one request per phone; only reset and hotkey recovery calls fan out over a comma list. The real ceiling is each phone's own AirPlay source rate (measured ~29 fps versus ~4.5 fps on two identical XRs), so on a slow link the post-action frame wait dominates the per-action budget, pushing worst-case overhead to roughly 0.3-1 second per check. In practice, throughput is dictated by the target app's own action cadence and the rate limit this detector exists to respect: treat anything above the low hundreds of actions per hour per phone as almost certain to trip the very dialog being watched for, which is the design's point, not a rig limitation.

Creative testing at panel scale

A/B creative split across device groups with an insights harvest

NEEDS SHORTCUT WITH CAVEATS

Partition the rack into two named groups, push creative A to one group's camera roll and creative B to the other, drive one identical posting flow across the union of both id lists, then hours later re-drive each phone to the post's Insights screen and capture the frame per phone into VariantA/ and VariantB/ folders. The grouping primitives exist precisely so a controller can address "the A half" and "the B half" as stable id lists across runs, but the harvest returns pixels only: with the OCR plug-in gated on this rig, nothing in the API turns an Insights screen into numbers.

HOW

1. `get_device_list` XP `GET /api/device/get` — confirm state, air_pid, vid/pid/uid and location_crc per phone, and record imgw×imgh (500×1080 on both XRs); exclude any phone whose mirror geometry differs.
2. `get_usb_list` XP `GET /api/config/usb/get` — confirms board enumerated and phone attached (state 2); says nothing about the video channel, which is independent.
3. `set_group` XP `GET /api/device/group/set?id=0&name=VariantA then ?id=0&name=VariantB` — id '0' creates a new group; capture the returned numeric ids.
4. `set_dev` XP `GET /api/device/set?id=<idsA>&gid=<gidA> and ?id=<idsB>&gid=<gidB>` — assigns phones to their group; id accepts a comma list, re-readable via `/device/group/get-dev`.
5. `mouse_reset_pos` XP `GET /api/mouse/reset?id=<idsA,idsB>` — clears the stuck-click state on the whole target set before any tap-critical sequence.
6. `shortcut#7` XP `POST /api/shortcut/album/upload {id:'<idsA>', files:[...], outtime:60000}` — two calls, each fanning out over a comma id list; response returns the 10 newest album records as confirmation the variant landed. Raise outtime well above the 15 s default for video.
7. `shortcut#10` XP `POST /api/shortcut/clipboard/set then GET /api/key/sendkey?fn_key=WIN+v` — optional per-phone caption; the only documented non-ASCII text route on XP.
8. `send_key / click / swipe` XP `GET /api/key/sendkey?fn_key=WIN+h (twice) then /api/mouse/click and /api/mouse/swipe` — normalises every phone to a known screen, then drives the posting flow once over the union id list.
9. `shortcut#1` XP `GET /api/shortcut/album/get?id=<one id>&num=10` — verifies the variant is the item the picker will select, one request per phone; XP has no date filter, so sort on create_time.
10. `mouse_reset_pos + click` XP `GET /api/mouse/reset then /api/key/sendkey?fn_key=WIN+h then /api/mouse/click` — hours later, re-normalises and drives each phone to Insights on blind fixed coordinates; no scheduler exists in either edition.
11. `get_device_screenshot` XP `GET /api/pic/screenshot?id=<one id>&binary=true` — captures the Insights frame per phone into VariantA/VariantB folders; single-id, run pollers concurrently.
12. `shortcut#3` XP `POST /api/shortcut/album/del or /api/shortcut/album/clear` — housekeeping so the next run's "newest item" is unambiguous; irreversible and needs Shortcuts delete permissions.

NEEDS

- Accounts owned or authorized, one per phone, with a creator/business account type if the platform gates Insights behind that.
- iMouse Shortcut installed on every phone, kernel IP configured, run once with prompts allowed, bound to an AssistiveTouch key — unverified on this rig; test with `/shortcut/device/ip?state=0` first.

- Shortcuts > Advanced permissions on each phone (run scripts, share large data, and delete permissions if album/del or album/clear is used).
- Standard iOS input prerequisites (AssistiveTouch, Full Keyboard Access, tracking speed, Auto-Lock Never, portrait lock) — none queryable over the API.
- Board bound and cabled per phone, IMConsole.exe running and logged in, and every phone in a mirrored id list sharing the same imgw×imgh and the same element at the same coordinate once the app is open.
- A controller process of your own to hold the delay between post and harvest — neither edition has a scheduling primitive.

WATCH OUT

- The harvest returns pixels, not metrics: `/api/pic/ocr` answers data.code 28, so reading impression and view counts off an Insights screenshot is a human job, a client-side OCR job on the raw frame, or requires the vendor plug-ins.
- The navigation to Insights is blind: with no server-side element location, the harvest runs on fixed coordinates captured once and re-verified by frame diffing, which breaks on any app update that moves the control, and equally on the post scrolling off position, being removed, or still processing — none of which is observable from the API, and a wrong-screen capture returns the same status 200/code 0 as a correct one.
- The group split is pure metadata: nothing in the kernel enforces that group A only ever receives file A; that discipline lives entirely in the controller.
- `/shortcut/album/upload` fans out over a comma id list, but the read-back paths (`album/get` , `album/down` , `clipboard/get` , `pic/screenshot`) do not — the harvest is N requests, not one.
- The "most recent project" grouping is a real strength here (each phone's Recents naturally holds only its assigned variant), but the shared picker-grid assumption is fragile: housekeeping (album/del) must run before each upload, not only after, or a phone with leftover items from a prior run has a different grid offset than a freshly cleared one.
- The stuck-click trap is silent: a phone can ignore every `/mouse/click` while `/mouse/move` and hardware keys keep working, and it returns status 200, code 0 regardless. Verify by frame diff, never by return code.
- Verification timing is per-phone: one XR delivers ~29 new frames/s under motion, its twin ~4.5, so a check on the slow phone must allow ~220 ms per new frame and compare by mean absolute pixel difference, never MD5.
- Whether the shortcut dispatcher is licence-gated on this edition is undetermined; check `user_state` and `total_license` before planning around it.
- Coordinated near-simultaneous posting of near-identical creative from multiple accounts is a pattern platform anti-abuse systems are specifically built to catch, independent of account ownership; this should stay a hard caveat rather than something the design tries to soften.
- The single WebSocket failure watch catches board- and mirroring-level failures but has no signal for "the post failed to publish" or a content-policy interstitial — the posting leg's own verification is thin, confirming the media landed in the album, not that the post published.
- Every album/clipboard/shortcut step depends on the Shortcut being installed, configured and bound on every phone, which is unverified here; until `/shortcut/device/ip` round-trips successfully per phone, roughly two-thirds of this chain, everything after group assignment, cannot run at all.

SCALE

The split is nearly free: two upload calls instead of one, and the posting flow is a single mirrored request per tap over the union id list, since the board walks all listed phones in parallel at the cost of one. The read side does not scale the same way: screenshot, album/get, album/down and clipboard/get are all single-id, so a 20-phone

harvest is 20-plus requests per stage, run concurrently since phones are measurably independent (ten pollers sustained 512 req/s aggregate). Use raw BMP for every capture (2 ms, 1.6 MB) rather than jpg (40 ms, ~21 req/s cap). The real wall-clock cost of the posting leg is pointer travel at ~0.6 ms/px, so a full-height swipe runs ~580 ms and may register as a drag; the slowest link of all is the Shortcut round trip, bounded by its outtime (default 15 s) and involving a hotkey, an iOS Shortcuts run and a LAN callback — a video upload can take far longer and needs outtime raised accordingly. A full A/B cycle across roughly 10 phones per side completes in low single-digit minutes for posting plus 10-30 seconds for a concurrent harvest sweep, well under the hours-long gap between posting and harvest, which is idle time, not compute time.

Ad-verification, outside the box

Geo/carrier ad-serving audit from a cellular-routed phone

NEEDS SHORTCUT WITH CAVEATS

Exploit the rig's two-channel design, video over Wi-Fi, app traffic over the SIM, to audit your own geo- or carrier-targeted campaigns on a genuine carrier-located iPhone. Read each phone's LAN and public IP through the Shortcut, switch on cellular data, open the placement, and capture the served creative with the phone's Wi-Fi still associated (so mirroring keeps working) but its routed traffic on the carrier. The evidence artifact is a timestamped, IP-attributed frame of the real ad on a real handset on a real carrier IP, which is exactly the population geo/carrier targeting was bought against and which a datacentre VPN or emulator cannot reproduce.

HOW

1. `get_device_list` XP `GET /api/device/get` — gate the audit on a live state and air_pid, and read imgw/imgh and the LAN `ip` field, which is free and needs no Shortcut round trip.
2. `mouse_reset_pos` XP `GET /api/mouse/reset?id=<list>` — clears the stuck-click state before any tap-critical sequence.
3. `shortcut#24` XP `GET /api/shortcut/device/ip?id=<id>&state=0` — reads the LAN IP; single-id only. A match against step 1's ip field is the cheapest end-to-end proof the Shortcut path works; data.code 30 means the hotkey never fired or the phone cannot reach the kernel.
4. `shortcut#24` XP `GET /api/shortcut/device/ip?id=<id>&state=1` — records the baseline public egress IP before the cellular flip, archived with deviceid and timestamp.
5. `none` XP `manual, on the phone: Settings > Wi-Fi > Edit > IPv4 > Manual` — enter an IP address and subnet mask and leave the router/gateway field empty, per the vendor's own settings-page instructions; the phone stays associated to Wi-Fi (mirroring and the Shortcut callback keep working) while routed traffic moves to cellular.
6. `shortcut#20` XP `GET /api/shortcut/switch/cdpd?id=<list>&state=1&outtime=20000` — turns cellular data on across the whole audit set in one fan-out request.
7. `shortcut#24` XP `GET /api/shortcut/device/ip?id=<id>&state=1` — re-reads the public IP and asserts it changed to the carrier's pool; a renewed timeout here means the manual IPv4 config broke the LAN return path.
8. `shortcut#13` XP `GET /api/shortcut/exec/url?id=<list>&url=<placement-url>` — opens the placement; a tap into an installed app's feed is the alternative when the placement is not a bare URL.
9. `swipe` XP `GET /api/mouse/swipe?id=<list>&direction=up&len=0.5` — scrolls the ad slot into view; keep len modest, since the board walks at ~0.6 ms/px and a full-height stroke may read as a drag.

10. `get_device_screenshot` XP `GET /api/pic/screenshot?id=<id>&binary=true` — captures the served creative; single-id, raw BMP measured at 2 ms.
11. `shortcut#21` XP `GET /api/shortcut/switch/cdpd?id=<list>&state=0` — ends the cellular session and stops metered data accruing between runs.
12. `NOT USED` XP `/api/shortcut/switch/wlan?state=0` — deliberately excluded: it kills both the AirPlay channel and the Shortcut's return path, leaving the phone controllable but blind and every later call timing out.
13. `NOT AVAILABLE` XP `/api/pic/ocr, /pic/find-image` — the kernel cannot identify the advertiser or read the creative's text on this rig (data.code 28); classification happens off-box on the archived frames.

NEEDS

- A SIM with an active data plan in each audit handset — neither XR on this rig is documented as having one, so this is unproven hardware, not a software gap.
- The iMouse Shortcut installed on each phone, kernel IP configured, run once with prompts allowed, and bound to an AssistiveTouch key — unverified here and load-bearing for the whole workflow, unlike the optional Shortcut use in the other entries in this section.
- The board bound and connected (state 2), since every Shortcut call is triggered by the board sending the hotkey.
- The manual Wi-Fi IPv4 configuration on every audit phone, per-phone, surviving reboots but redone if the LAN is re-addressed.
- Standard input prerequisites (AssistiveTouch, Full Keyboard Access, tracking speed, Auto-Lock Never, portrait lock) and IMConsole.exe running and logged in.
- A licensed account, and authorization to audit the campaigns and properties in question, plus awareness of the publisher's automated-access terms.

WATCH OUT

- Every function in this chain is documented and correctly named for this edition; nothing is invented. The caveats are all prerequisite and observability, not API existence.
- Two prerequisites are unverified and either one blocks the workflow entirely: whether the handsets have SIMs with data, and whether the Shortcut is installed and hotkey-bound. This should be read as "achievable once two unconfirmed hardware/software prerequisites are separately verified," not as ready to run today.
- `/shortcut/device/ip` and `/pic/screenshot` are single-device; only the cellular toggle, URL open and swipe/reset fan out over a comma list.
- No server-side recognition is available: the kernel hands you pixels, not an advertiser identity. All creative identification happens off-box.
- The no-gateway trick forces cellular egress at the IP-routing layer, but many ad exchanges use device location services for geo-targeting, not only IP geolocation. If the campaign's targeting is location-services-driven, this rig's fixed physical location keeps reporting that location regardless of which network carries the traffic, and the audit would silently validate the wrong geo dimension while looking successful end to end. State explicitly which signal is being audited, carrier/IP, and flag that location-based targeting needs physically relocating the handset, which this rig cannot do.
- iOS's behaviour when a Wi-Fi interface has no gateway but stays associated is not guaranteed to route every app's traffic over cellular equally; an app already running before the IPv4 change can hold stale sockets. Force-quit and relaunch the target app after the change, and verify egress IP with the Shortcut round trip before trusting a capture.
- Repeatedly loading ad placements, even passively for screenshot capture with no click, generates billable impressions and may be scored as invalid traffic by verification vendors; this is a hard constraint on run

frequency and volume, not a tuning knob.

- The technique rests on a vendor settings-page instruction that could not be independently cross-checked against this rig's own measured hardware record, which is silent on cellular/SIM capability entirely.
- The stuck-click trap and the mirror-pixel coordinate space apply here exactly as elsewhere, and the slow-phone framerate (~4.5 fps versus ~29 fps) lengthens, but does not break, each placement's render confirmation if the carrier-routed phone happens to be the weak-link handset.

SCALE

Per phone per placement, the fixed costs are two single-id IP round trips (seconds each, bounded by the 15 s default outtime), one URL open or tap, one or two short swipes, and a settle wait for the ad to render. Capture itself is negligible at 2 ms for raw BMP. Fan-out is asymmetric: the cellular toggle, URL open, click, swipe and reset all take a comma id list and cost the same for one phone or many, while the IP reads and screenshot are single-id loops. Phones do not contend, ten concurrent pollers sustained 512 req/s aggregate, so a rack scales close to linearly with one poller per phone; the binding constraint is cellular data volume and per-SIM plan cost, not API throughput. The realistic per-placement budget is 15-30 seconds end to end, dominated by app and ad-server latency rather than the kernel, and the number of SIM-equipped, Shortcut-bound handsets available is the actual ceiling on this rig, currently unconfirmed and possibly zero.

Platform-by-platform playbook

Every entry below is built from the same eight or nine XP primitives: a device read, an album upload, a key send, a mouse click, a clipboard set-and-paste, a mouse drag or wheel, and a screenshot. What changes per platform is the composer layout, whether the text needs Unicode, whether the platform itself offers scheduling, and how much the missing OCR and image-search plug-ins cost you at the verification step. Every entry needs the iMouse iOS Shortcut for anything beyond ASCII typing or a PC-to-phone file transfer, and every entry's proof of success is a pixel comparison, because this rig's OCR reports data.code 28, plug-in not connected, and the image-search plug-in should be assumed absent until proven otherwise.

Short-form video: TikTok, Instagram, YouTube Shorts

TikTok: album push to a verified post

NEEDS SHORTCUT · WITH CAVEATS

One `/shortcut/album/upload` call with a comma-separated id list lands an mp4 in Recents on every phone at once and returns the ten newest album records as proof it arrived. From there the TikTok composer is driven by taps in each phone's own mirror-pixel space, an ASCII caption goes in through `/key/sendkey`, and an emoji or CJK caption goes in through the clipboard-and-paste route. This is the workload the rack exists for, and it is also the clearest demonstration of its blind spot: with the OCR plug-in reporting code 28 on this rig, a raw pixel difference is the only success signal available, and it cannot tell a successful post from a guideline rejection.

HOW

1. `get_device_list` XP `/device/get?id=<a,b,c>`, reads state, air_pid and imgw/imgh (500x1080 on these XRs, not the 414x896 iOS point space); exclude any phone whose imgw/imgh differs from the master.
2. `shortcut#7` XP `/shortcut/album/upload`, pushes the clip into Recents on every listed phone in one call. The response's ten newest album records confirm the clip landed before any tile is tapped; raise outtime well past the 15000 ms default for a 30 MB clip.
3. `send_key` + `mouse_reset_pos` XP `/key/sendkey?fn_key=WIN+h (x2) then /mouse/reset`, two Home presses return every phone to page one, then the reset clears the silent stuck-click state that `/mouse/up` cannot fix.
4. `click` / `shortcut#13` XP `/mouse/click or /shortcut/exec/url`, opens TikTok. Coordinates are captured per phone: the two XRs on this rig have different home screens, so one coordinate hits a different app on each.
5. `click` XP `/mouse/click`, walks the composer: plus tab, Upload, first Recents tile, Next, Next. Each tap costs about 0.6 ms per pixel of pointer travel.
6. `send_key` XP `/key/sendkey?key=%23fyp+%23unboxing`, types the ASCII hashtags directly; ASCII punctuation including # is supported, but typing speed and maximum length are undocumented.
7. `shortcut#10` + `send_key` XP `/shortcut/clipboard/set then WIN+v`, the only Unicode path on XP, carries the emoji or CJK half of the caption. `sleep` is in seconds, `outtime` in milliseconds.
8. `click` + `get_device_screenshot` XP `/mouse/click then /pic/screenshot?binary=true`, taps Post, then verifies by mean absolute pixel difference (idle floor 0.02-0.03, real reaction 20-160) with a rect crop on the Post button, since MD5 always reports a false "reacted."

9. `click` + `shortcut#10` + `send_key` XP `comment sheet: /mouse/click, /shortcut/clipboard/set, WIN+v`, the same clipboard pattern for replies, addressed purely by screen position, since there is no OCR to read comment text.

NEEDS

- iMouse Shortcut installed, IP-edited, run once with all permissions, and bound to an AssistiveTouch key on every phone; not verified as done on this rig.
- Board bound and connected (state 2 in `/config/usb/get`) with mouse parameters present (`location_crc` non-empty).
- Phone accessibility settings the API cannot query: AssistiveTouch on, Full Keyboard Access on, max tracking speed, Auto-Lock Never, portrait lock.
- IMConsole.exe running and logged in; with only IMService.exe up, both phones read state 0 and the board list is empty.
- TikTok already signed in on each phone, on an account you own, with composer coordinates captured per phone and per app version.

WATCH OUT

- Verification is pixel-diff only: a guideline rejection, a duplicate-content warning and a successful post are indistinguishable at whole-frame level, and a rect-cropped check on the Post button narrows this without closing it.
- Every coordinate is app-version and phone specific; nothing in the API finds a UI element without a plug-in, so the macro breaks silently on a TikTok update.
- The two XRs deliver new frames at very different rates, about 29/s versus about 4.5/s under motion; a verification poll tuned to the fast phone will read a stale frame on the slow one as evidence nothing happened.
- Distinct captions per phone cost a full clipboard round trip each, adding 1-3 s per phone rather than folding into a flat post-time estimate.
- `/mouse/reset` must be resent whenever the batch's id list changes, including when a run is split into smaller posting groups.
- Album-upload outtime should scale with clip size and each phone's Wi-Fi energy-mode cap (roughly 5, 10 or 20 Mbps); a 30 MB clip can take 12-48 s depending on which cap applies.

> TRAP **A pixel match is not a success signal**
>
> Driving many phones from one comma-separated id list produces byte-identical actions at the same millisecond, which is exactly the signature integrity systems are built to find. That is a mechanical fact about the transport, not something to engineer around. Treat a rate-limit or "Action Blocked" dialog as a stop signal for that phone's queue, and capture its pixel-diff signature in advance, since there is no OCR to read the dialog text.

SCALE

About 20 taps per post, dominated by pointer travel at roughly 0.6 ms per pixel (a corner-to-corner tap on the 500x1080 mirror is about 600 ms). A shared caption across N identically laid-out phones costs the same wall time as one, since the board walks a comma-separated id list in parallel: call it 40-60 s per post, up to roughly 60-90 batch-cycles an hour run back to back. Distinct captions serialize the clipboard step, so a 19-phone batch

with distinct captions runs closer to 80-160 s per cycle, around 22-45 cycles an hour for the whole rack. Verification is about 2 ms per phone per check on a reused BMP buffer. These are transport ceilings; TikTok's own rate limits and detection will very likely force a lower real cadence.

Instagram: Reel, Story, DM reply and Follow

NEEDS SHORTCUT WITH CAVEATS

Instagram's four short-loop actions map onto the same primitives as the TikTok flow but lean harder on Unicode: the reel and story assets go to Recents in one album-upload call, captions and DM text arrive via clipboard-and-paste since `/key/sendkey` is ASCII-only, and a two-tap Follow is confirmed by reading the button's own pixels out of a raw screenshot. It is the platform where operators want the most non-Latin text and the shortest loops, and both are covered, the hard part is verification, not the action.

HOW

1. `get_device_list` XP `/device/get?id=<a,b,c>`, confirms mirroring is live and reads imgw/imgh for the coordinate space; excludes any follower whose mirror geometry differs from the master.
2. `shortcut#7` XP `/shortcut/album/upload`, pushes the reel mp4 and the story jpg in one call, since `files` is a string array. Read the response's newest-record list rather than assuming "first tile" ordering.
3. `shortcut#1` XP `/shortcut/album/get`, optionally re-reads the album (single id only) to disambiguate which upload is which by `create_time`.
4. `send_key` + `mouse_reset_pos` XP `WIN+h (x2) then /mouse/reset`, normalises state and clears the stuck-click trap.
5. `click` XP `/mouse/click`, drives the Reel composer: open app, plus, Reel, first tile, Next, Next, caption field. Coordinates are per phone, since the two XRs share no home-screen layout.
6. `shortcut#10` + `send_key` XP `/shortcut/clipboard/set then WIN+v`, the only Unicode caption route; prove the round trip once per phone with `/shortcut/clipboard/get`.
7. `shortcut#13` / `click` XP `/shortcut/exec/url?url=instagram://camera (unverified) or tap-through`, opens the Story camera. The URL scheme is documented in neither vendor doc and may fall through to Safari.
8. `click` + `shortcut#10` + `send_key` XP `DM reply: click, clipboard/set, WIN+v, ENTER`, the same clipboard pattern, addressed purely by position, with no way to confirm which thread is open.
9. `click` + `get_device_screenshot` + `find_multi_color` XP `/mouse/click then /pic/screenshot?binary=true, optionally /pic/find-multi-color`, taps Follow, then reads the button's own pixels (blue before, grey after) out of the raw BMP; treat the color-search endpoint as a bonus, not the primary method.
10. `get_device_screenshot` XP `/pic/screenshot?binary=true&rect=[l,t,r,b]`, verifies every write step by mean absolute pixel difference against the pre-tap frame, cropped to the button or dialog region.

NEEDS

- iMouse Shortcut installed, IP-edited and bound to an AssistiveTouch key on every phone; album upload and every clipboard call depend on it.
- Board bound and connected with mouse parameters present; Full Keyboard Access on for WIN+v and ENTER; AssistiveTouch, max tracking speed, Auto-Lock Never, portrait lock.
- Instagram signed in on each phone on accounts you own; Reels composer, camera, DM and profile coordinates captured per phone and per app version.
- A Follow-button color reference sampled at the phone's current settings, with brightness, True Tone, Night Shift and Dark Mode fixed and matching across phones.

WATCH OUT

- No OCR on this rig, so the DM step cannot confirm which thread is open and the Follow step cannot read the button label; both are positional or color-based guesses, and both should sit behind a screenshot review gate before running unattended.
- `/pic/find-multi-color`'s plug-in status here is unverified; treat client-side pixel reading of the raw BMP as the primary method.
- Gallery ordering is app behavior, not an API guarantee; confirm the intended asset from the album-upload or album-get response rather than trusting "first tile."
- Only the input and clipboard-set calls fan out; screenshot, find-multi-color, and album-get are all single-id.
- Identical-timing Follow actions fanned out to many accounts via one comma-separated id list is one of the most heavily policed automation signatures on the platform, independent of any attempt to look human; that is a direct, factual consequence of the transport.
- Video upload duration is unbounded and unmeasured on this rig; raise outtime for album upload well past 120 s for a video and poll `/shortcut/album/get` rather than using a fixed sleep.
- Only 2 of the rack's 19 boards currently have a phone bound and mirroring, so every fan-out claim is proven at n=2, not at rack scale.

> TRAP **Reel trims need two fingers, this rig has one**
>
> Instagram's Reels composer can require a pinch-to-zoom or a two-finger trim-handle drag when a source clip's aspect ratio does not already match 9:16. The rig drives exactly one HID pointer per phone, with no multi-touch, so any step needing two simultaneous contact points cannot be automated at all. Restrict the chain to source video already cropped to 9:16 before upload, or do that step by hand.

SCALE

A Reel post is roughly 15 taps (30-45 s), a Story about 6 taps (about 15 s), a DM reply 4 taps plus one clipboard round trip (about 5 s), a Follow 2 taps (about 3 s); shared inputs fan out at no extra cost, per-phone captions add one clipboard call each. The board-time ceiling is tens of such actions an hour per phone, but real throughput is capped lower by video-ingest wall time on a shared Wi-Fi link, by the mandatory human review of blind DM and Follow steps, and by Instagram's own throttling on Follow and DM, which is the real ceiling regardless of rig speed. A realistic sustained rate is low single digits of Reels an hour per phone and tens of Stories or Follows an hour per phone at most before platform-side risk dominates.

YouTube Shorts: album upload with in-app scheduling

NEEDS SHORTCUT WITH CAVEATS

The YouTube app's upload flow ends on a Visibility screen that includes Schedule, so a Short can be posted or scheduled entirely on the phone: push the mp4 into the album, normalise with two Home presses and a pointer reset, walk plus, Create a Short, gallery, Next with taps, paste the caption via clipboard, set the schedule with short drags, tap Upload. A clip already staged for another platform is reused with no second PC-to-phone transfer, and because Visibility carries Schedule, the rack does not have to be awake at publish time.

HOW

1. `shortcut#7` XP `/shortcut/album/upload?files=[...]&outtime=180000`, pushes the clip (skip if already staged for another platform); the response's ten newest album records confirm arrival before the UI walk begins.
2. `send_key` + `mouse_reset_pos` XP `WIN+h (x2) then /mouse/reset`, normalises state and clears the stuck-click trap; budget this before every Short, not once per session, since the trap has been observed to

reappear.

3. `click` / `shortcut#13` XP `/mouse/click or /shortcut/exec/url?url=youtube://`, opens YouTube; the custom URL scheme is unverified, and the icon-tap route cannot be mirrored across phones with different home screens.
4. `click` XP `/mouse/click`, walks plus, Create a Short, gallery icon, first tile, Next; coordinates are mirror-pixel (500x1080), not iOS points, and cannot be located at runtime without OCR.
5. `shortcut#10` + `send_key` XP `/shortcut/clipboard/set then WIN+v`, sets the caption for any non-ASCII text, since `key=` is English/numbers only.
6. `mouse_down` / `mouse_move` / `mouse_up` XP `/mouse/down, repeated /mouse/move, /mouse/up (or /mouse/swipe)`, drives Visibility, Schedule, and the date/time wheel; keep strokes to 100-200 px given the roughly 0.6 ms/px board walk.
7. `click` XP `/mouse/click`, taps Done then Upload Short; the transfer continues in the background, so Auto-Lock must be Never or the phone sleeping mid-upload both stalls it and drops the mirror.
8. `get_device_screenshot` XP `/pic/screenshot?binary=true&rect=[l,t,r,b]`, verifies the upload started and, later, completed, by mean absolute pixel difference on the banner region.

NEEDS

- iMouse Shortcut installed and IP-edited, with Allow Sharing Large Amounts of Data on for the video transfer; never probed on this rig.
- YouTube app logged in to a channel the operator owns, on each phone.
- Board bound, `location_crc` non-empty, AssistiveTouch, Full Keyboard Access, max tracking speed, Auto-Lock Never, portrait lock.
- Composer, gallery and Visibility coordinates captured per app version in 500x1080 mirror-pixel space.

WATCH OUT

- Video transfer duration is undocumented and unmeasured; a timeout returns code 30 with no partial-progress signal, so raise outtime well past the 15000 ms default.
- The "Create a Short" entry point and the Visibility screen move between app versions, and nothing in the API can find them at runtime while OCR and image search are unavailable.
- Upload completion cannot be read as text on this rig; a pixel diff on the banner region tells you something changed, not whether it says success or failure, since "Uploading," "Processing," and "Upload failed" look similar in a region diff without OCR.
- Two separate transfers are involved (PC to phone, then phone to YouTube over its own Wi-Fi), each with its own unmeasured bottleneck; a 50-100 MB clip can add tens of seconds to minutes before the UI walk can even begin.
- iOS can suspend background network activity depending on Background App Refresh permission, which is not queryable through this API and is a silent failure mode beyond Auto-Lock.

> TIP **The schedule wheel tracks the finger, it does not need a flick**
>
> YouTube's Visibility date/time control is a position-driven wheel, not a velocity-gated gesture like a swipe-to-dismiss list. A slow board-driven drag that ends at the correct offset should land the right value even at 0.6 ms/px, because the wheel tracks finger position continuously. Budget for the cost of the drag itself (a 200 px scroll is about 120 ms of board time, done once), not for extra retries against an assumed flick threshold.

SCALE

About 14 taps plus one clipboard round trip and one picker drag per Short, roughly 40-60 s per phone excluding file transfer, and the same whether one phone or N are listed since the driving calls fan out across a comma-separated id list. That is a physics ceiling of roughly 60-80 Shorts an hour per phone if content and transfer time were free. Real throughput is dominated by two things the chain-only number leaves out: per-clip file transfer time, likely tens of seconds to minutes, and YouTube's own upload-processing wall time before Visibility can be reached reliably. A defensible planning number is 10-20 Shorts an hour per phone sustained, not 60-80; the network and app are the bottleneck, not the API.

Text-first and native-scheduling platforms: Threads and Facebook

Threads: text posting and replies via clipboard fan-out

NEEDS SHORTCUT WITH CAVEATS

Threads needs no media pipeline; the whole value is text. Normalise every phone, tap into the composer, push the post body onto the phone clipboard with `/shortcut/clipboard/set`, which takes a comma-separated id list so identical copy reaches N phones in one request, paste with WIN+v, tap Post. Replies are the same six taps on a reply sheet. This is the cheapest platform per post on the rack and the cleanest template for any text-only surface, but the whole text path rides on an iMouse Shortcut round trip that has never been exercised on this rig.

HOW

1. `send_key` XP `/key/sendkey?fn_key=WIN+h (x2)`, presses Home twice; the first leaves the app, the second returns to page one, and it doubles as a liveness probe since a Home press produces a whole-frame diff around 142.
2. `mouse_reset_pos` XP `/mouse/reset?id=<ids>`, clears the silent stuck-click state before any tap-critical sequence; send it whenever the target set changes.
3. `click` XP `/mouse/click?id=<ids>&x=&y=`, opens Threads. Button is numeric on XP ('1' left), not the Phonefarm strings; the two XRs have different home screens, so one coordinate is not valid across both.
4. `click` XP `/mouse/click`, taps the compose control in the bottom tab bar; the position is an app-UI fact absent from any vendor doc and must be re-captured per app version.
5. `shortcut#10` XP `/shortcut/clipboard/set?id=<ids>&text=&sleep=1&outtime=15000`, pushes the post body, emoji included; `sleep` is seconds, `outtime` is milliseconds; POST JSON in UTF-8 rather than a query string.
6. `send_key` XP `/key/sendkey?fn_key=WIN+v`, pastes into the focused field; if the field was never focused the paste silently does nothing, and `fn_key=OPENKeyboard` can force the keyboard up as a focus check.
7. `click` XP `/mouse/click`, taps Post.
8. `shortcut#10` + `send_key` + `click` XP `reply sheet: clipboard/set, WIN+v, click`, the reply chain is the same three calls on the post's reply sheet, serialised per phone since the Shortcut is triggered by a HID hotkey through the board.
9. `get_device_screenshot` XP `/pic/screenshot?id=<single id>&binary=true`, verifies the post landed by mean absolute pixel difference; single-id only, so run one poller per phone.

NEEDS

- iMouse Shortcut installed, IP-edited and bound to a custom AssistiveTouch key on every phone; not yet verified present on this rig, no shortcut endpoint has ever been probed here.

- Shortcuts > Advanced: Allow Running Scripts, Allow Sharing Large Amounts of Data; phone must reach the kernel on TCP 9911 to post the Shortcut's result back.
- Board bound (state 2), `location_crc` non-empty; AssistiveTouch, Full Keyboard Access, max tracking speed, Auto-Lock Never, portrait lock.
- Threads app logged in on accounts the operator owns; composer and Post coordinates captured per app version in 500x1080 space.
- Licence headroom checked via `/config/user/info`: unlicensed phones are refused.

WATCH OUT

- The whole text path depends on the iMouse Shortcut, a per-phone manual install never verified on this rig; without it, posting is limited to ASCII via `/key/sendkey`, no emoji, no non-Latin script.
- XP has no `send_text` and no asynchronous completion signal for text delivery; the only confirmation is `/shortcut/clipboard/set` returning code 0 within outtime, optionally cross-checked with `clipboard/get`.
- Mirrored icon taps are invalid across phones with different home screens; fan-out is only safe for phones with identical layouts and identical imgw/imgh.
- No OCR on this rig, so a rate-limit or "Action Blocked" dialog cannot be read by the kernel; detecting it requires client-side template matching on raw frames.
- A positive pixel diff on the Post button region is necessary, not sufficient, evidence of success: a rate-limit interstitial or an error toast also changes enough pixels to register as a reaction.
- The 10-20 s per-post figure is an explicit guess; the Shortcut round trip has never been exercised end to end on this rig, so the whole text-posting path is unproven, not just slow by an unknown amount.
- Posting identical text from several accounts via one comma-id fan-out is a direct, mechanical consequence of the transport and is already treated as spam behavior by platform policy; that is a factual risk to name, not a detection problem to solve technically.

SCALE

Roughly six taps plus one clipboard round trip per post, about 2-4 s of pure board time; the clipboard call is bounded by outtime (default 15 s) and in practice is the long pole, so call it 10-20 s per post per phone with that figure unverified. Identical copy to N phones costs the same wall time as one, since input calls fan out; distinct copy costs one clipboard call per id. Verification does not fan out (single-id) but is nearly free, raw BMP p50 2 ms; the real ceiling on it is the AirPlay source rate, about 29 new frames/s on the fast phone against about 4.5 on its identical twin. If the unverified figure holds, the chain-only ceiling is roughly 150-350 posts an hour per phone, but Threads' own automated-posting throttling will cap real throughput long before the API or board time does.

Facebook: Business Suite scheduling, Marketplace listings, comment hiding

NEEDS SHORTCUT

WITH CAVEATS

Facebook is the one large surface with first-class in-app scheduling, so the rack only has to drive the Meta Business Suite composer and its date picker. Photos arrive in bulk through album upload, the caption comes from clipboard-and-paste, ASCII fields like price and title are typed directly, the date picker is driven with a real down-move-up drag, and hiding a comment is a long press expressed as a click with `time=800`. Because Business Suite schedules natively, a single supervised sitting can front-load a week of Page posts per phone instead of requiring the rack to be awake at each publish time.

HOW

1. `shortcut#7` XP `/shortcut/album/upload?files=[...]&zip=0&outtime=120000`, seeds listing or post photos from the PC; the response returns the 10 newest album records to confirm arrival, so a batch of more than 10 images cannot be fully confirmed from that one response alone.
2. `send_key` + `mouse_reset_pos` XP `WIN+h (x2) then /mouse/reset`, normalises state and clears the stuck-click trap.
3. `click` XP `/mouse/click`, opens Meta Business Suite, taps Create post, taps the text area; positions are app-UI facts absent from every vendor doc.
4. `shortcut#10` + `send_key` XP `/shortcut/clipboard/set then WIN+v`, sets a caption with emoji; required for anything outside ASCII.
5. `click` XP `/mouse/click`, taps Add photo and the Recents tiles; the freshly uploaded files are the newest items, checked against the upload response before tapping.
6. `mouse_down` / `mouse_move` / `mouse_up` XP `/mouse/down, repeated /mouse/move, /mouse/up (or /mouse/swipe, /mouse/wheel)`, toggles Schedule and sets the date/time wheel; keep strokes to 100-200 px given the 0.6 ms/px board walk.
7. `click` XP `/mouse/click`, taps Schedule to commit the Page post.
8. `click` XP `/mouse/click`, walks the Marketplace path: Marketplace tab, Sell, Item, Add photos, select; coordinates differ per region and account type.
9. `send_key` + `shortcut#10` XP `/key/sendkey for Title and Price, clipboard/set + WIN+v for Description`, types ASCII fields directly and pastes anything with a currency symbol, accent or emoji.
10. `click` XP `/mouse/click`, taps Next then Publish.
11. `click` (with `time`) XP `/mouse/click?time=800`, hides a comment via a long press, no separate API call needed; 800 ms is a starting value and more likely to over-fire than under-fire, tune downward first.
12. `get_device_screenshot` XP `/pic/screenshot?id=<single id>&binary=true&rect=[l,t,r,b]`, verifies each stage by mean absolute pixel difference, single-id only.

NEEDS

- iMouse Shortcut installed and IP-edited on every phone, with Allow Sharing Large Amounts of Data on; never probed on this rig.
- Meta Business Suite and Facebook logged in with Page admin rights per phone, since scheduling is per Page.
- Board bound and connected, `location_crc` non-empty, AssistiveTouch, Full Keyboard Access, max tracking speed, Auto-Lock Never, portrait lock.
- Date-picker behavior tested on one phone before any fan-out.
- Coordinate sets captured per app version for Business Suite, Marketplace and the comment sheet.

WATCH OUT

- Media upload timing is unbounded in the docs and unmeasured here; a timeout returns code 30 with no partial-progress signal.
- Business Suite and Marketplace coordinates are per app version, region and account type, and nothing in the API can locate them at runtime without OCR or image search.
- `/key/sendkey` cannot type a currency symbol, accent or emoji; anything outside ASCII must go through the clipboard.
- A rate-limit or "Action Blocked" interstitial cannot be distinguished from a successful publish by pixel diff alone, both change the screen well above the noise floor; this is a real, currently unclosed gap, not just a

documented limitation.

- Marketplace listing fan-out across phones is generally unsafe: identical listings from several accounts is exactly what commerce policy restricts, and an accepted offer is a real transaction the operator must fulfil, unlike reversible Page-post scheduling.
- Photo attachment relies on the uploaded file being the newest item in Recents; check the album-upload response's newest records rather than assuming the first tile.
- Only 2 of 19 boards currently have a phone bound and mirroring; fan-out and rack-scale framing are unverified beyond that.

> TIP **The date picker is a wheel, not a flick gesture**
>
> iOS's native picker tracks the finger 1:1 while pressed rather than requiring an inertial flick, so it is a direct-manipulation control, not a velocity-gated one. A short drag of roughly one row height, about 40-50 px, repeated a few times should move the wheel deterministically. Treat it as moderately risky and cheaply testable in a handful of short attempts before committing to fan-out, not as an unknowable step.

SCALE

A scheduled Business Suite post is roughly 15 taps plus one picker interaction, about 45-60 s per phone, dominated by app response and the picker, not the API; a Marketplace listing is about 20 taps plus ASCII typing, roughly 60-90 s. The composer half of this playbook mirrors for free across phones with identical app versions, layouts and imgw/imgh. For Page scheduling this gives a technical ceiling of about 40-70 scheduled posts an hour per phone. Marketplace listings imply a similar per-phone ceiling, but each one is a real binding commercial listing, so the meaningful constraint is how many genuine, deliverable items and prices a human can source and verify per hour, not board time. Comment hiding is a few seconds each, bounded by comment volume and by keeping a human in the moderation decision.

Niche surfaces: Pinterest and Snapchat

Pinterest: catalogue pins with native publish-later

NEEDS SHORTCUT · WITH CAVEATS

Pinterest is the most automation-friendly of the platforms covered here. The composer takes an album photo, an ASCII destination link and title typed directly, a Unicode description delivered by clipboard-and-paste, a board pick, and on business accounts a publish-at-a-later-date toggle. Saves from the feed are a three-tap macro. A whole day's catalogue can be pushed to N phones in one album-upload call with `zip=1`, after which each pin is a per-phone script; it is also the only platform here whose own product offers scheduling.

HOW

1. `get_device_list` + `get_usb_list` XP `/device/get and /config/usb/get`, reads imgw/imgh for the coordinate space, requires a live `air_pid` for video and board state 2 for control; excludes any phone whose imgw/imgh differs from the master.
2. `shortcut#7` XP `/shortcut/album/upload?zip=1`, pushes the day's catalogue images in one call; the doc recommends `zip=1` for many small images. The response's 10 newest records confirm arrival, but a catalogue larger than 10 needs spot checks or a split batch, since one timeout covers the whole request with no per-file granularity.
3. `send_key` + `mouse_reset_pos` XP `WIN+h (x2) then /mouse/reset`, normalises state and clears the stuck-click trap.

4. `click` XP `/mouse/click`, opens Pinterest and starts a Pin: icon, plus, Pin, image tile, Next; no label-based fallback exists without OCR.
5. `send_key` XP `/key/sendkey?key=... for title and URL`, types the title and destination link directly, since both qualify as ASCII and need no Shortcut round trip, the real efficiency of this playbook.
6. `shortcut#10` + `send_key` XP `/shortcut/clipboard/set then WIN+v`, delivers a Unicode description; use the documented `sleep` parameter rather than a blind delay, and prove the round trip with `clipboard/get`.
7. `click` + `mouse_wheel` XP `/mouse/click plus /mouse/wheel?direction=down&len=1-127`, picks the board and sets publish-at-a-later-date; whether an iOS native picker wheel honors HID scroll is unverified, with a down-move-up drag as the fallback if it does not.
8. `click` + `get_device_screenshot` XP `/mouse/click then /pic/screenshot?binary=true[&rect=[l,t,r,b]]`, creates the pin and verifies by mean absolute pixel difference, cropped to the composer chrome if the page animates.
9. `click` + `mouse_wheel` XP `/mouse/click, /mouse/wheel?direction=down`, saves from the feed, three taps; this cannot be comma-id mirrored even though the endpoint accepts a list, since each phone's feed shows different pins at the same coordinate.
10. `shortcut#3` XP `/shortcut/album/del or /shortcut/album/clear`, optional album cleanup after the run; `/shortcut/album/clear` is XP-only and deletes every item in the named album with no undo.

NEEDS

- iMouse Shortcut installed, IP-edited and bound to an AssistiveTouch key on every phone, for both album upload and clipboard set.
- Full Keyboard Access on, AssistiveTouch on with max tracking sensitivity, max tracking speed, Auto-Lock Never, portrait lock.
- Board bound (vid and pid set together), `location_crc` non-empty.
- Pinterest logged in per phone on an account the operator owns, a business account if native publish-later is wanted.
- Composer coordinates captured per phone and per app version, in 500x1080 mirror-pixel space.

WATCH OUT

- Without the Shortcut, only content already on the phone can be pinned and only ASCII text typed; no file can be delivered from the PC.
- No OCR or image search on this rig, so every composer control is a captured coordinate that breaks on a Pinterest UI update.
- Clipboard paste can lag the write on some handsets; use the documented `sleep` parameter rather than a blind delay.
- The two text-entry paths have different verification stories: `/key/sendkey` has no read-back, while `/shortcut/clipboard/set` has a documented get-back-and-compare round trip; use the round trip for the description and accept the title and URL fields are otherwise unverifiable except by re-reading the screen.
- Pushing many distinct catalogue images with distinct destination links at a mechanical per-phone cadence is the shape of activity spam classifiers key on for velocity and content-similarity; nothing in this chain should be tuned to vary timing or cadence to evade that.
- Claims about what volume of distinct pins a platform "rewards" are policy framing, not a physics constraint the rig can rely on; the real limiting factor is Pinterest's own anti-spam thresholds, not anything measured here.

TRAP **Album clear has no undo and no per-caller gate**

`/shortcut/album/clear` deletes an entire named album in one call, with no confirmation, and once the phone-side deletion permission is granted it stays armed for any future call from any LAN host, since the API has no authentication. Require an explicit human decision before wiring this to anything automatic, and treat it as a standing rack-level risk, not just a caveat for this one flow.

SCALE

One album-upload call with `zip=1` delivers the whole catalogue to every phone; from there each pin is roughly 16 taps plus two ASCII typing bursts and one clipboard round trip, realistically 45-60 s per pin per phone with taps dominated by pointer travel and the clipboard call bounded by its 15 s default outtime. Ten concurrent frame pollers measured 512 req/s total with no input/polling cross-talk, so throughput scales with handset count, not the kernel. Saves are about 3 taps and 5 s but strictly per-phone. The chain-only ceiling is 60-80 pins an hour per phone if the wheel-scroll step works, add 5-15 s per pin if it must fall back to an explicit drag; a defensible planning number once Pinterest's own spam-velocity limits are respected is 15-30 pins an hour per phone, not 60-80.

Snapchat: Memories Story post and per-phone streak keeper

PARTIAL WITH CAVEATS

Snapchat exposes no share-sheet upload and no deep-linked composer, so the whole flow runs through the camera screen: push the day's image into the phone album, swipe up to Memories, pick the Camera Roll tile, Send To, then either My Story or a multi-select of friend rows. Streaks and Stories run on a 24-hour clock, so keeping them alive on accounts you own is otherwise a daily manual pass over every handset; every call in the chain exists on this kernel, but the blocking dependencies are the iMouse Shortcut and per-phone captured coordinates, since there is no label-based fallback without OCR.

HOW

1. `get_device_list` + `get_usb_list` XP `/device/get and /config/usb/get`, reads imgw/imgh (500x1080), requires state != 0 with a live `air_pid` for video and board state 2 for control; video and control are independent channels, so a phone that mirrors perfectly can still ignore every tap.
2. `shortcut#7` XP `/shortcut/album/upload`, pushes the day's image from the PC in one call for all N phones; the response's newest album records are non-visual confirmation the file landed.
3. `send_key` + `mouse_reset_pos` XP `WIN+h (x2) then /mouse/reset`, normalises every phone and clears the stuck-click trap; re-verify board state 2 per target each run, since a board can silently drop to state 1 between runs with no visible symptom until a tap fails.
4. `click` / `shortcut#13` XP `/mouse/click or /shortcut/exec/url?url=snapchat://`, launches Snapchat; whether the scheme resolves to the app is Snapchat and iOS behavior, not anything the API documents.
5. `swipe` or `mouse_down` / `move` / `up` XP `/mouse/swipe?direction=up&len=0-1&brake=1`, opens Memories from the camera screen; XP's `len` is a percentage of the axis, not pixels, so a roughly 300 px stroke on a 1080 px axis is `len` around 0.28. Test both the swipe endpoint and a longer explicit drag before committing, with an icon-tap fallback if the drawer never opens.
6. `click` XP `/mouse/click`, taps the Camera Roll tab, the newest tile, then Send To; positions are captured coordinates with no layout-independent alternative on this rig.
7. `click` XP `/mouse/click`, taps My Story then the send arrow for a Story post; this can fan out to phones with matching imgw/imgh and matching app state, which a scrollable Send To list will not have.

8. `click` + `send_key` / `shortcut#10` XP `/mouse/click plus /key/sendkey?key=... or clipboard/set + WIN+v`, selects friends on the Send To screen for the streak keeper; ASCII names go through `/key/sendkey`, non-ASCII names need the clipboard route, since there is no `send_text` on XP.
9. `get_device_screenshot` XP `/pic/screenshot?id=<single id>&binary=true[&rect=[l,t,r,b]]`, verifies the send landed by mean absolute pixel difference; single-id only, one poller per phone.
10. `mouse_reset_pos` / `key_release_all` / `restart_usb` XP `/mouse/reset, /key/upall, /device/usb/restart?id=VID|PID`, recovers a step that did not land; the WebSocket `error_push` callback is the only asynchronous signal that an input call failed at the board.

NEEDS

- IMConsole.exe running and logged in; with only IMService.exe up, both phones read state 0 and the board list is empty.
- iMouse Shortcut installed, IP-edited and bound to an AssistiveTouch key on every phone, for album upload and clipboard set.
- Shortcuts > Advanced: Allow Running Scripts, Allow Sharing Large Amounts of Data.
- AssistiveTouch, Full Keyboard Access, max tracking speed, Auto-Lock Never, portrait lock; none queryable over the API.
- Board bound to the phone, `location_crc` non-empty.
- Snapchat installed and logged in per phone on an account the operator owns; coordinates captured for Memories, the newest tile, Send To, My Story and the send arrow; friend-row coordinates or ASCII-only friend names for search.

WATCH OUT

- Every function name written in Phonefarm style fails on this kernel: a bare `{"fun":"click"}` to 9911 returns HTTP 500, and 9912 is closed. Use the XP paths shown here.
- No OCR and no image search on this rig; every tap is a captured coordinate that breaks when Snapchat moves its UI, and the streak step cannot resolve a friend by name from the screen.
- Coordinates are mirror pixels (500x1080), not iOS logical points (414x896); screenshot-derived coordinates are already in the right space.
- The Shortcut round trip is itself unverified on this rig; prove it first with `/api/shortcut/device/ip` before relying on it, a code 30 timeout means the AssistiveTouch hotkey never fired.
- The Camera Roll "newest tile" coordinate is only stable if every phone's roll has exactly one new item since the last run; a phone that missed a prior push shifts its newest tile, so every step after image pick, not only friend selection, must run per-phone.
- Camera viewfinder screens produce a hot whole-frame diff by nature, since it is a live preview; a rect crop for verification needs to be confirmed empirically per app version, since chrome elements like a send checkmark can themselves animate and register as noise.
- A daily streak-keeping loop is a recurring, scheduled, low-variance automated action against a social app, exactly the shape of activity abuse detection is built to catch over time, independent of whether any single run's chain is technically correct; nothing here should be tuned to vary timing or behavior to evade that.

TRAP **Opening Memories may need a faster hand than the board has**

Snapchat's camera-screen navigation drawer is commonly a gesture-recognizer paging control, gated on distance and velocity, unlike a position-tracked picker wheel. A roughly 300 px stroke at the board's fixed 0.6 ms/px walk is only about 180 ms of travel, and the open-or-not decision may need a faster hand than the board's HID walking speed can deliver, since that speed is fixed by firmware and not adjustable per gesture. Test the swipe endpoint and a longer explicit drag before committing to either, and keep an icon-tap fallback route to Memories.

SCALE

The album push is one request for all N phones; everything after it is per-phone. A Story is roughly 8-10 taps and the streak keeper adds about 3 taps per friend, dominated by pointer travel at roughly 0.6 ms per pixel: budget 20-40 s per phone for the Story, more for a multi-friend send. Because friend-row order differs per account and cannot be comma-id mirrored, the streak keeper is inherently one script per account per day, not a batchable macro; the correct throughput unit is friends kept per run per phone, likely tens per phone per day, rather than a sustained hourly rate. Treating it as a high-frequency loop would itself be the kind of activity pattern that risks detection, so the physics-correct answer here is also the risk-correct one.

Commerce and seller-app forms

Shopify and TikTok Shop seller apps: album-fed listing forms and tap-through fulfilment

NEEDS SHORTCUT WITH CAVEATS

Seller apps are plain forms driven blind by coordinates: product photos land in the phone album via one `/shortcut/album/upload` call, title and price go in as ASCII with `/key/sendkey`, the Unicode description arrives by clipboard plus a paste, and Save is a tap. Fulfilment is a fixed tap path from Orders through to a typed tracking number. A rack can keep several owned seller accounts' catalogues current from one content folder on the PC, for stores where only the phone app is available to the operator. Nothing in the chain needs the OCR or image-search plug-ins, which is exactly why it survives on this rig, and exactly why nothing in it can read a field back to confirm what landed there.

HOW

1. `send_key` XP `/key/sendkey?id=<ids>&fn_key=WIN+h` (twice) — normalize every target phone to home page one; hardware keys still fire even when taps are silently dead, so this alone proves nothing about the tap path.
2. `mouse_reset_pos` XP `/mouse/reset?id=<ids>` — clear the silent stuck-click state (measured: 0.09 mean pixel diff on a click vs. 141.96 after reset) before any tap-critical run.
3. `shortcut#7` XP `/shortcut/album/upload` (id list, files array of PC paths, album_name, zip, outtime ms) — push SKU photos into the phone album in one call across the whole id list; the response's newest album records confirm the file arrived before the picker taps begin.
4. `click` XP `/mouse/click?id=<ids>&x=<px>&y=<py>&button=1` — walk Products > + > Add images > Photo Library > newest tiles > Add, then Save/Submit and the Active toggle; coordinates are mirror pixels (500x1080 here), never iOS points.
5. `send_key key=` XP `/key/sendkey?id=<id>&key=<ascii text>` — type title, price and later the tracking number; ASCII only, no Unicode.

6. `shortcut#10` + `send_key fn_key` XP `/shortcut/clipboard/set` then XP `/key/sendkey?fn_key=WIN+v` — paste the Unicode/emoji description, since `send_key` cannot carry it.
7. `mouse_wheel` XP `/mouse/wheel?id=<ids>&direction=down&len=30&number=1` — scroll the category picker or a long form; expect this to fail, since an iOS picker is normally a drag-driven control, and keep a swipe or a real down/move/up drag as the actual default.
8. `get_device_screenshot` XP `/pic/screenshot?id=<one id>&binary=true` — crop-diff each field's box right after typing it, before moving to the next field, not just the whole form after Save.

NEEDS

- Seller apps installed and signed in on each phone with product-edit rights, on accounts the operator owns; this flow logs into nothing and creates no accounts.
- iMouse Shortcut installed per phone, kernel IP edited in, run once with every permission allowed, bound to an AssistiveTouch custom key; album/upload and clipboard/set both travel that path and time out with code 30 if the binding never fires.
- Shortcuts > Advanced: Allow Running Scripts and Allow Sharing Large Amounts of Data.
- Board bound and physically attached (state 2), phone mirroring live with an air_pid.
- Full Keyboard Access, AssistiveTouch, max tracking speed, Auto-Lock Never, portrait lock — none of it visible over the API.
- A per-app-version coordinate set captured per phone, fields tapped top-down so the keyboard never covers the next target.
- IMConsole.exe running and logged in; with only IMService.exe up, phones showed state 0 and an empty board list.

WATCH OUT

- The Shortcut dependency was never probed live on this rig; a guessed `/shortcut/get` 404 is not evidence either way. Prove it first with `/shortcut/device/ip?id=<id>&state=0` and expect `data.text` to equal the device's own IP.
- Tapping is blind: there is no way to assert "the price field now reads 49.90," only that the frame changed, and a value typed into the wrong field because the keyboard covered it will pass a naive whole-frame diff.
- If the Shortcut binding is absent, album/upload and clipboard/set fail with code 30, killing photos and any Unicode description, while the ASCII-only steps (title, price, tracking number, Save) still fire — a partial failure that produces a live listing with no photos and no description unless the upload response is checked first.
- Mirroring one tap to several phones only holds when every listed phone shares app version, layout and imgw/imgh; typed content is always one call per phone.
- No character-set limit is documented for the clipboard route; confirm with a set/get round trip per phone before trusting emoji or non-Latin descriptions.
- Typing speed and inter-key delay for `send_key` are undocumented in either edition; chunk long strings and prefer the clipboard set/get round trip over trusting `send_key` for anything past short values.
- The Photo Library multi-select step assumes a fixed tile position after upload; if the album defaults to single-select or re-sorts by capture time instead of import time, a blind coordinate can attach the wrong image to the wrong listing — a materially worse failure than a blank field.
- Running many SKUs unattended with no field-level verification risks a batch of live, mispriced or mis-titled listings before a human notices; spot-check a sample rather than trusting a whole-form diff.

SCALE

One listing is roughly 25 taps plus three or four typed strings, 60-90 seconds per phone, dominated by app latency and pointer-walk time (~0.6 ms/px) rather than the API. Fulfilment is about six taps and a tracking number, ~15 seconds. The album upload fans out to every phone in one request regardless of count; verification does not fan out, one screenshot per phone at 2 ms per raw BMP frame, gated by each phone's AirPlay source rate (measured ~29 new frames/s on one XR, ~4.5 on its twin — budget 220 ms or more per fresh frame on the slow one). That works out to roughly 40-60 listings/hour/phone if every blind tap lands correctly; discount for whatever spot-check rate is needed to catch a mis-tap, since nothing here verifies field content.

Messaging and community broadcast

Telegram: near-coordinate-free channel broadcast by deep link, clipboard paste and ENTER

NEEDS SHORTCUT WITH CAVEATS

Telegram is the platform where this rack comes closest to layout-independent posting: a `tg://resolve` deep link opens the exact chat, the clipboard carries the message (emoji and CJK included), and paste plus ENTER sends it. All four calls in the chain accept a comma-separated id list, so a broadcast to N phones is four requests. One caveat survives scrutiny that an early draft missed: nothing guarantees the composer is focused after a deep link, so one `/mouse/click` on the input bar stays in the chain — exactly one coordinate per phone rather than a dozen, not zero.

HOW

1. `send_key` XP `/key/sendkey?id=<ids>&fn_key=WIN+h` (twice) — normalize to the home screen; requires Full Keyboard Access ON.
2. `mouse_reset_pos` XP `/mouse/reset?id=<ids>` — clear the stuck-click state; cheap insurance since the one remaining tap is the flow's single point of failure.
3. `shortcut#13` XP `/shortcut/exec/url?id=<ids>&url=tg://resolve?domain=<channel>` — open the exact channel on every phone at once; both vendor doc examples are bare web domains, so a custom scheme resolving is an inference to probe once per app build, not a guarantee.
4. `click` XP `/mouse/click?id=<ids>&x=<x>&y=<y>&button=1` — focus the message input; the input bar sits at a fixed y near the bottom of the mirror frame, so this one coordinate mirrors safely across same-geometry phones.
5. `shortcut#10` XP `/shortcut/clipboard/set` (id list, text, sleep seconds, outtime ms) — load the message onto every clipboard; the documented XP route for non-ASCII text, since XP has no `send_text` and no completion callback.
6. `send_key fn_key=WIN+v` then `fn_key=ENTER` XP `/key/sendkey` — paste and send; whether Telegram iOS treats ENTER as send or as a newline is app behavior outside any document here, verify once and keep a send-button tap as fallback.
7. `shortcut#7` XP `/shortcut/album/upload` then `click` — push a media file to the album, then paperclip > newest tile > send; the upload response's newest album records are the only step in this flow the kernel can actually confirm server-side.
8. `click time=700` XP `/mouse/click?id=<id>&x=<x>&y=<y>&time=700` — forward an existing message via long press, Forward, select, send; the message's y depends on scroll state, which differs per phone, so this branch is per-phone coordinate work.

9. `get_device_screenshot` XP `/pic/screenshot?id=<one id>&binary=true&rect=[0,820,500,1080]` — crop the bottom bubble region and threshold a mean absolute pixel difference (idle floor 0.02-0.03, real change 20-160); never MD5.

NEEDS

- IMConsole.exe running and logged in, reachable at 127.0.0.1:9911.
- Board bound at state 2 per phone, non-empty location_crc, Full Keyboard Access ON.
- iMouse Shortcut installed, IP-edited and bound to an AssistiveTouch custom key on each phone — never exercised on this rig.
- Telegram installed, logged in, with posting rights on the target channel.
- A one-time probe per app build that `tg://` resolves through the Shortcut's Open URLs action, and that hardware ENTER sends rather than inserting a newline.
- The input-bar coordinate captured once in mirror-pixel space; phones with differing imgw/imgh excluded from the mirrored id list.

WATCH OUT

- No `/shortcut/*` endpoint has ever been called on this rig; the whole flow rests on a manual Shortcut install and AssistiveTouch binding. Probe with `/shortcut/device/ip?state=0` first — code 0 with the right LAN IP proves install, binding and return path in one call.
- Custom URL schemes are outside what the vendor documents for exec/url; if the scheme isn't handled, the Shortcut may hand it to Safari instead, which shows nothing useful and leaves the subsequent paste and ENTER landing somewhere unintended, possibly the Shortcuts app or the home screen. Capture a screenshot after the exec/url call itself, not just the shortcut's return code — a success code only proves the Shortcut ran, not that the URL resolved to Telegram.
- No OCR or image-search plug-in (code 28), so nothing in the chat can be located on screen; screenshot diffing on raw BMP frames is the only verification channel, one request per phone.
- The clipboard is global phone state: a mirrored set overwrites whatever was there, and distinct per-phone texts need one call per phone, which removes the constant-cost property.
- Whether ENTER sends or inserts a newline is Telegram-iOS-version behavior that can change with any app update, with no signal from any endpoint here; re-probe after every Telegram update, not once per "build."
- The claimed "5-10 s end to end largely independent of N" figure only holds if the composer-focus click is issued as one mirrored call across the whole id list, not one call per phone — say so explicitly when building the flow.

SCALE

Identical text to N phones is four requests plus one mirrored click, roughly 5-10 seconds end to end regardless of N, since the board drives every listed id in parallel. Distinct texts or distinct chats collapse this to one clipboard call and one exec/url per phone, ~2-4 seconds each — linear in N. That yields on the order of 350-700 identical broadcasts/hour with only per-broadcast crop-and-diff verification, or roughly 900-1800 distinct sends/hour aggregate across many phones run concurrently. Verification is the serial tail: 2 ms per raw BMP fetch, gated by AirPlay source rate to roughly 35 ms per new frame on the fast phone and 220 ms on the slow one.

Discord: channel deep-link posting, long-press reactions and moderation from a member account

NEEDS SHORTCUT WITH CAVEATS

The same four-call pattern as Telegram applies: a deep link opens the channel, clipboard plus paste fills the composer, ENTER sends. Reactions and moderation hang off a long press, which needs no dedicated API since `/mouse/click` carries a `time` parameter — a click with `time=700` is a press-and-hold, and the sheet it raises holds the quick-reaction row alongside Delete Message and Pin Message. The technical chain is sound; the honest framing is that everything it does to a channel, a bot can do through Discord's own API with no phones involved.

HOW

1. `send_key` XP `/key/sendkey?id=<ids>&fn_key=WIN+h` (twice) — normalize to home; hardware keys work even when taps are silently dead, so this alone is not proof of a live tap path.
2. `mouse_reset_pos` XP `/mouse/reset?id=<ids>` — mandatory here rather than optional: this flow is tap-heavy (composer, long-press, sheet item, confirm), and a stuck phone follows every move and hotkey while silently swallowing every tap.
3. `shortcut#13` XP `/shortcut/exec/url?id=<ids>&url=https://discord.com/channels/<guild>/<channel>` — open the target channel; the https universal link is the safer primary form over `discord://`, since a universal link degrades to the web app rather than to nothing if the app is missing, but this is still an inference outside the doc's bare-http examples.
4. `click` XP `/mouse/click?id=<ids>&x=<x>&y=<y>&button=1` — focus the message bar; button is numeric on XP, not the string form. Its y is stable near the bottom of the frame across same-geometry phones.
5. `shortcut#10` then `send_key fn_key=WIN+v` XP `/shortcut/clipboard/set` — load and paste the message; emoji is exactly why the clipboard route is required over ASCII-only `send_key`.
6. `send_key fn_key=ENTER` XP `/key/sendkey?id=<ids>&fn_key=ENTER` — send; whether Discord iOS sends on hardware ENTER is undocumented app behavior, verify once and keep a send-button tap as fallback, treated as the primary path rather than a rarely-needed backup.
7. `click time=700` then `click` XP `/mouse/click?id=<id>&x=<x>&y=<y>&time=700` — react to a message; re-send the deep link first so every phone is scrolled to the bottom and the newest message sits at a predictable y, which mitigates but does not guarantee the target.
8. `click time=700` then two more clicks — moderation: Delete Message / Pin Message from the same long-press sheet; irreversible, gate behind explicit human confirmation or route through Discord's own API where the target is addressed by id.
9. `shortcut#7` then `click` x3 XP `/shortcut/album/upload` — attach media; the response's newest album records confirm the file landed before the blind picker taps.
10. `get_device_screenshot` XP `/pic/screenshot?id=<one id>&binary=true&rect=[0,700,500,1000]` — confirm the post appeared, mean absolute pixel diff on the crop, never MD5.

NEEDS

- IMConsole.exe running and logged in; boards bound at state 2; non-empty location_crc per phone.
- Full Keyboard Access, AssistiveTouch, max tracking speed, Auto-Lock Never, portrait lock — none visible to the API.
- iMouse Shortcut installed, IP-edited and bound to an AssistiveTouch custom key per phone — never exercised on this rig.
- Discord installed and logged in per phone, a member of the target guild, with Manage Messages permission for the moderation branch.

- One-time probes: channel link opens the app rather than Safari, hardware ENTER sends, and a 700-800 ms click registers as a long press on this Discord build.
- Per-phone message-bar and sheet coordinates captured in mirror pixels.

WATCH OUT

- The entire `/shortcut/*` dependency (clipboard for the caption, exec/url for the deep link) is completely unverified on this rig; a 404 on a guessed `/shortcut/get` path is not evidence either way. Run `/shortcut/device/ip?id=<id>&state=0` per phone first — if it fails, the flow degrades to ASCII-only typed text and manual app navigation with no deep link.
- A 700 ms click registering as a long press is a vendor-documented parameter, not a fact confirmed against this Discord build; probe once per app/iOS version, since press-duration thresholds vary by app.
- The https link opening the app rather than Safari's web view is an assumption, not documented kernel behavior; if it degrades to Safari, every downstream coordinate targets a browser, not the app.
- Reaction and moderation coordinates depend on scroll position and message height; with no OCR or image-search plug-in, nothing confirms what is under the pointer before the tap. A reply-preview banner, mention autocomplete strip, or sticker tray above the composer shifts the message-bar y in a way the API cannot detect — re-verify it is truly static for the accounts and servers actually used.
- Re-issue `/mouse/reset` before each tap-critical phase (composer focus, long-press, sheet tap, confirm), not only once at the top — the stuck-click state was observed to appear mid-session, and a hardware key press still works while stuck, giving no signal that a later tap silently failed.
- Because 9911 has no authentication, any other LAN host can fire the same moderation, reaction or post calls against these logged-in accounts concurrently with your automation, racing on the same coordinates this flow assumes are stable.
- Delete and Pin reach a variable-height sheet by blind tap and are irreversible; keep human confirmation on every destructive tap, or route moderation through Discord's own API where the target is addressed by message id.

SCALE

A text post to N phones with identical content is four comma-id requests plus one mirrored composer click, roughly 5-8 seconds regardless of N. Distinct channels or text make it linear, one exec/url and one clipboard/set per phone at 2-4 seconds each. Reactions and moderation are inherently per-phone, 2-3 taps plus the 700 ms hold and pointer-walk time (~0.6 ms/px), budget 3-5 seconds per phone per action. That is a theoretical hardware ceiling of roughly 200-400 posts/hour/phone or 700-1000 reaction/moderation actions/hour/phone; it is irrelevant in practice, since Discord's own anti-automation heuristics will trigger account action long before hardware limits bind. Treat this as a manual, occasional capability rather than a throughput target.

Feature-gap workarounds on the composer surface

X: media post and poll in the app, bookmarks, and scheduling through the mobile web composer

NEEDS SHORTCUT WITH CAVEATS

The X iOS app posts text, media and polls but exposes no schedule control; x.com in mobile Safari does. The rack opens the web composer, pastes text through the Shortcut clipboard, attaches media pushed earlier by `/shortcut/album/upload`, and drives the web schedule picker — giving scheduled posts from real handsets with no

new API capability, only a change of surface. Scheduling is the capability operators most often assume is unavailable on mobile, and the fix costs nothing beyond pointing the same primitives at Safari instead of the app.

HOW

1. `get_device_list` + `get_usb_list` XP `/device/get and /config/usb/get` — take imgw/imgh per phone as the tap space (500x1080 here); require state != 0 with a live air_pid and board state 2, since a phone can mirror perfectly and ignore every tap.
2. `shortcut#7` XP `/shortcut/album/upload` — push the promo image into the album for every phone in one request; the newest album records confirm arrival before the composer opens Photos.
3. `send_key fn_key=WIN+h` (x2) then `mouse_reset_pos` XP `/key/sendkey then /mouse/reset?id=<ids>` — normalize and clear the stuck-click state.
4. `shortcut#13` XP `/shortcut/exec/url?id=<ids>&url=https://x.com/compose/post` — open the web composer, the surface with the schedule control. A universal link to x.com may instead be claimed by the installed app, which has no schedule control; that is iOS link routing, not an API fault, and must be resolved on the handset. Confirm with a screenshot (App Store back button vs. Safari chrome) before trusting the schedule steps ran in a browser.
5. `shortcut#10` + `send_key fn_key=WIN+v` XP `/shortcut/clipboard/set` — put the post text in place; `send_key key=` is ASCII-only, so emoji and non-Latin text must take the clipboard route, and there is no completion callback to confirm delivery.
6. `click` + `send_key` XP `/mouse/click + /key/sendkey?key=Yes|No` — attach media and build the poll: media icon > Photo Library > newest tile > Choose; poll icon > option fields; all positional, since OCR is unavailable.
7. `click` + `mouse_wheel` XP `/mouse/click + /mouse/wheel?id=<id>&direction=up|down&len=1-127&number=n` — set the schedule; treat a wheel attempt as opportunistic and a real down/move/up drag as the primary path, since a picker wheel is normally drag-driven, not scroll-driven, and picker drags travel well under 1080 px so the pointer-walk cost is trivial regardless.
8. `click` XP `/mouse/click` — bookmark a post: open it, tap share, tap Bookmark. Per-phone even under a mirrored id, since two timelines don't show the same post at the same coordinate.
9. `get_device_screenshot` XP `/pic/screenshot?id=<single id>&binary=true[&rect=[l,t,r,b]]` — verify each post or schedule landed with a cropped mean-absolute-pixel-diff against an idle control; no step can be confirmed by the kernel as "posted" vs. "scheduled" vs. "drafted," only as a pixel change in the region that actually differs between those outcomes.
10. `mouse_reset_pos / key_release_all` XP `/mouse/reset, /key/upall` — recovery; watch the WS `error_push` callback for hardware-level failures.

NEEDS

- XP kernel on 9911 with IMConsole.exe running and logged in.
- Logged in to x.com in Safari on each phone (persistent session) for the web route; the app logged in for immediate posts, on accounts the operator owns.
- iMouse Shortcut installed, IP-edited, run once with all permissions allowed, bound to an AssistiveTouch custom key.
- Full Keyboard Access ON, AssistiveTouch at max sensitivity, max tracking speed, Auto-Lock Never, portrait lock.
- Board bound with a non-empty location_crc.
- Web composer coordinates captured in mirror-pixel space with Safari zoom left at default.

WATCH OUT

- Prove the Shortcut round trip first with `/shortcut/device/ip?id=<id>&state=0`; code 0 with the device's own IP means installed and reachable, code 30 after roughly 15 s means the hotkey never launched it.
- If the X app claims the universal link — normal iOS behavior when the app is installed — `/shortcut/exec/url` never reaches Safari, and the whole schedule-from-mobile-web premise collapses silently with a 200 status and no error signal from the kernel.
- No OCR or image search on this rig, so every composer control is a captured coordinate; a Safari zoom change or layout update invalidates the capture silently. Coordinates are mirror pixels, not logical points.
- Whether the web date/time pickers honor HID wheel events is unverified; budget the drag as the primary mechanism, not a rarely-needed fallback — the earlier full-height-swipe timing cost from the hardware measurements does not apply to a picker drag, which travels far less distance.
- XP gives no completion callback for text delivery, so confirmation is a clipboard read-back plus a frame diff, not an event.
- AssistiveTouch and Full Keyboard Access being ON is invisible to the API; a phone with either silently OFF accepts `/mouse/move` but ignores taps or paste, indistinguishable from the stuck-click state without a post-reset verification tap.
- The API has no authentication on the LAN; firewall 9911 and 17000.

SCALE

An immediate in-app post is roughly 8 taps (~20 s/phone); a scheduled web post is roughly 14 taps plus the picker steps (~50 s), with pointer travel dominant at ~0.6 ms/px (a 400 px move is 250 ms, a full-height traverse ~592 ms). The media push fans out to every phone in one request; composer work is per-phone but phones run fully independently (two concurrent frame pollers measured 83 and 85 req/s at p50 3 ms, ten pollers 512 req/s total). Verification is capped by AirPlay source rate — roughly 29 new frames/s on one XR versus 4.5 on its twin, so budget 220 ms or more per fresh frame on the slow handset. Mechanically this allows on the order of 50-70 scheduled posts/hour/phone run back to back, but a rate-limit or challenge interstitial on the real account is the actual ceiling, set by the platform, not the rig — treat the mechanical number as a limit, not a target.

LinkedIn: PDF carousel pushed into Files, composed and scheduled by blind coordinates

NEEDS SHORTCUT WITH CAVEATS

A PDF document carousel is the one LinkedIn format that normally cannot be produced on a phone, because the file has to reach the device first. `/shortcut/file/upload` puts the PDF into a fixed Files-app folder on every bound phone in one comma-id call; everything after that is `/mouse/click` at mirror-pixel coordinates through Add a document, Choose from Files, the upload folder and the composer, ending in Post or LinkedIn's own clock-icon scheduler. Getting a multi-page PDF onto twenty phones is the actual bottleneck for carousel posting, and that file transfer is the one part genuinely enabled by this rack rather than by a person holding a phone.

HOW

1. `shortcut id=8` XP `/shortcut/file/upload?id=<ids>&path=/iMouse/&files=[...]&zip=0&outtime=120000` — push the PDF into a fixed Files folder on every target phone at once; iOS 15+ only, verified in the shortcuts documentation.
2. `send_key fn_key=WIN+h` (x2) XP `/key/sendkey?id=<ids>&fn_key=WIN+h` — normalize every phone to home page one; requires Full Keyboard Access ON.

3. `mouse_reset_pos` XP `/mouse/reset?id=<ids>` — clear the stuck-click state before any tap-critical sequence.
4. `click` XP `/mouse/click?id=<id>&x=<x>&y=<y>&button=1` — open LinkedIn, tap Post, tap the text area; the two XRs have different home screens and wallpapers, so this app-icon tap is per-phone even though later in-app coordinates can be mirrored.
5. `shortcut id=10` then `send_key fn_key=WIN+v` XP `/shortcut/clipboard/set` — put the post copy, emoji included, on the clipboard and paste it; the only documented Unicode path, since `send_key key=` is English-only.
6. `click` (x5-7) XP `/mouse/click?id=<id>&x=<x>&y=<y>` — attach the document through Add a document > Choose from Files > Browse > On My iPhone > the upload folder > the PDF; purely coordinate-driven since the kernel cannot locate the file row for you. Keep the upload folder holding exactly one file so the row's y is deterministic, and re-verify the picker screenshot rather than trusting a hardcoded y after any prior run.
7. `send_key key=` XP `/key/sendkey?id=<ids>&key=<ascii title>` then click Done — type the ASCII document title; use the clipboard route instead for a CJK or emoji title.
8. `mouse_wheel`, or down/move/up drag XP `/mouse/wheel?id=<ids>&direction=up&len=30&number=1` — schedule: tap the clock icon, set date and time, Next; treat the wheel as a cheap first attempt and the drag as the primary mechanism, since nothing in the measured facts supports a picker wheel responding to HID scroll in either direction.
9. `click` XP `/mouse/click?id=<id>&x=<x>&y=<y>` — publish or leave it scheduled; irreversible, gate behind explicit human confirmation since no step above can be visually confirmed by the kernel.
10. `get_device_screenshot` XP `/pic/screenshot?id=<single id>&binary=true[&rect=[l,t,r,b]]` — verify each stage by mean-absolute-pixel-diff frame comparison; one request per phone, ten concurrent pollers measured 512 req/s.
11. `shortcut id=6` XP `/shortcut/file/del?id=<ids>&path=/iMouse/&list=[{name,ext}]` — delete the uploaded PDF afterward so the picker row stays deterministic for the next run; needs the Shortcuts "allow deleting without confirmation" permission or the call hangs on an unanswered prompt.
12. `shortcut id=13` then `click` + clipboard + `WIN+v` + click Send XP `/shortcut/exec/url?url=https://www.linkedin.com/in/<slug>` — connection request with a note; one exec/url mirrored to N phones opens the same profile on all of them, so distinct targets need one call per phone.

NEEDS

- XP kernel on 127.0.0.1:9911 (path-style API); Phonefarm function names return HTTP 500 on this rig, 9912 closed.
- IMConsole.exe running and logged in — with only IMService.exe up, both phones showed state 0 and an empty board list.
- Per phone: board bound at state 2, location_crc non-empty, or every mouse call returns an error code.
- Per-phone iOS settings invisible to the API: AssistiveTouch at max sensitivity, Full Keyboard Access ON, max tracking speed, Auto-Lock Never, portrait lock.
- iMouse Shortcut v1.3+ installed, kernel IP edited in, run once with every permission allowed, bound via Send Binding Key; Shortcuts > Advanced allowing scripts, large-data sharing and deletion without confirmation — not verified present on this rig.
- iOS 15+ for all `/shortcut/file/*` calls (the XRs qualify at 18.7.10).
- The `/iMouse/` folder created once in Files on each phone.
- LinkedIn logged in on each phone; composer and Files-picker coordinates captured per phone in mirror pixels.

WATCH OUT

- No `/shortcut/*` call has actually been exercised on this rig; the 10 Sep probe only touched device, USB, group, OCR and mouse/key paths. Prove the Shortcut round trip first with `/shortcut/device/ip?id=<id>&state=0`.
- No OCR and no image-search plug-in (code 28); nothing in the composer or the Files picker can be located by the kernel, only asserted and checked with a client-side pixel diff.
- The two XRs have entirely different home screens, so only in-app coordinates mirror, and only between phones whose imgw/imgh match.
- Verification is per-phone and one at a time, and one XR delivers only ~4.5 new frames/s — budget 220 ms or more per expected frame there rather than assuming 30 fps.
- The date/time picker is the one unverified mechanism in both directions, not merely unverified for this app version; plan the drag fallback as the primary path.
- A substituted USB cable leaves the board at state 1 with a phone plugged in and the whole chain silently dead.
- The 120000 ms upload outtime is fine for a single file, but a 20 MB carousel PDF at the rig's worst-case ~5 Mbps Wi-Fi mode takes roughly 32 seconds for the transfer alone; margin shrinks if the id list carries many phones sharing the same access point, since the upload call has no documented per-phone parallelism guarantee the way mouse and key calls do.
- A previous run leaving the Files folder in a different view state (list vs. grid, a stale cached row from before a delete) shifts the picker row's y even with the folder empty; re-verify the screenshot, don't just trust an empty folder.
- If this flow and the Telegram flow share phones in the same session, the phone's clipboard is single-slot shared state — serialize clipboard-dependent steps across use cases, not just within one.
- Stay inside LinkedIn's own published invite and posting limits and halt on the first restriction screen; keep a human in the loop on the final Post or Send tap, since a blind tap on a shifted layout can publish or send the wrong thing irreversibly.

SCALE

The file upload is O(1) in phone count, one call with a comma-id list, outtime raised to ~120 s for a multi-MB PDF. Everything else is roughly 18-22 taps plus one clipboard round trip, dominated by pointer-walk at ~0.6 ms/px (a 1000 px traverse measured 592 ms); a move mirrored to N phones costs the same as one, so a mirrored carousel post is ~45-70 seconds regardless of N when every phone shares layout. Screenshot verification is the only per-phone serial cost, 2 ms per raw BMP frame at 84 req/s per phone, gated by AirPlay source rate (29 fps vs. 4.5 fps between the two XRs). Connection requests run ~10-15 seconds each and cannot be mirrored when the target profile differs per phone. That supports on the order of 40-60 mirrored carousel-post cycles/hour across an arbitrary phone count, bounded by the composer walk rather than by phone count — but LinkedIn's own connection-request and posting caps bind well before the rig does for the connection-request branch specifically.

Cross-platform posting patterns

Share-sheet-first cross-platform posting from Photos

NEEDS SHORTCUT WITH CAVEATS

Push one clip to every phone with a single upload call, then start every post from the same place: Photos, newest item, Share, pick the target app in the share sheet. The first six taps are identical for every platform and every phone, so only the tail (caption, Post) is app-specific, which roughly halves the number of coordinate sets to maintain. A single entry path is what makes a many-platform playbook maintainable on a rack of phones; on iOS the share sheet is that path, removing the need to learn each app's own composer navigation.

HOW

1. `shortcut#7` XP `/shortcut/album/upload?id=<all ids>&files=[...]&zip=0&outtime=180000` — push the clip to every phone in one call; zip=0 is right for a single large file, and the response's newest album records confirm it actually arrived.
2. `send_key fn_key=WIN+h` (x2) then `mouse_reset_pos` XP `/key/sendkey then /mouse/reset?id=<all>` — normalize every phone to home page one and clear the stuck-click state.
3. `click` XP `/mouse/click?id=<same-layout group>&x=<px>&y=<py>&button=1` — Photos icon, Recents, newest tile, Share button; the newest-tile corner depends on the phone's own Photos sort order and must be captured per phone, standardized once per phone before it can be trusted at all.
4. `mouse_wheel` XP `/mouse/wheel?id=<ids>&direction=right&len=60&number=1` — scroll the share sheet's app row; expect this to fail, since the row is a touch-driven UIScrollView with no general scroll-wheel support, and build the horizontal swipe or drag as the default rather than a contingency.
5. `click` XP `/mouse/click?id=<ids>&x=<px>&y=<py>` — tap the target app's icon; its composer opens with the video already attached. Making icon order identical across phones is a one-time manual per-phone edit to share-sheet Favourites, with no API equivalent, and it is not permanently stable — iOS can reshuffle these suggestions after an OS update, so re-verify periodically rather than treating the one-time edit as durable.
6. `shortcut#10` then `send_key fn_key=WIN+v` XP `/shortcut/clipboard/set` — put the caption on the clipboard and paste it into the composer; the only documented Unicode text route on this edition.
7. `click` XP `/mouse/click?id=<id>&x=<px>&y=<py>` — tap Post/Share; the irreversible step, and share-sheet composers often use different coordinates from the in-app composer, so keep the two sets separate.
8. `get_device_screenshot` XP `/pic/screenshot?id=<one id>&binary=true` — verify per phone that the post went out, mean-absolute-pixel-diff, never MD5.
9. `shortcut#3` XP `/shortcut/album/del?id=<ids>&list=[{album_name,name,ext}]&outtime=30000` — delete the clip from the album afterward so the next run's newest-tile coordinate stays valid; confirm the deleted filename matches exactly what the upload step returned before issuing this, the one destructive call in the chain, and never reach for the album-wide clear.

NEEDS

- All target apps installed and signed in on each phone, on accounts the operator owns.
- iMouse Shortcut installed per phone, kernel IP edited in, bound to an AssistiveTouch custom key; Shortcuts > Advanced must allow running scripts, sharing large data, and deleting without confirmation, since the last two are what album deletion needs.
- Share-sheet Favourites edited once per phone so app icons sit in the same order — a manual, one-time human step with no API equivalent.
- Board bound and attached, phone mirroring live with an air_pid.

- Full Keyboard Access, AssistiveTouch at max sensitivity, max tracking speed, Auto-Lock Never, portrait lock.
- Two coordinate sets per platform where the share-sheet extension opens a reduced composer that differs from the in-app one.
- IMConsole.exe running and logged in.

WATCH OUT

- The three Shortcut-backed steps (upload, clipboard, delete) were never probed on this rig; prove the round trip per phone with `/shortcut/device/ip?id=<id>&state=0` before assuming any of this runs.
- Newest-tile determinism depends on the phone's Photos sort-order setting, a per-phone toggle invisible to the API; standardize it once per phone as a precondition, not just something the deletion step manages after the fact.
- Deleting the clip afterward only keeps the next run's coordinate valid under strictly serial upload-post-delete per phone; two pieces of content in flight for the same phone (a retry, or two campaigns queued back to back) makes newest tile ambiguous again even with deletion in place — serialize per phone.
- A single wrong coordinate anywhere in this long chain compounds: a wrong share-sheet tap can open the wrong app's composer, receive the pasted caption, and land a subsequent tap on that wrong app's own Post control, publishing to an unintended surface with nothing here to detect it.
- Publishing is irreversible and unverifiable beyond a pixel diff on this rig, since there is no OCR to read a success or error toast; budget a human review pass.
- The comma id list mirrors taps but not state — phones with a different app version, share-sheet order or imgw/imgh must be excluded from the fan-out group.

SCALE

The upload is one request for the whole rack (a 30 MB clip takes tens of seconds, give outtime ~180000 ms). After that, each platform is roughly 10-14 taps, so six platforms from one clip is under six minutes per phone, and the taps mirror across identical phones at no extra wall time since the board walks all listed phones in parallel. Captions and verification do not fan out, since both the clipboard read-back and the screenshot call are single-id. Screenshot verification is cheap (2 ms raw BMP, 84 req/s per phone, ten concurrent pollers measured 512 req/s total) but is gated by AirPlay source rate, roughly 29 new frames/s on one XR and 4.5 on its twin — the slower phone, not the API, sets the real pace of a multi-platform run.

Composers prefilled through URL schemes: one exec/url call instead of clipboard plus paste

NEEDS SHORTCUT · WITH CAVEATS

Where a platform accepts the message or caption as a URL parameter, `/shortcut/exec/url` alone delivers full Unicode straight into the composer with no keyboard and no clipboard, and it fans out to the whole rack in one comma-id request. Where a platform ignores the parameter, the fallback is the ordinary clipboard route. The valuable artifact is the matrix itself — which apps honor a prefill on which version — and that matrix is an external, app-version-dependent fact the vendor documentation cannot confirm and that has to be measured on this rack, then re-measured after every app update.

HOW

1. Host-side only, no call — build the URL on the PC with UTF-8 percent-encoding; send it as POST JSON to `XP /api` rather than a GET query string, since XP accepts POST application/json and encoding is easy to mangle in a query string.

2. `send_key fn_key=WIN+h` (x2) then `mouse_reset_pos` XP `/key/sendkey then /mouse/reset?id=<ids>` — normalize the phones and clear the stuck-click state.
3. `shortcut#13` XP `/shortcut/exec/url?id=<ids>&url=<prefilled link>&outtime=15000` — open the prefilled URL on every target phone in one request; both vendor doc examples are bare web domains, so custom schemes or universal links honoring the parameter is unverified, the load-bearing assumption of the whole idea.
4. `get_device_screenshot` XP `/pic/screenshot?id=<one id>&binary=true&rect=[l,t,r,b]` — confirm the composer is on screen before touching Send; with no OCR this can only prove the screen is not the home screen, not that the correct app or composer opened, nor that the injected text arrived complete and correctly encoded. A login wall, paywall or rate-limit interstitial produces the same pixel delta as a successful open.
5. `click` / `send_key fn_key=ENTER` XP `/mouse/click or /key/sendkey?fn_key=ENTER` — send, one tap or ENTER where the app sends on Enter; the irreversible step.
6. `shortcut#10` then `send_key fn_key=WIN+v` XP `/shortcut/clipboard/set` — fallback for platforms that ignore the text parameter; the only documented Unicode route on this edition, verify per phone with a clipboard read-back.
7. Measurement, not a call — record and re-verify the apps-that-honor-prefill matrix per app version; it is an external platform behavior, not documented or probeable from the API, and it changes with every app update.

NEEDS

- iMouse Shortcut installed on each phone, kernel IP edited in, bound to an AssistiveTouch custom key — exec/url fires through that hotkey path and times out with code 30 if the binding doesn't fire.
- Target apps installed and signed in on accounts the operator owns, with one manual test per app of whether the text parameter is honored on the current version, repeated after each app update.
- Full Keyboard Access ON for ENTER and for the clipboard fallback's paste.
- Board bound and attached, phone mirroring live, on the LAN so the Shortcut can reach the kernel.
- Portrait lock, Auto-Lock Never, AssistiveTouch at max sensitivity.
- A one-time probe of exec/url with a custom URL scheme and with a long URL, since neither is documented.
- IMConsole.exe running and logged in.

WATCH OUT

- The apps-that-honor-prefill matrix is an external claim about third-party apps, not a vendor-documented fact; treat any specific list as a hypothesis to measure on this rack, not a finding, and schedule a re-probe rather than hard-coding it into a production flow.
- Neither vendor doc's examples go beyond bare web domains; if only http(s) works, platforms whose prefill lives behind a custom scheme fall back to the web composer or the clipboard route. Probe this first, since it is the assumption everything else depends on.
- exec/url fires through the same single hardware-key-to-AssistiveTouch-to-Shortcut trigger path as every other `/shortcut/*` call; that binding is a manual, console-only, per-phone setup step with no API, and a failure there (unanswered permission dialog, unedited kernel IP, wrong board bound) returns code 30 after the full outtime window, not a quick failure — budget worst case up to the default 15 s per phone per attempt, not an optimistic happy path.
- No maximum URL length is documented; Safari handles several thousand characters but the Shortcut layer's own limit is unknown and should be found by bisection before a long caption is trusted.

- A universal link may open Safari or the app depending on the phone's prior per-domain choice, a state the API cannot read, making resulting coordinates non-deterministic across a fan-out group.
- The fan-out is only safe for identical messages; a per-recipient message is one call per phone.
- Since exec/url and clipboard/set share the same single trigger channel, concurrent use from a different script against the same phone races on outtime and clipboard state — there is no isolation between two automations sharing a rig.
- Treat any "Action Blocked" or rate-limit response as a hard stop per phone rather than a retry condition, and keep volumes within the platform's own published limits; the platform's anti-spam systems, not the kernel, set the real ceiling.

SCALE

One exec/url call per distinct message fans out to every phone in the id list, plus one tap or ENTER — roughly 2-4 seconds per message end to end, with the shortcut round trip (hotkey, iOS Shortcut, kernel over Wi-Fi) dominating rather than the HTTP call itself, versus three round trips for the clipboard route. Verification does not fan out, 2 ms per raw BMP frame per phone, gated by AirPlay source rate to roughly 35 ms on the fast phone and 220 ms on the slow one. That is a kernel-only ceiling on the order of 900-1800 messages/hour/phone with no relationship to a sustainable real-world send rate: a binding failure costs up to the full 15 s outtime per attempt, and target-platform rate limiting and anti-spam detection will bind well below that ceiling for any real messaging app, a platform-policy question this rig's telemetry cannot answer.

Also worth knowing

Domain index: where else these primitives apply

Every card above is built on the same five moves: an album push (`/shortcut/album/upload`), a clipboard fan-out (`/shortcut/clipboard/set` plus WIN+v), a composer deep link (`/mouse/click` on a captured icon, or the unverified `/shortcut/exec/url`), a golden-frame diff (`/pic/screenshot?binary=true` , mean absolute pixel difference against an idle floor of 0.02-0.03), and group targeting (`/device/group/get-dev` , XP path style; the Phonefarm edition calls the same idea `get_group_list` and `set_group` , function-name style, and neither name works against this XP kernel). None of that changes when the app on screen is a job board instead of TikTok. The table below is a map, not fifteen new cards: it says which of the five primitives a domain family actually needs and names the one thing that stops the chain short of full automation there. Two mechanical facts apply to every row. First, this rig's own device records report group `未分组` (ungrouped) and a live call to `/device/group/get` returns an empty list, so "group targeting" below is a documented, callable primitive, not something already configured here; a real group has to be created with `/device/group/set` before `/device/group/get-dev` has anything to return. Second, with OCR reporting data.code 28 (plug-in not connected) and image search unverified, every "confirms" in the chain column means a pixel region changed, not that the rack read what it says.

Domain family	Reused chain	Feasibility	One blocker that bites this family
Recruiting/HR, job boards	`/device/group/get-dev` picks a QA cohort; `/mouse/click` opens the ATS or job app; `/shortcut/clipboard/set` pastes cover-letter or screening-answer text; `/pic/screenshot` diff confirms the apply step advanced.	NEEDS SHORTCUT WITH CAVEATS	No OCR to read multi-field application or screening forms, and identical text pasted from many device-bound accounts into one job board is the duplicate-application signature ATS anti-spam already screens for. Stays at one account per phone testing your own listing's apply funnel, not scaled applying.
Dating and marketplace trust-and-safety review	`/shortcut/album/upload` stages a reported image on a review phone; `/mouse/click` opens the moderation queue; `/pic/screenshot` diff confirms an action registered; `/device/group/get-dev` routes report categories to separate phones once a real group exists.	NEEDS SHORTCUT WITH CAVEATS	No OCR to read the report reason or a chat transcript off screen, so a human still reads every screenshot before an action is taken. This is a queue router and evidence-capture aid, not an automated moderation decision-maker.
Gig and field-service operator apps	`/mouse/click` or `/shortcut/exec/url` opens the driver or technician app; `/pic/screenshot` diff watches the job-offer screen for a state change (idle floor 0.02-0.03 vs. a real reaction 20-160); `/device/group/get-dev` fans a status toggle to a fleet-test group; `/shortcut/clipboard/set` sends a canned customer reply.	NEEDS SHORTCUT WITH CAVEATS	A pixel diff says the offer screen changed, not what job or pay it shows, since there is no OCR. Accepting or declining a live gig offer is a real dispatch commitment, so this stays at online/offline status checks and canned-reply QA on accounts you operate.
Real estate listing apps	`/shortcut/album/upload` seeds listing photos; `/mouse/click` walks the listing composer; `/shortcut/clipboard/set` pastes the description; `/pic/screenshot` diff confirms the Published banner.	NEEDS SHORTCUT WITH CAVEATS	Same rule as the Facebook Marketplace card above: a published listing and any resulting lead are real. Fanning identical listings across many device-bound accounts is exactly what duplicate-listing detection is built to catch, so this stays to one account publishing per phone.
Travel and hospitality (OTA, airline, hotel apps)	`/mouse/click` or `/shortcut/exec/url` opens the booking app; `/pic/screenshot` diff confirms a fare-search screen loaded; `/device/group/get-dev`, once populated, lets N phones poll N routes or dates in parallel.	PARTIAL WITH CAVEATS	No OCR to read a fare or price number off the screen, only that the screen changed, so this is availability-presence monitoring and booking-flow UI QA, not fare scraping. The Prohibited-actions list also keeps this pipeline off any "Book" tap; that is a real purchase the operator has to make themselves.

Domain family	Reused chain	Feasibility	One blocker that bites this family
Education (LMS and course apps)	`/mouse/click` opens the LMS app; `/shortcut/clipboard/set` posts identical announcement or feedback text to an instructor or TA cohort; `/pic/screenshot` diff confirms the post landed; `/device/group/get-dev` separates a student-view group from an instructor-view group.	NEEDS SHORTCUT WITH CAVEATS	No OCR to read or grade free-text student submissions, so the reused mechanics cover posting and UI QA only. Auto-grading or scraping submitted work is out of reach on this rig.
Mobile gaming (regression testing on your own title)	`/mouse/click` launches the build and drives an input sequence; `/pic/screenshot` diff checks whether a menu or dialog advanced; `/device/group/get-dev` replays the same sequence across identical devices.	PARTIAL WITH CAVEATS	No OCR or image search to read a score, currency count or dialog text, only that pixels changed. That limits this to input-replay regression testing on a build you own, not reading or farming a live in-game economy.
Healthcare and telehealth apps	`/mouse/click` or `/shortcut/exec/url` opens the portal app for a UI smoke test; `/pic/screenshot` diff confirms a screen transition; `/device/group/get-dev` repeats the check across every phone on the same build.	PARTIAL WITH CAVEATS	Real content here is PHI, and the control API is plaintext and unauthenticated on the LAN, with no OCR to verify what is actually rendered. Stays to UI smoke-testing with synthetic test accounts and test data, never live patient data.
Finance and banking apps as targets, not as the account heartbeat	`/mouse/click` opens the app for its own UI regression test; `/pic/screenshot` diff confirms a post-tap screen change; `/device/group/get-dev` runs the same smoke test across a device bank.	PARTIAL WITH CAVEATS	The Prohibited-actions list bars this pipeline from executing any trade or transfer, and from entering financial credentials through it. Even short of that, there is no OCR to verify a balance or confirmation screen, so this narrows to pre-login launch and navigation checks on a test build.
Government and e-ID apps	`/mouse/click` or `/shortcut/exec/url` opens the portal or wallet app; `/pic/screenshot` diff confirms a known screen rendered.	PARTIAL WITH CAVEATS	e-ID and government apps sit behind device-bound biometric or hardware-backed key attestation the rack cannot see or replay: the API is a mouse-and-keyboard emulator plus a mirrored screen, with no camera, no biometric injection and no Bluetooth. This reduces to confirming an app opens and a known screen renders, nothing that touches the credential itself.

Domain family	Reused chain	Feasibility	One blocker that bites this family
IoT/smart-home and wearable companion apps	`/mouse/click` opens the companion app once a device is already paired by hand; `/pic/screenshot` diff confirms a control tap (a toggle, a scene) changed the phone-side UI.	PARTIAL WITH CAVEATS	The rack has no Bluetooth radio and no BLE peripheral role. Pairing a phone to a smart-home hub or wearable is a system-level Bluetooth prompt, Settings or a first-run pairing sheet, that nothing in this API can originate or answer. Onboarding a new accessory is not possible through the rack; only driving an app whose accessory is already paired is.
Media and streaming (video/OTT apps)	`/mouse/click` or `/shortcut/exec/url` launches the app and drives playback controls; `/pic/screenshot` diff confirms a play/pause state change.	NEEDS SHORTCUT WITH CAVEATS	No OCR to read a title, an episode name or an ad-countdown timer, so this confirms that something is playing, not which asset or whether an ad slot served. Inflating watch time or view counts across the fleet is an ad-fraud pattern this document will not detail as a technique; keep this to your own app build's playback-path smoke test.
Podcasts and music streaming apps	Same playback pattern as media and streaming, plus `/shortcut/clipboard/set` to post a caption when sharing an episode or track from a single account.	NEEDS SHORTCUT WITH CAVEATS	Identical share, follow or like actions timed to the same millisecond across many device-bound accounts is stream-count or engagement manipulation, the reward-farming loop this document excludes. The legitimate remainder is single-account posting and playback QA, not fleet-wide engagement.
Internal enterprise apps (MDM-managed line-of-business tools)	`/shortcut/exec/url` or `/mouse/click` opens the internal tool; `/shortcut/clipboard/set` delivers identical or per-phone form text; `/pic/screenshot` diff confirms a submit or sync screen changed; `/device/group/get-dev` runs the same check across a cohort once `/device/group/set` has actually populated one.	NEEDS SHORTCUT WITH CAVEATS	None of the external platform-policy risks above apply, since there is no outside platform to violate. The remaining blocker is entirely local: no OCR to read back a confirmation banner or a validation error, so a human still reads the final screenshot to confirm the workflow actually completed.

> OPEN **Group targeting is a real primitive with nothing in it yet**
>
> `/device/group/get-dev` takes a group id and returns the phones in it, and the doc shows it working with a populated group on a different install. On this rig every phone's `gname` reads 未分组 and a live call to `/device/group/get` returns an empty list, so any row above that leans on group targeting first needs a one-time `/device/group/set` call per cohort (Marketplace phones, LMS student-view phones, a fleet-test lane) before it does anything. That setup step is unverified on this rig, same status as the Shortcut round trip itself.

Account management

Logging in, logging out and rotating credentials on accounts you own reduces to the same mouse, key and screenshot primitives already proven on this rig, with the clipboard and the iMouse Shortcut picking up wherever `/key/sendkey`'s ASCII limit stops. The harder half is not driving the UI, it is knowing what happened afterward: with the OCR and image-search plug-ins both dark here (code 28), every login result, every OTP arrival and every signed-in check has to be read off a raw frame by your own pixel-diff code, not the vendor's recognition endpoints.

Login, logout and credential rotation

Credential vault to keystrokes: login, logout and session rotation

PARTIAL WITH CAVEATS

Keep every credential on the PC and never on the phone. A login job taps the username and password fields, types them through the HID keyboard path, submits, and classifies the result against stored crops for a signed-in state, a wrong-password state and a 2FA challenge. Logout and slot relabeling follow the same pattern, so one vault can own many accounts across many slots without a human touching a screen, as long as the passwords stay ASCII and every result stays auditable.

HOW

1. `get_device_list + get_usb_list` XP `GET /api/device/get?id=<id> and GET /api/config/usb/get` — confirm video (state != 0, live air_pid) and control (board state 2) are both up, and read imgw/imgh (500x1080) as the coordinate space.
2. `mouse_reset_pos` XP `GET /api/mouse/reset?id=<id>` — clear the silent stuck-click state before any tap-critical run.
3. `send_key fn_key + click` XP `GET /api/key/sendkey?fn_key=WIN+h (twice), then GET /api/mouse/click` — go Home twice, then navigate to the login screen.
4. `click then send_key` XP `POST /api/mouse/click, then POST /api/key/sendkey {key: username}` — focus and type the username, ASCII only, POST JSON never GET.
5. `click then send_key` XP `POST /api/mouse/click, then POST /api/key/sendkey {key: password}` — same for the password field, one device id at a time.
6. `shortcut#10 → send_key WIN+v → shortcut#10 → shortcut#11` XP `POST /api/shortcut/clipboard/set, GET /api/key/sendkey?fn_key=WIN+v, verify with GET /api/shortcut/clipboard/get` — non-ASCII secrets go through the clipboard instead.
7. `send_key fn_key` XP `POST /api/key/sendkey {fn_key: ENTER}` — submit.
8. `get_device_screenshot` XP `GET /api/pic/screenshot?id=<id>&binary=true` — capture a raw BMP and classify the outcome client-side.
9. `send_key` XP `POST /api/mouse/click (code field), then POST /api/key/sendkey {key: <TOTP>}` — type a PC-generated TOTP if the classifier sees a 2FA screen; an SMS code stops the job for a human.
10. `click + get_device_screenshot` XP `GET /api/mouse/click (Log out → confirm), then GET /api/pic/screenshot?binary=true` — log out and confirm the login wall is back before marking the slot closed.

11. `set_dev` XP `GET /api/device/set?id=<id>&name=Rack1-Slot07` — label the slot, never the account, in the device record.

NEEDS

- IMConsole.exe running and logged in; with only IMService.exe up, both phones showed state 0 and no boards on this rig.
- AssistiveTouch on with maximum tracking sensitivity, Full Keyboard Access on, Trackpad & Mouse speed maxed, Auto-Lock never, portrait lock; none of this is queryable over the API.
- Board bound (vid and pid together) and at USB state 2, location_crc non-empty (B0DBD800 shared-library profile here).
- Per-app, per-version golden crops for signed-in, wrong-password and 2FA states, captured per phone since the two XRs have different wallpapers.
- A PC-side secret store; no password manager or notes app on the handsets.
- iMouse Shortcut installed and AssistiveTouch-bound, only for the Unicode clipboard path.
- Authorization to administer every account the vault holds.

WATCH OUT

- OCR is dead on this rig (code 28), so every outcome classification is client-side pixel work on raw BMP frames, not a vendor recognition call.
- The Unicode clipboard path is Shortcut-gated, and the Phonefarm sibling doc marks its whole shortcut dispatcher professional-only; ASCII typing needs neither.
- XP has no `send_text`; clipboard/set plus WIN+v is the substitute, and outtime is milliseconds on XP versus seconds in the Phonefarm doc.
- Clipboard scrubbing is a documented write, not a documented erase: overwrite with a placeholder and read it back, do not try to blank the pasteboard.
- The ASCII claim on `/key/sendkey` does not enumerate which punctuation survives; validate the vault's character set once per board generation rather than assume full ASCII.
- `/mouse/click` can silently stop responding while `/mouse/move` and hotkeys keep working. Reset before every tap-critical run, or a password can be typed into nothing while the job reports success.
- `/mouse/*` and `/key/*` take comma-separated id lists; a stray comma mirrors one account's password to every listed phone.
- SMS-delivered 2FA is out of reach in both editions: no SMS endpoint, and reading a banner needs the missing OCR plug-in. TOTP typed from the vault works; SMS needs a human or an out-of-band feed.
- iOS can interpose a "Save Password to Keychain?" sheet or an autofill bar after submit that none of the three golden crops account for; add a fourth crop and a dismiss step.
- Nothing re-verifies that the username or password focus click actually landed before typing. Add a keyboard-visible or pointer-box check after each focus tap, not just at final classification, or a masked field can receive zero characters while step 8 reports success against a generic "try again" screen.
- No inter-keystroke wait or pre-submit check of the masked-dot count is built in; ENTER could fire before typing finishes if key injection lags the HTTP response.
- Only two phones are confirmed live and control-bound on this rig; a 19-phone claim is a capacity projection, not a measured result.
- The API has no authentication on the LAN, so a credential sent to `/key/sendkey` crosses as plaintext HTTP any host on the segment can read. POST JSON, never GET, and firewall 9911 and 17000 to trusted hosts. Treat a rate-limit or "too many attempts" response as a stop signal, not something to retry through.

SCALE

Logins run one phone at a time because the values differ per slot, but phones do not contend: two pollers held 83 and 85 req/s, ten held 512 req/s, and input traffic did not slow frame polling. Per-call cost is small (screenshot p50 2 ms raw BMP; never JPEG here, it is 40 ms with a roughly 21 req/s cap), but pointer travel and the target app's own response time dominate wall-clock: roughly 10 to 20 seconds per phone for a full login-verify-logout-verify cycle. That caps a sane cadence at tens of rotations per hour per phone, not hundreds, since backoff between attempts matters more than raw API throughput. Scaling past the two confirmed live phones to the rack's 19 boards is unverified.

Unicode-safe password rotation campaign across the rack

NEEDS SHORTCUT WITH CAVEATS

Rotate the passwords of accounts you own, fleet-wide, from the PC: push the new secret to each phone's clipboard through the iMouse Shortcut, drive the change-password screen with the pointer, paste with WIN+v, submit, verify on a captured frame, and only then commit the new secret to the vault. This is what makes long Unicode or emoji passphrases possible at all, since `/key/sendkey` is ASCII-only in both editions.

> TRAP **Some apps paste-block the new-password field**
>
> Secure text-entry fields on banking-grade or security-conscious apps can refuse a paste specifically to defeat clipboard-based credential tools. The rig cannot see or override that at the HID layer, and the ASCII fallback does not cover the non-ASCII passphrases this workflow exists for. Probe paste-acceptance on a given app (paste a known string, read it back from a frame) before committing to the clipboard path for it.

HOW

1. `get_device_list + get_usb_list` XP `GET /api/device/get?id=<id> and GET /api/config/usb/get` — confirm control and video, read imgw/imgh.
2. `mouse_reset_pos` XP `GET /api/mouse/reset?id=<id>` — clear the stuck-click state before touching the password screen.
3. `shortcut#10` XP `POST /api/shortcut/clipboard/set {id, text: <new password>, sleep, outtime}` — push the new secret to that phone's clipboard.
4. `click` XP `GET /api/mouse/click?id=<id>&x=&y=` — navigate to the app's change-password screen.
5. `send_key / shortcut#10 + paste` XP `GET /api/key/sendkey?key=<old password> or clipboard/set + fn_key=WIN+v` — fill the old-password field.
6. `send_key fn_key` XP `GET /api/mouse/click (focus), then GET /api/key/sendkey?fn_key=WIN+v (twice)` — paste the new secret into the new and confirm fields.
7. `send_key fn_key / click` XP `GET /api/key/sendkey?fn_key=ENTER or GET /api/mouse/click (Save)` — submit.
8. `get_device_screenshot` XP `GET /api/pic/screenshot?id=<id>&binary=true&rect=[l,t,r,b]` — capture the result banner and verify client-side.
9. `shortcut#10 then shortcut#11` XP `POST clipboard/set {text: decoy}, then GET clipboard/get` — scrub the pasteboard and prove the scrub took.
10. `(no API call)` XP `client-side bookkeeping` — commit to the vault only after the previous step passed; queue re-login for every other device holding that account.

NEEDS

- Accounts you own, with the right to change their credentials.

- iMouse Shortcut installed on every phone (Advanced settings: allow running scripts, allow sharing large amounts of data), IP edited to the kernel PC, run once with every prompt allowed.
- Shortcut bound to an AssistiveTouch custom key via IMConsole's "Send Binding Key".
- Phone able to reach the kernel on TCP 9911 over the LAN, since the Shortcut posts its result back through the kernel.
- Board bound and at USB state 2, location_crc non-empty.
- Full Keyboard Access on with a 1 s auto-hide, AssistiveTouch on, Trackpad & Mouse speed maxed, Auto-Lock never, portrait lock.
- Per-app, per-version change-password screen coordinates and a success-state crop, captured per phone.
- IMConsole.exe running and logged in.
- A vault that supports staged, propose-then-commit secrets so a failed verification does not overwrite the working password.

WATCH OUT

- Unicode entirely depends on the Shortcut. No Shortcut, no non-ASCII password, and the Phonefarm sibling gates its whole shortcut dispatcher behind a professional licence.
- Phonefarm-style bare function POSTs return HTTP 500 on this rig; use XP paths only, and remember outtime is milliseconds here, seconds in the Phonefarm doc. Code 30 means the hotkey never launched the Shortcut.
- Verification is unassisted. OCR and image-search are both effectively unavailable (code 28), so template matching happens in your own process; `/shortcut/other/ocr` and `/shortcut/other/find-image` exist but are documented by path name only, so probe before relying on them.
- `/pic/screenshot` and `/shortcut/clipboard/get` are single-id only; only the mouse, key and clipboard-set calls fan out over a comma list.
- Real change-password flows often demand a second factor or an out-of-band confirmation the rig cannot complete, and the SMS route is absent in both editions. Expect a human step on those apps.
- A full-height swipe is roughly 580 ms of pointer walk and iOS may read it as a drag rather than a flick; scroll long settings pages with `/mouse/wheel` instead.
- iOS's own "Save Password to Keychain?" sheet or an autofill bar is not one of the golden-crop states and reads as ambiguous. That correctly blocks the commit, but budget for the extra dismiss-and-reclassify step it forces.
- Per-phone Shortcut setup (install, edit the IP, grant prompts, bind an AssistiveTouch key) is manual, one-time labor per phone. For onboarding a real fleet this setup cost, not API throughput, is the actual bottleneck.
- The 19-phone throughput projection in the source draft was extrapolated from board count; only two phones were confirmed mirroring and bound in this rig's evidence. Re-probe all 19 before promising that scale.
- The password crosses the LAN as plaintext HTTP to an unauthenticated API, then briefly sits on the iOS pasteboard where any foreground app can read it. Firewall 9911 and 17000, and treat a failed verification as "keep the old secret, flag a human," never as a reason to commit on faith. A password change normally invalidates other sessions, so expect a re-login wave, and keep the automation's cadence sane against the platform's terms.

SCALE

Dominated by the Shortcut round trip, not the API: each clipboard call is a HID hotkey plus a Wi-Fi callback bounded by a 15 s outtime, with real completion usually 1 to 3 seconds, and this flow makes two to four of them per account. Pointer travel and verification screenshots add low hundreds of milliseconds. Budget roughly 10 to 25 seconds per phone end to end, so a 19-phone campaign is 3 to 8 minutes sequential, faster in parallel since phones do not contend (ten concurrent screenshot pollers held 512 req/s). Only `/pic/screenshot` and `/shortcut/clipboard/get` force one-at-a-time handling; give each phone its own worker. Real safe cadence is far below the theoretical 150 to 350 rotations/hour/phone, because password changes trigger re-authentication waves and confirmation emails; plan for single digits to low tens per hour per phone with backoff, and treat the 19-phone figure as unverified beyond the two confirmed live phones.

Knowing which slots are actually signed in

Session-state sweep: classify signed-in, login wall, captcha or ban from the mirror frame

NATIVE API | WITH CAVEATS

On a fixed interval, bring every mirroring phone to a known state, open the target app, grab one raw frame per phone, and classify it as signed-in, login wall, captcha, account-restricted, or an unexpected dialog. A rack's real inventory is accounts still signed in and unrestricted, not just phones that happen to be mirroring; without this sensor a scheduler can drive taps into a login wall or a captcha for hours and produce silent, zero-yield work.

HOW

1. `get_device_list / get_usb_list` XP `GET /api/device/get (optional id=a,b) and GET /api/config/usb/get` — enumerate targets with live video and control state, read imgw/imgh per phone.
2. `mouse_reset_pos` XP `GET /api/mouse/reset?id=<list>` — clear the stuck-click state on the whole sweep set in one call.
3. `send_key fn_key=WIN+h` XP `GET /api/key/sendkey?id=<list>&fn_key=WIN+h (twice, with a settle wait)` — normalize the whole set to Home in one request.
4. `click / swipe / send_key / shortcut#13` XP `GET /api/mouse/click, /api/mouse/swipe, /api/key/sendkey, or /api/shortcut/exec/url` — open the target app by icon tap, home-screen search, or Shortcut deep link.
5. `get_device_screenshot` XP `GET /api/pic/screenshot?id=<single>&binary=true` — one raw BMP per phone, one request each, run concurrently.
6. `(none, client-side)` XP `your own code on the returned bytes` — mean-absolute-pixel-diff fixed regions against golden crops for each state; the only classifier that works here today.
7. `find_multi_color` XP `POST /api/pic/find-multi-color` — optional distinctive-colour-block classifier, for example a restriction banner. Probe once for plug-in gating before relying on it.
8. `ocr / ocr_ex` XP `GET /api/pic/ocr, POST /api/pic/find-text` — optional text classifier, only once the OCR plug-in is actually installed; it answers code 28 today.
9. `get_device_screenshot (save) / send_key` XP `GET /api/pic/screenshot?save_path=..., GET /api/key/sendkey?fn_key=TAB+b` — act on the verdict: capture forensic evidence and pause the queue, or best-effort dismiss a benign dialog and re-check.

NEEDS

- IMConsole.exe running and logged in.

- Phones mirroring with a live air_pid matching AirplayMirroring.exe and an established socket on TCP 17000.
- Per-phone imgw/imgh (500x1080 here, not the 414x896 logical size); golden crops and coordinates cannot cross phones with different values.
- Your own bottom-up BMP decoder and mean-absolute-difference comparator; nothing in the API does this.
- Golden crops per app, per app version and per phone, at fixed display settings (lowest brightness, Standard view, no True Tone or Night Shift drift).
- AssistiveTouch, Full Keyboard Access, Trackpad & Mouse speed, Auto-Lock never; all unqueryable input prerequisites.
- Board bound and at USB state 2, location_crc non-empty.
- OCR plug-in (installed, ocr_num set) and OpenCV plug-in (opencv_num ≥ 1) only for the optional recognition legs; both are effectively absent here.
- iMouse Shortcut, only for the deep-link app-opening route.
- Authorization to operate the swept accounts under the platform's terms.

WATCH OUT

- Recognition is not native here: `/pic/ocr` answers code 28, so ocr, ocr-ex, find-text and find-text-ex are all unusable. Assume the image-search plug-in is in the same state until probed.
- `/pic/find-multi-color` sits on the same plug-in-gated page and has a Shortcut fallback, the same pattern OCR follows, so probe its data.code before designing a classifier around it.
- Screenshot-family endpoints are single-id only; navigation fans out with a comma list but capture is one request per phone.
- Frame freshness, not request latency, is the real constraint: one XR delivers roughly 29 new frames/s under motion, its identical twin roughly 4.5. Settle at least 220 ms on the slow phone and never assume 30 fps.
- Never compare frames by hash. MD5 or byte equality reports "changed" on an untouched phone every time from clock drift and encoder noise. Use mean-abs-diff (idle 0.02 to 0.03, real change 20 to 160).
- Golden crops are the real maintenance burden. They break on every app update and per-phone display drift, and cannot be shared across phones with different home screens or imgw/imgh.
- Getting to the app's home view is the weakest link, not the capture. The two XRs' different home screens mean shared coordinates are wrong by construction, and the only layout-independent route, a Shortcut deep link, pulls in the whole Shortcut prerequisite chain.
- A cached view can look signed-in while the session is dead. Classify a screen that cannot render without a valid session, not a feed that may be served from cache.
- Nothing here solves a captcha or a restriction notice. Both are terminal states: capture evidence, pause the queue, escalate to a person, and treat repeated notices as a signal to reduce or stop the automation, not to route around it.
- iOS system sheets (Keychain autofill or save prompts, "Allow Paste from X?" banners, low-power-mode banners) can look identical to an app dialog to a fixed-region diff classifier unless a golden crop exists for each, and these can shift wording or position across iOS point releases even on the same hardware. Crops need to be per-iOS-version, not just per-app-version.
- TAB+b is documented as "not supported by all apps," and on iOS a Tab-family key more plausibly moves focus than dismisses a sheet. A mis-fire could shift focus into an unintended field that gets typed over on the next sweep cycle.

- The frame-rate asymmetry between phones means a sweep interval tuned to the fast phone will more often catch the slow one mid-transition, systematically biasing which phone's states get misclassified. Settle-wait budgets need to be per-phone.
- A 19-phone throughput claim was extrapolated from board and hub count; only two phones showed a live device state and air_pid in this rig's evidence, and per-frame classifier CPU at higher concurrency is untested.
- An unrecognized frame, from a failed navigation or a swallowed keystroke, does not cleanly fall into any of the four semantic buckets. Define an explicit "navigation failed / unrecognized" bucket rather than forcing a low-confidence match; reporting a banned phone as healthy is the worse failure mode.
- Forensic frames are full screenshots of a signed-in account and contain personal data. Store them encrypted with a short retention window and off any shared or version-controlled path. The API has no authentication on the LAN, so firewall 9911 and 17000 to trusted hosts.

SCALE

Capture is cheap and embarrassingly parallel: two pollers held 83 and 85 req/s, ten held 512 req/s at p50 3 ms, and input traffic did not slow frame polling. A full 19-phone raw-BMP capture is well under a second of wall time and is bounded by your own decode and compare cost, not the kernel; reuse one buffer per poller, since naive allocation measured 488 MB/s and 496 gen-2 GCs per 8 s, versus 2.9 MB/s with reuse. Never switch to JPEG on loopback, it is 40 ms against 2 ms with a roughly 21 req/s cap. Navigation legs are one request each regardless of fleet size since `/mouse/*` and `/key/*` fan out over comma lists; the real ceilings are per-phone settle time, pointer travel at roughly 0.6 ms/px (prefer short strokes over full-height swipes), and your classifier's per-frame CPU, untested above two concurrent phones. A realistic sweep round lands in the tens of seconds for a rack this size, fine for a cadence measured in minutes, but treat the 19-phone figure as a capacity projection until all 19 boards are confirmed live.

Capturing one-time codes

Copy OTP and email codes out of Messages/Mail via the phone clipboard

NEEDS SHORTCUT

WITH CAVEATS

For codes that never reach the QuickType strip, such as email codes, in-app messages or untagged SMS formats, drive the phone's own UI to copy the code, read it off the phone with `/shortcut/clipboard/get`, and type it back with `/key/sendkey`. This is the only supported text-extraction channel on this rig: there is no message-reading API in either edition, and the server-side OCR fallback is dead here.

> TRAP **Messages and Mail are not the same gesture**
>
> iOS's context-sensitive "Copy Code" long-press menu is a Messages behavior. Mail has no equivalent auto-detection, so pulling a code from an email needs a manual drag-handle text selection (tap to place a cursor, then drag the handle to bracket the code) before Copy appears. As written, a single long-press-then-tap only works for Messages; treat Mail as a separate, harder sub-case with its own calibrated sequence.

HOW

1. `get_device_list + get_usb_list` XP `GET /api/device/get?id=<id> and GET /api/config/usb/get` — confirm control and video are up, read imgw/imgh.
2. `mouse_reset_pos` XP `GET /api/mouse/reset?id=<id>` — clear the stuck-click state before the sequence.

3. `send_key fn_key=WIN+h` XP `GET /api/key/sendkey?fn_key=WIN+h (twice)` — normalize to Home.
4. `get_device_screenshot → click` XP `GET /api/pic/screenshot?binary=true, then GET /api/mouse/click` — open Messages or Mail and tap the newest thread from a live frame.
5. `click time=800` XP `GET /api/mouse/click?id=<id>&x=&y=&time=800` — long-press the code text to raise the iOS context menu (Messages) or place a cursor (Mail).
6. `click` XP `GET /api/mouse/click` — tap Copy or Copy Code, using the frame taken after the long-press, not a cached coordinate.
7. `shortcut#11` XP `GET /api/shortcut/clipboard/get?id=<single>&outtime=<ms>` — read the code off the phone; validate data.text against a 4 to 8 digit regex.
8. `send_key fn_key=AppSwitch, then send_key key=<digits>` XP `GET /api/key/sendkey` — return to the app and type the digits, ASCII, no paste needed.
9. `shortcut#10` XP `GET /api/shortcut/clipboard/set?id=<list>&text=<placeholder>` — overwrite the pasteboard afterward; this is a scrub, not an erase.
10. `get_device_screenshot + ocr` XP `GET /api/pic/screenshot?binary=true&rect=[...] then client-side OCR` — plug-in-free fallback when the clipboard path is unavailable; server-side OCR answers code 28 here.

NEEDS

- iMouse Shortcut installed on every phone (Advanced: allow running scripts, allow sharing large amounts of data), IP edited in, run once with every "Allow" granted.
- Shortcut bound to an AssistiveTouch custom key; board bound and at USB state 2.
- Phone able to reach the kernel on 9911 for the Shortcut's Wi-Fi callback, alongside the USB trigger channel.
- AssistiveTouch, Full Keyboard Access, Trackpad & Mouse speed, Auto-Lock never, portrait lock.
- location_crc non-empty on the device record.
- Thread and menu coordinates read live per phone, never hard-coded or shared between the two XRs.
- IMConsole.exe running and logged in.
- Only phones and accounts the operator owns.

WATCH OUT

- The whole chain hinges on one Shortcut-gated call; without it, harvesting degrades to screenshot plus external OCR, since `/pic/ocr` answers code 28 here.
- `/shortcut/clipboard/get`, `/pic/screenshot`, `/pic/ocr`, `/pic/find-image` and `/pic/find-text` are all single-id only; only clipboard/set and the mouse, key and switch endpoints take a comma list. There is no way to harvest N phones in one request.
- The Phonefarm sibling gates its whole shortcut function family behind a professional licence; XP states no edition gate on `/shortcut/*`, but licensing is enforced per device through login, so treat it as licence-dependent.
- Phonefarm-style bare function POSTs return HTTP 500 on 9911, and 9912 is closed. Use XP paths only, and note outtime is milliseconds on XP, seconds in the Phonefarm doc.
- The long-press depends on iOS's "Copy Code" context menu, which neither vendor doc describes. Verify each frame rather than replaying coordinates, since a different app, iOS version or link-detected code can change the menu layout.
- A silent tap failure is invisible here too: `/mouse/click` can stop working while `/mouse/move` and hardware keys keep succeeding, returning code 0 throughout. Verify with mean-abs-pixel-diff, never MD5 equality.
- Overwriting the pasteboard is cosmetic. It does not clear iOS pasteboard history, does not un-send a Universal Clipboard sync to a paired Mac or iPad, and does not stop any app that already read the

pasteboard in the interval. Disable Handoff on rack phones if that matters.

- No message-read API exists in either edition; Phonefarm's SMS-send and read-aloud shortcut ids are struck through and absent from XP, so the clipboard is the channel, not a convenience.
- This rig has no Shortcut installed on either phone today, so the install-and-bind procedure, itself manual and on-phone, outside the HTTP API, is a blocking one-time setup cost, not a background assumption.
- Bubble geometry shifts with message length and wrapping, so a coordinate calibrated on one test message can miss a differently-worded real one. The per-frame verify step is necessary, not optional, and adds a round trip the throughput math should include.
- Typing digits back in after an AppSwitch round trip assumes the code field still has focus. If the OS backgrounds or reloads the app, the digits could land in the wrong field with no error, since `/key/sendkey` returns one code with no per-keystroke confirmation of where it went.
- The harvested code is a live authentication factor sitting briefly in PC memory and the kernel's HTTP response. Scope it to the single attempt, never log or persist it, and firewall 9911 and 17000 since any LAN host can otherwise call clipboard/get on any phone.

SCALE

Two costs dominate per phone: pointer travel at roughly 0.6 ms/px, so home to thread to code to menu is several hundred milliseconds of board time regardless of whether one phone or a comma-listed many is targeted, and the Shortcut round trip itself, a USB hotkey plus a Wi-Fi callback bounded by a 15 s outtime, so expect seconds, not milliseconds, and code 30 if the Shortcut never fired. Verification frames are cheap (2 ms p50 raw BMP) and phones are fully independent, two pollers ran 83 and 85 req/s with no interference, ten ran 512 req/s total, so run one worker per phone rather than serializing. The real ceiling is AirPlay source rate, roughly 29 fps on one XR and 4.5 on its twin, so a "wait for the menu" step on the slow phone needs 220 ms or more per frame, not a 30 fps assumption. Realistic figure: roughly 5 to 20 s per code per phone if messages are already present, on the order of 100 to 500 codes/hour/phone at best, dropping sharply once message-arrival latency or app-side rate limiting becomes the actual gate rather than the rig.

One-tap OTP from the iOS QuickType suggestion strip

NATIVE API WITH CAVEATS

When a code field is tagged as a one-time-code field, iOS itself offers the SMS code in the QuickType suggestion strip above the keyboard. Tapping it inserts the code without the rig ever reading the message. The chain is entirely XP-native, reset, click, force the keyboard open, poll a cropped frame, tap, ENTER, and the code never transits the PC. The premise, though, is iOS behavior no vendor document covers and this rig has not verified.

> TRAP **A hardware keyboard usually suppresses the software one**
>
> The control board presents to iOS as a HID keyboard, and iOS's normal behavior with an external keyboard attached is to suppress the full software keyboard for a thin predictive-text bar instead. `OPENKeyboard` is documented to force the on-screen keyboard up, but whether that produces a full keyboard with a QuickType row, the thin bar, or no visible change at all is exactly what the one-time manual calibration has to establish, not just the strip's coordinates within an assumed layout.

HOW

1. `get_device_list` XP `GET /api/device/get?id=<id>` — confirm the phone is mirroring and read imgw/imgh (500x1080 here).
2. `mouse_reset_pos` XP `GET /api/mouse/reset?id=<id>` — clear the stuck-click state first.

3. `click` XP `GET /api/mouse/click?button=1&x=&y=` — tap the code field so it takes focus.
4. `send_key (fn_key)` XP `GET /api/key/sendkey?fn_key=OPENKeyboard` — force the on-screen keyboard up so the suggestion strip sits at a repeatable height.
5. `get_device_screenshot` XP `GET /api/pic/screenshot?id=<id>&binary=true` — one-time manual step: trigger a real code, diff two frames, and record the strip band and tap point for this model and iOS build.
6. `get_device_screenshot` XP `GET /api/pic/screenshot?id=<id>&binary=true` — baseline capture of the strip band before the code arrives.
7. `get_device_screenshot (poll)` XP `GET /api/pic/screenshot?id=<id>&binary=true, polled` — poll the same crop every roughly 500 ms for up to 60 s; mean-abs-diff over the band (idle 0.02 to 0.03, real change 20 to 160).
8. `find_multi_color` XP `POST /api/pic/find-multi-color` — optional discriminator to confirm the band holds the AutoFill suggestion, not ordinary predictive text; its own plug-in gating is unverified.
9. `click` XP `GET /api/mouse/click?x=&y=` — tap the suggestion; iOS inserts the code, and the PC never learns it.
10. `get_device_screenshot + send_key` XP `GET /api/pic/screenshot?binary=true, then GET /api/key/sendkey?fn_key=ENTER` — confirm the field region changed and submit; verification is blind, without OCR.
11. `shortcut#11 (fallback)` XP `GET /api/shortcut/clipboard/get?id=<id>` — if the strip never appears, long-press-copy on the phone instead and read the pasteboard; needs the Shortcut installed.

NEEDS

- Only appropriate for accounts you own or your own app under QA; the codes are authentication material.
- An active SIM and cellular service on the handset; unverified on this rig, neither the probe nor HARDWARE.md confirms cellular service on either XR.
- The field must be declared a one-time-code field by the app (iOS `textContentType oneTimeCode`, or `autocomplete="one-time-code"` in Safari); nothing in the API can tell you whether it is.
- Messages notifications enabled and the phone unlocked; the suggestion does not appear on a locked screen.
- Board bound and at USB state 2, location_crc non-empty.
- AssistiveTouch, Full Keyboard Access, Trackpad & Mouse speed, Auto-Lock never, portrait lock.
- Strip band and tap point measured once per model and iOS build, stored in that device's own mirror pixels.
- IMConsole.exe running and logged in.
- The fallback path additionally needs the iMouse Shortcut installed, IP-edited, run once, and AssistiveTouch-bound.

WATCH OUT

- Every call in the core chain is verified on this kernel, but the premise, that the QuickType strip appears and a tap inserts the code, is iOS behavior no vendor document covers and this rig has not tested.
- Detection is blind: without OCR, a diff-triggered tap cannot distinguish an AutoFill code suggestion from an ordinary predictive-text candidate, so it can insert a wrong word. `/pic/find-multi-color` is the only plug-in-free discriminator, and its own plug-in gating is unverified.
- You cannot verify the inserted digits. Confirmation is limited to "the field region changed" and "the next screen appeared," so a wrong insert looks identical to a right one until submit fails.
- Strip geometry is model-, iOS- and layout-specific and will drift across iOS updates; treat it as a per-model-and-build calibration, never a hardcoded constant.
- Coordinates must come from that device's imgw/imgh (500x1080), not its logical width/height (414x896).

- XP has no push-stream screenshot, so this must be a poll, and XP's rect is a flat [left, top, right, bottom], not Phonefarm's four corner points.
- `/mouse/reset` is not optional housekeeping; the measured stuck-click state returns code 0 on every click while doing nothing.
- Nothing confirms the code was accepted by the app, only that the region changed and the next screen appeared. A masked field, a validation delay, or a second confirmation screen all read as identical to success or failure.
- AssistiveTouch and Full Keyboard Access being simultaneously required can themselves change where or whether a QuickType bar renders versus an AssistiveTouch floating menu intercepting the tap; an interaction this rig has not tested.
- Cellular and SIM presence on these two XRs is unconfirmed by any document. Without an active SIM, no SMS ever arrives and the whole premise cannot be exercised on this rig regardless of how well the pixel-diff logic is built.
- Automating OTP entry only makes sense on accounts you own or your own app under QA; against third-party accounts it is account takeover regardless of how the code was obtained. The API is unauthenticated on the LAN, so firewall 9911 and 17000 even though the code itself never touches the PC in this design.

SCALE

Per-phone by nature: each handset waits on its own SMS, and `/pic/screenshot` is single-id, so it is one poller per phone. Cost is trivial against measured headroom: a 500 ms poll is 2 req/s, against a measured 84 req/s per phone alone and 512 req/s across ten pollers. Input traffic and frame polling do not interfere. The real limiter is the AirPlay source rate, roughly 29 new frames/s on one XR under motion and roughly 4.5 on its twin, so allow 220 ms or more for the slow phone's crop to refresh, and discard identical polls as "no suggestion yet" rather than treating a still screen as a failure. Wall-clock is dominated by SMS delivery, seconds to a minute, not the rig; realistic planning figure is well under 60 to 100 OTP attempts/hour per phone even with a perfect rig, and zero if the phone has no active SIM.

Account provisioning

Apple ID sign-in on a fresh phone with same-device 2FA

PLUS EXTERNAL TOOLING · WITH CAVEATS

Sign a phone into an Apple ID entirely from the PC: tap the Settings fields with `/mouse/click`, type the Apple ID and password with `/key/sendkey` (ASCII only), then handle the six-digit verification code. An Apple ID gates every App Store install, TestFlight join, iCloud backup and iMessage/FaceTime identity on a phone, so this is the first commissioning step for any handset entering the rack. It is also the clearest test of this stack's ceiling: it exercises typing, tapping and screen-reading in one flow and shows exactly which links, the code read and the Keychain sheet, still need a person.

HOW

1. `get_device_list / get_usb_list` XP `GET /api/device/get?id=<id>, GET /api/config/usb/get` — confirm the phone is control-ready (board state 2, bound, non-empty location_crc) and video-ready (state != 0, live air_pid), and read imgw/imgh (500x1080) as the coordinate space.
2. `mouse_reset_pos / send_key` XP `GET /api/mouse/reset?id=<id> then GET /api/key/sendkey?id=<id>&fn_key=WIN+h` (twice) — clear the stuck-click state and return to home page one.

3. `click` XP `GET /api/mouse/click?id=<id>&x=&y=` — open Settings by tapping its per-phone icon; there is no Spotlight hotkey in the XP fn_key table.
4. `click` XP `GET /api/mouse/click?id=<id>&x=&y=` — tap "Sign in to your iPhone" at the top of the Settings root list.
5. `send_key` XP `POST /api {"fun":"/key/sendkey","data":{"id":"<id>","key":"<apple id>"}} then fn_key=ENTER` — type the Apple ID; the key parameter is documented ASCII-only.
6. `send_key (ASCII) or shortcut#10 + paste` XP `POST /mouse/click then /key/sendkey key=<password>; non-ASCII: POST /shortcut/clipboard/set then fn_key=WIN+v` — tap the password field and enter the password.
7. `get_device_screenshot` XP `GET /api/pic/screenshot?id=<id>&binary=true&rect=[l,t,r,b]` — capture the 2FA prompt region; rect is XP's flat array, single-id only.
8. `ocr / ocr_ex` XP `GET /api/pic/ocr?id=<id>&rect=[l,t,r,b]` — not usable on this rig (code 28, plug-in not connected); digits must be read client-side or by a human.
9. `click` XP `GET /api/mouse/click?id=<id>&x=&y=` — prefer the QuickType "From Messages" autofill chip when the code lands by SMS on this phone's own SIM; one tap fills all six boxes and sidesteps OCR.
10. `send_key` XP `POST /api/key/sendkey {"id":"<id>","key":"123456"}` — type the code manually if no chip appears.
11. `send_key (unverified)` XP `POST /api/key/sendkey key=<digits>` — the Keychain "enter the passcode of <device>" sheet; whether HID input reaches this secure field on iOS 18.7.10 is unproven and must be tested by hand first.
12. `get_device_screenshot + compare` XP `GET /api/pic/screenshot?id=<id>&binary=true&rect=[Settings header band]` — verify sign-in visually; no field anywhere in the device record reports Apple ID state.

NEEDS

- IMConsole.exe running and logged in; with only IMService.exe up, this rig showed both phones at state 0 and an empty board list.
- Board bound (vid+pid together, single device per call) at /config/usb/get state 2, location_crc non-empty, or every tap lands offset with code 11.
- iOS accessibility settings unreadable over the API: AssistiveTouch on at max sensitivity, Full Keyboard Access on with 1 s auto-hide, Trackpad & Mouse speed max, Auto-Lock Never, portrait lock.
- An Apple ID the operator owns, with this phone's SIM as the trusted number if the 2FA code is to land on-device at all.
- Client-side OCR or a human reading the mirror; the vendor OCR plug-in is not connected (code 28) on this rig.
- For non-ASCII passwords: iMouse Shortcut installed, IP-edited, hotkey-bound via AssistiveTouch; unconfirmed here, probe with `/shortcut/device/ip` first.
- A one-off manual test establishing whether the Keychain passcode sheet accepts hardware-keyboard digits on this iOS build.
- Per-phone stored coordinates for Settings, the sign-in banner, the credential fields and the QuickType chip; the two XRs have different home screens.

WATCH OUT

- "Same-device 2FA" only holds when this phone's SIM is already the trusted number. A fresh phone typically has Apple push the code to an already-trusted device instead of SMS here, so budget for the push-to-other-device case as the default, not the QuickType happy path this chain leans on.

- No OCR on this rig: `/pic/ocr`, `/pic/ocr-ex`, `/pic/find-text` and `/pic/find-text-ex` share the same absent plug-in, so any digit read is client-side or manual work.
- XP's rect on /pic/screenshot, /pic/ocr, /pic/find-image and /pic/find-text is a flat [left,top,right,bottom] array, not the Phonefarm four-corner form.
- /key/sendkey key is ASCII-only by the vendor's own wording; a non-Latin password needs the clipboard+paste route, which drags in the whole Shortcut prerequisite chain and leaves the password sitting in the phone's pasteboard until overwritten.
- XP has no send_text or send_text_ok callback; those are Phonefarm-only and this kernel rejects Phonefarm function names with HTTP 500.
- Secure-input behavior is undocumented in both editions. A field may silently discard keystrokes while /key/sendkey still returns code 0 — an API success is not evidence the phone accepted the input.
- No account-state readout exists anywhere in /device/get, /config/user/info (the kernel's own vendor license, not the Apple ID) or any callback; success detection is pixel-diff or client-side OCR only.
- /mouse/click can silently stop working while /mouse/move and hardware keys keep responding; only /mouse/reset clears it. Send it before the sequence and treat a near-zero frame diff as a stuck pointer, not a wrong coordinate.
- Screenshots are single-id ("Multiple is not allowed"), so the read side of this flow cannot be fanned out even though click and sendkey accept comma lists.
- Any Face ID or biometric prompt is unreachable through the API; the run becomes attended at that point.
- A byte-for-byte identical, machine-fast keystroke cadence across many phones is itself a signal Apple's sign-in risk engine can key on. That is a detection-risk fact to disclose, not something to engineer around.
- Apple ID sign-in commonly interposes an "unusual activity" check or a forced delay after a few sign-ins from the same PC/IP; do not plan a same-day multi-phone batch and expect it to run unattended.
- This flow is single-phone-at-a-time in practice: credentials differ per account, so the comma-id support on /mouse/click and /key/sendkey does not make parallel sign-ins as cheap as parallel screenshot polling.
- The API is unauthenticated on the LAN; the Apple ID, password, 2FA code and any device passcode travel in the clear to any host that can reach port 9911. Use POST JSON, never a query string, firewall 9911 and 17000 to trusted hosts, and have the operator supply credentials at run time rather than storing them.
- Only for accounts the operator owns; not a route to fabricate or bulk-provision identities.

SCALE

One phone at a time per PC, roughly 1-2 minutes each, dominated by Apple's server round trips rather than the API — the rig itself imposes no shared bottleneck (ten independent screenshot pollers sustain 512 req/s at 3 ms p50). /mouse/click, /key/sendkey and /mouse/reset accept comma id lists, but /pic/screenshot is single-id and credentials differ per account, so typing cannot be mirrored across the rack in practice. Expect on the order of 20-40 sign-ins/hour if the SMS-to-this-device path holds, dropping to roughly 5-15/hour when the code must route to another trusted device or be read by a human, and expect Apple's risk engine to slow or halt a same-IP batch well under 40/hour sustained after the first few accounts.

Profile setup

Profile completion: avatar from the album plus pasted Unicode bio

NEEDS SHORTCUT WITH CAVEATS

Push an avatar PNG from the PC into the phone's album via the iMouse Shortcut, tap through the app's photo picker and crop screen with mirror-pixel taps, then set a Unicode bio by pasting from the clipboard, since `/key/sendkey` types ASCII only. The shortcut album transfer is the only documented way to get an arbitrary file onto a phone in this stack, and the clipboard-paste route is the only way to put non-Latin text on screen, so this one chain covers both media and Unicode text entry for any profile-editing job.

HOW

1. `get_device_list` XP `GET /api/device/get?id=<id>` — read imgw/imgh (500x1080) and require state != 0 with a live air_pid plus non-empty location_crc.
2. `get_usb_list` XP `GET /api/config/usb/get` — confirm the board is enumerated and the phone is cabled (state 2), independent of video-readiness.
3. `mouse_reset_pos` XP `GET /api/mouse/reset?id=<id1,id2,...>` — clear the stuck-click state before any tap-critical sequence.
4. `send_key (fn_key WIN+h)` XP `GET /api/key/sendkey?id=<ids>&fn_key=WIN+h` (twice) — normalize to a known home screen before the first scripted tap.
5. `shortcut#7` XP `POST /api {"fun":"/shortcut/album/upload","data":{"id":"<id1,id2>","files":["C:\\farm\\avatars\\0421.png"],"album_name":"","zip":0,"outtime":60000}}` — push the avatar file from the PC into the phone's Recents album.
6. `shortcut#1` XP `the upload response's data.list, or GET /api/shortcut/album/get?id=<id>&num=5` — read back the name iOS actually gave the imported photo (never the PC filename).
7. `click` XP `GET /api/mouse/click?id=<ids>&x=&y=` — open the app's profile editor and reach "Choose from library".
8. `click` XP `GET /api/mouse/click?id=<ids>&x=&y=` — tap the target tile in the picker grid, then confirm the crop.
9. `shortcut#10` XP `POST /api {"fun":"/shortcut/clipboard/set","data":{"id":"<id1,id2>","text":"<bio>","sleep":1,"outtime":15000}}` — put the Unicode bio on the phone clipboard.
10. `send_key (fn_key)` XP `GET /api/mouse/click?id=<ids> then GET /api/key/sendkey?id=<ids>&fn_key=WIN+v` — focus the bio field and paste.
11. `click` XP `GET /api/mouse/click?id=<ids>&x=&y=` — save the profile; the irreversible step of the flow.
12. `get_device_screenshot` XP `GET /api/pic/screenshot?id=<id>&binary=true[&rect=[l,t,r,b]]` — verify the result client-side against a known-good reference frame.
13. `shortcut#3` XP `POST /api {"fun":"/shortcut/album/del","data":{"id":"<id1,id2>","list":[{"album_name":"","name":"<name from upload>","ext":"png"}],"outtime":15000}}` — delete the uploaded image by its exact returned name+ext.
14. `shortcut#10` XP `POST /api {"fun":"/shortcut/clipboard/set","data":{"id":"<ids>","text":" "}}` — optional clipboard scrub, cheap and reversible.

NEEDS

- iMouse Shortcut installed on every phone (Settings > Shortcuts > Advanced, all permissions allowed), IP-edited to the kernel, and bound to an AssistiveTouch custom key; the shortcut launches by a HID hotkey, so the board must be bound and working for any /shortcut/* call to fire.

- Phone reachable from the kernel on TCP 9911 over the LAN; Wi-Fi off or airplane mode breaks every shortcut call and mirroring, while USB control survives.
- USB board bound (/device/set with vid+pid together) at /config/usb/get state 2; mouse parameters present (location_crc non-empty).
- iOS accessibility settings unreadable over the API: AssistiveTouch on at max sensitivity, Full Keyboard Access on (required for WIN+v), Trackpad & Mouse speed max, Auto-Lock Never, portrait lock, phone unlocked.
- Photo-library permission granted to the target app once by hand, and its photo picker reached manually so tile coordinates can be measured from a raw BMP frame.
- A per-app, per-version coordinate profile in 500x1080 mirror pixels for every screen in the flow; no OCR or image-search plug-in on this rig, so nothing can be located at runtime.
- IMConsole.exe running and logged in; with only IMService.exe up, both phones showed state 0 and an empty board list.
- iOS 15+ as the safe floor for the file/album functions; these XRs run 18.7.10.

WATCH OUT

- Every /shortcut/* call in this chain is untested on this rig. Probe first with `GET /api/shortcut/device/ip?id=<id>&state=0` : code 0 echoing the phone's own IP proves the Shortcut is installed, hotkey-bound and reachable; a code 30 timeout after ~15 s means the hotkey never launched it.
- The Phonefarm doc marks the entire shortcut dispatcher "professional version only." XP gates no endpoint by edition and licenses per device instead, so whether this rig's license covers shortcut calls is unresolved until that probe runs.
- No runtime vision: without the OCR and image-search plug-ins, the flow is entirely open-loop on hard-coded coordinates, and a layout change in an app update silently taps the wrong thing.
- Picker tile order ("top-left is newest") is an app-specific claim, not an API guarantee, and it differs between the system picker and an in-app album grid; measure it per app before wiring it in.
- /shortcut/album/upload gives no progress signal, and the slow XR here delivers only ~4.5 new frames/s versus ~29 on its twin, so any screen-verification during the picker must not assume 30 fps.
- /mouse/click can silently stop working while moves and hardware keys still succeed; only /mouse/reset clears it, and skipping it is the likeliest cause of a run that reports success and does nothing.
- A shared coordinate is only safe across phones with identical imgw/imgh AND an identical app layout; the two XRs here have different home screens.
- outtime is milliseconds on XP (default 15000) and seconds on Phonefarm; a value copied across editions is wrong by 1000x, and a large avatar over Wi-Fi can exceed the 15 s default.
- The album name iOS assigns is not the PC filename; delete by the name+ext the upload response returns, never by the original filename.
- Saving the profile is an irreversible, published account change; gate the Save tap behind an explicit go-ahead rather than fanning it across the rack unattended.
- `/shortcut/album/clear` wipes an entire album on every listed phone with no undo; never wire it into automated cleanup — the targeted /shortcut/album/del does the job.
- A raw-BMP diff only proves the cropped region's pixels changed, not that the correct avatar was applied and saved; compare against a known-good post-save reference captured once by hand, not just an idle-floor threshold.

- AssistiveTouch emulates one simulated touch point per phone; a picker or crop sheet that needs a gesture beyond a plain tap (drag-to-resize) will not be satisfied by a coalesced click, and needs per-iOS-version validation.
- The API is unauthenticated on the LAN; firewall 9911 and 17000 to trusted hosts. Only use this against accounts the operator owns and is authorized to administer.
- Images carry EXIF and container metadata that travels with the upload, and the same file uploaded to multiple accounts is, factually, a linkage signal platforms act on.
- Automating profile edits with hardware input sits outside most platforms' automation terms even on owned accounts, and a profile edit is a visible account-level change that can trigger re-review; if the app surfaces a rate-limit or "action blocked" interstitial, stop that phone's run rather than retry.

SCALE

The media and text steps fan out cheaply: /shortcut/album/upload, /shortcut/album/del, /shortcut/clipboard/set and every /mouse and /key call take a comma id list, so mirroring a tap to N phones costs the same wall time as one. The per-phone cost is the shortcut round trip itself (outtime default 15000 ms; budget 30-60 s for a real image over Wi-Fi) plus verification, which is single-id but cheap and independent per phone (measured up to 512 req/s across ten pollers). A shared avatar and bio across a same-layout group costs roughly a minute per batch, dominated by the two shortcut round trips; unique avatars per phone force the upload into a per-phone loop, at which point plan on roughly 40-70 unique profile completions per hour per PC run serially, or 2-4x that pipelined.

Fleet app version management

TestFlight and App Store install/update rollout across the rack

NEEDS SHORTCUT WITH CAVEATS

Fan a TestFlight join link or an App Store product URL out to every bound phone in one `/shortcut/exec/url` call, tap Accept/Install/Get/Update at a measured coordinate, then poll a cropped screenshot band over the button until the label changes to Open. Version state can be carried in each device's custom name via `/device/set`, turning "which phones are stale" from a manual audit into a query. As written this is really an update or relaunch flow for phones that already have TestFlight and an Apple ID configured, not an install-from-scratch flow: there is no App Store install endpoint, and account sign-in cannot be scripted.

HOW

1. `get_device_list` XP `GET /api/device/get` — build the rollout id list from devices with state != 0, a bound board and imgw/imgh identical to the reference phone; cross-check /config/usb/get for board state 2.
2. `send_key (fn_key WIN+h)` XP `GET /api/key/sendkey?id=<a,b,c>&fn_key=WIN+h` (twice) — normalize every phone to a known screen.
3. `mouse_reset_pos` XP `GET /api/mouse/reset?id=<a,b,c>` — clear the stuck-click state before any install tap sequence.
4. `shortcut#13 (Open URL)` XP `GET /api/shortcut/exec/url?id=<a,b,c>&url=https://testflight.apple.com/join/XXXXXXXX&outtime=20000` — open the TestFlight or App Store URL on every phone at once.
5. `send_key + click` XP `/api/mouse/click, then /api/key/sendkey?key=<url>, then fn_key=ENTER` — fallback when the Shortcut is not installed: type the URL into Safari (per-phone icon coordinate).

6. `click` XP `GET /api/mouse/click?id=<a,b,c>&x=&y=&button=1` — tap Accept / Install / Get / Update; button is numeric on XP, coordinates are mirror pixels (500x1080), not iOS logical points.
7. `get_device_screenshot` XP `GET /api/pic/screenshot?id=<one id>&binary=true&rect=[l,t,r,b]` — poll a cropped band over the button until it reads Open; single-id only.
8. `ocr / find_image` XP `GET /api/pic/find-text?id=<id>&text=["Open","Update","Get"]` — not available on this rig (code 28); would read the button label server-side once the plug-in exists.
9. `set_dev` XP `GET /api/device/set?id=<one id>&name=Slot07-v3.2` — record the installed version per device, persisted in DeviceList.json and broadcast as a dev_change callback.
10. `callbacks (dev_connect / dev_disconnect / error_push)` XP `ws://127.0.0.1:9911/api` — watch the run asynchronously for a phone that lost its mirror mid-install.

NEEDS

- IMConsole.exe running and logged in; with only IMService.exe up, the live probe showed both phones at state 0 and an empty board list.
- Per phone: board bound at /config/usb/get state 2, location_crc non-empty.
- iOS settings invisible to the API: AssistiveTouch on at max sensitivity, Full Keyboard Access on, Trackpad & Mouse speed max, Auto-Lock Never, portrait lock.
- iMouse Shortcut installed, IP-edited and hotkey-bound on each phone for /shortcut/exec/url to fire; phone reachable to the kernel on 9911.
- TestFlight already installed and the Apple ID already signed in on each phone; neither is achievable through this API.
- Apple ID purchase/download authentication settings (Face ID for downloads, password prompts) configured by hand; the board has no path to satisfy a biometric prompt.
- Button coordinates measured once per iOS build and app, plus a stored crop of the settled "Open" state.
- A client-side comparator on raw BMP frames, since the OCR plug-in is not connected (code 28) — a hard prerequisite for verification, not an optimization.

WATCH OUT

- Nothing in this workflow is a vendor feature; TestFlight and the App Store appear nowhere in either API doc, so this is blind coordinate tapping with a screenshot diff as the only feedback, and any iOS or App Store UI revision breaks it silently.
- Only /shortcut/exec/url, /key/sendkey, /mouse/click and /mouse/reset fan out on one request; /pic/screenshot is documented "Multiple is not allowed," so confirmation is strictly per phone.
- A shared coordinate is only safe when every listed phone shows the same layout at the same imgw/imgh; the two XRs here have different home screens, so the Safari-fallback path in particular cannot be mirrored.
- Credential entry is out of scope by design: if the App Store asks for the Apple ID password, or a Face ID/passcode prompt appears, stop the run, flag the device, and hand it to a person.
- /shortcut/exec/url outtime is milliseconds on XP (default 15000) but seconds on the Phonefarm sibling; copying 15 across editions yields a 15 ms timeout.
- A code 30 "call timeout" from any /shortcut/* call usually means the hotkey never launched the Shortcut (check the AssistiveTouch binding and that the phone is unlocked), not that the URL failed.
- Install progress is invisible for long stretches on the slow phone (~4.5 new frames/s versus ~29 on its twin); do not treat frame scarcity as a stalled install.
- The device record's custom name is a convenient version tag but unvalidated free text with no schema; it is a label, not a source of truth for what is actually installed.

- A phone that loses Wi-Fi mid-install goes state 0 and disappears from the mirror while the install continues over cellular; USB control survives, so the run is blind rather than stopped.
- A cropped-button pixel diff cannot distinguish "install finished, button now reads Open" from a different system sheet or crash dialog occupying the same region; corroborate with a secondary check, such as relaunching the app via Spotlight, rather than trusting the button region alone.
- Apple enforces a device-association limit and a cooldown on associating new devices with one Apple ID, and TestFlight builds expire 90 days after upload with capped public tester slots; a 19-phone rack under one Apple ID hits that account-side wall well before any rig-side limit — a hard scaling ceiling, not just a background risk.
- The API is unauthenticated; firewall 9911 and 17000 to trusted hosts, and never print or copy the vendor password stored as reversible hex in `C:\iMouseXP\IMService.json`.
- An interrupted install leaves a phone in an unknown UI state; always re-normalize with two Home presses plus /mouse/reset at the start of each run rather than assuming where the previous run left off.
- Scripted redemption and installation across many devices likely sits outside what Apple's App Store and TestFlight terms contemplate; that legal exposure is the operator's to assess for accounts and builds they own.

SCALE

Link fan-out, taps and resets are one request regardless of fleet size, since /shortcut/exec/url, /key/sendkey, /mouse/click and /mouse/reset all take comma id lists and the boards walk in parallel; even a 500 px reposition costs about 300 ms. Confirmation is the only per-phone cost, at p50 2 ms and roughly 84 req/s per phone independently (ten pollers measured 512 req/s aggregate), so a 0.5 Hz poll across 19 phones is a small fraction of measured capacity. The real ceiling is external: per-phone Wi-Fi bandwidth and Apple's own servers, plus Apple ID device-association and tester-slot limits, not the rig. Realistic throughput is on the order of tens of phones per hour per PC across several legitimately owned Apple IDs on a reasonable uplink; an API-requests-per-second figure would misrepresent the actual bottleneck here.

Session hygiene

Lock between sessions, unlock by typed passcode

PARTIAL WITH CAVEATS

Put each phone on its lock screen between account sessions with the documented hotkey TAB+l, so no logged-in feed sits visible on the mirror wall or in a captured frame, then wake it, raise the passcode pad with a swipe, type the passcode, and confirm with a screenshot diff. The lock leg is fully native and needs no plug-in or Shortcut; the unlock leg depends on undocumented iOS behavior, whether mirroring survives a lock, whether the USB accessory path stays live while locked, whether the passcode field accepts HID digits, that must be proven once per phone model before this is trusted. It also runs opposite to the vendor's own documented rack setting of Auto-Lock Never.

HOW

1. `get_usb_list + get_device_list` XP `GET /api/config/usb/get ; GET /api/device/get?id=<id>` — confirm control-ready (board state 2, bound, non-empty location_crc) and video-ready (state != 0, live air_pid); read imgw/imgh.
2. `get_device_screenshot` XP `GET /api/pic/screenshot?id=<id>&binary=true` — capture a pre-lock reference frame and a small crop that differs between lock screen and app content.

3. `send_key (fn_key TAB+l)` XP `GET /api/key/sendkey?id=<id1,id2>&fn_key=TAB+l` — lock the whole target set in one request; TAB+l is documented on XP and has no Phonefarm counterpart.
4. `get_device_screenshot` XP `GET /api/pic/screenshot?id=<id>&binary=true` — confirm the lock landed via mean-absolute-pixel-diff against the reference, budgeting for the phone's own frame rate (never MD5-compare frames).
5. `send_key (fn_key WIN+h)` XP `GET /api/key/sendkey?id=<id>&fn_key=WIN+h` — wake the phone; whether this wakes a locked screen is unverified, fall back to a tap or fn_key=ControlBar.
6. `mouse_reset_pos then swipe` XP `GET /api/mouse/reset?id=<id> ; GET /api/mouse/swipe?id=<id>&direction=up&sx=250&sy=1000&ex=250&ey=500` — clear the stuck-click state, then raise the passcode pad with an explicit-coordinate swipe (single id).
7. `send_key (key)` XP `POST /api/key/sendkey {"id":"<id>","key":"<digits>"}` — type the passcode for that one phone; follow with fn_key=ENTER only for a custom alphanumeric passcode. Never comma-list this call.
8. `get_device_screenshot + send_key` XP `GET /api/pic/screenshot?id=<id>&binary=true ; GET /api/key/sendkey?id=<ids>&fn_key=WIN+h` (twice) — verify the unlock against a home-screen reference, then normalize before any coordinate-based work.
9. `n/a (client-side policy)` XP `n/a` — on failure, stop; do not retry the passcode. Raise it to a human and leave the phone locked.

NEEDS

- Phones the operator owns and administers, with passcodes held legitimately.
- Full Keyboard Access on with 1 s auto-hide; TAB+l, WIN+h and WIN+v are all HID hotkeys and are inert without it.
- AssistiveTouch on at max sensitivity, Trackpad & Mouse speed max; none of this is queryable over the API.
- Board bound and physically connected: /config/usb/get state 2 for that board and vid/pid/uid set together on the device record.
- location_crc non-empty (B0DBD800 shared-library profile on this rig), or every mouse call returns code 11.
- Mirroring live for verification: /device/get state != 0 with a live air_pid, and IMConsole.exe running and logged in.
- A one-time manual acceptance test per model and iOS build on an expendable phone: does mirroring survive TAB+l, does the swipe raise the passcode pad, does the pad accept typed digits, does it need ENTER.
- Per-phone stored reference crops for lock screen and home screen; the two XRs have different wallpapers, so references never transfer between phones.
- Explicit acceptance that this contradicts the vendor's own documented rack prerequisite of Auto-Lock Never.

WATCH OUT

- The observation channel may vanish exactly when needed: no reference file says whether AirPlay mirroring survives a lock. If it drops outright, /device/get goes state 0 and /pic/screenshot returns code 25. A more likely outcome is that mirroring stays up but every frame is frozen or blanked (iOS's own content-privacy behavior on a locked device), so a flat pixel diff after TAB+l is not proof the lock command failed, only that content may be suppressed.
- iOS can block Lightning/USB-C accessory data after a device has been locked for a period (Settings > Face ID & Passcode > Accessories, off by default). If that fires, the board cannot deliver the wake, swipe or digits and the phone needs hands on the rack; the exposure window is small for a between-job lock (minutes) but not zero for an overnight one.

- Never use this as a restart-recovery plan: a rebooted phone is in a before-first-unlock state that refuses accessory input until a human unlocks it. Do not combine it with /device/restart or a switch/device-state call.
- Face ID will attempt and fail in a rack with no face presented, which is what drops to the passcode pad, but a failed attempt still counts toward iOS's own retry accounting.
- No OCR on this rig (code 28 confirmed live); every check is a mean-pixel-diff against a stored crop, not a read of "Enter Passcode" or "iPhone Unavailable."
- A passcode sent as a GET query parameter is a plaintext secret on an unauthenticated LAN; use the POST JSON form, keep it out of logs, and firewall 9911 to trusted hosts regardless of verb.
- /mouse/reset and /mouse/swipe both assume the control channel survived the lock; if accessory access is blocked, /mouse/reset itself silently no-ops, so check for a state change after every step, not only after the final digit entry, or three no-ops in a row look identical to three successes.
- The claim that a standard 6-digit passcode submits without ENTER is unverified over a HID/AssistiveTouch path and should carry the same unverified flag as the rig's other unproven iOS behaviors, not read as settled.
- The single most destructive failure mode is a wrong-passcode retry loop: iOS escalates lockout delays on consecutive failures and can render a device unusable or erased depending on configuration. Hard-code one attempt, then escalate to a human; never retry.
- Never lock a phone you cannot reach to unlock; if any of the open unknowns resolves unfavorably for a given phone, that phone needs a human touch for that cycle, not an API retry.
- This is administration of devices and passcodes the operator already owns and holds; it carries no anti-abuse-evasion content, but reference crops and screenshots are real account-screen images and should be stored like the account data they are.
- The setup deviates from the vendor's documented Auto-Lock Never prerequisite, so any misbehavior traced to a locked phone falls outside supported configuration.

SCALE

Locking is effectively free: /key/sendkey with a comma id list locks the entire rack in one request with the board walking phones in parallel. Unlocking does not scale the same way, since /pic/screenshot is single-id and a passcode must be typed per phone and never comma-listed, but concurrency across phones is cheap (measured up to 512 req/s across ten independent pollers), so N unlock workers can run in parallel. Per-phone wall time is roughly 1-4 s of API-visible work on a healthy link (about 300 ms of pointer travel for the swipe plus one or more frame waits, worse on the slow XR at ~220 ms per frame), but the real throughput driver is the failure rate on the three unproven legs: any phone where mirroring, accessory access or passcode-field acceptance resolves unfavorably falls out of automation entirely for that cycle rather than costing extra time.

Navigation infrastructure

Layout-independent app launch through Spotlight typing

PARTIAL WITH CAVEATS

The rack's phones have different home screens and wallpapers, so a shared icon coordinate is not a fair multi-id target. Spotlight is layout-invariant: a mid-screen pull-down opens search on every iPhone, the search field accepts hardware-keyboard text, and ENTER opens the top hit, so one fan-out sequence launches the same app

on every bound phone by name rather than by pixel. This turns "open the app," the first step of nearly every account job, from N per-phone coordinate lookups into a single request per step; what remains unsolved is confirming which app actually opened, since there is no OCR on this rig.

HOW

1. `get_device_list` XP `GET /api/device/get (optionally ?id=a,b)` — read imgw/imgh, state, vid/pid and location_crc; drop any phone whose mirror geometry differs from the master.
2. `mouse_reset_pos` XP `GET /api/mouse/reset?id=<a,b,c>` — clear the stuck-click state on the whole target set before anything tap-critical.
3. `send_key (fn_key WIN+h)` XP `GET /api/key/sendkey?id=<a,b,c>&fn_key=WIN+h` (twice) — normalize every phone to home page one.
4. `swipe` XP `GET /api/mouse/swipe?id=<list>&direction=down&sx=250&sy=400&ex=250&ey=600` — open Spotlight with a short mid-screen pull-down, not from the top edge (Notification Center).
5. `send_key (key)` XP `GET /api/key/sendkey?id=<list>&key=instagram` — type the app name into the focused search field (ASCII only).
6. `send_key (fn_key)` XP `GET /api/key/sendkey?id=<list>&fn_key=ENTER` — wait roughly 1 s for the result list to settle, then open the top hit.
7. `click` XP `GET /api/mouse/click?id=<list>&x=&y=` — fallback when the top hit is a web or App Store suggestion: tap the first result row at its fixed, layout-independent coordinate.
8. `get_device_screenshot` XP `GET /api/pic/screenshot?id=<single id>&binary=true` — verify per phone by mean-absolute-pixel-diff against the pre-launch frame; single-id only.
9. `ocr / find_image` XP `GET /api/pic/find-text?id=<single>&text=["Instagram"]&contain=true&similarity=0.8 or /api/pic/ocr` — blocked on this rig (code 28); would confirm which app opened and pre-check the top hit before ENTER.
10. `shortcut#13 (open URL)` XP `GET /api/shortcut/exec/url?id=<list>&url=<scheme://...>&outtime=<ms>` — deterministic alternative worth benchmarking: deep-link the app instead of searching for it.

NEEDS

- Full Keyboard Access on with 1 s auto-hide; not queryable over the API.
- AssistiveTouch on at max sensitivity, and Trackpad & Mouse speed max; the menu only appears while the board is cabled, itself a diagnostic that control is alive.
- Each phone bound to its board (/device/set with vid+pid together) at /config/usb/get state 2.
- location_crc non-empty on every target, or /mouse/* returns code 11.
- Every phone in the id list sharing the same imgw/imgh as the master (500x1080 here); a different-sized model needs its own coordinate set.
- Portrait lock on, Auto-Lock Never, StandBy off.
- Siri Suggestions in Search turned off per phone, once, by hand, so the top hit is deterministic rather than usage-ranked; invisible to the API.
- IMConsole.exe running and logged in; with only IMService.exe up, both phones reported state 0 and an empty board list.
- For non-ASCII app names, and for the /shortcut/exec/url alternative: iMouse Shortcut installed, IP-edited, run once with all prompts allowed, and hotkey-bound; not verified present on this rig.
- For confirming which app opened: the OCR plug-in installed and connected, plus the image-search plug-in if used; currently absent, /pic/ocr returns code 28.

WATCH OUT

- The launch half is native and fully multi-id; the verification half is not. Without OCR you get "the screen changed by N pixel units," not "Instagram is open."
- Whether a board-driven pointer drag actually triggers iOS's Spotlight pull-down gesture is unverified on this rig. It is more likely to work than an even-odds guess, since the home-screen reveal is a distance-threshold gesture rather than the velocity-gated edge-swipe app switcher, but that raises confidence without removing the need to verify once against real hardware, since AssistiveTouch's synthesized touch path is not guaranteed identical to a finger's.
- Do not pass len together with explicit sx/sy/ex/ey on /mouse/swipe; the doc only auto-derives coordinates from the percentage when they are 0. The XP swipe table also mislabels sx/ex as "y coordinate," confirmed a copy-paste error; they are start/end x,y.
- /key/sendkey key is ASCII only in both editions; an app whose Spotlight-searchable name is not Latin cannot be typed this way and needs the clipboard+paste route instead.
- Spotlight ranking is suggestion-driven per phone; until Siri Suggestions in Search is off everywhere, ENTER can open a web result or an App Store card instead of the app, and without OCR you will not know which happened.
- /mouse/reset must run first: a phone in the stuck-click state still accepts every typed step and hotkey, so the sequence looks healthy right up to the fallback tap, which silently does nothing.
- Do not use fn_key=Home in place of WIN+h here; Home needs kernel 2026.2.27+ and targets the iOS 26.1+ fix, while these phones run 18.7.10.
- Timing is undocumented at three points, Spotlight's open animation, typing rate and query latency; the slow XR cannot be observed faster than ~220 ms per frame, so no timeout should assume 30 fps.
- /shortcut/exec/url is more deterministic than Spotlight ranking, but the Phonefarm sibling marks the whole shortcut dispatcher "professional version only" and the Shortcut install is unverified here; prove it with /shortcut/device/ip?state=0 before designing around it.
- Full Keyboard Access's on-screen affordances could race with the injected keystrokes; if the phone briefly shows its own predictive-text bar, the typed string could be autocorrected before ENTER, with no server-side way to detect this without OCR.
- A truncated search string, the field losing focus mid-stream, could still return a plausible top hit and open the wrong app silently; there is no way to detect a partial type without full-screen OCR.
- This is ordinary device automation on phones the operator owns and controls; it does not touch any platform's anti-abuse surface. The real risks are local: an open-loop launch can leave a phone sitting in Spotlight, Notification Center or Safari while later steps run against the wrong screen, so gate the next step on a frame diff; the stuck-click state makes a broken phone look healthy, so distinguish control-ready from video-ready rather than inferring either from "the app appeared to open"; and the API has no authentication at all, so firewall 9911 and 17000 to trusted hosts.

SCALE

Steps that move the pointer or type are one request for the entire rack regardless of size, since /mouse/reset, /key/sendkey, /mouse/swipe and /mouse/click all take comma id lists and the boards walk in parallel; a 200 px pull-down costs about 120 ms whether it drives 2 phones or 19. Verification is the only per-phone cost, at p50 2 ms for a raw BMP frame with pollers measured fully independent of the input stream (up to 512 req/s across ten pollers). The real ceiling is the phone's own AirPlay source rate, not the kernel: about 29 new frames/s on one XR here versus 4.5 on its identical twin. The full normalize-search-launch cycle is roughly 2-4 seconds of

pointer/API time for the entire bound id list, implying on the order of 900-1500 launch cycles per hour run back-to-back, with that aggregate figure essentially unchanged whether it targets 1 phone or 19, since fan-out cost does not scale with device count.

SMS and OTP retrieval

OTP and SMS relay by cropping the Messages or Notification Center screen for an external recogniser

PLUS EXTERNAL TOOLING · WITH CAVEATS

iOS hands SMS content to no API, and this rig has no server-side OCR, so the mirror frame is the only extraction surface: drive the phone to the message, crop the bubble or banner, run OCR client-side on the raw bitmap, and pattern-match the code out. This closes the one gap the API cannot close by design: Phonefarm's shortcut id 12 "Send SMS" is struck through in the vendor's own table, and XP has no SMS function in either direction. Scope this to SIMs and accounts you own, such as signing in to your own services, receiving your own 2FA codes across the rack, or QA-ing your own SMS delivery and message rendering.

HOW

1. `get_device_list` XP `GET /api/device/get?id=<deviceid>` — confirm the phone is live (state non-0, air_pid set) and read imgw/imgh, 500 x 1080 on these XRs, for every rect and tap that follows.
2. `mouse_reset_pos` XP `GET /api/mouse/reset?id=<id>` — clear a stuck-click state before any tap-critical sequence; measured on this rig, a tap can otherwise silently do nothing (0.09 mean pixel diff) while moves and hardware keys keep working.
3. `none` XP `GET /api/key/sendkey?id=<id>&fn_key=NoticeBar` — preferred, layout-independent route: open Notification Center by hotkey rather than hunting for the Messages icon, since the two XRs have different home screens and wallpapers.
4. `send_key WIN+h x2, then click` XP `GET /api/key/sendkey?id=<id>&fn_key=WIN+h (twice), then GET /api/mouse/click?id=<id>&x=<x>&y=<y>&button=1` — fallback route: two Home presses normalize to page one, then tap the Messages icon at coordinates mapped by hand for that specific phone. No OCR or image search is available to find the icon automatically, so this map is per device.
5. `get_device_screenshot` XP `GET /api/pic/screenshot?id=<id>&binary=true&rect=[l,t,r,b]` — crop the frame to the bubble or banner. rect is a flat [Left, Up, Right, Down] array in XP, not Phonefarm's four corner points. binary=true returns raw bottom-up BMP bytes with no JSON envelope, 2 ms for a full frame.
6. `ocr / ocr_ex` XP `none usable — /pic/ocr, /pic/ocr-ex, /pic/find-text and /pic/find-text-ex all return data.code 28` — recognition has to run client-side on the decoded bitmap; the server plug-in is not connected on this rig, and there is no verified fallback.
7. `none` XP `none` — pattern-match the expected code shape (length, charset) rather than grabbing any digits, and reject a low-confidence or wrong-shaped read instead of guessing.
8. `send_text / shortcut#10 + paste` XP `POST /api/shortcut/clipboard/set {id, text, sleep}, then GET /api/key/sendkey?id=<id>&fn_key=WIN+v` — optional: hand the code back into a field on a device you own. id takes a comma list. ASCII-only codes can skip the clipboard entirely and go straight in with `/api/key/sendkey?key=<digits>`.
9. `get_device_screenshot` XP `GET /api/pic/screenshot?id=<id>&binary=true` — confirm the new screen arrived by thresholding mean absolute pixel difference above roughly 1 (idle floor 0.02-0.03, a real reaction

20-160); never MD5-compare, since an untouched mirrored phone never repeats a frame.

NEEDS

- An external OCR engine running client-side on the decoded bitmap; the server plug-in answers data.code 28, and the /shortcut/other/ocr fallback is undocumented beyond its path name and needs iMouse Shortcut v1.3+
- A SIM in the phone receiving the SMS, and ownership of both the SIM and the account the code belongs to
- Mirroring live for the read, and a bound working board for the navigation taps
- A per-phone coordinate map for the Messages icon if using the hand-mapped route; none is shared across the two XRs
- Notification previews enabled on that phone (Settings > Notifications > Messages > Show Previews set to "Always") if using the NoticeBar route; the API cannot read that setting, and "When Unlocked" or "Never" leaves NoticeBar blank
- iMouse Shortcut installed and bound only if relaying via the clipboard; typing an ASCII code with /key/sendkey needs no Shortcut

WATCH OUT

- Every step except recognition is documented XP, already verified elsewhere on this rig; only OCR is off-rig, and there is no server-side fallback while the plug-in is missing, so "achievable" rests on an unverified external component with no measured accuracy or error rate
- rect on /pic/screenshot is written with literal brackets, [l,t,r,b], unlike every other multi-value XP parameter in this doc set which uses bare comma-separation; confirm the kernel actually parses it rather than silently returning the full frame, which would still work for OCR but destroy the throughput benefit of cropping
- rect combined with binary=true is documented as two independent flags but was not probed on this rig; check the returned byte count against the expected crop size before trusting it
- Fully per-device on the read side: /pic/screenshot is single-id, so navigation can fan out over a comma id list but every screenshot is one phone at a time
- Small text on a 500 x 1080 mirror is the weak link; crop tightly and treat a low-confidence read as a failure rather than a guess
- Sending SMS is not available in any form, Phonefarm shortcut id 12 is struck through and XP has no SMS endpoint at all, so this chain is read-only
- Codes expire in seconds while the slower XR yields only ~4.5 new frames/s; budget navigation and settle time, and gate the wait on a real change-detection threshold rather than a fixed sleep, or the frame you OCR was captured before the banner rendered
- Automating login or OTP retrieval through the UI, rather than an official API, is a pattern many services' fraud and bot-detection systems flag regardless of whether the account is self-owned; repeated automated app opens plus screen scraping around auth flows is a real operational cost to weigh, not just a legal footnote
- Some banking and 2FA apps suppress lock-screen and notification-center previews for security-sensitive codes by design; when that is the case, NoticeBar returns nothing and the hand-mapped Messages-app fallback, with its own per-device coordinate map, is mandatory rather than optional
- Reading SMS or relaying codes for a SIM or account you do not own, or without the holder's consent, breaks the terms of service of essentially every service that sends them and may be unlawful depending on jurisdiction; confine this to your own devices, SIMs and accounts

- Codes are short-lived secrets: do not write them to logs, on-disk screenshots or a shared queue, and delete the cropped frames after use
- The API is unauthenticated on port 9911; anything on the LAN can drive these phones and pull the same frames, so firewall it
- Crops of a Messages screen also capture unrelated content from real correspondents, a privacy decision to make before turning the capture on, not after

SCALE

Strictly serial on the read side: /pic/screenshot is single-device only, and recognition is per-frame and per-phone. Navigation is cheap and fans out, since /key/sendkey and /mouse/click take comma-separated id lists and a board walks every listed phone in parallel, but the sequence is dominated by pointer travel (about 0.6 ms/px) plus the settle wait for a fresh frame, roughly 35 ms on the faster XR and 220 ms on the slower one. End to end that is a few seconds per code on one phone. For a single OTP, external wait (SMS or carrier delivery, typically 3-30 s) dominates over the API calls themselves, and the realistic ceiling is tens of retrievals per hour per phone, set by upstream resend cooldowns rather than rig throughput. Screenshot polling itself scales linearly per phone (84 req/s/phone measured), so across the rack aggregate throughput tracks phone count up to the number of accounts and SIMs available; the API is not the bottleneck anywhere in this chain.

Commerce, customer and business operations

This is where the rack earns its keep on a resale or storefront operation: listing, restocking, price sweeps, support replies, live-shopping control and pre-launch verification. Every one of these builds on the same three primitives from Section 3 — `/mouse/*`, `/key/sendkey` and `/pic/screenshot` — plus, for anything that needs Unicode text or file transfer, the iMouse Shortcut app installed and hotkey-bound on each phone. Two facts shape almost every card below: the OCR and image-search plug-ins are not connected on this rig (every call returns `data.code 28`), so verification is client-side pixel comparison rather than reading text off the screen; and none of this makes a platform's anti-automation detection or its terms of service go away, so each card states the risk plainly and stops there.

Listing and catalog operations

Photo-to-listing publisher for mobile-only seller apps

NEEDS SHORTCUT · WITH CAVEATS

Turns a folder of edited photos plus a CSV of titles, descriptions and prices into draft or published listings inside apps that expose no listing API to ordinary sellers — Depop, Poshmark, Vinted, Etsy Seller and similar. The PC pushes photos into the phone's album, opens the app, taps through the Sell flow, pastes title and description, types the price, taps Publish and checks the result, all in mirror-pixel coordinates. This is the single largest labour cost in a resale operation, and it is the most demanding card in this section to stand up.

HOW

1. `get_device_list` XP `/device/get` — read `imgw` / `imgh` per phone (500x1080 on this rig); every coordinate below is in that space, not the 414x896 logical size.
2. `mouse_reset_pos` XP `/mouse/reset?id=a,b,c` — clear the silent stuck-click state before any tap-critical run; `/mouse/up` does not clear it.
3. `send_key` XP `/key/sendkey?id=a,b,c&fn_key=WIN+h` (twice) — normalise every phone to home page one.
4. `shortcut#7` XP `/shortcut/album/upload?id=<ids>&files=[...]&zip=1&outtime=60000` — push this SKU's photos; the response's newest-10 album records are the documented proof they landed.
5. `shortcut#13` XP `/shortcut/exec/url?id=<ids>&url=<scheme>` — open the app at its Sell screen, if the app publishes a URL scheme; otherwise tap its icon.
6. `click` XP `/mouse/click?id=<ids>&x=&y=` — tap Sell/+, select album cells, tap Done. Do not assume the newest photos are the first grid cells; confirm from the `/shortcut/album/upload` response before proceeding.
7. `send_text` XP `/shortcut/clipboard/set?id=<ids>&text=<title>` then XP `/key/sendkey?fn_key=WIN+v` — paste Unicode-safe title and description; XP has no async Chinese-text path, so clipboard+paste is the only route.
8. `send_key` XP `/key/sendkey?id=<ids>&key=1999` — type the ASCII price directly, after `fn_key=WIN+a` to select the pre-filled default.
9. `swipe` XP `/mouse/swipe?id=<ids>&direction=up&len=0.5` — scroll to category, size and condition pickers.
10. `get_device_screenshot` XP `/pic/screenshot?id=<one id>&binary=true` — tap Publish, then compare a crop against a golden crop by mean absolute pixel difference (idle floor 0.02–0.03, a real reaction 20–160).

Single id only, so one request per phone.

11. `shortcut#3` XP `/shortcut/album/del?id=<ids>&list=[...]` — remove the pushed photos so the next SKU's are again the newest. Irreversible once granted.

NEEDS

- iMouse Shortcut installed on every phone, IP-configured, run once with all permissions, and bound to an AssistiveTouch custom key via the console's Send Binding Key — a one-time per-phone setup, not something a comma-separated id list gives you for free.
- Board bound and connected (`/config/usb/get` state 2, non-empty vid/pid/uid) and a live control channel, since the Shortcut is launched by a HID hotkey through the board.
- Phone-side accessibility settings invisible to the API: AssistiveTouch on with tracking sensitivity max, Full Keyboard Access on, Trackpad & Mouse tracking speed max, Auto-Lock never, portrait lock.
- Seller app already logged in, on a version whose coordinate map you have captured from a raw BMP — per phone, since the two XRs on this rig have different home screens.
- IMConsole.exe running and the kernel licensed; with only IMService.exe up, the board list comes back empty.

WATCH OUT

- A mean-pixel-diff proves the screen changed, not that the listing published: a validation-error toast changes the screen by the same 20–160 magnitude as a real publish. With OCR unavailable, there is no way to read a "Listed successfully" string, so treat pixel-diff as "detects that something happened" and have a human review a sampled subset of listings before trusting a batch.
- Coordinate maps are captured per app version *and* per phone layout — the two XRs already differ, so the tap sequence must be authored and dry-run twice, not written once and mirrored across the fleet.
- App URL schemes are undocumented by the vendor and change between app versions; budget a tap-the-icon fallback.
- Full Keyboard Access must be on for the price keystrokes to register; there is no API signal if it is off, so a mis-configured phone silently leaves the price at its prefilled default while the console reports success for every keystroke.
- Automating listing creation on accounts you do not own or are not permitted to operate, or in a way built to evade a platform's anti-automation detection, is a terms violation; this card is scoped to owned, permitted accounts and includes no pacing or human-mimicry logic by design.

> TRAP **Pixel-diff is not proof of a correct publish**
>
> Because the change-detection threshold (20–160) does not distinguish "listed" from "rejected with an error banner," never let a passing diff alone trigger `/shortcut/album/del` . Delete only after a sampled human check, at least until the OCR plug-in is installed and can read the confirmation text.

SCALE

About 25 taps, 2–3 clipboard pastes and one album upload per listing, dominated by the board's ~0.6 ms/px pointer walk plus app settle time: 60–120 s per listing per phone is realistic. Content is per-phone (unique photos and text per listing), so the comma-separated id fan-out does not help the tap sequence itself — only the shared normalisation steps (`/mouse/reset` , WIN+h) batch. Phones are genuinely independent (measured 98.8

req/s while streaming moves and polling versus 78.7 idle), so N phones run N listings concurrently: roughly 30–60 listings/hour per phone, times phones in the rack, with a human sampling published listings as the real ceiling rather than the API.

Inventory photo round-trip: harvest camera roll, edit on PC, push back

NEEDS SHORTCUT

WITH CAVEATS

Staff photograph stock on any iPhone, dock it in a rack slot, and the PC pulls the new photos off with `/shortcut/album/get` plus `/shortcut/album/down`, runs background removal and colour correction externally, then pushes the finished set back with `/shortcut/album/upload` — to that same phone or fanned out to every seller phone in one request. The rack becomes the sync layer between camera roll and editing queue, replacing AirDrop-by-hand.

HOW

1. `dev_connect` callback or `get_device_list` XP `WS /api - dev_connect; or poll /device/get` — notice a freshly docked phone. A non-zero `state` only proves the video channel; control readiness is a separate check via `/config/usb/get`.
2. `shortcut#1` XP `/shortcut/album/get?id=<one id>&album_name=&num=200` — list the camera roll. Single id only, and XP has no date filter (unlike Phonefarm), so track "already seen" on the PC by name and create_time.
3. `shortcut#2` XP `/shortcut/album/down?id=<one id>&list=[...]&zip=1&outtime=120000` — pull new photos to the PC. Files land in `<install>\Shortcut\Media`; the response carries no file list, so poll that folder for arrivals.
4. External tooling — edit on the PC. The kernel neither transforms nor inspects images.
5. `shortcut#7` XP `/shortcut/album/upload?id=<a,b,c>&files=[...]&album_name=Ready to list&zip=1` — push the finished set back, to one phone or many; id is comma-separated here, so one request fans the set to every seller phone.
6. `shortcut#3` / `shortcut#5` XP `/shortcut/album/del or /shortcut/album/clear` — clean up originals once the PC copy is verified. Both irreversible once granted.
7. `get_device_screenshot` XP `/pic/screenshot?id=<one id>&binary=true` — optional, weak spot-check only; the upload response's newest-album list is the real proof of arrival.

NEEDS

- iMouse Shortcut installed, IP-edited, run once with all permissions and hotkey-bound on every phone that ever docks — including ad-hoc camera phones, which is the real friction here.
- Shortcuts > Advanced set to allow sharing large amounts of data and deleting without confirmation.
- Both channels up per docked phone: cabled to a bound board for the hotkey trigger, and on rack Wi-Fi for the byte transfer; airplane mode kills the transfer while leaving USB control alive.
- A stable slot-to-role map (`/device/group/set`, `/device/set gid+name`, `/device/group/get-dev`) so the PC knows which uid is a camera drop-box versus a seller phone.

WATCH OUT

- The harvest side is single-id only, so pulling from several docked phones is serialized one request per phone; only the push-back direction actually batches. State this asymmetry up front rather than reading "harvest camera roll… push back" as symmetric.

- The default `outtime` is 15000 ms — far too short for a hundred photos at once; raise it and poll the Media folder for arrivals rather than trusting the response.
- Comparing file size between the phone's record and the PC copy is a weak integrity check: a truncated transfer or a HEIC-to-processed transcode can coincidentally match on size. Prefer a hash check, or at minimum require a human confirmation before the first several deletes.
- Camera-roll harvesting pulls in whatever is on the roll, not just intentional stock photos — personal images and geotagged EXIF metadata included. With no server-side content filtering available, scope the harvest to a dedicated album rather than Recents, and strip metadata before anything downstream is published; this is a staff privacy matter, not just data hygiene.

SCALE

Three to five API round trips per phone; everything else is photo bytes moving over that phone's own Wi-Fi through the Shortcut, so tens of seconds to a few minutes per hundred photos depending on link quality (the two XRs on this rig already differ 6x in AirPlay throughput, so expect similar variance here). Harvest is serialized per phone by the single-id rule, but N docked phones harvest concurrently via N connections since phones are independent at the API layer (512 req/s measured across ten concurrent pollers). Push-back is genuinely O(1) in phone count. The true bottleneck is the external editing step and Wi-Fi transfer time, not the kernel API.

Fleet-wide storefront maintenance

Fleet price-drop and relist sweep with mirrored taps

NATIVE API WITH CAVEATS

Runs the same open-item, edit-price, save (or relist) sequence across a group of seller phones. Every XP input endpoint takes a comma-separated id list and the board walks all listed phones in parallel, so the taps themselves cost the same wall time for nineteen phones as for one. Prices are ASCII digits typed straight through `/key/sendkey`, so this is the one card in the set whose *input* half needs no Shortcut anywhere in the loop.

HOW

1. `get_group_list` / `get_device_list` XP `/device/group/get → /device/group/get-dev?id=<gid> → /device/get?id=a,b,c` — build the target id list. Exclude any phone with state 0, empty vid/pid, empty location_crc, or `imgw` / `imgh` different from the master.
2. `mouse_reset_pos` XP `/mouse/reset?id=a,b,c` — clear stuck-click state across the whole target set; the load-bearing step (a stuck phone measured 0.09 mean diff on click versus 141.96 after reset).
3. `send_key` XP `/key/sendkey?id=a,b,c&fn_key=WIN+h` (twice) — normalise to home page one; confirm per phone with a whole-frame mean diff since layouts differ.
4. `click` XP `/mouse/click?id=a,b,c&x=&y=&button=1` — open the seller app, its listings tab and the first item. `button` is numeric (1–8), not the "left"/"right" strings from Phonefarm.
5. `send_key` XP `/mouse/click on price field → /key/sendkey fn_key=WIN+a → /key/sendkey key=1799 → /mouse/click Save` — edit the price. Requires Full Keyboard Access on; there is no error signal if it is off.
6. `click` XP `/mouse/click?id=a,b,c&x=&y=` — relist / renew / share variants; ordinary taps, no dedicated marketplace-action API exists in either edition.
7. `get_device_screenshot` XP `/pic/screenshot?id=<one id>&binary=true&rect=[l,t,r,b]` — verify per phone before moving on; single id, so cheap but not batched.

8. `swipe` / `wheel` XP `/mouse/swipe?id=a,b,c&direction=up&len=0.4` or `/mouse/wheel?id=a,b,c&direction=down&len=30&number=3` — advance to the next row; prefer wheel for list traversal since it skips the ~0.6 ms/px pointer walk entirely.

NEEDS

- Every phone in the target set on the same seller-app version with the same list ordering and identical `img w` / `imgh` — a coordinate is only a fair shared target if the same element sits there on all of them.
- Boards bound at state 2, non-empty location_crc, and the full iOS accessibility checklist (AssistiveTouch, Full Keyboard Access, tracking speeds max, Auto-Lock never, portrait lock), proven per phone with `/mouse/reset` + `/mouse/move` and a pointer-box diff before trusting the sweep.
- Groups created and populated first via `/device/group/set` and `/device/set`.

WATCH OUT

- The fan-out is real for input but not for observation: `/pic/screenshot` is single-id, so verification is one request per phone even though the taps are O(1). Split the framing: input is fully kernel-native and O(1); verification is kernel-native but O(N) and only proves pixels changed.
- A `/mouse/click` response of `data.code 0` only means the kernel accepted the call — the stuck-click state returns the same code while doing nothing, so the sweep must verify by frame and `/mouse/reset` must precede every tap-critical run.
- Insert a screenshot check of the price field itself between typing and tapping Save, not only after Save — by the time a post-Save screenshot runs, a wrong or unregistered keystroke has already been committed.
- A stable, identical listing-tab layout across every phone in the id list is a strong and fragile assumption for a real operation where sellers may have different listing counts, sort orders or app versions; a mismatch produces silent wrong-price edits across the whole batch with no server-side catch.
- `error_push` is scoped to hardware/board failures and will not fire for a logically wrong outcome (wrong item selected, unfocused field) — it complements but cannot replace per-phone pixel-diff verification.

> TRAP **One bad coordinate, N wrong edits**
>
> The same fan-out that makes this cheap makes mistakes expensive: a mirrored id list replicates one wrong coordinate or wrong price string instantly across every phone in the sweep. Dry-run each new app version on a single phone before enabling the id list, and keep an explicit human confirmation before any sweep that writes prices.

SCALE

Input is genuinely O(1) in phone count: a click or move with nineteen ids costs the same wall time as one, since latency is a function of pointer distance (~0.6 ms/px), not fan-out. Verification is O(N) but cheap and independent (2 ms per raw BMP, no shared bottleneck). A 10-item sweep at ~6 taps per item runs to roughly 100–300 price edits/hour across the whole rack if verification checkpoints stay light — the real driver is the app's own settle time between taps, dropping sharply if a human reviews every screenshot before advancing, which the fragility of the shared-coordinate assumption argues for on any new app version.

Customer messaging and live commerce

Human-in-the-loop support reply desk over live mirrors

NEEDS SHORTCUT WITH CAVEATS

An agent works a console showing the live mirror of each support phone — WhatsApp Business, Instagram DMs, marketplace chat — picks a conversation and a reply, and the console taps and types on the phone in mirror-pixel space. ASCII replies go straight through `/key/sendkey`; anything with emoji, accents or non-Latin text goes through the phone clipboard. A cropped screenshot confirms the bubble appeared. Every reply stays human-composed or human-approved and human-triggered.

HOW

1. `get_device_list` XP `/device/get` — read `imgw` / `imgh` per phone, the coordinate space for every tap below.
2. `mouse_reset_pos` XP `/mouse/reset?id=<comma list>` — clear stuck-click state whenever the agent switches phone or before any tap-critical reply.
3. `get_device_screenshot` XP `/pic/screenshot?id=<one>&binary=true` — live mirror in the console; XP has no push-video stream, so this is a poller per phone on a fixed period, not a broadcast.
4. `click` XP `/mouse/click?id=<ids>&x=&y=&button=1` — tap the conversation row, then the compose field.
5. `send_key` XP `/key/sendkey?id=<ids>&key=<ascii>` — ASCII fast path, including a quick-reply trigger the app itself matches; the API only delivers characters.
6. `shortcut#10` XP `/shortcut/clipboard/set?id=<comma list>&text=<utf8>&sleep=1&outtime=15000` — Unicode/emoji/RTL reply text to the clipboard; one broadcast can reach many phones since id is comma-separated, but serialise per phone since the clipboard is a single shared slot.
7. `send_key fn_key` XP `/key/sendkey?id=<ids>&fn_key=WIN+v` — paste into the focused field.
8. `click` XP `/mouse/click` — tap the app's own Send control, kept bound to an explicit agent action; do not rely on `fn_key=ENTER`, since hardware Return behaves differently per app.
9. `get_device_screenshot` XP `/pic/screenshot?id=<one>&binary=true&rect=[l,t,r,b]` — confirm the bubble appeared by mean absolute pixel difference; no text verification is possible on this rig.
10. `shortcut#10` XP `/shortcut/clipboard/set?id=<ids>&text=` — clear the clipboard after each reply so customer PII does not sit there.
11. `shortcut#24` XP `/shortcut/device/ip?id=<one>&state=0` — cheapest proof the Shortcut path is alive on a phone; run before the desk opens, not per reply.

NEEDS

- Board bound per phone at state 2 with non-empty location_crc, and the full iOS accessibility checklist, none of it queryable over the API.
- iMouse Shortcut installed and bound (Send Binding Key) on every phone for the Unicode path, with Shortcuts > Advanced allowing scripts and large data.
- A per-app coordinate map (thread row, compose field, Send, attachment picker), refreshed after app updates and per phone layout.
- Agent accounts and business profiles the operator legitimately owns.

WATCH OUT

- One HID pointer per phone means one agent can drive a given handset at a time; two agents cannot work the same phone's thread list concurrently without racing the pointer and the single clipboard slot — enforce an explicit per-phone lock in the desk UI.

- No server-side verification exists: the mean-pixel-diff bubble check catches gross failures (nothing sent) but not a reply sent with wrong or garbled text from a stale clipboard, and it cannot read a "message not delivered" banner.
- A timed-out Shortcut launch (`data.code 30` , "Call timeout") pastes stale or empty clipboard content into a live reply; check the Shortcut's liveness before treating a paste as safe to send.
- `/key/sendkey` has no documented per-key timing, rate limit or maximum length — it has not been measured on this rig the way pointer movement has. Measure typed-length versus latency before relying on it for long templated replies.
- Platform terms forbid automated bulk or unsolicited messaging; this design is defensible only because a human composes or approves and triggers every send — never wire the Send tap to a scheduler.

> TIP **Blank the clipboard after every reply**
>
> Customer names, addresses and order numbers otherwise sit on the phone clipboard until overwritten, and every clipboard call crosses an unauthenticated LAN endpoint. Fold the clear-clipboard step into the send action itself rather than treating it as optional cleanup.

SCALE

Per-reply API overhead is small next to human read/compose time: ASCII path roughly 200–500 ms, Unicode/clipboard path 1–5 s bounded by `outtime` . Ten concurrent pollers sustained 512 req/s at p50 3 ms with no interference between input traffic and frame polling, so the mirror wall itself costs nothing. Realistic agent throughput is roughly 60–150 replies/hour across several multiplexed phones, gated by the human and by serialising the clipboard and pointer per phone, not by the rig.

Live-commerce control room: product pinning and comment replies

NEEDS SHORTCUT

WITH CAVEATS

A dedicated "producer" phone per live stream runs the seller's LIVE control surface — product showcase panel and comments — while the host streams from a separate handset outside the rack. A scheduled script rotates the pinned product by tapping captured coordinates; operator-approved comment replies go through the clipboard-and-paste path; a moderator watches every stream on a wall of screenshot pollers, one per producer phone.

HOW

1. `get_device_list` XP `/device/get` — read `imgw` / `imgh` , state and rotate per producer phone; exclude any whose `imgw` / `imgh` differs from the phone the coordinate map was captured on.
2. `set_group` / `set_dev` XP `/device/group/set?id=0&name=live-producers then /device/set?id=<list>&gid=<gid>&name=Slot-01` — create the producer group and assign slots.
3. `mouse_reset_pos` XP `/mouse/reset?id=<list>` — clear stuck-click state before each scheduled pin; a pin that silently never happened is invisible unless you verify.
4. `click` XP `/mouse/click?id=<one>&x=&y=&button=1` — tap the shopping-bag icon, the product row, then Pin; drive each phone with its own coordinate set since row counts can differ.
5. `swipe` XP `/mouse/swipe?id=<ids>&direction=up&len=0.3` — scroll the product list between pins.
6. `shortcut#10` XP `/shortcut/clipboard/set?id=<list>&text=<utf8>&sleep=1&outtime=15000` then `send_key fn_key` XP `/key/sendkey?fn_key=WIN+v` and `click` Send — operator-approved comment reply, kept human-triggered.

7. `get_device_screenshot` XP `/pic/screenshot?id=<one>&binary=true` — moderator wall, one poller per phone; measured a 30 fps wall costs the master view nothing.
8. `send_key fn_key` XP `/key/sendkey?id=<list>&fn_key=MuteSoundSwitch` — keep the rack silent without dropping the cast, unlike `/device/airplay/set air_audio` which disconnects and reconnects.
9. `dev_rotate` callback XP `ws://<pc>:9911/api` — react to orientation changes; every stored coordinate must be re-derived when `imgw` / `imgh` swap meaning.
10. `auto_connect_screen` XP `/device/airplay/connect (disconnect first, XP has no force flag)` — recover a producer phone that lost its mirror; may fail with code 15 or 28 since the plug-ins the vendor's own navigation relies on are absent here.

NEEDS

- Producer phones signed into the seller's own account with LIVE shopping enabled; the host stream runs on a separate handset that need not be in the rack.
- A coordinate map of the LIVE control layout per app version and per phone layout, with a re-capture routine after app updates.
- iMouse Shortcut installed and bound on each producer phone for non-ASCII replies.
- A persistent WebSocket connection for `dev_rotate`, `dev_disconnect`, `error_push` and `airplay_connect_log`, with its own reconnect logic — a dropped socket produces silence, not an event.
- Licence headroom: `dev_online_num` within `total_license`, or phones are refused with "not logged in, not allowed online."

WATCH OUT

- Every pin is fire-and-forget unless verified: with OCR absent, there is no way to read "Pinned" back, only a mean-pixel-diff against a golden frame — and a silently failed pin looks identical to a working one. Gate the next scheduled pin on that diff succeeding, not just alert after the fact, or a stuck-click phone will silently skip every future pin in the run.
- Only two of nineteen boards were ever observed mirroring simultaneously on this rig. Treat concurrent producer-phone streams as validated at n=2, single digits pending a real multi-phone test, not the board count.
- A pinned-product rotation script that fires on a fixed schedule regardless of what is on screen (a checkout modal, an ad interstitial, a permission prompt) taps blind — nothing in the chain checks current screen state before the next scheduled sequence.
- Per-phone AirPlay source rate varies enormously on identical hardware; some tiles will look laggy for reasons no setting fixes.

> TRAP **A muted tile is a dropped tile**
>
> `/device/airplay/set air_audio` disconnects and reconnects the cast to mute, which drops that moderator tile for the reconnect duration during a live. Default to the hardware `fn_key=MuteSoundSwitch`, which does not touch the cast, and treat `air_audio` as a fallback only.

SCALE

Pinning is a handful of taps per minute per phone, negligible against the board's parallel id-list dispatch. The moderator wall scales for free per this rig's own numbers (512 req/s at ten concurrent pollers, p50 3 ms), and streaming input alongside polling measured no interference. The binding limits are per-phone AirPlay

production (4.5–29 fps observed on identical hardware), LAN bandwidth per mirrored stream, and the licence's online-device count — not the rack's board count. Comment replies through the clipboard are seconds-scale round trips bounded by `outtime`, so they stay operator-paced by design.

Verification, compliance and monitoring

Coupon, loyalty and promo discount verification in the app

PLUS EXTERNAL TOOLING · WITH CAVEATS

Before a campaign goes out, drives a rack phone through the store app's cart, types the promo code, taps Apply, and captures a cropped screenshot of the totals box as evidence the advertised discount actually lands in the iOS app — catching the gap between a web-only discount and one that silently fails on mobile. Every input and capture step is proven on this rig; only turning the crop into a number has no working path today.

HOW

1. `send_key fn_key` XP `/key/sendkey?id=<ids>&fn_key=WIN+h` (twice) — normalise to home page one; doubles as a liveness probe since it works even while the pointer is stuck.
2. `mouse_reset_pos` XP `/mouse/reset?id=<ids>` — clear stuck-click state on the whole target set; skipping this is the single most likely cause of a false "coupon failed."
3. `click` XP `/mouse/click?id=<ids>&x=&y=&button=1` — open the app, open a known product, add to cart, open Cart, then tap the promo-code field to focus it.
4. `send_key` XP `/key/sendkey?id=<ids>&key=SAVE20` — type the ASCII coupon code; a non-ASCII code needs the clipboard-and-paste path instead.
5. `click` XP `/mouse/click` — tap Apply.
6. `get_device_screenshot` XP `/pic/screenshot?id=<id>&binary=true` — poll at a fixed period until the totals area settles (mean-diff falling back toward the idle floor).
7. `get_device_screenshot` XP `/pic/screenshot?id=<id>&binary=true&rect=[l,t,r,b]` — capture just the totals box, in the same mirror-pixel space as the clicks.
8. `ocr` / `ocr-ex` XP `/pic/ocr, /pic/ocr-ex, /pic/find-text, /pic/find-text-ex — ALL UNAVAILABLE ON THIS RIG (data.code 28)` — the intended way to read the discount amount; not usable until the plug-in is installed.
9. Client-side mean-pixel-diff, or `find_image` / `find_image_cv` XP `/pic/find-image or /pic/find-image-cv (plug-in status unverified here)` — alternative assertion needing no OCR: compare the totals crop against its own prior value from a known-good run.
10. `get_device_screenshot` XP `/pic/screenshot?id=<id>&save_path=D:\iMouseXP\Screenshot\` — archive the full frame as the campaign record; the kernel writes the file and returns the path.
11. `click`, `send_key` XP `/mouse/click ... then /key/sendkey?fn_key=WIN+h` — remove cart lines and go Home for the next code; cart state is not queryable, so another screenshot is the only proof it is empty.

NEEDS

- Store app signed in with a TEST account and a single-use test coupon code — a live code on a production account can consume a real redemption.
- A coordinate map in mirror pixels per app version and per phone.
- Control and video channels both ready (board state 2, phone bound; `/device/get` state non-zero with a live air_pid), plus the full iOS accessibility checklist.

- For server-side reading only: the vendor OCR plug-in installed and connected, or a PC-side OCR library — external tooling this rig does not currently have.

WATCH OUT

- The read-the-number step does not work on this rig today: `/pic/ocr` returns `data.code 28`, and `find-text`, `ocr-ex` and `find-text-ex` sit behind the same plug-in.
- A rejected code often re-renders the same totals with a small error toast, which can register as a real change (diff 20–160) even though the discount did not apply — diff the specific totals-number crop against its own prior value, not just "something changed" on the page.
- Applying a promo code is a security-sensitive cart action in most storefronts, often gated by bot-detection or fraud SDKs. A code that fails only under automation looks, in the API's eyes, identical to a genuinely broken coupon (`data.code 0` on every call) — the rig cannot distinguish "coupon broken" from "automation detected and silently blocked."
- Only two of nineteen boards have a phone actually mirroring on this rig, so checks are serialized across those two devices today, not parallel across a fleet.

> TRAP **"It worked" and "automation was blocked" look the same**
>
> Because every call still returns `data.code 0` whether the coupon applied or a fraud filter silently intervened, a passing pixel-diff cannot separate a genuinely broken promo from one that platform defenses quietly suppressed. Treat a failed check as "investigate," not as proof the coupon itself is broken.

SCALE

About 8–10 taps plus one settle-wait and one crop per code per phone: 15–30 s realistically, dominated by app response time, not the API. Input and capture do not contend (98.8 req/s with streaming input versus 78.7 idle). Tap steps batch across phones sharing a layout; screenshots do not (single-id, 2 ms each). Practical parallelism today is the two mirroring phones, giving roughly 240–480 checks/hour combined — not the 19 the board count implies.

Banking and PSP app availability heartbeat

NATIVE API · WITH CAVEATS

A dedicated monitoring phone opens a merchant's banking, PSP or payout app on a schedule and confirms it reaches its expected login or dashboard screen, timing the launch and archiving a frame, so payout-day outages and forced-update walls are caught hours before someone notices a blank screen. Scope is deliberately "does the app open and render the expected screen" — no automated login, no credential entry, ever.

HOW

1. `send_key fn_key`, `mouse_reset_pos` XP `/key/sendkey?fn_key=WIN+h (twice), then /mouse/reset?id=<ids>` — normalise and clear stuck-click state; WIN+h works even while the pointer is stuck, so it doubles as a board-liveness probe.
2. `click` XP `/mouse/click?id=<id>&x=&y=&button=1` — record to and tap the app icon.
3. `get_device_screenshot` XP `/pic/screenshot?id=<id>&binary=true` — poll frames at a fixed period until launch settles, recording elapsed time; no server-push frame stream exists on XP.
4. Client-side mean-pixel-diff, or `find_image` / `find_image_cv` XP `server-side equivalents plug-in-gated and unverified here` — compare the settled frame's stable crop against a golden crop, and diff the banner region to flag a forced-update or maintenance wall.

5. `get_device_screenshot` XP `/pic/screenshot?id=<id>&binary=true` — detect capture-protected apps (near-uniform histogram on an otherwise-fresh frame) and report "renders but capture-protected," never "down."
6. `get_device_screenshot` XP `/pic/screenshot?id=<id>&save_path=D:\iMouseXP\Screenshot\` — archive the evidence frame.
7. `send_key fn_key`, `swipe` XP `/key/sendkey?fn_key=AppSwitch then /mouse/swipe, or fn_key=WIN+h` — leave the phone clean for the next check.
8. `get_device_list`, `get_usb_list`, callbacks XP `/device/get, /config/usb/get, dev_connect/dev_disconnect/usb_change on ws://127.0.0.1:9911/api` — watch control and video channels independently rather than inferring health from one; a phone can be controllable with no picture, or visible with no control.

NEEDS

- A dedicated monitoring phone that holds no real funds and is not anyone's primary banking device, signed in only as far as the screen you intend to assert on.
- Golden crops per app per app version, captured at the same `air_ratio`, since crops are resolution-specific.
- Board bound and connected, the full iOS accessibility checklist, and a PC-side scheduler — the kernel has no cron of its own.

WATCH OUT

- Never automate login. This is a launch/render probe, not a session driver; Face ID, a passcode prompt or an SMS one-time code end the probe at that screen, full stop.
- Banking and PSP terms typically prohibit third-party automation and, often, screen mirroring of the app — confirm terms per app before wiring it into a schedule; this is a legal risk, not a technical one.
- Capture protection is real and unverified here: an app that blanks its content while the screen is captured yields a flat frame that the heartbeat must distinguish from "down," or it pages someone every run. Banking apps are among the likeliest to implement this or to detect the AssistiveTouch pointer outright, degrading the probe to "launched, no evidence" for every run — verify this once per app before trusting it long-term.
- A forced iOS update (Auto-Lock Never does not block it) can change the exact screen layout a golden crop was captured against, producing a false "down" alert that is really just a UI refresh; golden crops need a version-pinned refresh process.
- IMConsole.exe is a GUI app whose absence silently zeroes out `/device/get` and `/config/usb/get` with no API-level alert — the heartbeat needs its own external process watchdog on IMConsole.exe, or a false "all apps down" page-out is inevitable the first time the console crashes.
- A dropped WebSocket connection produces silence, not a callback; a heartbeat relying on `dev_disconnect` or `error_push` needs its own liveness ping and reconnect logic, since there is no documented WS heartbeat frame in the protocol.

> TRAP **A reboot looks exactly like every app being down**
>
> A restart of the phone or kernel drops mirroring and leaves the phone at the lock screen; the probe must re-cast, and a passcode-protected phone will simply sit there with Full Keyboard Access still on but no board-driven way past the lock — which correctly cannot be automated. Without a distinct "stuck at lock screen" state, this reads as every monitored app failing at once.

SCALE

A launch-and-capture cycle is a few seconds of wall time, almost all of it the app's own launch, so one monitoring phone can rotate through a dozen apps every 10 minutes comfortably. Raw BMP is 2 ms per frame at 84 req/s per phone, and input traffic does not slow polling or vice versa. Roughly 60–120 checks/hour on the one dedicated phone, bottlenecked by app launch time and by keeping the fleet to a single monitoring device, not by the API.

Automated video production

On-device video render farm driving CapCut/InShot templates

NEEDS SHORTCUT WITH CAVEATS

Push raw clips and stills into each phone's album, drive a pre-favourited CapCut or InShot template with taps and drags, paste text overlays through the clipboard, wait out the export by polling a cropped progress bar, then pull the finished MP4 back and clean up. Every step routes through the iMouse iOS Shortcut, so the phones become a batch renderer for template styles that exist only inside the iOS apps, with no cloud render cost. TikTok Shop and Reels listings need many short vertical videos, and this is the only way to reach templates a desktop editor cannot reproduce.

HOW

1. `shortcut#7 (album upload)` XP `POST /api/shortcut/album/upload` — seed raw clips and stills into every target phone's album in one call; `id` is a comma list, `files` is an array of PC paths, blank `album_name` means Recents, `outtime` is milliseconds (15 s default, raise it for large files).
2. `send_key, mouse_reset_pos, click, shortcut#13` XP `/api/key/sendkey (WIN+h); /api/mouse/reset; /api/mouse/click; or POST /api/shortcut/exec/url` — return Home, clear any stuck click, then open the editor app, either by icon tap or by URL scheme if the app publishes a usable one (unverified here).
3. `click, swipe, mouse_down/move/up` XP `/api/mouse/click, /api/mouse/swipe, /api/mouse/down+move+up` — walk the template flow: pick the template, place media cells in order, preview, export. Coordinates are mirror pixels (500x1080 on the XRs); use a down-move-up drag for timeline scrubs rather than the canned swipe.
4. `shortcut#10 + send_key` XP `POST /api/shortcut/clipboard/set then GET /api/key/sendkey?fn_key=WIN+v` — fill a text overlay, including non-ASCII, by setting the clipboard (comma id list, `sleep` in seconds) then pasting. XP has no async send_text/send_text_ok, so this two-call sequence is the only documented Unicode path.
5. `get_device_screenshot` XP `GET /api/pic/screenshot?binary=true&rect=[l,t,r,b]` — poll a crop of the progress bar, single-id only, and judge completion by mean absolute pixel difference against the prior crop, not by hash; poll every 250-500 ms since exports run minutes.
6. `click, shortcut#1 (album get)` XP `/api/mouse/click then GET /api/shortcut/album/get?num=1` — tap Save/Done, then read back the exported file's name, extension and create_time; single-id, and there is no date filter on XP so the newest item must be inferred from `create_time`, not position.
7. `shortcut#2 (album down)` XP `POST /api/shortcut/album/down {id, list:[{album_name,name,ext}], zip, outtime}` — pull the MP4 back to the PC; single-id, name and extension are split fields, lands in `<install dir>\Shortcut\Media`, and the response carries no file manifest.
8. `shortcut#3 (album del) + send_key` XP `POST /api/shortcut/album/del then /api/key/sendkey (WIN+h)` — delete the raw inputs and export, comma id list, then return the phone Home. `/shortcut/album/clear` wipes an entire album and has no Phonefarm counterpart; treat it as destructive.

9. `shortcut#4/#5 (file get/down)` XP `GET /api/shortcut/file/get then POST /api/shortcut/file/down` — alternate route when the app exports to Files instead of the album; iOS 15+ only (fine on the 18.7.10 XRs), single-id, lands in `<install dir>\Shortcut\File`.
10. `find_image / find_image_ex / ocr` XP `/api/pic/find-image-cv, /api/pic/ocr, /api/pic/find-multi-color` — the intended version-proof way to detect app state instead of tapping blind; OCR returns code 28 (unavailable) on this rig and the image-match plug-ins are unprobed, so this step does not work today.

NEEDS

- iMouse iOS Shortcut installed on every phone, kernel IP edited in, run once with all permissions allowed, and bound to an AssistiveTouch key. Not evidenced on this rig; prove it first with `GET /api/shortcut/device/ip` returning code 0 and the expected IP before building anything on top.
- Shortcuts > Advanced enabled per phone: allow running scripts, allow sharing large data, allow deleting without confirmation, allow deleting large data.
- iOS 15+ for the file endpoints (18.7.10 on the XRs qualifies); board bound and connected, since the Shortcut launches via a HID hotkey; mirroring live to see and drive the editor UI.
- CapCut or InShot installed, signed in, target templates pre-favourited, and a coordinate map per app version in 500x1080 mirror pixels.
- Free storage for raw inputs plus export on each phone; Full Keyboard Access on for WIN+v paste; Auto-Lock set to Never so a multi-minute export does not sleep the phone.
- Per-device licensing headroom: devices are refused online when unlicensed, so check `dev_online_num` against `total_license` via `/config/user/info`, not an edition gate.

WATCH OUT

- Until `/shortcut/device/ip` returns code 0, the upload, clipboard, album-get, album-down and album-del steps do not function at all. That is most of what makes this a render farm rather than plain template navigation, so treat those specific steps as not achievable, not merely caveated, until that one proof call succeeds.
- XP has no generic shortcut dispatcher: each function is its own path with typed parameters. Phonefarm's double-encoded `parameter` JSON, `devlist` array and string `retdata` do not exist here, and `outtime` is milliseconds on XP versus seconds on Phonefarm.
- album/get, album/down, file/get, file/down, clipboard/get and every /pic/* call are single-id; only upload/del/clear/clipboard-set/exec-url fan out to a comma list. Pull-backs and screen verification are per phone, so wall-clock scaling with phone count is much worse than the fan-out steps alone suggest.
- "Newest album item = first in the picker" is app behavior that breaks the moment anything else writes to the album, including a prior run's own export. Sequence runs so only the intended assets are recent, or select by `create_time` from album/get.
- No working server-side state detection exists today: OCR returns code 28 and the image-match plug-ins are unprobed. Every tap returns code 0 even when it lands on nothing, so a blind coordinate chain fails silently; the actual working method is client-side pixel-diff against a golden crop, which still breaks on layout drift between app versions.
- CapCut/InShot template editing routinely uses pinch-to-zoom and two-finger trims. The rig has one HID pointer per phone with no native multi-touch; AssistiveTouch's gesture menu can simulate a pinch only through its own on-screen menu, which is slower and less reliable than the single-pointer drag this chain otherwise uses, so not every template interaction is covered.
- Transfers and the phone's own encoder dominate wall clock, and they share Wi-Fi with AirPlay mirroring. Plan outtime and timing against the low end of the measured bandwidth range (roughly 5 Mbps), not an

average, since mirroring runs concurrently and is not controllable through the API; a slow phone's mirroring also degrades further during a transfer storm.

- The album/down response carries only code, id and message, no file manifest; verify the file actually landed in `<install dir>\Shortcut\Media` rather than trusting code 0.
- Watermarks, template licensing and app subscription tier are phone-local facts the API cannot see or change. CapCut and InShot terms of service may restrict automated use; check before running this at volume, and never point a delete-without-confirmation cleanup step at a phone's Recents if it holds anything a person cares about.
- The API is unauthenticated on the LAN, so any host on the network can upload into, or download out of, any phone's album. Firewall port 9911 and treat the kernel's IP as a trust boundary.

SCALE

Taps are cheap; transfers and the phone's own encoder dominate. A roughly 100 MB clip in and 50 MB out per phone means outtime needs to be in the minutes (180000 ms upload, 300000 ms download are reasonable starting points, counted in milliseconds on XP). Upload, delete and clipboard-set fan out to a comma id list in one call; album/get and album/down do not, so pull-backs run sequentially per phone though concurrently across phones. Progress-bar polling is independent per phone and does not contend with input traffic (measured at 83-512 req/s across 2-10 pollers, p50 3 ms). With the Shortcut verified and working, expect on the order of a few videos per hour per phone once transfer and export time are counted, roughly 2 to 6 videos/hour across the two phones actually online today. A 19-phone, 300-video/hour figure is a projection only; this rig has 19 boards but two phones mirroring, the wall has never been tested past two, and Wi-Fi bandwidth is the first thing likely to break above that.

Also worth knowing

Content and asset production

Store, documentation and marketing asset harvest across locales

NEEDS SHORTCUT WITH CAVEATS

Producing App Store screenshots, support and documentation screenshots, and app-preview or marketing video clips is recurring, scheduled work, done again for every release and every locale. This card treats it as its own production line rather than a one-off QA check, reusing the deep-link and hardware-hotkey chain from the cross-device and localization QA card and the Control Center recording chain from the QA screen-recording card, but running them for design and marketing deliverables instead of a layout diff. The rack's advantage over a person with one phone is repeatability across locales and screens, not resolution or device coverage, both of which stay bounded by whatever is physically docked.

HOW

1. `set_dev` / `get_device_list` XP `GET /api/device/set?id=<id>&name=xr-de-DE then GET /api/device/get`. Label one phone per locale, and per device model if more than the two XRs ever join the rack, so harvested files can be filed correctly. Read `imgw` / `imgh`, 500 x 1080 on this rig, before any capture.
2. `shortcut#13` XP `GET /api/shortcut/exec/url?id=<id1>,<id2>,...&url=myapp://screen/onboarding&outtime=15000`. Land every locale phone on the target screen in one fan-out request, the same deep-link approach as the QA card.

3. `get_device_screenshot` XP `GET /api/pic/screenshot?id=<one>&rect=[l,t,r,b]&save_path=D:\Assets\<locale>\`. For documentation and support screenshots, the mirror frame is good enough for a help article, so capture and crop straight to disk with no Shortcut hop and no album round trip at all. This is the cheap path, and it does not need the Shortcut app installed.
4. `send_key (shift+win+3)` XP `GET /api/key/sendkey?id=<id1>,<id2>,...&fn_key=shift+win+3`. For App Store screenshots, fire the phone's own hardware screenshot into Photos at native resolution, 828 x 1792 on this rig's XRs, above anything the AirPlay mirror can produce. Needs Full Keyboard Access on. The fan-out is free since `fn_key` takes a comma list.
5. `shortcut#1` then `shortcut#2` then `shortcut#3` XP `GET /api/shortcut/album/get?id=<one>&num=1, then POST /api/shortcut/album/down {id, list, zip:1, outtime:60000}, then POST /api/shortcut/album/del {id, list}`. Find the new still by `create_time`, pull it to `<install dir>\Shortcut\Media`, then delete it off the phone so the next locale's capture is unambiguous. All three are single-id, so a still harvest is strictly one phone at a time.
6. `send_key (fn_key=ControlBar) + click` XP `GET /api/key/sendkey?id=<ids>&fn_key=ControlBar then GET /api/mouse/click?id=<ids>&x=<tile x>&y=<tile y>`. For app-preview and marketing clips, open Control Center and tap the Screen Recording tile to start, run the scripted flow, then repeat the same tap to stop. This is the QA card's chain, reused for a deliverable instead of a bug report.
7. `shortcut#1` / `shortcut#2` / `shortcut#3` XP `as above, ext "mp4"`. Pull the RPReplay clip the same way as a still, with a generous `outtime` since a clip is tens of megabytes.
8. `none (manual)` XP `none`. Switch each phone's display language by hand for the next locale. There is no API for this on either edition, and it restarts SpringBoard, so schedule the switch outside the automated run and re-cast AirPlay afterward before capturing again.

NEEDS

- iMouse Shortcut installed on every phone used for stills or video, IP-edited, run once with all permissions, bound to an AssistiveTouch key; Shortcuts > Advanced set to allow large data and deleting without confirmation.
- One physical phone per locale to capture concurrently, and one physical phone per required App Store size class to cover more than a single size. Both boards mirroring on this rig carry the same model, iPhone XR, so today this pipeline can produce exactly one size class.
- Full Keyboard Access on, AssistiveTouch on, Auto-Lock never, portrait lock, the same accessibility checklist as every other Shortcut-dependent card in this document.
- Deep links for every screen that needs an asset, since nothing in the vendor docs describes how the Shortcut chain would navigate there without one.
- A naming and tracking scheme kept entirely on the PC: the kernel does not tag a captured file with locale, screen, or device model, and `/shortcut/album/get` returns only name, extension, size, and `create_time`.

WATCH OUT

- `shift+win+4` ("screenshot and edit") opens the Markup editor on the captured frame instead of saving it, which needs further blind taps to dismiss. Use `shift+win+3` for any unattended batch run; it saves to Photos on its own after the usual on-device thumbnail timeout.
- `/shortcut/other/screenshot` exists on shortcut v1.3 and later as a stated fallback for "special cases," but the vendor page gives it no parameter table of its own; it just reuses `/pic/screenshot`'s table under that heading. Whether it returns the AirPlay mirror frame or a native-resolution capture through the Shortcut app is not documented and was not probed on this rig; treat it as an unverified secondary route if AirPlay mirroring itself is down, not as a substitute for the confirmed `shift+win+3` hotkey.

- `get_devicemodel_list` is a static reference table of models the kernel recognizes, not a way to provision a size the rack does not physically have. Coverage of "every device size" is bounded by which phones are docked to a board, not by any API call.
- Album functions are single-id for get, down, and del alike, so both the still and the video harvest serialize per phone even though landing on the screen and firing the hotkey are free fan-outs. A ten-locale, six-screen still harvest is 180 serial Shortcut round trips before any video is pulled.
- A language switch drops the AirPlay session and needs a re-cast before the next capture; running it inside the same script as the capture loop, instead of as a separate manual step between locales, is what breaks a batch run silently.
- Nothing in the chain confirms a captured asset actually shows the intended screen in the intended language: with no OCR plug-in connected, a wrong deep link or a locale that silently fell back to English produces a file that looks correct to every check available here. Budget a human pass over the harvested batch before it goes into a listing or a help article.
- Video clips inherit the same caveats as the QA screen-recording card: a persistent AssistiveTouch overlay in the frame, a three-second start countdown to trim, and no report if an app blocks screen capture and records black.

> OPEN **What does the shortcut screenshot fallback actually capture**
>
> `/shortcut/other/screenshot` is named as one of the functions behind this card, but the documentation does not say whether it is a second route to the same 500 x 1080 mirror frame or a native-resolution capture taken through the Shortcut app itself. That distinction matters for App Store assets specifically. Confirm it against a real device before building a pipeline around it, and until then treat `shift+win+3` as the only proven native-resolution route.

SCALE

Landing on a screen is free per locale, one fan-out request regardless of count. Stills cost one Shortcut round trip per phone for get, down, and del each, on the order of 3 to 10 seconds per file measured elsewhere on this rig's Wi-Fi, so ten locales across six screens for stills alone is on the order of tens of minutes serial, not the minute a naive per-request estimate gives. A full-rack video pull runs 5 to 15 minutes serially at a generous 120-second `outtime`, per phone, for one clip apiece, since a one-minute recording is tens of megabytes. Documentation screenshots taken straight off the mirror with `save_path` are the fast path, at 2 milliseconds a frame with no Shortcut involved, and are the only piece of this card that scales the way the rest of the document's screenshot numbers do. Today's real ceiling is two phones, both iPhone XR, so a full store-asset refresh across many locales and multiple size classes needs more docked hardware, not a faster script.

Data collection, monitoring and verification

Everything in this section answers one question: how do you get exact information off a phone's screen and back onto the PC. On this rig the answer is narrower than the vendor docs suggest, because the OCR plug-in is not connected (`/pic/ocr` answers `data.code 28`, "插件未连接") and the image-search plug-in has never been probed. That removes every server-side recognition path and leaves three native channels: the mirror frame itself (compared by pixels, never by hash), the iOS clipboard (exact text, no plug-in needed), and the iMouse Shortcut's own photo album (native-resolution files). All three are documented XP-native endpoints; none of the Shortcut-dependent ones have actually been exercised on this rig's two phones, so every card below is gated on a one-call smoke test before it earns any throughput number.

> TIP **One call proves the whole Shortcut channel is alive**
>
> Every card in this section that depends on the iMouse Shortcut (album, clipboard, custom probes, deep links) rides the same install-and-binding chain, and none of it has been probed on this rig. The cheapest possible proof is `GET /api/shortcut/device/ip?id=<id>&state=0`: a `data.code 0` whose `data.text` equals the `ip` field from `/device/get` confirms the Shortcut is installed, IP-edited, hotkey-bound, and can reach the kernel on port 9911, all in one round trip. A 15 second wait ending in `data.code 30` means the AssistiveTouch binding never fired the Shortcut, and nothing downstream in this section will work until that is fixed. Run it once per phone before building anything on top.

Reading on-screen text and numbers without OCR

Region OCR of prices and counters inside native apps

NEEDS PLUG-IN · WITH CAVEATS

When a number lives inside a native app and is not selectable text (in-app pricing, follower counts, creator dashboards), deep-link to the screen, let the mirror settle, then crop a fixed rectangle and OCR it. On this rig the kernel OCR plug-in reports `data.code 28`, so `/pic/ocr` and `/pic/find-text` are dead code today; the only path that actually works is fetching the raw mirror frame and running OCR client-side on the PC against that same rectangle.

HOW

1. `shortcut#13` XP `/shortcut/exec/url?id=<ids>&url=<deep link>` — jump straight to the product/profile/dashboard screen, skipping in-app navigation
2. `get_device_screenshot` XP `/pic/screenshot?id=<id>&binary=true` — fetch two raw frames ~300 ms apart, require mean absolute pixel difference under ~1 before trusting the screen has settled
3. `ocr` XP `/pic/ocr?id=<id>&rect=[l,t,r,b]` — read the number from a fixed rectangle (flat four-integer array, not Phonefarm's four-corner-point form); confirmed dead on this rig, code 28
4. `—` XP `/pic/find-text?id=<id>&text=[…]&contain=true&similarity=0.8` — boolean check instead of parsing (e.g. "Out of stock" vs "Add to Cart"); same plug-in dependency, same dead result here
5. `—` XP `/shortcut/other/ocr` — named as a Shortcut-side "special case" fallback (v1.3+) but has no parameter table in the docs; probe once before designing around it

6. `get_device_screenshot` XP `/pic/screenshot?id=<id>&binary=true&rect=[l,t,r,b]` — the only path that works today: kernel crops server-side, PC runs Windows.Media.Ocr or Tesseract on the strip
7. `mouse_reset_pos` + `click` XP `/mouse/reset?id=<ids> then /mouse/click?id=<ids>&x=<centre.x>&y=<centre.y>` — optional: act on what was read, since OCR centres are already in mirror-pixel space

NEEDS

- OCR plug-in installed and connected — not satisfied here; `ocr_num` in the kernel config is GPU-build only and a change needs `/imserver/restart`
- For the deep-link and Shortcut-side fallback: the full iMouse Shortcut install/bind chain, plus Shortcut v1.3+
- Stable `air_ratio` per device — rects live in `imgw` × `imgh` mirror pixels (500×1080 here), never the iOS logical 414×896
- For the client-side fallback: an OCR engine on the PC plus a reused frame buffer (each 1.6 MB frame crosses the .NET Large Object Heap threshold)

WATCH OUT

- Treat steps 3-4 as dead code on this rig and build against the client-side fallback first; the kernel path is a drop-in once the plug-in is actually installed and a call returns code 0
- `rect` here is a flat `[left, top, right, bottom]` array that must be JSON-then-URL-encoded in GET mode — a Phonefarm-shaped four-corner rect is a parameter error on this kernel
- `/pic/ocr`, `/pic/find-text` and `/pic/screenshot` are all single-device — a rack sweep is one serialized round trip per phone at both capture and (once unblocked) recognition
- The mirror is roughly 60% of the XR's linear panel resolution (~36% of the area); small captions sit at the edge of reliable OCR cap-height once downscaled. Raising `air_ratio` helps but invalidates every stored rect and disconnects the cast — a real design fork, not a footnote
- `similarity` is the engine's own score, not a validated confidence figure — range-check the parsed value and require two consecutive settled frames to agree
- Settle timing is per phone: the rig's two otherwise-identical XRs deliver roughly 29 and 4.5 new frames per second under motion, so a fixed settle window is generous on one and starved on the other
- Many consumer apps only support top-level or category deep links, not item-level ones — those need an added in-app navigation phase (taps guided by earlier-frame template matching) before the rect read is valid, which reintroduces pointer-walk cost and mistap risk
- Reading numbers out of a third-party app's screens is still scraping that app; its terms of service apply regardless of the capture mechanism

SCALE

Capture is cheap and phones are independent: measured 84 requests/second per phone flat out, 512 req/s aggregate across ten concurrent pollers, no interference from input traffic. Recognition, not capture, is the ceiling — kernel-plug-in OCR latency is undocumented, and client-side OCR on a 500×1080 crop is an unmeasured estimate of tens to a few hundred milliseconds per call, so budget a handful of region reads per second per PC core and crop tight. Realistically that is roughly 200-800 region-reads per hour per phone for a workflow that navigates before every read, versus several thousand per hour for a fixed screen that never needs re-navigation. Only two phones are mirroring on this rig today; the 19 count is boards, not live devices.

Retail price watch by copying page text out of Safari

NEEDS SHORTCUT · WITH CAVEATS

Open a product URL in mobile Safari, select-all and copy the page with the board's hardware hotkeys, then read the clipboard back and regex the price, stock line and delivery promise out of plain text. It gets the mobile-personalized page the retailer actually serves (geo, app-vs-web pricing, member pricing) as exact characters, not an OCR guess. The whole approach rests on one unverified assumption: that iOS Safari treats Cmd+A as select-all-page on a non-editable web document, not just inside form fields.

HOW

1. `get_device_list` XP `/api/device/get` — enumerate targets, read each phone's `imgw` × `imgh` mirror geometry
2. `mouse_reset_pos` XP `/api/mouse/reset?id=<a,b>` — clear the stuck-click state before any tap-critical sequence
3. `shortcut#13` XP `/api/shortcut/exec/url?id=<a,b,c>&url=<product url>` — open the page on every listed phone in one request
4. `get_device_screenshot` XP `/api/pic/screenshot?id=<one id>&binary=true` — poll and diff by mean absolute pixel difference to detect settle, never a fixed sleep
5. `click` XP `/api/mouse/click?id=<id>&x=250&y=600` — give the web view keyboard focus, picking a known-blank region so the tap can't land on a link or ad
6. `send_key (fn_key)` XP `/api/key/sendkey?id=<a,b,c>&fn_key=WIN+a` — select all page text; the untested assumption the whole chain rests on
7. `send_key (fn_key)` XP `/api/key/sendkey?id=<a,b,c>&fn_key=WIN+c` — copy the selection
8. `shortcut#11` XP `/api/shortcut/clipboard/get?id=<one id>&outtime=<ms>` — read and parse; guard with a sentinel written before step 6
9. `get_device_screenshot` XP `/api/pic/screenshot?id=<id>&save_path=D:\evidence\` — optional evidence frame (kernel names the file itself, not timestamped)
10. `ocr / ocr_ex / find_image` XP `/api/pic/ocr, /pic/find-text, /pic/find-image` — not available on this rig, code 28; listed so no fallback is designed around a dead endpoint

NEEDS

- Full iMouse Shortcut install/IP-edit/AssistiveTouch-binding chain, verified with the `/shortcut/device/ip` smoke test before relying on anything past step 6
- Full Keyboard Access ON with 1 second auto-hide — without it WIN+a/WIN+c/WIN+v are not interpreted at all
- Board bound at state 2 with mouse parameters present, phone unlocked and mirroring, reachable to the kernel on 9911

WATCH OUT

- The core assumption is genuinely unverified: if Cmd+A only selects inside fields, the clipboard comes back empty or stale and the approach collapses to "open URL and screenshot." Test on one phone before building a pipeline
- `/key/sendkey` returns `code 0` once the hotkey was transmitted, not once iOS acted on it — without the sentinel guard a failed select-all silently reports last hour's price as this hour's
- WIN+a copies the entire page: navigation, cookie banner, recommendation carousels, footer. Anchor parsing on labels and currency symbols, never on line offsets, and expect Reader mode to reorder the copy

- Many retail sites render the price as an image, canvas element, or a lazy-loaded node that only materializes on scroll — select-all captures the DOM at copy time regardless of scroll position, so a price that only appears on intersection can be silently missing from the clipboard even though a screenshot would show it. That is a distinct failure from a cookie wall and needs its own handling
- Consent and cookie walls in several jurisdictions mean the first copy is the banner, not the product; accepting one automatically is a policy decision, not something to do silently
- A mistap that lands on a link or ad navigates away and the clipboard captures the wrong page with no distinguishing signal beyond the sentinel check — pick the tap point from the current frame, not a constant
- This rig has two phones mirroring, not 19 boards — every throughput figure below scales from that, not from board count

SCALE

Per page per phone: the URL open and both hotkeys fan out free over a comma id list; settle polling is cheap (raw BMP fetch measured at 2 ms, phones independent); the clipboard read is the one serial, single-id cost, bounded by `outtime`. Realistic estimate is 5-10 seconds per page per phone, dominated by real page load and the Shortcut round trip — roughly 8-15 pages/hour/phone sustained once retail load variance is accounted for, so 16-30 pages/hour across the rig's two live phones today. Whether the kernel serializes concurrent Shortcut calls to different phones is undocumented; measure with two simultaneous clipboard reads before promising a rack-wide number.

Change detection without recognition

Competitor profile monitoring by post-grid image diff

NEEDS SHORTCUT WITH CAVEATS

Drive a logged-in phone to a competitor's profile on a schedule, grab the mirrored frame, crop the top row of the post grid, and compare it to a stored baseline by mean absolute pixel difference. A score above threshold means the grid changed; archive the frame and re-baseline. The detection core is entirely native to this rig: a raw frame fetch and client-side arithmetic, no OCR plug-in and no platform API, so it survives the missing recognizer completely. Only navigation wants the Shortcut, and only an optional creative-tagging step wants the (unverified) image-search plug-in.

HOW

1. `get_device_list` XP `/api/device/get` — read geometry, mirroring state and board binding; exclude any phone whose `imgw` / `imgh` differs from the one the baseline was measured on
2. `mouse_reset_pos` XP `/api/mouse/reset?id=<a,b,…>` — clear stuck-click state before tapping
3. `send_key (fn_key WIN+h)` XP `/api/key/sendkey?id=<a,b,…>&fn_key=WIN+h` — press twice to normalize every phone to home page one
4. `shortcut#13` XP `/api/shortcut/exec/url?id=<a,b,…>&url=https://…/<handle>/` — deep-link straight to the profile, one request for every phone
5. `get_device_screenshot` XP `/api/pic/screenshot?id=<id>&binary=true` — fetch two frames ~500 ms apart, require mean absolute pixel difference under ~1 before treating the profile as loaded
6. `mouse_wheel` XP `/api/mouse/wheel?id=<ids>&direction=down&len=<1-127>&number=<n>` — scroll the grid into view; identical every run or the crop misaligns
7. `get_device_screenshot (rect)` XP `/api/pic/screenshot?id=<id>&binary=true&rect=[l,t,r,b]` — capture and crop the top grid row (flat array, JSON-then-URL-encoded)

8. `local` XP `(client-side)` — score the crop against the stored baseline by mean absolute pixel difference; idle measured 0.02-0.03, real change measured 20-160
9. `get_device_screenshot (save_path)` XP `/api/pic/screenshot?id=<id>&jpg=true&save_path=D:\social\<handle>\` — on a hit, archive the frame and set it as the new baseline
10. `find_image_ex` XP `/api/pic/find-image-cv` — optional: tag which of your own known creatives appeared; plug-in gated, unverified on this rig
11. `ocr / ocr_ex` XP `/api/pic/ocr, /pic/find-text` — not available: follower counts, post counts and captions stay out of reach; the design must stay purely diff-based

NEEDS

- Kernel reachable with IMConsole.exe logged in; each phone mirroring with a live `air_pid`
- A logged-in account you own or are authorized to operate — logged-out sessions typically hide or degrade profile grids
- AssistiveTouch, Full Keyboard Access, Auto-Lock Never, portrait lock, StandBy off and Do Not Disturb, none of which are queryable over the API
- The grid rectangle measured by hand once per app per phone, re-measured after every app update
- For deep-linking, the full Shortcut install/bind chain; for creative tagging, the image-search plug-in and a kernel restart

WATCH OUT

- A diff only proves the grid changed, never what changed or whether it was a post at all: stories rings, live badges, view counters, an avatar change, an A/B-tested layout and a reordered pin all cross the same threshold. Treat every hit as "review the archived frame," never as a confirmed new post
- A diff below threshold is equally ambiguous: it can mean no change, or that the app never loaded, hit a login wall, or served an error screen. Pair the diff with a positive-anchor rectangle over a known-stable element that must never disappear
- Repeated automated profile views on a real logged-in account routinely trigger a re-authentication prompt, an OTP challenge, a "suspicious activity" interstitial, or a rate-limit wall — this rig has no OCR to read any of them, so without the positive-anchor check a challenge screen either mis-scores as a grid change or silently stalls the schedule for that handle indefinitely
- Universal-link behavior is unverified per app: first-run "Open in app?" interstitials or signed-out mobile-web routing can serve a different grid layout and crop coordinates than expected
- Push and notification banners overlay exactly the profile-header/top-grid-row region and produce a diff spike unrelated to any post change; Do Not Disturb must be on for every monitoring phone
- `/pic/screenshot` is single-id; only navigation and hotkey steps fan out — capture cost scales linearly with phone count
- `binary=true` returns raw bytes with no JSON at all, so there is no `data.code` to read on that call; validate the body (BM magic, expected length) instead, and never mix `binary=true` with `save_path`

SCALE

Frame fetches are cheap and phones are fully independent: measured 83 and 85 req/s per phone concurrently, 512 req/s aggregate across ten pollers, no interference from input traffic. A 19-phone rack checking 50 handles each every 10 minutes is roughly 1.6 screenshots/second rack-wide, under 1% of measured capacity — the rig is nowhere near the bottleneck. Reuse one frame buffer per poller; each 1.62 MB frame crosses the .NET Large Object Heap threshold (488 MB/s allocation and 496 gen-2 collections per 8 seconds without reuse, versus 2.9 MB/s with it). The real limits are app cold-start and profile load (2-5 seconds), pointer travel if scrolling by

swipe, the weaker phone's slower settle window, and above all the platform's own tolerance for repeated views from one session — pace by handle-per-hour against that tolerance, not against the rig's request ceiling. Only two of 19 boards are actually mirroring today.

Native-resolution capture for evidence and QA

Native-resolution iOS screenshots pulled back as evidence

NEEDS SHORTCUT WITH CAVEATS

The AirPlay mirror is a re-encoded 500×1080 stream, the wrong artifact when a human, a client, or an OCR pass has to read small text. Firing the phone's own screenshot chord writes a native full-panel PNG into iOS Photos, which the Shortcut then pulls back to the PC. Unlike a mirror frame, the file is byte-stable and therefore hashable, with the iOS status-bar clock and EXIF creation time intact — the right input for ad-creative review, dispute and chargeback evidence, and anywhere font rendering or dark-mode contrast has to be judged at real pixels.

HOW

1. `click / shortcut#13` XP `/mouse/click or /shortcut/exec/url` — stage the page to capture; send `/mouse/reset` first, since taps can silently stop registering while moves and hotkeys keep working
2. `send_key fn_key WIN+h` XP `/key/sendkey?id=<ids>&fn_key=WIN+h` — two Home presses to normalize state before a cross-phone sweep
3. `send_key fn_key WIN+SHIFT+3` XP `/key/sendkey?id=<ids>&fn_key=shift+win+3` — fire the native iOS screenshot on every listed phone in one request; confirmed in the XP hotkey table
4. `shortcut#1` XP `/shortcut/album/get?id=<single>&num=1&outtime=<ms>` — read the newest album entry, one phone per call; sort order is undocumented
5. `shortcut#2` XP `/shortcut/album/down?id=<single>&list=[{…}]&outtime=30000` — pull the PNG to `<install dir>\Shortcut\Media`
6. `local` XP `(local file handling)` — rename with phone id and timestamp, hash the file, log the call (id, coordinates, fn_key, returned `create_time`) beside it
7. `shortcut#3` XP `/shortcut/album/del?id=<single>&list=[{…}]&outtime=<ms>` — clean the album so "newest entry" stays predictable for the next sweep
8. `get_device_screenshot` XP `/pic/screenshot?id=<single>&binary=true` — optional paired mirror frame, taken in the same moment, to scale OCR coordinates into mirror-pixel space
9. `mouse_wheel` XP `/mouse/wheel?id=<ids>&direction=down&len=<1-127>&number=<n>` — for long pages, scroll and repeat the chord rather than swiping

NEEDS

- Full Shortcut install/IP-edit/AssistiveTouch-binding chain, plus Shortcuts > Advanced permissions for running scripts and deleting without confirmation
- Full Keyboard Access ON — the screenshot chord is a hotkey iOS interprets, not a synthetic tap
- Board bound and connected, phone and PC on the same LAN, free storage and Photos access on the phone

WATCH OUT

- `/shortcut/album/get` and `/shortcut/album/down` are single-id only ("Multiple is not allowed") — only the capture chord fans out
- Native resolution is an iOS device property, not an API field: the API only exposes `imgw` / `imgh` (the mirror) and `width` / `height` (iOS logical points). Read the actual PNG dimensions from the downloaded file

- No screenshot plug-in path is involved, which is why this works at all here; the vendor also lists `/shortcut/other/screenshot` (Shortcut v1.3+) as an alternative "in special cases" but with no parameter table — probe before relying on it
- `/shortcut/album/del` is destructive and irreversible on the phone; run it per phone with that phone's own filenames, never with a rack-wide id list, and stay away from `/shortcut/album/clear` entirely
- `outtime` is milliseconds on XP (15000 default) but seconds in the Phonefarm sibling doc — mixing the two silently breaks a timeout
- Error codes to branch on: 30 call timeout (Shortcut never launched), 27 path does not exist, 35 file list empty, 2 device not bound, 3 hardware not connected, 25 device offline, 36 parameter error — match on `data.code`, never the (Chinese) message text
- iOS surfaces a screenshot thumbnail for a few seconds; a stray tap on it opens the markup editor and derails the next step. Screenshots of DRM-protected views (some banking and video apps) can come back blank
- This is fundamentally a serial-per-phone pipeline bookended by one free fan-out chord and one free fan-out cleanup precondition — describe it that way, not as rack-parallel

> TRAP **A wrong assumption here fails silently**
>
> Whether `num=1` returns the shot you just took is undocumented sort order. If it turns out to be oldest-first or unstable, the pipeline can archive an unrelated older photo as "evidence" with `data.code 0` at every layer and no error anywhere. Verify once against the chord's own `create_time` before trusting a sweep on it; a smoke test alone is not enough.

SCALE

The capture chord is a single request regardless of rack size — the board walks every listed phone for the same cost as one. Everything after that is serial per phone: one album read, one download, one optional delete, each a full hotkey-to-Shortcut-to-Wi-Fi round trip. No timing for this path has been measured on this rig; treat any per-shot figure (drafts suggest roughly 3-10 seconds, 1-2 minutes for a 19-phone sweep) as an unmeasured placeholder and instrument the first real run before quoting it. The rig's own AirPlay measurements already show a roughly 6x per-phone Wi-Fi asymmetry between two otherwise-identical XRs, which plausibly extends to the Shortcut's own file transfer.

The clipboard as a general data-exit channel

Select-all, copy, clipboard read: exact text without OCR

NEEDS SHORTCUT · WITH CAVEATS

On any surface where iOS exposes selectable text (Safari pages, text fields, Notes, a caption or bio field) the board can send select-all then copy, and the Shortcut hands the clipboard back as a UTF-8 string in `data.text` — no transcription error, no font guessing, Unicode intact. The same channel captures a permalink after tapping an app's own "Copy link" share action, though without OCR that step reduces to fixed per-app coordinates rather than anything text-aware.

HOW

1. `mouse_reset_pos` XP `/mouse/reset?id=<id1,id2,…>` — clear the stuck-click state before any tap-critical sequence
2. `shortcut#10` XP `/shortcut/clipboard/set?id=<ids>&text=<sentinel>` — write a per-phone sentinel first, so a stale read is detectable

3. `shortcut#13` XP `/shortcut/exec/url?id=<ids>&url=<url>` — Route A: push a URL onto the phone
4. `click` XP `/mouse/click?id=<ids>&x=<x>&y=<y>` — Route B: tap the app's own "..." menu then "Copy link"; coordinates only, since there is no OCR to find the row
5. `click` XP `/mouse/click?id=<ids>&x=<x>&y=<y>` — focus the field or page body (Route A only)
6. `send_key (fn_key)` XP `/key/sendkey?id=<ids>&fn_key=WIN+a` — select all; documented only in the XP hotkey table, absent from the Phonefarm doc
7. `send_key (fn_key)` XP `/key/sendkey?id=<ids>&fn_key=WIN+c` — copy
8. `shortcut#11` XP `/shortcut/clipboard/get?id=<single id>&outtime=<ms>` — read back, one phone per call; the only step that crosses Wi-Fi
9. `shortcut#10` XP `/shortcut/clipboard/set?id=<ids>&text=<placeholder>` — scrub the clipboard afterward, one request for the whole rack
10. `ocr / ocr_ex` XP `/pic/ocr, /pic/find-text, /pic/find-text-ex` — fallback if the plug-in is ever installed; all return code 28 today

NEEDS

- Full Shortcut install/IP-edit/AssistiveTouch-binding chain, Full Keyboard Access ON with 1 second auto-hide
- A text surface that is actually selectable by iOS — Safari bodies, fields, Notes and search boxes are; native feed cells, comment lists and DM bubbles generally are not
- Phone on the same LAN as the kernel, reachable on 9911; Handoff/Universal Clipboard turned off on rack phones

WATCH OUT

- Nothing in this chain has been exercised on this rig; run the `/shortcut/device/ip` smoke test before trusting anything past step 1
- `/shortcut/clipboard/get` is single-id — the hotkeys fan out to the whole rack, the read does not, so N phones means N reads
- Checking that a string "starts with https://" is not a sufficient staleness check, since a previous phone's permalink also starts that way — compare against the written sentinel, and pair it with a shape check: in a list-based UI, WIN+a can select objects or messages instead of text and still return a non-empty, URL-shaped clipboard payload
- WIN+a and WIN+c are XP-only documented hotkeys; the Phonefarm doc's fn_key examples and named-key list omit them entirely — don't present that column as vendor-confirmed
- Route B (tapping a share sheet's "Copy link") is exact-text only once a human already knows exactly where to tap; share sheets reorder by usage, so a fixed coordinate needs re-verification per app version
- Apps can show a "pasted from" or clipboard-access banner that changes what's on screen and can break a concurrent frame-diff check
- The API has no authentication and the clipboard text crosses the LAN as plain HTTP — never route credentials, one-time codes or payment details through it, and firewall 9911 to trusted hosts

> TRAP **A success code proves delivery, never outcome**
>
> `data.code 0` on the hotkey call means the keystroke reached the phone, not that iOS acted on it. A failed selection is silent — the read comes back with whatever was on the clipboard before. The sentinel-write-then-compare pattern in steps 2 and 8 is the only defense; a naive "non-empty string" check passes even when the wrong thing was copied.

SCALE

The reset and both hotkeys fan out free over a comma id list — 19 phones cost the same three requests as one. The serial cost is the clipboard read: a full round trip out over the board, through iOS, back over Wi-Fi to 9911, bounded by `outtime` (15000 ms default), realistically 1-2 seconds per phone when healthy. With the two phones actually live on this rig today, a full capture cycle is roughly 3-5 seconds; extrapolating to the full 19-board inventory (unverified) at 1-2 seconds/phone serial reads gives roughly 20-40 seconds per full-rack capture.

Turning the phone into a queryable sensor

Custom Shortcut as an on-phone probe answering through the clipboard

NEEDS SHORTCUT

WITH CAVEATS

Neither the XP nor the Phonefarm device record carries battery level, free storage, SSID, carrier, lock state or GPS. A Shortcut authored by hand on each phone — Get Battery Level, Get Network Details, Get Current Location, ending in Copy to Clipboard — turns Shortcuts into a sensor API the vendor never exposed: fire it from the PC, read the answer with `/shortcut/clipboard/get`. This is the only documented mechanism on this stack that runs arbitrary phone-side code and hands a result back without a jailbreak. Both possible triggers are unproven on this rig, so feasibility here is binary, not a gradient, until one of them is confirmed.

HOW

1. `get_device_list` XP `GET /api/device/get` — enumerate the rack and build the join key every probe reading files under; confirms the absence of battery/storage/SSID/carrier fields that justifies this entry
2. `shortcut#24` XP `GET /api/shortcut/device/ip?id=<one id>&state=0` — prove the vendor Shortcut round trip works before adding your own Shortcut to the picture
3. `local` XP `(manual, no API)` — author the probe Shortcut on the phone by hand, run once with every permission prompt answered, ending in Copy to Clipboard
4. `shortcut#13` XP `GET /api/shortcut/exec/url?id=<id1,id2,…>&url=shortcuts://run-shortcut?name=FarmProbe` — preferred trigger; unverified whether iOS honors the scheme from the vendor Shortcut's Open URL action
5. `send_key (fn_key)` XP `GET /api/key/sendkey?id=<id1,id2,…>&fn_key=<chord>` — fallback trigger via a spare AssistiveTouch binding; unverified whether the kernel forwards a chord absent from its published hotkey table
6. `local` XP `(client-side sleep)` — wait roughly 2-3 seconds for a local-sensor Shortcut; there is no completion callback on XP
7. `shortcut#11` XP `GET /api/shortcut/clipboard/get?id=<one id>&outtime=<ms>` — read the answer, one phone per request
8. `shortcut#10` XP `GET /api/shortcut/clipboard/set?id=<id1,id2,…>&text=` — scrub the pasteboard, one request for the whole rack
9. `get_device_screenshot` XP `GET /api/pic/screenshot?id=<one id>&binary=true` — optional visual confirmation of a Shortcuts banner or a stuck "Allow" prompt, since there's no OCR to read either

NEEDS

- The vendor Shortcut fully installed, IP-edited and bound, confirmed by the `/shortcut/device/ip` round trip

- A self-authored probe Shortcut per phone, run once by hand with the screen unlocked so permission prompts (Location especially) are answered
- For the chord fallback only: a spare AssistiveTouch binding that does not collide with the vendor's own, plus Full Keyboard Access ON

WATCH OUT

- Until one trigger is confirmed, there is no third API-only path to run phone-side code — if neither fires, this collapses to reading the vendor's own shortcut functions, none of which expose battery, storage, SSID or GPS. Treat the whole idea as conditional on a same-day acceptance test on one phone, not something to schedule into production first
- The chord fallback reasons by analogy from the Phonefarm edition's own `send_text` chord (CTRL+ALT+SHIFT+WIN+v) to argue the board emits 4-modifier chords at all — that only proves the vendor's own Shortcut consumes that specific chord on that edition, not that an arbitrary unpublished chord passes the XP kernel's dispatcher unfiltered. Weaker evidence than it sounds
- Fan-out is asymmetric: both triggers and the scrub take a comma id list; the harvest is single-id, so one request starts the rack and N requests read it back
- There is no completion callback on XP (no `send_text`, no `send_text_ok`) — a slow Shortcut and a dead one look identical until the read returns stale text or times out with code 30. Put a nonce or timestamp in the payload and reject answers that don't match the cycle just triggered
- `outtime` is milliseconds (15000 default); `clipboard/set`'s `sleep` is seconds — mixing the two silently gives a 15 ms or a 15,000 second timeout
- The probe overwrites the same pasteboard the vendor's own Unicode-paste flow (clipboard/set then WIN+v) depends on — never run a probe cycle concurrently with a paste-driven script on the same phone
- A rack-wide sweep with several stalled phones (unanswered prompt, no Wi-Fi) pays the full 15 second timeout per stalled phone if reads are issued serially; read concurrently across phones or the trigger's free cost is masked by a slow harvest
- This is legitimate first-party device administration on hardware you own, not a platform-evasion technique — the genuine risk items are the unauthenticated LAN and retaining GPS, carrier or SSID as identifying data

SCALE

Trigger cost is one request for the whole rack on either route, walking every listed phone in parallel. Harvest cost is one request per phone, each blocking up to `outtime` but typically returning in the Shortcut's own runtime — a local-sensor probe should complete in 2-3 seconds; anything using network or GPS is bounded by the phone's own connectivity, so raise `outtime` rather than assume a timeout means failure. Concurrency of the shortcut path itself has never been measured on this rig, only screenshot polling (512 req/s aggregate, no cross-interference) — read concurrently across phones, strictly serially per phone. A 10-15 minute sweep interval is comfortable; a per-minute sweep is not, since every cycle steals the pasteboard from anything else that pastes.

Ad and storefront verification

Ad verification and brand safety capture from real devices in real geos

NEEDS PLUG-IN

WITH CAVEATS

Turns the rack into an ad-verification panel: open a target app or deep link on each phone, step the feed with a single fan-out scroll request, and pull one raw-BMP frame per stop that captures the creative together with the content around it and your own timestamp. Ad-platform delivery reports confirm an impression was counted; they say nothing about what the creative looked like on a real iPhone, on a real carrier, in a real place, or what ran next to it, which is exactly what a mirror frame is. The rig's split network path, mirroring over Wi-Fi while the phone's own internet rides cellular, means the capture channel never contaminates the identity being tested. Recognition (matching the creative, reading a "Sponsored" label) is the gated half: the image-search plug-in is unconfirmed and the OCR plug-in is confirmed absent on this rig, so matching runs client-side on the decoded frame today.

HOW

1. `get_device_list + get_usb_list` XP `GET /api/device/get?id=<id> and GET /api/config/usb/get` — confirm the phone is mirroring (air_pid) and controllable (board state 2), and read imgw/imgh (500x1080) for the coordinate space.
2. `shortcut#24` XP `GET /api/shortcut/device/ip?id=<id>&state=1` — record the phone's public IP (state=0 for LAN, no Shortcut needed) and stamp it with your own clock.
3. `send_key WIN+h` XP `GET /api/key/sendkey?id=<ids>&fn_key=WIN+h`, sent twice — normalise to a known home screen before reusing a coordinate across phones.
4. `mouse_reset_pos` XP `GET /api/mouse/reset?id=<ids>` — clear the silent stuck-click state that lets /mouse/move keep working while clicks do nothing.
5. `shortcut#13 / click` XP `GET /api/shortcut/exec/url?id=<ids>&url=<url> or GET /api/mouse/click?id=<ids>&x=&y=` — open the app under test, by deep link or icon tap.
6. `mouse_wheel` XP `GET /api/mouse/wheel?id=<ids>&direction=down&len=60&number=2` — step the feed by a fixed increment in one request for the whole panel (iOS behaviour unverified, see Watch out).
7. `get_device_screenshot` (polled) XP `GET /api/pic/screenshot?id=<id>&binary=true` — poll until mean absolute pixel difference exceeds the idle floor, not a fixed sleep, to confirm a genuinely new frame.
8. `get_device_screenshot` XP `GET /api/pic/screenshot?id=<id>&binary=true (+ jpg&save_path for archive)` — capture the evidence frame, one request per phone.
9. `find_image_ex / find_image` XP `POST /api/pic/find-image-cv {img_list, similarity:0.8}` — locate the creative in the frame (plug-in gated).
10. `ocr / find_image_ex` XP `POST /api/pic/find-text {text:["Sponsored","Ad"]} or /api/pic/ocr` — read the disclosure label (plug-in gated, unavailable on this rig).
11. `(orchestration)` XP `repeat steps 6–9` — loop stops then phones; screenshots are one request per phone, input is one request for the whole panel.

NEEDS

- Vendor console (IMConsole.exe) running and logged in — without it phones show state 0 and the board list is empty.
- Per-phone iOS accessibility settings none of this API can query: AssistiveTouch on, Full Keyboard Access on, Auto-Lock Never, portrait lock.
- Board bound and cabled with the vendor cable, phone attached only after the board enumerates.

- Mouse parameters present (location_crc non-empty), or every mouse call returns data.code 11.
- iMouse Shortcut installed and bound, for the public-IP read and the deep-link route only.
- Image-search plug-in for in-API matching; the OCR plug-in is confirmed absent (code 28) on this rig.
- A SIM per phone plus the static-IP-without-gateway Wi-Fi setting, for a genuine per-geo egress.
- Accounts you own or are authorised to use, logged in normally.
- A reused per-poller buffer: un-pooled 1.62 MB frames measured 488 MB/s of allocation and 496 gen-2 GCs in 8 s.

WATCH OUT

- The capture spine (steps 1-8, wheel behaviour aside) is fully native on this rig today; only recognition and public IP are gated.
- /pic/ocr answers code 28 here; assume image-search is in the same state until probed. Decode the BMP and match client-side instead — the frame is already in the API's own coordinate space.
- The public-IP call needs the Shortcut and a working USB board (the hotkey rides USB) and defaults to a 15 s timeout, making it the slowest call in the loop — read it once per session, not once per stop.
- No carrier, SIM, locale or timezone field exists anywhere; geo is inferred from the public IP plus your own SIM-to-slot mapping. No server timestamp exists either — every time in the record is your controller's clock.
- Whether /mouse/wheel actually scrolls an iOS feed through AssistiveTouch is unmeasured on this rig. A raw wheel HID event has no guaranteed mapping to a swipe gesture, so default to /mouse/swipe on the short axis (~0.6 ms/px, ~300-500 ms/stroke) until a probe proves the wheel path works, rather than building the throughput model on it.
- Screenshots and recognition are single-id ("Multiple is not allowed"); only /mouse/*, /key/* and /shortcut/exec/url fan out to a comma-separated list.
- Frame rate is per-phone and not fixable in software: one XR here delivers ~29 new frames/s under motion, its identical twin ~4.5 — settle waits must be frame-driven, not a fixed sleep.
- Templates must be cropped from the mirror stream itself, since AirPlay re-compresses everything — match at similarity ~0.8, not ~0.95, and re-cut if the encoder ratio or display settings change.
- `save_path` chooses the directory only; the kernel names the file. Rename client-side and keep a manifest or the archive is unattributable.
- Ad platforms treat device-farm impressions as invalid traffic, a sharper risk than a plain ToS breach: take the minimum impressions needed to verify, never click the ad being verified, never run this as a volume loop, and pause on any block or rate-limit interstitial rather than working around it. Repeated automated scrolling from the same account can also simply reduce fresh ad delivery over time through ordinary frequency capping, a functional failure distinct from account risk. Get written authorisation naming accounts and campaigns before verifying on a client's behalf.
- Captured frames contain other people's names, faces and message previews next to the creative — that is personal data; scope retention, restrict access and crop or redact before anything leaves the organisation.
- The API has no authentication on the LAN; firewall 9911 and 17000 to trusted hosts.

SCALE

Transport is never the constraint: raw capture is 2 ms and 84 req/s on one phone, and phones are fully independent, up to 512 req/s aggregate across ten concurrent pollers with no contention from input traffic. The frame-driven settle sets the real pace instead, roughly 0.3-1.0 s per stop per phone, with the slowest-frame-rate device in the panel dominating the timing. If wheel scrolling turns out not to work and the design falls back to swipe, add ~300-500 ms of pointer-walk time per stop on top of the settle, which roughly halves throughput to

an estimated 200-400 ad-stop captures per hour per phone across 19 phones — still far inside the kernel's own ceiling and gated by ad-serving cadence rather than the API. Storage is the real cost: raw BMP is 1.62 MB per stop (~920 MB for 19 phones x 30 stops), so archive as JPEG (~96 KB, capped ~21 req/s) or crop with `rect` and keep the raw frame only for stops that matched; never use JPEG on the loopback analysis path.

App Store ranking, rating and review monitoring

NEEDS PLUG-IN WITH CAVEATS

Drives each phone to an App Store product page or the Search tab, lets the mirror settle, then reads star rating, review count and "What's New" text, or locates your icon among search results, from a cropped region of the frame. Every screenshot is kept as the dated record, so a rank change or a rating drop comes with an image of the page rather than a figure from a vendor's model. On this rig the navigation half works today; the reading half does not — /pic/ocr answers code 28 (plug-in not connected), so recognition has to run client-side on the raw BMP or wait on the vendor plug-ins.

HOW

1. `send_key (WIN+h)` XP `GET /api/key/sendkey?id=<id1>,<id2>&fn_key=WIN+h`, twice — normalise to a known home screen.
2. `mouse_reset_pos` XP `GET /api/mouse/reset?id=<id1>,<id2>` — clear the stuck-click state on the whole target set.
3. `shortcut#13 Open URL` XP `GET /api/shortcut/exec/url?id=<ids>&url=https://apps.apple.com/us/app/id<appid>` — deep-link to the product page (needs the Shortcut).
4. `click` XP `GET /api/mouse/click?id=<id>&x=&y=` — tap the App Store icon instead; works with no Shortcut at all.
5. `get_device_screenshot` XP `GET /api/pic/screenshot?id=<id>&binary=true` — capture the rating / review-count / What's New block, one request per phone.
6. `ocr` XP `GET /api/pic/ocr?id=<id>&rect=[0,250,500,420]` — read the block; blocked on this rig today (code 28).
7. `click` XP `GET /api/mouse/click?id=<id>&x=&y=` — open the Search tab and focus the field; coordinates must be measured per phone and iOS build, never assumed.
8. `send_key (key=, ENTER)` XP `POST JSON {fun:"/key/sendkey", data:{id, key}} then GET /api/key/sendkey?id=<id>&fn_key=ENTER` — type the keyword as POST JSON so spaces survive, then submit.
9. `find_image / find_image_ex` XP `POST /api/pic/find-image-cv {id, img_list, similarity:0.85, same:true}` — locate your icon among results (plug-in gated).
10. `mouse_wheel` (or `swipe`) XP `GET /api/mouse/wheel?id=<id>&direction=down&len=30&number=1` — scroll the result list and repeat until found or exhausted.
11. `(client-side)` XP `none — logic over the frames from steps 5/9` — reconstruct rank; no endpoint returns a list index or scroll offset.
12. `get_device_screenshot` XP `GET /api/pic/screenshot?id=<id>&jpg=true&save_path=` — archive the dated evidence frame.

NEEDS

- Vendor console running and logged in, with license capacity for the devices used.
- Each phone mirroring (live air_pid, an ESTABLISHED socket on TCP 17000) and controllable (board state 2, non-empty location_crc).
- iOS accessibility settings on every phone: AssistiveTouch, Full Keyboard Access, Auto-Lock Never, portrait lock, vendor cable only.

- Each phone signed into an Apple ID for the storefront being monitored — the API cannot set or read the storefront.
- iMouse Shortcut installed and bound, for the deep-link route only.
- OCR and image-search plug-ins connected, for the reading steps — currently absent, /pic/ocr returns code 28.
- Per-phone, per-iOS-build calibration captured once: App Store icon, Search tab, search field and rating-block coordinates.
- Competitor icon templates cropped from the same model at the same AirPlay compression ratio.

WATCH OUT

- Blocking today: /pic/ocr returns code 28; assume the image-search plug-in is in the same state until probed. Three ways out in order of cost: install the vendor plug-ins; run OCR and template matching client-side on the raw BMP; or the undocumented /shortcut/other/ocr-family fallbacks (Shortcut v1.3+) — probe before designing around them.
- Every /pic/* call is single-device; only /mouse/*, /key/* and /shortcut/exec/url take a comma-separated id list.
- Coordinates are mirror pixels (500x1080 here), never iOS points (414x896), and are per-phone and per-build — the two XRs on this rig have different home screens, so a shared coordinate hits a different app on each.
- The storefront follows the Apple ID, not the phone's network or IP — monitoring N storefronts needs N phones signed into accounts for those regions; the API cannot change it.
- `outtime` on Shortcut calls is milliseconds on XP (default 15000), not the seconds figure in the Phonefarm doc.
- `key=` is ASCII only; non-Latin keywords need clipboard/set plus WIN+v, which reintroduces the Shortcut dependency. Send multi-word keywords as POST JSON, not a GET query string.
- No endpoint returns a search-result index — rank is a client-side reconstruction over scrolled frames, and the paid slot at the top must be counted separately or every organic rank is off by one.
- An ENTER press does not by itself prove the results page loaded: App Store shows live autocomplete suggestions while typing, and a frame taken right after ENTER can still show that dropdown. Settle detection needs a second, positive check, such as the ranked-card layout region, not just "the diff stopped moving".
- Search ranking and suggestions are personalised to the signed-in Apple ID's install and search history — a farm account with an unusual history may not reproduce what a real prospective customer sees. State that scope limit next to any published rank number.
- Nothing proves a page finished loading except pixels; use mean-absolute-pixel-diff, never MD5 or byte equality — an untouched mirrored phone never repeats a frame byte-for-byte.
- Apple's terms prohibit automated scraping of listings and search results, and the account signed into a phone doing this at volume is exposed to search throttling or challenge interstitials. Treat any unexpected screen as a stop condition for that phone's run, not something to retry through, and keep cadence well below anything that would stress Apple's endpoints. The API has no authentication on the LAN.

SCALE

Bounded by UI settle time and by each phone's own AirPlay frame rate, not by the API — raw capture is 2 ms and phones are fully independent, up to 512 req/s aggregate measured. Budget roughly 3-5 s for a rating read and 10-15 s per keyword end to end (open, tap, type, settle, one or two scroll-and-match rounds); the same panel spread applies here as elsewhere on this rig, roughly 29 fps versus 4.5 fps on otherwise-identical phones.

Navigation and input fan out to N phones in one request, but screenshot and recognition are single-id, so the capture-and-read half runs as N concurrent requests — on this rig that is 2 phones actually mirroring, not the full 19 boards enumerated. With a full rack, tens of keyword checks per phone per hour is comfortable and the practical ceiling is what the store tolerates before throttling, not what the kernel delivers; at 2 live phones, expect on the order of 8-15 keyword-rank checks per hour per phone before that limit becomes the binding concern.

Network identity and provenance

Per-phone public IP truth table with carrier IP rotation

NEEDS SHORTCUT WITH CAVEATS

Builds an auditable per-slot record of the network identity each phone presents to the internet: the kernel's own LAN IP, the phone's self-reported LAN and public IP via the Shortcut, and, on cellular-routed phones, a forced IP churn by cycling the cellular radio off and on. The vendor's static-IP-without-gateway Wi-Fi setting is what makes this possible — AirPlay mirroring stays on the LAN while all internet traffic leaves over the SIM, so a phone's public IP can change without its mirror ever dropping. The output is provenance, not evasion: a log of device, timestamp, LAN IP, public IP and egress path attached to every geo-sensitive measurement a phone produced, so a run can be replayed knowing exactly where it came from and aborted the moment a phone silently falls back to the wrong path.

HOW

1. `get_device_list` XP `GET /api/device/get` — inventory phones and their kernel-observed LAN address.
2. `(Shortcut health probe)` XP `GET /api/shortcut/device/ip?id=<id>&state=0` — ask the phone for its own LAN IP; a match against the kernel's value proves the board hotkey, the Shortcut and the LAN round trip all work.
3. `shortcut#24` XP `GET /api/shortcut/device/ip?id=<id>&state=1` — read the public (extranet) IP per phone.
4. `shortcut#21/#20` XP `GET /api/shortcut/switch/cdpd?id=<ids>&state=0`, wait ~5 s, then XP `state=1` — cycle the cellular radio to force carrier IP churn; the only toggle that leaves Wi-Fi and the AirPlay mirror untouched.
5. `shortcut#24` XP `GET /api/shortcut/device/ip?id=<id>&state=1` — re-read and log the public IP against the pre-toggle value.
6. `(avoid)` XP `/api/shortcut/switch/flight and /api/shortcut/switch/wlan` — documented but wrong for this job: both drop Wi-Fi and blind the phone, killing the mirror and the Shortcut's own return path together.
7. `get_usb_list / get_device_list` XP `GET /api/config/usb/get and /api/device/get` — health gate: confirm board state 2 and a live air_pid before recording any Shortcut non-answer as a network fact.

NEEDS

- iMouse Shortcut installed and bound on every phone (Advanced permissions, kernel IP edited in, an AssistiveTouch custom key) — not verified present on this rig; no /shortcut/* call has ever been probed here.
- A SIM with an active data plan per phone — not verified on either live XR; without one, only the LAN round-trip health probe is exercisable.
- Wi-Fi set to a static IPv4 address and subnet mask with the router/gateway field left empty, per phone.
- Board bound and physically attached at state 2 — the Shortcut hotkey rides the board.

- Phone unlocked, portrait, Auto-Lock Never, AssistiveTouch on, Full Keyboard Access on.
- IMConsole.exe running and logged in.
- Kernel reachable from the phone on TCP 9911 — restrict that port at the firewall to trusted hosts.

WATCH OUT

- Nothing in this chain has actually been probed on this rig — the 10 Sep live probe covered device/get, usb/get, group/get, ocr and screenshot only. Run one live /shortcut/device/ip?state=0 call and confirm code 0 before scheduling anything on top of it.
- The `id` parameter shown in the shortcuts doc examples is a MAC address, not necessarily the same deviceid used by /device/get and /mouse/* — confirm which identifier this kernel actually expects before wiring deviceid straight through.
- No SIM is confirmed on either live phone, so the headline capability, distinct per-phone public IP and carrier rotation, is currently a paper capability here: steps 4-5 are inert, and step 3 reports the same shared building address for both phones, exactly the "one vantage point wearing twenty hats" failure this chain exists to catch.
- /shortcut/device/ip is single-id only, so a full audit is N sequential round trips of up to `outtime` (15 s default) each. The switch endpoints do fan out to a comma-separated list.
- A radio re-attach is not guaranteed to obtain a new address, and CGNAT means several SIMs on one carrier can legitimately share one public IP — an unchanged reading is not by itself proof the toggle failed. Retry a small bounded number of times, then record the result rather than looping.
- No carrier, ASN, country or geo field exists anywhere in the API; any such enrichment is an external lookup on the returned IP, run on the PC.
- A mismatch between the kernel's `ip` field and the Shortcut's own reading is not by itself a fault — a DHCP lease renewal can legitimately change a LAN IP between two reads. Treat a mismatch as a prompt to re-read /device/get, not an automatic alarm.
- `outtime` is milliseconds on XP (default 15000) and seconds in the Phonefarm doc — a value copied across editions is off by 1000x.
- Repeated cellular re-attach cycling is externally visible to the carrier's network as anomalous device behaviour, independent of anything the API does; budget it as an infrequent provenance stamp, not a polling primitive.
- Scope: this chain is for observing and recording egress identity for measurement provenance, logging which vantage point produced which data, not for concealing the relationship between accounts or evading platform limits. The API has no authentication on the LAN, so the resulting device/MAC/IP log is identifying data and should be kept out of version control.

SCALE

The IP read is the bottleneck: single-id only, one round trip per phone bounded by a 15 s default `outtime`, realistically 1-3 s when healthy — a 19-slot audit is roughly 20-60 s issued concurrently, or 1-5 minutes serialised, since the kernel showed no shared bottleneck at up to ten concurrent pollers (512 req/s aggregate). Rotation itself is cheap and fans out: cdpd off then on for the whole cellular set is two calls plus a ~5 s settle, about 10-15 s regardless of rack size, so a full audit-rotate-reaudit cycle is dominated by the two single-id read passes. As documented today this rig can run zero verified carrier-rotation cycles per hour, since no SIM is confirmed; once a Shortcut and SIM are confirmed live, a sane cadence is a handful of rotations per hour per

phone, an occasional provenance stamp rather than a streaming signal, both because re-attach cycling should stay low-rate and because none of this competes with screenshot polling or pointer input running on other macros.

Outside the box: the rack as a passive sensor

Notification Center sweep: harvest new alerts on every phone without opening an app

PARTIAL

WITH CAVEATS

One hotkey pulls down Notification Center on every listed phone at once, one screenshot per phone captures the stack, and a client-side pixel diff against the previous sweep says which phones have something new — no app-specific coordinate map, no foregrounding of any app, and because the whole loop uses only keyboard and screenshot calls, structural immunity to the stuck-click trap that silently kills /mouse/click. Notification Center is the one screen where iOS already aggregates every app's pushes, so detecting that something arrived is fully native on this rig; reading what a card says is not, since the OCR plug-in is absent (code 28) and needs either the plug-in, the Shortcut OCR fallback, or host-side OCR on the saved frame.

HOW

1. `send_key (WIN+h)` XP `GET /api/key/sendkey?id=<ids>&fn_key=WIN+h`, twice — normalise the whole target set to a static home screen.
2. `send_key (NoticeBar)` XP `GET /api/key/sendkey?id=<ids>&fn_key=NoticeBar` — pull down Notification Center on every phone in one request; an XP-only hotkey with no Phonefarm equivalent.
3. `(client-side wait)` XP `none` — settle ~400-600 ms for the pull-down animation, more on slow-frame-rate phones.
4. `get_device_screenshot` XP `GET /api/pic/screenshot?id=<id>&binary=true` — capture one frame per phone.
5. `(client-side diff)` XP `none` — mean-absolute-pixel diff of the stack region against the previous sweep; above roughly 5 means it changed.
6. `ocr / find-text` XP `GET /api/pic/ocr?id=<id>&rect=` — read card text; optional, blocked today (code 28), host-side OCR on the saved BMP is the dependency-free route.
7. `find_multi_color` XP `POST /api/pic/find-multi-color` — classify the source app by icon colour without OCR; plug-in status undocumented and unverified.
8. `mouse_wheel` XP `GET /api/mouse/wheel?id=<ids>&direction=down&len=60&number=3` — scroll a longer stack and re-capture; iOS honouring the wheel here is unverified.
9. `send_key (WIN+h)` XP `GET /api/key/sendkey?id=<ids>&fn_key=WIN+h` — dismiss and return home; notifications are not consumed by viewing, so the next sweep still sees them.
10. `mouse_reset + click` (optional) XP `GET /api/mouse/reset then GET /api/mouse/click?id=<id>&x=&y=` — escalate by tapping the top card to deep-link into the originating app.

NEEDS

- IMConsole.exe running and logged in; per-phone board bound at state 2, since NoticeBar is a HID hotkey through the board.
- Per-phone video channel ready (state != 0, live air_pid) — the screenshot comes from the mirror, not the board.
- Full Keyboard Access ON — without it iOS ignores fn_key combos entirely and the sweep silently does nothing.

- AssistiveTouch, Auto-Lock Never, portrait lock — none of these is queryable over the API.
- Phones unlocked; a locked phone's Notification Center is unreadable and the API must not be used to enter a passcode.
- iOS notification settings: previews shown, and grouping off per app if one card per event is needed.
- For text reading only: the OCR plug-in connected, the Shortcut OCR fallback, or a host-side OCR library able to decode the bottom-up 500x1080 BMP.
- Firewall 9911 and 17000 to trusted hosts before any sweep starts writing notification content to disk.

WATCH OUT

- Split the claim: "which phones have something new, and when" is fully native; "what it says" is not — OCR is blocked (code 28) on this rig today.
- NoticeBar exists only in the XP edition; a Phonefarm-edition rig would need an undocumented top-edge drag instead.
- Whether /mouse/wheel scrolls the Notification Center stack, and whether a second WIN+h reliably collapses it on iOS 18.7.10, are both unverified — fall back to a down/move/up drag or /mouse/swipe if the wheel does nothing.
- The NoticeBar / WIN+h / re-diff sequence has never been run end to end on this rig, and unlike a stuck click there is no equivalent of /mouse/reset to independently prove a stuck hotkey — run one live cycle before scheduling anything.
- Grouped notifications collapse many events into one card with a count; the pixel diff still fires but the count is unreadable without OCR.
- Notification Center is a rolling window — a sweep interval longer than a busy app's retention loses events. This is a sampler, not a guaranteed-complete log.
- /pic/screenshot is single-id, so captures scale with phone count even though the two hotkeys that open and close the panel are one request each regardless of rack size.
- At full-rack scale, wall-clock time is dominated by the single slowest phone; a 19-phone rack could plausibly contain several units as slow as the ~4.5 fps XR measured here, stretching any sweep-time estimate.
- Previews are among the most sensitive content a phone displays: names, order numbers, amounts, addresses, verification codes. Crop to the stack region rather than archiving whole screens, retain briefly and actually delete, keep the archive off shared or synced storage, exclude one-time codes from any log entirely, and sweep only phones and accounts you own or administer. The API has no authentication on the LAN, so anything on the subnet can pull the same content — firewall 9911 and 17000.

SCALE

Two hotkey requests address the whole rack regardless of size, so opening and closing the panel is one request each; capture is one request per phone but cheap — 19 phones at ~2 ms raw BMP each is ~40 ms of API time, and phones measured fully independent, up to 512 req/s aggregate across ten concurrent pollers with no contention from input traffic. Wall-clock cost is dominated by the ~400-600 ms animation settle plus extra time on slow-frame-rate phones, landing a full-rack sweep at roughly 1-2 s and repeatable every 30-60 s indefinitely, on the order of 60-120 sweeps per hour per phone for the native change-detection signal alone. Adding OCR identification, once unblocked, adds a serialised single-id read of unknown latency per phone that fired, so budget several seconds per changed phone for the read-and-identify half rather than treating it as free.

Push-delivery latency measurement by high-rate banner-strip polling

PARTIAL WITH CAVEATS

Turns the frame poller into a stopwatch: normalise to the home screen, then poll a narrow strip at the top of the mirror at 20-30 Hz per phone and flag the instant a banner slides in by a jump in mean-absolute-pixel difference against the phone's own idle floor. Differenced against your push provider's NTP-synced send timestamp, this yields an end-to-end delivery-observation time per phone, per SIM, per network path, with nothing installed on the phone and no taps at all, which makes it structurally immune to the stuck-click failure mode that affects every tap-based macro on this rig. Identifying which specific push arrived is a slower, separate step gated by the same absent OCR plug-in as the notification sweep.

HOW

1. `get_device_list` XP `GET /api/device/get?id=<id>` — confirm mirroring (air_pid) and read imgw/imgh for the strip's coordinate space.
2. `send_key (WIN+h)` XP `GET /api/key/sendkey?id=<ids>&fn_key=WIN+h`, twice — normalise every phone to a static home screen so the idle diff floor is real.
3. `get_device_screenshot` (rect+binary) XP `GET /api/pic/screenshot?id=<id>&binary=true&rect=[0,0,500,140]` — poll the banner strip and timestamp each response; rect and binary together are undocumented and unmeasured on this rig.
4. `(client-side)` XP `none` — mean-absolute-pixel diff detects arrival; never hash-compare, since an untouched mirror never repeats byte-for-byte.
5. `(external)` XP `none` — record the push provider's own NTP-synced send timestamp; embedding a sequence id in the banner body lets step 6 confirm identity rather than infer it from send order.
6. `send_key (NoticeBar) then ocr` XP `GET /api/key/sendkey?fn_key=NoticeBar, screenshot, GET /api/pic/ocr` — identify which push arrived; blocked today (code 28) and ends the measurement window when it runs.
7. `send_key (WIN+h)` XP `GET /api/key/sendkey?id=<id>&fn_key=WIN+h` — return the phone to baseline before arming the next send.
8. `shortcut#24` XP `GET /api/shortcut/device/ip?id=<id>&state=1` — attribute each phone to a network path; public IP only, carrier is an external lookup, and the Shortcut is unverified present on this rig.
9. `(phone-side configuration)` XP `none – vendor settings page section 4` — apply the static-IP-without-gateway Wi-Fi setting so the push actually traverses the carrier without killing the AirPlay mirror.

NEEDS

- IMConsole.exe running and logged in; board bound at state 2 and Full Keyboard Access ON, or the two hotkeys are silently ignored.
- Per-phone video channel live (state != 0, air_pid, an ESTABLISHED socket on 17000) — video is the entire measurement channel here.
- Auto-Lock Never, portrait lock, StandBy off, brightness pinned — the idle pixel floor is only stable if nothing else on screen animates.
- Banner-style alerts enabled with previews shown for the target app.
- PC and push-provider clocks NTP-disciplined.
- One SIM per phone plus the static-IP-without-gateway Wi-Fi setting, for per-carrier work.
- OCR plug-in, Shortcut OCR fallback, or external client-side OCR, for the identification step only.
- iMouse Shortcut installed and bound, for the network-attribution step only.
- Poll from loopback — the 2 ms full-frame figure is a loopback number, not a LAN one.

WATCH OUT

- The measured quantity is not APNs delivery time; it is banner-render-visible-through-AirPlay latency, bundling phone-side render, AirPlay encode, Wi-Fi transit and PC decode into one unmeasured constant bias. Report it by that real name — only differential comparisons, same phone, condition A versus B, are defensible, not absolute figures.
- Resolution is bounded by the phone's own frame production, not the poller: the two XRs on this rig deliver roughly 35 ms and 220 ms per new frame respectively under continuous motion, and that gap is the phone's own Wi-Fi link, not fixable in software. Publish a per-phone error bar, never a single rack figure.
- Those fps numbers were measured under driven motion; a static idle screen is explicitly unmeasured and may resolve worse. Calibrate with a known stimulus at a known PC time before trusting any result.
- `rect` and `binary` together are undocumented and unmeasured; the "~210 KB, ~2 ms" crop figures are arithmetic, not measurement. Smoke-test it, because if `rect` is silently ignored in binary mode the poller moves 1.62 MB per frame per phone instead, which changes the whole throughput budget.
- /pic/screenshot is single-id, so N phones means N requests per tick; the comma-separated fan-out that makes /mouse and /key calls cheap does not apply here.
- The identification step (NoticeBar plus OCR) destroys the measurement window it belongs to and is blocked today (code 28). Space test sends further apart than the worst plausible latency, or embed a visual marker decodable via /pic/find-multi-color instead, itself unverified on this rig.
- A 6x fps spread between two otherwise-identical phones means no per-phone resolution figure should be assumed for a third phone — measure each phone before trusting its own latency numbers.
- Scaling the measured "ten pollers give 512 req/s" ceiling to a 19-phone rack at 20-30 Hz (380-570 req/s) is extrapolation past the largest N actually tested. Treat the throughput figures as a plan to validate, step-tested at 12, 15 and 19 pollers, not a guaranteed ceiling.
- A poll gives an interval, not an instant — record both request-issue and response-receipt clock and report arrival as a bracket. The measured p95 tail (142 ms on ten concurrent pollers) is measurement error, not a phone property.
- This is legitimate QA on your own pushes to your own test accounts on devices you own; the risks are accuracy, privacy and network exposure, not platform abuse. Use dedicated test handsets signed out of personal accounts, crop the OCR region, retain crops only as long as the analysis needs them, and avoid calling an AirPlay-observed render time "APNs latency", which would itself be a false measurement claim. The API has no authentication on the LAN, and Auto-Lock Never plus a bright screen for long runs is a real battery and thermal load on the physical rack.

SCALE

Per-phone, per-tick cost is one single-id screenshot request; measured ceilings on this rig are 84 req/s for one poller flat out, ~83-85 req/s per phone with two running concurrently and no shared bottleneck, and 512 req/s aggregate at ten concurrent pollers. A 19-phone rack at 20 Hz (380 req/s) sits inside that envelope; 30 Hz (570 req/s) exceeds the largest number ever measured here, so reserve 30 Hz for one or two phones under study and use 20 Hz rack-wide. The binding constraint may be bytes, not requests, if rect cropping does not work in binary mode: 19 pollers at 20 Hz would then move roughly 615 MB/s on loopback, untested at this rig, so pool one buffer per poller regardless. Polling faster than ~30 Hz buys nothing once a phone's own frame production tops out around 29 fps, since duplicate frames are discarded on the diff anyway. Practical throughput is bounded by test-send cadence rather than the API: spacing pushes seconds apart to keep identification

unambiguous without OCR yields on the order of tens of timing measurements per hour per phone under a conservative, provider-rate-limit-respecting cadence, not the thousands per hour the raw polling rate alone would suggest.

Web and app content harvesting via the clipboard

Mobile web crawler: Safari page text harvested through the clipboard

NEEDS SHORTCUT

WITH CAVEATS

Feed a URL list to the rack: the Shortcut's URL opener loads each page in Safari on a real iPhone, a tap focuses the body, `WIN+a` / `WIN+c` select-all and copy the rendered text, and the clipboard read returns it verbatim. Because OCR is not connected on this rig (code 28), this is the only route to the mobile-rendered text of JS-heavy sites, and it returns exact characters rather than a recognition guess.

HOW

1. `shortcut#13 Open URL` XP `/shortcut/exec/url?id=<ids>&url=<page>&outtime=<ms>` — dispatch the next queued URL to Safari; id takes a comma list but each phone still needs its own URL, so this is one call per phone per page.
2. `get_device_screenshot` XP `/pic/screenshot?id=<id>&binary=true` — poll raw BMP frames (2 ms p50) until two consecutive frames differ by a mean pixel diff below ~1, so the tap lands after the page has actually loaded.
3. `click` XP `/mouse/reset?id=<id> then /mouse/click?id=<id>&x=250&y=700` — reset first, then tap a blank margin (mirror pixels, 500x1080 here) to focus the page body without navigating away.
4. `send_key fn_key` XP `/key/sendkey?id=<id>&fn_key=WIN+a then fn_key=WIN+c` — select all, then copy; needs Full Keyboard Access on.
5. `shortcut#10 To the phone clipboard` XP `/shortcut/clipboard/set?id=<ids>&text=SENTINEL-<url-hash>-<ts>&sleep=<s>` — poison the clipboard with a per-page sentinel before each cycle so a stale read is detectable.
6. `shortcut#11 Get phone clipboard` XP `/shortcut/clipboard/get?id=<id>&outtime=<ms>` — read the copied text back into `data.text`; single-device, so this step is per phone.
7. `mouse_wheel` XP `/mouse/wheel?id=<ids>&direction=down&len=100&number=5` — scroll to force lazy-loaded content in before the next copy.
8. `none` XP `no endpoint` — link discovery is not available from this chain: copied text carries no hrefs or structure, so drive the crawl from a known URL list or a conventionally fetched sitemap.

NEEDS

- iMouse Shortcut installed on every phone, kernel IP edited in, run once with every permission granted, and bound to an AssistiveTouch key via Send Binding Key. Never verified on this rig; the cheapest proof is `/shortcut/device/ip?state=0` returning the same address as `/device/get`.
- USB board bound and connected (`/config/usb/get` state 2), since every `/shortcut/*` call and hotkey travels over it.
- Full Keyboard Access on, AssistiveTouch on with tracking sensitivity maxed, trackpad speed maxed, Auto-Lock Never, portrait locked. None of this is queryable over the API.
- Phone on the same LAN as the kernel, reaching it on TCP 9911, with mirroring up.

- Safari as the default handler for http/https; cookie and consent walls handled once per phone by a person, not the automation.
- A per-phone clipboard round trip verified once: `clipboard/set` followed by `clipboard/get` returning the identical string.

WATCH OUT

- The whole chain is a paper design until `/shortcut/device/ip?state=0` succeeds: a 15 s timeout (code 30) is indistinguishable from "Shortcut not installed" versus "page slow to load."
- Only `exec/url`, `clipboard/set`, the mouse calls and the key calls take a comma list of ids. `clipboard/get` and `/pic/screenshot` are single-device, so the per-page work is per phone, not fanned out.
- Whether `WIN+a` selects the whole document or only the visible viewport is unverified here; if it is viewport-scoped this becomes a slower scroll-and-deduplicate loop.
- Copied text has no headings, hrefs or element boundaries: it is an undifferentiated text corpus, not anything resembling extracted "content," and the crawl cannot discover new URLs from it.
- The focus tap is the fragile step: land it on a link and the page navigates before the copy runs. A blank-margin coordinate cannot always be precomputed from a URL list alone; some pages need a first-pass screenshot to find one at all.
- The clipboard is shared state: a failed copy silently returns the previous page's text unless the per-page sentinel makes the failure visible.
- robots.txt, site terms and applicable data-protection rules apply exactly as they would to any crawler; respect published rate limits and stop on an explicit block rather than working around it.
- The slow XR's settle check needs a longer window, roughly 1.5-2 s rather than 500 ms, to reach the same confidence the fast phone gets, because it only delivers about 4.5 new frames a second.
- The Shortcut round trip (hotkey to app launch to Safari automation to HTTP post back) has never been timed on this rig; a cold Shortcut launch can itself run 1-3 s, which is a large slice of any per-page budget.

SCALE

This rig has two mirroring iPhone XRs, not 19 — 19 is the USB board count. At a realistic 8-15 s per page per phone, including an unmeasured Shortcut round trip, expect roughly 8-15 pages/minute combined, on the order of 12,000-20,000 pages/day running continuously with no stalls. That is below a 20-30k/day figure that ignores Shortcut latency, and far below a 150k/day projection that assumes 19 phones this rig does not have. This is orders of magnitude below a headless crawler, so spend it only where real-Safari fidelity is the point.

Clipboard harvest of your own native-app data: listings, orders and messages

NEEDS SHORTCUT

WITH CAVEATS

Native app content is not selectable on screen and OCR is unavailable on this rig, but the apps themselves expose an exact text channel: the share sheet's "Copy link," the long-press context menu's "Copy," and `WIN+a` / `WIN+c` in an editable field. Tap the affordance, then read the string back through the clipboard: a canonical URL, an order number, a buyer's message, verbatim and with zero recognition error. Applied to your own accounts this builds a live catalogue of your listing URLs, order IDs, tracking numbers and inbound messages with no transcription errors.

HOW

1. `send_key + mouse_reset_pos` XP `/key/sendkey fn_key=WIN+h (twice) then /mouse/reset?id=<comma list>` — bring every target phone home and clear the stuck-click state before starting.

2. `shortcut#13 open URL` XP `/shortcut/exec/url?id=<comma list>&url=<https or custom scheme>` — deep-link straight to your own account's profile, order list or thread without navigating the UI by hand.
3. `get_device_screenshot` XP `/pic/screenshot?id=<id>&binary=true` — locate the share, order-number or message target from a stored per-app coordinate table, since neither OCR nor image search is available to re-anchor a drifted layout.
4. `click` XP `/mouse/click?id=<id>&x=&y=&button=1` — tap the share control, then "Copy link" in the sheet; wait for the sheet animation before the second tap.
5. `click with time` XP `/mouse/click?id=<id>&x=&y=&time=800 then Copy` — long-press variant for captions, order numbers and message bubbles; `time` is documented as the press-to-popup delay, so no separate API is needed.
6. `shortcut#11` XP `/shortcut/clipboard/get?id=<id>&outtime=<ms>` — read the string off the phone into `data.text`; single-device.
7. `shortcut#10` XP `/shortcut/clipboard/set?id=<comma list>&text=<sentinel>&sleep=<s>` — clear the clipboard with a unique sentinel between items so a missed tap is detectable and no data lingers.
8. `mouse_wheel / swipe` XP `/mouse/wheel?id=<ids>&direction=down&len=90&number=2` — advance through your own order or listing history and confirm the next item is in position before the next copy.

NEEDS

- iMouse Shortcut v1.3+ installed on each phone, IP-edited, run once with every permission granted, and bound to an AssistiveTouch key. Not installed on this rig; `/shortcut/clipboard/get` is the load-bearing call and nothing works without it.
- Board bound and connected, since the Shortcut is triggered by a HID hotkey over it; a video-only phone cannot run this at all.
- Full Keyboard Access on for the field-copy variant; AssistiveTouch on with tracking sensitivity and trackpad speed maxed for the taps.
- Your own accounts logged in on each phone.
- A per-app, per-version coordinate table for the share icon, "Copy link" row and context-menu "Copy," re-measured after every app update.
- Handoff / Universal Clipboard turned off on rack phones, or every copied string syncs to the owner's other Apple devices.

WATCH OUT

- `/shortcut/clipboard/get` is single-id, so the harvest step is strictly per-phone even though the tap, the clipboard write and the URL open fan out over a comma list.
- The chain depends entirely on the iMouse Shortcut being installed and hotkey-bound, which it is not on this rig; a timeout surfaces as generic code 30, not as "not installed."
- `outtime` is milliseconds on XP (default 15000), while `sleep` on `clipboard/set` is seconds; swapping them produces either instant timeouts or 15-second stalls per item.
- No OCR and no image search on this rig, so every target must come from a stored coordinate table plus client-side pixel checks; an app update silently invalidates the table with no fallback to re-anchor it.
- iOS shows a "pasted from" banner on every clipboard read, visible in captured frames and sometimes to the phone's own user; it does not block the read but does make the activity visible on-device.
- The 15 s default `outtime` is a ceiling, not a target: if the bound hotkey misfires, each item silently costs up to 15 s instead of the assumed 2-5 s, which can drop realised throughput 3-5x.

- Universal Clipboard syncs every copied string, including order numbers and buyer messages, to the account owner's other Apple devices unless Handoff is explicitly off on every rack phone.
- Nothing in this chain can reliably distinguish an unexpected screen — a first-run permission prompt, a re-login, a 2FA/OTP screen triggered by a cold deep-link open — from normal content without OCR, so a run can stall or mis-tap into an unrelated field rather than failing loudly; a frame-diff-only heuristic can also false-flag an ordinary app-update dialog as a block.
- High automated volumes against your own account can still trip a platform's own anti-automation detection independent of intent; treat any such signal as a stop condition. Nothing here is built to route around it, and this chain is scoped to your own accounts and your own data.

SCALE

Each item is two or three taps plus one Shortcut round trip: the API costs are negligible (a click is one request, a raw screenshot 2 ms), so the 2-5 s per item is share-sheet animation plus the Shortcut's LAN round trip, roughly 10-20 strings a minute per phone if the Shortcut integration works as designed. Across the two phones mirroring here that is 20-40 items/minute combined, but it drops to roughly 4 items/minute per phone whenever the round trip stalls toward the 15 s ceiling, and the whole figure is aspirational until a single clipboard round trip is confirmed live on this rig.

Search and pricing intelligence from real handsets

Search-result monitoring from real iPhones on real carrier connections

NEEDS SHORTCUT

WITH CAVEATS

Dispatch the same search URL to every phone in one call, screenshot each first-result screen to a dated folder, and copy the page text through the clipboard so the order of domains can be logged as each device's own connection sees it, including the ad slots and local pack a rank-tracking API never returns. Each phone's egress IP is recorded alongside the sample so every result is labelled with the vantage point it came from.

HOW

1. `shortcut#13 Open URL with devlist` XP `/shortcut/exec/url?id=<id1>,<id2>,...&url=<search url>&outtime=<ms>` — open the same query on every phone in a single request; the fan-out is genuine, but page load afterwards is per-phone network time.
2. `get_device_screenshot` XP `/pic/screenshot?id=<id>&jpg=true&save_path=D:\serp\<date>\` — save a JPEG (40.5 ms p50, capped near 21 req/s kernel-wide) as the archival record; the directory must already exist or the call returns code 27.
3. `click + send_key + shortcut#11` XP `/mouse/reset -> /mouse/click -> /key/sendkey fn_key=WIN+a -> fn_key=WIN+c -> /shortcut/clipboard/get` — tap a blank margin, select all, copy, and read the text back to infer result ordering from copied domain order.
4. `mouse_wheel` XP `/mouse/wheel?id=<ids>&direction=down&len=80&number=3` — scroll to capture below-the-fold positions, then screenshot again per phone.
5. `shortcut#24 Get external network IP` XP `/shortcut/device/ip?id=<id>&state=1&outtime=<ms>` — record the egress IP each sample actually came from; single-device only.
6. `n/a` XP `/pic/screenshot (archive) + stop the queue for that phone` — if a CAPTCHA or interstitial appears, log it with a screenshot and stop querying from that device; there is no function for solving one and none should be built.

NEEDS

- Everything the clipboard-crawl chain needs: Shortcut installed and bound, board bound, Full Keyboard Access on, phone on the LAN reaching 9911, mirroring up. Unverified on this rig.
- The `save_path` directory must exist before the call, and the kernel process must be able to write it.
- For genuinely different vantage points, a SIM per phone with cellular data carrying internet while Wi-Fi carries only the mirror, using the vendor's documented static-IP-without-gateway setup so mirroring survives.
- Location Services on if local-pack results are wanted, or signed-out browsing for unpersonalised results.
- Consent dialogs on any storefront handled once per phone by a person, not by the automation.

WATCH OUT

- The geo premise does not hold on this rig as built: both XRs sit on the same Wi-Fi behind one egress, so today every phone samples from a single vantage point. It becomes real only with a SIM per phone plus the gateway-less static-IP setup, verified by `/shortcut/device/ip?state=1` returning genuinely different addresses.
- An iPhone needs a physical SIM with an active data plan per phone to make this real, a procurement cost the plan does not otherwise surface, and carrier CGNAT can mean the reported IP still doesn't match what the destination server logs.
- Automated querying of search results sits against the terms of service of Google and comparable engines. That is a legal and contractual constraint on the whole use case, not a technical one; where a first-party search API or licensed data source exists, it is the compliant route.
- When a block or CAPTCHA appears, the only supported response is to record it and stop. Nothing in this API solves, bypasses or evades bot detection, and building that is out of scope.
- Only the URL open and the wheel scroll fan out over a comma list; screenshot, clipboard read and IP lookup are all single-device, so per-phone work is serial.
- Rank inferred from copied-text domain order is approximate: ads, the local pack and organic results are not distinguishable in plain text. The screenshot is the authoritative artefact; the text is an index into it.
- "Same instant" describes the dispatch request, not a hardware-guaranteed simultaneous trigger: the kernel likely walks the id list within one HTTP call rather than firing every phone in true parallel.
- Results from a signed-in account are personalised and not comparable across devices; use signed-out Safari for a clean comparison.
- Only two phones are live today, so this is a two-sample pilot, not a monitoring fleet; scaling to a real panel needs more boards' phones brought online and the SIM/geo problem solved first.

SCALE

Dispatch is one request regardless of fleet size; everything after it is per phone. With the two phones actually online, roughly 100 keyword queries per phone at 8-15 s each, including unmeasured Shortcut latency, works out to about 15-25 minutes wall clock — slower than a simple 7-10 minute estimate, and far short of a 20-30 minute, 19-phone, 19-geo projection; 19 is the board count, and only two XRs are mirroring here.

Simultaneous multi-geo render comparison of one URL

NEEDS SHORTCUT · WITH CAVEATS

Open one identical URL on every mirroring phone with a single call, then pull one raw BMP frame per phone back-to-back at ~2 ms each so the captures land within milliseconds of one another. Each phone's public IP is recorded first as the geo column, and a client-side pixel diff, plus an optional exact-text lift through the clipboard, shows where the same page differs: a different price, a blocked asset, a consent wall, a redirect.

HOW

1. `get_device_list` XP `/device/get` — keep only phones with a live `air_pid` and identical `imgw` x `imgh`; different frame geometry makes a pairwise diff meaningless.
2. `shortcut#24 get external network IP` XP `/shortcut/device/ip?id=<id>&state=1` — record each phone's public IP as the geo label; single-device, roughly a 1 s Shortcut round trip each.
3. `shortcut#13 open URL` XP `/shortcut/exec/url?id=<id1,id2,...>&url=<target>` — the only step that is genuinely simultaneous; page load afterwards is per-phone network time.
4. `get_device_screenshot` XP `/pic/screenshot?id=<id>&binary=true` — grab one raw frame per phone in a loop; 2 ms p50 each, so the whole sweep is milliseconds even though it isn't batched.
5. `send_key + shortcut#11` XP `/mouse/click -> /key/sendkey fn_key=WIN+a -> fn_key=WIN+c -> /shortcut/clipboard/get` — optional: lift the exact rendered text (price string, banner copy) instead of guessing from pixels.
6. `get_device_screenshot (save)` XP `/pic/screenshot?id=<id>&jpg=true&save_path=D:\geo\<url hash>\` — archive the evidence frame with its IP and timestamp; the kernel writes the file and returns the path.
7. `mouse_reset_pos` XP `/mouse/reset?id=<id1,id2,...>` — clear the silent stuck-click state before any tap-driven step, across the whole target set.
8. `none` XP `no endpoint, client-side` — compare frames with mean absolute pixel difference, never MD5, and mask the status-bar band and the pointer dot before diffing.

NEEDS

- Shortcut installed, IP-edited and bound on every phone; board bound and connected. Not verified on this rig, so `/shortcut/device/ip?state=0` is the cheapest first proof.
- Phones on the kernel's LAN reaching TCP 9911, with the static-IP-without-gateway setup keeping AirPlay on the LAN while the phone's internet rides cellular.
- Each phone actually on the geo it is meant to represent, using accounts and SIMs you own or are authorised to test with, with Apple ID region, system language and login state held constant across phones.
- All phones mirroring with identical `imgw` x `imgh`; Full Keyboard Access on for the optional text lift.

WATCH OUT

- The single biggest hole is the premise, not the chain: if both phones ride the same LAN/ISP uplink, `/shortcut/device/ip?state=1` will often return the same public IP for every phone, or IPs that geolocate to the same city. Verify real IP diversity for every phone before trusting any diff, or the run produces N synchronised screenshots of one served variant that could be mistaken for "no personalisation detected."
- This rig has two iPhone XRs, not 19; 19 is the USB board count. Every 19-phone throughput figure needs rescaling to what two phones can actually do.
- Simultaneity is in the capture, not the render: the frame grabbed is the newest AirPlay frame, and the slow XR (~4.5 new frames/s) can be roughly 220 ms stale. Poll the slow phone until its frame stops changing rather than trusting a single grab after a fixed settle window.
- No OCR on this rig (`/pic/ocr` returns code 28), so reading a price off the frame needs either the clipboard lift or client-side OCR against the raw BMP.
- `/pic/screenshot`, `/shortcut/device/ip` and `/shortcut/clipboard/get` are all single-device, so the geo column and the text lift are serial per-phone Shortcut round trips at roughly a second each.
- The exact-text lift (`WIN+a` then `WIN+c`) may need a preceding long-press to raise a Select All menu on native views; plain hotkeys reliably work only on selectable web/text content.
- Evidence value depends on provenance you add yourself: store the IP, timestamp, device record and exact URL with every frame, since the API adds none of that.

- Repeated automated loads of a commercial site are scraping under most terms of service regardless of intent; pace requests, honour rate limits, and stop on any block or challenge page rather than working around it.
- Do not present a price difference as proof of geo discrimination without holding language, Apple ID region and login state constant — those confound geo far more often than IP does.

SCALE

The URL open is one request regardless of phone count, and raw capture is trivial at 2 ms per phone. The expensive parts are the per-phone Shortcut round trips, roughly 1 s each for the IP and clipboard reads, plus a settle wait conditioned on the slowest phone. A full cycle on this two-phone rig is realistically 10-20 s end to end, or roughly 180-350 URL comparisons an hour for the whole panel, bounded by the serial single-id Shortcut calls rather than by screenshot cost. A larger rack adds mostly to that Shortcut-bound stage; concurrent dispatch keeps the cycle near the same length regardless of phone count, since the control and video channels showed no shared bottleneck up to ten concurrent pollers.

Passive telemetry: training data and health signals

Training-data collection: screen, action, next-screen tuples at rack scale

NATIVE API

WITH CAVEATS

Instrument every input call so the raw frame immediately before it, the exact command and its mirror-pixel coordinates, and the settled frame after it are written as one dataset row. The action label comes from the request itself, not from perception, so it is exact and free, and the settle time comes from polling until consecutive frames stop differing. Raw BMP capture measured 2 ms on this rig, so before/after capture is essentially free, and the whole chain is native XP HTTP: no plug-in, no Shortcut, no professional edition.

HOW

1. `get_device_list` XP `/device/get` — record `imgw` / `imgh`, model, iOS version and `location_crc` once per session as row metadata; a coordinate is meaningless without the frame size it was expressed in.
2. `get_device_screenshot` XP `/pic/screenshot?id=<id>&binary=true` — capture `frame_before`; 2 ms p50, raw bottom-up BMP, single-device.
3. `click / swipe / mouse_wheel / send_key` XP `/mouse/click, /mouse/swipe, /mouse/wheel, /key/sendkey` — perform the action and log the request verbatim as the ground-truth label.
4. `(poll)` XP `/pic/screenshot?id=<id>&binary=true on a fixed period` — poll until the screen settles and record the settle time as a label; XP has no push frame stream, so polling is the only route (measured 84 req/s per phone flat out).
5. `mouse_reset_pos + pointer-box diff` XP `/mouse/reset?id=<comma list>` — verify the action actually landed by diffing a small box around the commanded coordinate before keeping the row; there is no API call that reports "the tap landed."
6. `(callback)` XP `ws://127.0.0.1:9911/api – error_push, dev_rotate, dev_connect, dev_disconnect` — drop rows whose action failed at the board, and tag session and orientation boundaries.
7. `send_key + shortcut#2` XP `/key/sendkey fn_key=shift+win+3 then /shortcut/album/get, /shortcut/album/down` — optional high-resolution label pair for a subset of rows via a native iOS screenshot; Shortcut-gated, not available on this rig today.

NEEDS

- IMConsole.exe running and logged in; each phone mirroring and bound to its board with a non-empty `location_crc`.
- iOS accessibility prerequisites on every phone — AssistiveTouch, tracking sensitivity max, Full Keyboard Access, Auto-Lock Never, portrait lock — none of which is readable over the API; verify with a per-phone input test.
- A frame differ using mean absolute pixel difference on the decoded BMP, never MD5 or byte equality.
- One reused buffer per poller: without reuse, this rig measured 488 MB/s allocation and 496 gen-2 GCs per 8 s; with reuse, 2.9 MB/s and 0-1.
- Disk budget of roughly 1.6 MB per raw frame, or ~10 KB per JPEG-encoded before/after pair after client-side encoding.
- Dedicated accounts on any app being driven, a review of that app's terms before recording its UI, and a scrubbing pass before any dataset leaves the machine.

WATCH OUT

- `/pic/screenshot` is single-id, so capture is one request per phone even though input calls fan out over a comma list; a mirrored action across N phones is one request but N capture requests.
- Settle-time labels are quantised by each phone's own AirPlay source rate, which differs even between identical XR units (~29 vs ~4.5 new frames/s here); record measured fps as row metadata rather than treating settle time as a general iOS transition-timing signal.
- An ignored tap still produces a well-formed "no change" row; without the pointer-box diff and `/mouse/reset` discipline, the silent stuck-click state contaminates the dataset with false negatives that look like legitimate no-op transitions.
- The pointer dot is baked into every frame; either keep it deliberately or mask a small box around the commanded coordinate, consistently, or a model will learn to read the cursor instead of the screen.
- The two XRs have different home screens, so mirroring one action for augmentation only produces a valid pair after normalising both phones to matching states; navigation fan-out to identical target screens is free, augmentation-by-mirroring generally is not.
- This rig has 19 boards but only 2 phones mirroring, so "rack scale" throughput figures are a projection, not a measurement; video capacity, not board count, bounds a real run.
- No text or element labels are available (OCR returns code 28), so the dataset out of this pipeline is (frame, action, frame) tuples only, not (screen_name, action, screen_name), unless a separate annotation pass adds those labels.
- Recording third-party app UI at volume for a training set carries the same terms-of-service and copyright exposure as scraping; treat dedicated accounts and a scrubbing pass as mandatory before any dataset export, not optional.

SCALE

Raw capture is not the bottleneck: 2 ms p50, 84 req/s per phone flat out, phones fully independent up to 512 req/s aggregate across ten pollers. The real cost is the action plus the settle wait: a realistic scripted or human-driven session yields roughly 600-2,000 labelled transitions an hour per phone, dominated by think time and in-app network waits rather than the near-free capture cost, so on the order of 1,200-4,000 rows an hour across the two phones mirroring today. That is an order of magnitude below a naive "one transition per second per phone" projection, and far below a 1.6M-row/day full-rack figure that assumes 19 mirroring phones this rig does not have.

Frame-rate as a signal: autoplay presence and app-hang detection

NATIVE API | WITH CAVEATS

Poll raw BMP frames while sending no input, and count how many successive frames differ by more than a mean-pixel threshold. A still screen produces almost none, an autoplaying video or animated banner produces close to the phone's own frame ceiling, and an app that has stopped responding produces none after an action that should have changed the screen. The output is a number, not a picture, so it needs no OCR, no image-search plug-in and no template library, which matters because neither recognition plug-in is connected here.

HOW

1. `get_device_list` XP `/device/get?id=<id>` — confirm a live `air_pid` and read `imgw` / `imgh`, the pixel space for frames and coordinates alike.
2. `mouse_reset_pos` XP `/mouse/reset?id=<comma list>` — clear the stuck-click state before any input-driven step, or a "frozen app" verdict can actually be a stuck pointer.
3. `mouse_move + get_device_screenshot` XP `/mouse/move?id=<id>&x=&y= while polling /pic/screenshot?id=<id>&binary=true` — calibrate this phone's ceiling by driving continuous motion and counting new frames flat out; measured 28.9/s on one XR, 4.5/s on its identical twin, so the ceiling is a per-phone property that must be measured, never assumed.
4. `send_key (WIN+h) + get_device_screenshot` XP `/key/sendkey?id=<comma list>&fn_key=WIN+h (twice) then poll /pic/screenshot` — establish the idle floor on a known-static home screen; measured ~0.02-0.03 mean abs diff frame to frame.
5. `click / swipe / shortcut#13` XP `/mouse/click, /mouse/swipe, or /shortcut/exec/url` — navigate to the screen under test; the click/swipe route keeps the whole technique native, the Shortcut route is optional.
6. `get_device_screenshot (loop)` XP `poll /pic/screenshot?id=<id>&binary=true at a fixed period` — send no input for a few seconds and count new frames as a fraction of the calibrated ceiling: near-ceiling means continuously animated content, near-zero means static.
7. `mouse_wheel` XP `/mouse/wheel?id=<id>&direction=down&len=40&number=1` — hang probe: act, then count new frames in the next second; zero after a scroll that should have moved the screen is the candidate signal.
8. `mouse_move + pointer-box diff; hotkey liveness` XP `/mouse/move then /pic/screenshot, plus /key/sendkey fn_key=WIN+h` — disambiguate "app is frozen" from "control channel is dead" before reporting a hang; a working hotkey proves the channel but says nothing about taps.
9. `(callback)` XP `ws://127.0.0.1:9911/api – error_push` — optional: subscribe to asynchronous hardware failures that otherwise look identical to a frozen app.

NEEDS

- Phone mirroring with Auto-Lock set to Never; a sleeping screen reads as a hang.
- Per-phone ceiling calibration before any verdict, re-run periodically since the ceiling is a property of that phone's live Wi-Fi link and can degrade over a session.
- A poller that reuses one ~1.6 MB buffer per phone and paces at a fixed period, recording frames on arrival rather than on a display tick.
- Nothing else touching the screen during the measurement window; the pointer dot moving, a notification banner or an auto-dimming screen all register as a frame change.
- Board bound and connected for the wheel/move/hotkey steps; the pure autoplay measurement needs only the video channel.

WATCH OUT

- On this rig the technique classifies autoplay reliably on only one of the two phones: the slower XR's ~4.5 new frames/s ceiling is well under the floor needed to separate "animated" from "slowly changing," so it

must be excluded from autoplay classification and used only for the coarser hang probe.

- This measures AirPlay mirror frame production, not the app's own render rate; a 60 fps app still can't deliver more than ~30 new frames/s over AirPlay, and a compressed stream can drop frames that did change. Treat it as a presence/absence classifier, never a performance metric.
- Never use byte equality or MD5 to decide "new frame": an untouched phone measures ~0.02-0.03 mean absolute pixel difference every frame from clock and encoder noise, so MD5 would report every frame as new on a dead-still screen.
- A zero-frame result after a wheel event has several causes beyond a hang: a stuck-click state, a screen that legitimately does not scroll (including one already fully scrolled), or a dropped mirror session. Rule those out with `/mouse/reset`, the pointer-box diff and `/device/get` state, and confirm the content actually has more to scroll before calling it a hang.
- A rack-wide figure built on 19 phones is hypothetical; 19 is the board count, and the measured envelope (512 req/s aggregate across ten pollers) is close to the edge for a genuine 19-phone fleet at 30 Hz, so stagger pollers or drop to 20 Hz if that hardware is ever added.
- The technique cannot distinguish "video paused deliberately by the app" from "app hang"; both look like a drop from near-ceiling to near-zero, so treat a rate drop as a hang candidate that still needs the independent liveness checks, not a hang signal on its own.
- A 5 s calibration window is too short on the slow phone, yielding only ~22 samples; widen calibration to 15-20 s for any phone whose running ceiling estimate is under ~10 fps, and calibrate on a plain wallpaper page, since live widgets or wallpaper motion can inflate the idle floor.
- AirPlay can buffer several frames under load, so a freeze under roughly one buffered frame interval on the fast phone may not register within a 1 s hang-probe window; treat the probe as reliable for freezes of at least a few hundred ms to a second, not instantaneous stutters.

SCALE

Per phone: 84 req/s flat out, or 2.0 ms p50 when paced; phones are fully independent, with input traffic and frame polling shown not to interfere with each other. A full check cycle — calibration, autoplay measurement, hang probe — is roughly 11-26 s, on the order of 140-330 classification cycles an hour per phone run continuously and serially, and since phones share no bottleneck those per-phone rates hold simultaneously across the rack rather than being divided among phones. A steady-state monitor that skips full recalibration can classify a screen roughly every 5 s per phone, about 700 five-second verdicts an hour per phone, well inside the measured throughput envelope; this scales with phones actually mirroring, not with the 19 boards present.

Also worth knowing

Recognition against a supplied image instead of the live mirror

NEEDS PLUG-IN · WITH CAVEATS

Six recognition functions carry a `target_img` parameter: `/pic/find-image`, `/pic/find-image-cv`, `/pic/ocr` and `/pic/ocr-ex`, `/pic/find-text` and `/pic/find-text-ex`, and `/pic/find-multi-color`. The docs describe it the same way on every one of them: "big picture of goal, generally you don't need to fill it in, our internal screen image will be used by default." Leave it blank and the plug-in reads the live 500×1080 mirror frame, the same frame every other card in this section is stuck with. Fill it in and the plug-in reads whatever file or string you hand it instead, at that image's own resolution. That is the one lever in the entire API that separates recognition from the mirror, which matters because the mirror is the document's own recurring complaint: it is

a re-encoded downscale of the XR's real panel, and small text sits at the edge of readable cap-height once it crosses that pipe. A native screenshot pulled back through the Shortcut's album chain and handed back in as `target_img` reads at full panel resolution instead. The same substitution also opens offline re-analysis of an already-archived frame corpus, and, since every call still needs a device id but that device's own screen is never touched once `target_img` is set, a queue of stored images run one at a time against whichever single bound phone is convenient.

HOW

1. `send_key (fn_key)` XP `/key/sendkey?id=<ids>&fn_key=WIN+SHIFT+3` — fire the native iOS screenshot chord (see the native-resolution capture card above for the full chain and its traps)
2. `shortcut#1` XP `/shortcut/album/get?id=<single>&num=1&outtime=<ms>` — confirm the shot landed, using its own `create_time`, not sort order
3. `shortcut#2` XP `/shortcut/album/down?id=<single>&list=[{…}]&outtime=30000` — pull the native PNG to `<install dir>\Shortcut\Media` on the PC
4. `local` XP `(client-side)` — point `target_img` at that file's path, or its contents, in place of leaving the parameter blank
5. `ocr / ocr_ex` XP `/pic/ocr?id=<id>&target_img=<file>&rect=[l,t,r,b]` — recognize text against the supplied image; `rect` must be measured in that image's own pixel space, not the 500×1080 mirror space used everywhere else in this section
6. `find_image` XP `/pic/find-image?id=<id>&target_img=<file>&img_list=[…]&similarity=0.8` — the same substitution for template matching; `find_image_ex` (XP `/pic/find-image-cv`) and `find_multi_color` (XP `/pic/find-multi-color`) take `target_img` the same way
7. `local` XP `(client-side)` — for offline re-analysis, skip steps 1-3 entirely and point `target_img` at any previously saved frame, e.g. one already written by `/pic/screenshot?save_path=…` or archived by an earlier sweep

NEEDS

- OCR and/or image-search plug-in installed and connected; on this rig all six functions return `data.code 28` today, and `target_img` changes what image the plug-in would read, not whether the plug-in is there to read it
- A valid, hardware-connected device id on every call regardless of `target_img` — nothing in the parameter tables drops the `id` requirement, so there is no documented way to recognize an image with no phone in the request at all
- For native-resolution sourcing: the full Shortcut install/IP-edit/AssistiveTouch-binding chain and Shortcuts > Advanced permissions, exactly as the native-resolution capture card above requires
- `rect` recalculated per source image — a rectangle measured against the 500×1080 mirror is meaningless against a native screenshot; read that file's actual pixel dimensions rather than assuming a fixed size

WATCH OUT

- The parameter table lists `target_img` only as type String. `img_list`'s own row on the same page spells out "base64 string image list or local path list"; `target_img`'s row does not repeat that. Whether it wants a bare base64 string, a data URI, or a path on the PC running the kernel is unstated — test with one known-good file before building anything on it
- Whether the id's device needs to be actively mirroring, or merely bound and hardware-connected, to accept a `target_img` call is untested on this rig. That answer decides whether "recognition on a phone whose video channel is down" actually works, or still fails with code 3 or 25 before `target_img` is ever read

- The Shortcut-side fallbacks named in the docs, `/shortcut/other/ocr`, `/shortcut/other/ocr-ex`, `/shortcut/other/find-image`, `/shortcut/other/find-image-cv`, `/shortcut/other/find-text`, `/shortcut/other/find-text-ex` and `/shortcut/other/find-multi-color`, are listed as v1.3+ "special case" alternates with no parameter table at all; whether they accept `target_img` or only ever read the phone's own screen is unknown
- None of this is a verified round trip on this rig: both plug-ins report code 28, so every claim above is a reading of the parameter table, not a call that has actually returned `code 0` with a supplied image
- All six functions are single-id, so feeding `target_img` fans out no better than live recognition would; a corpus of archived frames is worked one request at a time, same as a live sweep
- Codes 31 ("OCR did not recognize the text") and 32 ("No text found") are the plug-in's own outcome codes, not evidence of a resolution cause; a low-resolution mirror crop is one plausible reason recognition comes back empty, but the docs do not attribute either code to image size specifically

SCALE

`target_img` changes which image is read, not the cost of reading it: once a plug-in is connected, per-call latency should track whatever live-recognition latency turns out to be, which is undocumented on this rig either way. The only added cost is getting a native-resolution file onto the PC first, which is the same serial, single-id, per-phone album pipeline as the native-resolution capture card above (unmeasured; treat any per-shot figure there as a placeholder). Offline re-analysis of an already-archived corpus skips capture entirely and is bounded purely by plug-in throughput once one exists, so it is the cheapest of the three paths in this card once the plug-in gate is cleared.

Raising one phone's mirror resolution as a per-job lever, then dropping it back

NATIVE API

WITH CAVEATS

The mirror's pixel size is not fixed to the model, it is a per-device runtime setting. `air_ratio` in `/device/airplay/set` takes one phone from its working resolution up to 720, 1080, 1400, 1920 or 2650 for as long as a vision pass needs the extra pixels, independent of every other phone on the rack, then back down when the pass is done. That is the fix for "screen too small at the current ratio," a real condition on this rig: the mirror is roughly 60% of the XR's linear panel resolution, and small captions sit at the edge of reliable OCR cap height once downscaled to 500x1080. The catch is that the pixel space every stored rect and click coordinate is keyed to moves with it, and the doc states the change disconnects the cast to apply.

HOW

1. `get_device_list` XP `/device/get` — read the target phone's current `imgw`, `imgh` and `air_ratio` before touching anything; on this rig that is 500x1080 with `air_ratio 0` (following the global default), so this is the baseline every stored coordinate assumes
2. `set_airplay` XP `/device/airplay/set?id=<one id>&air_ratio=1920` — raise just this phone's mirror height for the pass; `air_refresh`, `air_fps` and `air_img_fps` ride the same call, so a higher frame rate for the pass costs no extra reconnect
3. `local` XP `(poll)` — the doc's own note under the parameter table says a non-zero value disconnects the cast immediately and the new geometry takes effect on the next screen projection; poll `/device/get` for that id until `state` is non-zero again and `imgw` / `imgh` reflect the requested ratio, never a fixed sleep
4. `get_device_screenshot` / `ocr` / `find-text` XP `/pic/screenshot`, `/pic/ocr?id=<id>&rect=[l,t,r,b]`, `/pic/find-text?id=<id>&text=[…]` — capture or recognize against the larger frame; the rect for this call must be measured at the new resolution, not carried over from the old one

5. `mouse_reset_pos` + `click` XP `/mouse/reset?id=<id> then /mouse/click?id=<id>&x=<x>&y=<y>` — act on what was read, using coordinates measured at the raised resolution, not the working one
6. `set_airplay` XP `/device/airplay/set?id=<one id>&air_ratio=0` — drop the phone back to its working ratio once the pass is done; this is a second disconnect and reconnect, not a free undo
7. `local` XP `(poll)` — poll `/device/get` again for `state` and `imgw` / `imgh` back at 500x1080 before resuming any job that assumed the working coordinate space

NEEDS

- The phone already bound and mirroring, since `air_ratio` only changes the geometry of an existing or next cast, it does not start one
- The coordinate registry keyed on (device id, resolution), not device id alone, so a rect or click measured at 500x1080 is never reused at 1920 or vice versa
- A settle-and-poll routine on `/device/get`, the same pattern used for screenshot settle elsewhere in this section, applied to `state` and `imgw` / `imgh` instead of pixel diff
- For the OCR half of the goal specifically: the kernel OCR plug-in actually connected. On this rig `/pic/ocr` and `/pic/find-text` answer `data.code 28` at any resolution, so raising `air_ratio` here only removes the "too small" half of the problem; the vision pass this buys is client-side OCR on the fetched frame, not the kernel path

WATCH OUT

- Two different resolution knobs exist and are easy to conflate. `air_play_ratio` in `/config/imserver/set` is the server-wide default, applies to every phone whose own `air_ratio` is 0, and is not on the doc's short list of settings that skip a kernel restart, so changing it needs `/imserver/restart` and touches the whole rack. `air_ratio` in `/device/airplay/set` is per device and needs no kernel restart, only a re-cast of that one phone. Only the per-device knob is the single-job lever described here
- `air_ratio` values (0, 720, 1080, 1400, 1920, 2650) are requested targets, not guaranteed output pixels. `imgw` / `imgh` in `/device/get` are the only confirmed result, read them back after the reconnect rather than computing them from the requested ratio
- The disconnect is stated as unconditional for any non-zero value, not an occasional side effect, so a job that raises and restores resolution around every read pays two full reconnects, not one
- Nothing in the docs gives a reconnect latency; anything mid-flight that assumed continuous mirroring, a diff-based watcher, an open Shortcut round trip, a queued click, breaks across the gap and needs to re-check `state` before resuming
- This has not been exercised on this rig's two live phones, only read from the documented parameter table and the device-connect/disconnect callback schema; treat the reconnect timing and the exact post-change `imgw` / `imgh` as unverified until run once

SCALE

`/device/airplay/set` accepts a comma id list, so raising resolution on several phones for the same pass is one call, but each listed phone pays its own independent disconnect and reconnect, there is no shared cast to break once for all of them. The lever is meant for one phone in one job at a time, not a rack-wide switch; a rack-wide change belongs on the global `air_play_ratio` instead, at the cost of a kernel restart and every phone's rects going stale together rather than one.

Server-side cropping as the default poll shape, not a special case

Every polling design earlier in this document, and elsewhere in this handoff, fetches the full mirror frame and crops on the PC: a wall thumbnail, a badge watcher, a banner-strip poll for push latency, the post-grid diff above all pull the complete `binary=true` payload (measured at 1.6 MB, 500x1080x3 plus a 54-byte header) and only then discard everything outside a few dozen pixels. `/pic/screenshot` does not require that. Its `rect` parameter, documented as a plain sibling of `binary` and `jpg`, crops on the kernel before the image leaves the process, so the reply body is the sub-rectangle, not the frame. `save_path` goes further and skips the transfer path entirely, writing the (optionally already-cropped) image straight to a file on the PC's own disk with no base64 field and no JSON image payload to build or parse. Neither costs anything extra to add to a call that already knows its target rectangle, and both are native, unauthenticated, plug-in-free XP behavior, no different in status from a call this document has already used elsewhere on this rig.

The same flat `[left, top, right, bottom]` array (never the Phonefarm four-corner form, see the coordinate-space note above) scopes the search area for `/pic/find-multi-color`, and for `/pic/ocr` where the OCR plug-in is connected, neither of which is this rig's case (`data.code 28`). For a badge watcher whose signal is a known color key rather than legible text, `find-multi-color` with a tight `rect` is the strongest version of this pattern: the response carries only `{index, centre}` pairs for a match, or an empty list, no image bytes at all, so the watcher's steady-state traffic is a few dozen bytes of JSON regardless of how large the full frame would have been. A banner strip that has to be read as pixels rather than matched by color (a diff strip, a piece of text once OCR is live) still goes through `/pic/screenshot?rect=…`, but the transferred image shrinks with the crop: a 500-pixel-wide, few-dozen-pixel-tall strip is on the order of tens of kilobytes rather than 1.6 MB, and a small badge glyph is a few kilobytes or less. Nothing in the docs or in HARDWARE.md benchmarks a cropped payload directly, so these are pixel-math estimates from the documented BMP layout, not measured figures; the one-call smoke test this section already recommends for the Shortcut chain has an equivalent here, run `/pic/screenshot` once with and once without `rect` on the same region and compare bytes and latency before sizing anything against it.

The reasoning connects directly to the GC pressure recorded in HARDWARE.md. That measurement (488 MB/s allocated, 496 gen-2 collections and a 96.9 ms GC pause in 8 seconds) is a direct consequence of a full 1.6 MB frame crossing .NET's 85 KB Large Object Heap threshold on every single poll, whether or not the poll needed more than a corner of it. A badge or banner-strip crop sized in kilobytes, not megabytes, most likely never reaches that threshold at all, which is a stronger fix than the buffer-reuse mitigation HARDWARE.md measured (2.9 MB/s allocated, 1 collection, 8.5 ms pause): reuse still allocates and refills a 1.6 MB buffer per pump, while a genuine server-side crop for a narrow job has no reason to touch an object that size in the first place. That distinction has not been measured on this rig either; the direction is well supported by the documented allocation mechanics, the magnitude is not.

This changes the arithmetic behind the throughput budget card, which currently prices every observation at the full-frame numbers (84 req/s flat out, 12.7 ms p50, 1.6 MB per call). A badge or banner watcher that adds `rect` to the same call is not bound by that ceiling; it is bound by whatever the kernel's own crop and encode cost turns out to be for a much smaller output, which nothing in the current documentation or measurements isolates. Until that is measured, the honest position is that per-phone poll rate for a narrow rect job is very likely higher than the full-frame figure implies, not lower, and any card that reuses 84 req/s or 1.6 MB as the per-observation cost for a badge or banner watcher should be flagged as pricing the wrong payload rather than treated as a conservative floor.

Testing, QA and research

The rack's most defensible use is as a bank of real iOS devices you can drive and observe programmatically: real link routing, real push rendering, real mail-client layout, and a frame-diff oracle cheap enough to run on every screen of every build. Everything below runs on the XP kernel at 127.0.0.1:9911 with path-style calls; the two-phone rig here proves the pipeline, not full 19-board throughput.

RAW FRAME CAPTURE	IDLE VS REACTION DIFF	PER-PHONE FRAME RATE	OCR / IMAGE-SEARCH PLUG-INS
2 ms / 84 req/s	**0.02–0.03 vs 20–160**	**4.5 vs 29 fps**	**Not connected**
single-id, per phone, no OCR needed	mean abs pixel diff, 0–255 scale	two identical XRs, same LAN, under motion	code 28 on this rig, both editions

Campaign link, push and email QA

Deep-link and universal-link verification matrix

NEEDS SHORTCUT · WITH CAVEATS

Fire every campaign URL, custom-scheme link and https universal link at the whole phone group in one request, then capture a raw-BMP frame per phone before and after and classify where each link actually landed: app opened to the right screen, Safari fallback, App Store page, "Open in app" banner, or error. App state is varied per phone first (cold, warm, and one phone deliberately without the app) so routing, fallback and not-installed branches are all covered, and each verdict is backed by a stored frame rather than a log line.

HOW

1. `get_device_list + get_usb_list` XP `GET /api/device/get, GET /api/config/usb/get` — inventory geometry, online state and board binding; exclude any phone whose imgw/imgh, state, vid/pid or location_crc doesn't match the set.
2. `mouse_reset_pos` XP `GET /api/mouse/reset?id=<ids>` — clear the silent stuck-click state across the whole target set before any tap-critical run.
3. `send_key WIN+h` XP `GET /api/key/sendkey?id=<ids>&fn_key=WIN+h` — two presses, normalize every phone to home page one.
4. `send_key AppSwitch + swipe` XP `GET /api/key/sendkey?fn_key=AppSwitch, GET /api/mouse/swipe` — set the row's app state: warm (WIN+h), cold (AppSwitch plus a dismiss swipe), or reserve one phone with the app absent.
5. `get_device_screenshot` XP `GET /api/pic/screenshot?id=<id>&binary=true` — capture the "before" frame, one request per phone.
6. `shortcut#13 Open URL` XP `GET /api/shortcut/exec/url?id=<ids>&url=<link>&outtime=<ms>` — fire the link at the whole group in one request; the only documented XP way to open an arbitrary URL from the PC.
7. `get_device_screenshot`, polled XP `GET /api/pic/screenshot?binary=true` — poll on a fixed period until consecutive frames stop differing, then capture the "after" frame.
8. `get_device_screenshot` with rect XP `GET /api/pic/screenshot?rect=[l,t,r,b]` — classify the landing by mean absolute pixel diff against the before frame and a per-phone golden of the expected destination.

9. `find_image / ocr` (optional) XP `POST /api/pic/find-image-cv, GET /api/pic/ocr` — assert a specific artifact (banner, badge, title text) instead of a whole-screen diff; not available on this rig, code 28.
10. `find_multi_color` (optional) XP `POST /api/pic/find-multi-color` — colour-signature classification of the fallback route without OCR; plug-in status unprobed, treat as unproven.
11. `send_key TAB+b / WIN+h` XP `GET /api/key/sendkey?fn_key=TAB+b|WIN+h` — return home and advance to the next URL; TAB+b is best-effort, WIN+h is the reliable reset.
12. `get_device_screenshot`, saved XP `GET /api/pic/screenshot?save_path=<dir>&jpg=true` — persist the classifying frame to disk, named by device, URL and app state.
13. `send_key + click` (fallback) XP `GET /api/key/sendkey?key=<url>, GET /api/mouse/click` — fully native fallback when the Shortcut is absent: type the URL into a note and tap the auto-linked text so the link is followed from a real in-app tap.

NEEDS

- The iMouse Shortcut installed on every test phone, IP-edited to the kernel PC, all permissions granted, and bound to an AssistiveTouch key — the core group-wide launch step depends on it and it is not confirmed present on this rig.
- IMConsole.exe running and logged in, and each phone's board bound (state 2 in /config/usb/get) with a non-empty location_crc.
- iOS accessibility settings on every phone that the API cannot query: AssistiveTouch on, Full Keyboard Access on, Auto-Lock never, portrait lock.
- A deliberately heterogeneous phone set (at least one without the app) and per-phone golden frames — home layouts differ, so coordinates and goldens are not shareable.
- An apple-app-site-association file actually deployed on the domain under test, matching the build's associated-domains entitlement.

WATCH OUT

- The headline "one request reaches all 19 phones" economics rests entirely on the Shortcut channel. Confirm it first with a single /api/shortcut/device/ip probe before quoting the 20–30 minute matrix figure; without it, fall back to the per-phone typing path, serial and no longer flat with rack size.
- Cold-start by swiping the app-switcher card is unlikely to work: the board walks the pointer at ~0.6 ms/px, so the stroke reads as a slow drag rather than a flick. Budget on /device/restart or /shortcut/switch/device state=1 instead, at a real cost of 60–90 seconds offline plus a re-cast.
- outtime on XP is milliseconds (default 15000), not seconds as in the Phonefarm edition — a value of 15 times out instantly with code 30.
- No OCR or image-search on this rig (code 28); every landing classification falls back to mean absolute pixel diff against a golden-frame library sized to (phone × app-state × destination) built before the run, not on the fly.
- /pic/screenshot is single-id — there is no group screenshot on XP, only input and hotkey calls fan out over a comma id list.
- Test traffic pollutes your own analytics unless links carry clearly-marked test parameters; scope the whole exercise to domains and apps you own or are authorised to test, and never let a run tap through a checkout or consent flow.

SCALE

Launch fans out for free: one /shortcut/exec/url call reaches all 19 phones inside its outtime window. Frame capture is the real cost: raw BMP polls run flat out at 84 req/s per phone (p50 2 ms), and ten concurrent pollers sustain 512 req/s aggregate with no shared bottleneck, so bandwidth is never the limit. The ceiling is each

phone's own AirPlay rate — about 4.5 new frames/s on the slow XR versus 29 on the fast one — which sets a 2–3 second settle window per cell. A 50-URL by 3-app-state matrix (150 cells) runs in roughly 20–30 minutes with the Shortcut confirmed live; without it, the step-13 fallback is serial and roughly 15–20x slower, no longer flat with rack size.

TRAP **The address bar is not a fallback**

Typing a URL into Safari's address bar is specifically the context where iOS does not hand off to an installed app. Treating that as the "native fallback" and then reporting a universal-link failure against it produces a false negative — use auto-linked text in a note or message body instead.

Push-notification render, arrival-latency and tap-through testing

PLUS EXTERNAL TOOLING

WITH CAVEATS

Put each phone into the app state under test (foreground, backgrounded, or locked), start a cropped raw-BMP poller on the banner strip, fire a test push from your own backend or ESP with a unique marker in the title, and take the first frame that crosses a mean-diff threshold as the on-screen arrival moment. Long-press the banner to expand rich media, tap through, and screenshot the landing screen to confirm the deep link routed correctly. Every step of the observation loop is native XP API; the push itself must come from outside the rig, and reading the notification text server-side needs a plug-in that is not installed here.

HOW

1. `get_device_list` XP `GET /api/device/get` — enumerate targets and read each phone's coordinate space and liveness.
2. `mouse_reset_pos` XP `GET /api/mouse/reset?id=<ids>` — clear the silent stuck-click state on the whole target set.
3. `send_key WIN+h` XP `GET /api/key/sendkey?fn_key=WIN+h` — two presses, normalize each phone to a known state.
4. `send_key TAB+l / WIN+h` XP `GET /api/key/sendkey?fn_key=TAB+l|WIN+h` — place the phone in the app state under test: locked, backgrounded, or leave the app open for foreground.
5. `get_device_screenshot`, polled XP `GET /api/pic/screenshot?binary=true&rect=[0,0,500,200]` — start a cropped raw-frame poller per phone on the banner strip.
6. `none` XP `external — your backend, APNs sandbox, or ESP` — fire the test push with a unique marker in the title; record the send timestamp on the PC clock.
7. `none` (client-side) XP `mean absolute pixel diff on consecutive raw BMP buffers` — flag the first frame crossing ~20 as the arrival moment; keep one untouched phone as an idle control.
8. `ocr / ocr_ex` (optional) XP `GET /api/pic/ocr?rect=[l,t,r,b]` — read the notification title/body text; not available on this rig, code 28.
9. `send_key NoticeBar` XP `GET /api/key/sendkey?fn_key=NoticeBar` — open Notification Centre for grouping and thread-summary evidence.
10. `get_device_screenshot` XP `GET /api/pic/screenshot?binary=true&save_path=<dir>` — capture the full-frame render evidence.
11. `click with time` XP `GET /api/mouse/click?x=<cx>&y=<cy>&time=700` — long-press the banner to expand a rich notification; coordinates from that phone's own arrival frame, never a hard-coded constant.
12. `click` XP `GET /api/mouse/click?x=<cx>&y=<cy>` — tap through to the deep-link destination.

13. `find_image / find_image_ex` (optional) XP `GET /api/pic/find-image-cv?img_list=[...]&similarity=0.85` — assert the landing screen against a golden, plug-in gated.
14. `send_key shift+win+3 + album` (optional) XP `GET /api/key/sendkey?fn_key=shift+win+3, GET /api/shortcut/album/get, GET /api/shortcut/album/down` — pull a lossless native-resolution copy of the render; needs the Shortcut.
15. `send_key VolumeUp` (optional) XP `GET /api/key/sendkey?fn_key=VolumeUp` — attempt to wake a dark screen before capture; unreliable, see watch-outs.
16. `swipe + send_key` XP `GET /api/mouse/swipe?direction=up, GET /api/key/sendkey?fn_key=WIN+h` — dismiss and reset between cases.
17. `WS callbacks` XP `ws://127.0.0.1:9911/api – error_push, dev_disconnect, usb_change` — subscribe for asynchronous failure signals so a board fault doesn't read as a push failure.

NEEDS

- An external push sender you control — your own backend against APNs, or a test-only audience in the ESP. The rig cannot originate a push.
- Board and mirror ready per phone, plus the iOS accessibility settings the API cannot query (AssistiveTouch, Full Keyboard Access, Auto-Lock never).
- The test app installed with notification permission granted, its push token registered to a dedicated test segment, and its deep-link scheme wired up.
- The OCR plug-in only for step 8's text read (confirmed absent, code 28); everything else in the chain is native.

WATCH OUT

- What you measure is on-screen arrival, not APNs delivery latency: the number is (first frame above threshold) minus (your send timestamp), inflated by that phone's AirPlay rate and poll period. Quote it as display-arrival latency with the per-phone frame rate attached, or it will be read as something it is not.
- The locked-screen branch is likely not achievable at all: locking turns the physical display off, and AirPlay mirrors the rendered framebuffer, so frames may freeze on the last image or go black once the display sleeps. Auto-Lock Never only suppresses idle timeout, not a deliberate lock. Verify on one phone (check air_pid and whether frames keep changing) before building any part of the matrix on this branch.
- VolumeUp does not wake a locked iPhone display on stock iOS with Face ID; drop that premise rather than relying on it.
- A 700 ms pointer hold via /mouse/click time is not confirmed to trigger iOS's haptic-touch expand gesture — treat it as optional, not a required chain link.
- /pic/screenshot, /pic/ocr and /pic/find-image are single-id; only /key/sendkey, /mouse/click, /mouse/swipe and /mouse/reset fan out over a comma id list.
- A tap that produces no reaction is ambiguous between "the deep link is broken" and "the phone is in the silent stuck-click state" — /mouse/reset before every tap and error_push are what disambiguate it.

SCALE

Hotkeys and taps fan out for free across the rack in one request each; frame capture does not, so budget one poller per phone. Measured headroom is large — 512 req/s aggregate at ten concurrent pollers, p50 3 ms — so the API is never the constraint. Foreground/background cases run at roughly 150–250 cases per hour for the whole rack, bounded by the slow phone's ~4.5 fps AirPlay rate and, more often, by the ESP or APNs sandbox's

own send-rate limits. Locked-state throughput is 0 through this API pending the freeze/black-frame check above; if the mirror dies on lock, that branch has to be re-scoped to a physical camera rig and dropped from any automated figure.

OPEN **Does the mirror survive a lock?**

Whether the AirPlay mirror keeps streaming after a synthetic lock (TAB+l) is undocumented and untested on this rig. If it freezes or goes black, as it does on stock iOS lock behaviour, the entire locked-screen half of any test matrix has to be dropped or redone by camera, not fixed in software. Check this on one phone before designing anything around it.

Email campaign rendering sweep on real iOS mail clients

PARTIAL WITH CAVEATS

Send the campaign to seed inboxes, then on each phone drive the real mail app (Apple Mail, Gmail, Outlook), open the newest message, and alternate raw-BMP frame grabs with mouse-wheel scrolls until the frame stops changing, stitching the tiles client-side into a full-height render. Because taps and scrolls take a comma id list, the identical sequence fans out to the whole rack in one request, so adding phones adds mail clients rather than wall-clock time. On this two-phone rig the sweep varies the client, not the device, since both are the same iPhone XR build.

HOW

1. `get_device_list` XP `GET /api/device/get` — preflight: read per-phone mirror geometry and liveness.
2. `mouse_reset_pos` XP `GET /api/mouse/reset?id=<ids>` — clear the silent stuck-click state on the whole target set.
3. `send_key WIN+h` XP `GET /api/key/sendkey?fn_key=WIN+h` — two presses, normalize every phone to home page one.
4. `click` XP `GET /api/mouse/click?x=<px>&y=<py>` — launch the mail client; per-phone coordinates are mandatory, home layouts differ.
5. `shortcut#13` (optional) XP `GET /api/shortcut/exec/url?url=<https url>` — open a landing page or deep link instead of tapping; needs the Shortcut, and no mail-app URL scheme is documented in either edition.
6. `click` XP `GET /api/mouse/click` — open the newest message; without OCR, pin the seed send as the only unread item and verify by whole-frame diff that the view changed.
7. `get_device_screenshot` XP `GET /api/pic/screenshot?id=<id>&binary=true` — capture one viewport tile.
8. `mouse_wheel` XP `GET /api/mouse/wheel?id=<ids>&direction=down&len=<1-127>&number=<n>` — scroll one viewport; covers every phone in the id list at once, but whether webview-backed clients honour it is unverified.
9. `none` (client-side) XP `mean absolute pixel diff on consecutive frames` — detect the bottom of the message; nothing under ~1 means nothing moved.
10. `click + get_device_screenshot` XP `GET /api/mouse/click, GET /api/pic/screenshot` — tap the CTA and capture the landing page; locating it by label needs OCR, absent here.
11. `send_key shift+win+3 + album` (optional) XP `GET /api/key/sendkey?fn_key=shift+win+3, GET /api/shortcut/album/get, GET /api/shortcut/album/down` — the only lossless native-resolution path for colour-critical QA; needs the Shortcut.
12. `none` XP `no API — blind /mouse/click through Settings > Display & Brightness > Dark` — the dark-mode pass; no appearance switch exists anywhere in either API.

NEEDS

- Seed addresses excluded from campaign reporting, with the test send delivered and unread before the run starts.
- Mail apps installed and signed in per phone (Apple Mail is stock, Gmail and Outlook must be added by hand), one client per phone or per group.
- Per-phone icon and message-row coordinate maps, and an empirical pixels-per-wheel-tick calibration per mail client before any stitch can be trusted.
- The iMouse Shortcut, unconfirmed on this rig, for the deep-link-open and native-resolution steps only.

WATCH OUT

- The OCR plug-in is not connected (code 28), which removes every text assertion — no confirming the subject line, no finding the CTA by label. The sweep produces images for a human or a client-side vision model to judge, not automated text pass/fail.
- Both phones here are the same model and iOS build, so the sweep currently tests mail-client rendering variance only, not device or OS variance; only 2 of 19 boards have a phone attached and mirroring.
- /mouse/wheel's behaviour inside iOS Mail or Gmail's webview is unverified — calibrate pixels-per-tick per client and keep a down/move/up drag as a committed fallback, not an afterthought, or the stitched composite will silently duplicate or drop rows.
- No dark-mode API exists in either edition — checked every /shortcut/switch/* path (device, brightness, torch, flight, cellular, Wi-Fi) and none touches appearance.
- Tapping a CTA in a real inbox fires the campaign's genuine open and click tracking; confirm seed-address exclusion from ESP reporting before the first run, or the rack will quietly inflate engagement metrics.
- The bottom-of-message stop condition proves the view changed, not that the right message opened — add a structural check (e.g. a fixed subject-line crop) before trusting the stitch, since there is no OCR to confirm it directly.

SCALE

API time per tile is negligible — 2 ms capture, sub-millisecond wheel — so wall clock is set by waiting for a genuinely new AirPlay frame (as little as 34 ms on the fast XR, 220 ms or more on the slow one) plus the mail app's own layout settle. A six-tile email render runs roughly 3–8 seconds per phone on the fast XR and 10–20-plus seconds on the slow one. Scaling this use case means adding phones for more mail clients in parallel, not compressing per-email time; testing several clients on one phone is serial and multiplies wall time by the number of clients.

Visual regression and rig health

Golden-frame visual regression and screen-change detection

NATIVE API WITH CAVEATS

Drive each phone to a known screen, wait for the mirror frame to actually settle, pull the raw BMP off the AirPlay decoder in about 2 ms, and compare it to a stored golden with a masked mean-absolute-pixel-diff threshold. No plug-in, no on-phone agent, no app instrumentation: a screenshot call plus a client-side comparator is the whole pipeline, which matters because it is the one vision capability that works today given the OCR and image-search plug-ins are not connected. Every phone is independent, so the suite scales across the rack without scheduling.

HOW

1. `get_device_list` XP `GET /api/device/get` — build the target set and read each phone's coordinate space; require rotate == 0.
2. `mouse_reset_pos` XP `GET /api/mouse/reset?id=<ids>` — clear the silent stuck-click state before any tap-critical walk.
3. `send_key WIN+h` XP `GET /api/key/sendkey?fn_key=WIN+h` — two presses, normalize every phone to home page one.
4. `shortcut#13` (optional) XP `GET /api/shortcut/exec/url` — navigate to the screen under test by deep link; Shortcut-gated, unverified here.
5. `click / mouse_wheel` XP `GET /api/mouse/click, GET /api/mouse/wheel` — navigate to the screen under test by tap; the no-dependency path.
6. `mouse_move + get_device_screenshot` XP `GET /api/pic/screenshot?binary=true` — confirm the tap actually landed with a diff on a ~40 px box around the commanded coordinate, before judging the screen.
7. `get_device_screenshot`, polled XP `GET /api/pic/screenshot?binary=true` — settle loop counted in new frames, not polls, at a fixed 33–50 ms period.
8. `none` (client-side) XP `mean absolute pixel diff, masked` — compare the settled frame to the golden.
9. `get_device_screenshot`, saved XP `GET /api/pic/screenshot?jpg=true&save_path=<dir>` — archive the live frame on a failure; the harness owns the golden and renders any side-by-side itself.
10. `find_image / find_image_ex` (optional) XP `POST /api/pic/find-image-cv` — assert the expected widget is present; plug-in gated.
11. `ocr / ocr_ex` (optional) XP `GET /api/pic/ocr, POST /api/pic/find-text` — assert the expected label text appears; plug-in gated, absent here.
12. `send_key TAB+b / WIN+h` XP `GET /api/key/sendkey` — leave the screen and advance the suite; watch the dev_rotate callback, which invalidates goldens on rotation.

NEEDS

- IMConsole.exe running and logged in, board bound per phone (state 2) with a non-empty location_crc, and the iOS accessibility settings the API cannot query.
- Golden images stored per screen and per phone — home layouts differ between phones, so a golden is not portable.
- A client-side BMP decoder and mean-abs-diff comparator with a reused buffer per poller, and masks maintained per screen for dynamic content (clocks, badges, carousels).
- A pinned app version on the monitoring phone for change-monitoring use, since there is no installed-app-version signal in the API.

WATCH OUT

- Settle detection must gate on new frames plus a per-phone timeout, not a poll count: a still screen produces almost no new frames (4.5/s on the slow XR), so a poll-counted quiet window fires falsely early, while genuinely continuous motion (video, spinners) never goes quiet and always hits the timeout.
- A tap that never landed looks identical to "nothing changed." The pointer-box diff is mandatory, and /mouse/reset, not /mouse/up, is the only documented recovery.
- Rotation swaps imgw/imgh meaning and invalidates every golden for that phone — check rotate == 0 and subscribe to dev_rotate.
- The assertion layer (OCR, image search) is plug-in gated and unavailable today, code 28 — keep it additive, never load-bearing.

- /pic/screenshot is single-id and XP has no server-push frame stream, so polling is the only route; only 2 of 19 boards are actually mirroring, so any full-rack throughput figure is a projection.
- Golden images and coordinate tables go stale together on any app or OS update, none of which the API can detect.

SCALE

Frame capture is effectively free (2 ms paced, 84 req/s flat out) and the client-side diff costs about 1 ms; the real cost per screen is the settle wait, roughly 1–2 seconds on the fast XR and 2–4 seconds on the slow one, plus pointer-walk time at 0.6 ms per pixel. A 20-screen suite runs in about 30–60 seconds per phone, and phones scale independently up to at least ten concurrent pollers (512 req/s aggregate, p50 3 ms). On this rig that is roughly 300–600 verified screen-checks per phone per hour, or 600–1,200 across the two phones actually mirroring today — the 19 boards are control capacity, not video capacity, and do not by themselves multiply this figure.

TRAP **Never hash-compare frames**

MD5 or byte-equality comparison reports "changed" on every single frame, because the AirPlay encoder, the clock, and wallpaper parallax vary from frame to frame even on an untouched phone. This project already reached a wrong conclusion that way once. Threshold on mean absolute pixel difference instead — measured idle floor 0.02–0.03, a real reaction 20–160 — on every use case in this section.

Crash-to-springboard and frozen-UI watchdog

NATIVE API WITH CAVEATS

A per-phone background watchdog that detects an app crash (the springboard dock reappears while the script expects to be in-app) and a hang (no frame change under a deliberate stimulus), and separates both from rig faults — a silent stuck-click pointer, a dropped AirPlay session, a failed board — using this rig's measured signatures. The core loop is native XP HTTP plus the WebSocket callback bus; no OCR or image-search plug-in is required, which matters because neither is connected here.

HOW

1. `get_device_list` XP `GET /api/device/get?id=<id>` — read imgw/imgh, state and air_pid once per watchdog start.
2. `send_key WIN+h` XP `GET /api/key/sendkey?fn_key=WIN+h` — two presses, normalize before capturing the signature.
3. `get_device_screenshot` XP `GET /api/pic/screenshot?binary=true` — capture the per-phone dock-strip signature as the home-screen reference.
4. `find_multi_color` (optional) XP `POST /api/pic/find-multi-color` — a plug-in-status-unconfirmed colour-table alternative to the pixel crop; probe for code 28 before relying on it.
5. `get_device_screenshot` XP `GET /api/pic/screenshot?binary=true` — the ~1 Hz crash poll, comparing the dock crop to the signature.
6. `mouse_wheel / mouse_move` XP `GET /api/mouse/wheel, GET /api/mouse/move` — the hang check: apply a stimulus and require the screen to move; prefer a 2 px move over a wheel event when the app must not be perturbed.
7. `send_key + mouse_reset_pos` XP `GET /api/key/sendkey?fn_key=WIN+h, GET /api/mouse/reset` — rule out the silent stuck-click state before declaring a hang.

8. `get_device_list` XP `GET /api/device/get?id=<id>` — rule out a dropped video channel (state, air_pid) before blaming the app.
9. `WS callbacks` XP `ws://127.0.0.1:9911/api – error_push, usb_change, dev_disconnect` — subscribe to the event bus so board- and session-level failures are logged next to app failures.
10. `shortcut#13 / click` XP `GET /api/shortcut/exec/url, GET /api/mouse/click` — on a confirmed crash, harvest the last frame and relaunch; the plain click relaunch needs nothing beyond a bound board.

NEEDS

- A per-phone dock/home signature and per-phone icon coordinates captured on that same phone during the same normalisation pass — the two XRs have different home screens.
- A per-phone frame-rate baseline measured under driven motion, used to size the hang timeout.
- Board and mirror ready per phone (state 2, non-empty location_crc), and the Shortcut only for the optional deep-link relaunch branch.

WATCH OUT

- A legitimate return to home — logout, deep-link bounce, user exit — looks identical to a crash by the dock signal alone. Gate the verdict on the script's own expected state, always.
- The hang window must be sized per phone from its measured frame rate: a generic 500 ms window fails on the slow XR, where a still screen delivers almost no new frames as the normal case, not a symptom.
- Crash-log harvesting through the Files-app shortcut path does not reach iOS's system crash-log store — only app-written, Files-visible containers are reachable.
- XP has no server-push frame stream, so the watchdog's own poll cadence bounds its detection latency, and /pic/screenshot is single-id.
- The two-Home-press normalisation itself takes non-zero time and counts as input — sequence it before, not during, an armed watch window, or a crash mid-normalisation is missed.
- Do not report "mirroring broken" without checking /device/get state and air_pid first — diagnosing a silent phone as an app hang when it is really a dropped AirPlay session is the specific wrong conclusion this design exists to prevent.

SCALE

A 1 Hz crash poll is under 1.2% of the measured per-phone capture ceiling (84 req/s), and hang checks at a several-second cadence add negligible load; the watchdog scales to at least ten phones concurrently at the same measured 512 req/s aggregate. The only real cost is memory — each frame is 1.62 MB, so reusing one buffer per poller matters, measured at 488 MB/s allocation and 496 gen-2 collections per 8 seconds without reuse, versus 2.9 MB/s with. Reset and relaunch actions are rare, triggered only on a confirmed fault, so they do not affect steady-state throughput.

Cross-device and localisation QA

Mobile web compatibility sweep in real Safari

NEEDS SHORTCUT WITH CAVEATS

Drive Safari on every mirrored phone from one controller: open a URL with a comma-id `/shortcut/exec/url` call, or Shortcut-free by tapping the address bar and typing; capture raw BMP frames and compare renderings client-side, since no OCR or image-search plug-in is connected here. The API supports the whole flow cleanly,

but the rig is the limiting factor: both phones are the same iPhone XR on iOS 18.7.10, so a sweep today produces one data point, not a version matrix. 19 is the count of USB control boards, not phones; 17 of those boards have nothing attached.

HOW

1. `get_device_list` XP `GET /api/device/get (optional id=a,b)` — enumerate targets, read imgw/imgh (500x1080, not the 414x896 logical points), and confirm each phone's iOS build; on this rig it returns exactly two iPhone11,8 / 18.7.10 records.
2. `mouse_reset_pos` XP `GET /api/mouse/reset?id=a,b` — clear the silent stuck-click state before any tap sequence.
3. `shortcut#13` XP `GET /api/shortcut/exec/url?id=a,b,c&url=<url>&outtime=15000` — open the same URL on every phone in one request; needs the iMouse Shortcut installed, IP-edited and AssistiveTouch-bound on each phone.
4. `click + send_key` XP `POST /api/mouse/click, then GET /api/key/sendkey?id=a,b&key=<url>, then &fn_key=ENTER` — Shortcut-free alternative: tap the address bar and type; ENTER belongs in `fn_key`, not `key`.
5. `get_device_screenshot` XP `GET /api/pic/screenshot?id=<single>&binary=true` — capture the rendered page; single-id only, one poller per phone.
6. `mouse_wheel` XP `GET /api/mouse/wheel?id=a,b&direction=down&len=60&number=1` — scroll to stitch a full-page capture; avoids the ~0.6 ms/px pointer-walk cost a swipe would carry.
7. `send_key` XP `GET /api/key/sendkey?id=a,b&key=<ascii text>` — fill a form field with ASCII.
8. `clipboard/set + fn_key WIN+v` XP `POST /api/shortcut/clipboard/set, then GET /api/key/sendkey?fn_key=WIN+v` — the only route to Unicode/CJK text, since XP has no `send_text` function.
9. `find-image-cv / ocr / find-text` XP `POST /api/pic/find-image-cv, GET /api/pic/ocr, POST /api/pic/find-text` — documented, but each answers `data.code 28` on this rig; do template matching and text assertions client-side on the raw BMP instead.
10. `send_key shift+win+3 + shortcut album` XP `GET /api/key/sendkey?fn_key=shift+win+3, then /api/shortcut/album/get and /album/down` — the only route to above-mirror, pixel-exact resolution.

NEEDS

- XP kernel on 9911 logged in via IMConsole.exe (the Phonefarm 9912 path does not exist here)
- AssistiveTouch, Full Keyboard Access, max tracking speed, Auto-Lock Never and portrait lock per phone
- Board bound via `vid` + `pid` together
- iMouse Shortcut installed, IP-edited and AssistiveTouch-bound, for exec/url, clipboard/set and album/down
- Additional phones on different iOS builds for any real version-matrix claim — none present here

WATCH OUT

- No Safari-version coverage exists today: two identical iPhone XR on 18.7.10 prove nothing about version differences
- Comma-id fan-out covers exec/url, /mouse/*, /key/sendkey and clipboard/set, but /pic/screenshot, /pic/ocr, /pic/find-image and /shortcut/album/* are single-id — frame capture is always one request per phone
- The 500x1080 mirror is a downscale of the XR's native 828x1792; sub-pixel CSS defects and font-smoothing differences are invisible without the native-resolution harvest

- No console errors, network waterfalls or HTTP status codes are observable through any API in either edition — only visible rendering is evidence
- Whether Safari honours HID wheel scrolling through AssistiveTouch was not measured on this rig; verify per phone before trusting a stitched capture, and never compare frames by hash
- Address-bar tap coordinates are state-dependent (tab count, keyboard state) and drift across phones and sessions; re-home to a known state before each URL
- Full-page stitching needs an explicit overlap/dedupe step for sticky headers and lazy-loaded content, not naive frame concatenation
- WIN+v paste into a WKWebView form field is plausible on iOS 18 but unverified here; probe per target site before relying on it for CJK data

SCALE

Navigation, scroll and typing are one request regardless of phone count; screenshot capture is strictly one request per phone (2 ms, 1.6 MB raw, phones independent up to 512 req/s aggregate across ten pollers). Page-settle time of 1-3 s per URL dominates, so a 100-URL sweep runs in roughly 3-5 minutes of wall time across the two phones today — yielding two same-version data points, not a mixed-iOS matrix — or on the order of 1,200-2,000 URL-loads/hour combined, scaling close to linearly if more same-geometry phones were added.

Localisation QA and App Store screenshot harvesting

NEEDS SHORTCUT · WITH CAVEATS

One phone per locale is driven by deep link to each marketing screen in a single multi-id request; the phone's own hardware screenshot hotkey (shift+win+3) writes a native-resolution PNG into Photos, which the Shortcut album functions pull back to the PC. The 500x1080 mirror frame is used only for a quick overflow and layout-shift check, since it sits far below native resolution. Every step that actually lands on a screen runs through the iMouse Shortcut, and per-locale language switching has no API at all — this rig has neither the Shortcut confirmed installed nor a way to verify the flow end to end.

HOW

1. `set_dev` XP `GET /api/device/set?id=<id>&name=xr-de-DE` — label each locale phone once so the harvest files correctly; re-read /device/get to confirm, since the response only echoes the ids that were set.
2. `none (manual)` XP `none` — set each phone's display language by hand; there is no XP or Phonefarm function for this, and a blind coordinate walk through Settings is unverifiable without OCR.
3. `shortcut#13` XP `GET /api/shortcut/exec/url?id=<id1>,<id2>,...&url=myapp://screen/onboarding&outtime=15000` — land every locale phone on the same screen in one request; the multi-id fan-out is what makes this cheap.
4. `get_device_screenshot` XP `GET /api/pic/screenshot?id=<id>&binary=true (poll until stable)` — take the mirror frame as a layout-check artefact only, diffed against an en-US golden for overflow and wrapped buttons.
5. `send_key (shift+win+3)` XP `GET /api/key/sendkey?id=<id1>,<id2>,...&fn_key=shift+win+3` — take the phone's native-resolution screenshot into Photos; needs Full Keyboard Access ON.
6. `shortcut#1` XP `GET /api/shortcut/album/get?id=<id>&num=1&outtime=20000` — find the new file; single-id, and ordering by `num` is undocumented, so match by `create_time`.
7. `shortcut#2` XP `POST /api/shortcut/album/down {id, list:[{album_name,name,ext}], zip:1, outtime:60000}` — pull the file to the PC; the response carries no file path, so locate it by name under <install dir>\Shortcut\Media.

8. `shortcut#3` XP `POST /api/shortcut/album/del {id, list, outtime}` — clean up so the next capture is unambiguous; /shortcut/album/clear empties an album entirely and is irreversible.
9. `find-text` XP `POST /api/pic/find-text {id, text:[...], contain:true, similarity:0.8}` — optional text assertion on the mirror frame; unavailable here (OCR plug-in code 28).

NEEDS

- iMouse Shortcut installed on every locale phone, IP-edited to the kernel, every permission allowed, bound to an AssistiveTouch custom key — none confirmed on this rig
- Shortcuts > Advanced toggles for running scripts and deleting without confirmation
- Full Keyboard Access, AssistiveTouch, Auto-Lock Never, portrait lock; each phone bound to its board and on the kernel's LAN
- Display language set by hand, per phone, outside the run
- A per-locale en-US golden mirror frame; App Store frame composition done externally
- OCR plug-in for the optional text assertion — absent here

WATCH OUT

- The entire pipeline from "land on the screen" onward is Shortcut-gated, not just the harvest half; only the mirror-frame diff and the hardware-hotkey screenshot are pure HID/HTTP
- album/get, album/down and album/del are single-id, so a full harvest of N locales x M screens costs N x M x 3 serial Shortcut round trips; only the fan-out steps (exec/url, sendkey) are free
- Any declined "Allow" prompt on the phone surfaces only as an undiagnostic code 30 "Call timeout"
- Album ordering is undocumented; select by `create_time`, not by assuming `num=1` is newest, since XP has no date filter
- The hardware screenshot needs Full Keyboard Access ON as a standing accessibility toggle; if an iOS update silently resets it, the harvest fails at that step with no error surfaced, since /key/sendkey reports success regardless of whether iOS honoured the key combo
- A language change restarts SpringBoard and drops the mirroring session; schedule it outside the run and re-cast afterward
- Neither phone on this rig is confirmed to have the Shortcut installed, so this use case is unverified end to end here
- Deep links must be static, vetted strings baked into the harness, never derived from anything a phone or webpage returns
- Nothing in the vendor docs specifies how the Shortcut itself navigates to a screen before firing the deep link; if a locale's screen has no myapp:// route and instead relies on blind coordinate taps, the same per-locale layout risk that rules out the Settings-walk fallback resurfaces for in-app screens too

SCALE

Fan-out steps are one request regardless of locale count. The harvest is the bottleneck: album/get and album/down are single-id Shortcut round trips over Wi-Fi, realistically 3-10 seconds each rather than the sub-millisecond kernel calls elsewhere in this API. Ten locales by eight screens by three calls is 240 round trips; if phones cannot be harvested concurrently — unverified on this rig, unlike the proven-independent /mouse/move and /pic/screenshot — that is 12-40 minutes wall clock, not the 5-10 minutes a naive estimate gives. Mirror-frame settle checks are effectively free (2 ms/frame, phones independent), but the settle dwell must respect each phone's own frame rate, several hundred milliseconds longer on the slow XR.

Release health: canaries, ad verification and flow recording

iOS update canary phones running rig preflight plus smoke suite

PARTIAL WITH CAVEATS

Keep one or two rack phones ahead of the fleet on the newest iOS; a job watches `/device/get` for the version change, runs a preflight (calibration lookup, pointer accuracy at three corners, hotkeys, a real tap) and then an app smoke suite, so an iOS release that breaks the app is told apart from one that breaks the control path itself. The vendor's own docs record exactly this failure mode: the Home fn_key exists only to fix a malfunction introduced in iOS 26.1, requiring kernel 2026.2.27 or later.

HOW

1. `get_device_list` XP `GET /api/device/get?id=<canary>` — watch for the version change; a WebSocket client can get the same record pushed on dev_change / dev_connect for an event-driven trigger instead of polling.
2. `get_devicemodel_list` XP `GET /api/config/devicemodel/get` — check the calibration library has a record for the new model+iOS pair; the response body is elided in the vendor doc, so probe the live shape before writing matching logic.
3. `mouse collection open/cfg/close` XP `GET /api/device/collection/mouse?id=<canary>&cmd=1 … &cmd=2` — collect mouse parameters if location_crc is empty or taps return code 11; progress arrives only on the collection_mouse WebSocket callback, and only one device can collect at a time.
4. `save_dev_location` XP `GET /api/device/collection/mouse/save?id=<canary>&describe=<note>` — publish the collected profile so the fleet auto-matches after it updates; there is no delete, so a bad profile can only be superseded.
5. `mouse_reset_pos + mouse_move + get_device_screenshot` XP `/api/mouse/reset, then /api/mouse/move to three corners, each followed by /api/pic/screenshot?binary=true` — pointer accuracy probe; detect the dot by mean pixel diff in a ~40 px box (measured 1.4-9.8 versus an idle floor of 0.02-0.03).
6. `send_key` XP `GET /api/key/sendkey?id=<canary>&fn_key=WIN+h (also TAB+b, AppSwitch, ControlBar)` — hotkey path check, verified by a whole-frame mean diff.
7. `click (with stuck-click guard)` XP `two Home presses, then /api/mouse/click?id=<canary>&x=&y=` — tap check; a click reading ~0.0 mean diff after a working move is the measured stuck-click state, cleared by /mouse/reset, not /mouse/up.
8. `shortcut#13 + get_device_screenshot` XP `GET /api/shortcut/exec/url?id=<canary>&url=<deep link> (or click-only navigation), then /api/pic/screenshot?binary=true` — app smoke suite diffed against previous-iOS goldens with a loose threshold for OS-chrome changes.
9. `auto_connect_screen` XP `GET /api/device/airplay/connect?id=<canary>, or the global auto_connect flag via /api/config/imserver/set` — recover the canary after the update reboot; progress on the airplay_connect_log callback.

NEEDS

- A dedicated canary phone holding only test data, enrolled in beta or with automatic updates on
- A human to enter the passcode after reboot and run the phone-side half of mouse-parameter collection (the console presents a QR code; no collection URL is published)
- Board bound with /config/usb/get showing state 2
- Stored goldens and icon-coordinate maps per phone
- A WebSocket client for the kernel version and any collection/cast progress — none of this is available over plain HTTP

- IMConsole.exe logged in with a license state that allows the device online; the full accessibility checklist re-verified after every update

WATCH OUT

- The version watch, pointer probe, hotkeys and tap check are fully native; calibration collection needs a manual phone-side step and the smoke suite's deep links need the Shortcut, so the job cannot run unattended end to end
- The kernel version needed to know whether the Home-key fix is present is WebSocket-only; a purely HTTP job can detect that a phone updated but not whether the kernel already supports that iOS
- Apple, not this API, controls when an enrolled device installs a downloaded update, typically overnight and only when idle; whether mirroring around the clock with Auto-Lock Never counts as idle to iOS's scheduler is unverified, so treat the job as event-driven off the version signal, not scheduled to a fixed nightly time
- Only one device can collect mouse parameters at a time, and a published bad profile is permanent and auto-matches every same-model/same-iOS phone the moment it updates; gate publishing behind the corner-probe pass criteria and never run it unattended before that gate is confirmed
- Whether the rig can type a passcode through the board after a reboot is unverified; plan for a manual unlock, and make the job fail loudly on a lock-screen frame rather than retry against it
- The three-corner pointer thresholds were measured on iOS 18.7.10; a canary running ahead on a newer iOS could see AssistiveTouch pointer rendering shift in ways that move those thresholds — treat them as a starting point to re-validate per iOS version
- Beta iOS can change AssistiveTouch or pointer behaviour in ways the vendor kernel has not caught up with; that is a genuine canary result, not a rig bug
- Do not update the whole rack at once: a calibration miss puts every tap in the wrong place on every phone simultaneously

SCALE

A preflight is roughly a dozen API calls and about ten raw-BMP screenshots per canary, seconds of kernel time; pointer travel for the three-corner probe is under 1.5 seconds at ~0.6 ms/px. The real costs sit outside the API: the reboot itself is 60-90 seconds offline plus an unbounded wait for a human to unlock, and mouse-parameter collection is one device at a time and human-operated, minutes not seconds. Screenshots must wait for a genuinely new AirPlay frame (up to ~220 ms on a slow phone) rather than assume 30 fps. This scales to one or two canaries per rig, run opportunistically rather than on a guaranteed cadence.

Ad slot fill, render and time-to-render measurement

NATIVE API WITH CAVEATS

Open each ad placement, poll the mirror frame and classify a fixed slot rectangle client-side from the decoded raw BMP: near-zero colour variance means no fill, a jump that stays stable for about a second means a creative painted, and the delta from the navigation timestamp to that first stable frame is time-to-render. The classification is pure arithmetic on pixels the kernel already returns, so it needs no plug-in — which matters, since none is connected on this rig. Repeated across sessions this builds fill-rate and render-time distributions from a real SDK on real iOS.

HOW

1. `get_device_list` XP `GET /api/device/get?id=<id>` — read the slot's coordinate space (imgw x imgh, 500x1080 here, not iOS points) and confirm state and air_pid before trusting any frame.
2. `mouse_reset_pos` XP `GET /api/mouse/reset?id=a,b` — clear the stuck-click state before the tap sequence.

3. `shortcut#13 / click` XP `GET /api/shortcut/exec/url?id=a,b&url=<deeplink>&outtime=15000, or POST /api/mouse/click {id,x,y}` — navigate to the placement and record to as the PC's send timestamp.
4. `get_device_screenshot` XP `GET /api/pic/screenshot?id=<single>&binary=true` — poll the frame containing the slot; whether `rect` combines with `binary=true` is undocumented and unprobed, so fetch the full frame and crop client-side.
5. `(client-side)` XP `none (arithmetic on the decoded BMP)` — classify the crop by mean absolute pixel difference against a calibrated idle control (measured ~0.02-0.03 idle, 20-160 for real change); never use MD5 or byte equality.
6. `find_multi_color` XP `POST /api/pic/find-multi-color {id, list:[{first_color:'x|y|RRGGBB-tolerance,...', rect, similarity}], same:false, all:false}` — detect the SDK's placeholder by colour signature; the plug-in dependency is unconfirmed on this rig, so treat this as an enhancement, not a foundation.
7. `find_image_ex` XP `POST /api/pic/find-image-cv {id, img_list, similarity}` — confirm the AdChoices badge; unusable here (plug-in code 28), do the template match client-side on the crop already held.
8. `ocr` XP `GET /api/pic/ocr?id=<single>&rect=[l,t,r,b]` — read ad copy or "no fill" text; probed 10 Sep 2026 and returns code 28.
9. `shortcut#23 / #18` XP `GET /api/shortcut/switch/wlan?id=a,b&state=0 or /api/shortcut/switch/flight?id=a,b&state=1` — test the offline/late-fill path; this blanks the mirror on the same phone, since AirPlay runs over Wi-Fi, so the condition cannot actually be observed on this rig.

NEEDS

- Test ad unit IDs with the device registered as a test device per network
- IMConsole.exe logged in, phone state non-zero with a live air_pid
- Board bound with location_crc non-empty for the tap-navigation route
- Per-screen slot rectangles measured once in mirror pixels from a raw BMP
- iMouse Shortcut installed, IP-edited and bound for exec/url and any switch/* call — not probed here
- Fixed display settings (brightness, True Tone, Night Shift) if colour-signature matching is used

WATCH OUT

- Time-to-render is measured at the PC: the clock starts on send and stops on frame arrival, carrying AirPlay encode and Wi-Fi latency as an unknown positive bias; treat it as a same-phone relative metric only
- Timing resolution is bounded by that phone's own AirPlay source rate — roughly 34 ms granularity on the fast XR, 220 ms on its identical twin — so a 100 ms poll on the slow phone mostly returns the same frame again
- /pic/screenshot is single-id, so the slot is sampled with one poller per phone even though navigation and taps are multi-id
- A video or animated creative can outrun the slow phone's mirror rate, making "stable for 1 s" an unsafe test there; run video placements on the fast phone or lengthen the window
- The rack holds two phones on one iOS build, so there is no per-OS or per-device-class breakdown available today
- The chain has no app-relaunch or ad-session-reset step: repeatedly re-navigating to the same placement can just redisplay a cached creative rather than trigger a fresh ad-server call, so a measured "fill rate" may really be a cache-hit rate — force a cold ad request (relaunch the host app) between samples to measure true fill
- The variance classifier is vulnerable to false positives from system UI (a low-battery banner, an incoming-call banner, a keyboard) landing inside a poorly chosen crop, especially near the top or bottom of screen

where ad slots often sit

SCALE

A single poller sustains ~84 req/s at p50 2 ms and phones are independent, so sampling a slot at 10 Hz per phone is nearly free, though 10 Hz only buys real resolution on the fast phone. With two phones, an overnight run at roughly 50-100 placement loads per phone per hour — bounded by placement-load and settle time, not the API — yields on the order of 100-200 impressions per hour rack-wide today; that does not extrapolate to the 19-board count, since 17 boards have no phone attached. Keep only the crop and the classification, not the 1.6 MB frames, or storage becomes the real cost.

UX flow recording with a timestamped input log

NATIVE API WITH CAVEATS

While a script drives a task through an app, record two synchronised streams: raw mirror frames polled at a fixed period, and a timestamped log of every input call with its coordinates and id list. Discard duplicate frames by pixel diff, encode externally, and overlay each commanded coordinate against the pointer's own low-contrast dot visible in the frame. The output is a flow video with the exact input sequence attached, plus steps/taps/screens/time-to-complete metrics pulled from the log rather than a stopwatch, re-runnable whenever the app ships a change.

HOW

1. `set_dev` XP `GET /api/device/set?id=<id>&name=AppA-v3.2` — label each phone with what it is running; re-read /device/get afterward, since XP echoes only the ids that were set.
2. `get_device_list` XP `GET /api/device/get?id=<id>` — establish imgw/imgh as the coordinate space and confirm a live air_pid.
3. `mouse_reset_pos` XP `GET /api/mouse/reset?id=a,b` — clear the stuck-click state before recording starts.
4. `send_key (fn_key WIN+h, twice)` XP `GET /api/key/sendkey?id=a,b&fn_key=WIN+h` — normalise the starting state; the two XRs have different home screens, so an identical coordinate opens different apps.
5. `loop_device_screenshot` XP `GET /api/pic/screenshot?id=<single>&binary=true, polled on a fixed ~33 ms period` — run the frame recorder, one per phone; XP has no server-push stream, so recording is client-side polling with a fixed period and a reused buffer.
6. `mouse_move / click / swipe / mouse_wheel / send_key` XP `the full input vocabulary, each call logged with its coordinate and id list` — drive the task; a real drag is down, streamed moves, up.
7. `error_push (WebSocket)` XP `ws://127.0.0.1:9911/api – {data.message, data.call_fun}` — catch input failures the HTTP reply does not report; the only asynchronous signal that an input actually failed at the board.
8. `(client-side)` XP `none – external ffmpeg or equivalent` — deduplicate frames by mean absolute pixel diff and encode; never MD5-compare, since encoder noise changes every frame.
9. `shortcut#7` XP `POST /api/shortcut/album/upload {id, files, album_name, zip, outtime}` — give every phone identical input assets so upload flows are comparable; Shortcut-gated.
10. `(manual + shortcut)` XP `GET /api/key/sendkey?fn_key=ControlBar, tap the Screen Recording control, then /api/shortcut/album/get and /album/down` — capture at native resolution instead of the 500 px mirror; there is no API that starts iOS Screen Recording, and the toggle's coordinate must be hand-calibrated per phone.

NEEDS

- IMConsole.exe logged in, phone state non-zero with a live air_pid, board state 2 and location_crc non-empty

- The full accessibility checklist per phone (AssistiveTouch, Full Keyboard Access, max tracking speed, Auto-Lock Never, portrait lock, StandBy off), none of it queryable over the API
- ffmpeg or an equivalent external encoder
- Accounts with synthetic test data
- iMouse Shortcut for album upload/download
- A WebSocket client for error_push and dev_change/dev_disconnect events

WATCH OUT

- The slow phone (~4.5 fps) records as a slideshow; put teardowns on the fast phone (~29 fps) and report per-phone fps in the output
- Recordings are a 500x1080 downscale of the XR's native 828x1792; native resolution needs the manual Control-Center recording toggle plus a Shortcut-gated harvest, and that toggle's tap coordinate is undocumented anywhere and must be hand-calibrated per phone, re-checked on every iOS update
- Pre-dedupe data rate is ~47 MB/s per phone at 29 fps, far less on the slow phone (~7.3 MB/s); size ingest buffers per phone's own measured fps, not a shared constant, and dedupe/encode on the fly rather than buffering raw frames
- /pic/screenshot is single-id, so N apps recorded means N pollers; the limit on parallel recordings is encoder CPU and disk, not the kernel
- A whole-frame diff cannot see the pointer dot; verify a move landed by diffing a ~40 px box around the commanded coordinate, and treat "the app opened" as inference, not proof an input worked
- A full-height swipe is ~580 ms of pointer travel, which iOS may read as a slow drag rather than a flick; prefer the short axis, /mouse/wheel, or a shorter length so velocity-dependent gestures still fire
- The input log and frame poller run on independent loops, so the frame nearest an input timestamp is not guaranteed to show the pointer having arrived, especially on the slow phone; carry an explicit per-phone uncertainty window on any overlay, not a single frame index
- A "flow video next month" diff is only valid if the app's layout has not shifted target coordinates; a layout change produces wrong taps with no error, which /mouse/reset discipline helps catch but does not eliminate
- There is no audio capture, no touch-event ground truth and no accessibility-tree export; the log records what was sent and the video records what the mirror showed, nothing about what the app actually received

SCALE

Frame capture is 2 ms per request and phones are fully independent (two pollers 83+85 req/s, ten pollers 512 req/s aggregate), and input traffic does not slow polling, so the API is never the bottleneck. The ceiling is each phone's own AirPlay source rate and PC-side encoding: today that means one fully usable smooth recording (fast phone, ~29 fps) plus one degraded recording (slow phone, ~4.5 fps) per PC, since only two phones exist and mirror. A monitoring wall of thumbnails alongside the recorder costs the master poller nothing (17.7 fps with it off versus 17.6 fps with it on at 30 fps). Scaling further is bounded by per-phone AirPlay fps and encoder/disk throughput, not by the kernel.

Outside-the-box measurement: latency, network transitions and accessibility

Tap-to-paint latency and scroll smoothness from frame arrival timing

PARTIAL WITH CAVEATS

Use the flat-out raw-BMP frame poller as a sampling oscilloscope: timestamp every frame whose pixel difference from its predecessor exceeds the noise floor, measure the interval from an input call to the first changed frame in the target region and to the point that region goes still again, and count distinct frames per second during a scripted scroll to spot jank against the phone's own baseline. Every call is native HTTP and needs no plug-in, but the API supplies no server-side frame timestamp, so these are relative, same-phone A/B figures, never absolute device latencies. It is the only performance signal this API exposes at all, which makes it genuinely useful for build-over-build regression checks on one fixed phone.

HOW

1. `get_device_list` XP `GET /api/device/get?id=<id>` — confirm mirroring and read imgw/imgh (500x1080, not the 414x896 logical points); record air_pid so a mid-run reconnection can be detected and that run discarded.
2. `swipe + get_device_screenshot` XP `GET /api/mouse/swipe?id=<id>&direction=left&len=0.5 (loop), while polling GET /api/pic/screenshot?id=<id>&binary=true flat out for 5 s` — baseline the phone's own frame production under continuous driven motion (measured 28.9 new frames/s on the fast XR).
3. `mouse_move` XP `GET /api/mouse/move?id=<id>&x=<bx>&y=<by>` — pre-position the pointer so the measured click carries no pointer-walk cost (measured ~0.6 ms/px); the pre-move itself paints the pointer dot into the frame, so the stable baseline must be read after it settles, not before.
4. `click` XP `GET /api/mouse/click?id=<id>&x=<bx>&y=<by> (optionally &button=1&time=<ms>)` — fire the input and timestamp the send; pin `time` to a fixed value across all A/B runs, since a changed press duration shifts when iOS registers the tap. Send /mouse/reset first, since a phone can silently stop honouring clicks while moves and hardware keys keep working.
5. `get_device_screenshot` XP `GET /api/pic/screenshot?id=<id>&binary=true (poll flat out; optionally &rect=[l,t,r,b])` — detect first paint as the first polled frame whose mean diff in the target region exceeds the reaction threshold (idle floor 0.02-0.03, real reaction 20-160); raw BMP only, since jpg=true costs 40.5 ms and caps at ~21 req/s.
6. `mouse_wheel or swipe` XP `GET /api/mouse/wheel?id=<id>&direction=down&len=30&number=30, or GET /api/mouse/swipe?id=<id>&direction=up&len=0.5` — drive a scroll and count distinct frames per second; wheel avoids the pointer-walk cost that would make a full-height swipe read as a slow drag rather than a flick.
7. `get_device_list` XP `GET /api/device/get?id=<id>` — repeat N times, take medians, and discard any run where air_pid or frame body size changed, or a dev_disconnect/dev_rotate/error_push callback arrived during the window.

NEEDS

- IMConsole.exe running and logged in, phone mirroring with a live air_pid
- Board bound (vid/pid/uid set, /config/usb/get state 2) and location_crc non-empty
- A per-phone frame-rate baseline, re-measured whenever air_ratio/air_fps change
- A fixed poll period with buffer reuse — never an adaptive interval fed by observed gaps
- One dedicated poller thread per phone; Auto-Lock Never and orientation locked
- A/B builds compared on the same phone only — cross-phone comparison is meaningless here

WATCH OUT

- The API exposes no server-side frame timestamp, so what is actually measured is time from issuing the input call to the client observing a changed frame, including HTTP round trip, poll interval and encoder pipeline delay; report it as a bound, never as device latency
- Sampling resolution is bounded by the AirPlay source: ~33 ms on the fast XR, ~220 ms on the slow one, which makes the slow phone's numbers close to useless for latency work, though its scroll-smoothness figures remain valid against its own baseline
- XP has no push stream; every sample is a poll, and the poll cadence is part of the measurement
- The click's own execution includes the board walking the pointer to (x,y); the pre-move reduces that to near zero but does not remove the click's own board latency from the figure
- Only relative, same-phone, same-session comparisons are defensible; any change to air_ratio, air_fps, air_img_fps, air_refresh, Wi-Fi conditions or a mirroring reconnect invalidates the comparison
- /pic/screenshot accepts one id only, so there is no way to sample several phones from one request, and re-verifying air_pid/imgw/imgh/air_ratio between build swaps is easy to miss if done manually via the console
- This is not read-only: the scroll pass moves app state and the tap opens whatever is under it, so run it only on a phone and build you are willing to perturb
- The API is unauthenticated on the LAN, so another host, or the vendor console itself, can inject input mid-measurement; close IMConsole's own activity on the phone during a run

SCALE

Each phone under test needs its own flat-out poller: 84 req/s at p50 12.7 ms, 1.62 MB per frame, with one reused buffer per poller — without reuse, measured 488 MB/s allocation and 496 gen-2 collections in 8 s, exactly the kind of periodic hitching that would corrupt timing work. Phones are independent (two concurrent pollers measured 83 and 85 req/s, ten measured 512 req/s aggregate at p50 3 ms, worst p95 142 ms), and input traffic does not slow polling nor vice versa, so stimulus and sampler run fully concurrently. Effective sampling resolution stays capped by the AirPlay source regardless: usable tap-to-paint samples on the slow phone top out around 16,000 per hour, far below the raw request ceiling. Never use jpg mode for this; it adds 40 ms of encode and caps the sampler at 21 req/s.

Offline and network-transition testing with the split-network trick

NEEDS SHORTCUT WITH CAVEATS

Give each phone a static Wi-Fi IP with no gateway so Wi-Fi carries only the LAN — AirPlay video plus the Shortcut's return path — while the internet rides cellular. Toggling cellular data with `/shortcut/switch/cdpd` then takes the app fully offline and back without ever losing the mirror, so a golden frame can be captured before, during and after the outage. The airplane-mode variant is also reachable but blinds the rig entirely and needs a pre-recorded blind-HID recovery path.

HOW

1. `get_device_list` XP `GET /api/device/get?id=<a,b,c>` — confirm each phone is mirroring and read imgw/imgh (500x1080).
2. `none (manual)` XP `none` — Wi-Fi > Configure IP > Manual, IP on the PC subnet, gateway left empty; a one-time per-phone setting with no API that takes effect immediately without dropping mirroring.
3. `shortcut#24` XP `GET /api/shortcut/device/ip?id=<one id>&state=1 (and state=0)` — verify the split holds, LAN vs. public IP; single-id, one round trip per phone; a code 30 here means the Shortcut isn't bound, not that the network is wrong.

4. `shortcut#13 / click` XP `GET /api/shortcut/exec/url?id=<a,b>&url=<deep link>, or /api/mouse/click?id=<a,b>&x=&y=` — drive the app to the screen under test; send /mouse/reset to the whole target set first.
5. `get_device_screenshot` XP `GET /api/pic/screenshot?id=<one id>&binary=true` — capture the golden online frame.
6. `shortcut#21` XP `GET /api/shortcut/switch/cdpd?id=<a,b,c>&state=0` — cellular off, one request for the whole cohort; outtime is milliseconds on XP, not the seconds the Phonefarm doc uses.
7. `click / swipe / wheel + get_device_screenshot` XP `/api/mouse/click, /api/mouse/swipe, /api/mouse/wheel, /api/pic/screenshot?binary=true` — exercise the app and observe the offline state; assertions must be client-side pixel diff or template match, since OCR answers code 28 here.
8. `shortcut#20` XP `GET /api/shortcut/switch/cdpd?id=<a,b,c>&state=1` — cellular back on; time-to-first-refresh measured client-side by diffing the content region across polled frames.
9. `shortcut#18 / #19` XP `GET /api/shortcut/switch/flight?id=<a,b>&state=1 then state=0` — airplane-mode variant; state=1 drops Wi-Fi too, so the Shortcut can no longer be reached and the recovery call cannot get through — plan the outage as one-way.
10. `send_key + click + auto_connect_screen` XP `GET /api/key/sendkey?id=<a>&fn_key=ControlBar, then /api/mouse/click on a pre-recorded tile coordinate, then /api/device/airplay/connect?id=<a>` — blind recovery from airplane mode over pure HID; the coordinate must be recorded per phone before the outage, since Control Center layout is per-phone.
11. `shortcut#22 / #23` XP `GET /api/shortcut/switch/wlan?id=<a,b>&state=0|1` — Wi-Fi toggle variant; severs both the video channel and the Shortcut's return path in one move, strictly worse than the cdpd route for this test.
12. `none` XP `none` — no bandwidth-throttling or lossy-link function exists in either edition; apply it on the router or carrier side, upstream of the AirPlay/LAN hop this split deliberately keeps untouched.

NEEDS

- A SIM with an active data plan in every phone under test — neither XR on this rig is documented as having one
- Static-IP-without-gateway Wi-Fi configuration applied by hand on each phone
- iMouse Shortcut installed, IP-edited and AssistiveTouch-bound — not verified on this rig
- Board bound and physically attached (/config/usb/get state 2)
- Control Center with two icon rows, portrait lock, Auto-Lock Never; IMConsole.exe running and logged in
- A per-phone pre-recorded mirror-pixel coordinate of the Control Center airplane tile

WATCH OUT

- Every network-toggle step runs through the iMouse Shortcut, which is not confirmed installed or bound on this rig; a code 30 after ~15 s is the signature of an unbound Shortcut
- `outtime` on XP is milliseconds, not the seconds the Phonefarm doc specifies — a value copied across editions is off by 1000x
- /shortcut/device/ip is single-id, so verification is one round trip per phone even though the toggles fan out; count that cost for every phone in the cohort, both before and after each toggle, not just once
- Airplane and Wi-Fi "off" are one-way over the API: the Shortcut's return path dies with the LAN, so only pre-recorded blind HID brings the phone back
- No text recognition is available on this rig, so "assert the offline banner says X" must be a client-side pixel diff or template match, not OCR
- The kernel exposes no throttling or packet-loss injection; that belongs on the router or carrier side

- Neither phone on this rig is documented as carrying an active SIM, so this use case is currently untestable here without adding cellular service
- iOS's automatic Wi-Fi reconnection and captive-portal detection can silently revert a static-IP-no-gateway configuration after a reboot or a brief Wi-Fi drop, and the only signal is a failed IP-readback
- Because /device/airplay/connect has no force flag on XP, a phone recovered from a dropped Wi-Fi mid-outage may need /device/airplay/disconnect first, adding another manual-recovery branch beyond the documented blind-HID tap
- The mechanism by which the vendor Shortcut actually flips cellular data is undocumented beyond the endpoint's parameters and is almost certainly a Settings deep-link or blind UI navigation — the same fragile-across-iOS-versions risk already flagged for language switching elsewhere, but sitting squarely on this entry's critical path; a future iOS release could silently break cdpd while still returning code 0

SCALE

The cdpd/flight/wlan toggles accept a comma-separated id list, so one HTTP request switches the whole cohort in a single Shortcut round trip (outtime default 15000 ms; observed Shortcut round trips run seconds, not milliseconds). Frame observation is unaffected by the outage and stays at ~2 ms per raw BMP poll per phone, phones independent. The real ceiling is each phone's AirPlay source rate: ~29 new frames/s on the fast XR, ~4.5/s on the slow one, so timing a reconnection on the slow phone has a ~220 ms resolution floor. IP-verification and screenshot steps are single-id and scale linearly with cohort size; a single online/outage/recovery cycle for a two-phone cohort runs on the order of 30-90 seconds of API/Shortcut time plus however long the app itself takes to notice and recover.

Keyboard-accessibility focus-order audit via Full Keyboard Access

NATIVE API · WITH CAVEATS

Full Keyboard Access is already on for every rack phone because the rig needs it to function at all; stepping focus with TAB from the control board and locating the focus ring by frame diff builds a focus-order map of every screen, exposing unreachable controls, focus traps and controls that take focus but do not activate. TAB fans out to every phone in one request, so this audit can run on every screen of every build rather than once a release, at essentially no extra prerequisite cost.

HOW

1. `mouse_reset_pos` XP `GET /api/mouse/reset?id=<a,b>` — park the pointer and clear any stuck-click state; the dot itself would otherwise show up in the diffs used to find the focus ring, so also move it to a fixed corner with a separate /mouse/move and mask that region.
2. `click (or shortcut#13)` XP `/api/mouse/click?id=<a,b>&x=&y=, or GET /api/shortcut/exec/url?id=<a,b>&url=<deep link>` — land on the screen under test.
3. `get_device_screenshot` XP `GET /api/pic/screenshot?id=<one id>&binary=true` — capture the baseline frame; one poller per phone.
4. `send_key` XP `GET /api/key/sendkey?id=<a,b>&fn_key=TAB` — advance focus one step; whether TAB belongs in `fn_key` or the `key` parameter is ambiguous in the doc, so probe once and pin it down. Comma ids step every phone together.
5. `get_device_screenshot` XP `GET /api/pic/screenshot?id=<one id>&binary=true` — locate the focus ring by mean absolute pixel diff against the previous frame (idle floor 0.02-0.03, real change 20-160), with the status bar and any animated region masked out.
6. `send_key + get_device_screenshot (loop)` XP `loop of /api/key/sendkey?fn_key=TAB and /api/pic/screenshot` — repeat until focus returns to the first control, recording order and rects; a cycle

shorter than the visible-control count is a focus trap, a control the cycle never reaches is unreachable.

7. `send_key` XP `GET /api/key/sendkey?id=<a>&key=<single space>, or &fn_key=ENTER` — activate the focused control; space is sent as a literal-space `key` parameter, not a named hotkey, and which of the two actually activates is app-dependent.
8. `send_key` XP `GET /api/key/sendkey?id=<a,b>&fn_key=TAB+b` — back-navigation check; the hotkey table itself notes it is "not supported by all apps."
9. `key_release_all` XP `GET /api/key/upall?id=<a,b>` — recover a stuck key or chord between screens.
10. `none (manual)` XP `none` — Dynamic Type / large-text variant is a manual per-phone setting with no API; run the same golden-diff suite against it and review flagged screens by hand.

NEEDS

- Full Keyboard Access ON on each phone, already required by the rig; the vendor's recommended 1 s Auto-Hide raised for this audit or accepted as lossy
- Board bound with /config/usb/get state 2, since TAB travels over USB, not Wi-Fi
- Live mirroring with a live air_pid; a per-screen mask for animated regions
- An inventory of visible controls per screen, since there is no OCR to name what the ring sits on
- One phone at the largest Dynamic Type size; apps and accounts you own or are permitted to test

WATCH OUT

- Auto-Hide at 1 s hides the ring a second after each TAB; on the slow phone (~220 ms between frames) that window holds only a handful of frames, so raise Auto-Hide on the audit phone or accept missed captures
- The audit's throughput is source-rate bound, not API-bound: the poll returns in 2 ms, but a new frame only arrives at the phone's own AirPlay rate
- Without OCR the audit produces rectangles and an order, not control names; naming needs the vendor plug-in or client-side template matching
- A missing focus ring is ambiguous between a genuine accessibility defect and a control that simply doesn't draw a system focus indicator — many custom-drawn UIKit/SwiftUI/WebView controls have none even though focus moved; pixel diffing cannot see accessibility-tree focus, only whatever the OS chooses to render. State this as a hard method limitation
- VoiceOver itself cannot be audited this way: the API has no audio capture, and enabling VoiceOver changes gesture semantics in ways that would break the control path the rig depends on
- TAB fans out to all phones, but screenshots are single-id, so observation scales linearly and each phone needs its own poller
- Blindly pressing Space or Return on every focused control will activate destructive actions (delete, send, purchase) as readily as harmless ones; gate activation to reviewed screens
- Both rack phones are the same model and iOS build, so this validates one iOS version only; a Full Keyboard Access regression specific to a different release would not be caught here
- Inter-screen navigation has no API-driven discovery without OCR; expect tens of screens audited per hour per phone, not hundreds, once navigation and ambiguous-diff review are included

SCALE

One TAB request steps every phone at once and costs a HID report rather than pointer travel, avoiding the ~0.6 ms/px board walk entirely. Observation is single-id and source-rate bound: on the fast phone a new frame lands roughly every 34 ms, so a 30-control screen audits in a couple of seconds; on the slow phone at ~4.5 fps the same screen takes 10-15 seconds, with some risk of missing the ring under a 1 s Auto-Hide. Frame polling and

input traffic do not interfere, and ten concurrent pollers sustained 512 req/s total, so the observation side scales to a rack. Including inter-screen navigation and manual review of ambiguous diffs, realistic sustained throughput is roughly 20-60 screens/hour on the fast phone and 5-15 screens/hour on the slow phone.

Long-running reliability and monitoring

Long-duration soak with a self-healing recovery ladder

PARTIAL WITH CAVEATS

Run an app scenario for 24-72 hours on every mirrored phone with periodic health probes, recovering with a cost-ordered ladder: pointer reset, key release, board reset, re-cast, phone restart, then app relaunch. Subscribing to the WebSocket callbacks (dev_disconnect, usb_change, error_push, airplay_connect_log) stamps rig events into the same log, so a gap in app data reads as rig degradation rather than an app defect. Memory leaks, slow-downs and rare crashes only surface over hours on real devices, and a soak that cannot tell "the app died" from "the board stopped honouring taps" produces worse than nothing.

HOW

1. `get_device_list` XP `GET /api/device/get (optionally ?id=a,b)` — enumerate the soak target set; read imgw/imgh, state, air_pid, bound vid/pid/uid and location_crc. Both XRs return 500 x 1080 and location_crc B0DBD800; all coordinates below are in this mirror-pixel space, never the 414x896 logical points.
2. `get_usb_list` XP `GET /api/config/usb/get` — read board health per slot before and during the soak. state 2 = board plus phone attached (control-ready), state 1 = board only. Do not parse ver/uver as a firmware version, it is mojibake on this build.
3. `mouse_reset_pos` XP `GET /api/mouse/reset?id=<comma list>` — hourly rung 0: clear any silent stuck-click state on the whole target set. There is no id=all keyword anywhere in XP, so build the explicit list from /device/get. This is the only call that clears the stuck-click state, and it stays cleared.
4. `send_key (fn_key)` XP `GET /api/key/sendkey?id=<list>&fn_key=WIN+h` — keyboard-channel liveness probe: Home press, then whole-frame mean-pixel-diff over ~20. Measured 142.15 mean diff while the pointer was stuck, so this proves the board and cable only, never that taps work.
5. `mouse_move` XP `GET /api/mouse/move?id=<list>&x=&y=` — pointer-channel liveness probe: move to a known point, then diff a ~40 px box around it. A landed pointer dot reads 1.4-9.8 in the box against a 0.02 idle floor; a whole-frame diff cannot see it.
6. `click / mouse_wheel / swipe` XP `/api/mouse/click (button 1-8, x, y, time ms), /api/mouse/wheel (direction, len 1-127, number), /api/mouse/swipe (direction, len, stepping, step_sleep, brake, sx/sy/ex/ey)` — scenario loop with per-iteration tap-to-paint latency logging. XP button is numeric ("1" left, "2" right), not Phonefarm's "left"/"right" strings; the swipe JSON examples spell the step count "steping" but the parameter table says "stepping" — send the table's spelling.
7. `get_device_screenshot` XP `GET /api/pic/screenshot?id=<one id>&binary=true` — frame capture for golden-diff assertions and the 5-minute time-lapse archive. id is single-only; binary returns raw bytes with no JSON. Measured 2 ms / 1,620,054 bytes raw BMP versus 40 ms and a ~21 req/s cap for jpg; one poller per phone, phones independent.
8. `mouse_reset_pos / key_release_all` XP `/api/mouse/reset?id=<list> then /api/key/upall?id=<list>` — recovery rung 1 (pointer) and rung 2 (keyboard). /mouse/up does not clear a stuck click; only reset does.
9. `restart_usb` XP `GET /api/device/usb/restart?id=720A|C241` — recovery rung 3 (control board). id is vid|pid, not the phone deviceid and not the 8-hex uid. Confirm success via usb_change codes 20 then 19

then 12, and /config/usb/get returning to state 2.

10. `discon_airplay + auto_connect_screen` XP `/api/device/airplay/disconnect?id=<list> then /api/device/airplay/connect?id=<list> (or /connect/all)` — recovery rung 4 (mirroring session). XP has no "force" flag, so disconnect is a separate mandatory call; progress arrives only on the airplay_connect_log callback. If the receiver has vanished from the phone's mirroring list, /api/imserver/regmdns re-advertises it without a kernel restart.
11. `restart_device` XP `Preferred: GET /api/key/sendkey?id=<list>&fn_key=CTRL+ALT+SHIFT+WIN+r. Shortcut route: /api/shortcut/switch/device?id=<list>&state=1. Last: /api/device/restart?id=<list>` — recovery rung 5 (phone restart). The hotkey needs no plug-in and no Shortcut, so it is the only rung-5 route certain to work here; with no OCR plug-in on this rig, /device/restart is expected to answer code 15 or 28.
12. `shortcut#13` XP `GET /api/shortcut/exec/url?id=<list>&url=<scheme or universal link>` — recovery rung 6: relaunch the app under test. Requires the Shortcut installed, IP-edited and AssistiveTouch-bound, and the app must expose a URL scheme or universal link. Without it, fall back to two WIN+h presses plus a tap on the app icon.
13. `shortcut#15` XP `GET /api/shortcut/switch/bril?id=<list>&state=0.1` — keep phones cool and dim through the soak. state is a float strictly between 0 and 1, Shortcut-dependent. Keep it consistent, since colour-based assertions sample the mirror frame.
14. `callbacks dev_disconnect / usb_change / error_push` XP `ws://127.0.0.1:9911/api` — stamp rig events into the soak log so a data gap has a cause. usb_change codes: 19 board-to-PC, 20 board off PC, 12 board-to-phone; error_push carries data.call_fun plus message and is the only asynchronous signal an input call failed at the board. The XP WebSocket path is /api — Phonefarm's /clinet path and port 9912 do not exist here.
15. `restart` XP `GET /api/imserver/restart` — absolute last resort, off-hours only. Drops every mirroring session and the WebSocket; expect im_connect on reconnect and every phone needing a re-cast. IMConsole.exe had to be running for the rig to come up at all, so a kernel restart can leave the rig needing a human.

NEEDS

- IMConsole.exe running and logged in — with only IMService.exe up, /device/get showed both phones state 0 and /config/usb/get returned an empty board list
- Every soak phone mirroring (non-0 state, live air_pid) and bound to its board (state 2, non-empty location_crc)
- iOS settings the API cannot see: AssistiveTouch on with max tracking sensitivity, Full Keyboard Access on, mouse tracking speed max, Auto-Lock Never, StandBy off, portrait lock
- For rungs 5-6 via Shortcut: iMouse Shortcut installed, IP edited, run once with all permissions, bound to an AssistiveTouch key; iOS 15+ for /shortcut/switch/device
- Passcode-free soak phones, or a human to unlock — neither API can type a passcode after a reboot
- A per-phone frame baseline and mean-pixel-diff threshold, since that is the only reliable "did it react" signal on this rig

WATCH OUT

- No id=all shorthand exists on any XP endpoint — build the comma list from /device/get; only /device/get with no id, and /device/airplay/connect/all, act on everything implicitly
- /device/usb/restart is addressed by vid|pid, never by phone deviceid — a controller that passes the device id silently targets nothing, and board firmware generation is only readable on the phone, never from ver/uver

- No OCR or image-search plug-in, so every assertion is a client-side mean-pixel-diff on raw BMP; never MD5-compare frames, an untouched phone still differs every frame from clock and encoder noise
- AirPlay frame rate is per-phone and motion-dependent — one XR delivers ~29 new frames/s, its identical twin ~4.5 — so screen-verification timeouts must be scaled per phone, not assumed at 30 fps
- The two XRs have different home screens; a shared coordinate is only fair after two WIN+h presses (leave the app, then return to page one)
- Control-ready (board state 2, bound) and video-ready (mirroring, live air_pid) fail independently — report them separately or the ladder climbs the wrong rung
- The API is unauthenticated on 9911; firewall 9911 and 17000 to trusted hosts before leaving an overnight soak running
- Auto-brightness drifts colour-signature thresholds mid-soak unless the Shortcut brightness pin is present and working; Auto-Lock Never does not disable auto-brightness
- The WebSocket bus has no documented delivery guarantee — if the WS client itself drops during a 24-72 h run, the soak loses its only rig-fault/app-fault signal until it reconnects, so give the harness its own WS-liveness heartbeat independent of HTTP polling
- Rung 5 does not guarantee the video channel recovers: an iOS reboot does not necessarily reselect AirPlay mirroring, and nothing in the API can browse or pick from the receiver list — test this unattended before relying on it
- Gate rungs 3+ on a control-channel-negative result specifically (the pointer probe fails), not merely "the scenario did not advance," or the ladder restarts hardware that was never the problem

SCALE

Health probes cost a handful of calls per phone per minute and do not interfere with input traffic (moves ran p50 0.5-3.4 ms while both phones were polled flat out; ten pollers sustained 512 req/s at p50 3 ms, so 19 slots of thumbnail polling is comfortable). Recovery costs: reset/upall in milliseconds, board restart in seconds, disconnect-plus-recast seconds to tens of seconds, phone restart 60-90 s plus recast. Time-lapse archiving at one raw BMP per phone per 5 minutes runs about 460 MB per phone per 24 h; crop with rect or re-encode client-side to cut that. The real ceiling on scenario throughput is app render time plus the pixel-diff verification wait: on the slow phone (4.5 fps) each verified step needs at least 220 ms, capping verified actions at roughly 500-2,000/hour on the fast phone and well under half that on the slow one; ladder overhead is near zero unless false-positive escalations are frequent.

Synthetic checkout monitor on real iPhones

PLUS EXTERNAL TOOLING · WITH CAVEATS

On a schedule, a rack phone walks the operator's own mobile checkout end to end: product page, add to cart, checkout, shipping form, stop at the payment sheet, recording a raw-BMP frame at every step and comparing settled frames against a golden crop from the last known-good run. A broken banner, a moved button, a layout shift or a slower step raises an alert carrying the exact frame. Web synthetic monitors run headless Chrome, not Safari on a real handset on a real network; a mobile-only checkout break is otherwise discovered by customers, and here the evidence is a 2 ms device-level frame with a timestamp.

HOW

1. `mouse_reset_pos + send_key fn_key` XP `/mouse/reset?id=<id> then /key/sendkey?id=<id>&fn_key=WIN+h (twice)` — normalise the phone to a known state. WIN+h is correct for these XRs on iOS 18.7.10; the newer "Home" fn_key needs kernel 2026.2.27+ and iOS 26.1+. Reset first, since a phone can silently ignore taps while hotkeys still work.

2. `shortcut#13` XP `/shortcut/exec/url?id=<comma list>&url=https://shop.example/products/sku123&outtime=15000` — open the product URL. Shortcut-free alternative needing only the board: tap Safari, tap the address bar, /key/sendkey?key=<ascii url>, /key/sendkey?fn_key=ENTER — the URL must be pure ASCII, which the key parameter requires anyway.
3. `click` XP `/mouse/click?id=<id>&x=&y=&button=1` — advance one step of the funnel. Mirror-pixel coordinates (500x1080 here), read per device from /device/get; every CTA position comes from a captured raw BMP for that site/app version.
4. `get_device_screenshot` XP `/pic/screenshot?id=<one>&binary=true` — wait for the step to settle and time it. Poll on a fixed 50 ms period and declare "settled" when the mean absolute diff between consecutive frames stays under ~1 for ~300 ms (idle floor 0.02-0.03; a real transition measures 20-160). Never MD5-compare.
5. `get_device_screenshot` XP `/pic/screenshot?id=<one>&save_path=D:\monitor\<phone>\<step>\` — write the evidence frame server-side. save_path is a directory; the kernel names the file itself, so record the mapping yourself. The directory must exist or the call returns code 27; save_path and binary are alternative return modes, this is a separate call from the settle poll.
6. `click + send_key` XP `/mouse/click on each field, then /key/sendkey?id=<id>&key=<ascii value>` — fill the shipping form. ASCII only, so a test name, street, postcode and phone number are fine and an accented address is not (that needs /shortcut/clipboard/set plus fn_key=WIN+v). Full Keyboard Access must be on; verify long values landed by screenshot, since typing speed and max length are undocumented.
7. `ocr / find_image` XP `none on this rig – /pic/ocr, /pic/find-text, /pic/find-image, /pic/find-image-cv all require plug-ins (code 28)` — compare against the golden frame. This is the load-bearing gap: assertions must be computed by the caller, mean absolute pixel difference of stable regions against the last good crop, alerting above ~20 or on a step time over threshold.
8. `get_device_screenshot` XP `/pic/screenshot (capture) then end the run` — stop at the payment sheet. Capture it as the final evidence frame and stop; do not automate entry of card numbers or credentials — the API is unauthenticated cleartext on the LAN, and Apple Pay's sheet requires biometric confirmation the board cannot produce.
9. `click` XP `/mouse/click on the cart's Remove control, then /key/sendkey fn_key=WIN+h` — reset state for the next run. Verify the cart is empty with a settled-frame diff rather than assuming the tap worked.
10. `get_device_list` XP `/device/get (per-phone air_* fields, state, air_pid)` — interpret a slow run correctly. Run the same flow on two phones: one XR delivers ~29 new frames/s under motion and its twin ~4.5, so step timings on the weak handset carry a ~220 ms quantisation floor versus ~35 ms on the good one.

NEEDS

- A dedicated monitor phone per store or app, signed into a flagged test customer account, boards bound (state 2), location_crc present, Auto-Lock Never plus the rest of the accessibility checklist
- Golden frames and a coordinate map per site/app version, captured from raw BMP on that specific phone, with a documented refresh step for intentional UI changes
- A test SKU (ideally a test payment mode) so synthetic carts do not consume real inventory or pollute analytics
- Writable evidence directories on the kernel host, pre-created — otherwise save_path returns code 27
- Optional: iMouse Shortcut installed and bound for /shortcut/exec/url; without it, the Safari-typing path needs only the board
- A scheduler in the controller — the XP API has no cron, scripting or scheduled-task concept

WATCH OUT

- Every assertion is client-side: OCR, find-text and find-image are all unavailable (code 28) — reading a price or an error string needs the vendor plug-in installed or your own OCR over raw frames
- Pixel-diff golden frames are brittle against legitimately dynamic content — clocks, countdowns, rotating banners, stock counters — mask those regions or the monitor cries wolf nightly
- Step timings are quantised by AirPlay frame production, not by the store: ~35 ms resolution on the healthy XR, ~220 ms on the weak one, so sub-200 ms regressions are not measurable on the slow handset
- save_path cannot name the file and is a separate return mode from binary=true — evidence capture is a second call or a client-side write of the polled bytes
- A phone can silently stop accepting taps, producing false "checkout broken" alerts in an unattended monitor — /mouse/reset at the top of every run plus a pointer-box sanity diff are mandatory
- No multi-id screenshot, so each monitored phone is its own poller and its own run
- The monitor stops at the payment sheet by design; completing a real purchase or entering card and credential data through this API is out of scope
- Long-unattended runs will eventually hit an iOS or carrier interstitial (SIM notice, update prompt, low-storage warning) a fixed coordinate map cannot navigate — budget a periodic full-reset-and-recapture step and a "run did not reach the expected step" alert, not just per-step diffs
- Concurrency is bounded well below the 19-board count: each monitored phone is an actively mirrored AirPlay stream plus the store's own traffic, realistically single digits of simultaneous monitored properties per PC/Wi-Fi segment
- Nothing detects or dismisses an unexpected iOS system prompt mid-run (update banner, tracking dialog, captive portal): the pixel-diff can only say "the frame changed a lot," not "this is a system dialog" — this produces false "checkout broken" alerts on a monitor left running for weeks
- A drifted pointer calibration is indistinguishable from a genuine UI break in this design's own instrumentation — a tap 20 px off target produces a large frame diff and reports as "checkout broken" with no way to tell it apart from a coordinate-map problem
- The "couple of minutes per phone" estimate undercounts real storefront load time — page render, cart update and shipping-form validation round trips commonly run several seconds to tens of seconds per step — treat it as a lower bound, not typical

SCALE

A full flow is roughly 10-15 taps and 10 evidence captures. Capture cost is trivial (raw BMP p50 2 ms, 84 req/s flat out per phone, phones independent, input traffic does not slow polling), so a settle detector can poll at 50 ms without competing with taps. Tap latency is dominated by the pointer walk (~0.6 ms/px: 100 px = 78 ms, 1000 px = 592 ms) and must be subtracted before blaming the store for a slow step. A realistic full-funnel run is more likely 2-5 minutes end to end once real page load time is counted, comfortably inside a 5-15 minute per-phone cadence. Practical concurrency across phones on one PC is limited to roughly 3-6 simultaneously monitored properties by Wi-Fi bandwidth and licence dev_online_num, not by the 19 physical boards; each raw frame is 1.6 MB, so crop with rect or re-encode client-side to bound evidence retention.

Test infrastructure: mocking and replay

Contract and chaos tests against a mock XP kernel, with record and replay of real 9911 traffic

NATIVE API VERIFIED

Extend the project's existing mock HTTP transport (TestHandler in SelfTests.cs, already serving five fixture devices with deliberate variety) into a full XP kernel simulator, plus an in-process WebSocket bus and a recording proxy for real 9911 traffic. It reproduces the real envelope and error signatures, models failure modes that cannot be reproduced on demand with hardware (the silent stuck-click phone, code 28 OCR, board loss mid-job), and lets a recorded incident replay deterministically with no phones attached. Four of the six test modes need the live rack; the hardest paths, a pointer that silently drops clicks, a board unplugged mid-fan-out, are exactly the states hardware will not produce on request, and both project mistakes to date (the MD5 frame comparison, the wall-starves-the-master bug) were reasoning errors a fixture with honest noise would have caught.

HOW

1. `POST {"fun":"click","msgid":n,...} on 9912` XP `GET /api/<path>?k=v, POST multipart with fun=/path, or POST JSON {"fun":"/mouse/click","msgid":n,"data":{...}} — all on 9911/api` — mock the request envelope and the two wrong-edition failure signatures. Status 200 means accepted while the real result is data.code; the mock must reply 200 + data.code, never a Phonefarm-style top-level status 0. The 404 "接口不存在" and 500 "List index out of bounds (1)" behaviours come from the 10 Sep live probe, not the docs — encode them as probe-derived.
2. `get_device_list` XP `GET /api/device/get` — mock device inventory including the exclusion cases. The existing TEST:01-05 fixture already covers online-with-board (01, 04), offline-and-unbound (02, 03) and different-geometry (05, imgw 375 x imgh 812) — exactly what a mirroring-exclusion assertion needs.
3. `get_usb_list / usb_change` XP `GET /api/config/usb/get and the usb_change WS callback` — mock board state and the board-loss event. state 1 = board on PC, 2 = phone attached; callback data.code 19 board-to-computer connected, 20 disconnected, 12 hardware-to-phone connected. Do not parse ver/uver, mojibake on this build.
4. `get_device_screenshot` XP `GET /api/pic/screenshot?id=<id>&binary=true (binary works over plain HTTP on XP)` — mock the frame source honestly. Return the exact 1,620,054-byte bottom-up BMP shape with ~0.02 mean-diff noise between frames and a scripted reaction over 20 only when a click is actually honoured. Generate the buffer once and mutate in place, the same Large Object Heap lesson as production.
5. `click / mouse_up / mouse_reset_pos` XP `/mouse/click, /mouse/up, /mouse/reset` — mock the stuck-pointer fixture. Measured 2026-09-10: the stuck phone returned success with a 0.09 mean diff, /mouse/up did not clear it, WIN+h still worked (142.15), and /mouse/reset restored it permanently (141.96). Reproduce all four, including the key path still working — the trap that makes the phone look healthy.
6. `ocr / find_image` XP `/pic/ocr, /pic/ocr-ex, /pic/find-image, /pic/find-image-cv, /pic/find-text, /pic/find-multi-color` — mock the plug-in-gated recognition endpoints. code 28 is what /pic/ocr actually returned here; the mock's default for every /pic recognition path except /pic/screenshot should be code 28, so no scheduler path can quietly depend on OCR this rig does not have.
7. `ws://host:9911/clinet callbacks` XP `ws://host:9911/api — im_connect, dev_connect, dev_disconnect, dev_change, dev_delete, dev_rotate, group_change, group_delete, usb_change, collection_mouse, airplay_connect_log, restart_log, user_info, im_log, error_push, im_config_change, logout, dev_sort_change` — mock the broadcast bus. All eighteen use the {fun,status,message,msgid,data} envelope;

user_info pushes every 30 s (accelerate in tests). error_push carrying call_fun is the only asynchronous signal an input call failed at the board.

8. `restart_usb / auto_connect_screen / restart_device` XP `/device/usb/restart?id=720A|C241 (vid|pid, not deviceid), /device/airplay/connect, /connect/all, /disconnect, /device/restart` — mock the recovery ladder's own endpoints. XP's usb restart takes vid|pid comma lists, a real regression risk; the docs do not specify which usb_change codes /device/usb/restart emits, so the 20-then-19-then-12 sequence is an assumption to validate against a recording.
9. `msgid echo` XP `documented: each interface returns msgid as-is` — record and replay real traffic. A proxy in front of 9911 logs path, params, id list, reply body, msgid and delta-t; replay serves the logged reply for the same path plus ids at recorded or compressed timing, so an observed incident reruns the same scheduler code with no rack.

NEEDS

- The existing --self-test / --ui-test harness and TestHandler in src/iMouse.Console/SelfTests.cs (75 self-tests, 76 UI checks passing as of 10 Sep 2026)
- Gesture stages BeginGesture / TrackGesture / EndGestureAsync taking plain Points, so a test can drive the exact path a real press follows
- One recording session against the live rack to capture realistic device records, the 19-board list and at least one genuine failure, for replay fixtures
- Somewhere to keep recorded fixtures that is not shared — this project is not a git repository and build/ already holds a live credential

WATCH OUT

- A mock that always answers code 0 is worse than no mock — the fixtures that carry value are the ones that lie the way the kernel lies: 200-with-no-effect, code 28 on every recognition path, code 25/26 on a phone that looks online, a board vanishing mid-fan-out
- Several mocked behaviours are probe-derived, not documented — the 404 body, the 500 "List index out of bounds (1)", the usb_change sequence from /device/usb/restart — mark them as dated observations so a future kernel build that changes them is diagnosable, not mysterious
- Replay fidelity is per path plus id list: it reproduces what the kernel said, not what the phones did, proving the client behaved identically given identical replies, nothing about whether a gesture actually landed
- Recorded fixtures embed real screenshots and device records (MAC, IP, user_name) — scrub or quarantine them, and budget for size at 1.6 MB per captured frame
- XP documents no WebSocket binary push, so there is deliberately nothing to mock there — a mock of Phonefarm's 261-byte binary frame header would model an edition this rig does not run
- A recording proxy in front of an unauthenticated LAN API is itself a new listening service capturing live screenshots and device identity — do not run it with elevated privileges or leave it running outside test sessions
- Contract tests that assert on Chinese-language error strings are fragile to a locale or firmware update — key assertions off the numeric or status codes (404/500, code 28) and treat the string as secondary
- Serving the full 1,620,054-byte BMP payload per screenshot call, thousands of times per second, is measurable memory and copy overhead — budget for it rather than assuming the mock is free just because it has no network hop
- Building a WebSocket bus for all 18 documented callback events is a bigger scope item than the rest of the chain; only error_push and usb_change are actually exercised by the recovery ladder today, so the other ~16 could be stubbed "defined but never fired" initially

SCALE

The mock side needs no rack and no network: mocked runs execute thousands of steps per second, bounded mainly by the BMP fixture, which should be allocated once and mutated in place (the production lesson: 488 MB/s allocation and 496 gen-2 GCs per 8 s before buffer reuse, versus 2.9 MB/s and 0-1 after). Recording against the live rack costs whatever the recorded session costs, a raw-BMP step is ~1.62 MB, so an hour of frame-inclusive recording is not storable — record request/reply metadata continuously and capture full frame bytes only at marked steps. Replay can run at recorded delta-t for timing-sensitive assertions, or compressed to zero for throughput; there is no phones-per-PC or actions-per-hour ceiling here, since nothing depends on a live phone or AirPlay.

Concurrency and race-condition testing

Synchronised N-phone race and contention testing

NATIVE API · WITH CAVEATS

One /mouse/click carrying a comma-separated id list makes every listed board tap at the same moment, measured latency was identical for one phone and for two, so N real devices with real client retry logic can hit "claim", "buy" or "join" within a very short window to exercise backend races, rate limiters and idempotency. Pre-position the pointer with a multi-id /mouse/move first so no per-phone pointer walk adds skew, then verify each phone's outcome from its own raw BMP frame. Load generators simulate concurrency at the HTTP layer but not the client; real apps with real local state and retry logic hitting one endpoint simultaneously is the scenario that produces double-bookings and duplicate redemptions in production.

HOW

1. `get_device_list` XP `GET /api/device/get` — build the eligible set: online, bound, identical mirror geometry. Exclude any phone whose imgw/imgh differs from the rest — the same coordinate on a differently sized mirror is a different place. Both XRs here are 500x1080.
2. `shortcut#13` XP `GET /api/shortcut/exec/url?id=<comma list>&url=<deep link>` — drive every phone to the identical screen. The only reliable normalisation, since the two XRs have different home screens. Without the Shortcut, fall back to two /key/sendkey fn_key=WIN+h presses plus a per-phone tap map.
3. `mouse_reset_pos` XP `GET /api/mouse/reset?id=<comma list>` — clear stuck-click state across the whole firing set. Mandatory before a race: a stuck phone follows /mouse/move and hotkeys while producing a 0.09 mean diff on a click (idle floor 0.02), so it would silently drop out and look like a backend loss. Measured 141.96 after reset.
4. `mouse_move` XP `GET /api/mouse/move?id=<comma list>&x=250&y=900` — pre-position every pointer on the target control so the firing call has zero travel. The board walks the pointer at ~0.6 ms/px; N phones walked in parallel cost the same as one.
5. `click` XP `GET /api/mouse/click?id=a,b,c&x=250&y=900` — fire: one request, all phones, log the send timestamp. Coordinates are mirror pixels; optional "time" sets the press-to-release hold; add &button=2 for a secondary click — XP uses numeric codes 1-8, not "left"/"right" strings.
6. `send_key` XP `GET /api/key/sendkey?id=<comma list>&fn_key=ENTER` — keyboard-triggered variant for focused buttons. Requires Full Keyboard Access on and the control focused; a second, independent trigger path to see whether the skew profile differs from the HID mouse path.
7. `get_device_screenshot` XP `GET /api/pic/screenshot?id=<one id>&binary=true` — capture each phone's outcome frame. Single-id, so N concurrent requests, one per phone. Raw BMP 2 ms / 1,620,054 bytes; jpg 40 ms, caps ~21 req/s. Start pollers before firing so the first post-tap frame is captured.

8. `ocr / find_image` XP `/api/pic/ocr, /api/pic/find-text, /api/pic/find-image-cv — NOT AVAILABLE on this rig` — classify each outcome. /api/pic/ocr answers data.code 28; substitute client-side classification: a golden-diff against stored "success"/"sold out" crops, or a colour-signature check.
9. `(no equivalent)` XP `ws://127.0.0.1:9911/api callback error_push (data.call_fun, data.message)` — watch for per-board dispatch failures during the volley. A failed /mouse/click reports only here, never in the HTTP reply — without this subscription a hardware drop-out is indistinguishable from a lost race.
10. `(outside the API)` XP `none — external` — correlate with the server side. The rack gives only the client-side send timestamp; actual request arrival spread must come from the service's own logs.

NEEDS

- One pre-existing test account per phone that you own, on a staging or test tenant — never production, and never accounts created in bulk for the test
- All firing phones online, bound, with identical imgw/imgh and identical app screen state at fire time
- Full Keyboard Access on for the ENTER variant; iMouse Shortcut installed and bound for deep-link normalisation
- A stored golden crop per outcome state (success, sold out, error) per phone, since no OCR or image-search plug-in is available
- WebSocket client attached to ws://127.0.0.1:9911/api before firing, to catch error_push

WATCH OUT

- N is 2 on this rig today: 19 boards exist, but only two iPhone XRs are present and mirroring — 19 is the board ceiling, not a demonstrated concurrency
- Sub-millisecond simultaneity across boards is unverified beyond two phones: three chained 7-port hubs are a plausible serialisation point at higher N — treat it as a hypothesis to measure at mid-N (4-8), not a specification
- thread_mode in /config/imserver/get/set plausibly governs parallel-versus-serial dispatch and was never probed here (ships false in the doc example) — read it and record the value before quoting any skew number, including the "identical latency for one or two phones" baseline itself
- Near-simultaneous taps are not the same as near-simultaneous backend requests: each phone still independently renders, runs app logic and issues its own network request over its own Wi-Fi link, and that jitter (plausibly tens to hundreds of ms, given the 4.5x fps disparity measured between identical phones) will dominate over any HID-layer skew — the rig controls tap simultaneity, not request simultaneity
- Verification is the weak link, not firing: with no OCR or image-search (code 28), outcome classification is client-side pixel work that must be validated against known-good frames
- The slow phone's outcome frame can lag ~220 ms or more behind — infer a lost race only from frame content, never from frame timing
- /pic/screenshot is single-id, so observation is N concurrent requests even though firing is one (phones are independent: measured 83+85 req/s at two phones, 512 req/s aggregate at ten pollers)
- A stuck-click phone silently drops out of the volley and reads as a backend loss — /mouse/reset to the full set before firing is part of the experiment's validity, not hygiene
- Golden-diff classification only holds once all firing phones are on pixel-identical UI state; an app that personalises the result screen (name, avatar, locale) breaks a shared golden crop even on a correct deep link — crop from a region free of per-account content, not just free of chrome or wallpaper differences
- Running N accounts against a service you do not operate can breach its terms and trip its rate limiter — use staging, and back off on any block or throttle response rather than retrying through it

SCALE

Firing and pre-positioning are each one HTTP request regardless of N. Observation is N concurrent /pic/screenshot calls at 2 ms each on loopback (measured 84 req/s per phone flat out, 512 req/s across ten pollers), so observation scales to the full 19-slot rack even though only two phones exist today. 2 phones are race-testable now; 19 is the board ceiling, not a working number, until thread_mode is read and a mid-N (4-8) skew measurement is taken — firing stays one request at any N, so the API itself is not the constraint.

Diagnostics extraction via the iOS Shortcut

Harvest iOS crash logs (.ips) through Files without Xcode

NEEDS SHORTCUT WITH CAVEATS

After a crash, drive the phone's own UI to Settings > Privacy and Security > Analytics and Improvements > Analytics Data, open the app's .ips entry, use Share > Save to Files into an /iMouse/ folder, then pull the file to the PC with /shortcut/file/get and /shortcut/file/down for symbolication, no Xcode, no Mac, and no free Lightning port, since the only port is occupied by the control board. A crash frame from the mirror tells you the app died; the .ips log tells you why, and getting it off a rack phone normally needs a human with a cable and a Mac.

HOW

1. `get_device_screenshot` XP `GET /api/pic/screenshot?id=<id>&binary=true (optionally &rect=[l,t,r,b])` — trigger: watch for a crash-to-home transition. rect is a flat [left, top, right, bottom] array in XP, not Phonefarm's four-corner-point form. Detect with a mean-absolute-pixel-diff against a stored home-screen crop, never MD5.
2. `send_key + click` XP `GET /api/key/sendkey?id=<id>&fn_key=WIN+h (twice), then /api/mouse/click on the Settings icon` — normalise to a known state, then walk into Settings. Two Home presses: first leaves the app, second returns to page one — the XRs have different home screens, so the icon coordinate is per-phone.
3. `click + mouse_wheel` XP `/api/mouse/click and /api/mouse/wheel?id=<id>&direction=down&len=30&number=3` — walk Settings > Privacy and Security > Analytics and Improvements > Analytics Data. Wheel scrolling avoids the ~0.6 ms/px pointer-walk cost of a swipe. Verify each step with a golden diff before the next tap; this is roughly 10-15 taps and the fragile part of the chain.
4. `ocr / find_text` XP `/api/pic/find-text?id=<id>&text=["MyApp-2026-09"]&contain=true&similarity=0.8 — NOT AVAILABLE, do client-side instead` — find the app's .ips row in the alphabetical list. /pic/ocr returns code 28 here, so find-text will too. Pull the raw BMP, run OCR on the PC, click the row's centre converted into mirror-pixel space.
5. `click` XP `/api/mouse/click?id=<id>&x=&y= per step` — share sheet: open the entry, tap Share, tap "Save to Files", navigate to On My iPhone > iMouse, tap Save. The suggestion row reorders between runs and iOS versions, so locate "Save to Files" by content each time and golden-diff after each tap.
6. `shortcut#4` XP `GET /api/shortcut/file/get?id=<one id>&path=/iMouse/&outtime=20000` — confirm the file landed. id single-only, outtime ms (default 15000, note Phonefarm's was seconds). Returns data.list[{name, ext, size, create_time}]. iOS 15+ only.
7. `shortcut#5` XP `POST /api fun=/shortcut/file/down, id=<one id>, path=/iMouse/, list=[{"name":"MyApp-2026-09-10-101010","ext":"ips"}], zip=1, outtime=60000` — pull it to the PC. Single id only; list entries are {name, ext} split, not a Phonefarm full-path string; files land in the iMouse install directory's Shortcut\File folder. Raise outtime well above the 15 s default for anything but a tiny log.

8. `shortcut#6` XP `POST /api fun=/shortcut/file/del, id=<comma list allowed>, path=/iMouse/, list=[{"name":...,"ext":"ips"}]` — clean up on the phone. Comma list allowed here, unlike get/down; returns the 10 newest directory entries. Destructive, no undo, only run after the download is confirmed on disk.
9. `(outside the API)` XP `none – external (the build's dSYM on the PC or a Mac)` — symbolicate. Neither API symbolicates; the rack's job ends at a raw .ips on the PC.

NEEDS

- iOS 15 or later — the whole /shortcut/file/* family is gated on it
- iMouse Shortcut installed, IP edited, run once with every prompt allowed, bound to an AssistiveTouch key via Send Binding Key; Advanced permissions: allow running scripts, allow sharing/deleting large amounts of data, allow deleting without confirmation
- An /iMouse/ folder created in Files > On My iPhone beforehand — a missing path answers code 27
- Settings > Privacy and Security > Analytics and Improvements > Share iPhone Analytics enabled, or there is no Analytics Data list to harvest
- Board bound and working (the Shortcut launches via a HID hotkey) and the phone reachable on TCP 9911 over the LAN
- A per-model, per-iOS coordinate map for the Settings walk in mirror pixels, plus a golden crop per step, or client-side OCR to locate rows

WATCH OUT

- No server-side row-finding on this rig: /pic/ocr answers code 28, so /pic/find-text will too — run OCR on the PC against the raw BMP and convert hits back into mirror-pixel coordinates
- The Settings and share-sheet walk is the fragile part: Apple reorganises it between iOS versions and the share sheet's suggestion row reorders between runs, so every tap needs a content check and the whole map must be re-captured per model and iOS build
- File list and download take a single id ("Multiple is not allowed"); only /shortcut/file/del accepts a comma list — this is one phone at a time, serially
- outtime is milliseconds and defaults to 15000 in XP, not seconds as in the Phonefarm sibling — a ported controller will time out on every transfer, and 15 s is also too short for a large .ips, so raise it and set zip=1
- XP's list entries are {name, ext} pairs plus a separate path, not a Phonefarm full-path array — sending "/iMouse/1.ips" as a name will simply not match
- Analytics entries are not reliably named after your app alone, .ips files may arrive suffixed (for example .ips.synced), and the list on a busy phone is long — budget scrolling and be prepared for the extension the file actually carries
- Crash logs are personal data: an .ips can carry the device name, installed process names and paths; the download lands unencrypted under the iMouse install directory's Shortcut\File folder on an API with no authentication — harvest only from phones you own
- This is a per-phone serial walk of roughly 15 taps plus two Shortcut round trips, and a stuck-click state mid-walk would silently abandon it partway — send /mouse/reset before starting and verify each tap actually moved the screen
- Analytics Data entries are timestamped when Apple processes the crash, not when it happens, and can lag the crash-to-home transition by seconds to a few minutes — a fixed-delay walk needs a retry or poll loop, not a single attempt
- The Shortcuts Advanced permissions have no API-visible status; only a live file transfer confirms them, and an iOS update can silently reset them

- The automation can assert what it sent and what changed on screen, but never "did the Share Sheet actually save this to /iMouse/" — a Share Sheet failure surfaces several steps later as a missing file, indistinguishable from a mis-tap, a slow analytics index, or a full disk
- No /shortcut/* path has actually been probed on this rig — run the cheap /shortcut/device/ip round-trip check before designing anything downstream; steps fail with code 30 "Call timeout" if the Shortcut is not installed or bound
- Share iPhone Analytics is normally an opt-in accepted during first-run setup; a rack phone provisioned unattended may have it off, in which case the walk fails at step 3 with no clean error the API can see
- A major iOS update can pop a full-screen consent prompt mid-walk; a coordinate-driven tap lands on whatever the prompt shows, not on Settings

SCALE

About a minute per crash per phone: roughly 15 taps (pointer walk up to ~600 ms per full-height traverse), a scroll pass through an alphabetical list, plus two Shortcut calls each bounded by outtime. File get and download are single-id, so harvesting is strictly serial across the rack. Comfortable for dozens of crashes a day, not a high-frequency CI loop. Budget 20-60 s of API and board time per crash normally, worst case another 1-2 minutes for the analytics-lag retry, plus a few extra seconds per walk on the slow XR (~4.5 fps) for content checks. For a 19-board rig, expect 15-30 minutes to sweep every phone once.

Native screen-recording harvest for bug reports and animation review

NEEDS SHORTCUT

WITH CAVEATS

Start iOS's own Screen Recording from Control Centre by hotkey plus one tap, run the flow under test, stop it the same way, then list the newest album item with /shortcut/album/get, pull the RPReplay .mp4 with /shortcut/album/down, and clear it with /shortcut/album/del. The result is a native-resolution, encoder-noise-free recording at the phone's own frame rate, something the 500x1080 AirPlay mirror cannot produce, and the only usable motion capture on the slower XR (~4.5 new frames/s). The vendor's own /shortcut/album/get response example already returns an RPReplay .mp4, so this is a documented path, not a hack.

HOW

1. `n/a (phone-side setup)` XP `none` — one-time per phone: add the Screen Recording tile to Control Centre. No API starts or stops iOS screen recording, nothing in the 62 documented XP paths covers it. Keep the vendor's required "two rows of icons at the bottom" Control Centre shape; re-check /device/airplay/connect after the change.
2. `send_key (fn_key) + click` XP `GET /api/key/sendkey?id=<ids>&fn_key=ControlBar then GET /api/mouse/click?id=<ids>&x=<tile x>&y=<tile y>` — open Control Centre and tap the record tile. Both calls take comma-separated id lists, so N phones start recording from two requests. The tile coordinate must be read off a screenshot frame in mirror pixels; no find-image or OCR is available to locate it automatically.
3. `send_key (fn_key WIN+h), click, swipe, wheel` XP `GET /api/key/sendkey?id=<ids>&fn_key=WIN+h then the regression script's mouse calls` — leave Control Centre and run the flow under test. These XRs are on iOS 18.7.10 (the newer "Home" fn_key needs kernel 2026.2.27+ for iOS 26.1+). Remember /mouse/swipe walks the pointer at ~0.6 ms/px, so a full-height 1080 px swipe is ~580 ms of travel and iOS may read it as a slow drag rather than a flick.
4. `send_key (fn_key) + click` XP `GET /api/key/sendkey?id=<ids>&fn_key=ControlBar then click the tile again` — stop the recording. Tapping the same tile is the deterministic stop; tapping the red status pill instead raises a Stop/Cancel confirmation sheet whose coordinates differ per iOS version.

5. `shortcut id 1` `XP` `GET /api/shortcut/album/get?id=<id>&num=1&outtime=<ms>` — find the new file. id single-only, album_name blank for the most recent project's album, outtime ms default 15 s. The doc's own example returns {"album_name":"","name":"RPReplay_Final1718703292","ext":"mp4","size":"5.4 MB","create_time":"2024-12-18 23:54:08"}. XP has no date filter, unlike Phonefarm.
6. `shortcut id 2` `XP` `POST /api fun=/shortcut/album/down, id=<id>, list=[{"album_name":"","name":"RPReplay_Final<ts>","ext":"mp4"}], outtime=120000` — pull the mp4 to the PC. Single id, zip 0/1, outtime ms default 15 s; list entries split name and extension unlike Phonefarm's whole filename. Files land under the iMouse install directory's Shortcut\Media folder; raise outtime well past the default for a clip of any length.
7. `shortcut id 3` `XP` `POST /api fun=/shortcut/album/del, id=<a,b>, list=[{"album_name":"","name":"RPReplay_Final<ts>","ext":"mp4"}]` — clear the clip off the phone. id IS comma-separated here; returns the 10 newest album records to confirm the file is gone. /shortcut/album/clear wipes a whole album, never wire it to an unattended step. Deletion needs the Shortcuts "allow deleting without confirmation" and "allow deleting large amounts of data" permissions.

NEEDS

- Screen Recording added to Control Centre on each phone, without breaking the vendor's "two rows of icons at the bottom" layout that automatic screencasting depends on
- iMouse Shortcut installed, IP edited, run once with all prompts allowed, bound to an AssistiveTouch key; Advanced permissions per the crash-log entry above
- USB board bound and connected (both the Control Centre taps and the Shortcut hotkey travel over it) plus LAN reachability from phone to kernel on 9911
- Free storage on the phone for the clip, and disk space under the iMouse install directory's Shortcut\Media folder on the PC
- The record-tile coordinate captured per phone/iOS version from a screenshot frame in mirror-pixel space, since no OCR or image-search is available here to find it
- Phone unlocked, portrait locked, Auto-Lock Never

WATCH OUT

- Shortcut status is unverified on this rig, no /shortcut/* path was probed on 10 Sep 2026 — prove it with the /shortcut/device/ip round trip before depending on the harvest
- Start and stop are blind UI taps at a fixed coordinate; with no recognition plug-in, verification is only a mean-pixel-diff on a small box around the tile (idle floor 0.02-0.03, real reaction 20-160) or the red recording pill appearing — never MD5-compare
- Adding the record tile changes the Control Centre layout, which the vendor docs explicitly tie to automatic screencasting — re-test /device/airplay/connect after the change, and re-locate every other required tile, since insertion shifts everything after it
- /shortcut/album/get and /album/down are both single-id, listing and downloading are strictly one phone at a time even though start, stop and delete fan out over a comma list
- XP dropped Phonefarm's date filter on the album list (album_name plus num only) — identify the new clip by create_time or by clearing the album first
- outtime defaults to 15 s; a multi-megabyte mp4 will exceed that with no documented ceiling for a raised value — a timeout surfaces as code 30
- Screen recording adds CPU and thermal load to the device under test and can itself change the performance being measured

- Apps that block screen capture (DRM video, some banking and streaming apps) record as black — iOS or app behaviour, not something the API reports
- Whether the AssistiveTouch pointer appears in an on-device recording is unverified — confirmed visible in the AirPlay frame, not established for the on-device recorder
- Every phone must keep the AssistiveTouch floating button on as a control prerequisite, and iOS screen recording captures it as a real on-screen overlay — every harvested clip shows a persistent floating button unless it is hidden, and hiding it either removes the pointer needed for the rest of the automation or needs its own blind, unverifiable re-show step afterward
- The 3-second start countdown means the flow under test should include a deliberate pause after "start recording," or the opening seconds of every clip show the countdown UI, not the app
- Recordings capture notifications, account names and any personal data on screen — record only on test devices and accounts you own, and review before attaching a clip to a bug report

SCALE

Start, stop and delete are multi-id, so a whole rack can begin recording the same flow from a handful of requests. Harvest is the bottleneck: get and down are single-id, and a one-minute clip is tens of megabytes per phone over the LAN, so a 19-phone rack's harvest phase alone runs on the order of 5-15 minutes serially with a generous (~120 s) outtime, meaning a handful of full-rack record-and-harvest cycles per hour rather than a per-test-case capture for a large regression suite. Screenshot polling for tile verification costs nothing measurable alongside this, since phones are independent.

Clipboard as a structured test channel: pull diagnostics out, push Unicode fixtures in

NEEDS SHORTCUT WITH CAVEATS

Give a test build a hidden "copy diagnostics" action that writes JSON (build hash, feature flags, last error, counters, memory) to the iOS pasteboard; a long-press via /mouse/click triggers it and /shortcut/clipboard/get returns the text for exact assertions with no OCR. In the other direction, /shortcut/clipboard/set seeds Unicode and hostile fixture strings onto many phones in one request, and fn_key=WIN+v pastes them into the focused field for rendering, truncation and crash checks. This rig has no server-side OCR at all, so the clipboard is a lossless Unicode side channel already exposed in both directions, turning screen-scraping assertions into structured ones.

HOW

1. `click` XP `GET /api/mouse/click?id=<id>&x=<x>&y=<y>&time=1000` — trigger the test build's copy-diagnostics action with a long press. time is the press-to-popup delay in ms, so a long-press needs no separate API. x/y must be mirror-image pixels (500x1080 here), never iOS points (414x896).
2. `shortcut id 11 (professional version only)` XP `GET /api/shortcut/clipboard/get?id=<id>&outtime=<ms>` — wait, then pull the JSON off the pasteboard. id single-only, outtime ms default 15 s. Response is data.text plus data.code/data.id; parse the JSON on the PC and assert build hash, flags, last error, counters. XP states no edition gate on this path.
3. `shortcut id 10 (professional version only)` XP `POST /api {"fun":"/shortcut/clipboard/set","data":{"id":"a,b,c","text":"<fixture>","sleep":1}}` — seed a fixture string onto every phone under test in one request. id comma-separated; sleep is a SECONDS delay ("on some phones paste may take a while to become effective"), outtime is ms. Use POST application/json UTF-8 rather than a GET query string for non-ASCII text.

4. `send_key (fn_key)` XP `GET /api/mouse/click?id=<ids>&x=&y= then GET /api/key/sendkey?id=<ids>&fn_key=WIN+v` — focus the field and paste. The hotkey table lists WIN+v Paste, plus OPENKeyboard, WIN+a Select All, WIN+x Cut, SwitchIme, BACKSPACE — useful for clearing a field between fixtures. Requires Full Keyboard Access on.
5. `send_key (key)` XP `GET /api/key/sendkey?id=<ids>&key=<ascii text>` — ASCII typing-speed cases (search-as-you-type, per-keystroke handlers) with no Shortcut at all. key supports only English, numbers and English characters; Chinese and Unicode are explicitly unsupported, which is why the clipboard route exists.
6. `get_device_screenshot` XP `GET /api/pic/screenshot?id=<id>&binary=true` — verify what landed on screen. Raw BMP measured 2 ms / 1,620,054 bytes; jpg 40 ms, capped ~21 req/s. Compare frames with mean absolute pixel difference, never MD5.
7. `shortcut id 10 then id 11` XP `/api/shortcut/clipboard/set?id=<id>&text=imouse-<ts> then /api/shortcut/clipboard/get?id=<id>` — round-trip self-test before trusting any assertion. The identical string must come back in data.text; this pair proves the Shortcut is installed, the AssistiveTouch hotkey fires, and the phone can reach the kernel on 9911. data.code 30 "Call timeout" means the hotkey never launched the Shortcut.

NEEDS

- iMouse Shortcut installed on every phone, IP edited to the kernel (192.168.4.29 here), run once with all prompts allowed, bound to an AssistiveTouch key via Send Binding Key; Advanced permissions as above
- USB board bound and connected (the Shortcut launches via a HID hotkey) and the phone reachable on the LAN, since the Shortcut posts its result back to the kernel on 9911
- Full Keyboard Access on with Auto-Hide 1 s, for WIN+v and OPENKeyboard
- A test build with a copy-diagnostics affordance — this is an app-side change, not an API capability
- Per-phone imgw/imgh read from /device/get so the long-press coordinate is in mirror-pixel space
- Phone unlocked, Auto-Lock Never, portrait locked

WATCH OUT

- Shortcut status is unverified on this rig, the 10 Sep 2026 probe never called any /shortcut/* path — run the cheap round-trip check before building anything on this
- Asymmetric fan-out: /shortcut/clipboard/set takes a comma-separated id list, but /clipboard/get is single-id only — harvesting diagnostics is strictly one request per phone
- Unit trap: sleep is in SECONDS, outtime is in MILLISECONDS (default 15000) on XP; the Phonefarm sibling's outtime was in seconds — getting this backwards produces either a 15 ms timeout or a 15,000 s stall
- The diagnostics JSON sits on the system pasteboard, readable by any app, over an API with no authentication — keep credentials, tokens and customer data out of it
- iOS shows a "pasted from <app>" banner and may prompt for cross-app paste permission the first time; this changes pixels (allow for it in diff thresholds) and needs answering once per phone
- No OCR fallback if the clipboard path fails: /pic/ocr and its variants sit behind the plug-in, which reported code 28 here — any screen-side assertion must be client-side on raw BMP
- A long-press is state-dependent: assert what you sent and verify by pixel diff, never assume the debug menu opened; normalise with two Home presses first, since the XRs have different home screens
- Round-trip latency for the Shortcut path is not measured on this rig, the "1-3 s" estimate is a guess; the only documented bound is outtime (default 15 s)
- Phonefarm's Chinese-entry hotkey (CTRL+ALT+SHIFT+WIN+v with a send_text_ok callback) does not exist on the XP kernel, it is absent from the endpoint list and bare Phonefarm names return HTTP 500 —

clipboard plus WIN+v is the only documented Unicode path here

- A full-rack pre-flight adds one serial 15-s-capped call per phone before any real test cycle — budget this into CI wall-clock
- No documented ceiling exists for clipboard text length; before relying on large fixtures, empirically determine the truncation point, since POST body size, Shortcut action limits and iOS pasteboard limits are three separate places it can be capped
- WIN+v only pastes if a field currently has keyboard focus; a bare tap does not always raise the keyboard on iOS (a field behind a picker, or one needing a second tap) — verify a text cursor or keyboard is visible before trusting a paste, since a failed focus produces no visible page change worth a pixel diff
- A "pasted from clipboard" banner or first-use permission prompt is itself UI state the automation must detect and dismiss — a phone stuck on an undismissed system alert fails every subsequent tap the same way a mis-click does

SCALE

Fixture seeding, the paste hotkey and the focusing tap are each one request for the whole rack, comma-id fan-out at board-walk cost. Diagnostics harvesting is the bottleneck: /clipboard/get is single-id, bounded by outtime (default 15 s, though a small read is likely well under 1 s once the Shortcut fires). Screenshot verification is also single-id but phones are independent (83-85 req/s per phone measured, 512 req/s across ten pollers) and does not compete with input traffic. A full rack can likely run one fixture cycle every 1-3 seconds on the push side; harvesting realistically adds well under a minute per fixture across 19 phones once the per-phone Shortcut pre-flight is amortised, giving on the order of 30-60 fixture cycles per hour, gated entirely by the unmeasured Shortcut round-trip.

Also worth knowing

Game QA and landscape apps

Game QA: onboarding runs, IAP sandbox checks, crash and hitch detection, landscape

PARTIAL

WITH CAVEATS

Games are one of the largest mobile-only categories and the one case where a rack of real handsets, not a simulator, is obviously the right tool: real touch latency, real StoreKit purchase sheets, real GPU-bound frame delivery. The tutorial and onboarding run, the frame-arrival crash and hitch signal, and the multi-device soak loop all reuse the visual-regression and watchdog machinery built above; the new pieces are held-key and drag-based game controls, and landscape, which is the default orientation for this whole category and is otherwise covered in this document only as a portrait-lock assumption.

HOW

1. `get_device_list` XP `GET /api/device/get?id=<ids>` — read rotate, imgw/imgh and state per phone before a run; rotate flags which phones are already in landscape.
2. `mouse_reset_pos` XP `GET /api/mouse/reset?id=<ids>` — clear the silent stuck-click state before any tap-critical sequence; a missed tutorial tap derails the whole run and looks identical to a game bug.
3. `send_key WIN+h` XP `GET /api/key/sendkey?id=<ids>&fn_key=WIN+h` — two presses, force every phone to home page one before launch.

4. `click` XP `GET /api/mouse/click?x=<px>&y=<py>` — launch the game icon; per-phone coordinates, home layouts differ.
5. `WS callback dev_rotate` XP `ws://127.0.0.1:9911/api` — subscribe before the game forces landscape; the callback carries the full device record with rotate flipped to 1 and imgw/imgh swapped, the cue to load that phone's landscape coordinate map and goldens.
6. `mouse_down + mouse_move + mouse_up` XP `GET /api/mouse/down, GET /api/mouse/move, GET /api/mouse/up` — the general touch-control path: press at a virtual joystick or button origin, stream move for a drag or analog gesture, release; works for any touch-driven title.
7. `key_down / key_up` XP `GET /api/key/down?id=<ids>&key=<key>, GET /api/key/up?id=<ids>&key=<key>` — hold and release a single key as a game control, for the minority of titles built to accept a hardware keyboard.
8. `key_release_all` XP `GET /api/key/upall?id=<ids>` — release every held key on that phone in one call; run before ending or restarting a session so a stuck hold does not bleed into the next run.
9. `click with time` XP `GET /api/mouse/click?x=<cx>&y=<cy>&time=<ms>` — a tap-and-hold action: charge a shot, skip a cutscene, or confirm on a store sheet that requires a long-press.
10. `get_device_screenshot`, polled XP `GET /api/pic/screenshot?binary=true` — the frame-arrival poll that stands in for a crash and hitch signal, reusing the watchdog design above; also captures the purchase-sheet, tutorial-step and landing frames for pixel-diff verification.
11. `send_key WIN+h` + `config/imserver/set flip_right` XP `GET /api/key/sendkey?fn_key=WIN+h, GET /api/config/imserver/set?flip_right=<bool>` — return to home between runs; flip_right is the kernel's landscape mirroring option, worth testing once against a game whose taps land offset.

NEEDS

- A per-phone, per-orientation coordinate map for the tutorial taps, joystick origin and purchase-sheet buttons; rotation swaps imgw/imgh, so a portrait map does not carry over to landscape.
- Orientation lock off, or set to allow landscape, on any phone running a landscape-only game. Every other use case in this section assumes orientation lock on for normalisation; a rack running both portrait apps and landscape games needs a per-phone accessibility policy, not one rack-wide setting.
- A sandbox or test Apple ID (an App Store Connect sandbox tester) signed in on each phone used for IAP testing, against the developer's own app.
- A game confirmed to accept hardware-keyboard input before building any control scheme around key_down/key_up; most touch-only titles will not react to a held key at all.
- The same board, mirror and IMConsole.exe prerequisites as every other use case in this section, plus per-phone golden frames captured separately for portrait and landscape.

WATCH OUT

- /key/down's own parameter table lists fn_key (combination hotkeys) alongside key, but its documented example URL calls fun=/key/sendkey, not /key/down. Confirm on this rig which fun value the kernel actually accepts for a held key before depending on it.
- key_down/key_up only reaches games that read a hardware keyboard (GameController-framework titles, some emulators, a few keyboard-compatible arcade or strategy games). The general-purpose control path for touch games is mouse_down, mouse_move, mouse_up, still subject to the measured ~0.6 ms per pixel board-walk latency, which reads as input lag against twitch, precision-timing gameplay.
- Frame-arrival timing detects the AirPlay mirror stalling, not the game engine's true internal frame rate. On the slow XR (~4.5 new frames/s under motion versus ~29 on its twin) a normal frame gap can look identical to a hitch, so baseline per phone first, exactly as in the crash and hang watchdog above.

- flip_right is one kernel-wide toggle, not per device: a rack cannot mirror some phones flipped and others not at the same time. Beyond "Horizontal screen flip right mode" its effect on the tap-coordinate mapping is undocumented; verify it empirically on one landscape phone before trusting any landscape tap coordinate.
- dev_rotate firing reliably is unconfirmed on this rig. Treat rotate in /device/get as the source of truth and dev_rotate as a push convenience until it has been observed live.
- The purchase sheet is real system UI: tapping its confirm control completes a real transaction unless the signed-in account is a sandbox tester. Never script past that tap without a human check that the account is a test account, and scope this only to the developer's own app.
- No OCR (code 28) means no reading of tutorial copy, currency counters or dialog text; tutorial-completion and IAP-success checks fall back to whole-frame or masked pixel diff against per-orientation goldens, the same ceiling as the rest of this section.

SCALE

Touch and key input fan out over id lists at effectively no marginal cost, so the pacing limit is the same one measured for visual regression: each phone's own frame-capture and settle budget, roughly 300 to 600 verified checks per phone per hour on this hardware. Running one tutorial-to-IAP loop continuously across the two live phones is a straightforward multi-hour soak today. Scaling to the other 17 boards means bringing them onto mirroring first, since board count here is control capacity, not video capacity, and does not by itself multiply a soak's throughput.

OPEN **What flip_right actually flips is undocumented**

The kernel config doc defines flip_right only as "Horizontal screen flip right mode," true by default, settable through /config/imserver/set without a kernel restart. It does not say whether the flip changes the mirrored image, the tap-coordinate mapping, both, or neither, and it is one value for the whole kernel, not one per device. Before trusting a single landscape tap coordinate, flip the setting on one phone, capture a screenshot, and confirm by test click where the coordinate space actually lands, the same verification discipline already used for portrait in this rig's measurements.

Dynamic Type, Bold Text, Reduce Motion, Increase Contrast and colour-filter sweeps against golden frames

PARTIAL WITH CAVEATS

Accessibility QA is not just focus order. Put one phone into a single Accessibility setting at a time, re-run the exact same navigation script and golden-frame suite used for visual regression, and diff every screen against its default-state golden. Larger text and Bold Text catch truncation and overlap; Reduce Motion and Increase Contrast catch missing states that only differ under those flags; a colour filter is the one variant a script can grade on its own, because grayscale and inversion are checkable arithmetic on the raw BMP rather than a judgement call. This reuses the golden-frame pipeline already in this section rather than inventing a new one.

HOW

1. `get_device_list` XP `GET /api/device/get` — confirm the phone is mirroring, read imgw/imgh, and require rotate == 0 before trusting any golden.
2. `mouse_reset_pos` XP `GET /api/mouse/reset?id=<id>` — clear the silent stuck-click state before the walk.
3. `send_key WIN+h` XP `GET /api/key/sendkey?fn_key=WIN+h` — two presses, normalize to home before touching Settings.

4. `shortcut#13 Open URL` (attempt) XP `GET /api/shortcut/exec/url?id=<id>&url=App-Prefs:root=ACCESSIBILITY` — try to land directly on the Accessibility pane; this URL scheme is Apple's own, not documented anywhere in the iMouse doc set, and is not confirmed to still work on iOS 18.7.10. Treat it as a probe, not a dependency.
5. `click` (fallback) XP `GET /api/mouse/click` — the no-dependency path when the deep link fails: tap through Settings by per-phone coordinate, same as any other native navigation in this section.
6. `none` (manual) XP `Settings > Accessibility, one toggle at a time` — turn on exactly one of: Larger Text at the maximum slider position, Bold Text, Reduce Motion, Increase Contrast, or one Color Filter (Grayscale or a colour-blindness filter). No XP or Phonefarm function reads or writes any of these; this step is off the API entirely.
7. `send_key WIN+h + shortcut#13 / click` XP `GET /api/key/sendkey?fn_key=WIN+h, then the same navigation calls as the golden-frame suite` — walk the identical screen list used to build the default-state goldens; a different script or a different order invalidates the comparison.
8. `get_device_screenshot`, polled XP `GET /api/pic/screenshot?id=<id>&binary=true` — settle loop on new frames at a fixed 33-50 ms period, exactly as the golden-frame card does, before capturing each screen.
9. `get_device_screenshot` with rect XP `GET /api/pic/screenshot?id=<id>&binary=true&rect=[l,t,r,b]` — crop to a specific control (a button, a label) when the whole-frame diff is too noisy to localize a clipped element under larger text.
10. `none` (client-side) XP `mean absolute pixel diff, masked, against the default-state golden` — for Larger Text, Bold Text, Reduce Motion and Increase Contrast this only flags "something changed here," since the setting is supposed to change the render; a human or vision-model pass classifies each flagged region as expected reflow or actual clipping.
11. `none` (client-side) XP `channel-level check on the raw BMP: near-zero colour saturation for Grayscale, inverted luminance ordering for Smart Invert` — the one variant with an objective automatic pass, since a true colour filter is a deterministic transform of the pixels, not a subjective layout change.
12. `find_image_cv` (optional) XP `POST /api/pic/find-image-cv?id=<id>&img_list=[...]&similarity=0.8` — assert a specific control template still matches under the filter instead of eyeballing the crop; plug-in gated, code 28 on this rig.
13. `send_key OPENKeyboard` XP `GET /api/key/sendkey?id=<id>&fn_key=OPENKeyboard` — raise the on-screen keyboard on a text field and capture it separately; keyboard chrome is a common place for Bold Text and Larger Text to clip a key label or push the return key off-frame.
14. `send_key TAB+b / WIN+h` XP `GET /api/key/sendkey?fn_key=TAB+b|WIN+h` — leave the screen and advance; watch dev_rotate, which invalidates every golden in the sweep for that phone.

NEEDS

- A complete set of default-state goldens per screen per phone, built by the golden-frame visual-regression card above; this sweep is a second pass over the same screens, not a standalone suite.
- One phone held in a single Accessibility state for the whole pass; none of Larger Text, Bold Text, Reduce Motion, Increase Contrast or a colour filter can be read or set through either XP or the Phonefarm API, confirmed absent across the configuration, shortcuts and equipment-related docs.
- A masking strategy that is per-region, not whole-frame: Larger Text and Bold Text change glyph size everywhere on every screen, so an unmasked whole-frame diff will flag every single screen as changed, hiding the genuinely broken ones among the expected ones.
- A reviewer, human or vision-model, for every flagged screen under the size- and contrast-based variants; only the colour-filter variant supports a fully automatic pass.

WATCH OUT

- No API reports which Accessibility setting is currently active on a phone. The script has to trust whoever set it; there is no readback to catch a state that silently reverted (an iOS update, an accidental tap) before or during a run.
- App-Prefs deep links are an Apple convention Apple has restricted release over release specifically because third parties abused them; treat step 4 as opportunistic, confirm it lands on the Accessibility pane on this exact iOS build before relying on it, and keep the tap-through fallback as the real path.
- Increase Contrast and colour filters can shift colours the app never renders under default settings (system-drawn borders, button fills), which is correct behaviour, not a defect; a naive full-image similarity score will read every screen as "failed" unless the comparator is filter-aware.
- Never hash-compare frames here either: the same AirPlay encoder noise applies under every Accessibility state, and the idle floor (0.02-0.03) and reaction range (20-160) from the base golden-frame suite still apply as the significance threshold.
- Running each state serially on one phone means the sweep cannot be parallelized across states the way the base suite parallelizes across screens; budget one full pass per state, not five states at once.

SCALE

Capture cost is identical to the base golden-frame suite: about 2 ms per raw BMP poll, a 1-2 second settle on the fast XR and 2-4 seconds on the slow one, so a 20-screen suite still runs in roughly 30-60 seconds per phone per state. The state change itself is the added cost, not the API: each of the five variants here (Larger Text, Bold Text, Reduce Motion, Increase Contrast, one colour filter) is a manual toggle plus a full re-run, so a five-state sweep over a 20-screen suite is five sequential 30-60 second passes on one phone, on the order of 5-10 minutes of capture, plus however long the human or vision-model review of flagged screens takes for the four variants that are not the colour-filter check.

WRONG **The rack is not silent**

An earlier draft refuted automated VoiceOver auditing on the premise that "the API exposes no audio capture at all." That overstates it. `air_play_audio` (global, configuration-related doc) and the per-device `air_audio` field returned by `/device/get` and set by `/device/airplay/set` control whether the AirPlay mirror itself carries system audio, VoiceOver's spoken output included; with it on, a human sitting at IMConsole's own mirror window hears the phone. What is actually true, and what the refutation should have said, is narrower: no documented HTTP endpoint captures, records or returns that audio stream. `/pic/screenshot` returns a video frame only, and no function in any of the nine reference docs offers an audio-equivalent capture call. So live human listening against a mirrored phone is possible today with `air_audio` enabled; turning that into an automated, scripted VoiceOver pass or fail is not, because there is nothing to poll. The separate point about VoiceOver changing gesture semantics (single-tap-to-select instead of single-tap-to-activate) still stands on its own and remains a real reason this rig's pointer-based control path cannot drive a phone with VoiceOver on.

SECTION 9 · FLEET OPS

Fleet operations and self-healing

A rack is not one phone repeated nineteen times, it is nineteen independent pairs of channels, control over USB and video over AirPlay, each of which fails on its own schedule. This part covers the registry that makes every slot findable, the health board that reports the two channels separately instead of collapsing them into one flag, the two background watchdogs that catch failures the vendor console does not surface (licence headroom, silent post-update miscalibration), and a scheduler pattern that keeps a job queue running against whichever channel is actually up rather than idling a slot the moment either one drops.

Rack registry and live health

Rack roster: slot names, groups and board bindings as the registry

NATIVE API · WITH CAVEATS

Treat the kernel's own device list as the rack's system of record instead of a side spreadsheet. Every phone record already carries a free-text slot name, a group id, the bound board's vid/pid/uid, video state plus air_pid, and location_crc, and every edit broadcasts to all connected consoles as `dev_change`, `group_change` or `dev_sort_change`. On this rig both XRs still show empty names, group 0 (未分组) and boards C241BFBD/C214BEBD, exactly the kind of identifier nobody can find by hand across nineteen boards.

HOW

1. `get_usb_list` + `get_device_list` XP `GET /api/config/usb/get ; GET /api/device/get` — read boards and phones, the two lists this whole roster joins on
2. `set_group` XP `GET /api/device/group/set?id=0&name=ClientA-iOS18` — create a fleet group (id=0 means add new)
3. `set_dev` XP `GET /api/device/set?id=<deviceid>&name=H10-P02&gid=<gid>` — label the slot and assign the fleet in one call, comma-list id for bulk renames
4. `set_dev` (vid+pid) XP `GET /api/device/set?id=<deviceid>&vid=720A&pid=<pid>` — bind the phone to its control board, one device per call by vendor rule
5. `mouse_collection_open/cfg/close` + `save_dev_location` XP `GET /api/device/collection/mouse?id=<id>&cmd=1|0|2 ; GET /api/device/collection/mouse/save?id=<id>&describe=...` — require a calibration before declaring the slot usable
6. none XP `GET /api/device/group/get-dev?id=<gid>` — query a fleet server-side instead of filtering the full list client-side (id=0 is the ungrouped bucket)
7. none XP `GET /api/device/sort/set?sort_index=<n>&sort_value=-1|0|1` — sort the roster into rack order, mirrors the vendor console's own column order
8. `callbacks` XP `ws://127.0.0.1:9911/api` — subscribe once, take `dev_change`, `dev_delete`, `group_change`, `group_delete`, `dev_sort_change`, `usb_change` instead of polling
9. `discon_airplay` + `del_dev` + `del_group` XP `GET /api/device/airplay/disconnect?id=... ; GET /api/device/del?id=... ; GET /api/device/group/del?id=...` — retire a slot (disconnect before delete; empty a group before deleting it, code 34 refuses otherwise)
10. none XP `none — Windows PnP` — enrich with the physical hub/port label; neither API edition exposes topology

NEEDS

- IMConsole.exe running and logged in; with only IMService.exe up, `/device/get` read state 0 for both phones and `/config/usb/get` returned no boards at all.
- A device record only exists after the phone has cast to the AirPlay receiver at least once; there is no create-device call.
- Board cabled to the PC before the phone is cabled to the board (state 1 then state 2), vendor cable only.
- Windows PnP read access if hub/port slot labels are wanted.
- No plug-in, Shortcut, OCR or Pro-edition gate applies to any call in this chain, the one capability class on this rig that needs none of them.

WATCH OUT

- The read side is proven live on this rig; the write side (`/device/group/set` , `/device/set` , the deletes, `/device/sort/set`) was never exercised in the read-only probe. The first write is integration testing against production hardware with two live, cabled phones.
- Board state 1 vs 2 carries the whole control-channel signal, but the XP field table only documents vid/pid/uid/ver, not state. The 1/2 reading is a measured rig fact, not a vendor guarantee, re-verify after any kernel update.
- "Controllable" and "visible" are independent and must display separately: board state 2 plus a bound vid/pid is control-ready, `/device/get` state non-0 with a live air_pid is video-ready. One rig snapshot showed 13 boards with a phone attached and only 1 phone mirroring.
- location_crc non-empty proves a calibration is assigned, not that it fits this handset. Both XRs share library CRC B0DBD800; a wrong CRC silently misplaces every tap, only a pixel-diff probe can catch it.
- Neither edition exposes hub or port; the slot label has to come from Windows PnP and be maintained by hand whenever a board is reseated.
- `sort_index` maps to the vendor console's own header columns with no published column list, probe which index is which before shipping it.
- Calibration is strictly serial, one device at a time (code 5), and it is a QR-scan flow driven from the console, not an instant API call. Budget a full-rack sweep as multi-hour, largely manual work, not a batch job.
- Names have no documented length limit, character set or uniqueness constraint; the kernel will let two slots share a name, enforce uniqueness in the controller.
- Nothing enforces matching imgw/imgh within a group; a controller that treats group membership as a safe-to-mirror-together signal needs its own filter on top.
- Auto-binding a board by pixel-diffing the pointer only works once a phone is already mirroring; a phone that is only control-ready has no board-to-screen confirmation available for its very first bind and must be identified by the cable and port it was physically plugged into.

SCALE

Every read returns the whole rack in one call, `/device/get` with no id, `/config/usb/get` , `/device/group/get-dev` per fleet. Renaming and regrouping batch over a comma id list; hardware binding is strictly one phone per call by vendor rule, so a fresh nineteen-slot rack costs nineteen bind calls, once. Calibration is the real cost center, serialized one device at a time and gated on an operator confirming a QR scan, so treat a fleet-wide sweep as operator-hours, a handful of phones per hour, not an API rate. Steady state after the initial build is free: the kernel broadcasts every edit to every connected client, so a roster refresh never needs to poll, and this traffic does not compete with the measured frame pipeline (84 req/s per phone) or with input traffic.

Two-channel health board: control state versus video state per slot

NATIVE API · WITH CAVEATS

A live dashboard that renders every slot as two independent lights, CONTROL (board enumerated on the PC with a phone cabled and bound) and VIDEO (an AirPlay session actually up), instead of one "online" flag. Two HTTP reads at startup join phone records to board records on uid; after that one WebSocket carries the kernel's broadcast callbacks, and a slow fixed-period reconcile poll covers the events the callback vocabulary does not emit.

HOW

1. `get_device_list` XP `GET /api/device/get` — read the phone inventory at startup (optional id filter)
2. `get_usb_list` XP `GET /api/config/usb/get` — read the board inventory at startup (19 boards, VID 720A on this rig)
3. none XP `client-side join on uid` — CONTROL green = board.state 2 AND a device record carries that uid; CONTROL amber = board.state 1
4. device record state XP `/device/get list[].state` — decide VIDEO on state alone, never on air_pid, which the vendor's own disconnect example leaves populated after disconnect
5. callbacks XP `ws://127.0.0.1:9911/api` — open one socket and take the broadcast stream, there is no subscribe call, holding it open is the subscription
6. `get_airplaysrvnum` XP `im_connect callback, data.ver` — read the kernel version once, the only source of it on XP
7. callbacks XP `dev_connect, dev_disconnect, dev_rotate` — drive the VIDEO light
8. callback XP `usb_change (data.code 19/20/12)` — drive the CONTROL light
9. none XP `dev_change, dev_delete` — catch a rebind, rename or delete made by any other client on the LAN
10. `get_device_list` + `get_usb_list` XP `GET /api/device/get + GET /api/config/usb/get on a fixed 5–15s timer` — reconcile poll as the safety net for events the callbacks do not emit
11. none XP `error_push callback, data.call_fun` — surface per-call hardware failures as a rack-level ticker
12. none XP `none – Windows PnP` — label each row with a findable physical slot

NEEDS

- IMConsole.exe running and logged in; otherwise the board renders all-dark, all-state-0, which looks like a hardware failure but is a missing software prerequisite.
- XP-edition kernel on TCP 9911 only, path style, Phonefarm-style function names return HTTP 500 on this kernel.
- One long-lived WebSocket client with reconnect-and-refetch on drop, there is no replay or sequence number, a missed broadcast is invisible until the reconcile poll catches it.
- Phones bound to boards via `/device/set` with vid and pid sent together (single-device call) so uid exists for the join.
- Everything in this chain is a GET or a callback, no `/mouse`, `/key` or write call is needed to render the two lights.

WATCH OUT

- The CONTROL light's whole premise, state 1 versus 2, is not in the documented field table, only vid/pid/uid/ver are. It is a measured rig fact, re-verify after any kernel update by pulling one cable and watching the value.
- `usb_change` has no documented event for "phone unplugged but board stays on the PC" (only connected/disconnected-from-computer and connected-to-phone). A callback-only CONTROL light latches

green after a cable pull, the reconcile poll is mandatory, not optional.

- air_pid is not a disconnect signal, the vendor's own dev_disconnect example carries state 0 with air_pid still populated. Use state as sole authority; air_pid is a hint only when it matches the running AirplayMirroring.exe PID.
- air_pid and air_handle are example-only fields, absent from the documented field table entirely.
- error_push's worked example carries an empty data.id, so per-slot attribution is not guaranteed, render it rack-level keyed by call_fun unless id is actually populated at runtime.
- Two green lights mean necessary, not sufficient. iOS-side prerequisites (AssistiveTouch, Full Keyboard Access, tracking speeds, Auto-Lock Never) are invisible to the API, and neither light can see the measured stuck-click state where `/mouse/move` keeps working while `/mouse/click` silently does nothing until `/mouse/reset`.
- The WebSocket has no replay or sequence numbers, a dropped connection during a state change is a permanently missed event until the reconcile poll catches it. Document the reconcile interval as the real staleness bound, not the socket.
- Use a fixed reconcile period, never one derived from observed event gaps, adaptive pacing on the frame pump measured decaying 23 fps to 16.5 fps through positive feedback on this rig, and the same trap applies to any self-timed polling loop.
- The hub/port slot label needs a Windows PnP or WMI reader entirely outside the HTTP API, real added engineering scope, not a REST call.
- "The same code handles roughly a hundred phones" is an assumption, not a measurement, only 19 boards and 2 phones were observed here.

SCALE

Two GETs at startup regardless of rack size, then one shared WebSocket that costs the kernel the same whether one console or several are watching, plus a reconcile poll of the same two GETs on a fixed low-rate timer, five to fifteen seconds, never adaptive. Measured control-plane and input traffic do not contend with frame polling (83-85 req/s per phone unaffected by concurrent input), so a health board running beside a live video wall is free, well under 1 req/s aggregate for this 19-board rack. The practical ceiling is the kernel's own device-count licensing and AirPlay bandwidth, not this board.

TRAP **A latched green light is worse than no light**

There is no `usb_change` code for "phone detached from a board that stays on the PC", so a health board driven only by callbacks will show CONTROL green forever after a cable pull. The fixed reconcile poll (five to fifteen seconds, never adaptive) is not an optimization here, it is the only thing standing between the dashboard and a false green.

Account and calibration watchdogs

Licence headroom and expiry watchdog on the kernel account

NATIVE API WITH CAVEATS

The kernel refuses to bring phones online when it is not logged in or over its authorised device count, and the only signal is a Chinese log line pushed on the WebSocket. This watchdog holds one connection, reads the licence and account fields the kernel already pushes every 30 seconds, computes headroom against the correct

ceiling for the account type, counts days to expiry from the kernel's own clock, and raises a dated warning before a phone silently sits at state 0. Re-login stays a manual operator action in IMConsole.exe, the controller only reads state, never a password.

HOW

1. callbacks XP `ws://127.0.0.1:9911/api` — open one WebSocket for the push channel, the Phonefarm /clinet path does not exist here
2. `get_airplaysrvnum` XP `im_connect callback, data.ver` — record the kernel version on connect, the only version readout on XP
3. none XP `GET /api/config/user/info` — read licence and account state on demand, no parameters
4. none XP `callback user_info` — consume the unsolicited 30-second push, same payload as the GET above
5. none XP `client-side arithmetic` — ceiling = total_license when user_state == 1, else local_total_license (cross-check test_num); headroom = ceiling minus dev_online_num
6. none XP `client-side, overdue_time vs network_time` — compute days to expiry from the kernel's clock, not the PC's, alert at 14/7/1 days
7. none XP `callback im_log, data.message` — surface the refusal string 未登录，不允许上线 as a hard alert, data.code is 0 even here
8. none XP `callback logout (and pass == false in user_info)` — alert on the account dropping out
9. none XP `GET /api/config/user/switch?id=1-10` — optionally move the rack to another sub-account pool
10. `get_device_list` XP `GET /api/device/get` — correlate the licence ceiling to actual slots, count state != 0 against dev_online_num
11. none XP `/config/user/login and /config/user/logout` — operator actions in IMConsole.exe, never automated

NEEDS

- XP kernel on TCP 9911, path style only, Phonefarm-style function names return HTTP 500 here and 9912 is closed.
- A WebSocket client to catch im_connect, user_info, im_log and logout; an HTTP-only fallback still works via polling `/config/user/info` but loses all three plus the kernel version.
- The kernel logged in via IMConsole.exe; with only IMService.exe running, both phones read state 0 and the board list comes back empty.
- Account credentials held only by the operator, entered in the vendor console, never by the controller.
- Nothing else: no plug-in, Shortcut, board or phone accessibility setting, this watches kernel-side account state only.

WATCH OUT

- `/config/user/info` was never exercised in the 10 September probe, which hit a wrong guessed path, `/user/get`, and 404'd. Probe the real path once before trusting the numbers.
- There is no Phonefarm equivalent for any of this, the sibling doc has no user, login or licence concept at all, this idea is XP-only by construction.
- The headroom formula must branch on user_state; `min(total_license, local_total_license)` is wrong and would report negative headroom on a healthy trial rig, per the vendor's own worked example.
- overdue_time is not a reliable gate by itself, in the vendor's own example it sits roughly 19 months in the past while pass is still true. Report it next to pass and user_state, not as a standalone verdict.
- dev_num is documented with two contradictory descriptions, "number of users" and "verification passed". Count phones from `/device/get` instead of trusting it.

- im_log carries data.code 0 even for the refusal message, detection is string-matching Chinese text. Surface any unmatched im_log verbatim rather than swallowing it, other over-licence wordings are undocumented.
- Sub-account switching is documented in one paragraph with no statement of how licence pools or online devices behave across the switch. Test on an idle rig before using it as remediation.
- Kernel version is only ever available from the im_connect callback, there is no HTTP version endpoint on XP.
- This watchdog explains only why a phone will not come ONLINE. It says nothing about the USB control channel, a licence-refused phone can still be board-state 2 and controllable.
- The 30-second push is not a guaranteed heartbeat by the doc's own wording, "when logging in and at other times". Give the watchdog its own staleness timer, alert if neither user_info nor im_connect arrives for 90-plus seconds, rather than assuming silence means nothing to report.

SCALE

Effectively free and independent of rack size. The kernel already pushes a small JSON message every 30 seconds on an existing socket whether the rig has 2 phones or 100, and `/config/user/info` is a single parameterless call used only at startup and after a reconnect. No screenshot traffic, no board traffic, nothing that can contend with the measured frame pipeline or input traffic. Evaluate on each push and de-duplicate so a persistent condition alerts once per threshold crossing, 14, 7 and 1 days, headroom at or below 3, not every 30 seconds.

TRAP **Do not take the minimum of the two ceilings**

The obvious headroom formula, `min(total_license, local_total_license)`, is wrong. The vendor's own worked example (user_state 2, total_license 0, local_total_license 100, test_num 100) would report negative headroom on a rack that is actually fine. Branch on user_state first: total_license only applies to a formal account (user_state 1), everything else is a test authorisation and reads against local_total_license.

iOS-update tripwire that queues recalibration and re-verification

PARTIAL WITH CAVEATS

The pointer-calibration library is keyed by phone model plus iOS version, so an overnight OTA can silently invalidate a phone's calibration while every other signal stays green, it mirrors, it moves the pointer, and it taps 30 px off. Every device record and the dev_connect/dev_change callbacks carry a version field, so a controller that stores the last-seen version per phone gets a free tripwire, quarantine on change, corner-probe and tap-test to confirm, recollect once per model and iOS version if needed, then fan the new calibration out to every same-model phone in one call.

HOW

1. `get_device_list` XP `GET /api/device/get (version field)` — store a per-device iOS version baseline in the controller, the kernel persists nothing beyond name and gid
2. callback XP `ws://127.0.0.1:9911/api – dev_connect, dev_change` — tripwire, compare data.list[0].version against the stored value on every callback
3. `set_group` / `set_dev` XP `GET /api/device/group/set?id=0&name=Quarantine then GET /api/device/set?id=<deviceid>&gid=<gid>` — quarantine the changed phone so the scheduler skips it, gid is inert metadata, the kernel does not enforce it
4. `get_devicemodel_list` XP `GET /api/config/devicemodel/get` — ask whether a calibration already exists for this model plus the new iOS version, response shape undocumented

5. `mouse_reset_pos` + `mouse_move` + `get_device_screenshot` XP `GET /api/mouse/reset?id=... ; GET /api/mouse/move?id=&x=&y= ; GET /api/pic/screenshot?id=&binary=true` — corner probe: reset, move to three points, screenshot each, mean absolute pixel diff in a box around the commanded point
6. `send_key` + `click` XP `GET /api/key/sendkey?id=&fn_key=WIN+h (twice) ; GET /api/mouse/click?id=&x=&y=` — functional tap test on a known home-screen icon, whole-frame mean diff
7. `mouse_collection_open/cfg/close` XP `GET /api/device/collection/mouse?id=&cmd=1 (0 query, 2 stop)` — if the probe fails, collect once on one phone of that model, operator opens the collection page from a console QR
8. `save_dev_location` XP `GET /api/device/collection/mouse/save?id=&describe=XR-18.7.10-rack1` — publish the collected parameters to the shared library, XP has no delete endpoint for this
9. `set_dev` location_crc XP `GET /api/device/get?id=<collected id> then GET /api/device/set?id=<id1,id2,...>&location_crc=<crc>` — read the resulting CRC back and fan it out to every remaining same-model phone in one multi-id call
10. `set_dev` XP `GET /api/device/set?id=<ids>&gid=<fleet gid>` — re-run the probe and tap test on every phone that received the new CRC, release on pass, leave quarantined on fail
11. `shortcut#24` (external IP) XP `GET /api/shortcut/device/ip?id=&state=0` — optional: re-check the Shortcut round trip, since an OTA can re-prompt its permissions

NEEDS

- XP kernel, path style only; IMConsole.exe running and logged in, the console is also the only documented way to present the phone-side collection QR.
- A controller that persists per-device iOS version history and a quarantine flag, plus a Quarantine group and a scheduler that actually honours gid.
- Client-side BMP decode and a mean-absolute-pixel-difference routine, no OCR or image-search plug-in required, which is the point since `/pic/ocr` returns code 28 on this rig.
- Per-phone imgw/imgh and a known home-screen icon coordinate for the tap test, the two XRs have different home screens.
- An operator at the rack for the collection leg only: phone bound to its board, portrait, no other device collecting.

WATCH OUT

- The collection leg cannot be fully automated, the phone-side page is opened by hand from a console QR code. Budget an operator per model-and-iOS wave.
- `/config/devicemodel/get`'s response shape is undocumented, "too many, omitted" in the vendor doc, and was never exercised live. Probe it once, or skip that step and let the corner probe be the sole authority.
- Quarantine is advisory only, nothing in the kernel blocks input to a quarantined phone, and the API has no authentication, so another host can still drive it.
- An offset probe result has two indistinguishable causes, a stale calibration or an OTA resetting AssistiveTouch or Trackpad tracking settings, and neither is queryable. Check phone-side settings before recollecting, or a bad collection is baked into the shared library permanently, there is no delete endpoint on XP.
- The tripwire only fires when a phone appears on a callback or poll; keep a periodic reconciliation poll as backstop for a phone that updates while offline or across a dropped socket.
- iOS auto-update is entirely outside the API, this detects an update after the fact and cannot prevent or defer one, and a passcode lock after an OTA reboot is a hard stop with no typed remediation.

- Screenshot verification must race the real AirPlay source rate, not a 30 fps assumption, the slow XR delivers roughly 4.5 new frames a second, so a timeout tuned to the fast phone at roughly 29 fps will false-fail the slow one.
- The three-point corner probe catches uniform scale or translation error but is weak against rotation or shear, a small rotational term can partially cancel across two far corners and still pass while the UI centre is off. Add a fourth, off-diagonal point if rotation error matters for a given model.
- Version-change tripwires fire more than once per real OTA, iOS commonly reboots twice and drops Wi-Fi while optimising. Debounce on a version value held stable across several polls before quarantining, or the scheduler thrashes a phone in and out mid-update.
- A raw inequality compare on version strings quarantines on every point release, not just the bumps that risk calibration drift, treat a version change as far more frequent than a version change that actually needs recollection.
- The home-screen tap test assumes the phone is already past any post-OTA prompts, Apple ID re-auth, Wi-Fi rejoin, updated terms. WIN+h will not dismiss those, and a whole-frame diff cannot distinguish an opened app from a phone stuck on a dialog.

SCALE

Costs nothing until an update actually lands, the tripwire is one WebSocket or one no-id `/device/get` poll per interval. Per-phone verification is dominated by pointer travel, roughly 0.6 ms per pixel, so the three corner moves cost about 1.1 seconds, plus three 2 ms raw-BMP screenshots and a wait for a fresh frame on the slow phone, call it two to four seconds per phone end to end, and phones verify in parallel (measured 512 req/s across ten concurrent pollers, input traffic does not slow frame polling). Reset and the two Home presses take comma-separated id lists, so they cost one request for the whole wave regardless of fleet size. The expensive step is the collection itself, and it is once per model and iOS version, not per phone, one collection on this rig covers both XRs; at 19 boards, a fleet-wide OTA wave is one manual collection, minutes, needing an operator, plus roughly 20 to 80 seconds of parallel automated re-verification.

TRAP **Every published calibration is permanent**

XP has no endpoint to delete a record from the mouse-parameter library. A collection run against a phone whose accessibility settings had already drifted bakes bad parameters into a record that auto-matches every future same-model phone on that iOS version, forever. Verify AssistiveTouch tracking sensitivity and Trackpad and Mouse tracking speed on the phone before collecting, and make the describe string identify the rig and date so a bad record is at least findable.

Scheduling through partial failure

Degraded-mode operation: keep running blind jobs when video is down

PARTIAL WITH CAVEATS

Control and video are separate channels, and either can die alone. Classify every phone continuously into FULL, BLIND (control only, no picture), MIRROR-ONLY (picture, no usable board) or DEAD, then let the job queue keep running whatever work each class can still do instead of idling the slot. A BLIND phone still accepts taps, swipes and keys over USB, so fixed-coordinate, idempotent, keyboard-driven work can run blind and be verified

once mirroring returns; a MIRROR-ONLY phone can still be screenshotted for observation-only jobs. On this rack the weak XR delivers roughly 4.5 new frames a second against its twin's 29 on the same LAN, video is the flaky channel, USB is the reliable one.

HOW

1. `get_usb_list` XP `GET /api/config/usb/get` — read the control channel for every slot
2. `get_device_list` XP `GET /api/device/get (optional id=comma list)` — read the video channel for every slot
3. none (derived) XP `client-side logic over the two reads` — classify FULL / BLIND / MIRROR-ONLY / DEAD; control-ready and video-ready are independent, the API offers no combined health field
4. callbacks XP `ws://127.0.0.1:9911/api – dev_connect, dev_disconnect, dev_change, usb_change, error_push` — re-classify on events, not only on a timer
5. `mouse_reset_pos` XP `GET /api/mouse/reset?id=<comma list>` — clear the invisible stuck-click state before any blind work, mandatory since there are no frames to diff it against
6. `send_key` fn_key WIN+h XP `GET /api/key/sendkey?id=<comma list>&fn_key=WIN+h (twice)` — normalise to a known screen, blind; the keyboard path is the liveness probe that survives a dead pointer
7. `click / swipe / send_key / mouse_wheel` XP `/api/mouse/click, /api/mouse/swipe, /api/mouse/wheel, /api/key/sendkey, /api/key/down, /api/key/up, /api/key/upall` — run only blind-safe work: fixed-coordinate taps, ASCII typing, documented hotkeys, all take a comma id list
8. `send_key` fn_key XP `/api/key/sendkey fn_key=TAB+l, MuteSoundSwitch, VolumeDown, Play/Pause` — give BLIND phones a verification-free housekeeping queue, one request drives the whole BLIND set
9. `shortcut / ocr / find_image` XP `/api/shortcut/* and /api/pic/* – exclude on BLIND phones` — exclude every job that needs the phone's own LAN path or a frame
10. `auto_connect_screen(_all)` XP `GET /api/device/airplay/connect?id=<comma list> or /api/device/airplay/connect/all` — recover the video channel; no force flag on XP, disconnect a degraded session first
11. none XP `GET /api/imserver/regmdns` — re-broadcast AirPlay mDNS if the receiver has vanished from the phone's mirroring list
12. `get_device_screenshot` XP `GET /api/pic/screenshot?id=<single id>&binary=true` — verify the blind work once the picture is back, single id only, compare by mean absolute pixel diff
13. `restart_usb` XP `GET /api/device/usb/restart?id=720A|C241 (vid|pid, comma list)` — recover the control channel for MIRROR-ONLY slots, video is untouched
14. `set_usb_autoairplay` XP `none per-device; nearest is the global auto_connect in /api/config/imserver/set` — no per-device auto-cast switch exists on XP

NEEDS

- IMConsole.exe running and logged in; with only IMService.exe up, both channels read as empty or state 0.
- A job queue where every job declares which channel it needs, control, video, or the phone's own LAN path, there is nothing to schedule against without that tag.
- Per-phone board binding present and location_crc non-empty, otherwise mouse calls return code 2, 3 or 11.
- The five iOS accessibility settings on every phone, none queryable over the API: AssistiveTouch on with tracking sensitivity max, Full Keyboard Access on, Trackpad and Mouse tracking speed max, Auto-Lock Never, portrait lock with StandBy off.
- A WebSocket client, HTTP polling alone cannot see `airplay_connect_log` and is blind to why a re-cast failed.

- Each phone's imgw/imgh cached from its last good `/device/get` record, so a BLIND phone's coordinate space is known without a fresh frame.

WATCH OUT

- The blind leg is genuinely blind, no observation channel exists at all on a BLIND phone. Screenshot, OCR and find-image all need the mirror; Shortcut fallbacks need the phone's own Wi-Fi, which is the thing that is down. The only feedback is data.code plus async error_push, and neither proves the UI reacted, only that the board accepted the command.
- The stuck-click state is undetectable blind, `/mouse/reset` is the only known cure and is confirmed only to clear that specific state, not to correct a drifted pointer, which is still unverified. A drifted pointer taps the wrong place on every call while still returning code 0.
- Re-casting is not proven on this rig, the OCR plug-in it may lean on reports code 28, not connected, here. Plan for manual re-cast from the phone's own Control Center until that path is probed.
- Auto-cast and a blind job fight over the same board and the same screen, serialise "auto-cast retry pending" and "blind job pending" as mutually exclusive locks per phone, not just as a stated risk, or the retry taps through whatever the blind job has on screen.
- There is no per-device auto-cast flag on XP, only a global switch, you cannot leave auto-cast on for just the flaky phone.
- There is no server-push frame stream on XP; observation-only work on MIRROR-ONLY phones is one polling thread per phone, since screenshot takes a single id.
- Board state semantics are measured, not documented, a kernel update that changes the encoding fails the classifier silently.
- A phone that reboots or locks comes back on the lock screen with no documented way to type a passcode, a BLIND phone that locks is effectively out of the rack until touched.
- A phone BLIND from a real hardware fault looks identical over the API, in the seconds right after the state changes, to one an operator just unplugged for a swap, both read board-state not-2, and a stale cached control-ready flag can send taps into nothing.
- The five iOS accessibility prerequisites are unqueryable, so a phone can pass every classifier check and still never confirm a blind tap landed even after mirroring returns, because the tap never registered on the iOS side at all, indistinguishable from a slow-mirroring phone until a human watches a replay.
- Cap the verification backlog, a phone stuck BLIND indefinitely needs an explicit limit, N unverified actions or T minutes, after which it is pulled from the blind queue and flagged for a manual check rather than trusting a reconciliation that may never come.
- Filter the reset target list by current board state first, sending `/mouse/reset` to a board with no phone attached is harmless but wastes a call across the whole id list every cycle.

SCALE

Classification is nearly free, one `/device/get` and one `/config/usb/get` per sweep, with WebSocket callbacks triggering off-cycle re-polls. Blind execution actually scales better than sighted execution: every input endpoint takes a comma-separated id list and the boards walk in parallel, so a hotkey sweep across the whole BLIND set is one request, not many, at the same latency as driving a single phone. Verification is the bottleneck and does not batch, screenshot is single-id, so budget one poller per phone, measured 83-85 req/s per phone independently, about 1.6 MB per raw BMP poll. The real ceiling is the per-phone AirPlay source rate, 29 fps on the strong XR versus 4.5 fps on the weak one, so a verification step on the weak phone has to tolerate roughly 220 ms per new frame rather than a 30 fps assumption. Command issuance batches across the whole BLIND set in one call and does not compete with frame polling, measured independent, so recovered-phone

verification can run concurrently with ongoing blind work on others. For a job with no branching, expect on the order of 2,000 to 4,000 fixed-coordinate blind actions per hour per phone; the moment a job needs to branch on screen state, blind execution stalls until video returns and the real ceiling becomes recast latency, not the API.

TRAP **Blind means idempotent and reversible, or it does not run**

Blind input into a phone whose screen has changed underneath you, a system dialog, an app-update prompt, a session that logged itself out, types into whatever field happens to hold focus. That is how text meant for one field ends up in a search box or a composed message. Restrict the blind queue to steps with no irreversible tail: never a send, submit, publish, post, confirm, purchase or delete while blind, and never any credential or personal data at all, that belongs in a password manager the operator drives, not a scripted keystroke. Treat any blind job as provisional until a frame confirms it.

Keeping the rack connected

Cast-on-connect plus periodic re-cast sweeper

PARTIAL WITH CAVEATS

Keeps a rack mirroring without a human, by combining a global kernel flag with a timed sweep. The `auto_connect` flag (with `connect_failed_retry`) tells the kernel to re-cast a phone on its own; a background sweeper calls the connect-all endpoint for anything still offline. AirPlay sessions drop on Wi-Fi hiccups, after any phone restart, and whenever a per-device cast profile changes, since that change disconnects by design. This recovers the video channel only: the USB control channel is independent of it, so it fixes "controllable but blind," never the separate stuck-click fault that makes a phone "visible but deaf."

HOW

1. `get_device_list` XP `GET /api/device/get` — enumerate what is actually offline (state 0) before acting; state says nothing about the USB control channel, so cross-check that separately.
2. `get_usb_autoairplay` (no per-device XP equivalent) XP `GET /api/config/imserver/get` — read the current global `auto_connect` / `connect_failed_retry` policy.
3. `set_usb_autoairplay` XP `GET /api/config/imserver/set?auto_connect=true&connect_failed_retry=0` — turn on cast-on-connect with unlimited retry; applies live, no kernel restart, no lost sessions.
4. `WebSocket ws://host:9911/clinet` XP `ws://127.0.0.1:9911/api` — open before casting; it is the only channel carrying `airplay_connect_log`, `dev_connect` and `dev_disconnect`.
5. `auto_connect_screen_all` XP `GET /api/device/airplay/connect/all` — sweeper: re-cast every offline device on a timer (roughly every 5 minutes); takes no id filter and cannot be scoped.
6. `auto_connect_screen` XP `GET /api/device/airplay/connect?id=<id1>,<id2>` — targeted re-cast of named phones, e.g. one rack row, without touching phones mid-task.
7. `auto_connect_screen force:true` XP `GET /api/device/airplay/disconnect?id=<ids> then /device/airplay/connect?id=<ids>` — XP has no force flag, so a degraded-but-connected session must be dropped before it can be recast.
8. (no Phonefarm equivalent) XP `WS callback airplay_connect_log` — trace every attempt's numeric code against the XP error table; never branch on the Chinese message text.
9. `callback dev_connect` XP `then GET /api/device/get?id=<id>` — confirm the cast really came up by comparing the new air_pid/air_handle to the pre-cast snapshot, not just checking state != 0.

10. `get_device_screenshot` XP `GET /api/pic/screenshot?id=<id>&binary=true` — state-free proof the picture is live: body starts "BM" and is exactly 1,620,054 bytes at this rig's geometry.
11. (none) XP `GET /api/imserver/regmdns` — re-advertise the receiver if every phone loses it at once, without the full-rack outage of `/imserver/restart`.
12. `mouse_reset_pos` XP `GET /api/mouse/reset?id=<ids>` — reset the pointer on the whole re-cast set before any tap-critical sequence; a re-cast does nothing for the separate stuck-click fault.

NEEDS

- XP kernel on 9911 with path-style calls; Phonefarm `fun` names return HTTP 500 here.
- IMConsole.exe running and logged in — without it both channels read as empty or offline even with the kernel service up.
- Each phone bound to its board (vid+pid together) with a non-empty `location_crc`, and board state 2 in `/config/usb/get` for every phone expected to auto-cast.
- Phone-side settings invisible to the API: AssistiveTouch on with tracking sensitivity maxed, Trackpad & Mouse speed maxed, Full Keyboard Access on, Auto-Lock Never, portrait lock, and the phone already unlocked on its home screen with no passcode or Face ID prompt pending.
- Control Centre arranged with two rows of icons at the bottom — the vendor states this affects automatic screencasting.
- Phone on the same LAN segment, the receiver name visible in the phone's Screen Mirroring list, AirplayMirroring.exe listening on port 17000.

WATCH OUT

- Every call in this chain is documented native XP with no plug-in gate in the docs — but the Phonefarm sibling's equivalent needs a point-capture step that explicitly requires the OCR and image-search plug-ins, and the OCR plug-in reports code 28 on this rig. If XP shares that dependency, auto-cast fails here until the plug-ins are installed; that is unverified, which is why this entry is graded partial rather than native. Test on one phone before enabling it rack-wide.
- `auto_connect` is a global flag with no per-device exemption — you cannot arm auto-cast for one row and leave a flaky phone out of it. The only per-phone lever is leaving the flag off and driving targeted connects yourself.
- The connect-all endpoint cannot be scoped; use the comma-list connect endpoint for anything less than the whole rack.
- No force flag exists: an already-connected but degraded session must be disconnected first, or the connect call returns code 22, "Equipment online."
- A re-cast changes air_pid/air_handle and can change imgw/imgh — re-derive any cached tap coordinates per device afterwards, and reset the pointer before the next tap-critical sequence.
- This fixes the video channel only. A phone that fails to re-cast is still fully controllable and simply blind; a phone that re-casts cleanly may still be in the stuck-click state. Report the two channels separately, never as one health light.
- Validated only on two calibration-identical iPhone XRs; a fleet with mixed models or iOS versions is exactly the case most likely to break the internal cast-swipe coordinates, and a phone lacking a matching calibration profile can return code 0 while having tapped the wrong control — a live screenshot proves a picture, not that the right control was pressed.
- An AirPlay receiver can optionally require an on-screen pairing PIN depending on the console's security setting; nothing in this API surface exposes or controls it, and no endpoint can enter one. If it is on,

unattended re-cast attempts stall silently and unlimited retry loops forever — confirm it is off before relying on this unattended.

SCALE

The connect-all and comma-list connect calls are O(1) regardless of fleet size; the cost is on the phone, not the wire. The board drives each phone's own Control Centre through USB at a measured ~0.6 ms/px, so a swipe-and-tap sequence costs low hundreds of milliseconds of pointer travel per phone before UI settling, and every listed phone is driven concurrently by one request. Run the sweeper as a background timer, not an inline retry inside a task, and cap `connect_failed_retry` rather than leaving it infinite, or a phone stuck at a lock screen gets tapped forever. Verification does not contend with any of this: `/device/get` is one call for the whole rack, and per-phone screenshot polling measured no interference with input traffic in either direction. This rig has only 2 of 19 boards actually mirroring, so batch-size and recast-reliability numbers beyond that are a design assumption, not a measurement.

Self-healing input and hardware

Stuck-click self-healing runbook with escalation ladder

PARTIAL WITH CAVEATS

Automated recovery for the measured fault in which a phone silently ignores `/mouse/click` while `/mouse/move` and hardware keys keep working. A prevent-detect-escalate loop climbs only as far as needed: pointer reset (the measured cure) first, then a two-channel triage read, then board reset, then phone reboot, then kernel restart as a last resort. Detection is state-independent: normalise with two WIN+h presses, fire one click at a per-phone known target, and threshold the mean absolute pixel difference of a raw BMP frame — measured 0.09 stuck versus 141.96 healthy against a 0.02-0.03 idle floor, never MD5. This is the one measured fault on this rig that returns `data.code 0` for a click that did nothing, so no status field catches it, and every rung except the screenshot takes a comma id list, so the preventive reset and each escalation is one request for the whole rack.

HOW

1. `mouse_reset_pos` XP `GET /api/mouse/reset?id=<idA>,<idB>,...` — Rung 0, preventive: reset the whole target set before any tap-critical sequence; cheap insurance that stays cleared across repeated clicks.
2. `send_key (fn_key)` XP `GET /api/key/sendkey?id=<ids>&fn_key=WIN+h (twice)` — Detect A, normalise: leave then return to a known home screen; the Home hotkey still works even while the pointer is stuck, which is exactly why a keyboard-only probe would miss the fault.
3. `get_device_screenshot` XP `GET /api/pic/screenshot?id=<id>&binary=true` — Detect B: baseline raw BMP frame, one request per phone (2.0 ms p50, 1,620,054 bytes).
4. `click` XP `GET /api/mouse/click?id=<id>&x=442&y=245` then screenshot — Detect C: one tap at a per-phone home-screen target (coordinates are per-phone, not portable across different home screens), then a second frame.
5. (client-side) XP `none – computed locally` — Detect D: mean absolute pixel diff between the two frames — idle 0.02-0.03, stuck click 0.09, healthy click 142-163; threshold above 20 is alive, below 1 is stuck. Never MD5.
6. `mouse_up + key_release_all` XP `GET /api/mouse/up?id=<ids> and /api/key/upall?id=<ids>` — Rung 1, cheap and expected not to fix it: release any held button or key, in case an aborted drag left one down, a different fault this call does cure.
7. `mouse_reset_pos` XP `GET /api/mouse/reset?id=<ids>` — Rung 2, the measured cure: reset, then re-run the detect probe; most incidents terminate here.

8. `get_usb_list + get_device_list` XP `GET /api/config/usb/get and /api/device/get?id=<id>` — Rung 2.5, triage before escalating: read control-channel state (board 2) and video-channel state (device non-zero plus air_pid) separately, at no cost, before rebooting a phone whose board simply fell off.
9. `restart_usb` XP `GET /api/device/usb/restart?id=720A|C241` — Rung 3: reset the control board, addressed by VID|PID rather than the phone id, the single biggest addressing difference from the Phonefarm doc.
10. `callback usb_change` XP `ws://127.0.0.1:9911/api` — Rung 3 confirmation: watch for code 20 then 19 then 12 for that board, falling back to polling `/config/usb/get` since the ordering is inferred, not documented.
11. `mouse_reset_pos` XP `GET /api/mouse/reset?id=<id>` — Rung 3 aftercare: reset again now that the board has re-enumerated with a fresh, unsynchronised pointer state.
12. `restart_device / shortcut switch device / send_key` XP `GET /api/device/restart?id=<ids> OR /api/shortcut/switch/device?id=<ids>&state=1 OR /api/key/sendkey?id=<ids>&fn_key=CTRL+ALT+SHIFT+WIN+r` — Rung 4: reboot the phone; the driven restart route is expected to fail here since its Phonefarm sibling needs the OCR plug-in, which reads code 28 on this rig.
13. `callback dev_disconnect / auto_connect_screen` XP `restart_log then dev_disconnect, then GET /api/device/airplay/connect?id=<id>` — Rung 4 aftercare: confirm via the callback plus a new air_pid, never via a screenshot diff since the mirror is gone during reboot; a passcode-locked phone stops here and needs a human.
14. `restart` XP `GET /api/imserver/restart` — Rung 5, maintenance window only: restarts the kernel, dropping every session and socket on the rack; try the far cheaper `/imserver/regmdns` first if the symptom is just a vanished receiver.
15. (no Phonefarm equivalent) XP `ws callback error_push` — continuous signal: the only asynchronous proof an input call failed at the board; it will not fire for this fault's `code 0`, but it catches neighbouring hardware faults.
16. (local) XP `none — local store keyed on deviceid + uid` — bookkeeping: log which rung cleared each incident; a phone repeatedly needing rung 3 or higher is a cable or board candidate, and the physical hub/port slot (from Windows PnP, not the API) is the only label a technician can act on.

NEEDS

- IMConsole.exe running and logged in — with only IMService.exe up, this rig showed both phones state 0 and an empty board list, leaving the runbook nothing to act on.
- Per phone: board bound and `/config/usb/get` state 2 for that board, plus a non-empty `location_crc`, or every mouse call returns "No mouse parameters."
- iOS-side settings that are invisible to the API and are the usual cause of "looks fine, ignores input": AssistiveTouch on with sensitivity maxed, Full Keyboard Access on (needed for both hotkey probes), Trackpad & Mouse speed maxed, Auto-Lock Never, portrait lock. The probe in this runbook is the only proof any of it is actually set.
- A per-phone detect target chosen from that device's own imgw/imgh, since the two XRs on this rig do not share a home screen.
- A WebSocket client for `usb_change`, `restart_log`, `airplay_connect_log` and `error_push`; rungs 0-3 work over plain HTTP, only the confirmations need the socket.
- Rung 3 only: board firmware and kernel floors per the Phonefarm doc, unverifiable here since `ver` / `uver` read as mojibake on this build. Rung 4's shortcut route needs the iMouse Shortcut installed and bound, not confirmed present on this rig.
- The API has no authentication at all — firewall 9911 before wiring this runbook to anything automatic.

WATCH OUT

- Rungs 0 through 3 and the whole detect loop are natively achievable and were confirmed answering on 9911 in the 10 September probe; rung 4's driven route is expected to fail here because its OCR-plug-in dependency reads code 28. Treat the hotkey route as the primary rung-4 attempt and the Shortcut route as unavailable until installed.
- The detect click opens whatever is under that coordinate. It must be followed by Home, must target something idempotent, and must never sit inside an open app where it could send, post, or delete. A long-press into icon-jiggle mode is a plausible lower-risk variant but is unverified here — measure before trusting it.
- `/mouse/reset` is confirmed only to clear the stuck-click state; whether it corrects a drifted (offset) pointer is unverified, so do not advertise it as a calibration fix. Its own round-trip and settle latency is also never measured — verify with a timed probe before wiring it into a high-frequency sequence.
- The usb_change sequence 20-19-12 after a board reset is inferred, not documented as an order; fall back to polling `/config/usb/get` for that pid returning to state 2. The board's `state` field itself is undocumented, a measurement from this rig, not the vendor's field table.
- The fault returns code 0, so no rung can be triggered by a status check alone — the probe is mandatory, and `error_push` does not catch this specific fault.
- Rung 5 drops every mirroring session and socket on the rack; gate it behind an explicit human maintenance window, never an automatic loop, and try `/imserver/regmdns` first when the symptom is a vanished receiver.
- Frame timing is per-phone — one XR delivers ~29 new frames/s under motion, its twin ~4.5 — so a detect loop assuming 30 fps will read the slow phone as stuck. Wait for a genuinely new frame (and on the slow phone, several of them, since one iOS UI transition needs 2-4 visually distinct frames to be judged stable) rather than a fixed sleep.
- The detect click is a real tap that opens whatever is under the coordinate, and repeated probing multiplies the chance it lands somewhere destructive if a phone has drifted off its expected state — the two-Home normalisation must run immediately before every probe, never be assumed still valid from a prior cycle.
- Everything here is XP path-style on 9911; Phonefarm function names return HTTP 500 and the WebSocket path is `/api`, not `/clinet`.
- The throughput reasoning has two different ceilings: screenshot latency describes HTTP capacity, not how soon a genuinely new frame exists — the real ceiling is the AirPlay source fps, 4.5-29 per phone, so a rack sweep's real wall time is bounded by the slowest phone's frame arrival, not by request count.

SCALE

Rungs 0, 1, 2, 4 and the WIN+h normalisation all accept a comma id list, so recovering 50 phones is one request each, not 50, and the board walks every listed phone in parallel. `/device/usb/restart` takes a comma list of VID|PID pairs too, so a bank of boards resets in one call. The only per-phone cost is the detect probe — screenshot is single-id, so two 1.62 MB frames per phone at ~2.0 ms each, plus the frame-arrival wait, which is the dominant and per-phone term (~35 ms on the fast XR, ~220 ms or more on the slow one). Phones are independent, measured at 83-85 req/s on two concurrent pollers and 512 req/s aggregate on ten, so probing fans out with one poller per phone and no scheduling. A whole-rack preventive sweep is two requests regardless of fleet size; a whole-rack verification sweep on 19 slots is on the order of one second of wall time. Given the detect click's own risk, run it once per job as a pre-flight gate rather than continuously; as a background watchdog, once per 30-60 s per phone stays well inside every measured ceiling.

USB flap watchdog: board reset driven by usb_change and a state-2 poll

NATIVE API

WITH CAVEATS

Holds one WebSocket and treats the `usb_change` callback as the control-channel event bus: code 19 board attached to PC, 20 detached, 12 phone attached to board, with a live board table joined to phone deviceids from `/device/get`. A board reset fires only where it can actually reach the board — a flap that returned but settled at state 1 with no code 12, or a board at state 2 whose control is dead per an `error_push` fault or a hardware-not-connected error code. A control-board flap takes a phone out of every job while the mirror still looks perfect, because video and control are independent channels; catching the transition from a callback is instant, discovering it from a click that did nothing is minutes to hours later and is easily misdiagnosed as a mirroring fault.

HOW

1. `WebSocket ws://host:9911/clinet` XP `ws://127.0.0.1:9911/api` — open and hold one socket; every attach/detach event arrives here, no polling loop needed for the events themselves.
2. `get_usb_list + get_device_list` XP `/config/usb/get and /device/get` — build the board table once at startup and join uid to deviceid; neither call takes an id filter for boards, so re-read the whole list.
3. `callback usb_change` XP `usb_change (WS callback)` — on every event, journal `data.code` (19/20/12) and `data.list` per uid with a timestamp; that log is the flap counter.
4. `restart_usb` XP `GET /api/device/usb/restart?id=720A|<pid>` — Trigger A: code 20 then 19 for the same uid, settling at state 1 with no code 12 — the board came back to the PC but not to the phone.
5. (none — no Phonefarm error callback) XP `error_push (WS) then /device/usb/restart` — Trigger B: an `error_push` naming `/mouse/down` or `/mouse/click` as failed, or a mouse call returning hardware-not-connected/not-working, while the board still shows state 2.
6. `callback usb_change + get_usb_list` XP `usb_change (WS) + /config/usb/get` — confirm recovery: expect 20 then 19 then 12, and re-poll until the pid is back at state 2; treat silence as failure on a bounded timeout, not success.
7. `mouse_reset_pos` XP `GET /api/mouse/reset?id=<deviceid>` — preventive after any control-path disturbance: the measured cure for the silent stuck-click state, sent to the whole affected set in one call.
8. `get_device_screenshot + mouse_move + send_key` XP `/pic/screenshot?binary=true, /mouse/move, /key/sendkey` — prove control is really back: move, then a mean-pixel-diff of a ~40 px box around the commanded point (landed pointer 1.4-9.8 vs an idle 0.02 floor); never MD5.
9. (none — Windows PnP) XP `none` — if the board settles at state 1 after a reset, stop retrying and flag the slot for a hands-on reseat labelled by hub and port, since the API exposes no topology.
10. `restart_usb` XP `/api/device/usb/restart?id=720A|C241,720A|C214` — count flaps per uid per rolling day; three or more marks a swap candidate, and a whole hub can be reset in one comma-separated call if an operator confirms it.

NEEDS

- IMConsole.exe running and logged in — with only IMService.exe up this rig showed both phones at state 0 and an empty board list, which would otherwise read as 19 simultaneous flaps.
- Each phone bound to its board so a usb_change uid resolves to a deviceid; unbound boards can still be reset by vid|pid but yield nothing to verify against.
- Firmware and kernel floors that cannot be checked from the API on this rig, since `ver` / `uver` read as mojibake.

- A persistent event journal keyed by board uid, so flap counts survive a watchdog restart, and a hub/port map from Windows PnP to turn an opaque pid into a rack slot.
- Per-phone iOS accessibility settings for the verification probe to mean anything — none is queryable, so a phone can be perfectly cabled and still ignore input.

WATCH OUT

- Resetting on a bare code 20 with no code 19 targets a board that has left the USB bus entirely — there is no addressee for a vid|pid reset. That is a physical-layer failure (board, cable, hub, or a human unplug) and belongs in a hands-on queue, not an automated reset.
- There is no callback for the phone detaching from an otherwise-present board; a state 2 to state 1 transition, the most common rack fault, may raise no event at all. Cover it with a slow poll of `/config/usb/get` diffed against the board table, and do not claim the design is purely callback-driven.
- The `state` field the whole design keys on is undocumented — the vendor's field table lists only vid/pid/uid/ver. The 1-versus-2 mapping is a measurement from this rig, corroborated by usb_change code 12, and should be re-checked after any kernel update.
- `/device/usb/restart` has never been exercised on this rig, so the 20-19-12 sequence, the return to state 2, and the time it takes are all inference, not measurement. Validate on one expendable slot off-hours before letting the watchdog act unattended. A reset is also electrically equivalent to unplugging and replugging the cable, which can trigger iOS's "Trust This Computer?" prompt on that reconnection; nothing distinguishes that state from a plain failure, and the automation cannot reliably dismiss it.
- Whether a code 20 callback carries the departed board's identity at all is unverified — the doc's only example is code 19. If it arrives empty, diff `/config/usb/get` instead of relying on the callback payload.
- The firmware precondition is unverifiable here; a board too old for the reset will simply return an uncertain error code or produce no event, so treat silence as failure and stop retrying that board.
- A reset is blind to whether a job is in flight — it can drop a held button mid-drag with no /mouse/up ever reaching the phone. Give the watchdog the same per-phone lock the job runner takes, or defer until the phone is idle.
- Video is untouched by a board reset — air_pid stays the same and frames keep arriving — which is exactly why the fault is invisible without this watchdog, and why "the mirror looks fine" must never be read as evidence the reset worked.
- Rate-limit resets (something like one per board per 10 minutes, a handful per day) or an automatic loop can mask a dying board, quietly bouncing a failing slot for weeks instead of escalating it. Hub-wide comma-list resets are tempting but blunt — every phone on that hub loses control at once, so keep them operator-confirmed only.
- A partial-fleet firmware mismatch, undetectable given the mojibake version fields, could silently no-op the restart call on older boards with no distinguishable error code, inflating the flap counter for boards that are simply outdated rather than failing.

SCALE

Event-driven and effectively free: one WebSocket carries callbacks for all 19 boards, with no per-board subscription, and the supplementary poll is a single small JSON list for the whole rack that can run every 2-5 seconds without noticeable cost. Recovery itself is a comma-list call, so N boards is one request. Measurements show input and control traffic do not interfere with frame polling in either direction, so the watchdog's own probes will not disturb a running wall. The only per-phone cost is the verification probe — one 1.62 MB raw BMP plus a short move — kept to the affected phone rather than swept across the rack. The expensive part is

wall clock, not CPU: budget seconds of control outage per reset plus whatever settle time the attach sequence needs, and at the recommended one-reset-per-board-per-10-minutes rate limit this stays well under ten driven resets per hour per board even during a bad patch.

Scheduled fleet maintenance

Rolling reboot maintenance window with re-cast and input self-test

NEEDS SHORTCUT

WITH CAVEATS

A scheduled overnight job that cycles a phone farm a batch at a time: disconnect the mirror so the drop is expected, reboot the batch in one call, wait for the boards to re-enumerate, re-cast, then prove each phone is genuinely usable again with a two-channel self-test before handing it back to the scheduler. The reboot is easy; proving the phone came back genuinely controllable is the real payload, and it is the difference between a rack that reports healthy and a rack that is healthy, since no health check that only watches "is it mirroring" will ever catch a phone that mirrors fine but silently ignores taps.

HOW

1. `get_device_list (+ get_group_list)` XP `/device/get, /device/group/get-dev` — select the batch and record a "before" snapshot of state, ip, vid/pid/uid, imgw/imgh and air_pid for each device.
2. `get_usb_list` XP `/config/usb/get` — snapshot the control channel before touching anything, so "never came back" can be told apart from "came back but did not re-cast" later.
3. `WebSocket ws://host:9911/clinet` XP `ws://<host>:9911/api` — open before the batch starts and hold it for the window; it is the only way to receive dev_disconnect, usb_change, dev_connect, restart_log, airplay_connect_log and error_push.
4. `discon_airplay` XP `GET /api/device/airplay/disconnect?id=<ids>` — announce the drop so it reads as maintenance, not a fault; wait for dev_disconnect before rebooting.
5. (no Phonefarm equivalent) XP `GET /api/shortcut/switch/device?id=<ids>&state=1` — reboot the batch via the iMouse Shortcut, needing no OCR plug-in and able to reboot a phone whose video channel is already dead.
6. `send_key` XP `GET /api/key/sendkey?id=<ids>&fn_key=CTRL+ALT+SHIFT+WIN+r` — fallback A if the Shortcut is not installed; needs Full Keyboard Access on but no Shortcut.
7. `restart_device` XP `GET /api/device/restart?id=<ids>` — fallback B, do not lead with this; it drives the phone's own Settings UI and its Phonefarm sibling needed the OCR plug-in first.
8. `callback usb_change` XP `usb_change (WS) / /config/usb/get` — wait for the phone using the control channel as the readiness signal (code 12, or polling for state 2), not a network ping that can answer before iOS accepts HID input.
9. `auto_connect_screen / auto_connect_screen_all` XP `/device/airplay/connect, /device/airplay/connect/all` — re-cast the batch, watching airplay_connect_log for code 0 then dev_connect; this is the weak leg on this rig, since auto-cast drives Control Centre by sight and the plug-in it may need reads code 28 here.
10. `set_usb_autoairplay / get_usb_autoairplay` XP `/config/imserver/set, /config/imserver/get` — alternative to a driven re-cast: set the global auto_connect flag once and let the kernel re-cast on its own; applies live with no restart.

11. `mouse_reset_pos` XP `/api/mouse/reset?id=<ids>` — on each dev_connect, clear the input channel first; mandatory after a fresh board enumeration, not optional.
12. `mouse_move + get_device_screenshot` XP `/mouse/move, /pic/screenshot` — self-test, pointer leg: move, then a mean-pixel-diff box around the target (landed 1.4-9.8 versus a 0.02 idle floor).
13. `send_key` XP `/key/sendkey?id=<ids>&fn_key=WIN+h (twice)` — self-test, keyboard leg and normalisation: a whole-frame diff above ~20 confirms hardware keys, but this alone proves nothing about taps.
14. `get_device_list + get_usb_list` XP `/device/get, /config/usb/get` — mark the phone healthy only if board state 2, a fresh air_pid, the pointer-box diff, and the WIN+h diff all pass; report "controllable but blind" and "visible but deaf" as distinct outcomes.

NEEDS

- IMConsole.exe running and logged in — without it the rack reports state 0 with an empty board list even though the kernel service is up, and the job cannot start it through the API.
- iMouse Shortcut installed, kernel-IP-edited, permission-granted, and hotkey-bound on every phone in scope — not confirmed present on this rig; the hotkey fallback is likelier to actually work since it needs no Shortcut.
- Each phone bound to its board with a non-empty `location_crc`, and a vendor-supplied cable only, since substituting a cable is a real failure mode.
- Phone-side accessibility settings invisible to the API, and phones without a passcode or a human available to unlock afterward — nothing in either API can type a passcode.
- A WebSocket client for the six callbacks this job depends on, and an external scheduler, since the kernel has no scheduling of its own.
- For the driven reboot and (probably) the re-cast: the OCR and image-search plug-ins, not installed on this rig.

WATCH OUT

- The re-cast leg is the unverified half of this job here: the API error table's "no coordinates" and "plug-in not connected" codes point at the same OCR/image-search dependency the Phonefarm sibling states outright, and this rig's OCR plug-in reads code 28. Until someone fires the connect call and sees code 0, assume a rebooted phone may need to be re-cast by hand — turning an unattended nightly job into one that files a work order.
- The reboot path itself is equally unverified: nothing in this rig's data confirms the iMouse Shortcut is actually installed on the two live XRs. Treat both the reboot and the recast as unproven in practice, not just the recast.
- A passcode-locked phone reboots to a locked screen and stays out of service until a human types the code; only run this on phones configured without one, or accept a morning unlock round.
- A data.code 30 "call timeout" on the reboot call is expected, not diagnostic — the phone is powering off and cannot answer in time. Do not retry on a 30, or you may reboot it again mid-boot; confirm by absence then return on the control channel instead.
- `auto_connect` is a global kernel flag, not a per-device switch — you cannot arm auto-cast for just the maintenance batch.
- Per-device air_* overrides disconnect the current cast and apply only on the next connect, so any override applied mid-window will not show up until the phone re-casts; order a config sweep before a cast sweep, not after.
- Screenshot, image-find and OCR calls are single-id only, so the self-test cannot be batched the way mouse and key calls can; run one poller per phone concurrently, which measurement shows has no shared

bottleneck.

- Several of the vendor doc's own curl examples in the sections this job uses point at the wrong endpoint — trust the parameter tables, not the examples.
- The API has no authentication and listens on all interfaces, so any host on the LAN can reboot the whole rack with one request; firewall 9911 before wiring anything to a scheduler.
- This rig has only 2 phones mirroring out of 19 boards, so nothing about batch size or recast reliability at scale extrapolates from measurement here — it is a design assumption.
- A reboot power-cycles USB enumeration too — expect the board to dip to state 1 before returning to state 2, and do not treat that intermediate dip as a fault.
- The pointer-box threshold was measured at one specific coordinate and wallpaper; compute an idle floor per phone rather than hardcoding these numbers, since the two XRs already have different home screens.

SCALE

Batch operations are genuinely one round trip — the shortcut, disconnect, connect, reset and sendkey calls all take a comma id list, and the board walks every listed phone in parallel. The per-batch cost is dominated by physics, not the API: roughly 60-90 seconds for an iPhone to reboot, plus the re-cast, plus a few seconds of self-test. The self-test is the only non-batchable part, since screenshot is single-id, but phones are independent and their polling measured no interference with input traffic, so the whole batch's self-tests run concurrently in well under a second each. A batch of 7 at roughly 3 minutes extrapolates to ~45 minutes for 100 phones, but that arithmetic is untested here; size batches by how many phones the rack can afford to have offline at once, not by API speed. If recast succeeds automatically every time, that is on the order of 20-30 verified cycles/hour/phone; if it needs manual Control Centre intervention, as the OCR code 28 result suggests it might, unattended throughput is effectively zero and this becomes a human-paced task rather than a script.

Proving the fleet is actually alive

Pointer-nudge liveness heartbeat that proves input is honoured

NATIVE API WITH CAVEATS

A per-phone background probe that proves the control path is actually alive, using an observation channel no API field provides: move the pointer between two fixed points in a visually dead zone and look for the small grey pointer dot arriving, by taking the mean absolute pixel difference in a small box around the destination. Every status field this rack exposes is a lie of omission — a bound board and a live mirror session say nothing about whether iOS is honouring input, and the accessibility settings that decide it are not queryable at all. This rig demonstrated the gap directly: a phone silently stopped accepting every click while moves and hardware keys kept working, and it looked perfect in every API field throughout. A landed pointer measures 1.4-9.8 against an idle noise floor of 0.02, needs no OCR plug-in, no image-search plug-in, no Shortcut and no professional edition, and turns "this phone has been silently dead since Tuesday" into a one-minute alert.

HOW

1. `get_device_list` XP `GET /api/device/get (optionally ?id=a,b)` — build the candidate set: only phones with a non-zero state and live air_pid, reading imgw/imgh per phone so a follower with different geometry is excluded from a shared move.
2. `get_usb_list` XP `GET /api/config/usb/get` — confirm the control channel is physically present before blaming the pointer; state 2 means the board is enumerated and the phone is attached.

3. `get_device_screenshot (binary=true)` XP `GET /api/pic/screenshot?id=<one id>&binary=true[&rect=...]` — baseline capture of the probe box; works over plain HTTP on XP, measured 2.0 ms p50 for a full raw frame.
4. `mouse_move` XP `GET /api/mouse/move?id=<id1,id2,...>&x=<X>&y=<Y>` — nudge the pointer at least ~40 px between two fixed points in a dead zone; one request drives every listed phone in parallel, and a 2 px hop is too small to move the low-contrast dot off its own prior position.
5. `get_device_screenshot` XP `GET /api/pic/screenshot?id=<one id>&binary=true[&rect=...]` — wait for a genuinely new frame, then diff client-side; a landed pointer measures 1.4-9.8, idle noise is 0.02-0.03. Never MD5.
6. `mouse_reset_pos` XP `GET /api/mouse/reset?id=<id1,id2,...>` — Rung 1 remediation: the only call confirmed to clear the silent stuck-click state; it also re-parks the pointer, so re-baseline after it.
7. `send_key (fn_key WIN+h)` XP `GET /api/key/sendkey?id=<id>&fn_key=WIN+h` — on-demand escalation only, never on the routine timer: disambiguates a frozen mirror from a dead pointer path by producing a large whole-frame change.
8. `restart_usb` XP `GET /api/device/usb/restart?id=720A|<pid>` — Rung 2 remediation: reset the control board, addressed by VID|PID from /device/get or /config/usb/get, not by phone id.
9. (no Phonefarm equivalent) XP `ws://127.0.0.1:9911/api – callback error_push` — complementary passive signal at no polling cost: the only asynchronous proof an input call failed at the board, naming the failing function.
10. `discon_airplay / auto_connect_screen` XP `GET /api/device/airplay/disconnect?id=<ids> then /device/airplay/connect?id=<ids>` — mirror recovery, only once the video channel is the confirmed fault; XP has no force flag, so disconnect first.

NEEDS

- IMConsole.exe running and logged in — without it, both channel reads go empty regardless of hardware state.
- Per phone: mirroring up, board bound and cabled, and mouse parameters present, or every mouse call fails outright.
- Phone-side accessibility settings, none queryable over the API: AssistiveTouch on with tracking maxed, Trackpad & Mouse speed maxed, Full Keyboard Access on, Auto-Lock Never, portrait lock.
- A per-phone probe pair at least ~40 px apart in a visually stable dead zone, expressed in that phone's own imgw x imgh, and a per-phone landed-pointer threshold calibrated from that phone's own signal rather than assumed.
- A client-side BMP decoder and mean-absolute-difference routine with a reused frame buffer, since unreused 1.6 MB frames measured 488 MB/s of allocation and 496 gen-2 collections per 8 s, falling to 2.9 MB/s and 0-1 with reuse.
- A lease/mutex shared with the job queue so the nudge never fires mid-drag, and, optionally, one WebSocket for error_push, dev_disconnect and usb_change.

WATCH OUT

- The probe is necessary but not sufficient for taps. The measured stuck-click phone passed a move probe cleanly while every click produced only 0.09 mean diff against a 0.02 floor — a green heartbeat proves only that the board moves this phone's pointer, never that the phone honours taps. Label the metric `control_pointer_alive`, not "healthy," and pair it with a periodic real click-and-verify against a safe dead-zone target.
- A failed probe is ambiguous between a frozen mirror and a dead pointer path — both read as roughly a 0.02 box diff. Only the state-changing WIN+h follow-up separates them, so the routine timer cannot classify the

fault on its own, and that hotkey must stay an on-demand escalation, never a periodic call, since it leaves whatever app is open.

- The pointer dot is low-contrast and only a few pixels across; on a bright or busy wallpaper the diff can fall under 1. Calibrate the threshold per phone from its own landed-pointer signal, not a fleet-wide constant, since wallpaper brightness changes the dot's contrast.
- A "visually stable dead zone" and its coordinates are per-phone, not per-fleet — the two XRs already have different home screens, and any phone with different mirror geometry must be excluded from a shared comma-list move.
- Frame arrival, not API latency, dominates the probe: one XR delivers ~29 new frames/s under motion, its twin only ~4.5, so a fixed timeout tuned for the fast phone will falsely fail the slow one. Wait on an actually-changed frame with a per-phone deadline.
- No server-side comparison is available on this rig — the OCR plug-in reads code 28 and the image-search plug-in is presumed absent too, so the mean-pixel-diff must be implemented entirely in the caller.
- The nudge is visible on the mirror and perturbs the hover target in whatever app is open; it must be gated by the job-queue lease so it can never land inside a drag, and server-side rect cropping's latency benefit is unmeasured on this rig, so do not assume it beats the full-frame baseline without checking.
- The whole scheme presumes the vendor console is running — with only the kernel service up, both channels read as empty, and a fleet-wide "all dead" reading usually means the console crashed, not that every board failed. Board-reset remediation needs a kernel and firmware floor that is only readable on the phone, since the version fields are mojibake on this build.
- The API has no authentication; a monitor that can nudge every pointer in the rack is reachable by any host on the LAN, so firewall 9911 to trusted hosts.

SCALE

Cost per phone per probe: two screenshot calls at ~2.0 ms p50 each, plus a share of two move requests that carry the whole comma-separated id list — because the board walks all listed phones in parallel, the two ~40 px hops cost roughly 25 ms of board time for the entire fleet, not per phone. For 19 phones once a minute, that is 38 screenshot requests and 2 move requests per minute against a measured ceiling of 84 req/s per phone and 512 req/s aggregate across ten concurrent pollers. Bandwidth is the real constraint, not CPU: 38 full frames a minute is roughly 62 MB/min on loopback, which is why rect cropping and a reused per-poller buffer matter more than the request count. The wall-clock probe duration is dominated by frame arrival — ~100 ms on the fast phone versus ~700 ms on the slow one — and screenshots are inherently one request per phone by documented rule, so run pollers concurrently and stagger start times. At a conservative once-a-minute cadence this is roughly 120 screenshot requests/hour/phone with orders of magnitude of headroom below the measured ceiling; the fastest sustainable cadence bounded by the slowest phone is on the order of one cycle per second, though a production heartbeat should run far below that to leave headroom for real job traffic.

Mirroring bandwidth policy

Energy-mode policy by time of day, and the capacity it buys

PARTIAL WITH CAVEATS

Run the rack on two mirroring profiles: high fidelity while an operator is watching a wall of live tiles, low bandwidth overnight while only scripts drive the phones. On this XP kernel there is no single mode switch, only per-device knobs on `/device/airplay/set` (air_ratio, air_refresh, air_fps, air_img_fps, air_audio) plus a

global floor in `/config/imserver/set`. Mirroring bandwidth, not CPU or USB, is what decides how many phones one PC and one access point can carry, and the vendor's own figures give a rough capacity model, though the switch is cheap while the re-cast and re-verification that follow it are not.

HOW

1. `get_airplay_mode` XP `GET /api/device/get` — snapshot air_ratio/refresh/fps/img_fps/audio and imgw/imgh per phone before changing anything; XP has no /device/airplay/get, so /device/get is the only read-back.
2. `none` XP `GET /api/config/imserver/get` — read the global air_play_* baseline that per-device zero fields fall back to.
3. `none` XP `GET /api/config/user/info` — read total_license and dev_online_num, the other half of the capacity ceiling.
4. `set_airplay_mode airplay_mode=0` XP `GET /api/device/airplay/set?id=<ids>&air_ratio=720&air_fps=10&air_img_fps=5&air_audio=1` — drop the whole fleet to a low-bandwidth night profile in one request.
5. `set_airplay_mode airplay_mode=2` XP `GET /api/device/airplay/set?id=<ids>&air_ratio=0&air_fps=30&air_img_fps=30&air_refresh=60&air_audio=0` — restore day fidelity; 0 means "use the global setting," not a literal maximum.
6. `restart` XP `GET /api/config/imserver/set?air_play_ratio=720&air_play_fps=10 then GET /api/imserver/restart` — move the kernel-wide floor instead of per-device overrides; needs a full restart that drops every session.
7. `auto_connect_screen(_all)` XP `GET /api/device/airplay/connect?id=<ids> or /connect/all` — re-cast, because the profile change itself dropped the picture.
8. `dev_connect callback` XP `WS callbacks airplay_connect_log then dev_connect / dev_change` — watch the re-cast rather than sleeping on a fixed delay.
9. `get_device_list` XP `GET /api/device/get?id=<ids>` — re-read imgw/imgh; the tap coordinate space is not guaranteed stable across a profile change.
10. `mouse_reset_pos` XP `GET /api/mouse/reset?id=<ids>` — clear the stuck-click state a re-cast can leave behind.
11. `loop_device_screenshot` XP `poll GET /api/pic/screenshot?id=<id>&binary=true` — confirm the new profile is actually delivering the frame rate you expect.
12. `shortcut#15` XP `GET /api/shortcut/switch/bril?id=<ids>&state=0.1` — optional phone-side brightness lever; not set up or confirmed on this rig.

NEEDS

- XP kernel reachable at 9911 with path-style calls; Phonefarm function names such as set_airplay_mode return HTTP 500 here.
- IMConsole.exe running and logged in — with only IMService.exe up, /device/get and /config/usb/get both come back empty.
- A way to re-cast after every switch: /device/airplay/connect (unverified, needs the OCR plug-in path), the global auto_connect flag, or a manual Control Centre re-cast.
- Every phone already mirroring and bound before the transition starts; an offline phone is not "restored" by the policy.
- Vision assets re-tuned per profile, since colour tables and coordinates are resolution-specific.
- An external scheduler — there is no cron, timer or schedule primitive in either API.

WATCH OUT

- The three named Phonefarm modes (0/1/2) do not exist on XP; the night/day air_* combinations here are a home-grown mapping, not a vendor-defined mode.
- The 5/10/20 Mbps and ~100/50/25-phone figures come from a single Phonefarm-doc sentence about "one landscape game"; nothing on this two-phone rig can confirm or refute them — treat any capacity number as a hypothesis.
- Every /device/airplay/set call, even one that only touches air_audio, is documented to drop the cast — schedule the switch inside a maintenance window, never mid-job.
- The global route needs /imserver/restart, which drops every mirroring session and every WebSocket client fleet-wide at once.
- Automated re-cast is unverified on this rig: /device/airplay/connect drives Control Centre through the board, and the OCR plug-in this depends on reports code 28 here.
- Type mismatch trap: global air_play_audio is Boolean, per-device air_audio is an Integer (0/1/2); crossing them is a silent-looking parameter error (code 36).
- imgw/imgh is not guaranteed stable across a profile change; a stale cached coordinate taps the wrong place with no error — taps return code 0 either way.
- Lowering the profile does not fix a slow phone: two identical XRs on this rig measured ~29 and ~4.5 new frames per second on the same LAN, which is the phone's own Wi-Fi link, not the profile.
- There is no per-device auto-cast flag on XP; auto_connect is one global Boolean, so "these phones re-cast themselves, those don't" is not expressible.
- No scheduler exists in the API — the time-of-day half of this idea is entirely external tooling holding unauthenticated control of every phone on the subnet.

SCALE

The switch calls themselves are free at any fleet size: /device/airplay/set, /connect, /disconnect and /mouse/reset all take a comma-separated id list, so flipping 100 phones is one request per operation, and as a scheduled transition it is two calls a day, not a continuous action. Verification does not scale the same way: /pic/screenshot is single-id only, so proving the new profile is delivering frames costs one poller per phone — measured at 83-85 req/s per two concurrent pollers and 512 req/s aggregate across ten, with input and video traffic non-interfering, but that is a two-phone loopback measurement and says nothing about 100 phones over real Wi-Fi. The vendor-sourced 5/10/20 Mbps figures translate to roughly 100/50/25 phones per typical access point in theory; treat that as an unverified extrapolation pending a real multi-phone contention test. Budget ~1.6 MB per raw BMP poll with a reused buffer per poller, and expect the re-cast, not the profile call, to dominate the real transition cost.

Physical fleet management

Torch beacon and slot roll-call: finding a phone physically in the rack

NEEDS SHORTCUT

WITH CAVEATS

Turn an opaque device id into something a technician can pick up: light the target phone's camera LED with `/shortcut/switch/torch` so it is the only glowing unit in a wall of identical XRs, and run a periodic roll-call that lights each slot in turn to audit whether the kernel's name labels still match physical positions. The LED is a camera flash, not part of the AirPlay picture, so this beacon has no observation channel on this rig; it is paired

with a second, plug-in-free fallback beacon over `/key/sendkey` whose whole-screen repaint IS visible in the mirror and confirmable by pixel diff. Nineteen boards on three hubs with identical cables and identical phones means a MAC address or PID tells a technician nothing about which unit to unplug, and the API exposes no physical location at all.

HOW

1. `get_device_list + get_usb_list` XP `GET /api/device/get?id=<id> and GET /api/config/usb/get` — confirm video-ready (state, air_pid) and control-ready (board state 2) before beaconing.
2. `none` XP `none — Windows PnP, not the API` — build the physical hub/port slot map once; no XP endpoint returns hub or port.
3. `set_dev` XP `GET /api/device/set?id=<id>&name=H10-P02` — persist the slot label into the device record's name field so an alert can name it.
4. `shortcut#16` XP `GET /api/shortcut/switch/torch?id=<id>&state=1&outtime=3000` — light the target phone's camera LED (Phonefarm marks the equivalent professional-version only; the XP docs carry no such gate).
5. `none` XP `none — human in the loop` — technician walks to the one glowing slot; there is no API way to confirm the LED actually lit.
6. `shortcut#17` XP `GET /api/shortcut/switch/torch?id=<id>&state=0` — beacon off, always sent from a finally block; a comma list makes an "all torches off" sweep one call.
7. `send_key (fn_key)` XP `GET /api/key/sendkey?id=<id>&fn_key=ControlBar` — USB-only fallback beacon (also NoticeBar, WIN+h) that repaints the phone's own screen, visible in the mirror.
8. `get_device_screenshot` XP `GET /api/pic/screenshot?id=<id>&binary=true` — confirm the fallback beacon fired via a before/after mean-pixel-diff.
9. `get_group_list / get_device_list` XP `GET /api/device/group/get-dev?id=<gid>` — enumerate a rack section, then loop the beacon per device in your own PnP-derived slot order, not the API's own order.
10. `set_dev (vid+pid)` XP `GET /api/device/set?id=<id>&vid=720A&pid=<pid>` — rebind when the light comes up in the wrong slot; one call per phone.
11. `mouse_reset_pos` XP `GET /api/mouse/reset?id=<ids>` — clear the stuck-click state before any tap-critical follow-up on the located phone.

NEEDS

- iMouse iOS Shortcut installed and AssistiveTouch-bound on every phone to beacon via torch — unverified and unprobed on this rig.
- Both channels alive at once for the torch: the USB board bound and connected (fires the hotkey) and the phone reachable on the LAN (the Shortcut answers over Wi-Fi).
- Phone unlocked, Auto-Lock Never, and no pending iOS "Allow" prompt blocking the Shortcut.
- Fallback path needs only a bound, connected board plus Full Keyboard Access on — works with Wi-Fi down and no Shortcut installed.
- IMConsole.exe running and logged in.
- A hand-built slot map joining Windows PnP hub and port to the device record's name field, since nothing in the API supplies it.

WATCH OUT

- No observation channel for the torch: the LED is a camera flash, not carried in the AirPlay stream, so data.code 0 only proves the Shortcut answered, never that the light came on — any console UI should say "beacon requested," never "beacon on."

- A multi-id torch call returns one aggregate result code, so an audit needing per-phone truth means one call per id — 19x the calls for a full rack.
- The beacon fails on exactly the phones you most want to find, since it needs both the board and Wi-Fi paths a broken phone has likely lost; the fn_key fallback is the primary tool for a genuinely dead slot, not an afterthought.
- Default outtime is 15000 ms; 19 unresponsive slots at default is nearly five minutes of dead air — lower it to roughly 3000 ms.
- The realistic state of this rig today: the torch path has never been demonstrated to work at all; only the fn_key fallback is verified-workable.
- /device/group/get-dev order is the kernel's, not physical order — the roll-call must be sequenced from your own PnP-derived slot map.
- Rebinding via /device/set is one phone per call, and vid/pid must be sent together — a rack-wide rebind is N calls.
- Prefer a steady-on LED over a blink pattern; a strobing wall of phones is a real photosensitivity hazard for little added visibility.
- The torch does not need the OCR or image-search plug-ins, so this rig's code-28 blocker does not apply here — the dependency is the separately unverified on-phone Shortcut.

SCALE

Beacon-on and beacon-off are each one call for the whole rack, since /shortcut/switch/torch takes a comma id list, at the cost of a single aggregate result code with no per-phone breakdown. A sequential per-phone roll-call is two calls per slot plus dwell: at a 2 s dwell with outtime lowered to ~3000 ms, 19 slots is roughly 40 seconds plus call latency. The fn_key fallback beacon also fans out on a comma list and costs the same wall time for one phone or all of them, since it never touches /mouse/move or the ~0.6 ms/px pointer-walk cost. Verifying the fallback is single-id only, measured at 84 req/s and p50 2 ms per raw BMP poll with phones fully independent (two pollers 83+85 req/s, ten pollers 512 req/s aggregate); the weak XR's ~4.5 fps means a before/after diff there needs 220 ms or more for a genuinely new frame. The torch path itself has no measurable throughput, because code 0 has never been confirmed to correspond to a lit LED on this rig.

Storage housekeeping

Storage hygiene: scheduled album and file cleanup with size-growth tracking

NEEDS SHORTCUT

WITH CAVEATS

Sweep every phone on a schedule through the iMouse iOS Shortcut: enumerate the camera roll and the Files directory the rig writes into, optionally harvest anything worth keeping to the PC, then delete by name or wipe a whole album. Each listing returns name, ext, size and create_time, so the controller can sum a per-phone byte total each run and trend it. The XP API has no storage, free-space or battery endpoint of any kind, so this enumerated total is the only available proxy for "how full is this phone getting," converting an otherwise unobservable failure into a trended number with a threshold.

HOW

1. `shortcut#24` XP `GET /api/shortcut/device/ip?id=<id>&state=0` — cheapest proof the Shortcut path is alive before trusting any listing.

2. `shortcut#1` XP `GET /api/shortcut/album/get?id=<id>&album_name=&num=200&outtime=60000` — enumerate the camera roll; size comes back as a formatted string, not a number.
3. `shortcut#4` XP `GET /api/shortcut/file/get?id=<id>&path=/iMouse/&outtime=60000` — enumerate the Files directory the rig writes into; per-directory only, not recursive.
4. `shortcut#2 / shortcut#5` XP `POST /api/shortcut/album/down and /api/shortcut/file/down` — optionally harvest anything worth keeping to the PC before deleting.
5. `shortcut#3 / shortcut#6` XP `POST /api/shortcut/album/del and /api/shortcut/file/del` — delete named items; a comma id list applies one shared filename list to every listed device.
6. `none (XP-only)` XP `POST /api/shortcut/album/clear {id, album_name, outtime}` — wipe a whole album; irreversible from the API's side, so harvest first.
7. `none` XP `none — controller-side` — parse each size string to bytes, sum per phone per run, persist and trend, alert on absolute total or daily delta.
8. `none` XP `none — external scheduler` — there is no cron or job endpoint in either edition; raise outtime well above the 15000 ms default for long listings and transfers.

NEEDS

- iMouse Shortcut installed on every phone with Allow Running Scripts, Allow Sharing Large Amounts of Data, and both delete-without-confirmation permissions on.
- Shortcut fetched from the console's QR code, its IP field edited to the kernel PC, run once with every "Allow" prompt answered.
- Shortcut bound to an AssistiveTouch key, triggered by a HID hotkey from a board that is bound and at /config/usb/get state 2.
- Phone reachable from the kernel on the LAN — the Shortcut posts its results back to port 9911 — and unlocked with Auto-Lock Never.
- iOS 15+ for every file call (these XRs qualify on 18.7.10).
- IMConsole.exe running and logged in.
- Deliberately does not need the OCR or image-search plug-ins; this chain never touches /pic/*.

WATCH OUT

- None of the /shortcut/* endpoints have actually been probed on this rig; the 10 Sep probe never touched them — treat the whole chain as unverified until the device-ip check passes.
- size comes back as a human-readable string ("5.4 MB"), so a summed total is a relative trend indicator, not an accounting figure.
- No date filter exists on XP; age-based retention has to be computed client-side from create_time, which means listing everything you might delete first.
- num defaults to 10 with no documented maximum, and ordering is only implied — a phone with thousands of photos may under-report growth from a truncated listing.
- The listing only sees the Photos library and the queried Files directory; app caches, mail attachments and system storage are invisible, so this is a leading indicator, not a storage gauge.
- Deletion likely moves items to Recently Deleted for 30 days on iOS, which would leave the enumerated total reduced without freeing space immediately — unverified by either vendor doc, so test before trusting cleanup to recover space.
- Multi-device addressing is asymmetric: listings and transfers are single-id only, while deletes take a comma id list applied to one shared filename list — only correct for identically named rig-generated files.

- outtime is milliseconds on XP and seconds in the Phonefarm doc; porting Phonefarm sample values gives 15 ms timeouts instead of 15 seconds.
- The Shortcut round trip (board hotkey, AssistiveTouch, app launch, Wi-Fi POST-back) is unmeasured and almost certainly does not share the concurrency headroom of the measured /pic/screenshot and /mouse/* numbers — budget serial execution or very low concurrency (2-4 phones) until measured.
- A phone with a weak Wi-Fi link should be expected to have a slower, less reliable Shortcut round trip too, since the POST-back rides the same link that delivers only ~4.5 fps on this rig's weak XR.

SCALE

Each phone's cost is dominated by the Shortcut round trip, not the HTTP call itself — board hotkey, AssistiveTouch, app launch, Wi-Fi POST-back — with a default 15 s ceiling per call, so a full sweep of one phone is seconds, not milliseconds. Listings are single-id, so a 19-slot rack is at least 38 sequential listing calls; deletes collapse onto a comma list only where filenames are shared across phones. Whether shortcut calls from different phones run in parallel is unverified — the measured per-phone independence on this rig (two pollers 83+85 req/s, ten pollers 512 req/s) applies to /pic/screenshot and /mouse/*, not to the hotkey-plus-LAN Shortcut path, which funnels through each phone's own board and Wi-Fi link. Plan a bounded concurrency of 4-8 phones inside a maintenance window and measure before widening; a realistic full-rack sweep runs on the order of 3-10 minutes, longer with any zip harvest, so this is designed as one sweep per maintenance window rather than a repeating loop.

Calibration and link-health diagnostics

Calibration library curation and pointer-accuracy drift audit

PARTIAL WITH CAVEATS

Treat the mouse-parameter calibration record as a curated fleet asset rather than something the kernel silently auto-matches. For every phone: confirm a library record exists for its model and iOS version, confirm the device record actually carries a non-empty location_crc, then measure real pointer accuracy by commanding three known mirror-pixel coordinates and locating the pointer dot in a raw BMP frame by pixel diff. Both XRs on this rig share one library CRC, byte-identical to the vendor's own doc example rather than a per-device measurement; nothing in the API tells you whether that profile is correct, and a wrong record does not error, it just puts every tap a few percent off.

HOW

1. `get_device_list` XP `GET /api/device/get` — read location/location_crc, model, version and imgw/imgh per phone; an empty location_crc fails every /mouse call with code 11.
2. `get_devicemodel_list` XP `GET /api/config/devicemodel/get` — confirm a library record exists for this model plus iOS version; the XP doc elides the response shape, so probe it once before parsing.
3. `mouse_reset_pos` XP `GET /api/mouse/reset?id=<ids>` — reset the pointer to a known origin before the probe.
4. `mouse_move + get_device_screenshot` XP `GET /api/mouse/move?id=<ids>&x=&y= then GET /api/pic/screenshot?id=<id>&binary=true` — command three known mirror-pixel points (near-origin, far corner, centre) and grab a frame after each.
5. `(analysis)` XP `(analysis, no call)` — a small constant offset is normal; an error that grows toward the far corner is a scale error needing a per-device collection.
6. `send_key (fn_key) + click` XP `GET /api/key/sendkey?id=<ids>&fn_key=WIN+h (twice) then GET /api/mouse/click?id=<id>&x=&y=` — functional cross-check to rule out the stuck-click state before

condemning a calibration record.

7. `mouse_collection_open + cfg` XP `GET /api/device/collection/mouse?id=<id>&cmd=1` — start a per-device collection; XP collapses Phonefarm's open/cfg/close trio into one cmd-coded path.
8. `mouse_collection_cfg_ret + collection` XP `WS callback collection_mouse` — watch progress codes (8 scan QR, 9 enter collection page, 14 start working, 21 working) rather than polling.
9. `mouse_collection_close` XP `GET /api/device/collection/mouse?id=<id>&cmd=2` — always stop collection; a stuck session blocks every other phone's collection with code 5.
10. `save_dev_location` XP `GET /api/device/collection/mouse/save?id=<id>&describe=<text>` — publish the record to the shared library so same-model phones auto-match on connect; irreversible on XP, no delete endpoint exists.
11. `set_dev (location_crc)` XP `GET /api/device/set?id=<ids>&location_crc=<crc>` — pin other same-model phones to the known-good CRC; a comma list covers a whole model group in one call.
12. `dev_connect callback` XP `WS callbacks dev_connect / dev_change` — re-arm the audit whenever a phone's iOS version changes, since the library key is model plus iOS version.

NEEDS

- XP kernel reachable at 9911 with path-style calls; IMConsole.exe running and logged in.
- Board bound and at /config/usb/get state 2 per phone — control readiness is a separate channel from video readiness.
- Phone-side accessibility settings the calibration was collected against (AssistiveTouch on at maximum sensitivity, tracking speed at maximum, Full Keyboard Access on, Auto-Lock Never, portrait lock) — none of these are queryable over the API.
- For collection only: the vendor console's QR code to open the phone-side collection page; the XP docs do not document that URL directly.
- Only one phone collecting at a time per kernel.
- No plug-in required for any step — the probe uses pointer-dot pixel diff rather than OCR or image-search.

WATCH OUT

- The audit half is fully native and already live-probed on this rig; the collection half needs a manual QR-driven step, which is what keeps this out of "native."
- /config/devicemodel/get's response body is elided in the XP doc ("too many, omitted") — the field shape must be probed before parsing, not assumed from the Phonefarm sibling's shape.
- The doc contradicts itself on collection concurrency (comma-list id versus a "device is collecting" error) — assume serial, one phone at a time.
- There is no delete endpoint for a bad library record on XP; a bad publish is permanent and can only be superseded by collecting again and re-pinning phones to the new CRC.
- A wrong location_crc is worse than an empty one — empty fails loudly with code 11, wrong succeeds and lands taps somewhere plausible. Never write a CRC that was not read from the library or freshly collected.
- The probe measures where the pointer went, not whether a tap registered — always reset first, since a move-lands-but-click-does-nothing result is the stuck-click state, not bad calibration.
- Frame timing bounds the probe: the board walks at ~0.6 ms/px, and the two XRs deliver wildly different frame rates (~29 versus ~4.5 new frames/s) — wait for a genuinely new frame rather than assuming a fixed rate.
- Grouping probe moves into one comma-separated request only works for phones sharing identical imgw/imgh.

- Only two of nineteen boards currently have a bound, mirroring phone — the full-rack throughput numbers are extrapolated, not measured beyond n=2; a realistic estimate is 5-15 seconds of wall time for a 19-board rack, not an optimistic 3-5 seconds, once per-phone frame-wait stacks on top of board-walk time.
- Publishing to the shared library carries the uploading phone's own device name in an "upname" field — rename the phone and use a neutral describe string before publishing, since this is a fleet-wide, irreversible record.

SCALE

Per phone the probe is one reset, three moves and three raw-BMP grabs: board time dominates at ~0.6 ms/px, roughly 1.05 s of pointer walk across the three probe points, while raw BMP grabs are cheap at p50 2 ms. The real per-phone cost is waiting for a new frame after each move, about 35 ms on the fast XR and 220 ms or more on the slow one, so budget roughly 2 s per phone end to end. Because /mouse/move fans out on a comma list and the board walks every listed phone in parallel for the same wall time, a whole geometry-matched group costs the same as one phone to move; only the screenshots are single-id and fan out independently (measured 83+85 req/s with two pollers, 512 req/s with ten). A more defensible estimate for a 19-board rack of identical geometry is 5-15 seconds of wall time, not an optimistic 3-5, once frame-wait is stacked on top of board-walk rather than assumed simultaneous. Collection is the opposite: strictly serial, one phone at a time with a manual QR step, so budget 1-3 minutes per phone and treat it as a per-model-per-iOS one-off rather than a fleet loop.

Wi-Fi link-quality scorer: rank phones by delivered frames per second

NATIVE API WITH CAVEATS

Drive every phone with one mirrored swipe request, poll each phone's raw AirPlay frames in parallel, and count how many are genuinely new by mean absolute pixel difference. The resulting new-frames-per-second figure is each phone's delivered mirror rate, trended per phone per day and fed into the scheduler as that phone's screen-verification timeout. Two identical iPhone XRs on the same iOS build and the same LAN measured 28.9 and 4.5 new frames per second under continuous motion; nothing in either API edition reports RSSI, link rate or retransmits, so delivered frame rate is the only observable the rack has for radio health, and every endpoint in this chain already answers on this rig.

HOW

1. `get_device_list` XP `GET /api/device/get` — enumerate the rack; only score phones with state non-0 and a live air_pid.
2. `none` XP `GET /api/config/imserver/get` — read the global cast configuration before attributing a low score to the radio; a throttled air_play_fps would explain it instead.
3. `get_usb_list` XP `GET /api/config/usb/get` — confirm the control channel (board state 2) so a dead board is never mistaken for a dead radio.
4. `mouse_reset_pos` XP `GET /api/mouse/reset?id=<ids>` — clear the stuck-pointer state across the whole target set before the swipe stimulus.
5. `send_key (fn_key)` XP `GET /api/key/sendkey?id=<ids>&fn_key=WIN+h (twice)` — normalise every phone to the home screen so the stimulus produces comparable motion.
6. `swipe` XP `GET /api/mouse/swipe?id=<ids>&direction=left&len=0.8` — fire one synthetic motion stimulus at every phone in a single request; two live doc traps in the parameter spelling and typing.
7. `get_device_screenshot` XP `GET /api/pic/screenshot?id=<id>&binary=true` — poll raw frames, one concurrent poller per phone; single-id only, no server push on XP.
8. `none` XP `none — client-side` — count genuinely new frames by mean absolute pixel difference above ~1; never MD5, which produced a documented false conclusion earlier in this project.

9. `none` XP `none — client-side` — validate the run before publishing a score: if the largest single-frame difference never exceeds ~20, the swipe never landed and the cause is unbound hardware, not a bad radio.
10. `set_airplay_mode` XP `GET /api/device/airplay/set?id=<ids>&air_ratio=720 (or air_fps / air_refresh / air_img_fps / air_audio)` — optional remediation: lower cast cost for a weak phone and re-measure; disconnects and re-casts, and changes imgw/imgh.
11. `get_device_list (ip, mac)` XP `GET /api/device/get` — optional correlation with AP association tables or DHCP leases outside the API; neither edition reports RSSI, channel or PHY rate.

NEEDS

- XP kernel reachable at 9911 with path-style calls; IMConsole.exe running and logged in.
- Each phone mirroring (state non-0, live air_pid) with an established AirPlay socket, since video delivery is what is being measured.
- Board bound at /config/usb/get state 2 with a non-empty location_crc, or the stimulus cannot be delivered.
- Invisible iOS prerequisites (AssistiveTouch on at maximum sensitivity, tracking speed at maximum, Auto-Lock Never, portrait lock) — none queryable over the API, and any one being off makes the stimulus silently do nothing.
- Phones parked on content where the swipe produces full-screen motion, such as a multi-page home screen; a still screen scores every phone near zero.
- One HTTP client per phone able to sustain ~1.6 MB per poll with buffer reuse; no plug-in, Shortcut or edition gate involved anywhere in this chain.

WATCH OUT

- The metric is delivered frame rate, not Wi-Fi link quality — no function in either edition reports RSSI, link rate or retransmits; earn the "Wi-Fi" label by re-running a phone alone and together, which is exactly how the 28.9-versus-4.5 fps gap on this rig was shown to be the phone's own link rather than kernel contention.
- Cross-phone ranking is only fair when phones show the same content, since source frame rate is content-dependent and the two XRs here have different home screens; treat per-phone trend against its own baseline as the defensible use, the league table as indicative only.
- The stimulus is real input on a real phone — run it only on a normalised, disposable screen, never mid-task.
- AirPlay tops out around 30 fps; treat anything above ~27 as "full rate" and do not chase the difference — the score has little headroom among healthy phones and is much better at catching a phone that has fallen far.
- Use a fixed poll period, never one derived from observed frame gaps — adaptive pacing fed by measured gaps decayed a 23 fps stream to 16.5 fps through positive feedback.
- Two live doc traps in the swipe call: the parameter table says "stepping," every JSON example spells it "steping," and direction is typed Integer in the table but passed as strings in every example.
- The remediation step is not free: /device/airplay/set drops the cast and changes imgw/imgh, invalidating every cached coordinate and template for that phone.
- /config/imserver/get was not exercised in the 10 Sep probe — confirmed as a real documented path, but treat that read as "should work," not "proven," until one live GET is run.
- Pin the stimulus to a single normalised state (home screen page 2 or later), since a vertical scrollable list under a horizontal swipe may not move at all and would falsely read as a dead link.

SCALE

Frame pollers are independent with no shared kernel bottleneck: two phones polled flat out gave 83 and 85 req/s at p50 3 ms, identical to either alone, and ten concurrent pollers sustained 512 req/s aggregate at p50 3 ms; input and video traffic do not interfere in either direction. The stimulus itself is a single request regardless

of fleet size, since /mouse/swipe fans out on a comma list and the boards walk in parallel, costing roughly 300 ms of pointer travel whether one phone or twenty are listed. A 5 s measurement window plus normalisation puts a full-rack pass well under a minute, and each phone's full cycle (two Home presses, swipe settle, 5 s window) is on the order of 6-7 seconds even though the calls fan out in parallel — comfortable for a nightly or per-shift cadence, not a continuous poll. The remediation path is the slow part, since each air_ratio change forces a disconnect and re-cast per phone.

Remote diagnostics and hands-on control

Remote-hands desk for one phone in a closet

NATIVE API WITH CAVEATS

A support engineer picks one phone anywhere on the rack and operates it as if it were in hand: a polled raw-BMP mirror from `/pic/screenshot`, real pointer control via `/mouse/move` plus down/up (or `/mouse/click` and `/mouse/swipe`), and the full ASCII-plus-hotkey set through `/key/sendkey`. Because control and video are separate channels, the same session catches the failure the vendor console cannot tell apart from a working phone: a perfect picture that ignores every tap. A four-rung recovery ladder, pointer reset, board restart, mirror reconnect, phone restart, sits behind it for when the phone itself is stuck.

HOW

1. `get_device_list + get_usb_list` XP `GET /device/get?id=<id> and GET /config/usb/get` — read both channel states first: video-ready is device state non-0 with a live air_pid, control-ready is that phone's board at state 2 with vid/pid/uid and location_crc populated. Also record imgw/imgh (500 x 1080 here), the coordinate space for everything that follows.
2. `get_device_screenshot` XP `GET /pic/screenshot?id=<id>&binary=true` — single-id polled mirror. Raw BMP measured p50 2 ms at 1.62 MB (84 req/s); `jpg=true` trades that for ~96 KB at a hard ~21 req/s cap. There is no server-push video stream on XP.
3. `mouse_reset_pos` XP `GET /mouse/reset?id=<id>` — open every session with this. It is the only call that clears the measured stuck-click state, where /mouse/click silently does nothing while moves and hardware keys keep working.
4. `mouse_move / down / up / click / swipe / wheel` XP `/mouse/move?id&x&y · /mouse/down?id[&button] · /mouse/up?id · /mouse/click?id&x&y · /mouse/swipe?id&direction&len · /mouse/wheel?id&direction&len&number` — a real drag is down, streamed moves, up, not a canned swipe. `button` is numeric (1 left, 2 right, 3 middle). The board walks the pointer at ~0.6 ms/px, so coalesce to the latest position rather than streaming every intermediate point.
5. `send_key / key_down / key_up / key_release_all` XP `/key/sendkey?id&key=<ASCII> or &fn_key=<hotkey> · /key/down · /key/up · /key/upall` — the documented hotkey table covers Home, Back, App Switch, Control/Notice bar, Lock, screenshot, pass-through mode, clipboard and volume/media keys. `key` is ASCII only in both editions.
6. Unicode text — XP `POST /shortcut/clipboard/set?id&text=<unicode>[&sleep] then GET /key/sendkey?id&fn_key=WIN+v` — XP has no direct Unicode send; paste via the iMouse Shortcut's clipboard is the documented route, and it needs the Shortcut installed and bound.
7. Recovery ladder — `mouse_reset_pos / key_release_all` XP `/mouse/reset · /key/upall`, then `restart_usb` XP `/device/usb/restart?id=<vid>|<pid>`, then `discon_airplay / auto_connect_screen` XP `/device/airplay/disconnect then /connect`, then `restart_device` XP `/device/restart or`

`/shortcut/switch/device?state=1 or fn_key=CTRL+ALT+SHIFT+WIN+r` — four rungs of escalating disruption, cheapest first.

8. Session feed — `XP ws://<host>:9911/api` — hold a socket for `im_connect` (the only place the kernel version is exposed on XP), `error_push` (the only async signal an input call failed at the board), and `airplay_connect_log` / `restart_log` during the session.

NEEDS

- A network path to the kernel on TCP 9911 restricted to a VPN or a firewalled management segment, never an open LAN — the API has no authentication of any kind.
- Target phone bound to its board (state 2) with mouse parameters present, and mirroring live (state non-0, matching air_pid).
- IMConsole.exe running and logged in; IMService.exe alone leaves the phone at state 0 and the board list empty.
- iOS settings no API call can read: AssistiveTouch on with tracking sensitivity maxed, Full Keyboard Access on, Trackpad & Mouse tracking speed maxed, Auto-Lock Never, portrait lock.
- iMouse Shortcut installed, IP-edited, run once with permissions granted, and bound to an AssistiveTouch custom key — only for Unicode typing and the Shortcut restart route.
- Board firmware generation read once from the phone's own AssistiveTouch → Devices screen; the API's ver/uver field is unusable on this build.

WATCH OUT

- No server-push video on XP: the live view is fixed-period polling, one request per phone. Over a WAN, prefer `jpg=true` with a `rect` crop of the working area.
- Bandwidth is the real remote-link constraint: raw BMP is 1.6 MB per frame at 2 ms, JPEG is ~96 KB but 40 ms with a hard ~21 req/s ceiling.
- Pointer feel degrades with distance, not load (~0.6 ms/px, ~600 ms edge-to-edge); coalesce operator input rather than sending every intermediate point.
- Each phone's own AirPlay link caps what "live" means regardless of the API — one XR here delivered ~29 new frames/s and its twin ~4.5 on the same network. A fixed poll period on the slow phone mostly re-fetches the same frame; verify with a pixel-diff wait, not a 30 fps assumption.
- `/device/restart` is expected to fail on this rig until the OCR plug-in is installed (code 15/28); use the Shortcut route or the CTRL+ALT+SHIFT+WIN+r hotkey instead. That dependency is specific to the UI-driven restart and to any script that drives Control Center's mirroring picker by coordinates — `/device/airplay/connect` itself is a direct call and does not require OCR.
- Restarting the phone (rung 5) is not actually remote-hands: it returns the phone to the Lock Screen and nothing in the API can type a passcode, so this rung converts the session into "needs a human at the rack afterward" rather than a same-tier recovery step.
- Verify a gesture landed by mean pixel difference in a small box around the target (1.4-9.8 against a ~0.02 idle floor) — `/mouse/click` and `/mouse/move` return only a status code, and MD5 or byte comparison of the whole frame is unusable because of encoder noise and live UI elements.
- The session is, functionally, unaudited remote access to whatever the phone is signed into, with no log beyond what your own tooling keeps.

SCALE

One operator per phone by design, since `/pic/screenshot` refuses multiple ids, but that single session is cheap: raw frames sustain 84 req/s at p50 2 ms, and input and polling measurably do not contend (streaming moves while polling raised throughput to 98.8 req/s). Several engineers can run independent sessions on different phones without scheduling, ten concurrent pollers sustained 512 req/s total, but on this rig only two phones currently show state non-0, so the real concurrency ceiling today is two simultaneous sessions, not the API's request budget. Per-session pace is bounded by pointer-walk physics and operator think time: a full-screen traversal costs ~600 ms, and a realistic move-click-verify cycle runs one to three actions every two to four seconds, on the order of 900-1,800 verified UI actions per hour per phone.

Rack-wide controllability matrix and cross-wired board detection

NATIVE API · WITH CAVEATS

A state-independent optical self-test that proves, per phone, that the board bound to its device record actually drives that screen. Pass A sends one mirrored `/mouse/move` to the whole id list and diffs a small pixel box per phone to separate controllable from mirroring-but-deaf. Pass B is the part that actually catches cross-wiring: it sends every phone its own distinct coordinate via N single-id calls and checks which coordinate showed up on which screen, because a mirrored move to a swapped pair looks identical from pass A alone. It runs natively today on this rig: no OCR, no image-search plug-in, no Shortcut, no WebSocket required.

HOW

1. `get_device_list` XP `GET /device/get` (no id = all) — imgw/imgh per phone is the coordinate space for the whole test; live-probed on this rig at 500 x 1080 for both XRs.
2. `get_usb_list` XP `GET /config/usb/get` — 19 boards, state 1 = on PC unbound, state 2 = phone attached; this is the denominator of control-ready slots. Never parse or display the ver field, it is mojibake on this build.
3. `send_key` XP `GET /key/sendkey?id=<list>&fn_key=WIN+h` (twice, ~1 s apart) — normalizes every phone to a known screen before probing; optional for the pointer test itself.
4. `mouse_reset_pos` XP `GET /mouse/reset?id=<list>` — clears the stuck-click state and gives the walk a known 0,0 origin so its duration is predictable.
5. `get_device_screenshot` XP `GET /pic/screenshot?id=<one id>&binary=true` — one baseline frame per phone; single-id only, no batching.
6. Pass A liveness — `mouse_move` XP `GET /mouse/move?id=<list>&x=470&y=1050` — one mirrored request drives every board in parallel; from 0,0 a ~1150 px diagonal is ~600-700 ms of walk.
7. Verify pass A client-side: re-screenshot and take the mean absolute pixel difference in a ~40 px box around the target — 1.4-9.8 means controlled, ~0.02-0.03 means no control; never MD5-compare whole frames.
8. Pass B cross-wire permutation — `mouse_move` XP `N x GET /mouse/move?id=<one id>&x=<Xi>&y=<Yi>` — a distinct coordinate per phone, issued concurrently, then box-diff every assigned coordinate on every screen. This is the step that actually reveals a swapped board, and it must cover every phone on every run, not just "suspect" ones.
9. `set_dev` XP `GET /device/set?id=<deviceid>&vid=720A&pid=<correct pid>` — remediate a confirmed swap, one device per call, vid and pid together, then a second call to rewrite the slot name. Re-read /device/get to confirm, since the response only echoes the ids that were set.
10. `mouse_reset_pos` XP `GET /mouse/reset?id=<list>` — park pointers and re-clear the stuck-click state before whatever runs next.
11. Optional — subscribe to XP `ws://127.0.0.1:9911/api` for `error_push` and `usb_change` during the run, so a board that disconnected mid-run is attributed correctly instead of read as a wiring fault.

12. Publish the matrix (phone x control-ok, video-ok, bound-to, actually-drives, hub/port) — entirely outside the API; there is no notification or webhook endpoint in either edition.

NEEDS

- IMConsole.exe running and logged in — with only IMService.exe up, /device/get reported both phones state 0 and /config/usb/get returned an empty board list.
- Every probed phone mirroring (state non-0, live air_pid, ESTABLISHED socket on TCP 17000); an offline phone cannot be optically proved at all.
- Each phone bound to a board (vid/pid/uid non-empty) and that board at state 2, or /mouse/move returns code 2 or 3.
- location_crc non-empty per phone, or every mouse call returns code 11 "No mouse parameters".
- iOS side, unqueryable over the API: AssistiveTouch on with tracking sensitivity maxed, Full Keyboard Access on, Trackpad & Mouse speed maxed, Auto-Lock Never, portrait lock.
- imgw/imgh read per device record, not assumed, so the probe grid matches each phone's own mirror geometry.
- A client-side BMP decoder (bottom-up, 54-byte header, 500x1080x3) and a mean-absolute-difference function. No OCR or image-search plug-in required, and none is installed here.

WATCH OUT

- A single mirrored move to the whole id list cannot detect cross-wiring: if A's record is bound to B's board and vice versa, the mirrored request moves both to the same coordinate and both screens show a dot, so both phones look healthy. Distinct per-phone coordinates (pass B) are the only mechanism that finds the swap, and it must run every time, not only against phones already under suspicion.
- Offline phones are invisible to this test: the proof is optical over the AirPlay channel, so a state-0 slot has no observation channel. Report such slots as unproven, never as ok, and never rely on a code-0 /mouse/move return as proof of anything, since that is exactly the condition under which the measured stuck-click state hides.
- The probe is blind to the stuck-click state by design: moves and hotkeys kept working on the stuck phone while /mouse/click silently failed. This matrix proves the pointer channel only; the tap channel needs a separate runbook.
- Signal margin is real but not huge (1.4-9.8 against a 0.02-0.03 idle floor). Place probe coordinates away from the clock, live widgets and high-contrast wallpaper edges, and diff against a same-run idle control rather than a stored baseline.
- Phones with different imgw/imgh must get their own grid rather than a shared coordinate; this rig's two XRs match at 500x1080, but the doc's own examples show 750x1334 and 460x818, so a mixed rack is the normal case.
- Remediation is manual-ish and destructive: /device/set is one device per call and rewrites persistent fleet state (DeviceList.json). Require two consecutive agreeing runs plus an operator confirmation before any rebind, and log the before/after vid/pid/uid — a false positive from a stalled Wi-Fi frame would swap a working binding into a broken one.
- Home-screen normalization changes what is on every screen; capture the baseline after both WIN+h presses settle, and give the slow phone (~4.5 new frames/s) several hundred ms rather than waiting for a frame to change.
- Pass B's "N concurrent single-id calls run in one ~600 ms walk, not N times" is a reasonable inference from board-independence data measured for two phones, but concurrent dispatch to N separate boards as N separate HTTP requests was not directly measured — treat it as inferred, not confirmed, at larger N.

- This test and stuck-click detection are two separate runbooks that happen to share the same reset/normalize steps; do not conflate a clean cross-wire matrix with a clean tap channel.

SCALE

Pass A is cheap and sublinear in requests: for 50 phones, about three fan-out calls plus 2N screenshots and one ~600-700 ms pointer walk, well under two seconds of wall time. Pass B is the expensive half and is now mandatory: N concurrent single-id moves (still one ~600 ms walk, not N times) plus another N screenshots, then an N x N box-diff that is trivial client-side CPU — a full 50-phone matrix including cross-wire recovery is a few seconds end to end. On this 2-phone rig a full pass is dominated by one diagonal walk plus four screenshot round trips, well under two seconds, so hundreds of passes per hour are technically possible; scaled to a 19-board rack, on the order of 100-300 full-rack passes per hour is achievable before the grid-capacity ceiling (~56 distinguishable probe points at >=60 px spacing on a 500x1080 mirror) forces pass B into batches for larger racks. The real limits are bandwidth and memory, not the kernel: 1.62 MB per raw frame means 3N frames is ~240 MB of loopback traffic for 50 phones, so reuse one buffer per poller. Realistic cadence, given that both passes move every visible pointer and press Home twice, is nightly or per-shift, not continuous, and it is mutually exclusive with running automation on the same phones.

Guarding fleet configuration

Config-drift guard on an unauthenticated kernel

NATIVE API WITH CAVEATS

Anything on the LAN can rewrite the kernel config, rebind a board, move a phone between groups, delete a device record or log the kernel out, the XP API has no authentication of any kind. Hold one WebSocket, keep a hashed baseline of the config, device, group and USB lists, and treat the broadcast callbacks as a free change feed: diff each callback against the baseline, auto-revert the fields that apply live, and alert-only on the fields that need a kernel restart. One attribution shortcut helps: `im_config_change` is documented as going to other connected clients only, so any copy you receive is by definition not your own write.

HOW

1. Open the feed — XP `ws://127.0.0.1:9911/api` — held for the controller's whole life. Callbacks are WS-only with no HTTP polling equivalent; the envelope is {fun, status, message, msgid:0, data}, and msgid is 0 on every broadcast, so it carries no correlation value.
2. Baseline the kernel config — XP `GET /config/imserver/get` — air_play_name, mdns_type, allow_ip_list, thread_mode, mouse_mode, air_play_ratio/fps and the rest, in one payload.
3. Baseline every device record — XP `GET /device/get` (optional id list) — hash only the policy fields: deviceid, name, gid, vid, pid, uid, location_crc. Exclude state, air_pid, air_handle, ip and rotate, which change legitimately every session.
4. Baseline groups and the board inventory — XP `GET /device/group/get and GET /config/usb/get` — hash vid|pid|uid only; exclude ver, which is mojibake on this build and would read as phantom drift on every poll.
5. Log your own writes (path, field, value, time) so an incoming callback can be matched by value within a short window — there is no client id or origin field anywhere in the API, and msgid is always 0.
6. On `im_config_change`: diff the pushed config against the baseline — the callback carries the full object, so no re-GET is needed, and any copy you receive is by definition someone else's write.
7. Revert live-apply fields immediately — XP `POST /config/imserver/set` — the nine fields that apply with no restart are lang, mdns_type, connect_failed_retry, allow_ip_list, auto_connect, auto_updata,

thread_mode, mouse_mode, flip_right.

8. Alert, never auto-revert, on restart-required fields — `XP GET /imserver/restart` — air_play_name/port/ratio/fps/img_fps/refresh_rate/audio, opencv_num, ocr_num and enable_hardware_acceleration all need a restart, which drops every mirroring session on the PC.
9. On `dev_change`: diff the pushed device record, then revert an unauthorized binding, group or calibration change with `XP GET or POST /device/set` — name/gid/location_crc can be batched across phones in one call, but a board rebind (vid+pid) is one call per phone, and there is no documented way to clear a binding.
10. On `group_change / group_delete / dev_delete`: restore structure with `XP /device/group/set and /device/set` — a deleted device record is only partly repairable; the record itself does not return until the phone next casts, and online devices cannot be deleted at all (code 7).
11. Watch the licence/session channel — `XP WS logout, im_log, user_info; baseline with GET /config/user/info` — anyone can log the kernel out over the LAN, which then refuses to bring devices online. Alert only; never auto-relogin.
12. Network layer, the only actual control — a Windows Firewall rule restricting inbound TCP 9911 and 17000 to the controller host and phone VLAN. mdns_type/allow_ip_list gate only AirPlay discovery, never the HTTP/WS API itself.

NEEDS

- XP-edition kernel on TCP 9911; Phonefarm-style POSTs return HTTP 500 and 9912 is closed, so the whole guard must use XP path style.
- A persistent WebSocket client with automatic reconnect and an immediate full re-baseline on reconnect — every observation channel here is a broadcast callback with no polling fallback, so a dropped socket is a blind spot.
- Your controller as the only intended writer, with a durable write log for self-echo filtering.
- IMConsole.exe running — a baseline taken with only IMService.exe up is garbage; gate baselining on a non-empty board list and at least one device record.
- Operator agreement on which host may write, since the vendor console is itself a legitimate writer and will otherwise trip the guard on every manual action.
- Local admin on the rack PC for the firewall rule (hardening, not required for detection itself).

WATCH OUT

- The entire detection half rests on WebSocket broadcast callbacks that were never live-probed on this rig, only documented. This is a materially bigger unverified leap than most other entries in this section, which are built on live-probed request/response endpoints; treat the mechanism's confidence accordingly until a callback is actually observed firing.
- `/config/imserver/get` and `/set` are documented but not probed here; the 10 Sep probe used /config/get, which 404s. Confirm the documented path answers before building on it.
- Partial-set semantics of /config/imserver/set are undocumented — every example sends the full field set, and nothing says omitted fields are preserved. Re-send the complete baseline object on revert until measured otherwise, but note that a stale baseline plus a full-object revert can clobber a concurrent legitimate change; tie baseline refresh to the write log, not to a fixed schedule.
- enable_hardware_acceleration is settable but appears in neither the GET response nor the im_config_change payload, so drift in that one field is invisible with no workaround.
- Attribution is best-effort everywhere except im_config_change: dev_change is documented as going to all clients, so self-echo must be filtered by value-compare, and two writers setting the same value in the same window are indistinguishable.

- A deleted device record is only partly repairable, and there is no documented call to clear a binding once set.
- This is detection and repair, not a security control — a hostile LAN caller can rewrite faster than you can revert. The firewall rule is the only thing that actually prevents anything, and a compromised guard host itself gets unauthenticated write access to every phone's binding, exactly like everything else on the LAN.
- The user_info push is described as "every 30 seconds," but nothing establishes that cadence as a guaranteed heartbeat rather than an on-change push; verify it before wiring liveness alerting to its absence.

SCALE

Effectively free at any rack size: the steady state is one open WebSocket and zero polling, since the kernel pushes change callbacks only when something actually changes, plus a user_info heartbeat roughly every 30 seconds. Baseline re-reads are four small GETs taken at startup, after reconnect and after each accepted change, not on a timer, and payloads scale linearly with the rack while staying far below the 1.6 MB of a single screenshot frame. Input and screenshot traffic measurably do not interfere with each other, so a guard loop costs the video and control pipelines nothing. Revert writes are batchable for name/gid/location_crc across a comma-separated id list; only board rebinds are one call per phone. Revert-action rate, not request rate, is the only throughput question that matters, and it should stay low by design: rate-limit reverts and escalate to a human after a small number of flaps rather than fighting a persistent second writer.

Scaling beyond one rack

Multi-PC scale-out: one XP kernel per receiver, mDNS-pinned phones, and an external federating coordinator

PLUS EXTERNAL TOOLING · WITH CAVEATS

Nothing in either API edition spans PCs, so scaling past one rack means one IMService/IMConsole per PC, each with its own AirPlay receiver name and its own 9911 API, plus an external coordinator holding one WebSocket per kernel and presenting the union as one rack under <pc>/<deviceid> keys. Two verified per-kernel levers make it workable: `mdns_type` plus `allow_ip_list` pins which phone IPs may discover a given PC's receiver, applied live with no restart, and `air_play_ratio/fps/img_fps/refresh_rate` (global, needs restart) or `/device/airplay/set` (per device, needs a reconnect) tune capacity. Recovery per kernel is `/device/airplay/connect/all` plus `/imserver/regmdns`. The coordinator itself is entirely your own code; XP has no cross-PC function, no cross-PC id namespace and no multi-receiver concept.

HOW

1. Give each PC a unique receiver identity, then restart — `XP /config/imserver/set?air_play_name=...&air_play_port=17000 then /imserver/restart` — both fields need a restart since neither is in the documented no-restart list.
2. Pin discovery so only this PC's phones see this PC's receiver — `XP /config/imserver/set?mdns_type=1&allow_ip_list=[...]` — applies live; the doc's own wording for rule 1 vs 2 is self-contradictory, so validate on a spare phone before relying on it.
3. Give each phone a static IP with the router/gateway field left empty, so allow_ip_list, which is keyed by IP, has a stable target — a phone-side iOS setting, not an API call.
4. Set the capacity profile per kernel or per phone — `XP /config/imserver/set (air_play_ratio, air_play_fps, air_play_img_fps, air_play_refresh_rate, air_play_audio) + /imserver/restart; or per`

`device /device/airplay/set?id=<ids>&air_ratio=&air_refresh=&air_fps=&air_img_fps=&air_audio=` — XP has no mode preset; the Phonefarm sibling's ~5/10/20 Mbps figures do not apply here.

5. Coordinator: one event-bus socket per kernel — `XP ws://<pc>:9911/api` — the same JSON envelope works over HTTP and WS, so poll over HTTP and keep the socket only for pushes.
6. Key every device as <kernel host>/<deviceid> — `XP GET /device/get (optional id list)` — deviceid is phone-derived and distinct from mac, so it likely survives a re-cast to a different kernel; namespace by host anyway, since the host is the routing address.
7. Cross-PC fan-out is N parallel requests, one per kernel — `XP /mouse/*, /key/sendkey, /device/set, /device/airplay/connect|disconnect, /device/usb/restart` — the comma-id list only collapses fan-out within a single kernel.
8. Per-kernel health — `XP /config/usb/get, /device/get, /config/user/info; WS user_info, im_log, im_connect(data.ver), error_push` — control-ready is board state 2 plus a bound uid; video-ready is device state non-0 with a live air_pid; im_connect is the only place the kernel version is exposed on XP.
9. Licence each kernel, but do not assume a utag partitions the pool — `XP /config/user/login (phone, password, utag 1–10), /config/user/switch (id 1–10)` — nothing in the docs states each utag carries its own licence headroom.
10. Move a phone between PCs — `XP /device/airplay/disconnect on the old kernel → edit allow_ip_list on both → /imserver/regmdns on the new PC → /device/airplay/connect (or auto_connect) on the new PC` — the board moves physically with the phone; XP has no force flag, so disconnect explicitly first.
11. Rack recovery after a PC reboot — `XP /device/airplay/connect/all (no parameters), per kernel; /config/imserver/set?auto_connect=true&connect_failed_retry=<n>` — progress arrives on the airplay_connect_log callback.

NEEDS

- One licensed XP kernel and a running, logged-in IMConsole.exe per PC — the coordinator cannot bring a rack up by itself.
- Phones and their rack PC on the same L2 segment, since mDNS/Bonjour does not cross subnets, with AirPlay's port reachable.
- Static IPs on every phone with the gateway field left empty.
- Each phone's control board physically cabled to the same PC that hosts its receiver — a board cannot be reached across machines.
- An external coordinator process you write; there is no federated id space or cross-PC addressing in the API.
- Firewall discipline on every rack PC: 9911 and 17000 restricted to trusted hosts.
- For programmatic re-casting, the OCR and image-search plug-ins installed and connected on each kernel — not satisfied on this rig, /pic/ocr returns code 28.
- Per-PC USB hub topology recorded from Windows PnP, since PIDs are opaque and slot labels do not survive a phone moving to another PC.

WATCH OUT

- The federation layer is entirely your own software, which is why this is "needs external tooling," not native.
- XP has exactly one AirPlay receiver per kernel; the Phonefarm sibling's N-receivers-per-PC concept has no XP equivalent, confirmed by an HTTP 500 on the bare function name. You add PCs, not receivers.
- allow_ip_list gates AirPlay discovery only, never authentication — any LAN host can still drive every phone on any rack PC's 9911, and each added PC multiplies that surface.

- Treat the ~50-100 phones-per-PC and Mbps figures as Phonefarm-edition planning estimates for "one landscape game," never measured on an XP build; XP has no mode preset, and this rig's own two identical XRs already show a 6x frame-rate spread on the same AP. Derive per-PC phone count empirically from measured new-frames/s per phone after placement, and expect the shared Wi-Fi AP's aggregate throughput, not USB hub slots, to be the true per-PC ceiling.
- Automated re-cast (steps 10-11) depends on /device/airplay/connect driving Control Centre via the OCR/image-search plug-ins; those are not connected on this rig, so expect phone-to-PC moves and post-reboot recovery to need a human at each rack until they are installed.
- Changing air_play_name or air_play_port needs a restart that drops every mirroring session on that PC; only the nine documented fields apply live.
- The kernel version is only readable from the im_connect WS callback; a coordinator that never opens the socket cannot tell which kernels are on which build.
- The coordinator, not the API, must enforce the one real invariant: a phone's control target (its board, cabled to one PC) and its video source (AirPlay, routable to any receiver it can see) must terminate on the same machine. A discovery misconfiguration silently violates this and produces exactly the split-channel failure this whole design exists to prevent.
- Scale-out is bounded by human onboarding speed: each new PC needs IMConsole.exe manually started and logged in, so a coordinator cannot auto-provision a new kernel unattended.

SCALE

The kernel API is not the constraint: ten concurrent pollers on one PC sustained 512 req/s, and input and screenshot traffic measurably do not interfere. The real per-PC ceilings are USB hub ports (21 slots, 19 populated on this rig) and shared Wi-Fi/AirPlay bandwidth, and frame size dictates the topology: a raw BMP is 1.62 MB at 2 ms on loopback, so a coordinator that pulls frames across the LAN should never do so, keep vision pipelines on the PC that decodes the frames and ship only results. Orchestration overhead across K kernels is just K simultaneous HTTP calls, no coordination cost beyond that; the comma-id list only collapses fan-out within one kernel. Start per-PC phone count from the vendor's unverified ~50-100 planning figures, then correct downward using measured per-phone new-frames-per-second, since this rig's own identical twins already differ 6x on the same AP; a defensible starting plan is 15-20 phones per PC at normal quality, validated post-deployment.

Readiness and resource tuning

App pre-warming across the rack before a traffic spike

NEEDS SHORTCUT WITH CAVEATS

Foreground and pre-load a set of apps on every phone ahead of a demo, sale or load test by pushing a deep link to the whole rack in one `/shortcut/exec/url` call, clearing any interstitial with a mirrored tap, and confirming the warm state on a sampled phone. Deep linking skips UI navigation entirely, so warming is layout-independent, no per-phone icon coordinates to hand-map, and the call accepts a comma-separated id list, making the whole rack one request.

HOW

1. Confirm control- and video-readiness — XP `GET /device/get (optionally id list) and GET /config/usb/get` — control-ready is board state 2 plus a bound uid, video-ready is device state non-0 with a live air_pid; the two channels are independent.

2. Normalize the target set — `XP GET /key/sendkey?id=<list>&fn_key=WIN+h (x2), GET /mouse/reset?id=<list>` — reset clears the stuck-click state that would otherwise make the interstitial-dismiss tap a silent no-op.
3. Deep-link the whole rack in one call — `XP GET /shortcut/exec/url?id=<id1,id2,...>&url=<scheme://path>&outtime=<ms>` — response carries only data.code/id/message: success means the URL was handed to the phone, not that the app reached the intended screen.
4. Clear any interstitial in the way — `XP GET /mouse/click?id=<list>&x=<x>&y=<y>` — only valid mirrored if every listed phone shows the same dialog in the same place; read what it says on a sampled frame first, never blind-tap.
5. Confirm the feed loaded on a sampled phone — `XP GET /pic/screenshot?id=<single id>&binary=true` — judge "loaded" by mean absolute pixel difference against the pre-launch frame, not byte equality; allow ~220 ms+ per new frame on the slow XR.
6. Keep the rack mirrored and re-cast anything that drops — `XP GET /config/imserver/set?auto_connect=true (global) and GET /device/airplay/connect?id=<list> or /device/airplay/connect/all` — auto_connect is a single global flag with no per-device override; turning it on affects the whole rack, not just the warmed phones.

NEEDS

- iMouse Shortcut installed on every phone with the kernel's LAN IP edited in, run once with every permission granted, and Shortcuts > Advanced set to allow running scripts — not verified present on this rig.
- The Shortcut bound to an AssistiveTouch custom key, since /shortcut/* is triggered by a HID hotkey from the board even though the payload travels over Wi-Fi.
- Each phone able to reach the kernel on TCP 9911 so the Shortcut can post its result back.
- Phones unlocked, Auto-Lock Never, portrait locked — a locked phone leaves the Shortcut unlaunched and the call times out at outtime with code 30.
- Apps installed and signed in, and a known URL scheme or universal link that lands on the exact screen you want warm.

WATCH OUT

- /shortcut/exec/url is unusable until the Shortcut is installed and bound on each phone; on this rig that is unverified, and the failure signature is a silent code-30 timeout after the outtime window, or code 2/3 if the board is unbound.
- Success only means the URL was dispatched, not that the app reached the target screen; "is the app warm" must come from the separate single-id screenshot diff, never from the exec/url response code alone. Not every app exposes a deep link to the exact screen you want, and universal links can bounce through Safari first, reintroducing the layout dependency the deep link was meant to avoid.
- Warming is not durable: a notification, a mirroring drop or auto-lock can silently un-warm a phone, and there is no API push for eviction — the only way to know warming held is to re-poll each phone right before the spike.
- A rack-wide call returns only when the slowest phone answers or times out, so one bad Shortcut binding taxes the whole call's wall-clock time even though the HTTP request itself is O(1); raise outtime for a large rack rather than splitting the call.
- Any bandwidth-lowering figure quoted for /device/airplay/set (the ~5/10/20 Mbps range) is the Phonefarm-edition's own vendor estimate, not measured on this XP rig; label it as a ballpark from the wrong product's docs.

- Do not blind-tap the interstitial-dismissal step: a consent sheet or purchase confirmation dismissed sight-unseen across a whole rack is the single most expensive failure mode in this cluster, since it can trigger a real transaction.

SCALE

Warming dispatch is O(1) for the whole rack, effectively instantaneous request cost, but wall-clock is bounded by the slowest phone's Shortcut round trip up to the outtime ceiling (default 15 s; raise to 30-60 s for a large rack to tolerate a slow-Wi-Fi phone). Verification is O(N) single-id screenshot diffs at ~2-3 ms server-side each, so confirming warm state across 20 phones costs well under a second of API time, though actually waiting for a fresh frame on the slow phone adds ~220 ms+ per phone. A full warm-and-confirm cycle across a rack of ~20 phones should complete in well under a minute if all phones respond promptly, and any pre-spike schedule should still budget up to the full outtime ceiling per attempt to avoid false negatives on the slow phone.

Brightness and cast-quality knobs as a rack power/heat lever

NEEDS SHORTCUT WITH CAVEATS

Hold idle phones at low screen brightness and a downgraded per-device AirPlay profile (low air_ratio/air_fps/air_img_fps, audio off), then restore full quality only on the phone an operator is actually watching. Brightness goes through the iMouse Shortcut (`/shortcut/switch/bril` , one request for a comma-separated id list); cast quality goes through `/device/airplay/set` , which is per device but disconnects that phone's cast and only applies on the next connect, so this is a batched mode switch, not a live dimmer.

HOW

1. Read current cast settings and mirror geometry — `XP GET /device/get?id=<deviceid>` — the record carries air_ratio/refresh/fps/img_fps/audio (0 = use global) plus imgw/imgh, which is the coordinate space for every /mouse call and can move when air_ratio changes.
2. Dim every idle phone in one request — `XP GET /shortcut/switch/bril?id=<id1>,<id2>&state=0.05` — state must be strictly greater than 0 and less than 1; use ~0.05 to dim and ~0.9 to restore, never 0.0 or 1.
3. Downgrade the idle set's mirroring profile — `XP GET /device/airplay/set?id=<id1>,<id2>&air_ratio=720&air_fps=5&air_img_fps=5&air_audio=1` — XP has no 0/1/2 preset, so the profile must be expressed as explicit numbers; the change disconnects the cast immediately and takes effect on the next connect.
4. Re-establish the dropped casts — `XP GET /device/airplay/connect?id=<id1>,<id2> (or /connect/all)` — XP has no force flag, so disconnect first if a session is stuck; progress arrives on the airplay_connect_log WS callback, not in the HTTP reply.
5. Restore the operator-selected phone — `XP GET /shortcut/switch/bril?id=<that>&state=0.9 then GET /device/airplay/set?id=<that>&air_ratio=0&air_fps=30&air_img_fps=30 then /device/airplay/connect?id=<that>` — air_ratio=0 returns that phone to the global adaptive setting, budget a full disconnect/reconnect round trip.
6. Verify the change took — `XP GET /device/get?id=<id> then GET /pic/screenshot?id=<id>&binary=true` — confirm state non-0, a live air_pid, and the echoed air_* values; if imgw/imgh moved, every cached coordinate and colour table for that phone is invalid until re-read.
7. Optional global fallback — `XP GET /config/imserver/get then /config/imserver/set?air_play_ratio=..&air_play_fps=..&air_play_refresh_rate=.. then /imserver/restart` — these fields are not in the no-restart list, so a global change drops every mirroring session on the rack; the per-device call is the cheaper lever.

NEEDS

- iMouse Shortcut installed on every phone, its kernel IP edited in, run once with permissions allowed, and bound to an AssistiveTouch custom key — brightness is a Shortcut function needing both the board hotkey trigger and the phone's LAN path to 9911. Not verified on this rig; the 10 Sep probe never called any /shortcut/* path.
- Board bound and connected (state 2) for the hotkey that launches the Shortcut.
- Phone unlocked, Auto-Lock Never — a locked phone cannot run the Shortcut.
- IMConsole.exe running and logged in — IMService.exe alone leaves the rig at state 0 with an empty board list.
- A per-phone record of imgw/imgh and any colour/template tables, so a later air_ratio change can be detected as invalidating them.

WATCH OUT

- The "power/heat" framing is not physics-checked anywhere in the source material: nothing in the measured hardware facts contains a watt, milliamp or thermal reading for board, hub, PC or phone. A phone screen at minimum brightness plausibly saves on the order of 0.3-1 W, a real but modest amount, while the dominant heat sources named for this rig, 19 USB HID boards, three hubs, a PC decoding up to 19 AirPlay streams, sit entirely outside the API and are unaffected by air_ratio or brightness. Present this as an unverified, plausible, low-magnitude side effect of a real bandwidth-reduction lever, not as validated thermal management.
- There is no airplay_mode on XP at all; the Phonefarm 0/1/2 presets and their Mbps/phone-count figures are that edition's vendor claims, not measurements of this rack, and there is no substitute Mbps table for XP's raw knobs.
- /device/airplay/set disconnects the target phones' casts immediately and applies only on the next cast, so it is strictly a batch, shift-level mode switch, never call it on a phone with an in-flight tap-critical sequence.
- Auto-cast recovery after the drop is uncertain on this rig: the OCR plug-in is confirmed not connected (code 28), and the XP code table lists 15 and 28 as cast failure modes, so run one real disconnect/reconnect cycle on a spare phone as a go/no-go gate before building any workflow that assumes automatic re-cast — a rig where re-cast silently fails turns every idle-dim into a one-way trip needing a human to re-cast by hand.
- Lowering air_ratio changes imgw/imgh, the coordinate space for every /mouse call and the scale for any colour table; re-reading /device/get after every /device/airplay/set should be automatic, not an optional check.
- Brightness via the Shortcut has a 15 s default timeout per phone, and a rack with even one broken binding turns a one-request brightness call into a 15+ second block, silently, unless code 30 is checked per id.
- This lever covers screen and radio only; board, hub and USB heat, and the PC's own decode load, are outside the API entirely.

SCALE

Both levers fan out, so dimming or re-profiling the whole rack is one request each, not N, but every /device/airplay/set drops and re-establishes the listed phones' casts, so batch the idle-set change once per shift rather than per operator focus change, and consider upgrading only the focused phone while leaving the rest at the global profile. Brightness has no cast side effect and can be toggled freely, but a rack-wide call is bounded by the slowest phone's Shortcut round trip, worst case the full 15 s timeout per stalled phone, so budget a handful of rack-wide brightness toggles per hour rather than continuous use. Quality-profile switching, since each call

> drops and re-establishes casts, should be treated as a per-shift operation, on the order of 1-4 switches per hour across the whole rack, and if auto-reconnect fails, plausible given this rig's code 28/15, effective throughput for that phone drops to zero until a human re-casts it by hand.

Outside the box

Every entry below rides the same load-bearing fact: an XP input endpoint that takes a comma-separated `id` list costs the same board time whether it addresses one phone or the whole rack, because the board walks the list in parallel rather than looping over it. That single property turns a phone farm into a synchronized display wall, a camera array, a Unicode input bus and a UI-driving harness for apps with no API of their own. None of it needs anything the vendor doesn't already ship, and all of it inherits the rig's two hardest lessons: control and video are separate channels, so "looks right in the mirror" is never proof a phone obeyed, and the API has no authentication, so anything built here is only as safe as the LAN it runs on.

Fleet-wide synchronized actuation

Mirrored tap/swipe loops: the mechanical core of a rack

NATIVE API · WITH CAVEATS

Every XP input call, `/mouse/click`, `/mouse/move`, `/mouse/swipe`, `/mouse/wheel`, `/mouse/reset`, `/key/sendkey`, accepts a comma-separated `id` list, and the board walks every listed phone in parallel. That is the primitive every other entry in this section builds on: normalize to a known screen, run a canned tap-swipe-wheel sequence, sample one phone to confirm progress. No emulator, no jailbreak, nothing installed on the phone.

HOW

1. `send_key (fn_key WIN+h)`, then `mouse_reset_pos` XP `GET /api/key/sendkey?id=<list>&fn_key=WIN+h` (send twice) then XP `GET /api/mouse/reset?id=<list>` — leave the app, return to home page one, then clear the silent stuck-click state before any tap is trusted.
2. `click` XP `GET /api/mouse/click?id=<list>&x=&y=[&button=][&time=]` — open the target app at one mirror-pixel coordinate shared by every phone on the list.
3. `click / swipe / mouse_wheel` XP `GET /api/mouse/swipe?id=<list>&direction=&len=0.9[&stepping=&step_sleep=&brake=&sx=&sy=&ex=&ey=]` and XP `GET /api/mouse/wheel?id=<list>&direction=&len=1..127&number=` — run the actual gesture loop, still one request per step for the whole set.
4. `get_device_screenshot` XP `GET /api/pic/screenshot?id=<one id>&binary=true` — sample a representative phone's raw BMP and threshold a mean absolute pixel difference against the idle floor; round-robin which phone you sample.
5. `find_multi_color` (optional gate) XP `POST /api/pic/find-multi-color` — branch on a colour-table match instead of a fixed delay; unverified on this rig, see the colour-detector card below.

NEEDS

- Per phone: AssistiveTouch on with tracking sensitivity maxed, Full Keyboard Access on, Auto-Lock Never, portrait lock, StandBy off — none of this is queryable over the API.
- Board bound and at `/config/usb/get` state 2, and `location_crc` non-empty in `/device/get` (shared library CRC B0DBD800 for iPhone11,8 on this rig), or every `/mouse` call returns code 11.
- Video up for verification (state non-0, live `air_pid`) — needs IMConsole.exe running, not just IMService.exe.

- Identical screen geometry and identical on-screen layout for any phone sharing a coordinate; the two XRs here have different home screens, so a shared tap opens a different app on each.

WATCH OUT

- A phone can silently stop honouring `/mouse/click` while `/mouse/move` and hardware keys keep working: measured 0.09 mean pixel diff on a click (idle floor 0.02-0.03) versus 141.96 after `/mouse/reset`. Moving and pressing Home prove nothing about taps.
- Phonefarm's documented swipe-jitter randomization does not exist on the XP kernel, no Phonefarm function name is even accepted (HTTP 500), and the XP swipe table has no randomization parameter at all: assume every gesture lands exactly where sent.
- A loop that never re-checks state will tap into a dialog, login prompt or rate-limit screen for hours. Gate on an observed state and stop on any unexpected screen, rather than trusting a fixed delay.
- Verification is single-id and sampled, not exhaustive, so a phone can drift into a stuck-click state or a wrong screen for many cycles before a sampled check happens to land on it.
- The slower of the two XRs on this rig delivers ~4.5 new AirPlay frames/s under motion against ~29 for its twin; any screen check on it must tolerate roughly 220 ms per new frame.
- `/mouse/wheel` 's latency and whether iOS honours a synthetic HID scroll on a touchscreen app are both unmeasured here; treat it as an unverified alternative to swipe, not an equivalent.

> TRAP **One reply code, no per-phone result**
>
> A mirrored call returns a single `data.code` and echoes the id list back. There is no per-phone success flag anywhere in the response, so a partial failure across the set is invisible until a screenshot or `error_push` says otherwise.

SCALE

Input is O(1) per action regardless of rack size: a 100 px move measured 78 ms whether it targeted one phone or two, and a full 1000 px move ~592-594 ms either way. Latency tracks pointer distance at roughly 0.6 ms/px, not fleet size. Verification is O(N): one raw-BMP poller per phone at up to 84 req/s each (ten concurrent pollers sustained 512 req/s aggregate at p50 3 ms), and input traffic measurably does not slow polling. A short-gesture loop could run on the order of 3,000-6,000 rack-wide gestures/hour if verification is sparse, dropping to roughly 500-1,500 verified actions/hour/phone once every cycle waits on the slow XR's ~4.5 fps ceiling.

Lockstep demo wall: one request drives every phone

NATIVE API · WITH CAVEATS

Apply the mirrored-input primitive to a showroom or retail fixture: a wall of identical phones scrolling the same feed or demoing the same app in visible unison, driven by one HTTP request per gesture instead of N. It is the single workload this API is unusually good at, and the clearest way to show a customer what the rack can do.

HOW

1. Batch hardware operation, once, optional `XP GET /api/config/imserver/set?thread_mode=true&mouse_mode=true` — one of the few live-apply config fields (no kernel restart needed); not probed on this rig, verify with `/api/config/imserver/get` first.
2. `get_device_list` `XP GET /api/device/get` — enumerate the wall and admit only phones with state non-0, non-empty vid/pid/uid, non-empty `location_crc`, and matching `imgw` / `imgh`.
3. `get_usb_list` `XP GET /api/config/usb/get` — confirm every board reads state 2 (enumerated and phone attached); 19 boards at VID 720A on this rig.

4. `send_key` XP `GET /api/key/sendkey?id=<list>&fn_key=WIN+h` (twice) then `mouse_reset_pos` XP `GET /api/mouse/reset?id=<list>` — normalize every phone to home page one and clear the stuck-click state across the whole set.
5. `click / shortcut#13` XP `GET /api/mouse/click?id=<list>&x=&y=` or XP `GET /api/shortcut/exec/url?id=<list>&url=` — launch the demo app on every phone at once.
6. `swipe / mouse_wheel` XP `GET /api/mouse/swipe?id=<list>&direction=up&len=0.9` and XP `GET /api/mouse/wheel?id=<list>&direction=down&len=30&number=3` — run the demo loop in lockstep.
7. `get_device_screenshot` XP `GET /api/pic/screenshot?id=<one id>&binary=true` — one request per phone, run pollers concurrently (measured 512 req/s aggregate across ten).
8. Callbacks XP `ws://<host>:9911/api → error_push, usb_change, dev_disconnect, dev_change` — the only asynchronous per-call failure signal there is.

NEEDS

- Every wall phone bound to its own board (state 2), identical mirror geometry (500x1080 on this rig), and identical app/home-screen layout across the set — the two XRs here already differ.
- The same per-phone iOS accessibility settings the mirrored-loop card lists, none of it API-visible.
- IMConsole.exe running and signed in; with only IMService.exe up, both phones read state 0 and the board list comes back empty.
- Only for the optional brightness/open-URL steps: the iMouse Shortcut installed and bound on each phone, unverified on this rig.

WATCH OUT

- Groups exist (`/device/group/set` , `/device/group/get-dev`) but no input endpoint accepts a group id: expand a group to a comma-separated id list client-side before every call.
- The documented brightness state range is "greater than 0 and less than 1" — `state=1` is out of range; use 0.99, and note this also contradicts the vendor's own rack checklist of keeping brightness lowest.
- Swipes are not instantaneous: a mirrored full-height swipe is ~580 ms of pointer walk before `/mouse/up` can follow, and velocity-dependent iOS gestures may not fire at that speed.
- Because `/pic/screenshot` is single-id, confirming N phones all advanced needs N reads; one phone's poller stalling (the slow XR, or one that dropped Wi-Fi) will make the wall look behind schedule when only the observation channel is lagging.
- Only the vendor console can bring the rack up in the first place; automation cannot bootstrap mirroring or board enumeration through the API alone.

> TRAP **Video bandwidth, not the kernel, is the real ceiling**
>
> Fan-out on the actuation side is free at any fleet size, but mirroring is a genuine Wi-Fi/AirPlay load per phone (roughly 5-20 Mbps depending on content), so 19 phones mirroring at once can need on the order of 100-380 Mbps just for video, well past what a typical access point sustains cleanly with that many clients. That, not the board, is the likely reason two identical XRs on this rig deliver such different frame rates.

SCALE

Actuation is fleet-size-independent, measured identically at 2, 10, 50, 100, 200, 400, 700 and 1000 px jumps for one phone and for two: a demo loop of short gestures can run roughly 2,000-5,000 lockstep actions/hour for the whole wall regardless of phone count, provided per-phone video bandwidth holds up, a real risk past roughly 10-15 simultaneously-mirroring phones on one access point. Verification scales per phone at up to 84 req/s each and does not bottleneck actuation. Beyond two phones this rig is untested.

Turning the rack into other hardware

Phones as network-addressable cameras with torch illumination

NEEDS SHORTCUT WITH CAVEATS

A rack of iPhones is also a rack of calibrated cameras with flashes, all addressable from one HTTP endpoint. Fan the torch on across phones, fire the shutter on every phone with a single mirrored `/mouse/click`, then pull full-resolution captures off one phone at a time. That makes synchronized multi-angle capture, time-lapse arrays and always-on visual sensors one-request operations, with the non-capturing phones' torches acting as controllable lighting for the one that shoots.

HOW

1. `send_key (WIN+h)`, `mouse_reset_pos` XP `GET /api/key/sendkey?id=<list>&fn_key=WIN+h` (twice), XP `GET /api/mouse/reset?id=<list>` — normalize and clear the pointer on every capture phone.
2. `shortcut#16` XP `/api/shortcut/switch/torch {id: <list>, state:1}` — turn on LED illumination for the whole rack in one request.
3. `click` XP `GET /api/mouse/click?id=<list>&x=&y=` — open Camera on every phone sharing that coordinate.
4. `click` (shutter) XP `GET /api/mouse/click?id=<list>&x=&y=`, unverified alternative XP `GET /api/key/sendkey?id=<list>&fn_key=VolumeUp` — fire the shutter; the coordinate tap is the documented, stable path.
5. `shortcut#17` XP `/api/shortcut/switch/torch {id: <list>, state:0}` — torch back off.
6. `shortcut#1` XP `/api/shortcut/album/get {id: <single>, album_name, num}` — list new captures on one phone; snapshot before capture and diff on `create_time`, since XP has no date filter.
7. `shortcut#2` XP `/api/shortcut/album/down {id: <single>, list:[{album_name,name,ext}], zip}` — transfer captures to the PC, into `<install dir>\Shortcut\Media`.
8. `shortcut#3` (optional) XP `/api/shortcut/album/del` or XP `/api/shortcut/album/clear` — irreversible; confirm with the operator before wiring to a button.
9. `get_device_screenshot` XP `GET /api/pic/screenshot?id=<single>&binary=true` — checks the viewfinder chrome only, never the sensor image.

NEEDS

- iMouse Shortcut installed, IP-edited, run once with every permission, bound to an AssistiveTouch custom key on every phone — torch and album transfer both ride this channel.
- Camera permission granted and the app left in Photo mode; mode is sticky and there is no API to read or set it.
- Board bound and at state 2 for the shutter tap; mirroring only needed if viewfinder confirmation is wanted.
- Per-phone coordinate map for the Camera icon and shutter, identical across phones for the fan-out to work.

WATCH OUT

- Opening Camera almost always releases any torch already lit on that same phone, because iOS gives the foreground camera session exclusive ownership of the flash unit. Treat "light this phone with its own torch, then shoot with it" as not possible; design lighting from idle phones onto the one that's shooting.
- `fn_key=VolumeUp` as a shutter is unverified: the hotkey exists and returns code 0, but whether iOS treats a HID volume key the way it treats a physical button is undocumented here.
- The mirror shows the Camera app's UI, not the sensor feed; the real image exists only after `album/down`, and retrieval, not capture, is the bottleneck by two to three orders of magnitude.
- Camera UI shifts a few pixels between iOS point releases and isn't queryable, so a coordinate map calibrated on 18.7.10 isn't guaranteed to survive a silent update on one rack phone.
- `album/get` and `album/down` both need an exact name+ext pair per file, and `album/get`'s own list is the only source of those names, so the pipeline must serialize get-then-down per phone.
- No API signal exists for iOS's thermal throttling of the flash; a hot, enclosed rack can silently produce dim or absent illumination with a code-0 success reply.

SCALE

Capture fans out cleanly (torch, click and hotkey all take comma-list ids, measured identical board time for 1 and 2 phones), so synchronized N-phone capture completes in well under a second. Retrieval does not fan out at all: `album/get` and `album/down` are single-id, so pulling frames is N sequential Shortcut round trips, each bounded by `outtime` (milliseconds on XP, raise it well past the 15 s default for full-resolution media) and by that phone's own Wi-Fi link. A 20-phone capture-and-retrieve cycle should be budgeted in the 1-2 minute range serialized, or pipelined per phone for roughly 500-1,000 capture+download cycles/hour/phone.

Phone rack as a pixel wall and stage-light array

NEEDS SHORTCUT WITH CAVEATS

Push each phone to a full-screen solid-colour page served from a LAN host, drive brightness, and pulse the camera LEDs, each addressed to a comma-separated id list so a whole colour region updates in one request. Device groups act as the region map, expanded client-side into id lists. It turns a rack that already exists into a reusable pixel wall, countdown board, tally display or audience-light effect, and is the clearest demonstration that the Shortcut channel is a general-purpose actuator, not just a file-transfer helper.

HOW

1. `set_group` XP `GET /api/device/group/set?id=0&name=Region-A` — create one group per physical region; `id=0` creates, any other id renames.
2. `set_dev` XP `GET /api/device/set?id=<list>&gid=&name=R2C4` — assign phones to regions and give them grid labels; comma-list id, but the vid+pid hardware binding stays single-device.
3. `shortcut#13` XP `GET /api/shortcut/exec/url?id=<region id list>&url=http://<host>/red` — paint a region by opening a colour page on every phone in it.
4. `shortcut#15` XP `GET /api/shortcut/switch/bril?id=<list>&state=0.99` — drive brightness for the region; no readback exists anywhere.
5. `shortcut#16 / #17` XP `GET /api/shortcut/switch/torch?id=<list>&state=1` then `state=0` — flash the camera LEDs across a region; invisible in the mirror.
6. `shortcut#13` once XP `GET /api/shortcut/exec/url?id=<all>&url=http://<host>/wall?slot=R2C4` — for animation, open one self-animating page and let it drive itself over its own WebSocket/SSE instead of re-issuing URLs per frame.

7. `get_device_screenshot` XP `GET /api/pic/screenshot?id=<single>&binary=true` — sample raw BMP pixels against the commanded colour; solid colour is the easy verification case.

NEEDS

- iMouse Shortcut installed, IP-edited, run once with all permissions, bound to an AssistiveTouch key on every phone — this entire use case is gated on it and it has never been probed on this rig.
- Phones unlocked with Auto-Lock Never so Safari stays foreground, and a LAN web host serving the colour/pattern pages.
- Phones physically arranged as a grid with positions recorded (name/hub/port), which the API does not expose on its own.

WATCH OUT

- Opening a URL launches Safari with its normal chrome (address bar, tab strip, home indicator) unless the page has already been added to that phone's Home Screen as a standalone web app, a step the API cannot perform. Without it every "pixel" tile shows a colour rectangle with visible Safari UI at the edges.
- No state readback exists for torch or brightness; the switch endpoints return `data.code` and nothing else.
- Frame rate for URL-swap animation is well under 1 fps (a Safari load plus a Shortcut round trip bounded by `outtime`); usable for cues and countdowns, useless for motion unless the page animates itself.
- No API call reports thermal state; iOS dims the display and disables the torch on its own when a phone gets hot, with nothing in the kernel to say so.
- Turning Wi-Fi or Airplane Mode on via the same `/shortcut/switch/*` family kills both the AirPlay picture and the Shortcut's own return path to the kernel — never mix them into a light cue.
- Even with Auto-Lock off, iOS can evict a backgrounded Safari tab under memory pressure, silently reloading the page and losing any client-side animation state with no API-visible signal.

> TRAP **One integration test gates all three effects**
>
> Screen colour, brightness and torch all depend on the same per-phone Shortcut bound to a HID-triggered AssistiveTouch key via the vendor console's Send Binding Key feature, a mechanism never probed on this rig. If that binding doesn't reliably trigger the Shortcut, the whole use case, not just one step, drops to not achievable as configured. Confirm with a cheap round trip on `/shortcut/device/ip` before building anything on top.

SCALE

`exec/url`, `bril` and `torch` all accept a comma-separated id list, so one colour region of any size is one request, the limit is round-trip latency (bounded by `outtime`, default 15 s), not width. Torch pulses are fastest since nothing renders. For discrete cue changes, expect on the order of 12-30 changes/hour per region if polled conservatively, effectively unusable for anything resembling motion. Once a self-animating page is loaded, frame rate is decoupled from the iMouse API entirely and limited only by that phone's own Wi-Fi and rendering, the only path to a usable pixel wall rather than a slow cue board. Nothing about Shortcut fan-out has been measured beyond the two phones on this rig.

Driving real consumer apps end-to-end

iMessage and FaceTime automation by driving the UI

NEEDS SHORTCUT WITH CAVEATS

Blue-bubble iMessage and FaceTime have no on-device API any script can call. Driving the genuine app from a real Apple ID and SIM, with taps and hardware-keyboard input, is the only route to automating them on accounts you own, useful for end-to-end testing of message-dependent flows and reproducing customer-reported bugs on real hardware. ASCII bodies go straight through `/key/sendkey`; anything non-ASCII needs the clipboard bus below, because the XP kernel has no `send_text` function at all.

HOW

1. `send_key (WIN+h)`, `mouse_reset_pos` XP `/api/key/sendkey?id=<ids>&fn_key=WIN+h` (twice) then XP `/api/mouse/reset?id=<ids>` — normalize the phone before any coordinate is trusted.
2. `click` XP `/api/mouse/click?id=<ids>&x=&y=` — open Messages or FaceTime and the compose/recipient field; per-phone coordinates, the two XRs have different home screens.
3. `send_key` XP `/api/key/sendkey?id=<ids>&key=<ascii text>` — type an ASCII recipient or body directly; needs Full Keyboard Access on.
4. `shortcut#10` XP `/api/shortcut/clipboard/set {id, text, sleep, outtime}` — for non-ASCII/emoji, put the text on the phone clipboard as a POST JSON body, not a query string.
5. `send_key` XP `/api/key/sendkey?id=<ids>&fn_key=WIN+v` — paste into the focused field; XP's paste key, not Phonefarm's combo.
6. `click` XP `/api/mouse/click?id=<id>&x=&y=`, or `/key/sendkey?fn_key=ENTER` — tap Send, or the FaceTime call control; FaceTime needs no dedicated endpoint.
7. `get_device_screenshot` XP `/api/pic/screenshot?id=<id>&binary=true` — verify by mean absolute pixel diff over the bubble region, never MD5.
8. (Wanted, not available) `ocr / find_image` XP `/api/pic/ocr, /pic/find-text, /pic/find-image` — plug-in gated; on this rig `/api/pic/ocr` answers code 28, so reading the sent bubble text back has no server-side path here.

NEEDS

- Apple ID signed in and iMessage enabled on each phone; automate only accounts you own or administer, and only to recipients who have consented.
- iMouse Shortcut bound (clipboard path only), Full Keyboard Access on, AssistiveTouch on, Auto-Lock Never, portrait locked — none of this is queryable over the API.
- A per-phone coordinate map captured from that phone's own screenshots; the two XRs here have different home screens and wallpapers.
- The phone unlocked — nothing in the API can type a passcode.

WATCH OUT

- No server-side vision exists on this rig (`/pic/ocr` code 28); any adaptive step, like reading a contact suggestion or locating the Send arrow, must be client-side template matching on the raw BMP.
- Recipient-suggestion rows reorder by recency, so blind-tapping a suggestion can address the wrong person; prefer typing the full number or Apple ID and pressing Enter.
- Paste timing is a real failure mode: fire the paste hotkey too early and a stale or empty clipboard lands. Use the `sleep` parameter and read back with `/shortcut/clipboard/get` when correctness matters.

- The stuck-click trap applies at full force here too; send `/mouse/reset` to the target before any send sequence.
- Apple-side security challenges, a "verify it's you" prompt, a 2FA code, an account lock, look identical to a stuck UI to this pipeline, and the automation has no way to complete one (it cannot read an SMS OTP).
- A fixed bubble region for verification breaks as a thread scrolls and bubble heights change; diff the whole visible transcript area, not a small fixed box.
- Full Keyboard Access can silently drop after an OS update or restart with no API-visible flag; verify it once per session with a canary keypress rather than assuming it stayed on.

SCALE

Clipboard set and key send both take comma-list ids, so seeding N phones with the same body is one request each, but navigation is not shareable in practice: each conversation sits at a different scroll position and the two XRs have different layouts, so the tap sequence is effectively per phone. Screenshot verification is one poller per phone (measured 83-85 req/s each, 512 req/s aggregate across ten). Counting navigation, the ~0.6 ms/px pointer walk, clipboard settle time and frame-gated verification, a realistic ceiling is roughly 4-12 messages per minute per phone, an order of magnitude below any "per second" assumption, and further reduced by Apple's own anti-abuse throttling on the account, which this pipeline cannot see or negotiate.

AirDrop transfers between phones on the same rack

PARTIAL WITH CAVEATS

Move a photo or file phone-to-phone by driving the iOS share sheet and AirDrop picker on the sender and the Accept sheet on the receiver, coordinating two control sessions from one controller. AirDrop is peer-to-peer over BLE discovery and a direct Wi-Fi link, leaving no copy on the PC, which makes it the only way to test AirDrop-dependent app flows and move an asset between two phones without routing it through the controller's filesystem.

HOW

1. `send_key (WIN+h)`, `mouse_reset_pos` XP `/api/key/sendkey?id=<sender>,<receiver>&fn_key=WIN+h` (twice) then XP `/api/mouse/reset?id=<sender>,<receiver>` — bring both phones to a known state.
2. `send_key` XP `/api/key/sendkey?id=<receiver>&fn_key=ControlBar` — receiver opens Control Center; an XP-only hotkey, use it instead of an edge swipe.
3. `click` (long press) XP `/api/mouse/click?id=<receiver>&x=&y=&time=800` — long-press the connectivity tile and set AirDrop receiving; `time` is the XP long-press mechanism, no separate endpoint.
4. `click` XP `/api/mouse/click?id=<sender>&x=&y=` — sender opens the asset and taps Share.
5. `get_device_screenshot + find_image / ocr` XP `/api/pic/screenshot?id=<sender>&binary=true`, then client-side matching (`/pic/find-image`, `/pic/ocr` exist as XP paths but are plug-in gated here) — locate the receiver's AirDrop tile; this step decides the whole design.
6. `click` XP `/api/mouse/click?id=<sender>&x=&y=` — tap the located tile.
7. `get_device_screenshot then click` XP `/api/pic/screenshot?id=<receiver>&binary=true` in a poll loop, then XP `/api/mouse/click?id=<receiver>&x=&y=` — catch and accept the incoming sheet; if sender and receiver share one Apple ID, iOS auto-accepts and no sheet ever appears.
8. `shortcut#1` XP `/api/shortcut/album/get {id: <single>, album_name, num}` or XP `/api/shortcut/file/get` — confirm arrival by snapshotting the list before the transfer and diffing on `create_time`; XP has no date filter.

NEEDS

- Both phones close together, on Wi-Fi with Bluetooth on by hand — there is no Bluetooth toggle anywhere in the XP API.
- Both phones bound (state 2) and mirroring (state non-0, live `air_pid`) — both sheets are only observable through their own mirror.
- Per-phone, per-model coordinate maps for the connectivity tile, AirDrop sheet, share glyph and Accept button.
- Client-side template matching, or the OCR/OpenCV plug-in installed and connected, to locate the dynamic AirDrop target tile.

WATCH OUT

- The AirDrop target tile's position depends on which devices have been discovered and in what order; without matching or a plug-in this step is a coin flip, and a wrong tap sends the asset to the wrong device.
- AirDrop receiving set to Everyone reverts to Contacts Only after 10 minutes on this iOS version; a long batch has to re-arm the receiver mid-run, and there is no API signal telling you it reverted.
- Only 2 phones on this 19-board rig are actually mirroring today, so on the current rig this is a single pair, not a fan-out scenario; the comma-list fan-out for setup steps is real but has nothing to fan out to yet.
- A realistic verified transfer runs 15-40 seconds once pointer-walk distance, the 800 ms long press (itself unmeasured, a guess pending per-iOS-version verification) and receiver-side confirmation polling are counted, not "several seconds."
- Reaching the AirDrop toggle needs the Control Center two-row icon layout so the connectivity tile sits at a queryable coordinate; without it the long-press coordinate is invalid.

SCALE

Inherently pairwise and sequential. Setup fans out (`ControlBar`, `WIN+h`, `/mouse/reset` all take comma-list ids), but the transfer itself needs a screenshot-decide-tap loop on both sides, and `/pic/screenshot` is single-id, so N pairs means 2N pollers, not the bottleneck since ten concurrent pollers measured 512 req/s. On the present rig's one possible pair, expect roughly 90-240 verified transfers/hour, bound by UI navigation and verification latency. For bulk staging, route around AirDrop entirely: `/shortcut/album/upload` and `/shortcut/file/upload` both take a comma-list id and push the same file to N phones in one request.

Filling API gaps: text and vision

Phone clipboard as a bidirectional Unicode data bus

NEEDS SHORTCUT · WITH CAVEATS

Use the phone clipboard as a two-way channel: push arbitrary Unicode, emoji or URLs to many phones in one request with `/shortcut/clipboard/set`, paste with `WIN+v` mirrored to the same list, and harvest what a phone copied with `/shortcut/clipboard/get`. It is the only documented way to get non-ASCII text onto an iPhone on this rig, since `/key/sendkey` is explicitly English/digits/ASCII-punctuation only and Phonefarm's `send_text` does not exist on the XP kernel.

HOW

1. `shortcut#24` `XP` `GET /api/shortcut/device/ip?id=<id>&state=0` — prove the Shortcut path works at all before trusting anything else; not yet run on this rig.
2. `shortcut#10` `XP` `/api/shortcut/clipboard/set {id: <list>, text, sleep, outtime}` — push text to the whole target set in one request, as POST JSON UTF-8, not a query string.

3. `click / send_key` XP `/api/mouse/reset?id=<ids>` then XP `/api/mouse/click?id=<ids>&x=&y=` (or `fn_key=OPENKeyboard`) — focus a text field; reset first, the stuck-click trap applies.
4. `send_key` XP `GET /api/key/sendkey?id=<list>&fn_key=WIN+v` — paste, mirrored to the same set; XP's own paste key, not Phonefarm's combo.
5. `shortcut#11` XP `GET /api/shortcut/clipboard/get?id=<id>` — harvest what a phone copied; single id only, so harvesting a rack is N requests.
6. `get_device_screenshot` XP `GET /api/pic/screenshot?id=<id>&binary=true` — verify the paste landed by mean absolute pixel diff; no OCR available here to assert the text directly.
7. `shortcut#10` then `shortcut#11` XP `/api/shortcut/clipboard/set?id=<id>&text=imouse-<timestamp>` then XP `/api/shortcut/clipboard/get?id=<id>` — a state-independent round-trip self-test, run once per phone before relying on paste.
8. `shortcut#13` XP `GET /api/shortcut/exec/url?id=<list>&url=` — adjacent use of the same channel for deep links.

NEEDS

- iMouse Shortcut installed, IP-edited, run once with every permission, bound to an AssistiveTouch key — not verified on this rig, the probe in step 1 never ran.
- Full Keyboard Access on with 1 s auto-hide, AssistiveTouch on, phone unlocked with Auto-Lock Never.
- Phone reachable to the kernel on TCP 9911, since the Shortcut posts its result back over the LAN.

WATCH OUT

- This rig's own probe already found `/api/shortcut/get` returns 404 inside the same `/shortcut/*` namespace the whole chain depends on. Treat the design as unverified-and-suspect, not just unverified, until the IP round trip actually returns data.
- A phone whose Shortcut binding is missing does not fail fast; it returns code 30 (call timeout) after the full 15 s `outtime`.
- Asymmetric fan-out: `clipboard/set`, `exec/url` and the paste hotkey all take comma-list ids, but `clipboard/get` is single-id only. Seeding is O(1) requests, harvesting is O(N).
- `outtime` is milliseconds on XP while `sleep` on `clipboard/set` is seconds; getting this backwards produces instant timeouts or 15-second stalls.
- Since iOS 16, a cross-app paste can raise an "Allow Paste" system sheet that is invisible to the API and blocks the paste until dismissed; a verification step must distinguish that from "paste hasn't landed yet," since both look the same in a pixel diff unless the sheet is captured in the crop.
- The clipboard is global on the phone and clobbers whatever a human operator had copied; a scripted read can also return stale data if something else wrote the clipboard between set and get.
- Displaying a QR via a rendered page is supported; reading one back from a neighbour's camera is not, there is no camera API and no OCR here, so treat that half of any QR-relay idea as not achievable on this rig.

> TIP **Round-trip self-test before you rely on it**
>
> Push a timestamped string and read it straight back per phone (`clipboard/set` then `clipboard/get`) before wiring the clipboard bus into any real flow. It is the only state-independent proof that Unicode survives the Shortcut path end to end, and it costs two requests per phone.

SCALE

Seeding the rack is two requests regardless of phone count (comma-list `clipboard/set` then comma-list paste), each fan-out request bounded by the slowest phone's Shortcut round trip, so send per-group rather than rack-wide if any phone is unhealthy. Harvesting is one request per phone since `clipboard/get` rejects multiple ids. A single push-paste-verify cycle chains a POST, a phone-side Shortcuts run (roughly 0.5-2 s including app-switch overhead), the paste hotkey, and a frame-gated verify, call it 1-3 s per phone end to end, or roughly 20-40 cycles per phone per hour run conservatively, well below the kernel's own 512 req/s ceiling because the bottleneck is iOS-side Shortcut execution and permission UI, not the API.

find-multi-color as a plug-in-free state detector

PARTIAL WITH CAVEATS

Gate automation on multi-point colour matches instead of OCR: sample a handful of pixels that uniquely identify a button or screen state, pass them as a colour table to `/pic/find-multi-color`, and branch on whether a match comes back with a centre to tap. No template image and no text recognition needed, but whether it runs without the vendor plug-in is unverified on this rig, and the docs' own placement of it argues against optimism.

HOW

1. `get_device_screenshot` XP `GET /api/pic/screenshot?id=<single>&binary=true[&rect=]` — grab a raw reference frame and read the pixels to key on; the BMP is bottom-up, flip y before converting an offset to mirror-pixel coordinates.
2. `find_multi_color first_color + offset_color + dir` (client-side) — build a colour table of 5-10 anchor pixels; XP's `first_color` format is a comma-separated list of absolute points `x|y|RRGGBB-tolerance`, there is no `offset_color` or `dir` on XP, those are Phonefarm-only.
3. `find_multi_color / find_multi_color_ex` XP `POST /api/pic/find-multi-color {id: <single>, list: [{first_color, rect, similarity}], same, all}` — run the search; returns `data.list[{index, centre}]` on code 0.
4. `click` XP `GET /api/mouse/click?id=<id or list>&x=centre[0]&y=centre[1]` — branch: tap the returned centre directly, no coordinate conversion needed.
5. (no XP equivalent to `loop_device_screenshot`) — repeat `/pic/find-multi-color` on a fixed period to implement "wait until this state appears"; XP has no server-push frame stream.

NEEDS

- Probe first, with a negative control: call once with a known-present table, then invert one anchor and call again. A code-28 reply on either call means it is plug-in-gated like OCR and this approach is unavailable until the plug-in is installed.
- Device online with a live `air_pid`, since the search runs against the kernel's internal mirror frame.
- Colour tables rebuilt whenever `air_ratio` or the device's `imgw` / `imgh` changes, since tables are resolution-specific, and fixed display settings (True Tone off, Night Shift off, constant brightness).

WATCH OUT

- The XP documentation does not describe this as kernel-internal or plug-in-free: it sits on the same "Picture Text Recognition" page as `/pic/ocr` and `/pic/find-image`, both plug-in-gated, and carries the same Shortcut-fallback footnote those carry. Given OCR already returns code 28 on this exact rig, the realistic default assumption is that this is gated too, not merely unknown.
- Single-id like every `/pic/*` endpoint, so state checks never fan out; detecting once on a "representative" phone is only safe for truly identical UI, like a system dialog, not app content, given the two XRs here

already have different home screens.

- Brittle compared with real OCR: a theme change, dark mode, a badge count or a localisation change silently breaks the table, and the failure mode is a confident wrong branch, not an error.
- The vendor's own example tolerance of 000000 will not hold against a live AirPlay stream; expect to tune tolerance and similarity per element.
- The pointer itself is visible in the frame as a small grey dot and moves with in-flight `/mouse/move` calls; anchors placed near where the pointer idles can pick up the pointer as a false match.
- If gated, the client-side fallback, matching the same colour table against the raw BMP you already fetch at 2 ms, works today with no server dependency; plan around it from day one rather than treating the server call as the default path.

> TRAP **Silent misdetection, not a clean failure**
>
> A colour table that matches the wrong element, or a control test that was never run, produces an automation that taps confidently in the wrong place, and on a mirrored fan-out it does so on every phone at once. Fail closed on codes 28, 29, 33 or 36 rather than falling through to the tap, and re-verify after any app update.

SCALE

Per phone, not fleet-wide, and cheap relative to any external vision path if it works at all: one call replaces a 1.6 MB frame transfer plus a round trip to a matcher, but its own cost is unmeasured on this rig. In the likely case it is gated, throughput collapses to the client-side path, raw BMP grabs sustain 84 req/s/phone flat out, with the real cost shifting to client-side pixel comparison time, which still needs benchmarking before quoting a number. Either way, a sane production polling loop runs at 1-5 Hz rather than the theoretical ceiling, since most polls see an unchanged frame.

Also worth knowing

Counting and enumeration without OCR: same/all as a quantity primitive

PARTIAL WITH CAVEATS

The state-detector framing above treats `find-multi-color` as a boolean: does this one pattern appear or not. The `same` and `all` flags turn it, and `find-image-cv`, into a counter and an indexer instead. `same` asks the search to return every occurrence of a match, not just the first; `all` asks it to evaluate every entry in the colour-table or template list, not stop at the first hit. Together they answer "how many" and "which one" for anything with a repeatable visual signature: unread badges, loaded grid tiles, toggled rows, the third of N identical icons, all without a coordinate map and without OCR.

HOW

1. `get_device_screenshot` XP `GET /api/pic/screenshot?id=<single>&binary=true&rect=` — pull a scoped reference frame over just the region holding the repeated items, to build the colour table or crop the template image from.
2. `find_multi_color` XP `POST /api/pic/find-multi-color {id: <single>, list:[{first_color, rect, similarity}], same:true, all:false}` — count every on-screen occurrence of one anchor pattern; returns `data.list[{index, centre}]`, one entry per match, length of the list is the count.
3. `find_image_cv` XP `POST /api/pic/find-image-cv {id: <single>, img_list:[<template>], same:true, all:false, rect, similarity}` — same idea with a template crop instead of a colour table; returns `data.lis`

`t[{index, centre, rect, similarity}]`, which also sizes each match via `rect`.

4. Either function, `all:true` with N distinct list entries (one colour table or template per control) — checks every entry instead of stopping at the first, so a list of per-row anchors comes back as exactly the rows currently in the "on" state.
5. Client-side, sort `data.list` by `centre` (row-major: y ascending, then x) before treating any index as an ordinal — the doc does not document the scan order the match list comes back in.
6. `click` `XP GET /api/mouse/click?id=<single>&x=<centre[0]>&y=<centre[1]>` — tap the Nth sorted centre directly; no coordinate map, no separate lookup step.

NEEDS

- The same plug-in this rig has not confirmed: `/pic/ocr` answers code 28 here, `find-image` and `find-multi-color` sit on the same doc page and are "assume the same" per the rig notes. Run the negative-control probe from the state-detector card before trusting any count.
- A tightly scoped `rect` or anchor pixel set per item, since a loosely bounded pattern can match chrome elsewhere on screen and inflate the count.
- For `find-image-cv`, a template as base64 or a local path resolved on the PC running IMConsole, not on the phone; a template built from step 1's screenshot has to be saved to a path the kernel host can read, or embedded as base64 inline.
- Device online with a live `air_pid`, same as any `/pic/*` call.

WATCH OUT

- Both endpoints are single-id, "multiple is not allowed" per the doc, same as every `/pic/*` function: counting is per phone, not fleet-wide. Enumerating badges across 19 boards is 19 sequential calls, each returning its own list.
- `find-multi-color`'s match entries carry only `index` and `centre`, nothing for size or confidence; `find-image-cv` additionally returns `rect` and `similarity` per hit, useful for confirming every matched tile is actually the same size before trusting the count.
- An off count is a quieter bug than a flipped boolean: a badge template that also half-matches a status-bar glyph, or a tolerance set too loose, silently returns one extra centre with no error and nothing distinguishes it from a real hit.
- No dedup or overlap rule is documented for adjacent or overlapping matches; a large tolerance value against a busy background can plausibly double-count a single item, unverified either way on this rig.
- If the plug-in turns out gated, this specific capability, an indexed list of all matches, has no equivalent client-side fallback: hand-rolled pixel diffing against the raw BMP can answer a yes/no state check but not cleanly enumerate and index N separate on-screen instances the way `data.list` does.

SCALE

Per phone, same as any `/pic/*` call: no fan-out, so counting across the rack is N sequential requests, one per phone, each returning its own indexed list in one round trip rather than one request per item. Cost scales with the number of distinct patterns searched under `all:true`, not with how many phones share the same layout. Whether the underlying search is plug-in-gated on this rig is unverified, so no throughput number for the server-side path exists yet; if gated, this specific quantitative capability is unavailable, not merely slower, since the client-side pixel-diff fallback proves state but does not enumerate.

SECTION 11 · ARCHITECTURE AND INTEGRATION

Building on it: architecture and integration

Everything so far has been about single calls. This section is about the shape of the program you wrap around them. The XP kernel is a local HTTP and WebSocket server on `127.0.0.1:9911` with no authentication, no outbound notification of any kind, no job queue, no locking, and, on this rig, no server-side recognition at all. That means the interesting architecture lives in your process, not in the vendor's: a scheduler that serializes the right resource, a socket that turns broadcast callbacks into events, a bridge that carries those events to a human, a perception step that substitutes for the missing plug-ins, and, at the far end, an on-phone Shortcut channel that can reach parts of iOS the API never touches.

KERNEL	LIVE WORKERS	FRAME COST	NO CONTENTION
9911	**2 of 19**	**2 ms**	**512 req/s**
HTTP and WebSocket, no auth, all interfaces	19 boards enumerated, 2 phones mirroring	raw BMP, 1,620,054 bytes, p50	ten concurrent pollers, p50 3 ms

The two things worth building first

A scheduler and an event socket. Between them they answer the two questions every later component asks: which phone is safe to drive right now, and what just changed. Both are pure XP, need no plug-in, and need nothing installed on the phone.

Per-phone job queue with board-serialized leases

NATIVE API · VERIFIED

Model the rack as N independent workers, one per phone and board pair, and hold exactly one lease per board `uid` so two jobs never interleave HID traffic on the same pointer. Jobs are declarative step lists, tap, drag, wheel, key, wait-for-screen-change, compiled to XP calls on 9911, with every coordinate expressed in that device's `imgw` x `imgh` mirror pixel space and every step carrying a mean-pixel-difference verification rule. The measurements say the kernel itself has no shared bottleneck, so the board is the only resource that must be serialized. Getting that boundary right is the difference between a rack that runs many jobs concurrently and one that mysteriously drops taps because two scripts fought over one pointer.

HOW

1. `get_device_list` XP `GET /api/device/get` — inventory state, `imgw` / `imgh`, `uid`, `vid` / `pid`, `gid`, `location_crc`, `air_pid`. Probed live here: two iPhone XR, imgw 500 x imgh 1080, width 414 x height 896, location_crc B0DBD800.
2. `get_usb_list` XP `GET /api/config/usb/get` — control-channel readiness per board, `data.list[{vid,pid,uid,ver,state}]`. Eligibility is the two-channel AND: device state != 0 with a live `air_pid`, board state 2, `location_crc` non-empty.
3. `none` XP `none (client-side)` — acquire an exclusive lease keyed by board `uid` (C241BFBD, C214BEBD here), not by `deviceid`. No API-side locking exists in either edition.
4. `mouse_reset_pos` XP `GET /api/mouse/reset?id=<ids>` — clear the silent stuck-click state once at lease acquisition, not once per step.

5. `click` XP `GET /api/mouse/click?id=<ids>&x=&y=&button=1&time=<ms>` — tap at an absolute mirror pixel, or long-press via `time`. XP's `button` is numeric, Phonefarm's is the string "left"/"right".
6. `swipe` / `mouse_wheel` XP `GET /api/mouse/swipe?id=&direction=up&len=0.9 | GET /api/mouse/wheel?id=&direction=up&len=1-127&number=` — flick or scroll. XP adds left/right to the wheel, which Phonefarm does not have.
7. `mouse_down` / `mouse_move` / `mouse_up` XP `/api/mouse/down → /api/mouse/move?x=&y= → /api/mouse/up` — arbitrary-path drag. Coalescing intermediate points to the newest position is free: 14 steps, 7 steps and 2 jumps produced the same 26.2 / 22.9 mean pixel change.
8. `send_key` XP `GET /api/key/sendkey?id=<ids>&key=<ascii> or &fn_key=WIN+h` — ASCII only on `key`; `fn_key` for the documented hotkey table, and the two are mutually exclusive.
9. `get_device_screenshot` XP `GET /api/pic/screenshot?id=<one id>&binary=true` — capture before and after. Single id, "Multiple is not allowed". XP defaults to BMP, the opposite of Phonefarm's JPEG default.
10. `none` XP `none (client-side)` — decide "did it land" on mean absolute pixel difference, never on MD5 or byte equality.
11. `none` XP `GET /api/device/get + GET /api/config/usb/get` — release the lease, and re-prime from these two GETs after a connection refusal. There is no session or heartbeat endpoint on the HTTP side.

NEEDS

- IMConsole.exe running and logged in. Measured: with only IMService.exe up, `/device/get` showed both phones state 0 and `/config/usb/get` returned an empty board list. Automation cannot bring the rack up through the API alone from that state.
- Every phone bound to its board with `/device/set?id=<deviceid>&vid=720A&pid=<pid>`. `vid` and `pid` must be set together, and hardware binding is single-device only, while `name`, `gid` and `location_crc` accept comma lists.
- Non-empty `location_crc` per phone (shared library B0DBD800 here), or every `/mouse` call returns `data.code` 11, "No mouse parameters".
- The iOS accessibility checklist on every phone: AssistiveTouch on with tracking sensitivity max, Full Keyboard Access on, Trackpad & Mouse tracking speed max, Auto-Lock Never, portrait lock. None of it is queryable over the API.
- Job coordinates authored per device in that device's own `imgw` x `imgh` space, read from its `/device/get` record, not a global constant and not iOS points.
- No plug-ins. This design touches only `/device/*`, `/config/usb/get`, `/mouse/*`, `/key/*` and `/pic/screenshot`, all of which answered on this rig.

WATCH OUT

- `status` 200 only means the request was accepted. The result is in `data.code`: 0 success, 2 not bound to hardware, 3 hardware not connected, 11 no mouse parameters, 25 offline, 26 screenshot failed, 36 parameter error. Match on the code, never on the message, which is Chinese on this build.
- The lease is advisory only. The API authenticates nobody and accepts commands from every LAN client, so another console or script can drive the same board while your lease is held. Treat a verification mismatch with no corresponding queued action as evidence of a lease collision and log it distinctly.
- A "screen changed" assertion can pass on the wrong screen. The two XRs here have different home screens and wallpapers, so the same coordinate is Clock on one and Reminders on the other. Normalize with two Home presses, the first leaves the app and the second returns to page one, and confirm the layouts match before replaying a job across phones.

- If the jobs are identical across phones this design is the wrong tool. Every mouse and key endpoint takes a comma-separated id list and the board walks all listed phones in parallel, so a mirrored action is one request, not N.
- Correction to the capacity claim: board state 2 only proves a phone is USB-attached. A usable worker also needs live mirroring and a non-empty `location_crc`, which on this rig is 2 phones, not 19. The other slots need someone to seat a phone, pair it with `/device/set`, and walk the full non-queryable accessibility checklist by hand, minutes per phone, none of it provisionable by the scheduler.
- Correction to the reset assumption: `/mouse/reset` is confirmed to clear the stuck-click state, but it is unverified whether it also corrects a drifted or offset pointer. The queue has no detection step that distinguishes "taps are landing progressively off-target" from "clicks stopped registering", because both currently rely on the same box diff. Also note that reset returns the pointer to the origin, so the first action after it pays a full-distance pointer walk.
- Verification latency is per phone, not per rack. The healthy XR delivers roughly 29 new frames/s under motion while its twin delivers roughly 4.5, so a wait-for-change step on the slow phone needs a 220 ms per frame budget and must not time out on a 30 fps assumption. A still screen produces almost no new frames, so most polls legitimately return an identical frame.
- Report worker count as a live computed value, not as the static 19-board inventory number. 19 is hub topology capacity.
- Automating third-party apps at scale can breach those apps' terms and can get the accounts or devices involved restricted. Confine this to accounts and apps you own or are authorized to operate, and stop a job when an app signals a block rather than retrying through it.

SCALE

Workers equal the number of eligible phones, currently 2. Per worker a tap-verify cycle is about 2 ms capture twice, plus 0 to 650 ms of pointer walk at the measured 0.6 ms/px, plus one to seven frames of settle (35 ms/frame on the fast XR, about 220 ms on the slow one). The fast phone sustains roughly 450 to 1,200 ten-step jobs per hour, about 4,500 to 12,000 discrete actions; the slow phone roughly 240 to 450 jobs per hour, because settle time dominates. Jobs run fully in parallel, so the rig total today is on the order of 700 to 1,650 ten-step jobs per hour. Quote capacity per phone class, never as an average: throughput here is bimodal. Cross-phone scaling is linear in principle, ten concurrent pollers measured 512 req/s at p50 3 ms with worst p95 142 ms and input traffic did not interfere with frame polling in either direction, but it is gated by the manual per-phone setup, not by the kernel. Reuse one 1.6 MB buffer per poller: without reuse the frame pump allocated 488 MB/s and forced 496 gen-2 collections in 8 s, with reuse 2.9 MB/s and 0 to 1.

TRAP **Your lock is a suggestion**

There is no locking, session, or ownership concept anywhere in either edition. A lease protects your own processes from each other and nothing else. On an open subnet the correct mental model is that the rack has no owner, which is why the firewall rule on 9911 and 17000 is part of the architecture rather than a hardening afterthought.

Kernel WebSocket callbacks as the rack's event bus

NATIVE API WITH CAVEATS

Hold one WebSocket to `ws://127.0.0.1:9911/api` and treat the kernel's broadcast callbacks as first-class triggers. `dev_connect` and `dev_disconnect` are video-channel health, `usb_change` codes 19/20/12 are control-channel health, and `error_push` is the only asynchronous signal that an input call was refused at the board

after the kernel already returned 200. Structural changes arrive push-style for the whole rack on one socket, so frame polling is reserved for when you actually need pixels. This converts a polled rack into a reactive one.

HOW

1. `ws://host:9911/clinet` XP `ws://127.0.0.1:9911/api` — open the event socket. The Phonefarm "/clinet" path does not exist here and 9912 is closed. There is no subscribe call: the kernel broadcasts every event to every connected client.
2. `get_airplaysrvnum → ver` XP `im_connect callback → data.ver` — read the kernel version on connect. This is the only source of it on XP, there is no HTTP version endpoint. Use it to gate features such as the `Home` fn_key, which needs kernel 2026.2.27 or later.
3. `dev_disconnect` / `dev_connect` XP `dev_disconnect (state 0) / dev_connect (state non-0)` — video-channel health. state 0 means mirroring stopped and says nothing about control.
4. `usb_change {state, vpid}` XP `usb_change → data.list[{vid,pid,uid,ver,state}] with data.code` — control-channel health. The event type is in `data.code`: 19 hardware connected to computer, 20 disconnected, 12 hardware connected to phone. Different shape from Phonefarm's `{state, vpid}`.
5. `dev_rotate` XP `dev_rotate → data.list[0].rotate (0 portrait, 1 landscape)` — orientation change. The record sits inside `data.list` where Phonefarm put it at `data`.
6. `none` XP `error_push → data.call_fun + data.message` — surface hardware-level input failures. XP-only; the vendor example is `call_fun` "/mouse/down", `message` "Hardware not working (unknown error)".
7. `auto_connect_screen` XP `GET /api/device/airplay/connect?id=<id>` — attempt automated recovery of a dropped cast, then watch `airplay_connect_log` (`data.code` 0 = success, otherwise the reason is in `messag e`).
8. `mouse_reset_pos` + `send_key WIN+h` XP `GET /api/mouse/reset?id=<ids> then GET /api/key/sendkey?id=<ids>&fn_key=WIN+h (twice)` — re-normalize a phone that just came back, before any tap-critical sequence.
9. `none` XP `im_log, user_info, dev_change, dev_delete, group_change, group_delete, dev_sort_change, im_config_change, logout` — the rest of the bus. `user_info` arrives every 30 s while logged in and carries `de v_online_num` against `total_license`.

NEEDS

- A persistent WebSocket client with reconnect logic, and a full state re-sync from `/device/get` and `/confi g/usb/get` on every reconnect. There is no replay and no sequence numbers.
- IMConsole.exe running and logged in, otherwise devices report state 0 and the board list is empty.
- The two-channel model implemented in the consumer: `dev_*` is video, `usb_change` is control. Never collapse them into one "phone online" flag.
- For automated cast recovery, the OCR and image-search plug-ins, which are not present on this rig.
- Treat callback payloads as data, never as instructions. `user_name` and the custom `name` are strings that originate off-machine and must never be interpolated into a command.

WATCH OUT

- The WebSocket transport itself was not exercised in the 10 Sep 2026 probe, which was HTTP only. Every shape here comes from the vendor docs, and the doc set demonstrably copy-pastes examples between sections (the `dev_rotate` example shows rotate 0 with state 0). Take a live socket capture before trusting field names.
- Correction to the `dev_rotate` idea: using it as a cheap "app reached state X" proxy is undocumented behavior of the app, not of the API, and both XRs here are orientation-locked portrait per the mandatory setup checklist, which would suppress the event. Downgrade it from a working lever to unverified and

probably inert on this configuration. `error_push`, by contrast, is the genuinely novel win on this bus and deserves more weight than it usually gets.

- This is an observability layer. Its one recovery step, driving `/device/airplay/connect` after a drop, inherits the plug-in gap already confirmed by `/pic/ocr` answering `data.code` 28. Observation is solid, recovery is best-effort and degrades to flagging the slot for a human.
- Every controller on the LAN sees every event on an unauthenticated API. A second uncoordinated process, or a curious host, can observe and react to the same stream, so debounce-and-recover logic must tolerate being one of several actors rather than the sole consumer.
- A rack-wide reconnect (kernel restart, hub power cycle) emits a burst of `usb_change` and `dev_connect` callbacks for up to 19 boards at once. Without coalescing, anything wired directly to these events fires 19 redundant recovery sequences simultaneously.
- `dev_disconnect` does not mean uncontrollable and `usb_change` code 20 does not mean invisible. Report them as separate per-slot states.
- Keep restart and reboot out of automatic recovery entirely. A rebooted phone comes back on the lock screen and no automation here can type a passcode.
- Never parse or display `ver` / `uver` as a version: it is mojibake on this build.

SCALE

This is the cheap half of the rack's observability. One socket carries every structural event for every device, so cost does not grow with phone count the way polling does: expect O(1) events per structural change plus one `user_info` broadcast every 30 s per connected client, negligible next to the 512 req/s measured across ten concurrent frame pollers. It composes with frame polling rather than competing with it, measured 98.8 req/s with input streaming against 78.7 without. Its value is latency, not volume: it replaces roughly one request per phone per interval of health polling with near-instant push notice. The one scenario needing consumer-side rate limiting is a full-rack reconnect storm, dozens of events within a few seconds. Filter by `fun` before persisting, or a naive log grows steadily on `user_info` alone.

Bridging the kernel to things it cannot reach

The kernel calls nothing out. A grep of all nine XP docs and the Phonefarm doc for webhook, notify, push URL and 回调地址 returns zero hits in either edition. It also cannot see anything on this rig beyond raw pixels, because the recognition plug-ins are not connected. Both gaps are filled the same way, by a process of yours sitting next to the kernel.

Alert bridge: kernel callbacks to Slack and email pages

PLUS EXTERNAL TOOLING · WITH CAVEATS

A small out-of-process daemon holds one WebSocket and turns the kernel's callbacks into human pages. It splits every event along the two-channel model, enriches each with the device record's `name`, `gname`, `ip` and `uid` from a cached `/device/get` plus the hub and port slot label from Windows PnP, debounces and rolls up per phone, runs a self-healing rung and reports its outcome, then posts outbound. A page that reads "Hub#0010-P02 (Slot-01, 192.168.4.68) lost CONTROL, usb_change code 20; USB restart 720A|C241 attempted, board did not return to state 2; cable suspect" is what turns a phone farm into something you can leave running overnight.

HOW

1. `WebSocket ws://{host}:9911/clinet` XP `ws://127.0.0.1:9911/api` — one socket, kept open. On connect XP pushes `im_connect` with `data.ver`, for example "2024.12.21.910".

2. `§3 server-initiated callbacks, {fun, data, message}` XP `envelope {fun, status:200, message, msgid:0, data:{...}}` — understand the delivery model first. No subscribe or filter call exists, so the daemon and IMConsole.exe coexist without contending. `status` is always 200; meaning lives in `data.code` and `data.list[]`.
3. `get_device_list` XP `GET /api/device/get → data.list[]` — cache the device inventory for alert enrichment, refreshed on the `dev_change` callback that any client's `/device/set` triggers.
4. `get_usb_list` XP `GET /api/config/usb/get → data.list[{vid,pid,uid,ver,state}]` — cache the board inventory for the control-channel side. Verified live: 19 boards, VID 720A, states 1 and 2.
5. `dev_connect` / `dev_disconnect` XP `record inside data.list[], state non-0 / 0` — classify video-channel events. Page them as "VIDEO down", never as "phone dead".
6. `usb_change {state, vpid}` XP `usb_change, data.list[] with the event type in data.code` — classify control-channel events, codes 19, 20 and 12.
7. `none` XP `error_push, data.call_fun + data.message` — catch input-path failures at the board, the signal nothing else gives you. No Phonefarm equivalent exists.
8. `none` XP `im_log (data.message) and user_info (user_state, total_license, dev_num, dev_online_num, overdue_time)` — catch license, login and kernel notices. The Phonefarm API has no user or account concept at all.
9. `n/a` XP `dev_change, dev_delete, group_change, group_delete, dev_sort_change, im_config_change, collection_mouse, airplay_connect_log, restart_log` — suppress planned and cosmetic churn instead of paging on it, correlated against your own outstanding jobs.
10. `mouse_reset_pos / restart_usb / discon_airplay + auto_connect_screen` XP `GET /api/mouse/reset?id=<ids> ; /api/device/usb/restart?id=720A|C241 ; /api/device/airplay/disconnect then /api/device/airplay/connect` — run the self-healing rung before paging and say which rung was tried. Note `usb/restart` takes VID|PID, not the deviceid and not the 8-hex uid. XP has no force flag on connect, so disconnect first.
11. `none` XP `none, nothing in the XP doc set` — deliver the page. Slack incoming webhook or SMTP, done by your daemon, which is the whole reason this is external rather than native.
12. `none` XP `user_info, pushed automatically every 30 s while logged in` — heartbeat the bridge itself. A 90 s silence is a reasonable "socket dead" trigger, but only while logged in.

NEEDS

- IMService.exe reachable on 9911 and IMConsole.exe running and logged in.
- A WebSocket client that reconnects with backoff and re-snapshots `/device/get` and `/config/usb/get` on every reconnect, because callbacks missed while disconnected are not replayed.
- An outbound Slack incoming-webhook URL or an SMTP relay reachable from the rack PC, with its secret stored outside the repo.
- A slot table mapping `deviceid` to board `uid` to hub and port, built from Windows PnP. Three 7-port hubs, VID 1A40 PID 0201; the two live XRs sit on Hub #0010 ports 2 and 3. Physical topology is not in the API.
- A maintenance-window and job-correlation list, so the daemon suppresses disconnects it or an operator caused.
- Windows firewall rules restricting who may reach 9911 and 17000.

WATCH OUT

- Elevate the WebSocket check from caveat to blocking pre-flight. The 10 Sep 2026 probe was read-only HTTP, so if `ws://127.0.0.1:9911/api` behaves differently from the HTTP surface, none of the

classification logic can be trusted yet. Connect and confirm an `im_connect` frame with `data.ver` before building on it.

- The highest-value bug this daemon could ship with: alerting keyed on "code != 0 means alert" silently misses `error_push`, whose own vendor example carries `data.code` 0 for a genuine hardware error. Gate that one on the presence of the callback and on `call_fun` / `message`.
- Match on `data.code`, never on message text. This build returns 成功, 插件未连接, 未登录，不允许上线 and 接口不存在.
- `error_push`, `im_log`, `user_info` and `im_connect` are XP-only. On the Phonefarm edition the bridge degrades to `dev_connect`, `dev_disconnect`, `dev_rotate` and `usb_change`, with no input-failure and no license signal at all.
- `usb_change` has no documented code for a phone being unplugged from its board; 12 only covers attach. Detect it from `data.list[].state` dropping 2 to 1, or re-poll `/config/usb/get` on every `usb_change`. Note also that `state` appears in the vendor example but not in its field table, so the 1 and 2 meanings are measured, not documented.
- False rack-wide outage: if IMConsole.exe exits, every phone reads state 0 and the board list empties while IMService.exe is healthy. Detect that combination and page it as "console down", not as 19 board failures.
- The self-healing rung is an unauthenticated write path fired automatically off a broadcast the daemon did not cause. Two bridge instances, or the daemon plus a human at the console, can race on the same `/device/usb/restart` with no documented idempotency. Add a per-slot in-flight lock and cooldown before letting it fire automatically.
- Keep the Slack integration strictly outbound. A slash-command or email-reply path that sends commands back into the rack hands rack control to anyone who can post in the channel.
- Never include the contents of `C:\iMouseXP\IMService.json` in an alert: it holds the vendor-account password as reversible hex obfuscation. Alert bodies also leak topology, phone names, LAN IPs and MACs into a third-party service, so alias them if the channel is broadly visible.
- Attaching a screenshot works only for a phone whose video is still up, which by definition excludes a `dev_disconnect` page. If you want a video signal independent of the kernel, check ESTABLISHED sockets on TCP 17000 per phone IP.
- A bridge that pages on every `dev_disconnect` without maintenance-window suppression trains its audience to ignore it. That is the real failure mode.

SCALE

One WebSocket per kernel regardless of phone count, since the kernel broadcasts rather than being polled: 2 phones or 100, it is the same socket, and it does not contend with the vendor console. Payloads are small JSON; the 1.62 MB screenshot path is not involved unless you deliberately attach an image, and `user_info` every 30 s doubles as a free keepalive. The load driver is alert volume, not kernel load. A flapping board can emit a `usb_change` every few hundred milliseconds, so the debounce and rollup window, roughly 30 s, is the actual rate limiter, capping pages at about 120 per hour per slot before rollup and at whatever the target Slack app's own webhook limit allows in aggregate. Measure that limit against your workspace rather than assuming it.

Vision-model computer-use loop: raw frame in, tap out

PLUS EXTERNAL TOOLING · WITH CAVEATS

Close a full see-think-act loop against a physical iPhone: pull the raw mirror frame as a bottom-up BMP, downscale it, hand it to an external multimodal model with the task and the allowed primitive set, take back one action in a normalized 0 to 1000 grid, map that grid to this device's `imgw` x `imgh` (500 x 1080 here, not the 414

x 896 logical points), and execute it. The external model substitutes for the OCR and image-search plug-ins that are not installed, so the loop reads arbitrary UI instead of matching fixed coordinates, which matters here because the two XRs have entirely different home screens. Perception costs 2 ms and input traffic does not slow frame polling, so the only expensive part of the loop is the model call. The phone sees nothing but a HID pointer and keyboard on the Lightning port.

HOW

1. `get_device_list` XP `GET /api/device/get?id=<deviceid>` — preflight the record and learn the coordinate space. Use `imgw` / `imgh` only; `location_crc` must be non-empty or every `/mouse/*` call returns code 11. Phonefarm's record has no `imgw` / `imgh` at all.
2. `get_usb_list` XP `GET /api/config/usb/get` — preflight the control channel. Video-ready and control-ready are independent; check both.
3. `send_key (fn_key WIN+h)` XP `GET /api/key/sendkey?id=<ids>&fn_key=WIN+h` — normalize the screen, twice. "Home" as an fn_key name also exists but needs kernel 2026.2.27 and targets iOS 26.1+; these XRs run iOS 18.7.10, so WIN+h is the right call.
4. `mouse_reset_pos` XP `GET /api/mouse/reset?id=<comma list>` — clear the stuck-click state before any tap-critical step. A responsive keyboard proves nothing about taps.
5. `get_device_screenshot` XP `GET /api/pic/screenshot?id=<id>&binary=true` — perceive. The body is the image bytes with no JSON envelope: 1,620,054 bytes = 500 x 1080 x 3 plus a 54-byte bottom-up header, p50 2 ms, against 40 ms and a 21 req/s cap for `jpg=true` and 26 ms for the base64 JSON default.
6. `none` XP `none` — think. Decode and row-flip the BMP, downscale, prompt with the task plus the primitive list, and ask for coordinates in a normalized 0 to 1000 grid. Map back with `x*imgw/1000`, `y*imgh/1000` and clamp.
7. `click` XP `GET /api/mouse/click?id=<ids>&x=&y=&button=1&time=<ms>` — act, tap. `time` is the press-to-release delay, so `time=800` is a long press.
8. `swipe` / `mouse_wheel` XP `GET /api/mouse/swipe?id=&direction=up&len=0.9 | GET /api/mouse/wheel?id=&direction=up&len=30&number=3` — act, scroll. A 1080 px stroke is about 580 ms of pointer walk and iOS may read it as a drag, so prefer the 500 px short axis or the wheel, which scrolls without moving the pointer.
9. `send_key` XP `GET /api/key/sendkey?id=<ids>&key=<ascii>` — act, type ASCII. English letters, digits and English punctuation only, no CJK and no emoji.
10. `send_text / shortcut#10 + paste` XP `POST /api/shortcut/clipboard/set {id, text, sleep} then GET /api/key/sendkey?id=&fn_key=WIN+v` — act, type Unicode. XP has no `send_text` and no `send_text_ok` callback, so this two-call path is the documented route. `sleep` is in seconds while `outtime` is in milliseconds.
11. `shortcut#13` XP `POST /api/shortcut/exec/url {id, url, outtime}` — optionally jump straight into an app screen, skipping UI navigation for anything with a URL scheme.
12. `get_device_screenshot` XP `GET /api/pic/screenshot?id=<id>&binary=true (optionally rect=[l,t,r,b])` — verify, then loop. Threshold a mean absolute pixel difference: idle floor 0.02 to 0.03, a real reaction 20 to 160, a landed pointer dot inside a 40 px box 1.4 to 9.8.
13. `none` XP `ws://127.0.0.1:9911/api → error_push` — listen for board-level failures the HTTP reply hides.

NEEDS

- An external multimodal model endpoint plus network egress from the rack PC or a bridge host. There is no on-rig recognition: `/api/pic/ocr` returned `data.code` 28, 插件未连接, on 10 Sep 2026, and the image-search plug-in is presumed absent too.
- The kernel on 9911 with IMConsole.exe running and logged in.

- Per phone: state non-0 with a live `air_pid`, board state 2 with `vid` / `pid` / `uid` bound, and a non-empty `location_crc`.
- The full iOS accessibility checklist on every phone, none of it readable over the API.
- A per-device coordinate mapper from the model's output space into that device's `imgw` x `imgh`, with clamping.
- The iMouse Shortcut installed, IP-edited and bound only if the loop needs Unicode text or `/shortcut/exec/url`. Taps and ASCII typing need no Shortcut.

WATCH OUT

- Correction, and it is a hard limit on the action space: there is exactly one HID pointer per phone and no multi-touch. Pinch to zoom, two-finger scroll and any simultaneous multi-point input are categorically impossible on this rig, not merely slow. The primitive list handed to the model must exclude them, because a hallucinated multi-touch instruction is not a retryable failure, it is a request the hardware cannot service.
- Correction to the downscale step: at 250 x 540 a standard 44 pt iOS touch target is about 25 px, and denser UI, list rows, keyboard keys, small icon grids, falls below what a vision model can reliably discriminate. Expect a materially higher misclick rate on dense screens, and note that clamping to the frame does not fix a coordinate that is confidently wrong rather than out of bounds.
- The perception step is entirely off-rig. With no OCR and no image search there is no slower-but-working fallback: a transient model outage stops the loop dead rather than degrading.
- `/pic/screenshot` is single id, so one poller per phone; only the act step fans out over a comma list. Fan-out is safe only across phones with identical layout and identical `imgw` / `imgh`, since the same coordinate is Clock on one XR and Reminders on the other.
- `binary=true` returns raw bytes with no envelope, so error codes 25 (offline) and 26 (screenshot failed) arrive as a JSON body where an image was expected. Sniff the "BM" magic and handle both shapes.
- Each frame is 1.6 MB, over the .NET Large Object Heap threshold. Without buffer reuse a poller measured 488 MB/s allocation and 496 gen-2 GCs per 8 s; with one reused buffer per poller, 2.9 MB/s and 0 to 1.
- Use a fixed poll period. Deriving it from observed frame gaps decayed a 23 fps stream to 16.5 fps through positive feedback.
- Phonefarm's `loop_device_screenshot` server-push stream does not exist in XP. There are no binary WebSocket frames, so poll.
- The pointer-box verification trick assumes AssistiveTouch's on-screen pointer dot is enabled and visible on every target phone. It is one more setting the API cannot query, and its absence makes every "did the tap land" check read as a false negative even when the tap worked.
- Sending frames of signed-in sessions to a third-party model is a data-egress decision each deployment makes explicitly, not an implementation detail, and it is easy to lose track of once the loop is wrapped in tooling. Frames and model transcripts are also a retention decision.
- Operating consumer apps with an autonomous agent conflicts with most platforms' terms of service, and the account-level consequence lands on accounts you own. There is no sandbox: a hallucinated coordinate acts on a live phone, so clamp, verify with a pixel diff before chaining anything destructive, and cap the step count. Never let the loop type credentials or complete a purchase; put those behind a human.

SCALE

Perception is per phone and cheap: p50 2 ms paced, 84 req/s flat out per phone, and phones are independent (two pollers 83 and 85 req/s at p50 3 ms each; ten pollers 512 req/s aggregate, worst p95 142 ms). Input and frame traffic do not contend, measured `/mouse/move` p50 0.5 ms while both phones were polled flat out. Per loop step: about 2 ms capture, a few ms downscale, 1 to 5 s of model latency, 0 to 600 ms of pointer walk at 0.6

ms/px, and a settle wait bounded by the phone's own source rate, about 35 ms on the .68 XR and about 220 ms on the .27 XR. That is 2 to 7 s per action per phone, roughly 500 to 1,800 actions per hour per phone as a kernel-and-hardware ceiling. Across the rack it scales close to linearly, but at 19 phones the bottleneck is entirely the model provider's concurrency and cost, not IMService or the boards.

Reaching into iOS through the Shortcut channel

Every `/shortcut/*` endpoint works the same way underneath: the board sends a HID key combination, iOS runs the Shortcut bound to that AssistiveTouch custom button, and the Shortcut talks back to the kernel over Wi-Fi. Naming that mechanism is useful, because it is the only path on this rig to structured, non-pixel data from inside iOS. It is also where the documented API ends, and the three cards below map that edge, from what the vendor ships to what is merely plausible.

Hotkey-to-Shortcut as an extension point, and its ceiling

PARTIAL WITH CAVEATS

The vendor half of this mechanism is fully supported and covers album, file, clipboard, URL, brightness, torch, airplane, cellular, WLAN, shutdown and restart, plus LAN and external IP, with comma-separated id fan-out. The tempting extrapolation is that anything Shortcuts can reach is therefore triggerable from the PC. The honest finding is narrower: the XP API exposes no way to bind, name, list or invoke a custom Shortcut, the binding is a console-only manual step, and XP's `fn_key` vocabulary is a closed documented list with no arbitrary-combination entry. Extending the rack means driving UI with the pointer or getting a custom Shortcut onto an AssistiveTouch button by hand.

HOW

1. `§5.1 setup` XP `none (no API)` — phone-side permissions, once, by hand. Settings > Shortcuts > Advanced: allow running scripts, allow sharing large amounts of data, allow deleting without confirmation. Documented only in the Phonefarm doc; the XP docs assume it.
2. `§5.2 setup` XP `none (no API)` — install the vendor iMouse Shortcut: console QR code, get the Shortcut, edit the kernel IP inside it (192.168.4.29 here), run once, allow everything. If the PC's LAN IP changes, it must be edited again on every phone.
3. `§5.3 setup` XP `none (no API)` — bind the trigger. Accessibility > AssistiveTouch > Devices > "iMouse4.x" > Customize More Buttons, then "Send Binding Key" in the console. This is the only documented binding step in either edition, and no endpoint performs it.
4. `shortcut (id 1–24 dispatcher)` XP `/shortcut/album/get|upload|down|del|clear, /shortcut/file/get|upload|down|del, /shortcut/clipboard/set|get, /shortcut/exec/url, /shortcut/switch/device|bril|torch|flight|cdpd|wlan, /shortcut/device/ip` — trigger the documented vendor functions. XP has no generic dispatcher: each is its own path with typed parameters and `outtime` in milliseconds, where Phonefarm's `outtime` was seconds and its parameter a double-encoded JSON string.
5. `click (button left/right only)` XP `GET /api/mouse/click?id=<ids>&button=<4..8>` — candidate route A to a custom Shortcut, a spare pointer button. The API side is documented; whether iOS routes an extra button to a Shortcut assigned under Customize More Buttons appears in neither the XP docs nor the vendor setup pages.
6. `send_key fn_key (CTRL+ALT+SHIFT+WIN+v)` XP `GET /api/key/sendkey?id=<ids>&fn_key=<name>` — candidate route B, an arbitrary combination. Refuted as written: the XP table is a closed vocabulary of named actions and the Phonefarm combo is absent from it. Nothing documents `fn_key` as a general combination encoding.

7. `shortcut#11` XP `GET /api/shortcut/clipboard/get?id=<id>&outtime=<ms>` — return path 1. Single id, returns `data.text`, and works regardless of what put the text on the clipboard, which makes it the cleanest confirmation channel available.
8. `send_text` XP `none` — return path 2, corrected: the kernel exposes no endpoint that accepts arbitrary posted data, so a custom Shortcut must call your own listener, not "the kernel's HTTP port".
9. `none (send_text_ok, Phonefarm only)` XP `none` — confirmation. `/key/sendkey` and `/mouse/click` return code 0 when the board accepted the input, which says nothing about whether a Shortcut ran. A vendor `/shortcut/*` call that never fires returns `data.code` 30, "Call timeout", after `outtime`.

NEEDS

- The iMouse Shortcut installed on each phone with the kernel's LAN IP edited in, run once with every permission granted, and bound to an AssistiveTouch custom button. None of this has an API.
- The board bound to the phone with `vid` and `pid` set together, and physically connected at board state 2. The trigger travels over USB HID, so a phone that is mirroring but unbound cannot be triggered at all.
- Full Keyboard Access on for any `fn_key` path; phone unlocked, portrait, Auto-Lock Never.
- Phone and kernel on the same LAN with the phone able to reach 9911, since the Shortcut's return leg is Wi-Fi and that port is unauthenticated.
- iOS 15+ for `/shortcut/file/*` and `/shortcut/switch/device`; iMouse Shortcut v1.3 or higher for the `/shortcut/other/*` recognition fallbacks.
- For a custom Shortcut: a spare trigger that does not displace the vendor binding, plus your own HTTP listener if it needs to report back.

WATCH OUT

- Correction to the framing: every documented working path here is the vendor's own pre-bound Shortcut occupying the single available trigger button. Reaching a genuinely custom Shortcut has not been verified end to end anywhere. Route A is an untested hypothesis about iOS behavior and route B is refuted by the `fn_key` table, so the only end-to-end-confirmed capability is triggering the vendor's own functions.
- Only one trigger binding is documented and the vendor's Shortcut occupies it. Rebinding it would plausibly break every `/shortcut/*` endpoint on that phone at once, so try it only on a phone you can fully re-provision, never on one in active use.
- Nothing in the `/shortcut/*` family has been probed on this rig; the live probe covered `/device/get`, `/config/usb/get`, `/device/group/get` and `/pic/ocr` only. Whether the iMouse Shortcut is even installed on either XR is unknown. The cheapest single proof is `GET /api/shortcut/device/ip?id=<id>&state=0` returning code 0 with `data.text` equal to the `ip` field in `/device/get`, and it should be run before anything else in this group.
- The binding step has no API and must be repeated by hand on every phone added, and redone after a board or phone swap. That is O(phones) manual work that does not compose with the comma-separated fan-out model the rest of the rack uses.
- Trigger fan-out is one request for many phones, but every return channel, `clipboard/get`, `device/ip`, `album/get`, `file/get`, is single id.
- Turning WLAN or airplane mode on via `/shortcut/switch/wlan|flight` kills the AirPlay video channel and the Shortcut's own return path while leaving USB control alive, so the call may time out even when it succeeded.
- Return path 2 means standing up another inbound service on the same LAN that already carries an unauthenticated kernel. Scope and access-control it independently rather than treating it as a free extension of an already open design.

- Only 2 of the 19 boards currently carry a mirroring, bound phone, so even the fully supported vendor half is exercisable on 2 phones today.
- `/shortcut/album/clear` and `/shortcut/file/del` are destructive with no undo, on a phone where "delete without confirmation" is explicitly enabled. Do not wire them to anything automatic.

SCALE

Triggering is cheap and parallel: `/key/sendkey`, `/mouse/click` and the set, switch, upload and exec paths all take a comma-separated id list, and the board walks every listed phone at the same cost as one. Reading back is serial: the single-id paths are each bounded by `outtime` (default 15000 ms) plus the phone's actual execution time, so confirmed round trips run roughly 1 to 16 s each and must be run one connection per phone to parallelize. Across a fully provisioned 19-phone rack that is on the order of a few hundred to low thousands of confirmed trigger-and-read cycles per hour, run concurrently. A phone also has only a handful of AssistiveTouch custom buttons, so a large Shortcut library has to be multiplexed behind one dispatcher Shortcut that reads its instruction from the clipboard, not one button each. For the custom-Shortcut idea no throughput number can be given in good faith: none of the required binding steps has been exercised on this hardware.

OPEN **One cheap call decides this whole group**

`GET /api/shortcut/device/ip?id=<id>&state=0` returning `data.code` 0 with `data.text` equal to that device's `ip` proves the vendor Shortcut is installed, the trigger fires, and the phone can reach the kernel. A `data.code` 30, "Call timeout", after 15 seconds means the chain is broken somewhere you cannot see. Run it on one XR before designing anything on top of the next two cards.

Phone-initiated reverse channel to your own gateway

NEEDS SHORTCUT WITH CAVEATS

The vendor's Unicode-text and album features work only because iOS runs a Shortcut on the phone and that Shortcut talks back over Wi-Fi. The same architecture can carry your own payload: author a second Shortcut that gathers what the API is blind to, battery level, Wi-Fi SSID, free storage, whether an app is installed, App Intents output from apps that publish actions, and have it POST JSON to an endpoint you run or write to the clipboard where `/shortcut/clipboard/get` reads it back. This matters because the API has no visibility into battery, thermal state, SSID or storage, yet those explain the rig's real failure modes: one XR delivers about 29 new frames/s while its twin delivers about 4.5, which is a Wi-Fi or AirPlay problem rather than a kernel one, and a phone that stops mirroring may simply be flat. On an OCR-less rig it is also the only structured-data path: results arrive as JSON rather than pixels you cannot read.

HOW

1. `get_device_list` XP `/device/get (optional id, comma list)` — enumerate targets and get correlation keys, `ip`, `user_name`, `mac`, `state`, `imgw` / `imgh`, so gateway posts can be joined to device records. Confirmed live: two iPhone XR at 192.168.4.68 and 192.168.4.27.
2. `shortcut#13 (Open URL)` XP `/shortcut/exec/url (id comma list, url, outtime ms)` — trigger path A, preferred and fully documented on the XP side. `shortcuts://run-shortcut?name=<name>` is an Apple URL scheme that the vendor doc neither documents nor forbids, so it must be probed on one phone.
3. `send_key (fn_key)` XP `/key/sendkey (id comma list, fn_key, key left blank)` — trigger path B, the AssistiveTouch chord. The endpoint and parameter are verified, but the published `fn_key` table is closed, so an arbitrary user chord has no documented syntax.

4. `none (Shortcut "Get Contents of URL")` XP `none, external tooling` — return path A: the Shortcut POSTs its JSON to a listener you run. The network path is proven, since that is how the vendor Shortcut returns data, but the listener is yours to build and must not be co-hosted on 9911.
5. `shortcut#11 (Get phone clipboard)` XP `/shortcut/clipboard/get (id, multiple not allowed, outtime ms)` — return path B, no listener needed. Returns `data.text`; a `data.code` 30 means the Shortcut never ran.
6. `shortcut#24 (external IP)` XP `/shortcut/device/ip (id single only, state 0 LAN / 1 external, outtime ms)` — the cheapest end-to-end proof that the Shortcut channel is alive, before you debug your own Shortcut. state=1 is a legitimate egress check on infrastructure you own.
7. `get_device_screenshot` XP `/pic/screenshot (id single only, binary=true)` — visual confirmation the first time, because a stuck iOS "Allow" prompt is invisible to every other call. You must read the prompt yourself: `/pic/ocr` answers `data.code` 28, 插件未连接, here.

NEEDS

- The vendor iMouse Shortcut installed on each phone with the kernel IP edited in, run once with every permission allowed, and bound via the console's "Send Binding Key". Neither HANDOFF.md nor HARDWARE.md records this being done on either XR, so treat it as unproven.
- Your own Shortcut authored on each phone, and for trigger path B bound to a spare chord under AssistiveTouch > Devices > Customize More Buttons.
- Board bound and connected at state 2, since both trigger paths travel over the board. Full Keyboard Access on, rack-wide, for any hotkey path.
- Phone unlocked, Auto-Lock Never, and on the same LAN as the kernel.
- An HTTP endpoint you control for return path A, with a per-phone token, since the LAN carrying this traffic also carries the entirely unauthenticated 9911 API.
- iMouse Shortcut v1.3 or later if you also want the `/shortcut/other/*` recognition fallbacks.

WATCH OUT

- Correction: nothing on this rig exercises `/shortcut/exec/url`, `/shortcut/clipboard/get`, `/shortcut/clipboard/set` or `/shortcut/device/ip`. Every measured number here is for `/pic/screenshot`, `/mouse/*`, `/key/sendkey` and `/device/get`, so treat the whole chain as unverified in practice, not just trigger path B.
- Correction to the verdict: because every step depends on the single unverified vendor-Shortcut install, the realistic status today is "blocked on a manual one-time setup step per phone" rather than "achievable". That install is a single point of failure for the entire use case.
- Correction: the first time `shortcuts://run-shortcut` runs on a phone it will very plausibly raise an iOS confirmation banner. Treat that as an expected one-time step per phone, cleared with a screenshot and a manual tap, since no API call can dismiss a system dialog.
- No rig evidence exists for what `/shortcut/exec/url` or `/key/sendkey` with an unrecognized `fn_key` actually returns, whether silently ignored, an HTTP error, or a 500 like the Phonefarm function names. Probe that before building retry or alerting logic around it.
- Return path B is single id, as are `/pic/screenshot` and `/shortcut/device/ip`. Only the trigger fans out. The measured lack of a shared bottleneck, 512 req/s across ten pollers, says to issue those 19 reads concurrently rather than serially.
- The Phonefarm sibling marks its whole shortcut dispatcher professional-version-only. The XP docs state no edition gate, but licensing here is per device through `/config/user/info` and has not been checked against shortcut use.

- A user Shortcut runs in the foreground with the phone owner's full app permissions and may show a banner, so it can interrupt whatever job is running, with no API signal that it did.
- Trigger failure and the stuck-click failure are independent: a phone mid-way through a stuck-click episode tells you nothing about whether its Shortcut trigger will fire, and `/mouse/reset` does nothing for a Shortcut trigger failure. Check both.
- App Intents coverage varies enormously by app and version. An action present in one app's Shortcuts pane may simply not exist in another.
- Firing a chord by mistake against a comma-separated id list runs it on every phone in the list, and two chords in the documented table are destructive (restart, pass-through toggle). A Shortcut that POSTs is a data path off the phone with the phone owner's permissions: scope it to facts you need, give each phone its own token, and point it only at an endpoint from your own configuration. Reading account or order data through App Intents is legitimate only on accounts and devices you own, and the app's own terms govern automated use.

SCALE

The trigger is O(1) per report cycle regardless of fleet size, one comma-separated fan-out request, and the board lease is held only for the keystroke or URL hand-off. Zero screenshot and zero OCR cost. A 19-phone sweep is therefore 1 trigger request plus up to 19 single-id return requests, which the measured concurrency ceiling says can be issued in parallel, so a full-rack report cycle likely completes in 15 to 20 s bounded by the slowest phone's Shortcut execution plus `outtime`, not by the API. Because each run interrupts the foreground app and may show a banner, a 5 to 15 minute cadence is the right ceiling, roughly 4 to 12 cycles per hour per phone, far below the API's raw capacity.

Fire your own Shortcut from a spare mouse button

NEEDS SHORTCUT WITH CAVEATS

The XP mouse endpoints expose buttons 1 to 8, not just left and right: `/mouse/click`, `/mouse/down` and `/mouse/up` all document "1 left button, 2 right button, 3 middle button, 4, 5, 6, 7, 8 button". iOS AssistiveTouch lets any button on a connected pointer device be taught an action, and the action list includes running a Shortcut. That is the same channel the vendor's own text-entry feature uses, except the vendor teaches a key combination rather than a mouse button. So: author a Shortcut on the phone, teach iOS that button 5 on the board runs it, and fire it from the PC with one down-and-up pair carrying a comma-separated id list. Buttons 3 to 8 are the only input primitives on this board with no pre-assigned iOS meaning, which makes them the natural place to hang a custom trigger, and unlike the vendor's Unicode path this needs neither plug-in nor a `send_text` endpoint that XP does not have.

HOW

1. `none (phone-side; the vendor equivalent is the §5.3 "Send Binding Key" console button)` XP `none` — one-time per phone: author the Shortcut, then Settings > Accessibility > Touch > AssistiveTouch > Devices > the board entry > Customize Additional Buttons, press the button when iOS asks, pick the Shortcut. No endpoint in either edition reads or writes iOS accessory-button mappings.
2. `mouse_move, click` XP `/api/mouse/move?id=<id>&x=&y= then /api/mouse/click?id=<id>&button=1&x=&y=, then /api/mouse/down?id=<id>&button=5 + /api/mouse/up?id=<id>&button=5` — optionally drive the teaching screen remotely, and use it as the test: if iOS never registers the press, the board does not transmit that button and the idea stops here.
3. `get_device_list, get_usb_list` XP `GET /api/device/get?id=<ids> and GET /api/config/usb/get` — pre-flight the control channel. Require state != 0 with a live `air_pid`, `vid` / `pid` / `uid` non-empty, `location_cr c` non-empty, and the board at state 2.

4. `mouse_reset_pos` XP `GET /api/mouse/reset?id=<id1>,<id2>,...` — clear the silent stuck-click state on the whole target set. Whether that state also swallows down and up on button 5 was never measured, so reset anyway.
5. `mouse_down + mouse_up` XP `GET /api/mouse/down?id=<id1>,<id2>&button=5 then GET /api/mouse/up?id=<id1>,<id2>&button=5` — fire on every phone in one round trip. Neither call takes x or y, so the pointer never moves and there is no 0.6 ms/px walk and no risk of a stray tap. Prefer this over `/mouse/click`, which requires coordinates.
6. `get_device_screenshot` XP `GET /api/pic/screenshot?id=<id>&binary=true` — confirm it visibly ran. Compare by mean absolute pixel difference, idle floor 0.02 to 0.03 against a real reaction of 20 to 160.
7. `shortcut#11 (professional version only on Phonefarm)` XP `GET /api/shortcut/clipboard/get?id=<id>&outtime=15000` — read a result back, route A: the Shortcut ends with "Copy to Clipboard". Catch: this endpoint is served by the vendor's own Shortcut, so it needs a second, separate binding alongside your button 5.
8. `none` XP `none (the Shortcut's "Get Contents of URL", aimed at a collector you run)` — route B. Point it at your own endpoint; the kernel has no "store a result for me" path, and `/shortcut/clipboard/set` writes to a phone.

NEEDS

- Board bound and cabled per phone, showing state 2, with mouse parameters present (`location_crc`, the B0DBD800 shared library on this rig).
- AssistiveTouch on with tracking sensitivity max, Trackpad & Mouse tracking speed max, Auto-Lock Never, portrait lock. None of this is queryable: a phone can mirror perfectly and ignore every input.
- iOS 13.4+ for pointer-device support and button customization, with the Shortcuts app present. The vendor's own shortcut features want iOS 15+, and the file endpoints are explicitly iOS 15+.
- The Shortcut authored on each phone and run once manually with every permission granted. An unanswered "Allow" prompt blocks the run silently and permanently.
- The unverified prerequisite: that this board actually transmits HID buttons 4 to 8 as distinct buttons and that iOS records one. The XP API documents 8 buttons, but the board's 9-byte vendor HID output report layout is an open unknown and nothing here has been exercised beyond button 1.
- For the clipboard readback only: the vendor iMouse Shortcut installed, IP-edited, and bound to its own separate trigger key.
- IMConsole.exe running and logged in.

WATCH OUT

- Correction, and it gates everything else: a 200 with `data.code` 0 from `/mouse/down?button=5` proves the HTTP layer accepted the parameter, not that a HID report with a nonzero button-5 bit left the PC. Test step 2 on one phone before treating any of this as achievable. If the board implements only left, right and middle, the mechanism collapses.
- The keyboard fallback is not a clean substitute on XP. The `fn_key` list is fixed, the Phonefarm "send Chinese" combo does not appear in it, no XP endpoint sends an arbitrary custom combination, and every combo XP does expose already carries an iOS meaning, so hanging your own Shortcut on one steals that function. That is what makes the spare button the better XP route rather than the exotic one.
- Correction to the framing: only 2 of the 19 boards currently have a phone attached and mirroring, so this is a 2-phone capability today. Each additional phone needs a fully manual AssistiveTouch binding that nothing in the API can set, verify or audit.

- Nothing about the binding is visible over the API on either edition. Your only evidence a phone is armed is a screen diff after firing, so budget one screenshot round trip per phone and keep a per-phone checklist. The mapping survives nothing but the phone itself: a restore, an iOS update or a cable swap to a different board can lose it.
- Silent failure modes cluster. An unanswered permission prompt, a Shortcut waiting on a dialog, a locked screen and a phone whose AssistiveTouch was turned off all look identical from the PC, because `data.code` 0 only reports that the button event reached the board.
- Verification cannot use MD5 or byte equality: the AirPlay encoder never repeats a frame. Use mean absolute pixel difference against an idle control, and remember a still screen produces almost no new frames at all.
- There is no operator-facing way to tell which physical board maps to which iOS button assignment once bound. LED color semantics are unconfirmed and `ver` / `uver` is known-unreliable mojibake, so a board swapped after a cable fault will silently fire the wrong phone's Shortcut, or none, with identical code-0 responses either way.
- A phone doing both the vendor text path and this one needs two independent bindings, the spare button for your Shortcut plus whatever binds the vendor Shortcut for clipboard reads. The AssistiveTouch button budget is not unlimited.
- A stray press of the mapped button later runs the automation unintentionally, including from any other host on the LAN, since the API authenticates nobody. Pick a button you will not use otherwise, firewall 9911, and do not bind anything destructive. A Shortcut that reaches the network is an egress path off the phone independent of the vendor stack.
- Anything the Shortcut drives inside a third-party app is governed by that app's terms. Use only accounts you own, keep to actions the app publicly exposes as Shortcuts actions, and stop on a rate limit or a block response rather than retrying into it.
- Edition split: Phonefarm's click, mouse_down and mouse_up document only "left" and "right", so buttons 3 to 8 are an XP-edition capability, and Phonefarm's generic shortcut dispatcher, including clipboard read, is professional-version only.

SCALE

Firing is one round trip regardless of fleet size: down and up both accept a comma-separated id list, the board walks all listed phones in parallel, and because neither carries coordinates there is no pointer-walk cost at all, against up to about 592 ms for a 1000 px walk on `/mouse/click`. Two requests, a few milliseconds, N phones. Everything after that is per phone: `/pic/screenshot` is single id at p50 2 ms, and `/shortcut/clipboard/get` is single id with a 15 s default `outtime`. The real ceiling is the Shortcut's own run time, roughly 3 s end to end for the vendor's comparable Shortcut and slower for anything touching Photos, Files or a third-party app, plus a confirmation bounded by the phone's AirPlay source rate, about 29 new frames/s on one XR here and about 4.5 on its twin. On the 2 live phones, budget on the order of 15 to 25 fire-and-confirm cycles per hour per phone. It does not extrapolate to 19, because the core HID-button assumption is untested at any N.

Addressing: groups, fan-out and coordinate spaces

Before anything else in a console, two questions have to have hard answers: which phones does this step go to, and what coordinate space is the step written in. The XP kernel answers the first with groups plus comma-separated id lists, which the Phonefarm edition does not have at all. It answers the second only through `imgw` and `imgh` on the device record, and it will never tell you that you got it wrong.

Group targeting and comma-id fan-out as the addressing layer

NATIVE API WITH CAVEATS

Use kernel groups as the rack's addressing layer, one per client, per campaign, per model, plus a quarantine group, and drive each group through the comma-separated id list that every `/mouse/*`, every `/key/*` and most `/shortcut/*` endpoints accept. One HTTP request steps the whole group and the boards walk their pointers at the same time. Capture cannot follow the same pattern, because `/pic/screenshot` refuses multiple ids, so verification is N concurrent raw-BMP polls with a mean pixel difference check, and any phone that reads no reaction drops out of the next step's id list.

Measured on this rack, a mirrored move to two phones costs exactly what a move to one costs (2 px = 1.0 ms, 1000 px = 592 ms, identical either way), and frame polling does not contend with input. That makes one input request per step plus N parallel capture verifications the cheapest correct batch architecture the kernel offers.

HOW

1. `set_group` XP `/device/group/set` — create the group with `id=0` (0 means add), read the new group id out of `data.list[{id,name}]`; a `group_change` callback goes to every connected client
2. `set_dev` XP `/device/set` — assign members with `id=<id1,id2,...>&gid=<gid>`; only the hardware binding is single-device, since pid and hardware id must be set together, and the response `data.list` names the ids that actually took
3. `none` XP `/device/group/get-dev` — resolve members (`id=0` lists ungrouped), keep records with `state != 0` and `imgw` / `imgh` / `rotate` equal to the master's, then join on `uid` against `/config/usb/get` and keep only boards at state 2
4. `mouse_reset_pos` XP `/mouse/reset` — reset the whole target set whenever its membership changes and before any tap-critical sequence; comma id list supported
5. `send_key` XP `/key/sendkey` — normalize with `fn_key=WIN+h` twice, the first press leaving the app and the second returning to page one
6. `click` / `swipe` / `mouse_wheel` XP `/mouse/click, /mouse/swipe, /mouse/wheel` — step the group; `button` is numeric on XP (1 left, 2 right, 3 middle, up to 8), `time=<ms>` makes a long press, swipe `len` is a fraction of the screen, and wheel scrolls without moving the pointer at all
7. `get_device_screenshot` XP `/pic/screenshot` — verify per phone, one poller each, `binary=true`, mean absolute pixel difference against the pre-step frame
8. `send_key` / `send_text` XP `/key/sendkey, /shortcut/clipboard/set` — type ASCII with `key=`, or push Unicode to the clipboard and paste it with `fn_key=WIN+v`
9. `del_group` XP `/device/group/del` — tear down, but move members back to `gid 0` first, because a populated group returns `data.code 34`

NEEDS

- Every member bound to its board (`/device/set` with vid and pid together, one device per binding call) and that board reading state 2 in `/config/usb/get`.
- Every member carrying a non-empty `location_crc`, or mouse calls return `data.code 11`, no mouse parameters.
- Members sharing `imgw` x `imgh` and `rotate`, and sharing the UI layout being targeted: same page, same app version, same icon at that coordinate.
- Phone-side accessibility satisfied on every member (AssistiveTouch, Full Keyboard Access for hotkeys, tracking speed at maximum, Auto-Lock Never, portrait lock). None of it is queryable over the API.
- For the Unicode branch only, the iMouse Shortcut installed on each phone, the kernel IP edited in, bound to an AssistiveTouch custom key, and the phone able to reach 9911 on the LAN.

- IMConsole.exe running and logged in. With IMService.exe alone the probe saw both phones at state 0 and an empty board list.

WATCH OUT

- A comma-list input call returns **one** `data.code`, one `data.id` and one message for the whole list. There is no per-device result, so a request can report success while one board did nothing. Per-phone screenshot verification is not defense in depth here, it is the only failure detection that exists today.
- The only asynchronous per-device failure signal is the `error_push` callback (`data.call_fun` plus message, for example `/mouse/down` and "Hardware not working (unknown error)"), and that requires an open WebSocket, which was never opened during the 10 September probe.
- Fan-out amplifies the silent stuck-click state rather than mitigating it. A comma-list click against N phones, one of which is stuck, returns the same success code as a fully successful call, so spot-checking a subset under-detects stuck phones in proportion to N.
- `/pic/screenshot`, `/pic/find-image` and `/pic/ocr` all refuse multiple ids, so verification is inherently N requests while input is one.
- The parallel-walk claim is measured on **two** phones. The "10 phones costs the same as one" line is a vendor-side inference. The ten-way numbers in the measurements are concurrent frame *pollers*, a read path, not simultaneous multi-phone pointer walks, which are a write path through shared USB HID bandwidth across three chained 7-port hubs. Do not use 512 req/s as evidence that a 19-phone click fans out for free.
- Groups are kernel-global and unauthenticated. Any LAN host, including the vendor console's own operator, can move a phone in or out of the target group between your resolve step and your click step, and there is no locking primitive. Re-resolve per job and react to `group_change` and `group_delete` rather than caching membership.
- Destructive adjacency: `/device/group/del` and `/device/del` are one path element away from `/device/group/get-dev`, and they refuse only the specific guarded cases (code 34, code 7).
- Prefer `/mouse/wheel` for scrolls. It moves no pointer, so it skips the roughly 0.6 ms per pixel walk entirely, which swipe and click sequences cannot.
- The Unicode branch is not merely unverified, it is a separate manual prerequisite chain: a companion app installed per phone, the kernel IP typed in, an AssistiveTouch key bound. None of that is exposed by any API call in this document, so it cannot be provisioned by automation.
- On a Phonefarm-edition kernel this whole design collapses. Phonefarm has no multi-id addressing, its only fan-out being the shortcut dispatcher's `devlist` array, so every step becomes N requests.
- Driving the same account or session across many phones from one controller is visible to the destination service as coordinated automated behavior from a single operator. That is a terms-of-service and fraud-detection exposure to weigh before scaling group size, and the correct response to a rate limit or an "action blocked" interstitial is to stop the job.

SCALE

Input costs exactly one request per step regardless of N, with latency set by the longest pointer walk in that step: about 592 ms for a 1000 px move, about 580 ms of travel for a full-height 1080 px swipe (which iOS may read as a slow drag rather than a flick, so prefer the 500 px short axis or the wheel), plus 1 to 12 ms of HTTP. Verification is the real bottleneck, because it needs N independent captures *and* a fresh AirPlay frame: on this rack one XR delivers about 28.9 new frames per second and its identical twin only about 4.5, so a mixed fleet's cycle time is set by its slowest phone, roughly 0.8 to 1.5 s per step for a short move plus settle and verify. That is on the order of 2,000 to 4,000 group-level steps per hour for the rack as a whole, not per phone, and it is only

trustworthy at the measured scale of two phones. Consider `thread_mode` in `/config/imserver/set` ("use thread mode to batch operate hardware", applies live with no restart) before scaling the id list. The hard ceiling on phones per PC is the hub topology, 21 physical slots with 19 populated, not the API.

Coordinate-space registry keyed by model, iOS and mirror geometry

NATIVE API WITH CAVEATS

A client-side template store in which every named UI target ("Settings icon", "Safari address bar") is keyed by model, iOS version, `imgw` x `imgh` and `rotate`, and stored in mirror-image pixels. A resolver reads each phone's live geometry from `/device/get` and refuses to fan a coordinate out to any phone whose space differs from the master's. Templates are authored by cropping raw BMP frames from `/pic/screenshot`, so the author's pixel is literally the pixel `/mouse/click` receives.

This encodes the single most expensive trap measured in the project: using iOS logical points (414 x 896) where the kernel wants mirror pixels (500 x 1080 on these XRs). Only `/device/get` exposes `imgw` and `imgh` at all, and the Phonefarm device record does not carry them, so making the space an explicit registry key is the difference between a script that ports across models and one that silently mistaps.

HOW

1. `get_device_list` XP `/device/get` — read `imgw`, `imgh`, `width`, `height`, `model`, `version`, `rotate`, `state`, `gid` and `location_crc` per record; live-probed here as two iPhone11,8 on iOS 18.7.10, imgw 500 x imgh 1080, physical 414 x 896
2. `dev_rotate` XP `ws://127.0.0.1:9911/api` — invalidate a cached space entry on `dev_rotate` (orientation changed) and on `dev_change` (name, group, binding or calibration changed by any client); Phonefarm has `dev_rotate` but no `dev_change`
3. `get_device_screenshot` XP `/pic/screenshot` — author a template from a full raw frame (`binary=true`), or add `rect=[l,t,r,b]` to crop and `save_path=<dir>` to have the kernel write the crop and return its path in `data.image`; record name, model, version, imgw, imgh, rotate, x, y and the crop file
4. `none` XP `none (client-side)` — resolve at run time as `registry[model, iOS major, imgw x imgh, rotate]`, and on a miss fail the job rather than falling back to another entry; there is no kernel-side check that a coordinate belongs to the device it is sent to
5. `none` XP `/device/get + /config/usb/get` — fan-out guard: build the comma id list only from phones whose geometry equals the master's, whose `state != 0`, whose `location_crc` is non-empty, and whose board reads state 2
6. `send_key` XP `/key/sendkey` — normalize with `fn_key=WIN+h` twice before applying a page-one template; the newer `fn_key=Home` exists only for iOS 26.1 and later and needs kernel 2026.2.27, so WIN+h is correct on these XRs
7. `mouse_reset_pos, mouse_move` XP `/mouse/reset, /mouse/move, /pic/screenshot` — drift probe: reset, move to the template point, then take the mean absolute pixel difference in a roughly 40 px box around it; log offsets over about 5 px as needing calibration
8. `mouse_collection_*` XP `/device/collection/mouse, /device/collection/mouse/save, /device/set` — remediate a drifted phone with `cmd=1` (0 query, 2 stop), watch the `collection_mouse` callback, save with a describe remark, then write `location_crc` back
9. `set_airplay_mode` XP `/config/imserver/get, /device/airplay/set` — freeze the space the templates were authored in: keep `air_play_ratio` fixed globally and leave per-device `air_ratio` at 0

NEEDS

- XP kernel reachable on 127.0.0.1:9911 with IMConsole.exe running; with IMService alone both phones read state 0 and the board list was empty.
- `air_play_ratio` fixed globally and per-device `air_ratio` left at 0, or the whole template set rebuilt whenever either changes.
- Templates authored per phone wherever home-screen layouts differ. The two XRs on this rack have different home screens and wallpapers, so a shared page-one coordinate is not a shared target.
- Phone-side input settings for the drift probe to mean anything: AssistiveTouch on at maximum tracking sensitivity, Trackpad and Mouse tracking speed at maximum, Auto-Lock Never, portrait lock. None is queryable.
- A client-side decoder for bottom-up 24-bit BMP and a mean absolute difference routine. There is no server-side compare.

WATCH OUT

- Three of the nine steps rest on things never exercised on this rig. The WebSocket bus and its `dev_rotate` and `dev_change` callbacks were never opened, so cache invalidation on rotate is a paper design; ship a periodic `/device/get` poll every 30 to 60 s as a fallback in case a dead socket silently freezes the cache.
- `rect` and `save_path` on `/pic/screenshot` are documented but were never live-probed. Only the plain full-frame `binary=true` capture was measured (2 ms, 1,620,054 bytes). Verify them before building the authoring tool on them; the fallback of full frame plus client-side crop works but pays the full 1.6 MB transfer.
- Whether `imgw` and `imgh` swap when `rotate` flips to 1 is stated nowhere in the XP docs. Re-read `/device/get` after a rotation instead of transposing the cached pair, which means the registry key cannot be computed offline from one portrait pair.
- Landscape behavior is also affected by the kernel-wide `flip_right` ("horizontal screen flip right mode"). If it is ever changed it belongs in the registry key, and it applies live with no restart.
- A phone with an empty or uncollected `location_crc` returns `data.code 11`, no mouse parameters. The resolver should check that field before adding a phone to any fan-out list, not only geometry equality.
- Calibration remediation is a hard human-in-the-loop step, not an API gap a retry fixes: the XP docs never publish the phone-side collection URL, so starting a collection needs the vendor console's QR code and a person holding the phone. Budget a manual queue; drift correction cannot run unattended overnight.
- The pointer-dot box diff is only valid once the phone's AirPlay stream has produced a new frame. On the slow twin at 4.5 fps the probe must wait about 220 ms or more, or it reads the pre-move frame and reports a false "needs calibration".
- There is no OCR or image-search fallback on this rig to re-find a moved element: `/api/pic/ocr` answers `data.code 28`, plug-in not connected, and image search is presumed equally absent. A stale template fails blind. Keep the crop so a client-side matcher can be added later.
- Because `/mouse/click` returns code 0 for a coordinate that lands nowhere useful, a bug in the registry itself (wrong crop, stale cache) is indistinguishable from a genuinely drifted phone until the box-diff check runs. The registry's correctness rests entirely on its own client-side test coverage.
- The kernel returns BMP by default or JPEG, nothing else, so a PNG template store re-encodes client-side.
- Templates are per model, per iOS major, per ratio. An iOS update can move elements without changing any other component of the key, so re-author after updates.
- The API has no authentication, so any LAN host can move the same phone's pointer while a probe is running and corrupt the measurement. Single-flight the probe per phone.

SCALE

Resolution is an O(1) dictionary lookup with no kernel round trip. The only kernel cost is one `/device/get` at start plus invalidations. Authoring is a one-time cost per combination of model, iOS major, geometry and rotation, one capture of roughly 2 to 40 ms plus a human placing the crop, and this two-XR rig needs one combination since both phones share model, iOS and ratio. The recurring cost is the drift probe: reset plus move (worst case about 600 ms for a full 1080 px jump, usually far less because template points are local) plus a 2 ms capture plus the diff, roughly 0.7 to 1.3 s per phone end to end. Phones are independent, ten concurrent pollers having measured 512 req/s at p50 3 ms, so a full-rack drift sweep across all 19 boards run in parallel finishes in under two seconds of wall time rather than 19 times that. The practical ceiling is hub capacity, not this feature's software cost.

The control plane: events, and actions that prove themselves

Two components sit under everything else in a console built on this kernel. One owns the single WebSocket and the rack's cached state. The other owns the rule that no input is believed until a frame proves it landed. Both are buildable today on documented XP calls, and both carry the same honest gap: the socket itself was never opened during the 10 September probe.

WebSocket event bus: msgid-correlated RPC plus broadcast state cache

NATIVE API

WITH CAVEATS

One long-lived client on `ws://<pc>:9911/api` doing two jobs. As an RPC channel it sends the identical `{"fun":"/path","msgid":n,"data":{...}}` envelope the HTTP POST form uses, keeps a pending map from msgid to promise, and resolves on the reply whose msgid matches. As a state cache it consumes the kernel's unsolicited broadcasts into an in-memory rack model that every other component reads instead of polling on a timer.

Two signals exist only on this socket. `error_push` carries `{call_fun, message}` and is the single asynchronous notice that an input call failed at the board. `im_connect`'s `data.ver` is the only kernel-version readout in the entire XP API, since there is no HTTP version endpoint. `user_info`'s `dev_online_num` against `total_license` is likewise the only way to see a license cap before phones start being refused.

HOW

1. `connect ws://host:9911/clinet` XP `ws://127.0.0.1:9911/api` — open the socket and record `data.ver` from the `im_connect` callback; Phonefarm's path is literally `/clinet` and its HTTP lives on 9912, neither of which applies here, and 9912 is closed on this rig
2. `get_device_list / get_group_list / get_usb_list` XP `/device/get, /device/group/get, /config/usb/get, /config/imserver/get` — subscribe first, buffer what arrives, then prime the cache with these four and apply the buffer over the snapshot
3. `same envelope, different fun names` XP `{"fun":"/mouse/click","msgid":17,"data":{...}}` — send any request over the socket with a monotonic msgid; every XP endpoint page states that the WebSocket accepts the same JSON, and bare Phonefarm-style names are rejected with HTTP 500
4. `dev_connect / dev_disconnect / dev_rotate` XP `dev_connect, dev_disconnect, dev_rotate, dev_change, dev_delete` — patch device state; XP nests the record in `data.list[]` where Phonefarm sent a bare record, renames fields (`air_handle`, `air_pid`, `srv_name`, `user_name`, plus `imgw`/`imgh`, `uid`, `gname`, `uver`, integer `gid`), and sends `dev_delete` as an array of deviceid strings

5. `usb_change {state, vpid}` XP `usb_change` — patch control-channel state from `data.list[{vid,pid,uid,ver,state}]` with the event type in `data.code`: 19 hardware connected to computer, 20 disconnected, 12 hardware connected to phone
6. `set_group / del_group` XP `group_change, group_delete` — patch groups, noting that `group_change` carries `data.id` and `data.name` at the top level of `data` and **not** a list (the `/device/group/set` response is the one returning `data.list`), while `group_delete` does carry a list of deleted gids
7. `none` XP `error_push, im_log, user_info, logout, im_config_change, dev_sort_change` — route operational signals; `error_push` has no Phonefarm counterpart, `im_log` carries notices such as "not logged in, going online not allowed", and `user_info` arrives at login and every 30 s with pass, license counts, device counts, overdue time and user state
8. `mouse_collection_cfg_ret` XP `collection_mouse, airplay_connect_log, restart_log` — track long-running operations; `collection_mouse` folds Phonefarm's two callbacks into one code space (0 success, 4 failed, 5 a device is collecting, 6 this id is not, 8 scan code opened, 9 entered the collection page, 14 start working, 21 at work)
9. `loop_device_screenshot` XP `none, poll GET /api/pic/screenshot` — keep frames off this socket; XP documents no push frame stream and no binary frame protocol, and `binary=true` returns raw bytes instead of a JSON envelope, so its behavior over a JSON-framed socket is undefined
10. `none` XP `reopen ws://127.0.0.1:9911/api` — reconnect and re-prime on close; `/imserver/restart` drops the socket by design, and no resume token or replay buffer is documented

NEEDS

- XP path style on 9911 only. Phonefarm's `/clinet` path and bare fun names are rejected by this kernel with HTTP 500 and a Delphi-flavored "List index out of bounds (1)" message, and 9912 is closed.
- A client that ignores message strings and branches on `data.code`. This build returns Chinese text.
- IMConsole.exe running and logged in, or the primed cache is empty.
- Only documented paths. Guessed ones return 404 with a JSON-shaped body you might otherwise parse as a real error; `/config/get`, `/user/get`, `/shortcut/get` and `/device/model/get` all 404'd here.

WATCH OUT

- **Documented, unprobed.** Every callback shape above comes from the vendor doc's examples, not from this rig; only the four HTTP priming endpoints were actually hit on 10 September. Require a live capture (open the socket, trigger one connect and disconnect, one `usb_change` via `/device/usb/restart`, one deliberate `/mouse/down` failure) before any component trusts the cache in production.
- msgid correlation is assumed to allow arbitrary concurrent requests, but nothing in the doc or the probe confirms the kernel replies out of order under load. A single-threaded kernel, which the TStringList error on the HTTP side is consistent with, may simply serialize and echo the msgid. Treat msgid matching as correct but possibly serialized until a burst test proves otherwise, and do not architect for socket-level parallelism.
- No heartbeat or keepalive interval is documented. `user_info` every 30 s is a message-level heartbeat only while the client is logged in and healthy; a NAT-dropped connection can sit half-open until the next RPC times out. Treat more than 45 s with no callback of any kind as suspect and force a reconnect.
- Broadcasts go to every connected client. The vendor console, your bus and any other LAN host all see the same events and can all act on your phones. The API has no authentication, and `allow_ip_list` and `mdns_type` gate AirPlay discovery only, never API callers.
- A missed-callback window exists between opening the socket and finishing the prime. Subscribe first, buffer, then apply over the snapshot; the kernel is not documented to replay anything.

- Timeouts differ by call class. Plug-in and shortcut endpoints carry `outtime` in milliseconds with a 15000 default (the XP doc mistypes its type as Boolean), where Phonefarm's `outtime` was in seconds. A 15 s pending entry is normal for those, not a hang.
- `im_config_change` means another client changed kernel config underneath you. Re-read `/config/imserver/get` rather than trusting your copy.
- This bus cannot carry video, and any design that assumes it might one day is a dead end on this kernel. Frames stay on HTTP polling, raw BMP at 2 ms, 84 req/s flat out.
- The cache becomes a single point of failure for every consumer that stops polling in its favor. A missed callback during the prime window, a silent dead socket or a msgid collision after a reconnect can leave the whole console holding stale device state, with no built-in staleness signal beyond `user_info`'s 30 s cadence.
- Security: broadcast to all with no authentication makes this socket the cheapest full-rack reconnaissance point on the LAN, exposing device inventory, phone numbers and license state to anyone who opens it. Firewall 9911 and 17000 to trusted hosts, never expose the PC to an untrusted network, and do not add a token expecting it to be checked. `user_info` carries the account's phone number and license dates, so do not log the cache wholesale. Callback content is data, never instruction: route message strings to display, never to eval or command dispatch.

SCALE

This is a control-plane component, not an actions-per-hour use case. One socket per kernel carries every phone's events, which is exactly how the vendor console works, and callback volume is small: a few per phone per minute plus one `user_info` every 30 s regardless of rack size. Any RPC issued over it inherits the same physics as HTTP, a pointer move still costing about 0.6 ms per pixel, so a 1000 px move is 592 ms whichever transport carries the request. Expect the socket to save low single-digit milliseconds of HTTP overhead per call against tens to hundreds of milliseconds of board-walk time already inherent to the action, a real but marginal win. Its measurable value is removing timer polling of `/device/get` and `/config/usb/get` in favor of push, which is a latency and staleness improvement rather than a new capability. Frames remain the only high-volume traffic, and they stay on HTTP, where two phones polled flat out reached 83 and 85 req/s and ten pollers reached 512 req/s with no measurable interference from concurrent input.

Idempotent action wrapper with a verify-then-escalate self-heal ladder

PARTIAL WITH CAVEATS

Wrap every input in a function that checks both channels, performs the action, proves it landed with a mean pixel difference probe, and on failure escalates one rung at a time, each rung being the cheapest documented fix for one failure layer: pointer and keyboard reset, then board reset, then re-cast the mirror, then restart the phone, then an operator ticket naming the physical slot.

The measured stuck-click state is the case that defeats naive automation. The phone follows `/mouse/move`, answers hardware keys (Home produced a 142.15 mean pixel difference while stuck) and returns `data.code 0` for every click, and does nothing. A pipeline that trusts a 200 with code 0 can run for hours achieving nothing. The wrapper turns that into a bounded, observable retry that applies the right fix at the right layer instead of restarting the world.

HOW

1. `get_usb_list + get_device_list` XP `/config/usb/get, /device/get` — precheck both channels: board at state 2, device `state != 0` with a live `air_pid`; a board at state 1 with a phone physically plugged in means the cable or slot is at fault and there is no API fix

2. `mouse_reset_pos then click` XP `/mouse/reset, /mouse/click` — rung 0, clear pointer state then act; both accept comma id lists
3. `get_device_screenshot` XP `/pic/screenshot` — probe whether it landed, one id only, `binary=true`, 2 ms and 1,620,054 bytes measured; whole-frame mean absolute difference over 20 means it reacted (141.96 for a landed click), under 1 means nothing happened (0.09 stuck, 0.02 to 0.03 idle)
4. `key_release_all + mouse_up + mouse_reset_pos` XP `/key/upall, /mouse/up, /mouse/reset` — rung 1, release stuck keys and buttons and retry once; this targets an interrupted chord or an abandoned drag and will not fix the stuck-click state on its own
5. `restart_usb` XP `/device/usb/restart` — rung 2, reset the control board at both ends; the id is `vid|pid` (doc example 720D|F735), not the deviceid and not the 8-character uid, and the pipe must be URL-encoded on GET
6. `discon_airplay then auto_connect_screen` XP `/device/airplay/disconnect, /device/airplay/connect` — rung 3, rebuild the video channel; XP has no force flag, so an explicit disconnect first is the documented substitute, and `/device/airplay/connect/all` exists for whole-rack recovery
7. `restart_device` XP `/device/restart | /shortcut/switch/device | /key/sendkey` — rung 4 by three routes: the kernel restart with `restart_log` progress, the Shortcut route (`state=0` shutdown, `state=1` restart, iOS 15 and later, comma id list), or the `CTRL+ALT+SHIFT+WIN+r` hotkey
8. `none` XP `none (Windows PnP, not the API)` — rung 5, open a ticket with the physical slot; hub plus port is the only stable label, three 7-port hubs (VID 1A40 PID 0201), 19 of 21 slots populated, the two live phones on hub 0010 ports 2 and 3, and the API exposes no topology at all

NEEDS

- The WebSocket event bus for `usb_change`, `airplay_connect_log`, `restart_log` and `error_push`. Rungs 2 to 4 have no synchronous completion signal, so without the socket you are guessing.
- A repair queue separate from the work queue, because rungs 2 to 4 take a phone out of the pool for seconds up to roughly 90 s.
- For rung 4 via Shortcut: iMouse Shortcut installed with the kernel IP edited in, run once with every permission allowed, bound to an AssistiveTouch custom key, iOS 15 or later. Via hotkey: Full Keyboard Access on.
- A decision about what happens after a restart. The phone boots to the lock screen, mirroring drops, and a passcode-locked phone cannot be unlocked by this automation. Set `auto_connect` in `/config/imserver/set` or call `/device/airplay/connect` explicitly, and expect to strand passcode phones for a human.
- A probe of the plug-in state. This rig has no OCR plug-in (`/api/pic/ocr` answers `data.code 28`) and image-search status is unknown; assume absent.

WATCH OUT

- Rungs 0 to 2 are documented XP calls with no plug-in dependency and are usable today. Rungs 3 and 4 are documented but unverified here: the XP doc never states a plug-in requirement for `/device/airplay/connect` or `/device/restart`, yet its code table carries 15 "no coordinates" and 28 "plug-in not connected", and the Phonefarm sibling explicitly required OCR plus image search for the equivalent calls. That is inferred from doc silence, not confirmed absence of a dependency, so treat both rungs as expected to fail until probed and make the ladder fall through to a human rather than loop.
- Rungs 3 and 4 depend architecturally on the WebSocket bus for their completion signals, and that bus is itself unverified on this rig. Today, over HTTP alone, those rungs are fire and hope exactly like the Phonefarm predecessor, because `/device/airplay/connect` returns before the cast completes and there is no synchronous alternative.

- The "over 20 means it reacted" threshold is generalized from two data points, both large high-contrast transitions. A tap that only toggles a checkbox, a keyboard key or a low-contrast control can produce a whole-frame mean under 20 and be misclassified as stuck, triggering a destructive escalation for an action that actually succeeded. Calibrate per action class with a box diff around the expected change, not one global whole-frame constant.
- The probe cannot catch a tap that landed 30 px off target, and whether `/mouse/reset` corrects a *drifted* pointer is explicitly unverified; it is confirmed only to clear the stuck-click state. Drift needs a positional check.
- `/device/usb/restart` is addressed by VID and PID, not deviceid. Passing a deviceid is a silent no-op class of bug.
- Rung 2's precheck, and the whole state-2 gate, rest on the `state` field of `/config/usb/get`, which is absent from the documented field table and whose meaning is reconstructed from the `usb_change` code table and this rig's measurements. A kernel update could change it silently, producing wrong routing decisions with no error. Monitor it rather than treating it as a contract.
- The 20 then 19 then 12 `usb_change` sequence after a board reset is inferred from the documented code meanings, not stated. Treat any of the three arriving as progress rather than requiring that order.
- Restarting a passcode-locked phone strands it on the lock screen, and screenshot verification is useless during the reboot because the mirror is gone. Use the callback plus a reachability check plus `/device/get` state instead, and give the queue a terminal "needs human" state distinct from "retry eligible", or the ladder will re-strand the same locked phone forever and consume repair slots.
- Give the ladder an explicit numeric backoff and circuit breaker. Without a cap on escalations per phone per hour, a flapping fault such as a marginal cable can cycle a phone through rungs 2 to 4 repeatedly at 60 to 90 s of downtime each, degrading a healthy-looking rack far more than the original fault would have.
- Rung 0's reset forces a full-distance walk on the next move, up to about 650 ms. That is a per-phone latency budgeting concern for the calling code, not a rack-wide throughput limit, since the boards walk phones in parallel with no shared kernel bottleneck measured.
- Every rung above 1 is destructive to some degree and should be rate-limited and logged. Do not use the restart rung to work around an app that is deliberately blocking automation; if the app signals a block, stop that job and surface it rather than escalating hardware. The kernel is unauthenticated, so these same destructive calls are available to anything on the subnet. Keep `/device/del` and `/shortcut/album/clear` out of any automatic ladder.

SCALE

Rung 0, the steady-state guarded path, adds two roughly 2 ms raw-BMP captures per guarded action plus an occasional full-distance reset walk of up to about 650 ms, paid only when the target set changes or before a tap-critical sequence rather than per call. For typical UI taps 100 to 500 px apart, 78 to 300 ms each, one phone can sustain roughly 3,000 to 6,000 guarded actions per hour before verification overhead, and because `/mouse/reset`, `/key/upall` and `/mouse/up` all take comma-separated id lists, resetting an entire target set before a batch is one request that the board walks in parallel. There is no shared kernel bottleneck to contend for, ten concurrent pollers having reached 512 req/s at p50 3 ms with input traffic not slowing frame polling, so rungs 0 and 1 scale close to linearly with rack size. Escalations are the rare and expensive path: rung 2 costs seconds, rung 3 seconds to tens of seconds, rung 4 roughly 60 to 90 s plus a re-cast. At even a 2 percent fault rate needing rung 4, a 19-phone rack loses on the order of 0.4 phone-equivalents of throughput per restart cycle, which is why the separate repair queue is load-bearing rather than optional.

Boundaries: an authenticated gateway and a measured rack

The kernel has no authentication, no outbound calls, no audit trail and no metrics. Those four gaps are the same gap seen from four angles, and both components here exist to close it: one is the boundary anything outside the LAN must cross, the other is the instrument that tells you whether the rack is actually working.

Authenticated HTTP gateway so n8n, Zapier or Make can drive the rack

PLUS EXTERNAL TOOLING

WITH CAVEATS

A small reverse proxy in front of 9911 that requires a bearer token, exposes a handful of high-level verbs (health, tap-group, open-url-on, type-on, screenshot-of), translates each into XP path calls with the reset-and-verify wrapper, and returns re-encoded screenshots as URLs. It also holds the one WebSocket session and turns `dev_disconnect`, `usb_change` code 20, `error_push` and `im_log` into outbound webhooks, because the kernel can only push to connected clients and cannot call out on its own.

This is simultaneously the only safe integration surface and the authentication boundary the product lacks. The kernel's error model is also unusable by no-code tools: HTTP status is 200 for a failed operation, the real result is `data.code` against a 41-entry table, and messages come back in Chinese.

HOW

1. `none` XP `none (network layer)` — run the gateway on the rack PC or a jump host on its own port, and restrict 9911 in the Windows firewall to localhost, the gateway host **and the phone subnet**, since the phones need 9911 for the Shortcut return path and 17000 plus mDNS for AirPlay
2. `get_device_list, get_usb_list` XP `/device/get, /config/usb/get` — verb "health", merged per slot into control (board state 2 and a bound uid), video (device state non-zero with a live `air_pid`) and an `observed Fps` the gateway computes itself
3. `shortcut (id 13)` XP `/shortcut/exec/url, /device/group/get-dev, /pic/screenshot` — verb "open-url-on": resolve members, call the Shortcut with `outtime=15000`, then one capture per phone, re-encoded and uploaded, replying with the URLs
4. `mouse_reset_pos, click, get_device_screenshot` XP `/mouse/reset, /mouse/click, /pic/screenshot` — verb "tap-group": registry resolves the coordinate in that group's space, reset the id list, click the id list, then diff per phone and reply `{ok:[ids], failed:[ids]}`, which is the whole reason the gateway is worth building
5. `send_key, send_text` XP `/key/sendkey, /shortcut/clipboard/set` — verb "type-on": ASCII through `key=`, non-ASCII through the clipboard then `fn_key=WIN+v`
6. `dev_disconnect, usb_change` XP `ws://127.0.0.1:9911/api` — outbound: hold one socket and POST `dev_disconnect`, `usb_change`, `error_push` and `im_log` to the n8n webhook URL, since there is no outbound webhook facility anywhere in the API
7. `none` XP `msgid echoed on every request` — audit: log every inbound call with token id, verb, resolved id list, msgid and before and after frame fingerprints, because the kernel keeps no audit trail of its own

NEEDS

- A gateway process to write and run. This is external tooling, not an API feature.
- n8n, Zapier or Make reachable from the gateway over outbound HTTPS. The rack PC must never be reachable from the internet.
- Firewall rules that restrict 9911 and 17000 to localhost, the gateway host and the phone subnet.

- Object storage or a static file host for screenshots, plus a BMP decoder and a PNG or JPEG encoder on the gateway.
- For the open-url-on and Unicode type-on verbs: iMouse Shortcut installed on each phone, kernel IP edited in, bound to an AssistiveTouch custom key.
- IMConsole.exe running and logged in, or `/device/get` reports state 0 and no boards enumerate.
- Per-group coordinate templates in the mirror-pixel space of those phones.

WATCH OUT

- The "health" verb as specified cannot detect the stuck-click state. A phone can be board state 2, bound, mirroring with a live `air_pid`, while `/mouse/click` produces a 0.09 mean pixel difference no-op and moves and hardware keys still work. Document **control-ready** and **tap-verified** as two different guarantees, or the gateway will report a broken phone as healthy.
- `observedFps` computed by counting frames whose mean difference exceeds about 1 reads near zero on an idle screen, because a still screen produces almost no new frames. That is a property of content, not a health signal. Either drive a trivial 2 px move before sampling, or label an idle reading "not measured" rather than "slow phone".
- `air_fps` in the device record is a configuration override where 0 means use the global value. It is not a measured rate, and reporting it as fps would be wrong.
- The reset-and-verify wrapper only protects the click it immediately precedes. A multi-step verb can enter the stuck-click state between two of its own taps, so re-verify mid-sequence rather than resetting once at the top.
- The kernel cannot initiate an outbound call, so the event-to-webhook bridge is a hard requirement for any "notify me when a phone drops" flow, and the gateway's single socket becomes an unmonitored single point of failure for all of it. Its reconnect and backoff behavior is unaddressed and was never live-tested.
- Do not wire `/config/user/login` into any verb. It takes the vendor account phone and password, and this rig already stores that credential as reversible hex obfuscation in `C:\iMouseXP\IMService.json`, so a gateway that proxies it turns a token leak into an account compromise. Leave account login to the vendor console and the user.
- Exclude destructive endpoints from the verb surface or gate them behind explicit human confirmation: `/shortcut/album/clear`, `/shortcut/album/del`, `/shortcut/file/del`, `/device/del`, `/device/group/del`, `/device/restart`, `/shortcut/switch/device` with `state=0`, `/imserver/restart`. An automation platform's retry can fire any of them twice.
- Make every verb idempotent by requiring a client-supplied job id and deduplicating on it. The kernel offers no idempotency of its own; msgid is only a response-correlation token.
- Diff with mean absolute pixel difference thresholded around 1, never byte or MD5 equality. Clock ticks, encoder noise and wallpaper parallax make an MD5 check report a false positive on every idle phone.
- Verified actions are bounded by the phone's own AirPlay rate, about 28.9 new frames per second on one XR here and about 4.5 on its twin, so do not use a fixed timeout tuned to the fast phone.
- `/pic/screenshot` takes a single id, so screenshot-of a group of N costs N concurrent requests.
- Shortcut-dependent verbs are unverified on this rig and will fail with `data.code 30`, call timeout, if the Shortcut is not installed or its binding does not fire.
- There is one HID pointer per phone, so multi-touch gestures such as pinch and rotate cannot be added to this design without a different mechanism.
- A leaked gateway token is a full rack takeover, because the thing behind it authenticates nobody. Scope tokens per group, rate-limit them, rotate them and log every call. Over-firewalling is its own outage:

blocking 9911 from the phone subnet silently breaks every Shortcut-backed verb, and blocking 17000 or UDP 5353 breaks mirroring. Verbs that type into third-party apps carry whatever terms-of-service exposure the underlying activity carries, amplified by a webhook that can fire a job thousands of times, so cap job rates at the gateway and stop a job when a platform returns a rate limit or an "action blocked" response. Finally, map `data.code` to HTTP status yourself (0 success, 2 not bound, 3 hardware not connected, 11 no mouse parameters, 25 offline, 26 screenshot failed, 28 plug-in not connected, 30 call timeout, 36 parameter error, 41 incompatible command) and never match on message text.

SCALE

Per-verb latency is the right unit, not throughput. Gateway overhead on loopback is roughly 1 ms, and unverified control calls (health, reset, click, sendkey) are sub-10 ms at the kernel, scaling to something over 500 req/s aggregate across phones on the measured evidence. Verified verbs are bounded by the phone's own AirPlay source rate: a confident before-and-after diff needs roughly two fresh frames, about 70 to 140 ms on a 29 fps phone against about 450 to 700 ms on a 4.5 fps phone, on top of real iOS settle time the API cannot shorten. Practically, expect hundreds to low thousands of verified taps per hour per phone for real UI navigation, not the thousands the raw click latency alone would suggest. A URL open through the Shortcut is typically seconds and bounded by `outtime`. Screenshots are 1,620,054 bytes raw at 2 ms each; re-encode on the gateway rather than paying the kernel's 40.5 ms JPEG path and its roughly 21 req/s cap. One WebSocket session carries every callback for the whole rack.

Observability stack: frame archive, per-phone metrics and a rack SLO

PLUS EXTERNAL TOOLING

WITH CAVEATS

A collector process that holds one WebSocket and turns broadcasts into events, polls each phone's raw BMP, keeps only frames whose mean difference from the last kept frame exceeds a threshold, re-encodes client-side and uploads keyed by deviceid and time, and exports metrics: video state and `air_pid` from `/device/get`, control state from the board, new frames per second from the diff meter, a pointer probe result, `error_push` counts labeled by `call_fun`, license headroom from `user_info`, and kernel version and liveness from `im_connect`. The SLO is the fraction of phones simultaneously control-ready, video-ready and pointer-probe-good.

Every real trap in this project was found by measuring, and none of them is exposed as a kernel metric: that one XR delivers about 29 new frames per second while its identical twin delivers about 4.5, that a phone can accept moves and hardware keys while silently swallowing every tap, that board state and mirroring state are two independent channels. Without a diff-based frame meter and a scheduled pointer probe, a half-dead rack looks healthy in the vendor console.

HOW

1. `dev_* / usb_change callbacks` XP `ws://127.0.0.1:9911/api` — one socket for all broadcasts; the XP envelope is `{fun, status:200, message, msgid, data:{list:[...], code, id, message}}` with device records inside `data.list`, and callbacks are WebSocket-only, with no HTTP long-poll or event feed
2. `get_device_list` XP `/device/get` — video-state gauge from `state` (0 not online, non-zero online) plus `air_pid`, measured here as the AirplayMirroring.exe PID and the most reliable live proof that mirroring is up
3. `get_usb_list` XP `/config/usb/get` — control-state gauge from `data.list[{vid,pid,uid,ver,state}]`, where 1 is a board on the PC with no phone and 2 is a board with a phone attached
4. `get_device_screenshot` XP `/pic/screenshot` — frame meter at a **fixed** period, since deriving the interval from observed frame gaps was measured to decay 23 fps to 16.5 fps through positive feedback

5. `get_device_screenshot` XP `/pic/screenshot with rect and save_path` — archive with dedupe: keep a frame only when its mean difference against the last kept frame exceeds about 5, then re-encode client-side; `rect` crops server-side for a cheap detector and `save_path` writes to disk, two XP-only options the Phonefarm edition lacks
6. `mouse_reset_pos + mouse_move` XP `/mouse/reset, /mouse/move` — scheduled control probe, reading a box diff of 1.4 to 9.8 around the commanded coordinate against an idle floor of 0.02, which a whole-frame diff cannot see
7. `none` XP `error_push, im_log, user_info, /config/user/info` — error and license telemetry; a counter labeled by `call_fun` is genuinely new information, and `user_info` arrives at login and every 30 s with license and device counts
8. `get_airplaysrvnum` XP `im_connect (data.ver)` — kernel identity and liveness; this is the only source of the kernel version in XP, so export it as an info metric and set `kernel_up=0` on socket close
9. `derived` XP `/config/usb/get + /device/get + the probe` — the SLO: phones with control state (board at 2 and the phone bound to that board's uid) and video state (device state non-zero with a live `air_pid`) and a good pointer probe, over total enrolled; the rack has been observed with 13 boards carrying phones while only one phone was mirroring

NEEDS

- A metrics scraper and an object-storage endpoint, both external. The scraper must reach the collector, never the kernel, so 9911 stays closed to everything but the collector host.
- Buffer reuse in the poller. Each raw frame exceeds the 85 KB .NET Large Object Heap threshold; without reuse this measured 488 MB/s of allocation and 496 gen-2 collections per 8 s, against 2.9 MB/s and zero to one with reuse.
- A board lease shared with the job queue, so a probe never moves the pointer during a human-driven or scripted session.
- IMConsole.exe running and logged in, or every gauge reads zero for the wrong reason.
- A WebSocket client that reconnects. All event telemetry is socket-only.

WATCH OUT

- The 19-phone extrapolation (190 req/s, roughly 300 MB/s) is bandwidth-only math that ignores client-side cost: diffing nineteen concurrent 1.6 MB BMPs at 10 Hz is 300 MB/s of decode and compare work inside the collector. The only measured concurrency point is ten pollers against the *same two* devices, not 19 distinct AirPlay sources feeding 19 diff buffers. Pilot at four to six phones and measure CPU and GC before committing.
- A uniform 10 Hz poll is partly self-defeating. On the 4.5 fps phone roughly 55 percent of polls return the identical previous frame. Make the rect-cropped cheap detector the default poll and reserve full frames for archive-on-trigger, or the fast and slow phones share one period tuned for neither.
- `state` in `/config/usb/get`, and `air_pid` and `air_handle` in `/device/get`, are present in live responses but absent from the vendor's field tables. They are safe as liveness evidence and unsafe as a stable contract, so alarm loudly if they ever vanish from a response, since that would zero both readiness gauges at once.
- The pointer probe mutates the rig: `/mouse/reset` snaps the pointer to 0,0 and `/mouse/move` walks it. Gate it on the lease, keep hops short, and when the lease is held for a whole scrape interval report **unknown** rather than not-ready, or every busy phone fails the SLO purely from lease contention.
- The box-diff signal was validated on two XRs with one particular pointer rendering. A different model or iOS build could render the cursor differently, or not at all if AssistiveTouch is off, and silently break the probe. Calibrate the threshold per device model.

- `new_frames_per_second` is a property of the phone's own Wi-Fi and AirPlay link as much as of the kernel, and the vendor console hits the same wall. Alert on a drop relative to each phone's own baseline, never on a single global threshold.
- Never compare frames by MD5 or byte equality. An untouched mirrored phone changes every frame; threshold the mean absolute difference instead, under about 1 being nothing and a real reaction 20 to 160.
- License headroom must read both `total_license` and `local_total_license`, and `user_state` 1 against anything else distinguishes a formal license from a trial.
- Size archive storage from the kept-frame rate after dedupe, not the raw poll rate. An idle rack approaches zero, but a rack under active automation can sustain close to each phone's native rate, up to about 29 kept frames per second on a fast phone, times however many phones are active at once. Budget for the worst case.
- Archived frames contain whatever is on the phone, including signed-in accounts, message previews and one-time codes, so the bucket is as sensitive as the devices. Encrypt it, restrict it, set retention. Exposing metrics means exposing a process that can drive the rack, so scrape the collector and never open 9911 to a remote scraper. A probe firing during an operator's session will visibly move their pointer. And do not read `C:\iMouseXP\IMService.json` for inventory: it holds the account password under trivially reversible obfuscation.

SCALE

Measured headroom: raw BMP capture at p50 2.0 ms when paced at 50 ms, 84 req/s flat out at p50 12.7 ms per phone, 1,620,054 bytes per frame. Phones are independent, two concurrent pollers reaching 83 and 85 req/s at p50 3 ms each, identical to either alone, and ten reaching 512 req/s aggregate at p50 3 ms with a worst p95 of 142 ms. A 30 fps thumbnail wall cost the master poller nothing (17.7 fps at p50 31.9 ms with the wall off against 17.6 fps at p50 32.3 ms with it on), and input and screenshot traffic do not interfere in either direction. The probe costs one reset, one short move (about 1 ms for a 2 px hop) and two 2 ms captures per phone per interval. Today's two phones sit comfortably inside that envelope. Scaling to all 19 boards at a uniform 10 Hz raw poll is an untested extrapolation; a safer default is adaptive rect-cropped polling per phone at 3 to 5 Hz with full frames only on a detected change or a slow archive cadence, which at 19 phones is about 95 req/s for the cheap check plus one 1.6 MB frame only when something actually changed. Archive volume after dedupe is near zero on idle phones and bounded by the source rate under motion, so cap the kept-frame rate per phone.

Further out: the rack as a render lab

The last entry in this part is the one that uses the rack for something other than driving accounts. It is worth reading even if you never build it, because it is the clearest case where the hardware's real advantage, genuine iOS devices with a live video channel, is the product rather than a means to one.

Campaign send to on-device render proof to CRM event loop

PLUS EXTERNAL TOOLING

WITH CAVEATS

A webhook from the marketing platform reaches an n8n or gateway bridge, which drives a small QA group of real iPhones: open the tracked link in Safari through the Shortcut, or open Mail and Messages by tapping registry coordinates, capture the rendered screen as raw BMP, scroll below the fold, then upload the frames and post a "rendered on iPhone XR, iOS 18.7.10" event with image URLs back to the CRM profile or a Slack channel.

Emulator-based inbox previews do not reproduce iOS Mail dark mode, Messages link-preview rendering or Safari landing-page behavior on a real 414 x 896 XR. The rack already owns the two things such a service sells. Every rack-side call is a documented XP endpoint on 127.0.0.1:9911; the CRM, the bridge and the object store are entirely outside the kernel, which never holds a CRM credential.

HOW

1. `get_group_list / get_device_list` XP `/device/group/get-dev` — ingest the send event and resolve the QA target set, reading `imgw` and `imgh` per phone, because every later coordinate is a mirror pixel (500 x 1080) and not the 414 x 896 in `width` and `height`
2. `mouse_reset_pos` XP `/mouse/reset` — clear the stuck-click state across the whole target set before any tap; measured, `/mouse/up` does not clear it and reset does, and it stays cleared
3. `send_key fn_key=WIN+h` XP `/key/sendkey` — normalize every phone to home page one with two presses; requires Full Keyboard Access on the phone, which is invisible to the API
4. `shortcut#13` XP `/shortcut/exec/url` — web channel: open the tracked landing page, with `outtime` in milliseconds (default 15000) rather than the seconds Phonefarm used; the only step that depends on the iMouse Shortcut being installed and bound
5. `click` XP `/mouse/click` — email and SMS channel: open Mail or Messages and the top message row from registry coordinates, `button` numeric, `time=<ms>` for a long press
6. `mouse_wheel / swipe` XP `/mouse/wheel, /mouse/swipe` — scroll below the fold; the wheel takes direction, `len` 1 to 127 and `number`, and moves no pointer, so prefer it, while swipe takes `len` as a fraction plus stepping (spelled "steping" in the doc's JSON), `step_sleep` and `brake`
7. `get_device_screenshot` XP `/pic/screenshot` — capture the rendered frame, one request per phone run concurrently, `binary=true` giving raw bytes with no JSON at 2.0 ms and exactly 1,620,054 bytes, with `rect`, `jpg` and `save_path` available
8. `send_key fn_key + click` XP `/key/sendkey fn_key=ControlBar, /mouse/click` — dark and light variants; there is no API for the iOS appearance setting, so reaching dark mode still means a long press on the brightness slider then a tap on the tile, both from the registry
9. `get_device_screenshot (x2)` XP `/pic/screenshot before and after` — verify something actually rendered using mean absolute pixel difference, never MD5, and budget for the source frame rate before claiming proof
10. `send_key fn_key=WIN+h` XP `/key/sendkey, /shortcut/clipboard/set` — clean up and post the CRM event; uploads and the CRM write are entirely external, and credentials live in the bridge because the kernel has no auth and no secret store

NEEDS

- A webhook bridge plus a token-scoped gateway in front of 9911. The kernel has no authentication of its own, so the bridge is also the access control.
- Object storage for the frames and CRM API credentials, both held by the bridge and never by IMService.
- IMConsole.exe running and logged in.
- Per phone: the board bound with vid and pid set together, one device per binding call, board state 2, a non-empty `location_crc` (B0DBD800 here, a shared-library profile), and device state non-zero with a live `air_pid`.
- Phone-side accessibility settings, none of them queryable: AssistiveTouch at maximum tracking sensitivity, Full Keyboard Access on, tracking speed at maximum, Auto-Lock Never, portrait lock.
- For the URL channel only: the iMouse Shortcut installed, kernel IP edited in, run once with all permissions granted, bound to an AssistiveTouch custom key, and the phone able to reach 9911.

- A test mailbox and number on the QA phones, enrolled in the CRM's own test segment, on accounts the operator owns.
- A per-model, per-iOS coordinate registry for Mail and Messages, maintained by hand, because no server-side OCR or template matching is available here.

WATCH OUT

- The rack-side half is fully native XP. The loop only closes with external pieces the kernel cannot provide.
- The Safari channel depends on the Shortcut, which is not confirmed installed here. The cheapest single probe is `/shortcut/device/ip?id=<id>&state=0`, which should return `data.code 0` with `data.text` equal to the device record's IP; `data.code 30` means the hotkey never launched the Shortcut.
- The Mail and Messages channels have no element lookup at all on this rig. Locating the top message row means fixed registry coordinates or template matching written into your own client. Installing the vendor OCR and OpenCV plug-ins, and setting `ocr_num` and `opencv_num`, which need a kernel restart, would move this to `/pic/find-text` or `/pic/find-image-cv`.
- Mail's inbox row position shifts with unread state, and opening a message marks it read, so the "top row" template drifts between runs. Prefer the tracked-URL channel where the campaign offers one.
- One reset at job start is not enough. The stuck-click state can occur at any point, so a later tap, the Mail or Messages tap or the long press on the brightness slider, can silently no-op and be misreported as "rendered". Reset, or at minimum diff, immediately before every tap-critical step.
- Verification inherits the AirPlay asymmetry: on the 4.5 fps XR a before-and-after pair needs about 450 ms or more just for a second fresh frame to exist, before any real render time. A short timeout tuned to the fast phone produces false "nothing rendered" verdicts on the slow one.
- The wheel avoids the pointer walk, but the taps and the long press do not, so chaining several 400 px or longer moves inside one job adds hundreds of milliseconds of board time on top of the settle wait.
- Dark-mode toggling has no API and depends on a long press then a tap on a slider whose position can shift with iOS version and layout. Treat dedicated per-mode phones, one permanently light and one permanently dark, as the default design rather than the fallback.
- `/pic/screenshot` accepts one id, there is no multi-device capture, and there is no push frame stream in XP; the Phonefarm `loop_device_screenshot` binary protocol has no equivalent here.
- Screenshots of a rendered marketing email or SMS contain real recipient data: subject lines, personalization, numbers in Messages threads. Treat the bucket as sensitive, set retention, and do not reuse a shared test mailbox or number, or prior test data leaks into new captures.
- Repeatedly hitting the same tracked link from the same LAN address on a fixed cadence can trip the destination's own bot and rate-limit heuristics even though the devices are genuine. That is a real detection risk distinct from the terms-of-service argument for using owned devices and accounts, and the correct response to a rate limit or a block page is to stop the job and surface it.
- Never use `jpg=true` in the loop. It costs 40.5 ms and caps at roughly 21 req/s; encode client-side.
- The two failure modes that masquerade as "the campaign broke" are the silent stuck-click state, fixed only by `/mouse/reset`, and a phone-side accessibility setting being off. Neither is visible over the API, so a per-phone input self-test belongs in the job's preflight.

SCALE

Per phone the wall-clock cost is app launch and render settle measured in seconds, not API time: clicks and key presses are sub-millisecond to a few milliseconds, a raw capture is 2 ms, and a wheel scroll is trivial. A realistic light-plus-dark render-proof cycle, normalize, open the channel, scroll, capture, verify, toggle appearance, capture, verify, clean up, is roughly 10 to 20 s on the fast phone and 20 to 40 s or more on the slow one once

verification waits are budgeted honestly, giving on the order of 100 to 300 render jobs per hour per phone. Frame capture does not contend: two pollers flat out gave 83 and 85 req/s each, ten gave 512 req/s aggregate at p50 3 ms, and input and screenshot traffic were measured not to interfere. Concurrency across phones is not kernel-limited, but concurrent AirPlay video for more than two phones is unverified on this rig, so treat phones per PC for this workflow as unproven above two until it is tested. Mirroring a click to N phones is one request that the board walks in parallel, but only where the layout is genuinely identical.

Hardening the control plane

Everything else in this section assumes something is standing between the world and port 9911. Nothing in the kernel provides that, so it is the first thing to build and the last thing to skip.

Security posture for an unauthenticated control plane, with emergency stop

PLUS EXTERNAL TOOLING

WITH CAVEATS

The XP kernel accepts any HTTP or WebSocket caller on 9911 with no authentication of any kind. There is no auth setting to turn on and no vendor fix, so every control lives at the network and gateway layer: phones and rack PC on a dedicated VLAN, Windows Firewall allowing 9911 only from localhost and one gateway host, and every human or machine call routed through a token-scoped gateway that keeps an append-only audit ledger and denies destructive endpoints by default. Bolted on top is a one-verb emergency stop that releases held keys, releases held mouse buttons, resets every pointer, and optionally blinds remote viewers, each as a single comma-list request for the whole rack. The `mdns_type` and `allow_ip_list` settings gate AirPlay discovery only, not the API, so they are defence in depth and nothing more.

HOW

1. `none` XP `no API involved`. Network isolation. Phones and rack PC on their own VLAN or SSID with no route from the office network, with the gateway host the only peer permitted to reach 9911. This is the only control the kernel cannot ignore.
2. `none` XP `host firewall, ports confirmed from the docs and the live probe`. Inbound 9911/TCP from 127.0.0.1 and the gateway IP only. That single rule covers both the HTTP API and the WebSocket, because XP serves `/api` for both on the same port. Allow 17000/TCP (`air_play_port` , default 17000 in `/config/imserver/get` , matching the live `AirplayMirroring.exe` listener) and 5353/UDP mDNS from the phone subnet only.
3. `none` XP `/config/imserver/set?mdns_type=1&allow_ip_list=["192.168.4.68","192.168.4.27"]`. Narrow screencast discovery. Documented as "Screencast Discovery Rules" with `allow_ip_list` taking effect only when the rule is 1. Both are in the no-restart set, so they apply live and broadcast `im_config_change` to other clients.
4. `(policy layer, no kernel support)` XP `deny by default: /device/del, /device/group/del, /shortcut/album/clear, /shortcut/album/del, /shortcut/file/del, /config/user/logout, /imserver/restart, /device/restart, /shortcut/switch/device, /device/usb/restart`. All of these exist, which is exactly why they need a gate. `/shortcut/album/clear` accepts a comma id list, so one request can wipe an album on every phone in the rack. `/shortcut/switch/device?state=0` shuts a phone down on iOS 15 and later. The kernel offers no scopes, roles or rate limits at all.
5. `dev_delete / group_delete` XP `WS callbacks dev_delete, group_delete, dev_change, im_config_change, logout, im_log, error_push, dev_sort_change on ws://127.0.0.1:9911/api`. Audit ledger. The kernel broadcasts these to every connected client when any client deletes a device, changes settings or takes one

offline, which is what makes a second vendor console on the LAN visible to the ledger even though it bypasses the gateway entirely. Record token, verb, ids, msgid, args, resulting `data.code` and before and after frame hashes for gateway calls, plus every broadcast for everything else.

6. `key_release_all` XP `/key/upall?id=<every id, comma separated>`. Emergency stop, keyboard. Releases all held keys, comma id list, no other parameters. One request for the whole rack.
7. `mouse_up` XP `/mouse/up?id=<all>&button=1`. Emergency stop, mouse buttons. `button` is documented as "1 left button, 2 right button, 3 middle button, 4, 5, 6, 7, 8 button", defaulting to left when empty. There is no `/mouse/upall` counterpart to `/key/upall`, so plan for one call per button your automation presses until the list behaviour is tested (see Watch out).
8. `mouse_reset_pos` XP `/mouse/reset?id=<all>`. Emergency stop, pointer state. Comma id list. Measured on this rig, this is the only call that clears the silent stuck-click state, and `/mouse/up` does not, so reset is the load-bearing step of the stop rather than an afterthought. It also returns the pointer to 0,0.
9. `discon_airplay` XP `/device/airplay/disconnect?id=<all>`. Optional: cut the video channel. Comma id list. Blinds anyone watching a mirror without touching the USB control channel, because the two channels are independent. Recovery is `/device/airplay/connect?id=<ids>` or `/device/airplay/connect/all`, with progress on the `airplay_connect_log` callback. If phones cannot see the receiver afterwards, `/imserver/regmdns` re-broadcasts it.
10. `restart_usb` XP `/device/usb/restart?id=720A|C241[,720A|C214]`. Harder stop for a board that will not behave. Note the addressing difference from the Phonefarm edition: XP takes `vid|pid`, not the phone's `deviceid` and not the 8-character `uid`. Read vid and pid from `/device/get` or `/config/usb/get`. Expect `usb_change` callbacks with code 20, then 19, then 12.
11. `none` XP `/config/user/login, do NOT call from an integration`. Secrets stay with the operator. This endpoint takes phone, password and utag in plaintext over an unauthenticated local API. Licence login belongs to the operator at the console over an operator-only path, never to a gateway or a script.

NEEDS

- Administrative control of the LAN or VLAN and of Windows Firewall on the rack PC.
- The token-scoped gateway process, the same one fronting the other integrations, with an append-only ledger store.
- A WebSocket client inside the gateway holding `ws://127.0.0.1:9911/api`, so out-of-band kernel broadcasts land in the ledger.
- An operator-only path, console or RDP, for licence login, kernel restarts and anything on the deny-list.
- An inventory of every host that legitimately needs 9911, ideally exactly one.

WATCH OUT

- The emergency stop is three or four requests, not three. `/key/upall`, then `/mouse/up` once per button held, then `/mouse/reset`, plus `/device/airplay/disconnect` if you want to blind viewers.
- The one-call-per-button reading is conservative but not confirmed either way. The `button` field says "up to 8 keys are supported", the same wording the `id` field uses for comma-list semantics elsewhere in the same document, and no example shows whether `button` accepts a list. Test `/mouse/up?id=<all>&button=1,2` against a phone with both buttons held before finalising the script. If it releases both, the stop is one call shorter and you avoid maintaining an open-ended list of every button the automation might ever press.
- Order the stop as `key/upall`, then `mouse/up` per button, then `mouse/reset`. That order is correct because `/mouse/up` does not clear the stuck-click state and `/mouse/reset` does. State plainly in the design that `/mouse/reset`'s effect on a currently held-down button is unverified: the rig data confirms reset clears the silent

stuck-click state and returns the pointer to 0,0, not that it releases a physically held mouse-down. Up before reset is the safe sequence for exactly that reason.

- Nothing cancels an in-flight gesture. The board walks the pointer at roughly 0.6 ms per pixel, so a swipe or long move already dispatched keeps consuming board time for up to about 592 ms on a 1000 px stroke after the stop calls return 200. The emergency stop has a soft tail latency, not an instant cutoff, on whichever phone had a gesture in flight. This applies to any in-flight `/mouse/move`, not only to `/mouse/swipe`.
- `/device/usb/restart` is not part of the fast stop path. Each restart triggers a full re-enumeration through `usb_change` codes 20, 19 and 12 before the board is control-ready, which takes seconds. Keep it in a separate "hard reset a misbehaving board" runbook.
- `mdns_type` and `allow_ip_list` are screencast discovery rules, not API access control. The document's own description of values 1 and 2 is self-contradictory, so verify the behaviour on this build before relying on either.
- Any use of `/shortcut/*` requires the phones themselves to reach 9911, which re-exposes the unauthenticated API to the phone subnet. Either accept that with a tightly scoped per-IP rule, or forgo the shortcut functions.
- A second vendor console anywhere on the reachable subnet bypasses the gateway completely. The firewall rule is the only real control. The broadcast callbacks merely make the bypass visible after the fact.
- Match every response on `data.code`, never on message text. The kernel replies in Chinese, guessed paths return a 404 with `接口不存在` rather than a structured error, and Phonefarm-style function names return HTTP 500 "List index out of bounds".
- `/imserver/restart` and `/config/user/logout` are each one unauthenticated GET away from taking the whole rack offline. Keep both admin-scoped and expect the WebSocket to drop and every phone to need re-casting after a restart.
- A VLAN posture assumes physical and network control the automation layer cannot verify. No endpoint confirms the rack is actually isolated, so the ledger can prove what happened through the gateway but never that no other path to 9911 exists. That is inherent to an API with no authentication, not a gap in the design.
- Screenshots expose logged-in accounts and message contents. `/shortcut/album/clear` and the file and album delete calls are irreversible from the API's side. Several deny-list calls destroy data, so require an explicit admin scope plus an operator confirmation and never put them behind a single-click button. `C:\iMouseXP\IMService.json` stores the account password under reversible hex obfuscation: no tool should read it, print it, copy the scheme, or let a build carry it forward.

SCALE

Firewall rules and a loopback gateway hop add no measurable latency, roughly 1 ms on loopback against a 2 ms raw capture and sub-millisecond input calls. The emergency stop is a fixed three to four comma-list requests regardless of phone count, because every input endpoint accepts an id list and the board walks all listed phones in parallel, so resetting 19 phones costs what resetting one costs. Each request is sub-10 ms on loopback given the measured input latencies of 0.5 to 11.7 ms p50 for small moves, so the request layer completes in well under 100 ms for the whole rack. The real bound is the uncancellable gesture tail of up to about 592 ms, and seconds more if a board restart is invoked. The exception to the flat cost is `/device/usb/restart`, which is addressed per board as `vid|pid`. Ledger volume is a few hundred rows per job plus the broadcast stream, which is low-rate except for `user_info` every 30 s.

Pricing the work before you run it

The rig's timing is dominated by two things that are invisible to an HTTP latency histogram: pointer distance and per-phone AirPlay frame production. A planning model that knows both is worth more than a faster client.

Throughput budget calculator built from the rig's measured constants

NATIVE API · WITH CAVEATS

A client-side planning function that turns a job description, meaning step count, pointer distance per step, target phone set and verification policy, into a wall-clock estimate plus a kernel-load figure. It uses only constants measured on this rig: raw BMP capture at p50 2.0 ms when paced (12.7 ms and 84 req/s flat out, 1,620,054 bytes per frame), the JPEG variant at p50 40.5 ms with a hard cap near 21 req/s, the board's roughly 0.6 ms per pixel pointer walk, per-phone new-frame rate (28.9 fps on 192.168.4.68 against 4.5 fps on 192.168.4.27), and the fact that a comma-separated id list costs the longest single move rather than the sum. The model emits wall-clock per phone, rack wall-clock, requests per second, MB/s of frame traffic, and names the slowest phone from the device record's `name` and `user_name` fields.

HOW

1. `get_device_list` XP `GET /api/device/get (optional id=a,b comma list)`. Returns `data.list[]` with `img w` / `imgh` (500 x 1080 on these XRs, the space every distance in the model must use), `width` / `height` (414 x 896 logical points, not the input space), `state` (0 means offline), `air_pid` (a live AirplayMirroring PID is proof of mirroring), and the two name fields for the slow-phone callout.
2. `mouse_move` XP `GET /api/mouse/move?id=<ids>&x=&y=`. Drive continuous motion so the frame-rate calibration measures something. Alternate two points 200 px apart, measured at 133 ms per move and identical for one phone or both. A still screen produces almost no new frames, so calibrating an idle phone measures nothing.
3. `get_device_screenshot` XP `GET /api/pic/screenshot?id=<id>&binary=true`. Poll raw frames for 5 s per phone to derive that phone's fps. The document states id "Multiple is not allowed", so this is one request per phone. Run one poller per phone concurrently: measured 83 and 85 req/s for two, 512 req/s total for ten. Do not add `jpg=true`, which costs p50 40.5 ms and caps near 21 req/s because the kernel re-encodes.
4. `none (client-side analysis)` XP `none, the XP kernel exposes no frame-change or fps field`. Count new frames by mean absolute pixel difference, not byte equality. Threshold above 1 on the 0 to 255 scale: idle measures 0.02 to 0.03, a real reaction 20 to 160. MD5 or byte comparison reports "changed" on an untouched phone and has already produced one wrong conclusion in this project.
5. `click` XP `GET /api/mouse/click?id=<ids>&x=&y=&button=1&time=<ms>`. Price one step. `time` is the delay between pressing and popping up, in milliseconds, handled internally if omitted, and that is the hold term. t_step = 0.6 ms x distance_px + hold + settle + captures. Use 2.0 ms per capture when the poller is paced near 50 ms and 12.7 ms when it is saturating.
6. `click / swipe / mouse_move with devlist` XP `same endpoints with id=<id1>,<id2>,...`. Price the fan-out as a maximum, not a sum. Measured, a 1000 px move costs 592 ms for one phone and 594 ms for both. Frame capture is the exception, single id only, so N phones means N concurrent pollers.
7. `mouse_reset_pos` XP `GET /api/mouse/reset?id=<ids>`. Add a pointer-reset allowance whenever the target set changes. One request, comma list accepted. A phone can silently stop honouring `/mouse/click` while `/mouse/move` and hotkeys still work; `/mouse/up` does not clear it and `/mouse/reset` does, and it stays cleared. Budget one reset per target-set change and before any tap-critical sequence.
8. `loop_device_screenshot (Phonefarm push stream)` XP `none, XP documents no server-push frame stream`. Sum kernel load and frame bandwidth. This absence is why requests per second is a real budget line on this rig: there is no push channel to amortise, every frame is a poll. Load is the sum over pollers of 1 divided by

poll period, with a measured ceiling of 512 req/s at ten pollers (p50 3 ms, worst p95 142 ms). Bandwidth is req/s multiplied by 1.62 MB.

NEEDS

- `IMConsole.exe` running and logged in. With only `IMService.exe` up, `/device/get` reported both phones in state 0 and `/config/usb/get` returned an empty board list.
- A per-phone fps calibration run, or a live frame-rate meter, before any estimate is trusted. The two XRs differ by about six times.
- Per-step pointer distances known in the target phone's `imgw` x `imgh` space, measured from the previous step's end point, not from iOS logical points.
- One reusable 1.62 MB buffer per poller. Without reuse the measured cost was 488 MB/s of allocation and 496 gen-2 collections in 8 s.

WATCH OUT

- The "fan-out costs max, not sum" rule only holds when every listed phone receives the same x,y in its own `imgw` x `imgh` space, meaning a genuinely mirrored action. It does not generalise to the common pattern where each phone runs its own script against its own screen content: on this rig the same coordinate is Clock on one XR and Reminders on the other. For that pattern the model must cost N separate requests, one id each, because the coordinates differ. Expose both modes explicitly, mirrored fan-out and independent per-phone, rather than defaulting to the cheap max formula.
- The `time` parameter on `/mouse/click` is documented only as handled internally if omitted, with no stated default duration. The t_step formula treats hold as a small known constant, so until the internal default is measured, which could be tens or hundreds of milliseconds, any estimate that omits an explicit `time=` value is assumed rather than verified. Call that out in the output.
- Frame-count settle rules are heuristics, not measured constants. Nothing in the XP docs or the rig measurements defines a settle time. Every settle wait needs a hard wall-clock ceiling, because a step that does not change the screen produces no new frames and the wait would never terminate.
- AirPlay frame delivery is motion and content dependent, so inter-frame gaps on the 4.5 fps phone are not a steady 222 ms. They cluster and stall. Treat "3 frames, roughly 660 ms" as a best-case average, not a bound; real settle time on the slow phone can run into seconds in the tail.
- Frame capture cannot be fanned out. `/pic/screenshot` documents "Multiple is not allowed", so verification cost scales linearly with phone count even though input cost does not. Any verification-heavy plan across more than a couple of phones is bandwidth-bound on raw BMP well before it is CPU or board bound, so surface MB/s alongside req/s rather than letting a 19-board plan silently assume loopback bandwidth a remote client will not have.
- Per-phone fps is recalibrated daily in this design, so a phone that degrades mid-session through Wi-Fi interference or thermal throttling silently invalidates the estimate until the next pass. Flag calibration age in the output, not only at input time. The worked example already shows one slow link tripling a verified job's wall clock against its identical twin.
- The rig has 19 boards but only 2 phones mirroring. Any example sized at 19 pollers is hypothetical; the aggregate poller floor beyond ten concurrent pollers is listed as unverified in the project handoff.
- Every constant is from this rig: two iPhone XRs on iOS 18.7.10, one board generation, loopback HTTP. The 0.6 ms per pixel walk is a property of this board and firmware and must be re-measured on other hardware. The 84 req/s and 512 req/s ceilings are loopback figures and will not hold over a LAN link.
- The model estimates time, not success. It cannot predict a stuck pointer, an unsatisfied iOS accessibility setting (none are queryable over the API), or a licence cap. Those appear as repair-queue allowances against

an assumed failure rate, which is an input, not a measurement. Publishing an estimate as a commitment is the business risk: the numbers are honest about the rig and silent about the app being driven.

- Calibration frames are live screenshots and may contain account content. Do not archive them by default, and remember that with no authentication on 9911 any LAN host can read the same frames this tool polls.

SCALE

Worked example on this rig, a 12-step verified job averaging 400 px of pointer travel per step: fast XR, 12 x (240 ms walk plus about 104 ms settle at 28.9 fps plus 4 ms capture), roughly 4.2 s; slow XR, 12 x (240 plus about 667 plus 4), roughly 10.9 s; mirrored to both in one comma-separated id list, the rack wall clock is the slower phone's 10.9 s, not the sum. In actions per hour that is about 350 ms per verified step on the fast XR, roughly 10,300 verified actions per hour per phone, against about 911 ms and roughly 3,950 per hour on the slow twin, with a mirrored pair bounded by the slow phone at about 3,950. Open-loop, with no screen verification, a 400 px step costs 240 ms, roughly 15,000 actions per hour per phone, and N mirrored phones cost the same as one. That parallelism does not extend to independently scripted phones, where wall clock across phones is limited by the client's concurrent request capacity and the 512 req/s poller ceiling rather than by the board. Frame load: each raw poll is 1,620,054 bytes, so ten pollers at 10 Hz is 100 req/s and about 162 MB/s on loopback, inside the measured 512 req/s ceiling; the same ten on JPEG would hit the 21 req/s per-stream cap at 40.5 ms each. Input traffic and frame polling were measured not to interfere, 98.8 req/s with `/mouse/move` streaming against 78.7 without, so the two budgets add independently.

Capturing operator work and replaying it

With no OCR or image-search plug-in connected on this rig, a recorded pixel signature is the only screen-side verification available without adding software. That constraint turns out to shape the whole design.

Record-and-replay macro compiler with per-step pixel signatures

NATIVE API WITH CAVEATS

Record an operator's session on the master phone, meaning real `/mouse/down`, streamed and coalesced `/mouse/move`, `/mouse/up`, discrete `/mouse/click` and `/key/sendkey` hotkeys, capturing a raw BMP frame immediately before and after each step. Compile that into a job in which every step carries its primitive, its arguments already in the master's `imgw` x `imgh` mirror-pixel space, its pointer distance from the previous end point, the expected whole-frame mean absolute pixel difference, the bounding box of the region that actually changed, and a reference crop of that box. Replay onto phones whose `imgw`, `imgh` and `rotate` match the master, addressed as one comma-separated id list per step, then verify each step per phone against the recorded signature rather than a generic "something changed". Recording also solves the classic coordinate trap for free, because iOS logical points (414 x 896) are not the input space (500 x 1080 mirror pixels) and a recorded session is in the right space by construction.

HOW

1. `get_device_list / get_group_list` XP `GET /api/device/get, GET /api/device/group/get, GET /api/device/group/get-dev?id=<gid> (0 = ungrouped)`. Resolve the replay set and confirm it is geometrically comparable. `get-dev` is XP-only and returns full device records. Filter on identical `imgw` / `imgh` and `rotate`: a coordinate on a different-sized screen is a different place. On this rig `/device/group/get` is empty and both XRs are gname `未分组`, so group-based targeting needs groups created first with `/device/group/set?id=0&name=...`, and until then `get-dev?id=0` is the only populated list.
2. `mouse_reset_pos + send_key fn_key WIN+h` XP `GET /api/mouse/reset?id=<ids> then GET /api/key/sendkey?id=<ids>&fn_key=WIN+h twice`. Normalise every phone before recording and before replay.

WIN+h is the documented Home hotkey for these phones; the distinct `Home` fn_key exists but needs kernel 2026.2.27 or later and targets iOS 26.1 or later, and these XRs are on 18.7.10. Two presses: the first leaves the app, the second returns to page one. The reset is not optional, being the only measured cure for the silent stuck-click state.

3. `click / mouse_down / mouse_move / mouse_up / send_key` XP `/mouse/click (button 1–8, x, y, time ms), /mouse/down, /mouse/move, /mouse/up, /key/sendkey (key ASCII only, or fn_key)`. Record the primitives as the operator performs them. Hook the console's existing gesture stages, `BeginGesture`, `TrackGesture` and `EndGestureAsync`, which already take plain points, plus `InputTargetIds()`. Record a real down, move, up stroke rather than a canned `/mouse/swipe`, so the path is reproduced instead of approximated.
4. `get_device_screenshot (binary)` XP `GET /api/pic/screenshot?id=<id>&binary=true (optionally &rect=[l,t,r,b])`. Capture before and after frames for the signature. Single id only, 1,620,054-byte bottom-up BMP, p50 2 ms paced. XP's `rect` is a flat left, top, right, bottom array, not Phonefarm's four corner points, and can crop the reference region server-side; `save_path` can write the crop straight to disk.
5. `none (client-side compiler)` XP `none, no XP endpoint stores or executes a macro`. Compile each step into primitive, arguments, distance, expected diff, bbox and crop. The kernel has no macro, schedule or job concept in either edition, so the compiler, the store and the runner are all the console's responsibility.
6. `devlist fan-out` XP `same mouse and key endpoints with id=<id1>,<id2>,...`. Replay each step to the whole set in one request. The board walks all listed phones in parallel, a 1000 px move measured 592 ms for one phone and 594 ms for two. Coalescing intermediate move points is free: the same stroke sent as 14, 7 or 2 points produced identical reactions.
7. `none (client-side comparison); ocr / find_image would be the alternative` XP `/pic/ocr and /pic/find-image exist but return data.code 28 插件未连接 on this rig`. Verify per phone against the recorded signature. Two-part test: whole-frame mean absolute difference within a tolerance band of the recorded value, for example 141.96, and crop diff inside the recorded bbox below a calibrated threshold. Idle floor is 0.02 to 0.03, a real reaction 20 to 160. Exclude the status-bar band from every bbox, because the clock changes on its own.
8. `mouse_reset_pos / restart_usb / auto_connect_screen` XP `/mouse/reset?id=, /device/usb/restart?id=720A|C241, /device/airplay/connect?id= (progress on the airplay_connect_log callback)`. Handle divergence without stalling the rest of the set. Retry the single failed step in place first, meaning reset, re-issue that exact step, re-verify, since reset is the documented cure for this failure mode. Only then drop the phone from the id list, archive its frame, and escalate to the board (addressed `vid|pid`, not `deviceid`) and then the mirror session. Watch `error_push{call_fun}` on the WebSocket bus for board-level failures the synchronous reply does not report.

NEEDS

- The 2.0 console's gesture path (`BeginGesture`, `TrackGesture`, `EndGestureAsync`) as the recording source, and `InputTargetIds()` for the fan-out list.
- A homogeneous replay set: same model, same iOS, same `imgw` / `imgh`, same `rotate`, and genuinely the same on-screen layout. The two XRs in this rack have different home screens and wallpapers, so the same coordinate is Clock on one and Reminders on the other.
- Recording performed on a normalised phone (`/mouse/reset` then two WIN+h presses) with Auto-Lock Never and portrait orientation locked.
- Each phone's iOS accessibility prerequisites satisfied: AssistiveTouch on with tracking sensitivity max, Full Keyboard Access on, Trackpad and Mouse tracking speed max. None of these is queryable over the API, and a phone can mirror perfectly while ignoring every input.

- A per-follower calibration check. A board with an empty or mismatched mouse-parameter library returns `data.code` 11 "No mouse parameters" on every mouse call, not a diff divergence. Add a `/device/get location_crc` check, or an explicit `/device/collection/mouse` capture, to the pre-flight, or a code 11 reply will be misread as "the tap did not land" and trigger the wrong recovery.
- `IMConsole.exe` running. With only `IMService.exe` up, both phones read state 0 and the board list was empty.

WATCH OUT

- The recording step is inherently a manual operator session. The value comes from a human performing the task once, so this is a human-in-the-loop tool, not an unattended one.
- Two raw captures per step undercounts verification cost. Because a still screen yields almost no new AirPlay frames, a single post-action poll can return bytes identical to the pre-action frame even though the tap landed, especially on the 4.5 fps twin. The compiler needs a bounded retry-poll loop sized to each phone's measured fps, polling at roughly one over fps with a timeout of a small multiple of that, not a single grab and diff.
- Resetting once at the start of a run is not enough. The stuck-click state was observed appearing mid-session with no known trigger, and `/mouse/up` does not clear it.
- Carry the 0.6 ms per pixel pointer-walk time as a scheduling constraint, not just as metadata. If the runner fires the next step before pointer walk plus settle plus verify window has elapsed, queued moves coalesce to the newest position, which was measured to happen silently and can merge a recorded double tap or a deliberate two-step drag into a single motion on replay.
- Any step that types non-ASCII text cannot be recorded as `/key/sendkey`, which is documented ASCII-only in both editions. The XP route is `/shortcut/clipboard/set` with text, sleep and outtime, followed by `/key/sendkey fn_key=WIN+v`, and that requires the iMouse iOS Shortcut installed, IP-edited and bound to an AssistiveTouch custom key on every phone, which is not verified present on this rig. A macro containing Unicode typing is therefore not natively replayable here.
- The verification thresholds are unvalidated. "Within plus or minus 50 percent of the expected whole-frame diff" and "crop diff below about 10" are proposals, not measurements. The only measured anchors are an idle floor of 0.02 to 0.03 and real reactions of 20 to 160. Calibrate per macro against an idle control, or the tool will manufacture both false divergences and false passes.
- Signatures encode the recording phone's wallpaper and theme. Keep bounding boxes tight around the element that actually changed and exclude the status-bar band; a whole-frame signature will not transfer between the two XRs at all.
- Screen-side verification on the slow phone is expensive: 4.5 new frames per second against 28.9 on its twin, so any settle-by-frames wait needs a wall-clock ceiling.
- A near full-height 1080 px stroke is about 580 ms of pointer travel before `/mouse/up` can even be sent, which iOS may read as a slow drag rather than a flick. Velocity-dependent gestures recorded by hand may simply not fire on replay, and replay timing for such steps will not match the human gesture's real duration. Prefer short-axis strokes.
- App updates invalidate a macro silently: the coordinates still land, the signature diverges, and nothing explains why. Store the app version and the recording date with every macro.
- No server-side recognition is available as a fallback. `/pic/ocr` answered `data.code` 28 on this rig and the image-search plug-in is presumed absent too, so if pixel signatures prove too brittle for a given app there is no documented alternative short of installing the vendor plug-in.

- Verification does not fan out the way input does. N phones need N concurrent HTTP pollers against `/pic/screenshot`, measured comfortable to ten pollers at about 512 req/s aggregate and p50 3 ms. Scaling the replay set well past that needs the poller pool explicitly sized and paced, even though the mouse and key relay stays one request per step.
- Recorded frames and reference crops are screenshots of live, possibly logged-in sessions, and the 9911 API has no authentication protecting the files or the console that stores them. Local-only storage and scrubbing before any sharing is a hard requirement, and never publish a macro bundle that still contains full frames.
- Replaying the same recorded interaction across many phones against a third-party app is exactly the kind of automated use many consumer app terms of service restrict, and platforms detect and may action it. Whether a given macro is permissible depends on that app's terms, and that judgement belongs to the operator before the macro is built, not to the tool.

SCALE

Recording adds two raw captures per step, about 2 ms each when paced and 1.62 MB each, so keep only the diff bbox crop, which is kilobytes. Replay costs one request per step regardless of set size, with per-step wall clock equal to the longest single pointer walk plus hold plus settle. On the fast phone that is roughly 30 to 150 ms of walk for typical short taps and drags, plus about 35 ms of settle and one to three verify polls, so 100 to 300 ms per step: a ceiling near 12,000 to 36,000 steps per hour and realistically 3,000 to 8,000 actions per hour per phone once retries and periodic resets are budgeted. On the 4.5 fps twin the verify wait alone is about 220 ms per new frame and often needs two or three polls to escape a still-screen run, 440 to 660 ms, capping practical throughput near 1,000 to 2,000 actions per hour per phone. A macro dominated by full-height swipes at 580 ms of travel each caps even the fast phone near 4,000 to 6,000 such gestures per hour. Verification does not fan out, so N phones means N concurrent pollers, measured comfortable at ten. A compiled job is portable to another PC running the same phone model, because the coordinates are in that model's mirror-pixel space, but not to a different `imgw` / `imgh`.

Past the boundary of one PC

The comma-id fan-out that makes mirroring cheap stops at the PC boundary. This is the one pattern that carries it across, and it is also the one with the least measurement behind it.

Cross-PC follow-the-master input relay over XP kernel WebSockets

NATIVE API · WITH CAVEATS

The XP kernel accepts the identical JSON envelope `{"fun":"/mouse/click","msgid":n,"data":{...}}` over HTTP POST and over `ws://<host>:9911/api`, because every endpoint page in the XP docs prints the same body with the note that WebSocket can also send the same JSON request. A relay process can therefore hold one WebSocket per kernel PC, take each input the master console emits, rewrite `data.id` to that kernel's eligible follower list, assign a fresh `msgid` per socket, and fan one operator gesture out to phones on several racks. Followers are eligible only when their device record matches the master on `imgw`, `imgh` and `rotate`, `state` is not 0, and their board reports state 2 in `/config/usb/get`. Frames for verification are polled per phone over HTTP on each kernel, because XP has no server-push frame stream, so the WebSocket carries input and callbacks only.

HOW

1. `(ws connect to ws://host:9911/clinet)` XP `ws://<pc>:9911/api`. Open one WebSocket per kernel and wait for the `im_connect` callback, which carries `data.ver`, the only kernel-version readout in XP since

there is no HTTP version endpoint. Note the XP WS path is `/api`, not the Phonefarm `/clinet` typo. `msgid` is echoed verbatim on every reply, which is what makes many in-flight requests on one socket matchable.

2. `get_device_list` XP `/device/get (optional id=a,b filter)`. Prime each kernel's inventory. Returns `data.list[]` with `imgw`, `imgh`, `rotate`, `state`, `vid` / `pid` / `uid`, `location_crc` and `gid`. `imgw` / `imgh` is the coordinate space for every mouse call, 500 x 1080 on this rig's XRs against logical 414 x 896, and a follower whose geometry differs from the master must be excluded because the same coordinate is a different place. Phonefarm has no `imgw` / `imgh` field at all.
3. `get_usb_list` XP `/config/usb/get`. Check the control channel per follower. Returns `data.list[{vid,pid,uid,ver,state}]`, where state 2 means the board is enumerated and a phone is attached. State semantics come from the `usb_change` code table (19 board to PC, 20 disconnected, 12 board to phone) and the measured rig. Do not parse `ver` or `uver`, which are mojibake on this build. `/device/get` state only proves video.
4. `mouse_reset_pos` XP `/mouse/reset?id=<comma list>`. Re-sync the pointer whenever the follower set changes. This is the only call that clears the silent stuck-click state, where a phone obeys `/mouse/move` and hotkeys but produces a 0.03 mean pixel difference on a click. `/mouse/up` does not clear it, and it stays cleared after reset.
5. `click` XP `/mouse/click {id: comma list, x, y, button, time}`. Relay a tap. `button` is numeric, "1" left, "2" right, "3" middle, up to "8", not the Phonefarm "left"/"right" strings. `time` is the press-to-release delay in milliseconds, so a long press is a click with time near 800. Coordinates are mirror-image pixels.
6. `mouse_down / mouse_move / mouse_up` XP `/mouse/down, then repeated /mouse/move {x,y}, then /mouse/up, each with the comma id list`. Relay a real drag. Coalesce queued moves to the newest position per kernel: the board traverses the path regardless, measured with 14 steps, 7 steps and 2 jumps producing identical 26.2 and 22.9 mean pixel change, and latency is roughly 0.6 ms per pixel of distance rather than jitter.
7. `swipe / mouse_wheel` XP `/mouse/swipe {direction up|down|left|right, len (0.9 = 10% to 90%), stepping (spelled steping in the doc's JSON examples), step_sleep ms, brake, sx/sy/ex/ey} and /mouse/wheel {direction, len 1–127, number}`. Relay canned gestures and scrolling. XP adds left and right wheel plus `step_sleep` and `brake` over Phonefarm. Trust the parameter table, not the examples. A full-height 1080 px swipe is about 580 ms of pointer walk, so prefer the 500 px short axis.
8. `send_key` XP `/key/sendkey {id: comma list, key (ASCII only) | fn_key}; /key/down, /key/up, /key/upall for held chords`. Relay keystrokes and system hotkeys. XP's fn_key vocabulary is larger than Phonefarm's: WIN+h, Home (needs kernel 2026.2.27 or later, for iOS 26.1 or later), TAB+b, AppSwitch, ControlBar, NoticeBar, TAB+l, OPENKeyboard, shift+win+3 and +4, CTRL+ALT+SHIFT+WIN+r, CTRL+ALT+WIN+p, WIN+c/v/a/x, SwitchIme, VolumeUp and Down, Play/Pause. Two WIN+h presses normalise every phone to home page one before a mirrored sequence.
9. `none` XP `error_push WS callback, data.call_fun plus data.message`. Catch per-kernel input failures the synchronous reply hides. XP-only, and the only asynchronous signal that an input actually failed at a board, which matters because a comma-list reply carries one `data.code` for the whole list.
10. `dev_connect / dev_disconnect / dev_rotate / usb_change` XP `same callback names on XP but a different envelope: {fun, status:200, message, msgid:0, data:{list:[record], code, id, message}}`. Track membership changes without polling. Records live inside `data.list`, and `usb_change` carries the event type in `data.code`. XP adds `dev_change`, `dev_delete`, `im_log`, `user_info` every 30 s, and `im_config_change`. Rebuild the follower list on `dev_disconnect`, `dev_rotate` (which swaps the meaning of `imgw` / `imgh`) and `usb_change` code 20, then re-send `/mouse/reset` to the new set.

11. `get_device_screenshot` XP `HTTP GET /api/pic/screenshot?id=<single id>&binary=true`. Verify per phone on each kernel. The document says id "Multiple is not allowed", and with `binary=true` no JSON is returned at all. XP documents no binary WebSocket frame protocol, and Phonefarm's `loop_device_screenshot` push stream has no XP equivalent, so frames must be polled over HTTP, one request per phone. Measured: raw BMP p50 2 ms, 1,620,054 bytes, 84 req/s; JPEG p50 40 ms capped near 21 req/s.

NEEDS

- Each kernel PC running the XP kernel with its vendor console (`IMConsole.exe`) started and logged in. Measured here, with only `IMService.exe` running, both phones reported state 0 and `/config/usb/get` returned an empty board list.
- Each kernel's account licensed, checked via `/config/user/info` for `pass` true and `dev_online_num` under `total_license`. Unlicensed devices are refused with the `im_log` push `未登录，不允许上线`. Re-check this after every WebSocket reconnect, because a kernel that drops below its licensed count after a restart refuses new devices with no HTTP-visible error and surfaces it only as an `im_log` push.
- Followers bound to a board, with `vid` and `pid` set together via `/device/set`, one device per binding call, and a non-empty `location_crc`. Verify that `location_crc` at the same pre-flight step as geometry, or every mouse call returns `data.code` 11 "No mouse parameters" regardless of network health and gets misdiagnosed as a relay or WebSocket fault.
- Follower device records matching the master on `imgw`, `imgh` and `rotate`, with phones normalised to the same screen state first. The two XRs on this rig have different home screens and wallpapers, so the same coordinate is a different app on each.
- Per-phone iOS settings, none of which are queryable over the API: AssistiveTouch on with tracking sensitivity max, Full Keyboard Access on, Trackpad and Mouse tracking speed max, Auto-Lock Never, portrait orientation lock.
- A network path from the relay host to each kernel's TCP 9911, opened deliberately at the firewall. The API authenticates nobody, so every kernel the relay can reach is also reachable by anything else on that path.

WATCH OUT

- No XP function publishes the master console's outgoing input stream. The relay works only if you own or instrument the master, meaning your own console emitting a copy of each envelope. There is no API to subscribe to another client's requests, and the WS callbacks broadcast state changes, not other clients' input calls. This is a lawful relay of the operator's own console, not a mechanism to observe or intercept anyone else's session.
- "One gesture becomes one request per kernel" holds only for a discrete `/mouse/click`. A drag is down plus N moves plus up, so one drag becomes 2 plus N relay messages per kernel, and cross-kernel jitter compounds per move message, not once per gesture. Coalesce intermediate moves to the newest position per kernel, which is verified safe on a single kernel, both to bound WAN traffic and because the 0.6 ms per pixel board walk already dominates fine-grained sampling.
- A comma-separated id list returns one `data.code`, `data.id` and `data.message` for the whole list. There is no per-phone acknowledgement anywhere in the XP response shape, so per-phone verification by frame diff stays mandatory and `error_push` is the only asynchronous per-call failure signal.
- Verification gains nothing from the relay. Control and video stay independent per kernel exactly as on one PC, so confirming a tap landed rather than merely sent still requires one HTTP poller per phone against that phone's own kernel. Input traffic scales as total kernels; verification traffic scales as total phones.
- Latency skew is real but unmeasured for this pattern. The 1 to 5 ms per-kernel relay cost is an estimate. What is measured is loopback: `/mouse/move` p50 0.5 to 11.7 ms depending on concurrency, and a distance-

proportional board walk of roughly 0.6 ms per pixel that dominates anything longer than a tap. Taps will look simultaneous; a fast drag on a remote kernel will visibly trail the master.

- Cross-PC operation is untested on this rig, which has exactly one kernel and two phones. The "10 kernels by 50 phones" figure is arithmetic from the comma-list fan-out, not an observation.
- Per-kernel phone count is also gated by Wi-Fi airtime, not only by licence count. The 28.9 against 4.5 fps figures were measured with two phones total on one access point, and per-AP phone density has not been measured in this project, so the real per-kernel ceiling from Wi-Fi contention alone is unproven.
- A dropped WebSocket silently orphans that kernel's followers. Alarm on close and, on reconnect, re-prime `/device/get` and `/config/usb/get`, re-check the licence, and re-send `/mouse/reset` before resuming. A stale follower list sends taps into the wrong coordinate space.
- A WebSocket dropped mid-drag leaves that kernel's phones pointer-down with no `/mouse/up` ever sent. Track outstanding down-state per kernel and issue `/mouse/reset` on reconnect, since `/mouse/up` alone is documented not to clear the related stuck state.
- Kernel restarts via `/imserver/restart` and per-device changes via `/device/airplay/set` drop mirroring sessions. Followers return only after re-casting through `/device/airplay/connect` or the global `auto_connect` flag.
- The stuck-click trap is per phone and invisible: moves and hotkeys keep working while taps do nothing. Send `/mouse/reset` to the whole target set before any tap-critical relayed sequence.
- A relay makes one operator's mistake land on every rack at once, with no undo and no per-phone confirmation. A mis-mapped coordinate taps whatever sits at that spot on each follower. Relayed input also inherits each follower's own session, so verify which accounts are logged in before broadcasting anything that posts, messages or purchases.
- Reaching kernels across PCs means exposing an unauthenticated control port. Anything that can reach the relay's path can drive the phones directly, read frames of logged-in sessions, or restart the kernel. Restrict TCP 9911 to the relay host at the Windows firewall and keep phones on their own VLAN.
- Mirroring identical actions across many accounts can violate a destination platform's terms even when every account is legitimately yours. Pause on any "Action Blocked" or rate-limit screen rather than retrying.

SCALE

Inside one kernel, mirroring to N phones is one request and the board walks all listed phones in parallel: a 1000 px move costs about 592 ms whether it drives one phone or the whole list. Across kernels the relay issues one request per kernel, so ten kernels is ten requests regardless of phones per kernel. One drag gesture becomes roughly (2 plus N_moves) times K WAN messages, so a typical 7 to 14 point drag across ten kernels is about 100 to 160 relay messages; at a 50 to 100 gesture per minute operating pace that is roughly 5,000 to 16,000 relay messages per minute, well inside what loopback measurements suggest a kernel absorbs (512 req/s aggregate across ten concurrent HTTP pollers, with WS framing cheaper still). Verification is the bottleneck, scaling as total phones rather than kernels, with the slowest AirPlay link on any kernel setting that kernel's confirmation floor. Input and frame traffic do not contend: streaming `/mouse/move` to both phones while polling flat out gave 98.8 req/s against 78.7 with no input. A 50-phone, 10-kernel fleet with even one 4.5 fps class phone per kernel is gated at 220 ms or more per settle per phone, so serialised fleet-wide confirmation runs at only about 3 to 4 confirmed gestures per second, while confirmation pipelined per phone and asynchronously is bounded only by each kernel's own board-walk time, not by the relay.

Also worth knowing

x-callback parameters and Personal Automations: the two halves the ceiling analysis missed

NEEDS SHORTCUT WITH CAVEATS

The hotkey-to-Shortcut card above concludes the channel cannot be parameterised and can only be triggered from the PC. Both conclusions are about the vendor's own pre-bound Shortcut, not about the mechanism underneath. iOS exposes two documented ways past both limits: the `shortcuts://x-callback-url/run-shortcut` scheme, which carries a per-call `input` value into a Shortcut and can chain a result back out, and Personal Automations, which bind a Shortcut to a phone-side trigger, time of day, charger state, battery level, an NFC tag, so iOS runs it with no board, no kernel, and no PC involved at all. Both ride entirely on top of the already-documented `/shortcut/exec/url`, clipboard and album endpoints; neither adds a new XP endpoint.

HOW

1. `shortcut#13 (Open URL)` `XP GET /api/shortcut/exec/url?id=<ids>&url=<encoded shortcuts://URL>&outtime=<ms>` — the parameterised trigger. `url` is documented only as a String with no format check, and the same action already used to jump into an app screen can equally hold `shortcuts://x-callback-url/run-shortcut?name=<Shortcut>&input=<text>&x-success=<callback>`. The GET form needs the whole scheme URL-encoded as one query value; the JSON POST form documented alongside it avoids that by sending `url` as a plain string.
2. `none (Shortcut "Shortcut Input" action)` `XP none, on-device` — the target Shortcut reads `input` as its starting value, so a fan-out over a comma id list can no longer carry one shared payload: each phone needs its own `exec/url` call if the input differs per phone.
3. `shortcut#10 (clipboard/set)` `XP POST /api/shortcut/clipboard/set {id, text, sleep}` — an alternative way to seed input when the payload is long or has characters that would not survive two layers of URL-encoding cleanly; the Shortcut reads "Clipboard" instead of "Shortcut Input".
4. `none (Shortcut "Output" / "Stop and Output")` `XP none, on-device` — the callback half. `x-success` reopens on the phone, it does not reach the PC or the kernel by itself. In practice it is a second `shortcuts://x-callback-url/run-shortcut?name=<Reporter>&input={result}` call, chaining the result into a Shortcut that actually reports out.
5. `shortcut#11 (clipboard/get)` `XP GET /api/shortcut/clipboard/get?id=<single id>&outtime=<ms>` — read the chained result once the reporter Shortcut's last action is "Copy to Clipboard". `data.code` 30 means the chain stalled before that action ran, not that the input never arrived.
6. `none (Take Photo, Record Audio, and other built-in or App Intent actions)` `XP none, on-device` — once a Shortcut is running, any action installed on the phone is available to it, not just the vendor's list. Take Photo saves straight to an album without a single tap coordinate, which is the direct fix for a camera card built on UI taps that break on any Camera app layout change. Record Audio, HomeKit scene control, a Health category read, setting a Focus mode, and showing or adding a Wallet pass are the same story: real actions, reachable only by authoring a Shortcut, with no XP endpoint of their own.
7. `shortcut#1 (album/get)` then `shortcut#3 (album/down)` `XP GET /api/shortcut/album/get?id=<single id>&album_name=&num= then GET /api/shortcut/album/down?id=<single id>&list=[{album_name,name,ext}]` — retrieve what Take Photo produced, both already-documented and unrelated to the pixel-tap chain they replace.
8. `none (Shortcuts app, Automation tab, Create Personal Automation)` `XP none, no API, phone-side only` — the autonomous half. Bind the same kind of Shortcut to Time of Day, Charger Connected or

Disconnected, Low Battery, or an NFC tag, with "Ask Before Running" turned off. iOS runs it on its own schedule; nothing on 9911 is called, so it fires whether the kernel is up, the board is unplugged, or the video channel is showing `dev_disconnect`.

9. `none (automation's own "Get Contents of URL" action)` `XP none, external tooling` — point the automation straight at your own listener, the same gateway built for the reverse-channel card, so a phone can report battery, storage, or "still alive" on its own schedule. Anything it cannot push out this way still lands on the clipboard for `/shortcut/clipboard/get` once the control channel is back.

NEEDS

- The vendor iMouse Shortcut already bound as described in the hotkey card, since `/shortcut/exec/url` and every clipboard and album call still travel over that same HID-triggered channel.
- A second, custom Shortcut authored by hand on each phone. Both x-callback-url and Personal Automations target Shortcuts you write, never the vendor's own.
- The phone-side Shortcuts permissions from the setup checklist: allow running scripts, allow sharing large amounts of data.
- Per action: a configured Home app and the phone signed into it for HomeKit, a one-time Health permission grant for Health reads, and the pass already sitting in Wallet for a Wallet action. None of this is settable or queryable over the API.
- For the autonomous half, the automation created directly in the Shortcuts app on the phone. There is no remote call to create, list, edit, or delete a Personal Automation.
- A listener endpoint if an automation reports over the network rather than the clipboard, scoped and tokened the same way as the reverse-channel gateway.

WATCH OUT

- `shortcuts://x-callback-url` is Apple's own scheme, documented by Apple, not by the vendor or by either edition of this API. Nothing in this card has been probed on this rig. Confirm on one phone that `/shortcut/exec/url` passes such a URL through unmodified before designing anything on top of it.
- `x-success` is an on-device relay, not a network return path. It only becomes PC-readable data once the shortcut it reopens takes a further action, Copy to Clipboard or Get Contents of URL. Calling this "a structured callback to the PC" without that second hop overstates what Apple's mechanism actually does.
- Turning off "Ask Before Running" removes the automation's own confirmation, not iOS's. The first time an automation touches Health, HomeKit, or the camera it can still raise a one-time system permission prompt, and an unattended phone with an unanswered prompt blocks silently, the same failure mode as an unanswered AssistiveTouch prompt elsewhere in this group.
- Nothing in `/device/get` or `/config/usb/get` reports whether a Personal Automation exists, is enabled, or last fired. It is phone state the rack cannot see; the only evidence is whatever the automation itself chooses to report.
- Take Photo captures whatever the camera is actually pointed at in the rack, not the mirrored screen. It produces a photo, not a screenshot, so check its content by hand at least once before trusting an unattended capture.
- HomeKit, Health, and Wallet actions carry real-world and account-level side effects on the phone owner's own accounts: a HomeKit scene changes something in a physical home. Confine every one of these to accounts, homes, and health records you are authorized to operate, and stop rather than retry if an app signals a block, consistent with the automated-use caveat on every other Shortcut card in this section.
- Fan-out changes shape here: the vendor's own `/shortcut/*` calls take one comma id list for identical work on every phone, but a personalised `input` value makes that one call per phone again, not one call for many.

SCALE

The triggered half costs what `/shortcut/exec/url` already costs, one request per distinct input, so a shared payload is still O(1) but a personalised fan-out is O(phones). Reading a result back stays governed by whichever documented endpoint the reporter Shortcut ends on: `clipboard/get`, `album/get`, and `album/down` are each single id at their usual `outtime` ceiling. The autonomous half costs the kernel nothing until an automation chooses to call in: zero polling, zero HID, and it runs on 19 phones or 2 with identical marginal cost to the API, since iOS is the one doing the scheduling. That makes it the cheapest possible source of a self-report from a phone whose video and control channels are both down, at the price of being entirely unverified on this hardware and invisible to every rack-facing endpoint until it speaks.

Device-as-a-service: partition boards per client with groups, not with sub-accounts

PLUS EXTERNAL TOOLING · WITH CAVEATS

19 boards and 2 mirroring phones is exactly the shape of a rentable device farm, and every piece needed to sell it that way is already in this doc set: name a group of boards after a client, gate the gateway with a key, log who touched what. The one primitive that looks purpose-built for multi-tenancy, `/config/user/switch` with its `utag` 1 to 10 sub-accounts, is not that primitive. It switches which vendor licensing account the whole kernel process is logged in as. It is global process state, not a per-request or per-connection scope, and nothing in the API lets a caller attach a `utag` to an individual call the way `id` scopes a device. Build tenancy on `gid` groups and your own gateway instead, and never wire client onboarding to `/config/user/switch`.

HOW

1. `none` XP `GET /api/config/user/login?phone=&password=&utag=` — log IMConsole into the vendor account once, at rack setup, not per client. `utag` here just selects which of that account's own 1-10 licensing buckets is active; it is not a tenant id you hand out. Phonefarm has no equivalent call at all, no user concept in that edition's API.
2. `set_group` XP `GET /api/device/group/set?id=0&name=<client>` — create one group per client. `id=0` means "new group"; the response returns the generated numeric `gid`. This is the real partition boundary: it lives under the one logged-in account and needs no `utag` switching to use.
3. `set_dev` XP `GET /api/device/set?id=<deviceid list>&gid=<gid>` — assign that client's boards to the group. `id` takes a comma list so a whole allotment moves in one call; `gid` is independent of `vid` / `pid` hardware binding, so grouping and USB pairing are two separate steps.
4. `none` XP `GET /api/device/group/get-dev?id=<gid>` — the gateway's source of truth for "which device ids may this client's key touch." `id=0` means ungrouped, not wildcard, so a board left unassigned is invisible to every client's allow-list, which is the safe default while onboarding.
5. `none` XP `none (client-side, your gateway)` — front 9911 with your own authenticated reverse proxy. Every `/mouse/*`, `/key/*`, `/pic/*` and `/device/*` call gets rewritten to check the caller's key against the current `/device/group/get-dev` snapshot for their `gid` before it is allowed through, because the kernel itself checks nothing.
6. `none` XP `none (client-side, your gateway)` — per-client frame retention. Tag every `/pic/screenshot` capture with client id, device id and timestamp before writing it to storage; the kernel's response carries none of that, only raw bytes or a JSON error shape.
7. `none` XP `none (client-side, your gateway)` — per-client audit trail. Log caller key, resolved client id, `fun`, device id list and `data.code` for every call at the gateway, because the kernel's envelope carries no caller identity anywhere, and two clients' traffic is otherwise indistinguishable on the wire.

8. `none` XP `GET /api/config/user/info` / XP `user_info callback` — treat this as an operator canary, not a tenant API. It carries `utag`, `dev_num` and `dev_online_num` for whichever account is currently active and repeats every 30 s while logged in; a value that does not match what you expect means someone called `/config/user/switch` underneath you.

NEEDS

- One IMConsole login for the whole rig, holding the licensing account that authorizes all 19 boards. Devices cannot come online unlicensed under that account (the kernel pushes `im_log` "未登录，不允许上线" otherwise), which is one more reason not to be logging in and out of different `utag` buckets on a live rack.
- A gateway process you write and run, since neither edition authenticates callers on 9911. This is the same shape as the alert-bridge daemon described earlier in this section, extended to also authorize and log, not just observe.
- A provisioned `gid` per client, created and populated with `/device/group/set` and `/device/set` before that client's key goes live, plus a cached `/device/group/get-dev` snapshot the gateway refreshes on its own schedule, since `gid` membership changes are not pushed as callbacks.
- Storage outside the kernel for the audit log and the retained frames, keyed by client id, since `C:\iMouseXP\IMService.json` and the kernel's own state hold none of it.
- A decision made once, in the design, not discovered after the second client signs up: either one kernel process per tenant (separate IMConsole instances, separate PCs or VMs, true isolation), or one shared kernel with strict gateway-side `gid` filtering and a hard rule that nothing in the tenant-facing path ever calls `/config/user/switch`.

WATCH OUT

- This is the trap the surface catalogue does not carry: no endpoint anywhere in the nine XP docs accepts a `utag` or account parameter except `/config/user/login`, `/config/user/switch` and the two info calls. Every other call, from every connected client, is answered against whichever account is currently active in the one IMService.exe process. There is no way to pin a call to a specific sub-account. If tenant B's flow ever calls `/config/user/switch`, tenant A's in-flight jobs on shared boards start executing under B's licensing context immediately, with no request-level warning.
- Because devices need an active license to come online, a switch is not cosmetic: boards licensed only under the account just switched away from can drop offline mid-job under the other tenant's context, surfacing as an `im_log` push, not as an error on the call that was actually running.
- `gid` groups are not account-scoped either. They live under whichever single account is logged in, so switching `utag` does not hand you a second isolated group namespace to go with it; groups and accounts are two unrelated axes, and only the group axis behaves per-object rather than per-process.
- No locking exists on `/device/set`. Two gateway instances (or the gateway racing a human at the console) assigning different `gid` values to the same device id resolve last-write-wins, with nothing to detect the collision after the fact.
- The gateway is the entire security boundary. If it is bypassed, is compromised, or simply is not running, 9911 is still open and unauthenticated to anyone on the LAN, per-client isolation included; the trap above is a reason to avoid `switch` even for a well-meaning operator working the console directly while clients are connected.
- Reusing accounts, phones, or a rented board pool to run third-party apps for multiple paying clients raises the same terms-of-service exposure already flagged for automation in general, now multiplied by however many clients' accounts pass through the same hardware; keep each client's own app accounts on their own boards via `gid`, and do not let one board serve two clients' logins.

> SCALE
>
> Grouping and gateway filtering cost nothing extra at the kernel: they are the same `GET` calls this section already measured at 512 req/s across ten concurrent pollers, filtered client-side against a cached allow-list. The limiting resource is still the board-serialized lease from the scheduler card above, now partitioned per tenant instead of per job; a client's usable capacity is exactly the boards in their `gid` that are also state 2, bound, and mirroring, which today is at most the 2 live phones, not a share of 19. One shared kernel scales the gateway's own request handling roughly linearly with clients until the boards themselves saturate; one kernel per tenant scales instead in PCs and IMConsole logins, trading a global-switch risk of zero for a hardware cost per tenant. Given the trap above, quote capacity per client the same way the rest of this section insists on quoting it per phone class: as a live computed number from `/device/group/get-dev` plus `/config/usb/get`, never as 19 divided by client count.

Skipping straight to the vendor's own client: imouse-py

Before hand-rolling any of the HTTP or WebSocket plumbing above, read `imouse-py` (`pip install imouse-py`), the XP edition's official Python client, documented at `/iphone-farm-setup/xp-api-documentation/python-library`. It wraps the same path-style endpoints in two layers: a thin `imouse.api(host=...)` that exposes one method per call, and a recommended `imouse.helper(api)` that groups them the way this handoff does, a `console` object with `device`, `airplay`, `usb`, `group`, `imconfig` and `user` sub-objects for rack-wide operations, and a `helper.device(id)` object with `image`, `keyboard`, `mouse` and `shortcut` sub-objects for per-phone ones. Callbacks are handled the same way: an `@event.on("im_connect")`-style decorator per event name, with the payload already parsed into typed models (`DeviceInfo`, `UsbInfo`, `UserData`, `ImServerConfigData`) instead of a raw dict. Phonefarm has nothing equivalent, it ships only a loose `ios_at_api.py` plus a PyQt6 sample script, so this is an XP-only advantage, not a cross-edition one.

Its value here is as ground truth, not as a runtime dependency. The vendor's own `mouse_click(deviceid, button, x, y)` signature is a direct, code-level confirmation of the parameter order and defaults in the keyboard-and-mouse doc, positional `button` as an empty string defaults to left-click, matching `curl .../mouse/click?id=...&x=100&y=100` with `button` omitted. Where this handoff has had to hedge, for example whether `dev_rotate`'s payload sits inside `data.list` or at `data`, or whether `error_push` ever names the failing device, the library's typed models and its `helper` grouping are a second independent source to check a field name or a default against before committing it to a C# type. It will not resolve every open question: the library was not exercised against this rig any more than the WebSocket transport was, so treat it as a stronger prior, not a substitute for a live capture.

Two concrete uses follow from that. Someone building the Windows console this API implies should still read `imouse-py`'s source, even though it is Python and the target is C#, because its class boundaries (console-level versus device-level, plus the event dispatcher) are a tested precedent for where to draw the same seams, and its method signatures are a faster way to settle a disputed parameter than re-deriving it from the doc's prose tables. Someone who only wants to prototype a job or validate an idea against the rack, rather than ship the console, can skip the transport layer entirely: `pip install imouse-py`, call `helper.console.device.list_by_id()`, `helper.device(id).mouse.swipe(...)` and `@event.on(...)` directly, and never hand-write an HTTP GET or open the WebSocket at `ws://127.0.0.1:9911/api` themselves. Both readers still inherit every constraint measured elsewhere in this handoff, single-id-only on `/pic/*` and the shortcut read-backs, one shared parameter set per comma-separated id list, no authentication on the LAN, since the library is a thinner client over the same kernel, not a different one.

SECTION 12 · LIMITS AND RISKS

Hard limits, misconceptions and risks

Everything up to here described what the rack can do. This section is the boundary: the capabilities buyers assume are present and are not, the ones that exist in the product but are missing on this rig, and the ones that exist but are shaped differently from the mental model people bring to a phone farm. Every limit below is either documented in the vendor material or measured on this hardware on 10 September 2026. Where the XP edition installed here differs from the Phonefarm edition the vendor documents separately, that is called out.

XP SHORTCUT FUNCTIONS	WEBSOCKET CALLBACKS	OCR ON THIS RIG	/PIC/* ADDRESSING
19	**18**	**code 28**	**1 id**
none of them touches Messages	none carries message or notification text	HTTP 200, data.code 28, plug-in not connected	"Multiple is not allowed" on every recognition path

API AUTHENTICATION
none
any host on the LAN can drive the rack

Absent from the API in both editions

These two are not configuration problems or plug-in problems. No function exists, in the XP documentation set installed on this rig or in the Phonefarm document, that performs them. Both are among the first things operators assume a rack of SIM-carrying phones can do.

Sending an SMS or iMessage by API call

NOT POSSIBLE · REFUTED

There is no messaging endpoint. The Phonefarm shortcut table lists id 12 "Send SMS" struck through, alongside struck-through id 14 "Read text aloud", and the XP edition on this rig has no messaging path at all: its shortcuts page documents 19 functions covering album, file, clipboard, open URL, restart and shutdown, brightness, torch, flight mode, cellular data, WLAN and IP, and none of them is messaging. A full-text search of all nine XP doc files for sms, iMessage and the Chinese equivalent returns no endpoint. What remains is ordinary UI automation through the visible Messages app, one thread per phone.

HOW

1. `shortcut#12 Send SMS` XP `no XP path` — the capability people look for. Struck through by the vendor in one edition, absent from the other. Do not call it.
2. `shortcut#13 Open URL` XP `/shortcut/exec/url` — nearest workaround: open the composer from an `sms:` URL. Endpoint verified, custom URL schemes are not (every doc example is an http domain).
3. `shortcut#10` XP `/shortcut/clipboard/set` — place a non-ASCII or long body on the pasteboard, then paste with `/key/sendkey?fn_key=WIN+v`.
4. `send_key / click` XP `/key/sendkey?fn_key=ENTER` — or `/mouse/click` on the send arrow. Whether Return sends in Messages or inserts a newline is an iOS behavior no doc covers.

5. `get_device_screenshot` XP `/pic/screenshot?binary=true` — confirm the bubble by mean absolute pixel difference, single id only.

NEEDS

- A SIM with active SMS service in the handset. Neither XR on this rig has SIM state recorded, so even the workaround is unproven here.
- The iMouse iOS Shortcut installed, its IP edited to 192.168.4.29, run once with every permission granted, and bound to an AssistiveTouch custom key.
- Board bound and at `/config/usb/get` state 2, `location_crc` non-empty, AssistiveTouch and Full Keyboard Access on, and IMConsole.exe running.
- Send-arrow and QuickType coordinates measured per phone in mirror pixels (500 x 1080), not iOS points (414 x 896).

WATCH OUT

- No step in the workaround chain is verified on this hardware or iOS build. `/shortcut/exec/url` is a general-purpose Open URL action with a 15 s default `outtime` and no confirmation that the composer opened, so a blind ENTER or a blind tap at a guessed coordinate lands on whatever app was already frontmost.
- `/shortcut/exec/url` and `/mouse/click` both fan out to a comma-separated id list, so one wrong URL or one wrong shared coordinate is broadcast identically to every phone in the batch.
- Nothing reads, enumerates or exports Messages. There is no inbox endpoint and no thread list.
- Legal exposure is the dominant risk and it is severe. Unsolicited commercial SMS to US numbers falls under the TCPA with statutory damages assessed per message, CAN-SPAM covers commercial content, and PECR plus GDPR apply in the UK and EU. Carrier acceptable-use policies prohibit bulk or automated messaging from consumer SIMs, and Apple's terms prohibit automated bulk iMessage sending. Driving the UI carries the same liability an API send would.
- Every message is visible on the screen and persists in the thread and in carrier records.

SCALE

Not applicable to the claimed capability. For the UI workaround, budget roughly 3 to 6 seconds of wall time per message per phone with everything working first try, so on the order of 600 to 1,200 messages per hour per phone as a pure rig ceiling. That number is not the real ceiling: carrier anti-spam systems, platform abuse detection and legal liability stop this design far below what the hardware could sustain. This is not a gateway.

Reading an incoming SMS, one-time code or notification text

NOT POSSIBLE REFUTED

No function in either edition returns the body of an SMS, a push notification or Notification Center content, and no callback fires when a message arrives. The 18 documented WebSocket callbacks are device, board, cast and license state only, so an event-driven code relay cannot be built at all. The only text-out channels are `/shortcut/clipboard/get`, which reads whatever is already on the pasteboard, and the OCR family, which is plug-in gated and dead on this rig. Everything real here is a per-phone UI flow with vision done on the PC.

HOW

1. `none` XP `no /shortcut/sms, no /device/notification` — enumerated across all 60-odd XP paths. Phonefarm id 12 is outbound and struck through, and XP has no generic numbered-shortcut dispatcher to reach it.

2. `callbacks` XP `ws://127.0.0.1:9911/api` — 18 callbacks, none carrying message content. Detection is polling only, never an event.
3. `send_key fn_key` XP `/key/sendkey?fn_key=NoticeBar` — workaround A: open Notification Center so the text is on screen, then capture the frame and run your own OCR on the PC. Documented, not exercised here.
4. `screenshot + click` XP `/pic/screenshot?rect=[l,t,r,b] then /mouse/click` — workaround B: when a code arrives while a field is focused, iOS offers it in the QuickType bar. Poll a crop of that strip for a mean pixel difference above about 20 against the 0.02 idle floor, then tap it. No reading required.
5. `click time=800 + shortcut#11` XP `/mouse/click?time=800 then /shortcut/clipboard/get` — workaround C: long-press the bubble, tap Copy, read `data.text` back. Exact characters rather than a guess.

NEEDS

- The phone must actually receive the message. Codes sent to a third-party virtual number never reach the rack.
- Board at state 2, `location_crc` present, and the invisible iOS accessibility set (AssistiveTouch, Full Keyboard Access, tracking speed max, Auto-Lock Never, portrait lock).
- Workaround C and any `/shortcut/other/*` path need the iMouse Shortcut provisioned, v1.3 or higher for the recognition fallbacks.
- An OCR engine on the PC, because the kernel provides none here.

WATCH OUT

- `fn_key=NoticeBar` and the QuickType tap are documented but were not exercised in the live probe. Treat them at lower confidence than the verified mouse and screenshot mechanics.
- The QuickType strip is not at a fixed position: keyboard height varies with input mode, third-party keyboards and per-app toolbars, so the crop box is a per-app constant you must measure, not a rig constant.
- A notification that iOS collapses into Notification Center without a banner (Focus modes, Deliver Quietly) never appears on the mirrored screen, and that phone setting is invisible to the API.
- Polling has a latency window that can miss a short-lived code, and the miss is silent.
- Reading messages is only defensible on handsets and accounts the operator owns. Doing it on anyone else's raises privacy and computer-misuse exposure quite apart from platform terms, and automated handling of login codes is often prohibited by the platform's own terms even for your own account.
- A code captured into a PC-side screenshot is a live secret on disk. Purge it rather than logging it, and remember the kernel API is unauthenticated on the LAN, so any host on the subnet can pull the same frames.

SCALE

Never mirrored, because each handset's code is different. Only the normalizing Home presses and `/mouse/reset` can be fanned out. The polling itself is free (raw BMP frames cost 2 ms at p50 and phones are independent), but SMS delivery of 5 to 30 seconds dominates, plus seconds of UI navigation. Budget tens of seconds per code, not milliseconds.

Missing on this rig, not from the product

TRAP **The blocked call answers HTTP 200**

A missing plug-in does not produce a transport error. `GET /api/pic/ocr` returns HTTP 200 with an empty result and `data.code` 28, message 插件未连接. A client that checks only the HTTP status, or the top-level status field, will record a success. Branch on `data.code` everywhere, never on message text, which is Chinese in this edition.

Server-side OCR and find-text to read the screen

NOT POSSIBLE REFUTED

Every "read the number, then decide" idea routes through the kernel's OCR endpoints, and on this rig they are dead. The probe on 10 September 2026 returned `data.code` 28, which the XP code table defines as plug-in not connected. That kills `/pic/ocr`, `/pic/ocr-ex`, `/pic/find-text` and `/pic/find-text-ex`, and with them the OCR-dependent point-capture helpers behind automatic screen connection and device restart. This is a rig fact, not a product fact: the endpoints are fully documented with defined parameters and response shapes, and installing the vendor plug-in brings them back. Note also the scope: the kernel cannot read text here, but the rack still can, client-side, and that path is the recommended one.

HOW

1. `ocr / ocr_ex` XP `/pic/ocr, /pic/ocr-ex` — blocked. Single id, optional `rect` as [left, top, right, bottom], returns list of text with center and rect.
2. `none` XP `/pic/find-text, /pic/find-text-ex` — blocked, XP-only endpoints on the same plug-in. These are the calls a "detect the throttle banner" flow would need.
3. `find_image / find_image_ex` XP `/pic/find-image, /pic/find-image-cv` — untested, presumed blocked. Gated by `opencv_num`. Probe with one template crop before designing on it.
4. `get_device_screenshot` XP `/pic/screenshot?id=<id>&binary=true` — works. Raw bottom-up BMP, 1,620,054 bytes, p50 2 ms. Feed the decoded buffer, or a `rect` crop, to Windows.Media.Ocr, Tesseract or PaddleOCR on the host. The frame is in the same 500 x 1080 space `/mouse/click` expects, so host-OCR boxes are directly tappable.
5. `send_key + shortcut#11` XP `/key/sendkey?fn_key=WIN+a, then WIN+c, then /shortcut/clipboard/get` — exact strings instead of a probabilistic read, where the text is selectable.
6. `n/a` XP `/config/imserver/set?ocr_num=<n> then /imserver/restart` — the real fix. The plug-in binary is not obtainable through the API, and the restart drops every mirroring session.

NEEDS

- To restore the blocked calls: the vendor OCR plug-in package for the XP edition, installed and connected to IMService.exe, then `ocr_num` set and the kernel restarted. `ocr_num` above 1 only means something on the GPU build.
- For the host-OCR path: an OCR engine on the PC and a decoder for a 54-byte-header bottom-up 500 x 1080 x 3 buffer. Reuse one buffer per poller; each 1.6 MB frame is far above the .NET Large Object Heap threshold, and without reuse the measured cost was 488 MB/s of allocation and 496 gen-2 collections per 8 s, against 2.9 MB/s with reuse.
- For the clipboard and native-album paths: the full iMouse Shortcut prerequisite chain, plus Full Keyboard Access for the WIN+a and WIN+c combinations.

WATCH OUT

- The image-search verdict is inference, not measurement. Only `/pic/ocr` was probed. Record the real answer for `/pic/find-image-cv` before repeating the claim, and treat `/pic/find-multi-color` being plug-in-free as an argument from silence.
- Host OCR on a lossy 500 x 1080 mirror frame is marginal for small digits: 1,234 can read as 1,284. Route anything where an exact numeric or currency value matters through the clipboard path, or through a native-resolution album capture, never through host OCR on the mirror frame alone.
- The clipboard path only works where the text is genuinely selectable. Text baked into an image, a WebView that blocks selection, or a custom-rendered element returns empty or stale content, silently.
- The `/shortcut/other/*` recognition fallbacks are named in the XP doc with no parameter table, no example and no response body. They are an unexplored lead, not a solution.
- Every `/pic/*` path is single id, as are `/shortcut/album/get` and `/shortcut/album/down`, so text checks scale linearly with phones while input does not.
- Pixel-color state checks break on dark mode, accessibility contrast settings, Night Shift and app redesigns. Pin the display settings and re-capture after any app update.
- Installing the plug-in is rig-wide and costs a kernel restart, so every phone must re-cast. Plan a maintenance window, not a rolling upgrade.
- The Phonefarm doc marks `ocr_ex` professional-version-only while XP states no edition gate. The mapping between Phonefarm professional and the installed XP International Edition is defined nowhere, so do not assume the license covers it.

SCALE

The 2 ms screenshot is never the bottleneck, the recognizer is. Host OCR on a frame or crop runs roughly 50 to 300 ms per phone on CPU, so a 19-phone rack checked once a minute costs about 6 s of CPU per sweep while a per-second loop across the rack does not fit. Frame supply is genuinely parallel: two phones polled flat out gave 83 and 85 req/s, ten concurrent pollers gave 512 req/s total. The floor is the AirPlay source rate, which is per-phone: 192.168.4.68 delivers about 29 new frames per second under motion while its identical twin 192.168.4.27 delivers about 4.5, so a verification step on the slow phone waits about 220 ms per frame and must not time out on a 30 fps assumption. A still screen produces almost no new frames, so discard repeats with a same-as-previous check rather than re-running OCR.

Present, but not the shape people assume

The remaining three are the expensive misconceptions. In each case the endpoint exists, accepts the call and returns success, and the wrongness shows up later, in the posted content, in the typed field, or in a rack that has gone dark.

Per-phone parameters in one multi-id request

NOT POSSIBLE REFUTED

A comma-separated `id` list is a fan-out, not a batch. Every endpoint that accepts multiple ids carries exactly one parameter set, applied identically to every id, and there is no array-of-per-device-payloads form anywhere in the XP docs, the Phonefarm doc or the Python library. The other half of the wall is read-back: every `/pic/*` path and every shortcut read (clipboard/get, album/get, album/down, file/get, device/ip) is marked "Multiple is not allowed". Distinct content per phone is always N calls, and reading state back from N phones is always N calls. Read this as a throughput model, not a broken feature: the mirrored half really is O(1) requests.

HOW

1. `mouse_reset_pos + send_key` XP `/mouse/reset?id=<list> then /key/sendkey?fn_key=WIN+h` — shared baseline for the whole target set in one request. Two Home presses: the first leaves the app, the second returns to page one.
2. `shortcut#10 / #13 / #7` XP `/shortcut/clipboard/set, /shortcut/exec/url, /shortcut/album/upload` — these accept an id list, but `text`, `url` and `files` are single shared values. Loop one id per call whenever the payload differs.
3. `send_key / click` XP `/key/sendkey?fn_key=WIN+v then /mouse/click?id=<list>` — back to one request for all, provided every follower's `imgw` and `imgh` match the master.
4. `get_device_screenshot` XP `/pic/screenshot?id=<id_k>&binary=true` — verification is N requests, by documentation. Cheap, but N.
5. `set_dev / del_dev` XP `/device/set, /device/del` — the one exception: these return `data.list` of the ids that actually succeeded. Input paths return a single `data.code` with `data.id` echoing the whole list.

NEEDS

- A controller that structurally forbids combining a personalized field with a comma id list. The API will accept it and fan the same payload out to every phone.
- A follower list filtered to matching `imgw` and `imgh` before any shared coordinate call, and built only from phones that are online, bound and on the intended screen.
- For personalized payloads, the full Shortcut provisioning on that phone, plus iOS 15 or later for `/shortcut/file/*` and `/shortcut/switch/device`.

WATCH OUT

- The failure is silent and one-directional. Passing an id list with a personalized field is a valid request that applies the same text, URL or file to every phone, with no error, because input endpoints report one code for the whole list. Guard it in the controller's types, not at review time.
- A shared coordinate sent to a phone with different `imgw` or `imgh` lands somewhere else entirely and still returns code 0. The two XRs here have different home screens, so the same coordinate is Clock on one and Reminders on the other.
- A multi-id tap that hit the stuck-click state on one board reads as success. Send `/mouse/reset` to the whole set before any tap-critical sequence and verify per phone with a pointer-box pixel difference (1.4 to 9.8 for a landed pointer against a 0.02 idle floor).
- `outtime` is milliseconds in XP (default 15000) and seconds in Phonefarm, and the doc's type column calls it Boolean, which is a translation artifact. `sleep` on `/shortcut/clipboard/set` is a separate parameter in seconds.
- Unverified: whether concurrent Shortcut round trips across phones serialize inside IMService. Mouse, key and screenshot traffic were measured as independent, but a Shortcut adds a second, Wi-Fi-bound leg that was never load-tested. Treat the personalized loop as serial until measured.
- The Phonefarm sibling looks like it has a batch form and does not: its shortcut function takes a device list but one shared parameter string, and it is professional-version-only. Moot here, since this kernel rejects every Phonefarm function name with HTTP 500.

SCALE

Shared steps cost one request regardless of N, and the board walks all listed phones in parallel, so the cost is pointer-walk distance at about 0.6 ms per pixel (1,000 px measured at 592 ms), identical for one phone or nineteen. Personalized steps are N requests, each a Shortcut round trip bounded by `outtime`; if each fires

promptly, budget 1 to 3 s per phone, and note the unhandled worst case is 19 phones at up to 15,000 ms each, about 4.75 minutes for one round. Verification runs up to 84 req/s per phone and is never the constraint. The Shortcut round trips dominate by three orders of magnitude.

Typing non-ASCII text with the keyboard endpoints

NEEDS SHORTCUT WITH CAVEATS

The direct limit is confirmed. `/key/sendkey`, `/key/down` and `/key/up` all document `key` as English, numbers and English characters only, the Phonefarm sibling states it harder as "Chinese is not supported", and there is no Unicode typing endpoint at all: the string send_text appears zero times in the XP documentation set. Emoji, CJK and accented Latin cannot be typed through the HID keyboard path. The documented route around it is two calls, clipboard then paste, and both fan out to an id list, so one shared Unicode string reaches N phones in two requests. On this rig that route is a design, not a capability: no `/shortcut/*` endpoint has ever been probed here, so treat non-ASCII as unavailable until a clipboard round trip returns code 0 on these phones.

HOW

1. `send_key key="Nouveautés"` XP `/key/sendkey?key=<text>` — works for ASCII only, and neither doc defines behavior for out-of-domain bytes, so there is no error code to detect the failure.
2. `send_text` XP `no XP path` — refuted for this rig. No such path, no send_text_ok callback, 9912 closed, and bare Phonefarm function names to 9911 return HTTP 500.
3. `click / send_key fn_key` XP `/mouse/click or /key/sendkey?fn_key=OPENKeyboard` — focus the target field. Send `/mouse/reset` to the set first.
4. `shortcut#10` XP `/shortcut/clipboard/set` — place the Unicode string on the pasteboard. Send it as POST application/json in UTF-8, not as a GET query string.
5. `send_key fn_key=WIN+v` XP `/key/sendkey?id=<list>&fn_key=WIN+v` — paste. Needs Full Keyboard Access on, or the combination is never interpreted.
6. `shortcut#11` XP `/shortcut/clipboard/get` — round-trip an identical string. This is the cheapest single proof that the Shortcut is installed, the hotkey binding fires and the phone can reach the kernel.

NEEDS

- The iMouse Shortcut installed on every phone that needs Unicode, with the kernel IP edited inside it, run once with every prompt allowed, and Shortcuts, Advanced set to allow running scripts and sharing large amounts of data.
- The Shortcut bound to an AssistiveTouch custom key through the console's Send Binding Key, and the board bound and at state 2, because the Shortcut is launched by a HID hotkey.
- Full Keyboard Access on with 1 s auto-hide, phone unlocked, portrait, Auto-Lock Never.
- Provisioning is manual and per phone. At 19 boards that is 19 GUI passes with no API to batch it, and it must be redone on every phone whenever the kernel IP changes, for example after a DHCP re-address.

WATCH OUT

- The single observable symptom of any missing piece in the phone-side chain is `data.code` 30, call timeout, after `outtime`. Nothing distinguishes a missing Shortcut from a denied permission dialog.
- WIN+v pastes whatever is on the pasteboard. If `clipboard/set` silently failed and the paste is sent anyway, the previous string, possibly another phone's text, gets posted into a live field. Gate the paste on `clipboard/set` returning code 0, and treat the first production use on each phone as a dry run verified by `clipboard/get`.
- Pasting destroys the previous pasteboard contents, and there is no read-modify-restore primitive beyond get then set. The text also sits on the pasteboard afterward, where any app that reads it on foreground can see it,

and Apple's Universal Clipboard may propagate it to other signed-in devices. That last point is field behavior, not vendor documentation, so verify it rather than assuming it.

- Glyph-level verification must be client-side or by eye. `/pic/find-text` would be the clean check and is dead here.
- The IME-switching alternative cannot produce emoji and depends on undocumented per-phone keyboard state; picking a candidate blind needs recognition this rig does not have. The URL alternative only works for apps that honor a text parameter, which is an app fact with no vendor documentation behind it.
- Pasting the identical string across many phones is ordinary bulk-content behavior, and platforms commonly treat identical repeated posts as spam under their terms. Whether bulk posting is permitted at all is governed by each platform's terms for the accounts involved.

SCALE

ASCII is effectively free: one call carries a whole string to the whole rack. Unicode costs a Shortcut round trip, but both write calls fan out, so one shared string reaches N phones in exactly two requests, bounded by the slowest phone's round trip. Cost is per distinct string, not per character: a 3-character caption and a 2,000-character bio are the same two calls. Distinct text per phone serializes at one `clipboard/set` each. Verification does not fan out, and its latency is dominated by the slow phone's 4.5 fps AirPlay link rather than by request throughput.

Turning Wi-Fi off and back on as a round trip

NOT POSSIBLE REFUTED

Turning Wi-Fi off from the API is a one-way door. `/shortcut/switch/wlan` with `state=0` is fully documented and takes an id list, but the iMouse Shortcut that executes it is a phone-side Shortcut that fetches its instruction from the kernel over the LAN. The USB board delivers only the trigger hotkey. The moment `state=0` lands, the phone can no longer be told `state=1`, and the AirPlay video channel dies with it, leaving a controllable phone you cannot see and cannot instruct. Airplane mode through `/shortcut/switch/flight` has the same trap. The general rule for this rig: every shortcut-backed function is a LAN function, so no shortcut may be used to disable the LAN.

HOW

1. `get_usb_list + get_device_list` XP `/config/usb/get + /device/get` — the last full snapshot you will get. Control is board state 2, video is device state non-zero with a live `air_pid`.
2. `shortcut id 23` XP `/shortcut/switch/wlan?state=0` — the outbound half, which genuinely works and fans out to the whole rack in one request.
3. `dev_disconnect` XP `WS callback + /pic/screenshot` — the video channel collapses, device state drops to 0, the socket to AirplayMirroring.exe on TCP 17000 tears down, and screenshots answer code 25, device offline.
4. `shortcut id 22` XP `/shortcut/switch/wlan?state=1` — the break. The board still delivers the hotkey and iOS still launches the Shortcut, but the Shortcut's first act is to fetch its instruction from an IP it can no longer reach. The call ends in code 30 after `outtime`.
5. `mouse_reset + send_key + click` XP `/mouse/reset, /key/sendkey?fn_key=ControlBar, /mouse/click` — blind, open-loop recovery over USB at coordinates memorized in advance. Unverified on two counts and not a fallback with known odds.
6. `none` XP `phone-side setting, no API` — the supported alternative. Set IPv4 to Manual with the router and gateway field left empty, so the phone keeps LAN reachability for mirroring and shortcuts while default traffic leaves over the SIM.

7. `shortcut ids 24, 21, 20` XP `/shortcut/device/ip, /shortcut/switch/cdpd` — network QA that never surrenders the control plane: confirm LAN reachability, and cycle a stalled cellular data session with `state=0` then `state=1`.

NEEDS

- The full Shortcut prerequisite chain for any of these calls to fire at all, plus the invisible iOS accessibility settings and IMConsole.exe running.
- A frame captured and coordinates recorded before Wi-Fi is cut, per phone, if blind recovery is to be attempted at all.
- For the supported alternative: a SIM with an active data plan in each phone, and the manual IPv4 edit applied by hand on each phone. There is no API for that setup, so it does not scale the way the broken toggle appeared to.

WATCH OUT

- The outbound half works, so do not read this as "the endpoint does not exist". Only the round trip is impossible.
- Whether the `state=0` call returns code 0 or code 30 is undetermined and depends on whether the Shortcut acknowledges the kernel before the radio drops. Do not branch recovery or alerting logic on that code, because the phone goes dark either way.
- Blind recovery rests on two untested assumptions: that `/mouse/*` still acts on a device whose state is 0 (code 25 exists and may gate it), and that the Control Center tile restores what the Shortcut switched off. It also assumes the AssistiveTouch and Full Keyboard Access prerequisites survived, which no software can confirm. Budget for a physical trip to the shelf with a cable.
- There is no observation channel to close the loop on: no mirror frame, and no server-side recognition either.
- A rack-wide `state=0` is one comma-separated, unauthenticated HTTP GET away from taking every phone off-network at once. That is a self-inflicted outage needing a person at every affected phone. Keep 9911 firewalled and never put this behind an unconfirmed UI button. If it must be tested, test one expendable phone off-hours with someone at the shelf.
- A widely reported iOS behavior is that Wi-Fi re-enabled by hand while airplane mode is on survives later airplane-mode cycles, which would make the flight toggle recoverable. That is not stated in any vendor document and was not tested here. Prove it on one expendable phone before trusting it.
- Before assuming any shortcut call fires on this rig at all, run `GET /api/shortcut/device/ip?id=<id>&state=0` and check it returns the same address as the device record's `ip` field. Phonefarm gates its whole shortcut dispatcher as professional-version-only; XP attaches no edition note, but the path was never exercised here.

SCALE

Not applicable to the round trip, since it does not complete. The dispatch side is cheap, one comma-list GET for the whole rack, which is exactly what makes the failure mode dangerous rather than throttled. `/shortcut/device/ip` is single id, so reachability checks are one request per phone bounded by `outtime`; run them in parallel rather than in a loop. The supported alternative is a one-time manual setup of a few minutes per phone, after which cellular cycling is a single rack-wide call with no ongoing cost and mirroring never drops.

OPEN **What is still unprobed on this rig**

Three gaps matter more than the rest, because designs get built on them. No `/shortcut/*` endpoint has ever been called on these two phones, so every Shortcut-backed capability in this document is documented-but-unexercised here. The image-search plug-in was never probed, and the OCR result makes its absence likely rather than certain. And `/pic/find-multi-color` is presumed kernel-internal only because no doc names a plug-in for it, which is an argument from silence given that `/pic/ocr` turned out to be gated. Each of these is a single cheap probe. Run them before the console's feature set is promised to anyone.

Telemetry and phone-side configuration

The two requests a fleet console asks first, "how is this phone doing" and "is this phone set up correctly", are the two the API answers least. Both are documented absences, verified by reading the field tables end to end rather than by failing to find a parameter.

Phone telemetry: battery, temperature, storage, position, carrier

NOT POSSIBLE REFUTED

Neither edition exposes any phone telemetry. The XP device record returned by `/device/get` is a closed list covering network identity, model and iOS build, screen geometry, board binding and mirroring state, and nothing else. Grepping both doc sets for battery, temperature, storage, GPS, carrier and signal returns no hit outside `location` and `location_crc`, which are mouse calibration parameters. The 19-entry XP shortcut catalogue covers albums, files, clipboard, URL, restart, brightness, torch, airplane mode, cellular, Wi-Fi and IP, with no reader for power, heat, free space or position.

HOW

1. `get_device_list` XP `GET /api/device/get` — confirm the ceiling. Returns `ip`, `mac`, `user_name`, `version` (18.7.10), `model` (iPhone11,8), `width` / `height` (414x896), `imgw` / `imgh` (500x1080), `state`, `air_pid`, `vid` / `pid` / `uid`, `gid` / `gname`, `location_crc`, the five `air_*` overrides and `uver`. No power, thermal, storage, position or carrier field exists.
2. `get_device_list + get_usb_list + callbacks` XP `/api/device/get, /api/config/usb/get, ws://127.0.0.1:9911/api` — the health telemetry that does exist, split along the two channels. Video ready is `state` non-zero with a live `air_pid` (22236 here); control ready is board `state` 2 with the phone bound to that board's `uid`. Callbacks `dev_connect`, `dev_disconnect`, `usb_change` and `error_push` carry the transitions.
3. `none` XP `client-side frame-rate counter` — add the measured new-frame rate as a link-quality proxy per phone. On this rig 192.168.4.68 produces about 29 new frames per second under driven motion and its identical twin 192.168.4.27 about 4.5.
4. `shortcut#15, set_airplay_mode` XP `GET /api/shortcut/switch/bril?id=<ids>&state=0.05, GET /api/device/airplay/set?id=<ids>&air_fps=<n>&air_ratio=<n>` — the nearest thing to a thermal or battery control loop. Both are inputs only, with no corresponding reading.
5. `shortcut#24` XP `GET /api/shortcut/device/ip?id=<id>&state=1` — the public egress IP, single id only. `state=0` returns the LAN address, which should match the `ip` field from `/device/get`, making this the cheapest end-to-end proof that the Shortcut is installed and the hotkey binding fires.

NEEDS

- Nothing at all for the negative finding. `/device/get` with no `id` returns every device in one call and answers today.

- For any `/shortcut/*` route, the iMouse Shortcut installed on each phone with the kernel IP (192.168.4.29) edited in, run once with every permission allowed, Shortcuts Advanced set to allow running scripts and sharing large data, and the Shortcut bound to an AssistiveTouch custom key through the console's Send Binding Key. `/shortcut/file/*` and `/shortcut/switch/device` additionally need iOS 15 or later.
- For any screen-reading route, the vendor OCR plug-in installed and connected, which is not the case here, or your own OCR engine running on the raw BMP frames on the PC.

WATCH OUT

- `location` and `location_crc` are pointer calibration, not geolocation. Both XRs carry the shared-library CRC B0DBD800, byte identical to the vendor's own doc example. A dashboard that maps this field is mapping a checksum.
- Reading battery off a Settings screenshot is not a degraded version of telemetry, it is a from-scratch vision and automation project. On this rig it is blocked end to end: `/pic/ocr` answers but returns `data.code` 28, plug-in not connected, so `/pic/ocr`, `/pic/ocr-ex`, `/pic/find-text` and `/pic/find-text-ex` are all unavailable. Label it blocked pending plug-in install, not available but slow.
- Estimating free space by summing Photos and On My iPhone through the Shortcut is not a weak proxy, it is not a proxy at all. It excludes apps, caches and system data entirely, so the error against true free space is unbounded. Do not carry it into a capacity plan.
- Both levers are open loop and one is disruptive. Brightness has no getter, and changing any `air_*` value disconnects the cast immediately and applies on the next projection.
- Public-IP geolocation is coarse and wrong behind CGNAT or a carrier's shared egress, and every phone on one LAN reports the same address. Treat it as network egress information, never as device position, and keep it out of logs you would share.
- A scrape built on UI navigation can appear to succeed while reading stale data for hours, because a failed tap and an unchanged screen look identical. The stuck-click state makes this concrete: `/mouse/click` can stop landing while `/mouse/move` and hardware keys still respond, and only `/mouse/reset` clears it.
- A thermal shutdown presents as `dev_disconnect` and `usb_change` code 20 arriving together. A cable pull hits `usb_change` alone and a Wi-Fi loss hits `dev_disconnect` alone, so correlate the two callbacks before naming a cause.

SCALE

The negative check is one request for the whole rack. The workarounds scale badly and in different ways. Every shortcut reader is single id (`/shortcut/album/get`, `/shortcut/file/get`, `/shortcut/clipboard/get`, `/shortcut/device/ip` all state "Multiple is not allowed") with a 15,000 ms default `outtime`, so a 19-phone sweep is 19 sequential round trips. Only the write and switch calls take a comma-separated id list and fan out in one request. A UI-navigation read costs roughly 3 to 5 seconds per phone, dominated by pointer travel at about 0.6 ms per pixel and by per-phone frame settle, which is about 35 ms on the fast XR and 220 ms or more on the slow one. That is on the order of 15 to 20 phones per hour for a one-shot manual check, far below any monitoring cadence, and it interrupts whatever the phone was doing.

WRONG **"location_crc is where the phone is"**

It is the CRC of an entry in the vendor's shared mouse parameter library, used to match calibration across devices of the same model. Both phones on this rig carry B0DBD800. There is no geolocation anywhere in either API edition, and the only position-adjacent value available, the public IP, describes a network, not a phone.

Auditing the phone-side accessibility settings from the PC

NOT POSSIBLE REFUTED

The vendor requires five phone-side settings on every slot: AssistiveTouch on with tracking sensitivity at maximum, Full Keyboard Access on with auto-hide at 1 s, Trackpad and Mouse tracking speed at maximum, Display and Brightness with Auto-Lock set to Never and View set to Standard, and a Control Center laid out with two rows of icons at the bottom, which the vendor notes affects automatic screencasting. None of the five is readable through either edition. A search for "accessibility" or "assistive" across the whole XP documentation set returns zero hits, and the consequence is the vendor's own support case: a phone can mirror perfectly, report `state` 1 with a live `air_pid`, and silently discard every input.

HOW

1. `get_device_list` XP `GET /api/device/get` — 26 fields of geometry, network, binding and mirroring state. Not one describes an iOS setting.
2. `none` XP `GET /api/config/imserver/get` — kernel-side fields only (`air_play_name`, `lang`, `mdns_type`, `open cv_num`, `ocr_num`, `allow_ip_list`, `thread_mode`, `mouse_mode`, `flip_right` and the rest). Phonefarm has no configuration endpoint at all.
3. `shortcut ids 1-24` XP `the 19 documented /shortcut/* paths` — every one writes or reads content. None reads a Settings toggle.
4. `mouse_reset_pos + mouse_move + get_device_screenshot` XP `GET /api/mouse/reset?id=<ids>, GET /api/mouse/move?id=<id>&x=380&y=880, GET /api/pic/screenshot?id=<id>&binary=true` — behavioral probe 1. Diff a roughly 40 px box around the commanded point: a landed pointer measures 1.4 to 9.8 mean absolute difference against a 0.02 idle floor.
5. `send_key` XP `GET /api/key/sendkey?id=<ids>&fn_key=WIN+h` — behavioral probe 2, the keyboard path. A whole-frame mean difference above 20 (142 measured) means the key was interpreted. The plain `Home` key needs kernel 2026.2.27 or later and iOS 26.1 or later, so on these 18.7.10 phones `WIN+h` is the correct choice.
6. `click` XP `GET /api/mouse/click?id=<id>&x=<x>&y=<y>&button=1` — behavioral probe 3, the one that matters. Press Home twice first to normalize, then tap a target whose position you read from that phone's own frame.
7. `shortcut#24` XP `GET /api/shortcut/device/ip?id=<id>&state=0` — an indirect tell. A `data.code` 0 whose `data.text` equals the device record's `ip` proves part of the phone-side setup, but only the Shortcut part.

NEEDS

- Mirroring live (`state` non-zero with a live `air_pid`) so the probes are optically verifiable, and the board at `state` 2 in `/config/usb/get` with non-empty `vid` / `pid` / `uid` in the device record.
- A non-empty `location_crc`, otherwise every `/mouse` call returns `data.code` 11, no mouse parameters.
- `IMConsole.exe` running and logged in. With only `IMService.exe` up, this rig reported both phones at `state` 0 and an empty board list.
- A client-side frame differ computing mean absolute pixel difference on the bottom-up BMP (1,620,054 bytes). Byte or MD5 comparison is useless, because encoder noise and the clock change every frame.

WATCH OUT

- Passing every probe is not a settings audit and must never be reported as one. A phone can land the pointer, honor `WIN+h` and register a tap while still having mid-range tracking sliders or a one-row Control Center, and that misconfiguration surfaces later as offset taps or a dropped mirror, not as a failed probe.
- `WIN+h` succeeding proves only the keyboard path. It was measured still working on a phone whose taps were being silently discarded, so it is never a proxy for input health.
- Auto-Lock Never, View Standard, lowest brightness and StandBy off are not testable by any probe at all.

- The shortcut round trip is ambiguous on failure. `data.code` 30, call timeout, cannot distinguish Shortcut not installed from binding not configured from phone unreachable, so do not sell it as diagnostic of any single failure mode.
- Probe 3 has a real side effect: it opens whatever is at the tapped coordinate. Run the sweep as a scheduled, bounded pass, not as a continuous background health check, or a fleet ends up in unexpected app states.
- A failed probe must produce a work order for a person at the rack, never an automated remediation. The settings cannot be changed from the PC either, so any automatic "fix" can only be a benign retry, and a retry loop hides exactly the failure this entry is about.
- A probe run against an unlocked phone captures whatever it is displaying into the 1.6 MB frames you diff. Treat probe frames as disposable.
- Passing today says nothing about tomorrow. Nothing on the PC can observe a toggle being changed on the phone, so results carry a timestamp and are re-run on events: shift change, phone re-seated, iOS update.
- Dropped from earlier drafts: the claim that reaching a phone-side collection page proves LAN connectivity. The XP docs document no such URL. The Phonefarm `?fun=collection` form belongs to the other edition. Use the `ip` field or the shortcut round trip instead.

SCALE

The audit itself is impossible at any rate. The substitute sweep is cheap: `/mouse/reset`, `/mouse/move`, `/mouse/click` and `/key/sendkey` all take a comma-separated id list, so the input half costs one request for N phones and the board walks them in parallel. Verification is single id (`/pic/screenshot` states "Multiple is not allowed") but phones are independent, measured at 83 and 85 req/s with two concurrent pollers and 512 req/s aggregate with ten. Budget pointer travel, not requests: a 1000 px reposition is about 592 ms. Allow at least one new frame after each action, roughly 35 ms on the fast XR and 220 ms on the slow one, never a fixed 30 fps assumption. A three-probe sweep of 19 phones lands around 20 to 40 seconds, which is affordable far more often than it is useful.

Transport and frame fidelity

Server-pushed frame streaming and a second AirPlay receiver

NOT POSSIBLE REFUTED

The Phonefarm doc advertises two capacity features this rig does not have. `loop_device_screenshot` makes the kernel push screenshots at a chosen interval as binary WebSocket frames, retunable live and stoppable with `stop:true`. `get_airplaysrvnum` and `set_airplaysrvnum` read and set how many AirPlay receiver services one kernel advertises, each appearing to the phones as a separate target. The installed XP kernel has neither: no loop or push endpoint, no binary WebSocket frame format, and a single receiver in `/config/imserver/get` with no count field. The live probe confirms it, since `POST {"fun":"get_airplaysrvnum"}` to 9911 returns status 500 and port 9912 is closed.

HOW

1. `loop_device_screenshot` XP `none` — do not design around a pushed stream. Every one of XP's documented callbacks is a JSON envelope; there is no binary opcode, no frame header and no interval parameter anywhere in the XP doc set.
2. `get_device_screenshot (binary, ws-only in Phonefarm)` XP `GET /api/pic/screenshot?id=<one id>&binary=true` — the substitute. One poller per phone at a fixed period, 33 to 50 ms for a live view and 100 to 500 ms for wall tiles. XP states plainly that with `binary` true the JSON is not returned and raw image data comes back directly, so HTTP binary works here where Phonefarm forbids it.

3. `none` XP `client-side buffer reuse and SameAsPrevious` — reuse one 1.62 MB buffer per poller and discard frames whose mean absolute difference from the previous frame is under 1.
4. `get_device_screenshot isjpg / gzip` XP `/api/pic/screenshot jpg=true` — never on loopback. The kernel re-encodes at 40.5 ms p50 with a hard cap near 21 req/s. Reserve it for a remote client that cannot afford 1.6 MB per frame. XP has no `gzip` and no `original` flag.
5. `get_airplaysrvnum / set_airplaysrvnum` XP `none` — no receiver count exists. The nearest call, `/config/imserver/set` with `air_play_name` or `air_play_port`, renames or moves the one receiver and needs `/imserver/restart`, which drops every mirroring session.
6. `get_airplaysrvnum → data.ver` XP `WS callback im_connect → data.ver` — the kernel version comes from the WebSocket on connect, not from any HTTP path. Features are gated on it, and the two editions use different version schemes, so do not compare them.

NEEDS

- `IMService.exe` on TCP 9911 for both HTTP and WebSocket at `/api`, plus `IMConsole.exe` running and logged in.
- Target phone mirroring, with an established socket on TCP 17000 from the phone's IP.
- One poller thread and one reusable 1.62 MB buffer per phone, since `/pic/screenshot` is single id and every raw frame exceeds the .NET Large Object Heap threshold.
- `imgw` / `imgh` read per device. Frames and input coordinates share that space (500 x 1080 here), not the 414 x 896 logical size.

WATCH OUT

- The substitute is not a literal equivalent. Phonefarm's push let the kernel pace the stream and multiplex many devices over one socket using the frame's `deviceid` header. On XP the client owns pacing and demultiplexing and holds N sockets, so ten tiles means ten sockets and ten times the request overhead even though wall-clock latency is fine.
- Quote the spread, not just the median, in any capacity claim: ten concurrent pollers measured 512 req/s aggregate at p50 3 ms with a worst p95 of 142 ms, which is 47 times the median.
- No transport beats the phone's own AirPlay production rate, measured at about 29 new frames per second on one XR and 4.5 on its identical twin on the same iOS build. That is the second phone's Wi-Fi link, and the vendor console hits the same wall.
- Use a fixed poll period. Deriving the interval from observed frame gaps is positive feedback and was measured decaying 23 fps to 16.5 fps.
- Only two phones are live here, so nothing establishes how many phones one PC's single receiver can actually carry. Do not extrapolate a per-PC capacity number from a two-phone test, and do not port Phonefarm's 50-to-100-phone mode table, whose measurement basis is undefined and which does not exist on XP.
- `mdns_type` and `allow_ip_list` were not probed and the doc's wording does not clearly separate the two modes. Treat them as an untested lever, and note they gate AirPlay discovery only, never the API.
- Any client library ported from the Phonefarm docs fails loudly but unhelpfully on every call: the XP kernel splits `fun` on "/" and expects at least two segments, so bare function names return status 500, "List index out of bounds (1)", rather than a clean unsupported error. Guessed paths return 404 with a Chinese "interface does not exist" message. Porting needs a full path rewrite, not a compatibility shim.

SCALE

Polling scales linearly with no shared kernel bottleneck. One phone sustains 84 req/s at 2.0 ms p50; two phones flat out measured 83 and 85 req/s, identical to either alone; ten pollers reached 512 req/s aggregate. A wall A/B showed the master unaffected, 17.7 fps with the wall off against 17.6 fps with it running. Input traffic and frame

polling do not interfere in either direction. Receiver count is the hard stop: one receiver per kernel, so capacity grows by adding PCs, with the per-PC ceiling unmeasured.

Pixel-exact or color-accurate assertions on mirror frames

NOT POSSIBLE REFUTED

`/pic/screenshot` returns the AirPlay decoder's frame at `imgw` x `imgh`, 500 x 1080 on these XRs, against an iOS logical size of 414 x 896 and a physical panel of 828 x 1792. It is a rescaled, lossily encoded stream whose idle frame-to-frame noise measures 0.02 to 0.03 mean absolute difference with nothing happening. One-pixel misalignment, anti-aliasing changes, sub-pixel text rendering and exact color values cannot be asserted from it, and XP's screenshot parameter table has no `original` flag to escape the rescale.

HOW

1. `get_device_screenshot` XP `GET /api/pic/screenshot?id=<deviceid>&binary=true` — fast structural check. Raw BMP, 2 ms, 1,620,054 bytes.
2. `none` XP `none (client-side)` — compare by mean absolute pixel difference. Measured on this rig: untouched phone 0.02 to 0.03, a real UI reaction 20 to 160.
3. `none` XP `GET /api/pic/screenshot?id=<deviceid>&rect=[l,t,r,b]&binary=true` — crop server-side to the region under test to cut both noise and payload. `rect` and `save_path` are XP-only additions.
4. `send_key fn_key` XP `GET /api/key/sendkey?id=<ids>&fn_key=shift+win+3` — the exact route, per screen. The phone takes its own screenshot into Photos at the panel's native 828 x 1792.
5. `shortcut#1` XP `GET /api/shortcut/album/get?id=<deviceid>&num=1` — list the new entry. XP has no date filter, so pick the newest by `create_time` or diff the listing before and after.
6. `shortcut#2` XP `POST /api/shortcut/album/down {id, list:[{album_name:"", name:"IMG_xxxx", ext:"png"}], zip:1, outtime}` — pull the PNG to `<install dir>\Shortcut\Media` and diff it exactly on the PC. Name and extension are separate fields on XP, unlike Phonefarm's single filename string.

NEEDS

- Mirroring live for any `/pic/screenshot` call, and a bound, connected board plus Full Keyboard Access for the `shift+win+3` hotkey.
- For the exact route, the iMouse Shortcut installed, kernel-IP-edited, run once with all permissions allowed and bound to an AssistiveTouch custom key, with Shortcuts Advanced set to allow sharing large amounts of data and Photos permission granted on first run with the phone unlocked.

WATCH OUT

- Fixing the phone's display settings does not recover color accuracy. Brightness, True Tone, Night Shift and `air_ratio` all shift sampled RGB, but the deeper reason is structural: the AirPlay stream's H.264-family encoding uses 4:2:0 chroma subsampling by design and discards color detail on every frame regardless of any iOS setting.
- Raising `air_ratio` is a red herring, not a fidelity fix. It changes the encoder's output resolution without touching the lossy compression or the per-phone source cadence, so it makes images larger without making them more exact, and it invalidates every template and color table built at the previous ratio.
- The exact route is doc-derived, not measured. No `/shortcut/*` call has been exercised on this rig, and a missing binding fails silently for about 15 seconds before returning `data.code` 30. Verify it with a single dry run, send `shift+win+3` then poll `/shortcut/album/get`, before writing it into a deliverable as supported.
- Even the native PNG is exact to what the screen displayed, not to a render target. It is still a composite from the OS compositor and its font rendering.

- Never assert on frame equality. A mirrored screen never repeats a frame exactly, and this project already drew a wrong conclusion once by byte-comparing frames and deciding an untouched phone had reacted.
- Harvesting native screenshots fills the camera roll. Plan cleanup with `/shortcut/album/del`, and note that `/shortcut/album/clear` is destructive and should stay behind a human confirmation.

SCALE

Mirror diffs are cheap: 2 ms per raw frame, about 84 req/s per phone, phones independent, and input traffic does not interfere. A full-fleet structural sweep of 19 phones is well under a second of API time, bounded in practice by how fast the UI under test actually changes and by new-frame availability, which is 29 per second on one phone here and 4.5 on the other. The native-PNG route is orders of magnitude slower: a hotkey, a Photos write, an album listing and a LAN transfer, four serialized single-id steps per screen per phone, estimated at 5 to 15 seconds each. Budget it for the handful of screens that genuinely need exactness.

TRAP **Hashing frames to detect change**

MD5 or byte equality reports "changed" on an idle, untouched phone every single time, because the clock, the AirPlay encoder's noise and wallpaper parallax move pixels on every frame. Any change detection must threshold a mean absolute pixel difference, ideally against an idle control. The measured floor on this rig is 0.02 to 0.03; a real reaction measures 20 to 160; a landed pointer, visible only in a small box around the commanded coordinate, measures 1.4 to 9.8.

App state, traffic and builds

Structured app state: DOM, accessibility tree, element list, logs

NOT POSSIBLE REFUTED

There is no "get elements" call in either edition. Neither API exposes the iOS accessibility tree, a view hierarchy, a web DOM, app variables, console output, crash logs or anything inside an app sandbox. The only observable state is pixels, colors within those pixels, plug-in recognition results on those pixels, and whatever the iOS Shortcut can put on or read from the clipboard and the Files app. This single fact makes the whole automation model on this rack coordinate-and-vision based, so every use case elsewhere in this document that assumes element-level targeting inherits the same limitation and needs its own per-app, per-iOS-version coordinate baseline.

HOW

1. `none` XP `none` — the capability itself. The full XP endpoint set and the Phonefarm function list were searched; nothing returns element structure, and guessed paths return HTTP 404.
2. `click + send_key + shortcut#11` XP `GET /api/mouse/click?id=<id>&x=&y=&time=600, then /api/key/sendkey?fn_key=WIN+a, then fn_key=WIN+c, then GET /api/shortcut/clipboard/get?id=<one id>` — the strongest substitute for text. `time` is the delay between press and release, so 600 ms is a long press that raises the Select All menu where `WIN+a` is not honored. Returns exact Unicode with zero structure.
3. `find_image / find_image_ex` XP `GET /api/pic/find-image or /api/pic/find-image-cv` — element positions by template. Returns `data.list[{index, centre, rect, code}]` in mirror pixels, so `centre` feeds `/mouse/click` directly. Plug-in gated.
4. `ocr / ocr_ex` XP `GET /api/pic/ocr, /api/pic/ocr-ex, /api/pic/find-text, /api/pic/find-text-ex` — labels and on-screen strings as a flat list of strings with boxes. Unavailable here: `/pic/ocr` returns `data.code` 28.

5. `find_multi_color` XP `POST /api/pic/find-multi-color with list:[{first_color:"x|y|RRGGBB-tolerance,...", rect, similarity}]` — an "is the UI in state X" check. No plug-in is named for it in either doc, which makes it the most likely recognition function to work on this rig, but that is an inference from the docs and not a measurement: it has not been probed, and a `data.code` 28 would prove it gated after all.
6. `get_device_screenshot` XP `GET /api/pic/screenshot?id=<one id>&binary=true` — the fallback that always works. Run your own OCR or template matching on the PC, budgeting 100 to 300 ms of client-side recognition against 2 ms of capture.
7. `mouse_reset_pos` XP `GET /api/mouse/reset?id=<id1,id2,...>` — before any tap-critical vision flow. A flow that clicks a correctly located center and sees nothing happen is usually the stuck-click state, not a bad match.

NEEDS

- For the clipboard route, the iMouse Shortcut installed, IP-edited and bound on each phone (unverified here), Full Keyboard Access on so iOS honors `WIN+a` and `WIN+c`, and a view whose content is genuinely selectable text. Native table cells, images and canvas content are not.
- For the template and OCR routes, the corresponding vendor plug-in installed and connected via `opencv_num` or `ocr_num` with a kernel restart, which is not the case here.
- For any vision flow, a per-app, per-iOS-version coordinate and template baseline, re-measured whenever the app or iOS updates.

WATCH OUT

- Every substitute returns flat data: a string, a center point, a match or no match. Never hierarchy, role, enabled state or identity.
- The clipboard route is asymmetric in cost. `WIN+a` and `WIN+c` fan out to N phones in one `/key/sendkey` call, but `/shortcut/clipboard/get` is single id and each round trip costs roughly a second over Wi-Fi.
- Apps can remove Select All from the callout menu entirely, common in banking and DRM-protected reading apps, and the automation cannot distinguish that from an empty field: both produce no meaningful clipboard change. Run a positive control against a known-good selectable field alongside the target to confirm the mechanism is even active for that app.
- The clipboard is a single shared slot. A scrape overwrites whatever a paste step was about to use, so sequence them deliberately.
- Correction to a common overstatement: the Files app and the photo albums are listable and downloadable through the Shortcut on iOS 15 or later. What is unreachable is another app's private container, which is where app state, databases and logs live.
- The two phones here already have different home screens and wallpapers, so the same coordinate hits a different app on each. Normalize with two Home presses before reusing any coordinate across phones, and do not assume identical layouts.
- Lifting screen content or clipboard text from apps holding other people's data is personal-data processing the moment it lands in a PC-side log, and the frames capture notifications and account names incidentally. Restrict scrapes to accounts you own or have written authorization for, keep derived fields rather than whole frames, and set a retention limit. Automated extraction from third-party apps and sites is also typically a Terms of Service breach regardless of technique.

SCALE

Raw frame capture is 2 ms per phone with phones independent, so vision input is effectively unbounded. Client-side recognition adds 100 to 300 ms per frame and is bounded by PC CPU, not the kernel. The clipboard scrape is the bottleneck at roughly 1 to 2 seconds per phone, because `/shortcut/clipboard/get` is a single-id

Shortcut round trip with a 15 s default timeout. Once the long-press selection step, the positive control and the per-app copy-availability check are included, call it a few hundred reliable text lifts per hour per phone for real content. Every recognition endpoint, `/pic/find-image`, `/pic/find-image-cv`, `/pic/ocr` and `/pic/find-multi-color`, is explicitly single id, so recognition never batches across phones either.

Capturing the phone's network traffic, ad requests or push payloads

NOT POSSIBLE REFUTED

Neither edition exposes a proxy, packet capture, HAR export or any traffic hook. A grep of the whole XP doc set returns no endpoint that reads a request, a response body or a push payload. The only network-adjacent calls are the Shortcut IP lookup and the connectivity toggles `/shortcut/switch/flight`, `/cdpd` and `/wlan`. What the rack can observe is what renders on screen, so an ad-verification or attribution deliverable can only be "the rendered creative, the timestamp, the phone identity and the egress IP", never the request and its response.

HOW

1. `get_device_screenshot` XP `GET /api/pic/screenshot?id=<deviceid>&binary=true` — pull a raw mirror frame of the rendered creative or banner. Pixels only; no network metadata accompanies a frame.
2. `send_key fn_key + ocr` XP `GET /api/key/sendkey?id=<ids>&fn_key=NoticeBar, then GET /api/pic/ocr?id=<deviceid>` — open Notification Center and read the banner text. `NoticeBar` is in the XP hotkey table and absent from Phonefarm's. This reads rendered text, never an APNs payload.
3. `find_image / find_image_ex` XP `POST /api/pic/find-image-cv {id, img_list, rect:[l,t,r,b], similarity, all, same}` — identify the creative by template instead of by request body. Plug-in gated on `opencv_num`, unprobed here.
4. `none` XP `GET /api/device/get → ip` — the LAN address for phone-to-request correlation on the local segment, with no Shortcut needed.
5. `shortcut#24` XP `GET /api/shortcut/device/ip?id=<deviceid>&state=1` — the extranet address, only needed if requests leave via NAT with a different apparent IP per phone, which is network-dependent and unverified here.

NEEDS

- Mirroring live for any frame- or recognition-based observation.
- The OCR plug-in connected for `/pic/ocr` and `/pic/find-text`, which is not the case here, and `opencv_num` at 1 or more for `/pic/find-image-cv`, unprobed.
- The iMouse Shortcut installed, kernel-IP-edited and hotkey-bound for `/shortcut/device/ip`.

WATCH OUT

- The notification fallback is doubly blocked here, not singly. Even with the OCR plug-in connected, opening Notification Center surfaces a push banner only if the phone is unlocked and the app has not suppressed banners.
- Mirror frames carry no timestamp or provenance of their own, so an evidence package must record the request you sent, the id, the coordinates and the wall-clock time alongside the frame.
- XP's `rect` is a flat `[left, top, right, bottom]` array. The Phonefarm four-corner-point form does not apply here.
- The nearest real workaround, an HTTP interception proxy set by hand in each phone's Wi-Fi settings, sits entirely outside this API and cannot be scripted through the kernel at all. It is a footnote, not a chain step. Certificate pinning in most SDKs defeats it anyway, installing a CA profile weakens that phone's TLS trust until it is removed, and it must be confined to apps and accounts you own, since intercepting other people's traffic engages wiretap and computer-misuse law as well as those apps' terms.

- A static-IP-without-gateway Wi-Fi arrangement means cellular traffic never crosses a LAN proxy at all.

SCALE

Zero for the capability itself. For the observation fallback, raw frames are 2 ms p50 at about 84 req/s per phone with phones independent, and `/pic/screenshot` is single id so capture is one request each. The useful evidence rate is bounded by the phone's AirPlay source rate, 29 new frames per second on one XR and 4.5 on its twin. Because OCR is non-functional here, the effective rate of verified text extractions today is zero per hour per phone.

Installing builds, reading device internals, driving the app by element ID

NOT POSSIBLE REFUTED

There is no endpoint in either edition to install an IPA or TestFlight build, to read battery, CPU, memory, thermal state or console logs, to report an app's version, or to query a UI element tree the way XCUITest does. A grep across the XP doc set for battery, install, log, element and crash returns only kernel-side logs (`airplay_connect_log`, `restart_log`, `im_log`, `error_push`) and the account pages. Every CI pipeline assumes install the build, drive by element ID, collect logs; on this rack each of those three is absent or a weaker, pixel-based substitute.

HOW

1. `shortcut#13` XP `GET /api/shortcut/exec/url?id=<ids>&url=<https or itms-beta URL>` — open the store or TestFlight page. This opens a URL, it does not install anything. Accepts a comma-separated id list.
2. `click` XP `GET /api/mouse/click?id=<ids>&x=<px>&y=<px>&button=1` — tap Install, Update or Open by coordinate. XP's `button` is numeric 1 to 8, where Phonefarm used the strings "left" and "right". Send `/mouse/reset` to the target set first.
3. `ocr / find_image_ex` XP `GET /api/pic/ocr, POST /api/pic/find-text, /api/pic/find-image-cv` — verify the installed version by reading the app's own About screen. All plug-in gated, so on this rig the check must be done client-side on a raw BMP.
4. `shortcut#11` XP `GET /api/shortcut/clipboard/get?id=<deviceid>&outtime=<ms>` — the only documented value channel back from the phone. It returns whatever is on the clipboard and cannot itself obtain battery, CPU or memory.
5. `shortcut#4 / shortcut#5` XP `GET /api/shortcut/file/get?id=<deviceid>&path=<path>, then POST /api/shortcut/file/down` — retrieve only files the app itself writes into the Files app, iOS 15 or later, both single id, landing in `<install dir>\Shortcut\File`. Crash `.ips` files and `os_log` are not exposed to Shortcuts.
6. `find_image_ex / ocr / find_multi_color` XP `/api/pic/find-image-cv, /api/pic/find-text, /api/pic/find-multi-color` — navigation is coordinate, template, text or color based. All single id.

NEEDS

- The iMouse Shortcut installed, kernel-IP-edited, run once with all permissions allowed and bound to an AssistiveTouch custom key, for every `/shortcut/*` step, plus iOS 15 or later for the file endpoints.
- The board bound (`vid` and `pid` set together through `/device/set`) and connected at board `state` 2, plus the iOS accessibility prerequisites, for any tap-based flow.
- App Store or TestFlight access on each test Apple ID, already signed in, with the phone unlocked so store and trust prompts can be answered by a person.

WATCH OUT

- Triggering a user-authored Shortcut from the PC to report battery is not a supported capability and should not appear in any deliverable that claims it. XP's `fn_key` vocabulary is a fixed enumerated table (`WIN+h`, `Ho`

`me`, `TAB+b`, `TAB+l`, `AppSwitch`, `ControlBar`, `NoticeBar`, `OPENKeyboard`, `shift+win+3` and `+4`, the clipboard chords, IME and volume keys), with no entry for an arbitrary user Shortcut.

- `/pic/find-multi-color` is the one recognition endpoint with no documented plug-in dependency, which makes it plausibly available here, but that is an inference and not a probe. Carry the same hedge it gets elsewhere: assume gated until a call returns `data.code` 0.
- Never script an Apple ID sign-in through these endpoints. The API is unauthenticated and speaks plain HTTP, so anything typed through `/key` is visible to any host on the subnet. Hand account sign-in to a person at the rack.
- The phone's single wired port is occupied by the control board, so USB-side tooling cannot share the device. Crash-log extraction with a desktop utility is a manual, one-phone-at-a-time cable swap, which is throughput zero for unattended automation. Unplugging also drops that phone's control channel from board `state` 2 back to 1 and requires re-binding, and the vendor states the board works only with its supplied cable.
- The fan-out claim holds only for phones with identical layouts. Taps are in mirror pixels and the two XRs here have different home screens, so a coordinate that hits Install on one does not necessarily hit it on the other. Per-phone calibration is required and was not verified.
- There is no rollback. An install driven by taps cannot be uninstalled by any API function either.
- Repeated installs across many devices under one Apple ID can trip App Store limits and TestFlight tester counts. Use accounts you own and stay inside their terms. Enterprise or ad-hoc distribution profiles change a phone's trust posture and should be a deliberate, documented decision.

SCALE

Install by UI is roughly five taps plus download time per phone. Navigation calls fan out in one request where layouts match, but every verification step is single id and must be looped, and pointer travel adds about 0.6 ms per pixel regardless of fleet size. Serialized behind download completion that is on the order of 1 to 3 attempts per minute per phone under ideal conditions, lower in practice because store-side throttling of many downloads from one LAN applies and because OCR verification is unavailable here, forcing slower client-side inspection. Device internals and element-ID driving are zero: no endpoint exists, and any number quoted for them describes a manual screen-reading workflow, not an API capability.

The input surface

Pinch, rotate, two-finger and pressure gestures

NOT POSSIBLE REFUTED

The control board is a single-pointer HID mouse and keyboard bridge. Every input endpoint, `/mouse/click`, `/mouse/down`, `/mouse/move`, `/mouse/up`, `/mouse/swipe` and `/mouse/wheel`, commands one pointer position, and `button` 1 to 8 selects a mouse button, not a finger. There is no second-pointer call, no touch-count parameter and no pressure parameter anywhere in either edition, so pinch-to-zoom, two-finger rotate, multi-finger swipes and pressure gestures cannot be emitted. "Swipe" in this API means a one-finger drag, not gestures in general.

HOW

1. `click / mouse_down / mouse_move / mouse_up / swipe / mouse_wheel` XP `that is the complete mouse surface` — establish the limit from the endpoint set itself. `/mouse/move` takes only `id`, `x`, `y`: one position. `/mouse/swipe` takes one start and one end.

2. `click x2 at the same coordinate` XP `GET /api/mouse/click?id=<ids>&x=250&y=540 twice in quick succession` — workaround A, app-native double-tap zoom. Do not move between the taps: the board walks the pointer at about 0.6 ms per pixel, so any repositioning breaks the interval.
3. `mouse_wheel` XP `GET /api/mouse/wheel?id=<ids>&direction=up|down|left|right&len=30&number=1` — workaround B, where content scrolls rather than zooms. `len` is 1 to 127 and XP adds the horizontal pair Phonefarm lacked. Wheel scrolling avoids the pointer-walk cost entirely.
4. `click` XP `/api/mouse/click on the app's own +/− buttons or zoom slider` — workaround C, the only one that is parameterizable, repeatable and verifiable. If the app under test is yours, a debug zoom control costs less than any of the above and gives an exact factor instead of an opaque gesture.
5. `get_device_screenshot` XP `GET /api/pic/screenshot?id=<id>&binary=true before and after` — verify by mean absolute pixel difference, in a box around the expected content region rather than whole-frame.

NEEDS

- Board bound and connected plus the full iOS accessibility checklist, none of it queryable over the API.
- Coordinates in mirror pixels read per device from `/device/get`, never iOS points.
- Two Home presses to normalize before mirroring any coordinate across phones.

WATCH OUT

- This is a hardware and protocol limit, not a configuration gap. The board presents a 9-byte vendor HID output report as a mouse and keyboard bridge, and the kernel exposes exactly one pointer. No plug-in, license tier or iOS setting adds a second contact point, and iOS treats a Lightning-connected pointer accessory as one AssistiveTouch cursor in any case.
- The double-tap workaround is inferred from a timing budget, not observed. iOS's recognition window is roughly 300 to 500 ms in practice and Apple does not publish the figure; what is measured here is only that the input channel is fast enough to fire both taps inside it. Confirm with a before and after diff showing an actual scale change before relying on it. Apps that do not implement double-tap zoom simply get two taps.
- Ctrl plus wheel is not expressible: `/key/down` accepts single keys and documented `fn_key` combinations, and a bare CTRL is not in that vocabulary.
- Replaying a recorded AssistiveTouch custom gesture is an iOS feature, not an API feature, and it is location-locked: apps that zoom about the pinch center will zoom about the point where the gesture was recorded, and the replay cannot be parameterized per call. It also requires re-enabling the floating AssistiveTouch menu against the vendor's prescribed configuration, which puts a permanent button in every frame and invalidates templates and pixel-diff baselines captured without it. Worst of all, the drift is undetectable through the API, since no gesture-list endpoint exists, so a silently wrong replay looks identical to a correct one in logs.
- A whole-frame diff cannot distinguish a successful zoom from an accidental app switch or a keyboard appearing: all three land in the 20 to 160 range. Box-diff the content region.
- None of the recognition-based verification paths are available on this rig, so success or failure has to be judged by client-side pixel diffing of raw BMP frames.
- Pressure and haptics are unobservable regardless: the mirror carries pixels only, so even a pressure gesture, if it could be emitted, could not be verified.

SCALE

Irrelevant to the limit itself. Of the substitutes, double-tap and wheel scroll fan out to the whole rack in one request each, since both take comma-separated id lists and the board walks every listed phone in parallel at the cost of one. Verification is single id but phones are independent, so a before and after pair costs roughly 5 to 15

ms per phone and a 19-phone sweep stays well under a second. The real gate is the app's own animation settle, roughly 300 to 600 ms, which puts a fully confirmed action at single digits per minute per phone and an unconfirmed, pipelined one in the thousands per hour.

Walls the API cannot cross

Apple Pay, Face ID and side-button confirmations

NOT POSSIBLE REFUTED

The rack cannot cross the Apple Pay sheet, a Face ID prompt, or any step that needs a physical button press. The control board is a HID pointer and keyboard bridge, and the XP `/key/sendkey` hotkey table, read in full in both places it appears, contains no side button, no power button and no biometric action. The nearest entries are `TAB+l` (lock) and `CTRL+ALT+SHIFT+WIN+r` (restart). The furthest the API reaches is tapping the control that raises the sheet, then screenshotting the result.

HOW

1. `click`, `swipe` XP `/api/mouse/click, /api/mouse/swipe` drive the cart and checkout up to the payment sheet. XP `button` is numeric (1 left, 2 right); `len` is a fraction, 0.9 meaning 10% to 90%. Coordinates are mirror pixels, 500 x 1080 here.
2. `click` XP `/api/mouse/click` taps the Apple Pay or Confirm control. This is the last action the API can take in the flow.
3. `get_device_screenshot` XP `/api/pic/screenshot?id=…&binary=true` captures the sheet as evidence of where the flow stopped. Raw BMP is 2 ms and 1,620,054 bytes on this rig, and the call can be repeated to watch the sheet time out or be cancelled, so the API is not blind after the tap, only unable to act.
4. `send_key` XP `/api/key/sendkey?id=…&fn_key=…` confirms the absence: the XP hotkey enumeration is closed, and the Phonefarm list is a strict subset of it.
5. No API call. Auto-Lock set to Never on each phone, in Display & Brightness, keeps the lock gate out of the run. This is per-phone iOS configuration and invisible to the API.
6. No API call. A merchant sandbox with a stored non-biometric method is the only route that lets a checkout flow complete unattended, and it must be a genuine test environment.

NEEDS

- Board bound and connected, valid mouse parameters, and the iOS accessibility settings, for the taps that reach the sheet at all.
- Auto-Lock set to Never on each phone so the lock gate does not appear mid-run.
- A merchant test or sandbox environment if the flow has to finish without a person present.

WATCH OUT

- This is a platform boundary, not a rig limitation. No plug-in, professional edition, Shortcut or firmware version in either document changes it.
- Never push payment credentials or device passcodes through `/key/sendkey` or `/shortcut/clipboard/set`. The API is unauthenticated and reachable by any host on the LAN, and it would breach payment-network and platform rules in any case.
- A restart by any route, `/device/restart`, `/shortcut/switch/device state=1` or `fn_key=CTRL+ALT+SHIFT+WIN+r`, leaves the phone at the passcode screen. Mirroring drops too, so `air_pid` goes empty and `state` returns 0. An automation loop must detect that and route to a human queue rather than keep tapping a dead mirror.

- The two XRs here have different home screens and app layouts, so fanning a checkout tap out to a comma-separated id list risks hitting a different control on each phone. In a live payment flow that is a real charge, not a cosmetic error.
- `/pic/screenshot` is single-id, so evidence capture is one request per phone even though the taps that got there fan out.

SCALE

Not improved by batching. Driving N phones to the sheet costs one comma-separated tap sequence and well under a second of board time regardless of N, so arrivals at the sheet are as cheap as the browse flow allows. Completions are zero per hour by platform design, at any fleet size, because every arrival then queues on a person. Plan the rack around queueing sheets for a human, or around a sandbox that avoids the sheet entirely.

No outbound events, no caller authentication

Kernel-originated webhooks or outbound event calls

NOT POSSIBLE REFUTED

Neither edition can be told to POST an event, such as device offline, board unplugged or cast finished, to a URL you own. The kernel's entire event surface is a set of JSON callbacks broadcast to clients that have already connected to its own WebSocket, `ws://127.0.0.1:9911/api` on XP and `ws://host:9911/clinet` on Phonefarm. The XP configuration endpoint exposes 19 fields and not one is a URL, endpoint, notification or webhook field. The only way an event leaves the rack is a bridge process you run, holding a persistent socket, making the outbound call itself with its own authentication.

HOW

1. `none` (Phonefarm has no kernel-configuration function beyond airplay server count and restart) XP `/api/config/imserver/get` confirms the absence: `air_play_name`, `lang`, `mdns_type`, `connect_failed_retry`, `air_play_ratio`, `opencv_num`, `ocr_num`, `allow_ip_list`, `air_play_fps`, `air_play_img_fps`, `air_play_refresh_rate`, `air_play_port`, `air_play_audio`, `auto_connect`, `auto_updata`, `thread_mode`, `mouse_mode`, `flip_right`, plus `enable_hardware_acceleration` on `/set`. No callback address anywhere.
2. `ws://host:9911/clinet` (the vendor's own spelling) XP `ws://127.0.0.1:9911/api` opens the only event channel, and the client opens it. XP serves HTTP and WebSocket on the same port. On connect the kernel sends `im_connect` with `data.ver`, the only place the XP kernel version is exposed.
3. `dev_connect`, `usb_change`, `collection`, binary loop-screenshot frames XP `im_connect, dev_connect, dev_disconnect, dev_rotate, dev_change, dev_delete, group_change, group_delete, usb_change, collection_mouse, airplay_connect_log, restart_log, user_info, im_log, error_push, im_config_change, logout, dev_sort_change`: all 18 XP callbacks are verified in the callback page. The envelope is `{fun, status:200, message, msgid:0, data:{list, code, id, message}}`, with device records inside `data.list`. `usb_change` carries `data.code` 19 (board to PC), 20 (board off PC), 12 (board to phone).
4. `none` XP `none`: debouncing and the outbound call with your own credentials is entirely your code.
5. `none` XP `user_info callback plus a periodic GET /api/device/get` monitors the bridge itself. `user_info` is pushed at login and every 30 s, so it doubles as a socket heartbeat, but only while the kernel is logged in. The periodic `/device/get` is the transport-independent liveness check.

NEEDS

- A bridge host, which can be the rig itself, with outbound network access to wherever the events are going.

- IMConsole.exe running and logged in. With only IMService.exe up, the live probe showed both phones at state 0 and an empty board list, so no meaningful events would ever fire.
- Kernel reachable on TCP 9911, restricted at the Windows firewall rather than trusted to be reached only by the bridge.
- A WebSocket client library, and reconnect logic, because `/imserver/restart` drops every socket.

WATCH OUT

- There is no HTTP long-poll, no server-sent-events endpoint and no queue to drain. Anything broadcast while the bridge was down is lost, so reconcile with `/api/device/get` and `/api/config/usb/get` on every reconnect.
- XP has no server-push frame stream at all, so job-finished signals derived from the screen must be polled with `/pic/screenshot`, never awaited as an event.
- `error_push` is the only asynchronous failure signal, and it does not cover this rig's most consequential failure mode. A stuck click returns `data.code` 0 while taps are not landing, measured at 0.09 against 141.96 mean pixel difference. Nothing in the source material says a stuck click raises `error_push`, so treat that as no rather than as an open question. The only working detector is a pointer-box pixel diff on the AirPlay frame, which lives on the video channel, not the event channel.
- Any outward alert claiming action X succeeded on phone Y must be backed by a screen diff, not by the absence of an error callback.
- Multiple clients each receive every broadcast, so a second subscribed bridge is legitimate redundancy. It also means a rogue LAN client can read every broadcast, including device names, ip, mac and user_name, with no way for your bridge to know it is not alone.

SCALE

Callback volume is negligible and does not grow with phone count: connect and disconnect events, board plug events, config changes, and a `user_info` heartbeat every 30 s, all on one shared socket. One WebSocket client comfortably serves the whole 19-board rack, and extra clients cost the kernel almost nothing. Frame data never travels on this channel; video is polled per phone at 2 ms and 1.62 MB a frame, single id, with phones independent at 83 to 85 req/s each.

Authenticating API callers: the XP kernel is open to anything on the LAN

NOT POSSIBLE REFUTED

Neither edition has any concept of a client credential. The two settings integrators mistake for access control do something else. `/config/user/login` licenses the kernel to the vendor account and returns `total_license`, `overdue_time` and `user_state`; `mdns_type` and `allow_ip_list` are documented as screencast discovery rules, governing which phones may find the AirPlay receiver. No XP endpoint takes a token, key or session, and every vendor example calls a remote host with no credential. Any host that can reach TCP 9911 can enumerate devices, poll frames, inject taps and text, rewrite kernel config, and call `/imserver/restart`, which takes no parameters at all.

HOW

1. `get_device_list` XP `GET http://<rig ip>:9911/api/device/get` returns the full device list with no challenge. The equipment doc's own examples are remote-host calls to 192.168.9.9:9911 with no credential of any kind. On this rig the same call returns both XRs with ip, mac, user_name, model and board binding.
2. `none` XP `/api/config/user/login, /info, /logout, /switch` confirms the login endpoint is licensing, not caller auth: the response is entirely licence state. After logout the kernel refuses devices but the HTTP and WebSocket API keep answering.

3. `none` XP `/api/config/imserver/get | /set` confirms `mdns_type` (0 allows all, 1 specifies the IP, 2 the IP in the list) and `allow_ip_list` ("It only takes effect when the rule is 1") gate AirPlay discovery only. Setting `mdns_type=1` narrows who can find the advertised receiver name, not who can call port 9911.
4. `restart` XP `/api/imserver/restart` shows the blast radius: its parameter table is `fun` only, no id and no credential. Stated for threat modeling, not as a procedure to run against a rig you do not own.
5. `none` XP `none`: the only real fix is outside the API. A Windows Firewall inbound rule scoping TCP 9911 to the controller hosts, TCP 17000 and UDP 5353 scoped to the phone VLAN or SSID, optionally a reverse proxy or gateway in front of 9911 that adds a token, rate limits and an audit ledger, and an ACL on `C:\iMouseXP\IMService.json`.

NEEDS

- Nothing. The exposure is the default state and cannot be turned off from inside the API.
- To mitigate: administrator rights on the kernel PC's firewall, and control of the switch or AP to put phones on their own VLAN or SSID.

WATCH OUT

- There is no documented token, key, session, per-endpoint permission, caller allow-list or audit log in either edition. The XP error table, codes 0 to 41, contains no unauthorized code, which is itself evidence that callers are never checked.
- The absence is documented by omission plus the vendor's own remote-host examples, and corroborated by the live probe here: unauthenticated GETs to `/api/device/get`, `/api/config/usb/get` and `/api/device/group/get` all answered.
- A reverse proxy helps only if 9911 itself is firewalled to localhost or the proxy. Otherwise callers simply bypass it.
- `C:\iMouseXP\IMService.json` stores the vendor account password as trivially reversible hex obfuscation, so file access to the PC is a live credential regardless of network rules. Firewalling 9911 does nothing about that.
- The kernel keeps no caller log, so a firewall rule applied to the wrong interface or VLAN is undetectable from inside the product. Catching it needs an external port scan or a deliberate read-only probe from a second host you own.
- Describing the exposure is not permission to reach a rig you do not own. Probing or driving someone else's kernel over a network is unauthorized access.

SCALE

Not throughput-bound. This is a binary exposure, and one firewall rule covers an entire rack regardless of phone count. In a multi-kernel deployment each additional PC is an additional open surface needing its own rule, because there is no central policy point inside the vendor stack; a shared gateway in front of them is the only place a token, rate limit or audit trail can exist. The exploit is cheap: a raw BMP frame costs an uninvited caller 2 ms and 1.62 MB per phone per call, and ten concurrent pollers sustained 512 req/s against this kernel with no degradation, so continuous video exfiltration of the whole rack is trivial once a host is on the LAN.

WRONG **allow_ip_list is not an API access control**

It is a screencast discovery rule, and by its own description it applies only when `mdns_type` is 1. Setting it changes which phones can find the AirPlay receiver. It changes nothing about who can call `/api`. Every mitigation for caller access lives at the network layer or on the host.

Input primitives and their ceilings

Keyboard: ASCII only, one key per down or up, no cadence or repeat control

PARTIAL

WITH CAVEATS

`/key/sendkey` types a string of "English, numbers and English characters" or fires one hotkey from a closed vendor list. `/key/down` and `/key/up` take a single key each; `/key/down` additionally accepts `fn_key` for a combination, but `/key/up` has no `fn_key` parameter, so `/key/upall` is the only documented way to clear a held chord. None of the four keyboard endpoints has a parameter for inter-key delay, key repeat, typing speed or maximum length, and non-ASCII text is not supported at all. The documented XP route for Unicode is the clipboard plus `WIN+v`.

HOW

1. `send_key` XP `/api/key/sendkey?id=…&key=user123` types an ASCII string in one call. The XP parameter table is exactly `fun`, `id`, `key`, `fn_key`. `id` takes a comma list, so one request types the same string on N phones. Send text as POST JSON in UTF-8 so punctuation is not mangled in a query string.
2. `send_key`, one call per character or chunk XP `/api/key/sendkey, one call per character or chunk` is the only cadence lever. Whatever timing happens inside a single string is the kernel's and is undocumented.
3. `key_down`, `key_up`, `key_release_all` XP `/api/key/down (key, or fn_key for a combination), then /api/key/sendkey, then /api/key/up (key only), then /api/key/upall` holds a modifier or chord.
4. `send_key` with `fn_key` XP `/api/key/sendkey?id=…&fn_key=…` fires a documented hotkey. The XP list is closed: WIN+h, Home, TAB+b, AppSwitch, ControlBar, NoticeBar, TAB+l, OPENKeyboard, shift+win+3, shift+win+4, CTRL+ALT+SHIFT+WIN+r, CTRL+ALT+WIN+p, WIN+c, WIN+v, WIN+a, WIN+x, SwitchIme, BACKSPACE, a bare space, TAB, ENTER, the four arrows, MuteSoundSwitch, VolumeUp, VolumeDown, Play/Pause.
5. `send_text` (not accepted by the XP kernel) XP `/api/shortcut/clipboard/set?id=…&text=…&sleep=…&outtime=… then /api/key/sendkey?id=…&fn_key=WIN+v` enters Chinese, emoji or any non-ASCII text. `sleep` is in seconds because "on some mobile phones it may take a while for the paste to be effective"; `outtime` is milliseconds, default 15 s. Round-trip proof is `/shortcut/clipboard/get`, single id, returning the identical string.

NEEDS

- Full Keyboard Access on with auto-hide 1 s on every phone, or hotkeys and typed characters are not interpreted.
- AssistiveTouch on, the phone bound to its board with `vid` and `pid` set together, and that board at state 2 in `/config/usb/get`.
- A focused text field. No endpoint checks focus, and a send into an unfocused screen silently does nothing.
- For the Unicode route only: iMouse Shortcut installed with the kernel IP edited in, run once with every prompt allowed, and bound to an AssistiveTouch custom key. Not verified on this rig.

WATCH OUT

- No delay, repeat-rate, speed or length parameter exists on any of the four endpoints. This was checked field by field against the XP tables.
- XP's key-name vocabulary is narrower than Phonefarm's. SHIFT, CTRL, ALT, ESC, PAGEUP, PAGEDOWN, F1 to F12 and FN are Phonefarm-only names and are unverified on this kernel; XP's documented route for a combination is `fn_key` on `/key/down`. An interrupted chord leaves a modifier held, and the next plain type is silently corrupted until `/key/upall`.

- Verify with a box diff cropped to the caret or field region, not a whole-frame threshold. The 20-and-above reacted threshold used elsewhere in this handoff was calibrated on full-icon taps (141.96 and 162.66 against an idle floor of 0.02 to 0.03); a single character in a small field can sit close to that floor and read as a false negative.
- A silent no-op and a slow-AirPlay false negative look identical. On phone .27, at roughly 4.5 new frames per second, a real reaction can take 200 ms to appear at all, so poll 2 to 3 frame intervals, 400 to 700 ms, before concluding a keypress failed.
- Nothing on this rig proves the correct characters landed. Frame diff proves the screen changed, and the clipboard round trip proves the clipboard holds string X; `/pic/ocr` answers `data.code` 28, so confirming typed text needs a human or a vision model reading the raw BMP.
- Pair `/mouse/reset` with the focus-a-field-then-type pattern this use case depends on, not just once at session start. The measured stuck-click state is cleared by reset and not by `/mouse/up`.
- The `fn_key=Home` variant needs kernel 2026.2.27 or later. The iOS build on these phones, 18.7.10, is an independent fact and does not tell you the kernel build, which has not been checked here. `WIN+h` is the safe default.

SCALE

The send side is fleet-wide and cheap: every keyboard endpoint accepts a comma-separated id list, so typing to all 19 phones costs the same round trip as one phone, with no pointer-walk cost. There is no measured latency for `/key/sendkey` specifically; `/mouse/move`'s p50 of 0.5 to 11.7 ms is a plausible floor for a similar payload, so budget tens of milliseconds per call when splitting a long string. Verification is the real ceiling and is per phone: at 2 to 3 confirming frames per action, a .68-class phone (about 29 new frames/s) supports roughly 3,000 to 8,000 verified actions per hour, and a .27-class phone (about 4.5 new frames/s) roughly 1,000 to 1,500.

Scroll wheel: capped at 127 per event, with no pixel or position semantics

PARTIAL WITH CAVEATS

`/mouse/wheel` takes `len` 1 to 127, documented only as "Scroll length 1-127" with no unit, times `number` events, in direction up, down, left or right. XP adds the horizontal pair that Phonefarm lacks. Nothing states how `len` maps to pixels, nothing reports the current scroll offset, and there is no scroll-to-position or scroll-to-element call among the seven mouse endpoints. "Scroll exactly 300 px" is not expressible, so the robust pattern is scroll then look, at one screenshot per iteration per phone.

HOW

1. `mouse_wheel` XP `/api/mouse/wheel?id=…&direction=down&len=40&number=3` scrolls. Phonefarm documents up and down only and calls the field `length`. Wheel costs no pointer travel, unlike a swipe of the same distance. Position the pointer over the view you intend to scroll with `/mouse/move` first.
2. `get_device_screenshot` XP `/api/pic/screenshot?id=<single id>&binary=true` grabs a frame. `binary=true` returns raw bytes and no JSON, p50 2 ms and 1,620,054 bytes, bottom-up BMP; `jpg=true` costs 40 ms and caps near 21 req/s; the default base64 JSON form is 26 ms. `rect` crops server-side, `save_path` writes to disk.
3. `find_multi_color`, `find_image`, `ocr` XP `/api/pic/find-multi-color, or client-side matching on the raw BMP` looks for the target. XP's colour format is absolute points, `x|y|RRGGBB-tolerance`, inside a list of configs, returning `data.list[{index, centre}]`.
4. Same calls in a client loop, with a hard iteration cap. Budget the wait per iteration from the phone's own AirPlay source rate, not your poll rate: roughly 220 ms per genuinely new frame on the slow XR.

5. Compare consecutive frames to detect the end of the list. Threshold the mean absolute pixel difference: the idle floor measured 0.02 to 0.03 on a 0 to 255 scale and a real reaction 20 to 160, so anything under about 1 after a scroll means nothing moved. Never compare hashes; MD5 reports changed on an untouched phone because of clock, encoder noise and wallpaper parallax.
6. `swipe` XP `/api/mouse/swipe with sx, sy, ex, ey` is the fallback when positional control matters, at the cost of the roughly 0.6 ms per pixel pointer walk. Two documentation traps: the XP table labels `sx`, `sy`, `ex` and `ey` all as "y coordinate", and the JSON examples spell the parameter `steping` while the table says `steppin g`.

NEEDS

- Board bound with `vid` and `pid` set together, showing state 2 in `/config/usb/get`.
- `location_crc` non-empty on the device record, or every mouse call returns `data.code` 11, no mouse parameters.
- Coordinates in that device's `imgw` x `imgh` mirror space, 500 x 1080 here, never the 414 x 896 logical points.
- Trackpad & Mouse tracking speed at maximum on every phone, since the setting is part of the wheel-to-scroll mapping and a mirrored wheel call only lines up if the phones share it.

WATCH OUT

- The 127 ceiling is per event. Larger movement comes only from raising `number`, which issues more discrete events rather than one bigger one.
- `len` has no documented unit and no documented mapping to pixels or rows in either edition. Treat identical values in different apps, or on different phones, as producing different amounts of scroll until measured.
- No endpoint reports a scroll offset and none accepts a target, so every loop must be bounded by an iteration cap plus a frame check rather than by arithmetic.
- A wheel event delivered while the pointer sits over the wrong view scrolls the wrong thing with no error. `dat a.code` 0 means the board accepted the event, not that anything scrolled.
- All six `/pic/*` endpoints are single-id, so the scroll fans out and the verification does not. OCR is unavailable here (`data.code` 28) and the image-search plug-in status is unverified, so client-side matching on the raw BMP is the only channel known to work today.
- If taps stop registering mid-loop while moves still work, send `/mouse/reset` to the whole target set.

SCALE

One `/mouse/wheel` call drives every listed phone. Verification is one 2 ms raw-BMP request per phone per iteration, and pollers are independent: two phones flat out measured 83 and 85 req/s at p50 3 ms each, and ten concurrent pollers 512 req/s in total. Nineteen phones at five iterations is negligible against the phones' own frame production, which is the real ceiling at about 29 fps on the good XR and 4.5 on its twin. Wheel is the only scroll primitive whose cost does not grow with the distance travelled.

Pointer-walk speed caps swipes: velocity-dependent flicks are out of reach

PARTIAL

WITH CAVEATS

The board walks the pointer to its destination at a measured 0.6 ms per pixel, so a full-height 1080 px swipe takes roughly 580 ms of travel and iOS reads it as a slow drag rather than a flick. Neither `/mouse/swipe` nor `/m ouse/move` exposes a duration or velocity parameter. The only levers are distance, the `stepping` and `step_slee`

`p` loop, and `brake`. Momentum fling scrolling, fast card-swipe UIs, velocity-thresholded swipe-to-dismiss and many game gestures cannot be produced directly.

HOW

1. `mouse_move` twice XP `/api/mouse/move?id=…&x=20&y=540 then x=480` measures the floor. Measured ladder: 2 px 1.0 ms, 100 px 78 ms, 400 px 250 ms, 1000 px 592 ms. Latency is a function of distance, not jitter, so never read a 264 ms move as noise.
2. `swipe` XP `/api/mouse/swipe?id=…&direction=left&len=0.8` prefers the short axis. Read the mapping carefully: 0.9 slides from 10% to 90% and 0.8 from 20% to 80%, so `len` is the far edge and the span is twice `len` minus one. On the 500 px width, `len` 0.8 gives a 300 px stroke, about 180 ms, and `len` 0.6 gives only 100 px. The 1080 px height is the expensive axis.
3. `swipe` with explicit coordinates XP `/api/mouse/swipe with sx, sy, ex, ey in mirror pixels` finds each app's page-turn threshold empirically. A 150 to 200 px stroke is usually enough. XP adds `stepping` (loop count), `step_sleep` (ms between steps) and `brake` (stop immediately after sliding); Phonefarm's equivalent is the single `for` loop count.
4. `mouse_wheel` XP `/api/mouse/wheel?id=…&direction=down&len=127&number=k` substitutes for momentum scrolling. It buys distance, not velocity.
5. `click` with `button:'right'` or `time:700` XP `/api/mouse/click?id=…&button=2 or &time=700` substitutes a long press or secondary click for a fast reveal swipe. Swipe-to-delete rows usually have an equivalent context menu behind a long press.
6. `mouse_up`, `mouse_reset_pos` XP `/api/mouse/up?id=…&button=1, /api/mouse/reset?id=…` recovers after any composed gesture. A real drag is down, streamed moves, then up; an interrupted one leaves the button held.

NEEDS

- Board bound and at state 2, `location_crc` non-empty, or mouse calls return `data.code` 11.
- All coordinates in the device's `imgw` x `imgh` mirror space, 500 x 1080 on these XRs.
- Per-phone iOS accessibility settings: AssistiveTouch on with tracking sensitivity at maximum, Trackpad & Mouse tracking speed at maximum, portrait lock, Auto-Lock never. None of these is queryable over the API.
- Optional: `/config/imserver/set mouse_mode=true`, documented as "use fast and accurate mouse movement" and applying without a kernel restart. Whether it changes the walk rate is unverified here.

WATCH OUT

- This is a hardware floor, not a missing parameter. No duration, velocity or speed argument exists in either edition, so a genuine high-velocity flick is not achievable.
- The substitutes are not equivalent: the wheel gives distance without momentum, and a long press opens a menu rather than performing the swipe the UI expected.
- A stroke slow enough to read as a drag can pick up and move an icon or reorder a list instead of scrolling.
- `brake` and `step_sleep` get one line each with no timing semantics. Measure their effect per app.
- Phonefarm documents that percentage-mode swipes randomly vary their start and end coordinates, and does not say whether that still applies with explicit coordinates. XP documents no randomization at all, so verify stroke geometry from the frame on either edition.
- `data.code` 0 means the board accepted the call. Follow every composed drag with `/mouse/up` and confirm with a frame diff.

SCALE

Walking the pointer on N phones in parallel costs the same as one, because every mouse endpoint takes a comma id list and the board drives the listed phones together: a 300 px stroke on 19 phones is still about 180 ms of wall time. The binding constraint is per-gesture latency, not throughput. Input traffic was measured not to slow frame polling, 98.8 req/s while streaming moves against 78.7 with no input, so verification can run concurrently. Exclude from any mirrored gesture a phone whose `imgw` or `imgh` differs from the master, since the same coordinate is a different place on a different-sized mirror.

Rotation: dev_rotate is the only signal, and the landscape mapping is undocumented

PARTIAL

WITH CAVEATS

Rotation is detectable, not controllable, and not documented. XP exposes `rotate` (0 vertical, 1 horizontal) in every device record and broadcasts `dev_rotate` with the full record in `data.list`. There is no endpoint to rotate a phone, lock orientation or read the lock state; that is a Control Centre setting. What neither document states is how `imgw`, `imgh` and the coordinate origin behave when `rotate` flips to 1, so any landscape transform must be measured on the rig before it is trusted. XP's only landscape knob is the kernel-wide Boolean `flip_right`, documented by name alone.

HOW

1. `none` (phone-side setup) XP `none`: lock portrait on every phone, close StandBy, and give Control Centre two rows of icons, which the vendor says "affects automatic screencasting". No API in either edition sets or reads orientation lock, verified by absence across all nine XP doc pages.
2. `get_device_list` XP `/api/device/get` reads `rotate` and the mirror geometry per phone. `imgw` and `imgh` are the actual image size and the input coordinate space; `width` and `height` are logical points, 414 x 896 here, and are not. Live: both XRs `rotate` 0, 500 x 1080.
3. `callback dev_rotate` XP `dev_rotate on ws://127.0.0.1:9911/api` subscribes to rotation events. The record is inside `data.list`, not `data`, and the XP socket path is `/api`, not Phonefarm's `/clinet`.
4. `mouse_move` plus `get_device_screenshot` XP `/api/mouse/move then /api/pic/screenshot?id=…&binary=true` measures the landscape mapping instead of assuming one. Rotate a phone by hand, reset, move to three known points, and mean-abs-diff a 40 px box at each commanded point: a landed pointer measured 1.4 to 9.8 against an idle floor of 0.02.
5. `none` XP `/api/config/imserver/set flip_right` sets the landscape handling mode if the measurement calls for it. It is in the no-restart list, and other connected clients receive `im_config_change`. Its actual effect on frame or coordinate orientation is described nowhere, so treat it as an experiment.
6. `click`, `swipe`, `mouse_move` XP `/api/mouse/click, /api/mouse/swipe, /api/mouse/move, /api/mouse/reset`: partition every mirrored id list by `rotate` and by `imgw` and `imgh` before sending input. One bad member sends the whole batch to the wrong place.
7. `send_key fn_key=WIN+h` XP `/api/key/sendkey?id=…&fn_key=WIN+h, twice` returns a phone to a known portrait state. Measured working on this rig even while the pointer was in the stuck-click state. Two presses are needed, the first to leave the app and the second to return to page one.

NEEDS

- Orientation lock on and StandBy off on every phone, plus the two-row Control Centre layout, before any calibration or auto-cast.
- A WebSocket client on `ws://127.0.0.1:9911/api` to receive `dev_rotate`. Polling `/device/get` is the fallback and adds latency.

- A measured, per-model coordinate set for landscape. There is no documented transform to derive one.
- Board bound and mouse parameters present, or `/mouse/*` returns code 2, 3 or 11 regardless of orientation.

WATCH OUT

- XP code 15 is "No coordinates", not "Must be in portrait orientation". The portrait error text exists only in the Phonefarm table and does not apply to this kernel.
- No document in either edition says `imgw` and `imgh` swap on rotation, and this rig has never run a phone in landscape. The XP `dev_rotate` example itself shows `rotate` 0 with the geometry unchanged. Treat the swap as an untested hypothesis, and treat any x-to-y transform as something to measure, not a recipe.
- Whether the shared-library calibration, `location_crc` B0DBD800, stays accurate in landscape is unstated in both docs and untested here.
- With OCR returning code 28 and the image-search plug-in presumed absent, there is no server-side way to re-find an element after an unexpected rotation. Recovery is Home-press normalization plus re-measurement, or client-side matching on raw BMP frames.
- `/pic/find-multi-color` tables are absolute pixel coordinates and are therefore orientation-specific too. A portrait table will not match a rotated frame.
- Because the API is unauthenticated, another LAN client can flip `flip_right` underneath you. Watch `im_config_change`.

SCALE

Rotation state is per phone, so a mirrored id list must be partitioned by `rotate` value and by mirror size. Detection costs nothing: `dev_rotate` is pushed to every connected client, and frame polling is per-phone independent at 83 and 85 req/s with two pollers, 512 req/s across ten, p50 3 ms. Re-measuring a landscape map costs three `/mouse/move` round trips plus three raw BMP grabs per model, with pointer travel dominating at 0.6 ms per pixel.

TRAP **An unnoticed rotation mirrors taps into the wrong place on every phone at once**

Every mouse and key endpoint takes a comma-separated id list, and one phone that rotated mid-batch relocates every coordinate in that list. Guard each batch with a `rotate` check and a `/mouse/reset` when the target set changes, and drop any phone whose `rotate`, `imgw` or `imgh` differs from the master. In a real app, and not a test screen, that misdirected tap can be destructive.

Licensing, boards and calibration

Edition gating is a Phonefarm concept; XP licenses per device

PARTIAL WITH CAVEATS

The Phonefarm doc marks the whole shortcut dispatcher and `ocr_ex` "professional version only". The XP docs gate no endpoint by edition at all. XP licenses through a vendor account instead, whose payload carries `pass`, `user_state` (1 is formal authorization, anything else is test), `total_license`, `local_total_license`, `dev_num`, `dev_online_num` and `overdue_time`, pushed unsolicited as `user_info` every 30 seconds. There is no documented mapping between the two schemes, so the correct stance here is to assume nothing about `/shortcut/*` or `/pic/ocr-ex` until a round trip has returned `data.code` 0.

HOW

1. `none` XP `/api/config/user/info` reads licence and account state. Not probed on this rig: the 10 September probe used `/user/get`, which 404s. The field table lists `dev_num` twice with conflicting glosses; read it as the device count and `dev_online_num` as the online count.
2. `none` XP `user_info and im_log WebSocket callbacks` keep watching without polling. `im_log` carries notices such as the vendor's own example, not logged in so going online is not allowed.
3. `shortcut#24` XP `/api/shortcut/device/ip?id=<one id>&state=0` proves the shortcut channel with the cheapest round trip. `state` 0 is LAN and 1 extranet; id is single only; `outtime` is milliseconds with a 15000 default, where Phonefarm's was seconds. Pass condition is `data.code` 0 and `data.text` equal to the `ip` field from `/device/get`. That one call proves the Shortcut is installed, the AssistiveTouch binding fires, and the phone can reach the kernel.
4. `shortcut#10` and `#11` XP `/api/shortcut/clipboard/set then /api/shortcut/clipboard/get` proves the write and read pair before relying on Unicode text entry.
5. `ocr_ex` XP `/api/pic/ocr-ex` is worth probing only after the plug-in exists. XP names it in prose under `/pic/ocr` with no parameter table of its own, and `/pic/ocr` answers `data.code` 28 here.
6. `click`, `swipe`, `send_key` XP `/api/mouse/click, /api/mouse/swipe, /api/key/sendkey` is the fallback where the shortcut layer is unavailable: slow, screen-dependent, and with every step verified by pixel diff on raw BMP frames.
7. `none` XP `none (IMConsole.exe, outside the API)`: measured, with IMService.exe up but the console not running, `/device/get` showed both phones state 0 and `/config/usb/get` returned an empty board list. The API cannot bring the rack up on its own.

NEEDS

- A vendor account logged in by the operator, through the console, with enough device authorizations for the phones brought online.
- iMouse Shortcut installed per phone with the kernel IP edited in, run once with every permission allowed, bound to an AssistiveTouch custom key, plus the Shortcuts advanced allowances for running scripts, sharing large data and deleting without confirmation.
- Board bound and connected, since the shortcut is launched by a HID hotkey, and the phone reachable on the LAN, since the shortcut answers the kernel over Wi-Fi.
- OCR plug-in installed and connected for `/pic/ocr` and `/pic/ocr-ex`, with `ocr_num` set and a kernel restart. iOS 15 or later for the file shortcuts and `/shortcut/switch/device`, Shortcut 1.3 or later for the `/shortcut/other/*` fallbacks.

WATCH OUT

- "Professional version only" exists solely in the Phonefarm document. Nothing maps it onto the XP licence model, so do not code against a professional flag on this kernel.
- No `/shortcut/*` endpoint has been proven on this rig, and the phone-side install state is unknown. A first call may legitimately time out with `data.code` 30 after 15 seconds without anything being broken in software, which is hard to tell from a network fault at a glance.
- The vendor's own example shows `total_license` 0 alongside `local_total_license` 100 and `user_state` 2. The relationship between the counters on trial versus formal accounts is undocumented, so do not compute capacity from one alone.
- XP status semantics differ from Phonefarm's entirely. Top-level `status` 200 means the request was accepted; the outcome is `data.code` against XP's own 0 to 41 table, where 28 is plug-in not connected, 29

plug-in call timeout, 30 call timeout, 36 parameter error, 37 login failed, 41 incompatible commands. Reusing Phonefarm's 1 to 32 codes mis-reads every result.

- Messages are Chinese on this build. Branch on `data.code`, never on message text.
- Exceeding authorization surfaces as an `im_log` notice, not a call failure, so an over-provisioned rack fails quietly at the point where phones try to come online.

SCALE

The licence count caps the rack independently of hardware: 19 populated boards are useless beyond the account's authorization, and `dev_online_num` against `total_license`, or `local_total_license` on a trial, is the live headroom reading, refreshed free every 30 s. Shortcut calls are the slow path at a 15 s default timeout, and the read-shaped ones (album get and down, file get and down, clipboard get, device ip) are single-id, so a rack-wide probe is N sequential round trips. The write-shaped ones (clipboard set, exec url, the switches, album upload, delete and clear, file upload and delete) take a comma id list and fan out in one request.

One board per phone, single-id capture, and a 19-slot rack

NATIVE API · WITH CAVEATS

Control is addressed one board per phone. Each board enumerates as VID 720A with a unique PID and a `uid` of PID plus a four-character suffix, and `/device/set` makes the binding with `vid` and `pid` set together, one device at a time. Input mirrors to N phones in one request, because every `/mouse/*` and `/key/*` endpoint takes a comma-separated id and the board walks the listed phones in parallel. Capture does not: every `/pic/*` endpoint is marked "Multiple is not allowed". This rack has 19 boards across three seven-port hubs, 21 slots, and two phones mirroring today.

HOW

1. `get_usb_list` XP `/api/config/usb/get` inventories the control channel, returning `data.list[{vid,pid,uid,ver,state}]`. `state` appears in the example but not the field table; its meaning comes from the `usb_change` code table plus measurement here: 1 is a board on the PC with no phone, 2 a board with a phone attached.
2. `get_device_list` XP `/api/device/get` inventories the video channel. `state` 0 is not online, non-zero is online, and `air_pid` is the AirplayMirroring process id and the live proof of mirroring. Control-ready and video-ready are independent readings and must be reported separately.
3. `set_dev` XP `/api/device/set?id=…&vid=720A&pid=…` binds a phone to its board. The response is the list of ids successfully set, not the full record, so re-read `/device/get` to confirm. To learn which uid a slot is, replug the phone cable and watch `usb_change` code 12.
4. `set_dev` with a new pid XP `/api/device/set with the new vid and pid` moves a phone to a different board. No unbind endpoint is documented in either edition. `/device/del` removes records entirely, refuses online devices with code 7, and broadcasts `dev_delete`.
5. `restart_usb` XP `/api/device/usb/restart?id=720A|C241` resets a wedged board without touching the phone or the kernel. XP addresses boards, not phones, so an unbound board can be reset too. Expect `usb_change` 20, then 19, then 12 when the phone re-attaches.
6. `callback usb_change` XP `usb_change` tracks hardware changes without polling, with the event type in `data.code`.
7. `set_group`, `set_dev name` XP `/api/device/group/set`, `/api/device/set name+gid`, `/api/device/group/get-dev` labels the rack. PIDs are opaque, so hub and port is the stable physical label, and it comes from Windows PnP rather than the API.

NEEDS

- Connection order enforced by the hardware: board to PC (state 1), then phone to board with the vendor cable (state 2), then mirroring, then bind the uid.
- IMConsole.exe running and logged in, or the board list comes back empty and every phone reads state 0.
- A device authorization per phone. Board count is not the only ceiling.
- Hub and port topology recorded out of band, so an unresponsive board can be found physically.

WATCH OUT

- One board per phone is a vendor product-page statement, not an API guarantee. The API simply offers no way to share a board.
- Neither doc states a maximum device count for the kernel. Nineteen is this rack's physical fact, not an API limit.
- Cable substitution leaves a board at state 1 with a phone visibly plugged in, a silent control-channel failure no API field explains. Surface the vendor-cable requirement in diagnostics.
- A wrong pid in `/device/set` binds a phone to another slot, and every tap then lands on a different handset with no error. Never route or display by `ver` or `uver`, which are mojibake on this build.
- Phonefarm's per-device auto-cast switch has no XP equivalent; XP's `auto_connect` is a single global flag, so "this phone re-casts itself" is not expressible per slot. Phonefarm's three airplay presets and their bandwidth guidance do not exist in XP either, so any per-phone bandwidth budget must be measured, not quoted.
- `/device/airplay/set` disconnects the current cast immediately and applies on the next cast. Never call it mid-run on a working phone.
- There is no supported path to drive the boards directly: the nine-byte vendor HID report layout on usage page 0xFF00 is undocumented, so IMService stays a hard dependency.

SCALE

Control scales linearly and cheaply: a comma id list on click, move, swipe, reset, sendkey, up, upall, airplay connect and disconnect, collection and usb restart drives N phones in one request, and a move of a given distance costs the same wall clock for one phone as for two. Video does not scale that way: one AirPlay session per phone, one screenshot request per phone, measured 83 and 85 req/s with two pollers and 512 req/s across ten, raw BMP at 2 ms and 1.62 MB against JPEG at 40 ms and about 21 req/s. Source frame rate is the real limit and is per phone: 28.9 new frames per second on 192.168.4.68 against 4.5 on its identical twin at .27, a Wi-Fi and AirPlay property the vendor console hits too.

Device floor and calibration coupling: iOS 15, and a parameter record per model and iOS version

PARTIAL WITH CAVEATS

Supported hardware is iPhone 6s and later. The vendor settings page says iOS 13.4 and above, the product page says 15 and above, and the XP doc independently requires 15 or later for the file shortcuts and `/shortcut/switch/device`, so design to 15. Pointer accuracy depends on a mouse-parameter record: `/device/get` exposes `location` and `location_crc`, and with neither present every mouse call fails with code 11. Both XRs here carry an empty `location` and `location_crc` B0DBD800, a shared library profile byte-identical to the vendor doc's own example.

HOW

1. `get_devicemodel_list` XP `/api/config/devicemodel/get` checks the model and parameter library. The endpoint exists and returns `data.list`, but the XP doc elides the payload entirely, so the record shape is

undocumented on this edition. Probe it before depending on it. `/device/model/get` is not the path and 404s.

2. `get_device_list` XP `/api/device/get?id=…` checks each phone actually has parameters. Live: B0DBD800 on both XRs, `location` empty, which is a library match rather than a per-device measurement.
3. `mouse_collection_open` plus `mouse_collection_cfg` XP `/api/device/collection/mouse?id=…&cmd=1` starts a collection when no record matches. `cmd` 0 queries, 1 starts, 2 stops, one endpoint replacing Phonefarm's three. Progress arrives on the `collection_mouse` callback: 0 success, 4 failed, 5 a device is collecting, 8 scan code opened, 9 entered the collection page, 14 start working, 21 at work.
4. Collection page opened full-screen from a Home Screen icon XP `none (undocumented)` is the phone-side step. XP gives no phone-side collection URL; the console presents a QR code, and codes 8 then 9 are the only signals. The console route is right-click the mirrored phone, Set Mouse Parameters, profile 1 for iOS 14.2 and above.
5. `save_dev_location` XP `/api/device/collection/mouse/save?id=…&describe=XR-18.7.10` publishes to the shared library, single id plus free text, then stop with `cmd=2`.
6. `set_dev location_crc` XP `/api/device/set?id=a,b,c&location_crc=…` copies the calibration to same-model phones. Here `id` does take a comma list; only `vid` plus `pid` binding is single-device.
7. `mouse_reset_pos`, `mouse_move`, `get_device_screenshot` XP `/api/mouse/reset, then moves to (20,20), (480,1060), (250,540), each followed by a binary screenshot` probes accuracy after any iOS update or settings change. Find the pointer by mean-abs diff in a 40 px box: measured 1.4 to 9.8 against an idle floor of 0.02, with a commanded (380,880) landing at (377,875). Error growing toward the far corner means the per-pixel scale is wrong.

NEEDS

- Automatic iOS updates disabled on every phone. Phone-side, with no API to read or set it.
- Tracking Speed and AssistiveTouch tracking sensitivity at the same settings the library record was collected with. Both invisible to the API.
- Board bound and connected, or codes 2, 3 and 13 appear, with the phone online and in portrait.
- Only one device collecting at a time, and the vendor console available for the QR step.

WATCH OUT

- The XP doc never says library records are keyed by model plus iOS version; that is Phonefarm's statement and matches observed behavior. Treat it as behavioral, not contractual. The corollary, that an iOS update silently loses the match, has not been observed here: these XRs are on 18.7.10 and still match B0DBD800.
- XP documents no way to delete a record from the library, so a bad `describe` or a bad CRC is permanent on this edition.
- The collection endpoint's parameter table advertises a comma id list while code 5 forbids concurrent collection. Follow the code, not the table.
- Never invent or guess a `location_crc`. Only a collected or library value belongs there, and a wrong one mis-positions every tap with no error.
- Calibration accuracy cannot be confirmed by OCR or template matching on this rig, so verification is pointer-box and whole-frame pixel diffs on raw BMP frames. Never MD5-compare frames; an untouched mirrored screen never repeats byte for byte.
- Send `/mouse/reset` before any accuracy probe. The stuck-click state otherwise makes a correctly calibrated phone look mis-calibrated.
- A heterogeneous rack still needs mirrored coordinate lists split by `imgw` and `imgh` regardless of calibration, and the two XRs here have different home screens, so a shared coordinate is only fair after two `WIN+h`

presses normalize the state.

- Because the API is unauthenticated, another LAN client can overwrite `location_crc` on any device. Watch `dev_change`.

SCALE

One successful collection per model plus iOS version serves the whole rack: the resulting CRC is distributed to every same-model phone in a single `/device/set` with a comma id list, and same-model phones auto-match from the library on connect. Collection itself is serialized and drives real UI, so a mixed rack costs one manual console session per model and iOS combination. The accuracy probe is cheap and fully parallel: three moves plus three 2 ms raw BMP grabs per phone, with phones polling independently at 83 to 85 req/s each.

Vision on this rig

Vision plug-ins absent: OCR and image search answer code 28

NEEDS PLUG-IN WITH CAVEATS

On this rig `/api/pic/ocr` answers HTTP 200 with `data.code` 28, plug-in not connected. The image-search pair, `/pic/find-image` and `/pic/find-image-cv`, is backed by the same plug-in architecture, since the kernel config exposes `opencv_num` and `ocr_num` as instance counts, so assume both are absent until probed. `/pic/find-text` and `/pic/find-text-ex` sit on the OCR plug-in and go with it. Every "locate the element, then tap its centre" design therefore moves client-side, or waits on the vendor plug-in.

HOW

1. `ocr` XP `/api/pic/ocr` baselines the gate. Verified live on 10 September 2026: status 200, `data.code` 28. The XP code table reads 28 plug-in not connected, 29 plug-in call timed out, 31 OCR did not recognize the text, 32 no text found. The success shape is `data.list[{text, centre, rect, similarity}]`.
2. `find_image` XP `/api/pic/find-image (DM-compatible) and /api/pic/find-image-cv (OpenCV)` probes the image-search gate, which has not yet been done here. Both take a single id and `img_list` as base64 strings or local PC paths, returning `data.list[{index, centre, rect, code}]`.
3. No Phonefarm equivalent XP `/api/pic/find-text and /api/pic/find-text-ex`: an XP-only convenience that OCRs then matches strings. Same plug-in, therefore unavailable here.
4. `get_device_screenshot` XP `/api/pic/screenshot?id=…&binary=true` is substitute 1, client-side vision on raw frames. Measured p50 2.0 ms, 84 req/s flat out, 1,620,054 bytes of bottom-up BMP, which is 500 x 1080 x 3 plus a 54-byte header. Decode, then run an OCR or template stack on the PC at roughly 50 to 300 ms per full frame, an order of magnitude less on a cropped rect. Coordinates come out already in the space `/mouse/click` wants.
5. `find_multi_color` XP `/api/pic/find-multi-color` is substitute 2, colour-table matching, gate unknown. XP's table is a comma-separated list of absolute points, `x|y|RRGGBB-tolerance`, not Phonefarm's anchor-plus-offset form. The doc offers `/shortcut/other/find-multi-color` as a fallback in exactly the phrasing it uses for the plug-in functions, so do not assume it is kernel-internal.
6. `shortcut` dispatcher XP `/api/shortcut/other/screenshot, find-image, find-image-cv, ocr, ocr-ex, find-text, find-text-ex, find-multi-color` is substitute 3, on-phone recognition. Named once each in the picture-recognition page, with no parameter tables at all, and requiring the full Shortcut prerequisite chain. It trades a plug-in dependency for a phone-side dependency, not for no dependency.
7. `none` XP `/api/config/imserver/set opencv_num, ocr_num, then /api/imserver/restart` applies if the plug-ins are bought. Neither field is in the no-restart list, so a kernel restart is required, which drops every mirroring session. `ocr_num` is annotated "Only the GPU version is effective".

8. `restart_device` XP `/api/device/restart, or /api/shortcut/switch/device state=1, or /api/key/sendkey fn_key=CTRL+ALT+SHIFT+WIN+r` shows that a driven restart has two OCR-free routes. The hotkey needs no vision at all, only Full Keyboard Access and a bound board, and is the route to prefer here. The shortcut route also covers shutdown, which Phonefarm has no equivalent for.

NEEDS

- To restore server-side vision: the vendor OCR and OpenCV plug-in packages for the XP edition, installed so the kernel stops returning code 28, and a GPU build if `ocr_num` above 1 is to mean anything.
- To use the client-side substitute: a PC-side OCR and template stack consuming raw bottom-up BMP frames, with one reusable buffer per poller. Each 1.6 MB frame exceeds the .NET Large Object Heap threshold, and without reuse the measured cost was 488 MB/s of allocation and 496 gen-2 collections in 8 s.
- To use the `/shortcut/other/*` fallbacks: iMouse Shortcut 1.3 or later installed per phone, kernel IP edited in, run once with permissions granted, bound to an AssistiveTouch key, plus a working board.
- Templates and colour tables built from the same phone at the same `air_play_ratio`. They are specific to the 500 x 1080 stream.

WATCH OUT

- OCR is confirmed unavailable; image search is presumed unavailable but has not been probed. Say unknown, not absent, until `/pic/find-image-cv` has actually been called.
- Every recognition endpoint is single-id, so recognition never fans out the way `/mouse/click` does.
- XP `rect` is a flat [left x, top y, right x, bottom y] array everywhere. Phonefarm's four-corner-point form will not parse here.
- Changing `air_play_ratio` globally, or `air_ratio` per device, invalidates every template and colour table at once, and the per-device change disconnects the cast and applies only on the next connect.
- Designing a flow around `/pic/ocr` here produces code 28 at runtime, not at build time, because the endpoint answers HTTP 200 and looks healthy. Guessed paths return 404 and Phonefarm-style bare function names return HTTP 500, so a client that checks only HTTP status will misreport all three failure modes.
- Exact-colour matching with zero tolerance produces intermittent false negatives because of AirPlay encoder noise. Match on `data.code`, never on the Chinese `data.message` strings.

SCALE

Client-side vision is bounded by PC cores, not by the kernel. Frame fetch is 2 ms and phones are independent, two pollers at 83 plus 85 req/s and ten at 512 req/s in total, but 19 phones at four full-frame OCR passes per second at about 100 ms each is roughly eight cores of work. Cropping to a rect before recognizing is the single biggest lever, with small-region template matching about ten times cheaper than a full frame. Server-side plug-ins would not parallelize better, since their endpoints are single-id and concurrency is a config knob rather than a per-request property. A still screen produces almost no new frames, so discard near-identical polls with a mean-pixel-difference test and never with MD5.

TIP **Restarting a phone does not need vision**

Phonefarm's `restart_device` depends on `save_restart_point`, which needs the OCR and image-search plug-ins. XP has two routes that need neither: `fn_key=CTRL+ALT+SHIFT+WIN+r` on `/key/sendkey`, which needs only Full Keyboard Access and a bound board, and `/shortcut/switch/device` with `state=1` on iOS 15 or later, which also covers shutdown at `state=0`. Both leave the phone at the passcode screen with mirroring dropped, so pair either with a check for `state` 0 and a human queue.

The phone itself is a limit: locks, prompts and reboots

The rack drives a mirror and a HID board. It does not drive iOS. Anything iOS puts in front of the app, a lock screen, an Apple ID prompt, a software update, is a wall the API has no call to pass.

Phones must stay unlocked: nothing in the API unlocks a screen or answers a system prompt

PARTIAL WITH CAVEATS

Across the whole XP surface, `/device/*`, `/mouse/*`, `/key/*`, `/pic/*`, `/shortcut/*`, `/config/*` and `/imserver/*`, there is no unlock function and no channel that reports "locked" as distinct from "offline". After a lock, a reboot, a battery event or iOS re-authenticating on its own, the phone sits on a screen the rack cannot pass, and a passcode, Face ID, Apple ID, 2FA, software update, Screen Time or "Trust This Computer" prompt stops it just as hard. What you do get is prevention, which is phone-side, detection, and recovery of the mirror once a human has unlocked.

HOW

1. `(none)` XP `none` — prevention is phone-side and invisible to the API: Auto-Lock Never, StandBy off, portrait lock on, applied at commissioning and re-checked after every iOS update
2. `send_key` XP `/key/sendkey fn_key=TAB+l` — the documented lock hotkey. Never script it. There is no unlock counterpart anywhere in the hotkey table
3. `get_device_list` XP `/device/get` — state 0 means not online, non-zero means online; `air_pid` is the live AirplayMirroring.exe process id, 22236 for both XRs here
4. `callback dev_disconnect` XP `ws://127.0.0.1:9911/api` — the mirror dropping arrives as a broadcast with the full record at state 0, instead of a polling loop
5. `get_device_screenshot + ocr` XP `/pic/screenshot?id=<id>&binary=true` — lock-screen detection has to be client-side matching here, because `/pic/find-text` and `/pic/ocr` return data.code 28
6. `auto_connect_screen` XP `/device/airplay/connect` — re-cast after a human unlock; `/device/airplay/connect/all` takes no parameters, and the global `auto_connect` flag applies without a kernel restart
7. `restart_device` XP `/device/restart` — expect a lock screen after every reboot, roughly 60 to 90 seconds, tracked on the `restart_log` callback rather than by screenshot, because there is no mirror during the reboot

NEEDS

- Dedicated phones with Auto-Lock Never, StandBy off and portrait lock, none of which is queryable over 9911
- A stored per-phone lock-screen reference at the current `air_play_ratio`. The two XRs here have different wallpapers, so one reference cannot serve both
- A WebSocket client for `dev_disconnect` and `airplay_connect_log`
- A named human escalation path per physical slot, because an unlock cannot be automated

WATCH OUT

- state 0 conflates locked, Wi-Fi dropped, AirPlay session died and phone off. The board channel (`/config/usb/get` state 2) is the only independent second signal, and it says nothing about the screen
- A retry loop against a locked phone never succeeds and buries the real cause in `airplay_connect_log` noise. `connect_failed_retry=0` means infinite retries and makes this worse
- `auto_connect` is a global kernel flag on XP. There is no per-device cast-on-connect flag as in the Phonefarm edition

- Hardware-keyboard passcode entry at the lock screen is undocumented and untested. Do not build on it. Repeated wrong entries hit iOS lockout timers that no API call can clear, and entering a passcode on a device you are not authorized to access is a computer-misuse offence (CFAA in the US, the Computer Misuse Act in the UK) however the keystrokes are delivered
- Scripting TAB+l against a comma-separated id list can lock the entire rack in one request with no scripted way back

SCALE

Detection parallelizes perfectly: `/device/get` returns the fleet in one call, `dev_disconnect` is broadcast, and pollers are independent (two at 83 and 85 req/s, ten at 512 req/s aggregate). Recovery partly parallelizes. The unlock is the one rack operation that does not parallelize at all: an iOS update or a power blip locks all 19 phones at once and each needs an individual human touch, so rack-wide recovery time is linear in phone count.

Exposure you cannot configure away

What the rack inherently exposes: shared egress, simultaneous input, pointer-type taps, hardware-keyboard text

PARTIAL WITH CAVEATS

Two properties of this stack are not configurable. Network identity: each phone reaches the internet through whatever egress its Wi-Fi gives it, and the only API readout is `/shortcut/device/ip`, useful for geo and ad verification and for confirming a test device egresses where you expect. Input identity: `/mouse/*` and `/key/*` deliver HID mouse and hardware-keyboard events through the board, not synthesized finger touches, `/key/sendkey` is ASCII-only, and AssistiveTouch, which the rack requires, is a runtime accessibility state. A third property is timing: every input endpoint takes a comma-separated id list, so a mirrored action lands on N phones inside one request.

HOW

1. `shortcut#24` XP `/shortcut/device/ip` — state 0 gives the LAN address, state 1 the extranet address; single id, default outtime 15000 ms. XP adds the LAN readout Phonefarm lacked, and comparing state 0 against the `ip` field from `/device/get` is the cheapest end-to-end proof the Shortcut chain works
2. `shortcut#18–#23` XP `/shortcut/switch/wlan, /flight, /cdpd` — a factual constraint, not a technique: turning Wi-Fi or flight mode on kills the AirPlay video channel while USB control survives. The vendor's static-IP-without-gateway setup exists so a phone stays on the LAN for mirroring while cellular carries its internet traffic
3. `click / swipe / mouse_move / send_key` XP `/mouse/*, /key/* with "Separate multiple devices with commas"` — the board walks all listed phones in parallel and a move to N phones costs the same as to one, about 0.6 ms per pixel either way
4. `click` XP `/mouse/click` — button is numeric on XP ("1" left, "2" right, up to "8"), x and y are mirror pixels, `time` is press-to-release in milliseconds
5. `send_key` XP `/key/sendkey key=<ASCII>` — both XP key pages say "Only English, numbers and English characters are supported". For anything else, `/shortcut/clipboard/set` then `fn_key=WIN+v` is the documented Unicode path
6. `swipe` XP `/mouse/swipe` — for a repeatable stroke supply explicit sx/sy/ex/ey, or build it from `/mouse/down`, `/mouse/move`, `/mouse/up`. The Phonefarm doc says percentage mode varies start and end coordinates; the

XP doc documents no such variation

7. `(none)` XP `none` — there is no rate limiter in either edition. Backing off on a platform rate-limit response is entirely the client's job

NEEDS

- The iMouse Shortcut installed, permissioned and bound to an AssistiveTouch key for every `/shortcut/*` call, including the IP readout
- Full Keyboard Access on and AssistiveTouch on with tracking sensitivity at maximum, or neither hotkeys nor coordinates work
- Written authorization for every account and service the rack touches
- A client-side scheduler that can stop a phone's queue, because the API provides none

WATCH OUT

- `/shortcut/device/ip` and `/shortcut/clipboard/get` are single-id only; the switch and clipboard-set endpoints take comma lists
- Toggling wlan or flight drops the video channel and the Shortcut's own LAN return path, so the call may time out with code 30 and the phone will need re-casting
- Nothing in this API exposes a knob for what iOS reports about the device, the input type or the accessibility state
- Automation, multi-accounting and device farms breach the terms of service of every major platform (Meta, TikTok, X, Google, Apple). Enforcement is account termination and, for commercial operators, civil action. That is a property of the use, and the mitigation is authorization and rate discipline, not concealment
- A rig that delivers pointer-type touches and hardware-keyboard text cannot faithfully reproduce a real finger or the on-screen keyboard, so any test depending on touch semantics or IME behavior needs a human
- The pointer-type and accessibility-flag observations are iOS behavior and appear in none of the reference documents. Treat them as context, not as verified API facts

SCALE

The comma-separated id list is the throughput story: one request drives every listed board in parallel and measured input latency is unchanged between one phone and two. Input traffic and frame polling do not interfere, 98.8 req/s of screenshots with input streaming versus 78.7 without. The ceilings are elsewhere: the 0.6 ms/px pointer walk (a 1000 px move is 592 ms), the per-phone AirPlay source rate, and the 15 s default outtime that makes Shortcut-mediated operations roughly two orders of magnitude slower than a mouse call.

Legal and platform exposure is a property of the use: no audit log, no consent store, no rate limiter

PARTIAL WITH CAVEATS

The kernel sends a mirrored tap to every id in the list without regard for what the tap does. The exposure lives in the use: unsolicited commercial SMS or iMessage (TCPA, PECR), commercial email or DM without opt-out (CAN-SPAM), processing other people's personal data harvested from screens or clipboards (GDPR, CCPA), access without authorization (CFAA, Computer Misuse Act), and breach of platform terms, which is civil rather than criminal but ends accounts. What the API contributes is metadata only: groups, custom names, per-phone identity and frame evidence on disk.

HOW

1. `set_group` XP `/device/group/set?id=0&name=ClientA` — id "0" creates a group, any other id renames it; broadcast to every client as `group_change`. On this rig the group list is empty and both XRs sit in the default

ungrouped bucket

2. `set_dev` XP `/device/set?id=<ids>&gid=<gid>&name=A-slot-01` — comma list for name and gid; returns only the ids that succeeded. Names and groups persist in DeviceList.json, the only operational metadata the kernel keeps
3. `get_device_list` XP `/device/get` — deviceid, mac, ip, user_name, version, model, state, air_pid, imgw, imgh, gid, vid, pid, uid. It does not include the board's hub and port, which comes from Windows PnP
4. `shortcut#24` XP `/shortcut/device/ip?id=<id>&state=1` — record the egress actually used, and re-take it after any radio change rather than caching it
5. `get_device_screenshot` XP `/pic/screenshot?id=<id>&save_path=D:\evidence\` — XP-only: the kernel writes the file and returns its path in data.image. Default BMP is about 1.6 MB per frame, `jpg=true` is about 96 KB at a 40 ms encode
6. `(none)` XP `none` — cap actions per phone per hour in your own scheduler before calling `/mouse/click`, `/key/sendkey` or `/shortcut/exec/url`
7. `(none)` XP `/config/imserver/set allow_ip_list` — see the trap below. The only real access control is a Windows Firewall rule on TCP 9911 and 17000
8. `(none)` XP `ws://127.0.0.1:9911/api` — dev_change, dev_delete, group_change, group_delete, im_config_change, dev_sort_change, error_push, im_log, user_info (every 30 s). Live broadcasts, not a log

NEEDS

- Written authorization for every account, device and system the rack touches. The tool cannot distinguish an account you own from one you do not
- A consent and opt-out record for any outbound messaging, maintained outside the kernel
- A lawful basis and a retention policy for stored screen captures and clipboard content, both of which routinely contain other people's personal data
- A firewall rule limiting 9911 and 17000 to the controller host, and a client that holds the WebSocket open if any audit trail is required

WATCH OUT

- There is no field for purpose, client, consent, operator or authorization. Everything substantive lives in your own store
- `/device/group/del` returns code 34 ("Please remove the device in the group first") and `/device/del` returns code 7 for an online device, so the metadata layer resists tidy-up while the rack is running
- `C:\iMouseXP\IMService.json` holds the vendor account password as reversible hex obfuscation. Any tool that reads it holds a live credential and must never print or commit it
- The callbacks identify no caller and persist nothing, so there is no server-side record of who drove the rack or on whose authority

SCALE

Compliance work scales with phones, accounts and messages, not with API calls. Labelling 19 phones is two calls. Evidence capture does not fan out: `/pic/screenshot` is single-id, cheap in kernel terms but about 30 MB per rack-wide BMP snapshot. `/shortcut/device/ip` is the slow link at a 15 s default timeout per phone.

TRAP **allow_ip_list is not access control**

It is documented under "Screencast Discovery Rules" and only takes effect when the rule is 1. It gates which phones may discover the AirPlay receiver, not who may call the HTTP or WebSocket API. Verified live: the API on 9911 accepts unauthenticated calls from any LAN host, and the vendor's own examples call it from a remote 192.168.9.9.

Physical limits: radio, heat and battery

Wi-Fi is the AirPlay bottleneck: per-phone frame rates differ sixfold

PARTIAL WITH CAVEATS

Video is one-way, iPhone to Wi-Fi to AirplayMirroring.exe on TCP 17000, one session per phone. Nothing in the XP API touches the radio. Measured here: two identical iPhone XRs on the same iOS build and the same AP deliver 28.9 and 4.5 new frames per second under continuous driven motion, alone or together, so the slow one is its own link rather than kernel contention. The only documented levers are per-device `/device/airplay/set`, the global `air_play_*` block, and network design outside the API.

HOW

1. `get_device_list` XP `/api/device/get` — state, air_pid, imgw/imgh, srv_name, ip and the per-device air_* overrides (0 means use global). Verified: two iPhone11,8 on iOS 18.7.10, state 1, air_pid 22236, imgw 500 by imgh 1080, logical size 414 by 896
2. `(none)` XP `/api/config/imserver/get` — read the global casting configuration before changing it. Documented but not probed here. The guessed path `/config/get` returns 404
3. `swipe + get_device_screenshot` XP `/api/mouse/swipe driving motion, /api/pic/screenshot?binary=true polled per phone` — measure the real source rate by counting frames whose mean absolute pixel difference from the previous frame exceeds about 1
4. `set_airplay_mode` XP `/api/device/airplay/set?air_ratio=720&air_fps=15&air_audio=1` — air_ratio is one of 0/720/1080/1400/1920/2650, air_refresh 0 to 60, air_fps and air_img_fps 0 to 30, air_audio 0 global / 1 off / 2 on. The change disconnects the current cast and applies on the next one
5. `(none)` XP `/api/config/imserver/set air_play_ratio=720&air_play_fps=15 then /api/imserver/restart` — the global `air_play_audio` is Boolean, while the per-device `air_audio` is Integer 0/1/2. Every air_play_* field needs the restart
6. `(none)` XP `/api/imserver/regmdns` — XP-only: re-advertise the receiver (mDNSResponder.exe, "iMouseXP-5429" on UDP 5353) instead of restarting the kernel again
7. `auto_connect_screen` XP `/api/device/airplay/connect` — confirm recovery with state non-zero, a live air_pid, an ESTABLISHED socket on 17000 from that phone's IP, and a frame that changes under motion. XP has no force flag, so disconnect first

NEEDS

- Phones and PC on the same LAN segment with mDNS (UDP 5353) allowed, and the receiver name matching `air_play_name`
- IMConsole.exe running and logged in. With only IMService.exe up, `/device/get` showed both phones at state 0 and `/config/usb/get` returned an empty board list
- A client that decodes the 1,620,054-byte bottom-up BMP and computes mean absolute pixel difference, because there is no server-side frame comparison

WATCH OUT

- The Phonefarm three-mode bandwidth table (about 5, 10 and 20 Mbps, "about 100 or 50 phones") does not exist on XP, is described there as being for one landscape game with no stated measurement method, and the derived "19 phones is about 190 Mbps" is an extrapolation, not a measurement
- Only two phones have ever been measured here. Whether 19 concurrent sessions starve each other on one AP is untested: plausible physics, not an observed result
- That lowering `air_ratio` changes imgw/imgh, and so invalidates stored coordinates and templates, is a reasonable inference from the parameter descriptions but is neither documented nor measured. Re-read imgw/imgh from `/device/get` after any ratio change
- `/pic/screenshot` accepts one id only, unlike every `/mouse/*` and `/key/*` endpoint, so frame measurement is N concurrent requests
- A still screen produces almost no new frames, so any fps measurement without driven motion measures nothing. Do not derive the poll interval from observed frame gaps: doing so decayed a 23 fps stream to 16.5 fps through positive feedback. Use a fixed period
- XP has no multi-receiver concept. Phonefarm's `get_airplaysrvnum` and `set_airplaysrvnum` have no XP equivalent, so spreading load across several named receivers on one PC is not available
- Losing Wi-Fi kills only video. A phone stays fully controllable over USB while blind, so a health model must report control-ready and video-ready separately or a radio problem will be misdiagnosed as a dead board

SCALE

The kernel is demonstrably not the constraint: raw BMP p50 2.0 ms when paced at 50 ms, 84 req/s flat out at p50 12.7 ms; two phones concurrently gave 83 and 85 req/s at p50 3 ms each; ten pollers gave 512 req/s total. JPEG costs p50 40.5 ms and caps at about 21 req/s because the kernel re-encodes, so choose it only where 1.6 MB per frame is unaffordable, never on loopback. The real ceiling is per-phone production, 28.9 fps on 192.168.4.68 and 4.5 fps on 192.168.4.27. Budget 1.6 MB per poll with a reused buffer: without reuse, 488 MB/s of allocation and 496 gen-2 collections per 8 s; with reuse, 2.9 MB/s and zero to one.

FAST XR SOURCE RATE	SLOW XR SOURCE RATE	NEW FRAME INTERVAL	RAW FRAME
28.9 fps	**4.5 fps**	**220 ms**	**1,620,054 B**
192.168.4.68, under driven motion	192.168.4.27, same AP and iOS build	slow phone; verification timeouts must allow for it	500 x 1080 bottom-up BMP, p50 2 ms

No thermal or battery telemetry exists: heat management is open-loop

PARTIAL WITH CAVEATS

Rack phones run with the screen permanently on, the board charging them continuously and the SoC encoding an H.264 stream around the clock. Nothing in the XP API reports temperature, battery level, charge state or thermal throttling. The `/device/get` record carries air_handle, air_pid, rotate, state, imgw/imgh, gid, the air_* overrides, name, srv_name, width/height, ip, mac, user_name, version, model, deviceid, device_name, location, location_crc, vid, pid, uid, gname, uver, and nothing else. No `/shortcut/*` path reports battery either.

HOW

1. `shortcut#15` XP `/api/shortcut/switch/bril?id=<comma list>&state=0.05&outtime=15000` — comma list confirmed; state is a float documented as greater than 0 and less than 1; outtime is milliseconds on XP and seconds on Phonefarm

2. `set_airplay_mode` XP `/api/device/airplay/set?air_ratio=720&air_fps=10&air_audio=1` — cut encode and radio load per phone. The Phonefarm energy-saving preset does not exist on XP; these explicit knobs replace it
3. `discon_airplay / auto_connect_screen` XP `/api/device/airplay/disconnect then /api/device/airplay/connect` — a rest phase stops the encoder while USB control stays alive. Both take comma id lists
4. `find_multi_color / get_device_screenshot` XP `/api/pic/find-multi-color, or safer here /api/pic/screenshot?binary=true analysed client-side` — detect the iOS thermal warning screen. XP encodes absolute anchors as "x|y|RRGGBB-tolerance,..." inside a list of configs, not Phonefarm's first_color plus offset form
5. `get_device_list` XP `/api/device/get, then rebuild the comma id list` — there is no API-level exclude flag; the target set is whatever list your client sends

NEEDS

- The Shortcut chain for `/shortcut/switch/bril`, which is not verified on this rig. Without it the only brightness lever is manual, per phone, and the API cannot read brightness back
- A captured reference of the actual iOS thermal warning screen on this model, which is itself a hardware-stress exercise to produce
- Acceptance that rest phases blind the phone: no screenshots, no find-image, no OCR while disconnected

WATCH OUT

- A thermally throttled phone answers every call with data.code 0 while the iOS temperature screen blocks all UI, so mirrored taps land on nothing. It looks identical to the silent stuck-click state: use the pixel oracle, not return codes
- Airplane mode as a cooling measure is a trap. `/shortcut/switch/flight?state=1` kills Wi-Fi, which kills both the video channel and the Shortcut's own return path, and the reverse call has to travel that same dead path. The same applies to `/shortcut/switch/wlan?state=0`. Never wire either to an automated thermal response
- The claim that iOS pauses charging above about 40 C, visible on the lock screen, is outside every reference file and is not checkable here. With Auto-Lock set to Never there is no lock screen in the mirror to read, so it is not an observation channel
- A swelling battery lifts the display and can stress or short the board's cable. There is no software signal at all; the first symptom is physical, so inspect on a schedule
- The vendor setup page warns that inserting the control board in the wrong orientation can cause a short-circuit burnout at your own risk, and that no cable end or metal part may touch the yellow PCBA area

SCALE

Brightness, cast settings and disconnect/connect all take comma id lists, so a 19-phone adjustment is one request each. Detection is the only per-phone cost: a once-a-minute thermal check across 19 phones is roughly 30 MB/min of loopback traffic and a few tens of milliseconds of kernel time. Rested phones consume no AirPlay bandwidth, which also relieves shared-AP pressure. Shortcut calls round-trip over each phone's Wi-Fi, so a rack-wide brightness change is bounded by the slowest link.

The limits of what a response tells you

A code 0 response only means the board accepted the report: the frame is the only oracle

NATIVE API VERIFIED

Status 200 means the request was accepted and data.code 0 means the kernel executed it. Neither says iOS acted. Measured on this rack on 10 September 2026: a phone entered a state where `/mouse/move` visibly moved the pointer and `fn_key=Home` still worked (whole-frame mean difference 142.15), while every `/mouse/click` returned code 0 and produced 0.09 against an idle floor of 0.02 to 0.03. `/mouse/up` did not clear it. `/mouse/reset` did, and it stayed cleared: the same click then measured 141.96. The same false-success shape covers a mis-calibrated pointer, a banner over the target, a locked phone and the thermal screen.

HOW

1. `mouse_reset_pos` XP `/api/mouse/reset?id=<comma list>` — the only call measured to clear the silent stuck-click state, and the clearing persists. Send it before any tap-critical sequence and whenever the target set changes
2. `get_device_screenshot` XP `/api/pic/screenshot?id=<single id>&binary=true` — capture the reference frame. Raw bytes, no JSON wrapper; unlike the Phonefarm sibling, binary works over plain HTTP here
3. `mouse_move` XP `/api/mouse/move?x=380&y=880 then a re-capture` — prove the control channel by diffing a small box around the commanded point. A move to 380,880 put the pointer at 377,875 in the captured frame
4. `click` XP `/api/mouse/click?x=<x>&y=<y>[&button=1][&time=<ms>]` — require a whole-frame mean difference above about 20. A real reaction measures 20 to 160 (162.66 and 141.96 observed); under 1 is nothing happened
5. `(none)` XP `ws://127.0.0.1:9911/api callback error_push` — data.call_fun and data.message, the vendor example being "/mouse/down" with "Hardware not working (unknown error)". The only asynchronous signal that an input call failed at the board, and it arrives after the synchronous code 0
6. `send_key fn_key WIN+h` XP `/api/key/sendkey?fn_key=WIN+h (twice)` — normalize the screen before a shared-coordinate probe. The first press leaves the app, the second returns to page one

NEEDS

- A per-phone idle control frame and threshold, captured while nothing is driving that phone
- imgw/imgh per device from `/device/get`. Coordinates are mirror pixels, 500 by 1080 here, not iOS logical points at 414 by 896
- A WebSocket client for `error_push`, `dev_disconnect` and `usb_change`. HTTP alone cannot see them
- Client-side BMP decoding and mean-absolute-difference arithmetic. The API offers no frame comparison of any kind

WATCH OUT

- Never MD5-compare frames. The clock, encoder noise and wallpaper parallax change every frame, so byte equality reports "reacted" for an untouched phone. An earlier diagnostic in this project drew a wrong conclusion exactly that way
- A probe tap on a shared coordinate is only fair if the phones have the same element there. The two XRs here have different home screens, so the same coordinate is Clock on one and Reminders on the other. Prefer a move-only probe with a pointer-box diff, which is state-independent

- A phone at 4.5 new frames per second needs about 220 ms before the post-action frame can differ. Wait for a genuinely new frame, not a fixed short delay
- The pointer renders as a small low-contrast grey dot that a whole-frame diff cannot see. A 40 px box around the commanded point measured 1.4 to 9.8 against a 0.02 idle floor
- `/mouse/reset` is confirmed only to clear the stuck-click state. Whether it corrects a drifted pointer is unverified; an offset pointer is a calibration problem and needs `/device/collection/mouse`
- `error_push` has no per-request correlation id, only call_fun and message, so matching it to a specific in-flight request is the client's problem

SCALE

`/mouse/reset` fans out to the whole target set in one request, so a 19-phone reset is one call. The probe screenshots are the per-phone cost at about 2 ms each, so a 19-phone control check is well under 100 ms of kernel time plus one source-frame interval per phone, which is the real wait: about 35 ms on a healthy link and about 220 ms on the slow XR. Move latency scales at 0.6 ms/px identically for one phone or many, so a long probe move is hundreds of milliseconds of board time, not jitter.

TIP **Reset is free, detection is not**

Sending `/mouse/reset` costs one request and has no downside, so send it whenever the target set changes rather than trying to detect the stuck state first. Diagnosing that state as a mirroring fault and restarting the cast wastes the session and does not clear it.

The limits of reach: files, audio and events

File access stops at Photos and the Files "On My iPhone" tree

NEEDS SHORTCUT WITH CAVEATS

The entire on-device file surface is the Shortcut layer: `/shortcut/album/{get,upload,down,del,clear}` for the Photos library and `/shortcut/file/{get,upload,down,del}` for the Files app tree on iOS 15 and later. That is the whole list. Nothing in either edition reads another app's documents, caches, databases, logs or keychain, installs apps or configuration profiles, or pulls a device backup. Anything an app does not surface through Photos or Files is reachable only as pixels, and only pushed in through that app's own import UI driven by taps.

HOW

1. `shortcut#7` XP `POST /api/shortcut/album/upload` — id comma list, files as an array of PC paths the kernel can read, blank album_name means Recents, zip 0/1, outtime in milliseconds. The response returns the 10 newest album records
2. `shortcut#8` XP `POST /api/shortcut/file/upload` — iOS 15 and above, id comma list, path defaults to the My iPhone root. Code 27 is "Path does not exist", so create the directory first
3. `shortcut#1 / #4` XP `GET /api/shortcut/album/get, /api/shortcut/file/get` — both single id only. XP filters albums by album_name and num and has no date filter. An empty `ext` denotes a directory; code 35 means the list is empty
4. `shortcut#2 / #5` XP `POST /api/shortcut/album/down, /api/shortcut/file/down` — single id only, and list entries split name and extension into separate fields. Files land in the install directory under \Shortcut\Media and \Shortcut\File
5. `click / swipe` XP `/api/mouse/click, /api/mouse/swipe` — handing a seeded file to a specific app is UI automation in mirror pixels, verified only by frame differencing

6. `shortcut#3 / #6` XP `POST /api/shortcut/album/del, /api/shortcut/file/del, GET /api/shortcut/album/clear` — the del calls take comma id lists and return the 10 newest remaining records. Clear wipes an entire album
7. `shortcut#24` XP `GET /api/shortcut/device/ip?id=<id>&state=0` — the cheapest proof the whole channel works before building on it. A 15 s wait ending in code 30 means the hotkey did not launch the shortcut

NEEDS

- The iMouse Shortcut installed with the kernel IP edited in, run once with every permission allowed, and Shortcuts > Advanced set to allow running scripts, sharing large amounts of data, deleting without confirmation and deleting large amounts of data
- The shortcut bound to an AssistiveTouch key, with the board connected and the phone on the LAN. The trigger travels over USB while the payload travels over Wi-Fi, so both channels must be up
- iOS 15 or later for every `/shortcut/file/*` path; the XRs here are on 18.7.10
- Target directories that already exist, and outtime raised well above the 15000 ms default for large transfers

WATCH OUT

- None of this is verified on this rig. The Shortcut install state was never probed, `/api/pic/ocr` answering code 28 shows the plug-in layer is incomplete, and the guessed path `/shortcut/get` returned 404. Run the IP probe before designing around any of it
- Asymmetric multi-device support: upload and delete accept comma lists, but album/get, album/down, file/get, file/down, clipboard/get and device/ip are all single-id, so a rack-wide harvest is N requests
- Download calls return only a status code. The bytes land on the kernel PC's disk, so a client on another machine gets no file back from the API
- Deletion permissions are granted once on the phone and then apply to every later call. A wrong name in a del list, or a stray album/clear, destroys user data with no undo and no prompt
- Uploads read PC paths with the kernel process's privileges, and the API on 9911 is unauthenticated, so any LAN host can push files into every phone's Photos library or pull them off
- The "path must start and end with /" rule comes from the Phonefarm doc and is not stated in the XP parameter table. Only the millisecond outtime unit is confirmed for XP

SCALE

Seeding is one request for the rack, but the transfer crosses each phone's own Wi-Fi link, so wall time is bounded by the slowest radio. Harvesting is strictly one request per phone. Use zip=1 for many small files. Shortcut transfers share the LAN with AirPlay, so a large push further depresses the frame rate of an already-slow phone: schedule bulk transfers outside verification windows. A 50 MB video to 19 phones on a contended AP is minutes, and the default timeout fires long before that.

Text-to-speech is withdrawn in both editions; a user-authored Shortcut is the only substitute

NEEDS SHORTCUT · WITH CAVEATS

Phonefarm shortcut sub-function 14, "Read text aloud", is struck through in the vendor table, and the XP shortcut page has no TTS function at all. Neither edition has any audio-output or audio-capture endpoint; `air_audio` only decides whether the AirPlay stream carries the phone's sound to the PC receiver. The feature can be rebuilt on the phone: put the words on the clipboard, then launch a user-authored "Get Clipboard, Speak Text" Shortcut by URL scheme. Nothing reports back that anything was spoken.

HOW

1. `n/a (phone-side)` XP `none` — author a Shortcut named for example "Speak Clip" on each phone. There is no API to install, list or trigger Shortcuts by name
2. `shortcut id 10` XP `/shortcut/clipboard/set` — id comma list, text, sleep in seconds, outtime in milliseconds. Unicode-safe, so non-ASCII works where `/key/sendkey` cannot. Returns data.code 0 with no payload
3. `shortcut id 13` XP `/shortcut/exec/url` — id comma list, url, outtime. Passing `shortcuts://run-shortcut?name=Speak%20Clip` is documented iOS behavior but undocumented vendor behavior, so probe it on one phone. It briefly foregrounds the Shortcuts app
4. `send_key fn_key` XP `/key/sendkey fn_key` — refuted as an alternative trigger: the XP fn_key vocabulary is a closed set and a custom combination cannot be defined through the API
5. `set_airplay_mode` XP `/device/airplay/set air_audio` — receive-side only. It governs whether the mirrored stream carries audio to the PC, not whether the phone's own speaker sounds

NEEDS

- The iMouse Shortcut installed and permissioned, plus a second hand-authored Shortcut on each phone
- The phone reachable from the kernel on the LAN and cabled to a bound board at state 2, so both the trigger and the callback path work
- IMConsole.exe running and logged in. XP paths on 127.0.0.1:9911 only; Phonefarm function names return HTTP 500 here and 9912 is closed

WATCH OUT

- The vendor TTS function itself is not achievable in either edition. Only the home-made substitute works
- No completion signal. XP has no `send_text_ok` equivalent, so code 0 means accepted, not spoken
- With air_audio set to on, the sound is carried to the PC receiver, which can make an ear test misleading about where playback happened. Changing air_audio also drops the current cast
- A Shortcuts permission prompt left untapped blocks every later triggered Shortcut on that phone, and shows up in the mirror as a dialog
- Unverified on this rig: no `/shortcut/*` endpoint has ever been called here. Prove the channel with the IP probe first

SCALE

Clipboard-set and exec-url both take comma id lists, so making the whole rack speak the same line is two requests. Different text per phone is one clipboard call each, bounded by outtime, so 19 distinct lines is roughly 20 s serial or a few seconds concurrent. Playback is local to each speaker with no mixing or sequencing control, so simultaneous speech on a rack is unintelligible by design, and genuinely a noise problem in a shared room.

No push, badge or background-event listener exists; only what is drawn on the mirror can be polled

PARTIAL WITH CAVEATS

Nothing in either edition subscribes to APNs, badge counts or app events. The XP callback page has 19 sections and every one is about the rig itself. The only substitute is pixels: poll a cropped band of the mirror, threshold a mean absolute difference, then open Notification Centre and read the text with client-side OCR, because the kernel OCR plug-in is not connected here.

HOW

1. `get_device_screenshot` XP `/pic/screenshot with rect=[0,0,500,120]` — id single only, binary true for raw bytes, jpg false is the XP default, plus save_path. Crop to the banner band; read imgw/imgh per phone first
2. `n/a (client-side)` XP `none` — threshold a mean absolute pixel difference: 0.02 to 0.03 on an untouched phone, 20 to 160 for a real reaction. There is no server-side change detector
3. `send_key fn_key` XP `/key/sendkey fn_key=NoticeBar` — open Notification Centre for the full text instead of the truncated banner. Related entries: ControlBar, AppSwitch, WIN+h
4. `ocr / ocr_ex` XP `/pic/ocr, /pic/ocr-ex, /pic/find-text, /pic/find-text-ex, or the plug-in-free /shortcut/other/* equivalents` — all single id. On this rig `/api/pic/ocr` answers data.code 28, so OCR must run client-side or the Shortcut route must be stood up
5. `click` XP `/mouse/click, preceded by /mouse/reset` — act on the notification in mirror pixels, then dismiss with two presses of WIN+h
6. `shortcut id 13` XP `/shortcut/exec/url` — where the app allows it, deep-link to the inbox and poll that screen instead of waiting for a banner
7. `(none)` XP `ws://127.0.0.1:9911/api` — the real event bus covers the rig only: im_connect, dev_connect, dev_disconnect, dev_rotate, dev_change, dev_delete, group_change, group_delete, usb_change, collection_mouse, airplay_connect_log, restart_log, user_info, im_log, error_push, im_config_change, logout, dev_sort_change

NEEDS

- The phone mirroring and unlocked with Auto-Lock Never, and notifications set to show as banners for the apps of interest, none of which is queryable through the API
- Client-side OCR or per-app pixel templates on the PC
- Per-phone imgw/imgh read before any rect or click coordinate is computed, and the board at state 2 if the flow acts on what it finds

WATCH OUT

- Subscribing to push notifications, badge counts or background app events is not achievable. Only visible pixels can be polled, and only while the phone is awake and mirroring
- Detection latency is bounded by the phone, not the kernel. A banner shown for 3 s can be missed on the 4.5 fps XR
- Opening Notification Centre covers the screen, invalidates any coordinate the flow was about to use, and must be dismissed before the next action
- A banner covers the top of the mirror and can swallow a tap that a mirrored click intended for a different phone. Exclude any phone whose imgw/imgh differs from the master
- The rect and binary combination is documented as two independent parameters but has not been measured here; the 2 ms and 1.62 MB figures are for the uncropped frame
- Reading notification content from accounts that are not yours is personal-data processing subject to privacy law, and this rig's API is unauthenticated on all interfaces, so any LAN host can pull the same frames. Automating replies to other people's messages may also breach the messaging platform's terms

SCALE

Capture is per-phone independent and nearly free: two pollers at 83 and 85 req/s, ten at 512 req/s total, with input traffic and polling measurably not interfering. So 19 phones at 4 polls per second is trivial, and a 120-row crop is a fraction of the 1.6 MB full frame. Reuse buffers, since each full frame exceeds the .NET Large Object Heap threshold. Use a fixed poll period. The binding constraints are per-phone source rate and, for client-side OCR, PC CPU per frame.

Also worth knowing

Audio: real at the receiver, narrower than "no audio capture"

WRONG **"The API has no audio capture" overstates the limit**

Two cards elsewhere in this handoff, the VoiceOver audit in the QA section and the text-to-speech entry in this section, state flatly that the API has no audio capture. That is true of the HTTP surface: no endpoint hands back audio bytes the way `/pic/screenshot` hands back image bytes. It is not true of the rack. `air_play_audio` in `/config/imserver/set` and the per-device `air_audio` override in `/device/airplay/set` exist to route each phone's sound into the PC's AirPlay receiver, and that receiver is an ordinary Windows process, `AirplayMirroring.exe`, whose output is capturable like any other Windows audio session. The correct claim is narrower: no call returns a WAV. The audio exists at the receiver and is capturable host-side, and a second, independent route captures sound at the phone itself. That reopens playback QA, ad-audio verification, alarm and notification-sound checks, and, directly against the QA-part2 claim, VoiceOver auditing, since spoken accessibility output leaves the phone through the same audio path as any other app sound.

Capturing a phone's AirPlay audio as a file, host-side

PLUS EXTERNAL TOOLING WITH CAVEATS

Whatever a phone plays through its own speaker, an ad's soundtrack, a podcast, a VoiceOver announcement, an alarm, is mirrored to the PC over the same AirPlay session that carries the picture. `air_play_audio` is the global switch and `air_audio` (0 use global, 1 off, 2 on) is the per-device override, both documented with a boolean and an integer enum respectively, nothing more exotic. The API never returns the samples; a Windows-side loopback recorder does. This is the missing half of the bandwidth story: `air_audio` was already known here as a knob for shrinking mirroring traffic, and it is that, but it is also the thing that decides whether there is anything to record.

HOW

1. `get_im_config` XP `/config/imserver/get` — read `air_play_audio` first. Not queried on this rig; the vendor's own example response ships it `true`, but that is the doc default, not a measurement here.
2. `set_im_config` XP `/config/imserver/set?air_play_audio=true` — global enable if it reads false. Not on the no-restart list the doc publishes, so this one requires a kernel restart and drops every mirroring session, exactly like the OCR fix earlier in this section.
3. `set_dev_airplay` XP `/device/airplay/set?id=<id>&air_audio=2` — force one phone's audio on, or `air_audio=1` to force a phone's audio off. The doc states that setting these per-device overrides disconnects that phone's cast immediately and the new value applies on the next mirroring connection; not exercised live here, so budget a reconnect, not an instant switch.

4. `send_key` XP `/key/sendkey?fn_key=Play/Pause` — start the media under test without touching the mirror-image coordinates, useful before a capture window opens.
5. `none` XP `Windows process loopback capture, e.g. via NAudio, against AirplayMirroring.exe's PID` — no XP path; this is the external tool. Read the target PID from the device record's `air_pid` field so the capture always points at the process actually holding that phone's session, not a hardcoded PID.
6. `get_device_screenshot` XP `/pic/screenshot?binary=true` — a timestamped frame alongside the audio buffer, so a review can confirm which visible moment produced which sound.
7. `send_key` XP `/key/sendkey?fn_key=MuteSoundSwitch or VolumeUp/VolumeDown` — normalize the phone's own volume before capture, or mute it afterward, without the reconnect cost of touching `air_audio` again.

NEEDS

- A confirmed `air_play_audio` read and, if it was off, one kernel restart accepted as a maintenance window, the same cost already paid for the OCR plug-in fix.
- A Windows audio capture library that supports per-process loopback (the Windows process-loopback capture API, available since Windows 10 2004, exposed by NAudio and similar libraries). Nothing in the iMouse product supplies this.
- The device record's live `air_pid` polled before each capture, since a phone that reconnects gets served by whichever `AirplayMirroring.exe` instance answers next.

WATCH OUT

- Measured on this rig on 10 September 2026: both XRs showed `air_pid 22236`, the identical `AirplayMirroring.exe` process, while both were mirroring. A process-scoped loopback capture on that PID captures every phone routed through it mixed together, not one phone. Whether `air_audio=1` on the phones you are not testing actually removes their contribution to that shared mix, or only stops their video-adjacent overhead while the mixed audio session includes them regardless, is undocumented and unmeasured here.
- The practical isolation method is therefore one of two disruptive options: set `air_audio=1` on every phone except the target before each capture and accept a reconnect on all of them, or use `/device/airplay/disconnect` to take every other phone off mirroring for the duration of the test. Neither is a background operation; both interrupt phones that are not under test.
- The Phonefarm sibling's `set_airplaysrvnum` can run multiple AirPlay receiver instances, which would give a real per-receiver isolation boundary. XP has no receiver-count concept, only the single `air_play_name` in `/config/imserver/get`, so that isolation path does not exist on this edition.
- Whether the hardware volume keys (`VolumeUp`, `VolumeDown`, `MuteSoundSwitch`) act on the same audio session AirPlay mirrors, versus a separate ringer or call-volume path, is an iOS behavior no vendor doc covers. Verify with a capture before trusting a mute.
- A restart to flip the global switch invalidates every live capture and every bound coordinate sequence that assumes the phone is still mirroring; sequence it like the OCR fix, as a planned window, not a per-test toggle.

SCALE

The global switch and the screenshot calls are cheap and already fan out or run at the rig's usual per-id rates. Isolation is what does not scale: proving a clean single-phone recording costs one reconnect cycle per phone you silence, so testing all 19 boards one at a time against a shared receiver is 18 mute-and-reconnect operations per test round, not a parallel sweep. Treat this as a one-phone-at-a-time QA lane, not a rack-wide audio pipeline, until the mixing question above is measured.

Recording a phone's microphone through a self-authored Shortcut NEEDS SHORTCUT WITH CAVEATS

This is a second, unrelated audio route: not the phone's outgoing sound, its incoming microphone. iOS Shortcuts ships a general-purpose Record Audio action that saves a clip to Files or the Shortcut's own output. Nothing in the iMouse documentation names it; it is a capability of Apple's Shortcuts app that a rack operator would have to build themselves, on top of the one fixed iMouse Shortcut this handoff documents elsewhere.

HOW

1. `none` XP `no XP path` — build a distinctly named Shortcut by hand on the phone: Record Audio, a fixed duration, then Save File to Files or iCloud Drive. Run it once locally so the microphone permission prompt is already answered; a remote trigger cannot see or dismiss that dialog.
2. `shortcut#13` XP `/shortcut/exec/url?url=shortcuts://run-shortcut?name=<name>` — trigger it remotely. `/shortcut/exec/url` is a general Open URL action, and every example in the XP, equipment and shortcuts docs uses an http or https domain; a `shortcuts://` scheme reaching this endpoint is inferred from Apple's own URL scheme, not demonstrated anywhere in the vendor material.
3. `shortcut#6` XP `/shortcut/file/get?id=<id>` — poll for the new file by name and timestamp once the recording should be done. There is no completion callback; `exec/url` returning code 0 only means the OS accepted the URL, not that the Shortcut finished.
4. `shortcut#7` XP `/shortcut/file/down?id=<id>&list=[{name,ext}]` — pull the audio file back to the PC. Single id, like every other `/shortcut/*` read path.

NEEDS

- iOS's own microphone permission granted by hand, once, per phone. No endpoint grants it remotely.
- The same invisible iOS accessibility baseline the rest of this document assumes: unlocked screen, IMConsole.exe running, board at state 2. iOS 15 or later for the `/shortcut/file/*` family.
- A distinct, identically spelled Shortcut authored and saved on every phone that needs this, by hand, with no API to push or template it, the same manual-per-phone cost already described for the Unicode Shortcut in this section.

WATCH OUT

- No `/shortcut/*` endpoint of any kind has been exercised on this rig. This route inherits that at full force, on top of an untested URL scheme.
- The microphone hears whatever is acoustically present, the phone's own speaker if something is playing, room noise, or, depending on iOS version and the calling app, audio from an active call. Call and FaceTime audio capture through this route is a guess about iOS behavior, not a documented one, and should be tested on an expendable line before being relied on.
- This captures a live voice, not a UI state. Two-party consent recording statutes in a number of US states, and wiretap and interception law generally, apply to recorded calls regardless of what the platform's own terms say. Room or ambient recording without the knowledge of the people present raises the same exposure. Confirm consent and jurisdiction before pointing this at anything but test content the operator controls end to end.

SCALE

One phone at a time in practice: a personalized `shortcuts://` URL only reaches every phone in an id list if every phone carries a Shortcut of that exact name, and even then each phone's recording, poll and download are three separate single-id calls. Budget a Shortcut round trip per phone plus the recording's own duration, the same order of magnitude as the Unicode clipboard round trip elsewhere in this document, not the sub-millisecond fan-out of a shared mouse or key call.

Automating apps that refuse simulators, emulators and jailbroken devices

NEEDS SHORTCUT

WITH CAVEATS

Banking and fintech apps, government e-ID and passport apps, insurance and healthcare portals, and DRM-gated media apps commonly run their own root, jailbreak and emulator checks and refuse to load, or drop to a degraded mode, on an Android emulator, an iOS simulator or a jailbroken phone. A device cloud built on cloud-hosted emulator VMs is exactly what those checks exist to catch. This rack is 19 stock iPhone XRs with no jailbreak and no app installed on the handset, driven through a USB HID accessory board and mirrored over AirPlay, so an app running on it sees genuine consumer hardware and a stock OS, the same as it would from the owner's hand. That is the rack's strongest point of difference against a simulator farm for this exact class of app, and it is worth stating plainly rather than leaving it implicit.

HOW

1. `click / send_key` XP `/mouse/click, /key/sendkey` — log in and navigate the app's screens. Board HID, no phone-side Shortcut needed, and both fan out to a comma id list in one request.
2. `get_device_screenshot` XP `/pic/screenshot?id=<id>&save_path=<dir>` — capture evidence of screen state (a balance, a claim status, a verification screen) as a file on the PC. Single id, as documented.
3. `shortcut#10 / #11` XP `/shortcut/clipboard/set then WIN+a, WIN+c, /shortcut/clipboard/get` — where an exact string matters more than a screenshot (an account number, a policy ID, a confirmation code), select and copy, then read `data.text` back. Needs the full iMouse Shortcut chain on that phone.
4. `get_user_info` XP `/config/user/info` — check `dev_num` and `dev_online_num` before committing a regulated-app workflow to the whole rack, since it says how many devices the license actually covers.

NEEDS

- Accounts the operator owns, or is specifically authorized by the account holder or the platform to test with. Nothing here changes what the app's own terms of service permit.
- For the clipboard steps, the same iMouse Shortcut prerequisite chain documented elsewhere in this section: Shortcut installed and IP-configured on the phone, AssistiveTouch and Full Keyboard Access on, board at state 2.
- Coordinates and timing measured per app screen, the same as any other UI-automation flow on this rig.

WATCH OUT

- Face ID, Touch ID, Apple Pay, Secure Enclave-backed key confirmation, and a physical side-button double-press all stop the rack cold. No function in the API can satisfy a biometric prompt or a Secure Enclave confirmation, so a KYC selfie liveness step or a payment confirmation step is the one part of these flows that cannot be automated end to end. Everything up to that gate can be driven; the gate itself cannot.
- Frames and clipboard reads captured from these apps carry regulated data by construction: account numbers, health information, ID document images, policy details. `/pic/screenshot?save_path` writes that data unencrypted to local disk in plain BMP or JPG with no retention control of its own, and port 9911 has no authentication, so any other host on the LAN can request the same screenshot or the same clipboard read

while the workflow runs. Purge captured frames rather than archiving them, and keep 9911 off any network you do not control.

- Being real hardware defeats false positives from an emulator check, not the anti-fraud or anti-abuse logic behind it. This is a statement about what the app's device check sees, not a way to get past controls it is not designed to catch.
- Scope every use of this to apps and accounts the operator owns or is authorized to test. Running it against an account that is not the operator's, or against a platform whose terms forbid automated interaction, carries its own legal exposure under computer-misuse, banking, health-privacy or platform-terms law well before the technical wall above is reached.

SCALE

Navigation is cheap and shared: `/mouse/click` and `/key/sendkey` take a comma id list, so stepping the whole rack through an identical login or menu sequence is one request regardless of phone count. Verification is not: `/pic/screenshot` and `/shortcut/clipboard/get` are single id, so confirming the result on each phone is always N requests, one per phone, the same throughput ceiling documented for the rest of the rig. Check `dev_num` and `dev_online_num` from `/config/user/info` before scaling a regulated-app workflow past what the license and the online fleet actually support.

The one avenue that could move the walls: talking to the boards directly

Every limit above is a limit of the documented HTTP API sitting on top of `IMService.exe`. Underneath it there is a second, undocumented surface: each board enumerates as a USB HID device, vendor ID `720A`, on a vendor-defined usage page (`0xFF00`, usage `0x00`) that Windows cannot decode from the descriptor alone. What is measured is only the shape of the channel: a 9-byte output report (1 report ID plus an 8-byte payload) carries PC-to-board commands, and a 17-byte input report (1 report ID plus 16 bytes) carries board-to-PC replies. The content of those bytes is unread. `analysis/HARDWARE.md` names the fix as its own second recommended next step: run the original vendor software, capture its USB traffic with USBPcap or Wireshark while triggering known actions such as Home, a single tap and a swipe, and diff the reports to recover the command layout. That work has not been started, and it has not appeared anywhere else in this handoff as a project option.

It matters because it is the only lever in sight against three things this document otherwise treats as fixed. First, the 0.6 ms-per-pixel pointer walk: `/mouse/move` /mouse/move measured at 592 ms for a 1,000-pixel jump, and `/mouse/swipe` /mouse/swipe rides the same walk underneath its `steping`, `step_sleep` and `brake` parameters, which govern how many loop iterations a slide takes and the pause between them, not how fast the board itself moves the cursor. A 1,080-pixel swipe still takes about 580 ms of travel regardless of how those three parameters are set, which is why velocity-dependent flicks do not fire. Whether that pacing is a firmware limit of the board or an artifact of how `IMService.exe` frames each move as a step sequence is exactly the question a decoded report would answer; today there is no way to tell the two apart. Second, the comma-list fan-out: the "one parameter set for every id" constraint is a property of the HTTP layer, where a single call carries one shared payload to a list of ids. Each board is a separate USB device with its own PID, so writing distinct 9-byte reports to distinct device handles is a structurally different operation from a shared HTTP parameter, and might allow real per-phone payloads in one coordinated pass instead of one HTTP call per phone. Third, total dependence on the vendor kernel: `/config/usb/get` /config/usb/get is the only window onto board attach state (`vid`, `pid`, `uid`, `ver`, `state`), and `/device/usb/restart` /device/usb/restart is the only reset path, and both exist only because `IMService.exe` is running and licensed. Direct HID access would let a console enumerate and drive boards through the Windows HID APIs with no kernel process, and no license, in the loop at all.

None of that is close to free. It needs a capture rig: a machine running the original iMouse software, USBPcap or Wireshark recording the USB traffic, and one board set aside that can be sacrificed if a command byte is guessed wrong. It needs protocol work with no starting point better than diff-and-guess, first the 8-byte command space, then the meaning of the 16-byte reply, with nothing in any vendor document to check the guesses against. It costs the vendor relationship: no more firmware updates, no more support, and whatever validation `IMService.exe` currently does on a report before it reaches the board has to be rediscovered or done without. And it carries a real risk of bricking a board, because a malformed report sent straight to unknown firmware has no documented safe-failure behavior, unlike an HTTP call that answers a clean status 500 or a `data.code` the docs define.

Nothing here depends on which edition's kernel is installed; the boards and their HID bridge sit underneath both, and the capture-and-diff evidence for the report sizes was gathered against this XP rig. Treat this as a parallel investigation, run against a spare board once the documented-API console is otherwise working, never as a prerequisite for it. Everything else in this handoff already ships on `/config/usb/get`, `/device/usb/restart` and the rest of the documented surface; this is the one place where a locked door is at least a door, not a wall.

The Apple-ecosystem checklist

Ten features a buyer will ask about by name before asking about anything below this line. None of the underlying vendor terms (Bluetooth, NFC, Wallet, Siri, eSIM, Guided Access, Screen Time, Find My, Family Sharing, iCloud) appears anywhere in the nine XP doc files or the Phonefarm doc: a full-text search across all ten returns zero hits. That absence cuts two ways. Where the feature needs a radio or a physical button the API does not expose, it is a hard wall, not a gap to work around. Where the feature is just another app on the home screen, it is ordinary UI automation, no different from anything else in this document, and in three cases it is a better answer to a real operator problem than anything already covered.

Feature	Feasibility	Verdict	Mechanism, and why
Apple Watch and wearable pairing	Not possible	Refuted	No endpoint. Pairing is a Bluetooth handshake plus a confirmation tap on both the phone and the watch, and neither edition documents a single Bluetooth call of any kind, not a scan, not a connect, not a state read. There is no channel to start this, driven or otherwise.
AirPods and audio routing	Not possible	Refuted	Same wall. Even switching output among AirPods already paired by hand would mean a blind `/mouse/click` on the Control Center audio card, a coordinate never measured on this rig and not worth designing on when the API cannot see or set the Bluetooth state behind it.
CarPlay	Not possible	Refuted	CarPlay needs a wired or Bluetooth session with a head unit that does not exist in a rack, and no function in either doc names it. Not a rig limitation, a category mismatch.
NFC and Wallet	Not possible	Refuted	A Wallet tap-to-pay or transit pass is triggered by a physical double-click of the side button. The `/key/sendkey` hotkey table (`WIN+h`, `ControlBar`, `NoticeBar`, `TAB+l`, and the rest) opens every other hardware-button surface but has no side-button entry at all, and there is no software trigger for the NFC radio either.

Feature	Feasibility	Verdict	Mechanism, and why
Find My	Partial, UI only	With caveats	Opening the app is ordinary automation: `/key/sendkey?fn_key=WIN+h` to a known home screen, then `/mouse/click` on the icon. Nothing reads a location back through the API, only through the mirrored screen, so this is a screenshot-and-look tool, not a query. Still genuinely useful here: 19 owned handsets on a shelf is exactly the loss and theft scenario Find My is for, and it costs nothing beyond one Apple ID sign-in per phone.
Family Sharing	Partial, UI only	With caveats	Reachable the same way as any Settings screen, `/mouse/click` through Settings, the account banner, then Family Sharing. Little reason to use it on a rack of independent farm phones, and any per-account change it makes is one phone at a time: the shortcut fan-out endpoints (`/shortcut/exec/url` and the rest) share one payload across an id list, but Family Sharing has no shortcut path at all, so there is no batch form to fall back on.
Screen Time and Guided Access	Partial (Screen Time) / Not possible (Guided Access trigger)	With caveats	Screen Time app limits are a one-time `/mouse/click` walk through Settings per phone, and once set they enforce themselves with no further API calls, which is a real guardrail against an automation that drifts into the wrong app. Guided Access does the same job more tightly, pinning the screen to one app, but its standard trigger is a physical triple-click of the side button, again absent from the `/key/sendkey` hotkey table, so nothing here can turn it on or off once configured.
iCloud backup and restore	Partial, UI only	With caveats	Settings > [name] > iCloud > iCloud Backup is reachable by `/mouse/click`, and restoring a fresh phone from an existing backup is the fastest route to identical home screens across the rack, faster than laying out icons by hand on each one. The wall shows up at sign-in: a new Apple ID on a phone needs two-factor approval, a code delivered by SMS or push that this rig cannot read (see the messaging entries above), so this only scales across phones that already share one signed-in account.
eSIM	Partial, UI only	With caveats	The Settings walk to Cellular, Add eSIM is ordinary automation, and an activation code supplied as text rather than a QR image can be typed in with the documented Unicode route, `/shortcut/clipboard/set` then `/key/sendkey?fn_key=WIN+v`. Carrier activation by QR code cannot be automated at all: that flow points the phone's camera at a printed or displayed code, and nothing in the API can present an image to the camera or trigger the scan. Treat provisioning as a manual, per-phone task like the rest of the one-time setup chain.
Siri and dictation	Not possible (Siri) / Partial (dictation)	Refuted (Siri) / With caveats (dictation)	Siri has no software trigger in either edition: it is invoked by a side-button long-press or by "Hey Siri," and neither is a key the hotkey table defines. Keyboard dictation is only the microphone glyph on the on-screen keyboard opened with `fn_key=OPENKeyboard`, tappable like any other icon, but there is no endpoint that returns what it transcribes; the result still has to leave the phone through the clipboard round trip documented for Unicode text.

TIP **Three of these are worth building, not just noting**

Find My, Screen Time app limits and iCloud backup or restore all clear the same bar as everything else marked "with caveats" in this document: reachable by `/mouse/click` and `/key/sendkey` through the visible UI, verifiable by screenshot, and each solving a real problem for a rack of owned handsets rather than a workaround for a missing feature. None of the three has been exercised on this hardware; treat the coordinates and timings as unmeasured until probed here, the same caveat that applies to every UI-automation entry in this handoff.

TRAP **The pattern behind six refusals is one missing radio and one missing button**

Apple Watch pairing, AirPods, CarPlay, NFC and Wallet, and Siri's own trigger all fail for one of two reasons: the API defines no Bluetooth function anywhere, in either edition, and the `/key/sendkey` hotkey table has no entry for the side button, single-click, double-click or long-press. Any future vendor firmware or edition that adds either one would reopen this whole list at once, so it is worth re-checking both facts, not just re-testing each feature individually, before assuming they are still closed.

TRAP **Regulated data has no separate lane on this rack**

GDPR and CCPA are not the only regimes this rig can walk into, and nothing in the kernel treats regulated data differently from any other pixel. The facts are scattered across this document; here they are in one place. `/pic/screenshot` with `save_path` set writes a full, unencrypted BMP or JPG of whatever the phone was displaying to a plain filesystem path on the kernel PC, no encryption parameter exists. `/shortcut/clipboard/get` returns `data.text` for whatever is on the pasteboard, which on a banking, health or support app is routinely a password, a one-time code or a card number, in plaintext, in the HTTP response. Every saved frame accumulates in that same folder with no expiry, no size cap and no delete-by-age call anywhere in the XP or Phonefarm docs, so `save_path` is an unbounded archive of everything ever shown on screen. `C:\iMouseXP\IMService.json` holds the vendor account password behind `/config/user/login` as reversible hex obfuscation, recoverable from that file alone, and because `/config/user/switch` moves between the ten `utag` sub-accounts without asking for the password again, that one recoverable string is a skeleton key to all ten. And `allow_ip_list` and `mdns_type` under `/config/imserver/set` gate AirPlay discovery only; the HTTP and WebSocket API on port 9911 authenticates no one, so any host on the LAN reaches all of the above. None of this is a bug. It is the documented shape of the product. It becomes a compliance problem the moment the screen being mirrored belongs to a banking, health, government or customer-support workflow, several of which this document proposes elsewhere.

Regime	What triggers it on this rig	Control to add, since the API has none
GDPR / CCPA	Any frame from `/pic/screenshot` or clipboard text from `/shortcut/clipboard/get` that identifies an EU, UK or California resident becomes personal data the instant it lands on disk or in a response body.	A lawful basis and a written retention limit for the `save_path` folder, enforced by your own scheduler, since no kernel call deletes or ages out a saved frame.
HIPAA	A frame or clipboard read taken while a patient portal, telehealth or pharmacy app is on screen is electronic protected health information the moment it is captured, whether or not anyone reads it.	Never capture the full frame near such an app: use `/pic/screenshot`'s `rect` to crop to only the field needed, encrypt the volume the archive lives on, and treat the operator as a business associate needing a signed agreement with whoever owns the data.

Regime	What triggers it on this rig	Control to add, since the API has none
PCI-DSS	A card number or CVV copied inside a payment app and read back with `/shortcut/clipboard/get`, or a card-entry screen caught by an unscoped `/pic/screenshot`, puts the rack in scope as a system that can see cardholder data.	Never persist the `text` field from a clipboard read; verify success by length or hash, not by logging the value. Crop screenshots away from card-entry fields entirely rather than after the fact.
KYC	Where a `utag` sub-account under `/config/user/login` represents a verified identity, for example an exchange or bank account, the phone screens driving its onboarding are the KYC evidence trail, and the single recoverable password in `IMService.json` is the key to every one of that login's ten sub-accounts at once.	One phone number and password per regulated program, never shared across `utag` slots with unrelated workloads, so a leaked `IMService.json` exposes one program instead of ten.

> TRAP **allow_ip_list will not do the job people assign it**
>
> Operators reach for `/config/imserver/set`'s `allow_ip_list` and `mdns_type` as if they were the access control for regulated workloads. They are not: they are documented as screencast discovery rules, and take effect only when `mdns_type` is 1. The HTTP and WebSocket API on 9911 has no authentication in either edition and accepts calls from any LAN host regardless of these settings. The only real boundary is a Windows Firewall rule scoping TCP 9911 and 17000 to the controller host, applied outside the API, before any regulated data touches this rack.

WRONG **The SMS refutation covers the vendor endpoints, not the rack**

The verdict above is correct as written for what it measures. Phonefarm's shortcut id 12, Send SMS, is struck through in the vendor's own function table, and no XP path replaces it. A full-text search of all nine XP doc files finds nothing. That part of the entry should stand.

But it is a claim about two vendor endpoints, not about the rack. The custom-Shortcut channel this document already builds elsewhere is ordinary Apple Shortcuts automation, and Shortcuts ships a native Send Message action that composes and sends an iMessage or SMS itself, with no on-screen tap and no Messages UI involved. That action is not an iMouseXP function, but the rack already has two ways to reach it. `/key/sendkey?fn_key=` fires the AssistiveTouch key bound to the phone's provisioned Shortcut, and `/shortcut/clipboard/set` already lands a recipient and body on that phone's pasteboard in one fanned-out call, so a Send Message step added to that Shortcut, reading its inputs from the clipboard, turns an existing text-delivery mechanism into a real send. `/shortcut/exec/url` is a second, unverified path to the same result: iOS's shortcuts://run-shortcut URL scheme can launch a named Shortcut with an input string directly, which would let one Open URL call pass the message text without a clipboard round trip, but every documented example of this endpoint is an http domain, so treat that shortcuts:// route as presumed rather than confirmed until it is probed on this rig.

None of this is a case for unsolicited or bulk sending. Confine it to messages the recipient has already agreed to receive, an operator-owned second number or an account that opted in, the same population the rest of this section assumes. The channel is still outbound only: no function in either edition returns a message body, and none of the 18 WebSocket callbacks fires on receipt, so there is no way to read a reply and no two-way flow to build on top of this. What actually constrains the capability is carrier acceptable-use terms on the sending SIM, Apple's terms for Shortcuts-driven Messages automation, and the consent and anti-spam law already named for the UI workaround: TCPA, CAN-SPAM, PECR and GDPR. That is a legal and contractual limit, not a technical one.

Re-test before this gets promised to anyone: add the Send Message step to the provisioned Shortcut, fire it through both trigger paths, and record whether a blocked permission or bad recipient comes back as anything other than the same `data.code` 30 timeout every other missing-Shortcut symptom produces. Until that probe runs, this entry should read Needs Shortcut, With caveats, not Not possible, Refuted.

Prerequisites and setup checklist

Every capability in this document rests on a stack of things the API cannot see: iOS accessibility toggles, a bound board, a calibrated pointer, a running console, and on two functions, a plug-in that is not installed on this rig. This section lists what each capability class needs, how to prove it is actually working rather than merely configured, and the order to bring a new phone onto the rack.

Global one-time setup (per rig, not per phone)

- Install the VC runtime and add the iMouseXP install/download folder to Windows Security or 360 exclusions; several install prompts must be allowed or the kernel service and AirPlay receiver never register.
- Plug boards into the PC before cabling phones. Order is enforced by the hardware: a board reaches `/config/usb/get` state 1 first, state 2 only once a phone is cabled to it.
- Start `IMConsole.exe` and log in. On this rig, with only `IMService.exe` running, both phones read state 0 in `/device/get` and the board list in `/config/usb/get` was empty. The API alone does not bring the rack up.
- Set `air_play_name`, `air_play_port`, `air_play_ratio` / `fps` / `refresh_rate`, and the plug-in counts `opencv_num` / `ocr_num` in `/config/imserver/set`, then call `/imserver/restart` once (most of these fields need a restart; `lang`, `mdns_type`, `connect_failed_retry`, `allow_ip_list`, `auto_connect`, `auto_updata`, `thread_mode`, `mouse_mode` and `flip_right` apply live).
- Open the firewall for TCP 9911 (API), TCP 17000 (AirPlay) and UDP 5353 (mDNS) on the phones' LAN segment, and restrict who else can reach them: the API on this kernel has no authentication.

> TRAP **Two editions, one kernel, wrong function names fail silently-ish**
>
> This rig runs the XP kernel on port 9911 with path-style endpoints (`/api/mouse/click?id=...`). Sending a Phonefarm-style function call such as `{"fun":"get_device_list"}` to this kernel returns HTTP 500 "List index out of bounds", not a normal JSON error. Port 9912, which the Phonefarm doc uses, is closed here. Confirm the edition before trusting any function name from the Phonefarm doc: check the response shape of a known-good XP path first.

New-phone bring-up order

Steps in the order the hardware and software enforce them. Each has an immediate check; do not skip ahead on a step that has not passed its check.

1. **Board to PC.** Plug the board into a free hub port. Check: `/config/usb/get` lists its `vid` / `pid` at state 1.
2. **Cable to phone.** Use the vendor-supplied Lightning or USB-C cable only; a substitute cable is a real failure mode. Check: the same board entry moves to state 2.
3. **Phone-side accessibility settings.** On the phone: AssistiveTouch on with tracking sensitivity at maximum and "Always Show Menu" off; Full Keyboard Access on with 1 s auto-hide; Trackpad & Mouse tracking speed at maximum (this menu only appears while the board is attached — its absence is itself a diagnostic); Auto-Lock Never; portrait orientation locked; StandBy off; Control Center with two rows of icons at the bottom. None of this is queryable over the API. Check: only by driving input in a later step.

4. **Mirror the phone.** Control Center > Screen Mirroring > select the receiver named in `air_play_name` (this PC advertises as iMouseXP-5429). Check: `/device/get` shows a non-zero `state` and a non-zero `air_pid`, and `imgw` / `imgh` are populated (500×1080 on this rig's XRs).
5. **Bind the board to the device record.** `/device/set?id=<deviceid>&vid=720A&pid=<pid>` (vid and pid must be set together; binding is single-device). If unsure which physical board is which, unplug and replug the phone cable and watch which `uid` changes state. Console equivalent: right-click the phone > Bind hardware. Check: `/device/get` shows the `vid` / `pid` / `uid` populated.
6. **Calibrate (mouse parameters).** Check `/device/get` for a non-empty `location_crc` first — the shared library (this rig's value: B0DBD800) may already cover the model and iOS version. If empty, run `/device/collection/mouse?id=<id>&cmd=1`, complete the collection page on the phone's home-screen icon, then `cmd=2` to stop and `/device/collection/mouse/save` to publish it. Check: the corner-accuracy probe below.
7. **Install and bind the iMouse Shortcut** (only if you need clipboard/Unicode text, album, files, device switches, or the shortcut OCR/image-search fallbacks). Turn on the four Shortcuts > Advanced permissions, scan the console's QR code, edit the IP inside the shortcut to the kernel PC's LAN address, run it once and allow every prompt, then bind the launch hotkey under AssistiveTouch > Devices > Customise More Buttons and click "Send Binding Key" in the console. Check: `/shortcut/device/ip?id=<id>&state=0` returns `code` 0 with the phone's known LAN IP.
8. **Plug-ins** (only if OCR or image-template search is needed). Install the vendor OCR and image-search plug-ins alongside the kernel, set `ocr_num` / `opencv_num`, restart the kernel. Check: `/pic/ocr?id=<id>` returns `code` 0 instead of 28. On this rig it currently returns 28 — treat OCR and template search as unavailable until this changes.

Prerequisites by capability class

Capability class	Phone-side needs	PC-side / plug-in needs	Edition note
Pointer/keyboard input (tap, drag, swipe, keys)	AssistiveTouch, Full Keyboard Access, tracking speed, portrait lock, Auto-Lock Never	Board bound, mouse parameters present (`location_crc` non-empty)	XP path names only; Phonefarm function names return HTTP 500 on this kernel
Screenshots / live frames	Same Wi-Fi as PC; mirroring to the named receiver; portrait lock, Auto-Lock Never	None beyond the AirPlay receiver; independent of the board	Works on both editions; coordinate space is `imgw` × `imgh`, not iOS points
Image search (find-image / find-image-cv)	Mirroring only; shortcut fallback needs Shortcut v1.3+	Image-search plug-in connected (`opencv_num` ≥ 1); status on this rig unverified, assume gated (code 28) until proven otherwise	XP exposes it via `opencv_num`; Phonefarm lists it as a base function but still needs the plug-in for auto-cast point capture
OCR / find-text	Mirroring only; shortcut fallback needs Shortcut v1.3+	OCR plug-in connected. **Not connected on this rig** — `/pic/ocr` returns code 28	`ocr-ex` is professional-licence only per the Phonefarm doc; GPU build required for `ocr_num` > 1

Capability class	Phone-side needs	PC-side / plug-in needs	Edition note
Multi-colour search	Mirroring only; fixed brightness/True Tone so sampled RGB is stable	No plug-in named in either doc; presumed kernel-internal but unverified on this rig	Colour-table syntax differs: XP uses anchor pixels with per-channel tolerance, Phonefarm uses offsets from a first colour
Shortcut-based functions (album, files, clipboard, URL, power, brightness, torch, radios, IP)	Four Shortcuts > Advanced permissions; shortcut installed with correct kernel IP; launch hotkey bound under AssistiveTouch; phone unlocked, reachable on the LAN	None beyond the kernel being reachable on 9911; target directories must pre-exist for uploads	Phonefarm gates the whole feature behind a professional licence via one `shortcut` function; XP splits it into separate paths with no licence note in the XP docs
Chinese / Unicode text entry	Full Keyboard Access on; Shortcut installed and bound (same as above)	None extra; route is clipboard set, then a paste hotkey	XP has no `send_text` function; the only documented route is `/shortcut/clipboard/set` then `WIN+v`. Phonefarm's async `send_text` is not accepted by this kernel
Auto screen-cast / cast-all / cast-on-connect	Board-driven UI navigation prerequisites (same as input); two icon rows at the bottom of Control Center; unlocked, no passcode prompt pending	OCR and image-search plug-ins installed (per the Phonefarm sibling's stated requirement for point capture); board bound with mouse parameters	XP: `auto_connect` global switch, no documented per-device autoairplay flag; Phonefarm exposes `set_usb_autoairplay` per device and three named quality tiers
Phone restart / shutdown	Driven route: input prerequisites, unlocked home screen. Shortcut route: iOS 15+. Hotkey route: Full Keyboard Access	Driven route needs the OCR plug-in (per Phonefarm sibling) for the restart coordinate; shortcut/hotkey routes do not	All three routes exist on XP; only the driven one is plug-in-gated
USB control-board restart	Board firmware 3.0+ (read from AssistiveTouch > Devices on the phone, not from the API)	Kernel 1.2.0.3+; addressed by `vid|pid`, not the phone id or the 8-char uid	Same on both editions; control channel only, video is unaffected
Mouse-parameter calibration	Same accessibility settings as input, matched to whatever settings the library entry was collected with; phone cabled, portrait; only one device collects at a time	Model + iOS version present in `/config/devicemodel/get`, or collect and save one	Same mechanism both editions; XP: `/device/collection/mouse` + `/save`; this rig uses the shared library, `location_crc` B0DBD800
Groups, names, sorting, device records	None	None beyond the kernel; metadata persists in `DeviceList.json`	Kernel-side only, no plug-in or phone dependency on either edition

Capability class	Phone-side needs	PC-side / plug-in needs	Edition note
Kernel config, licence, service control	None directly; a config restart forces every phone to re-cast	Licensed account logged in via console or `/config/user/login`; correct paths only (`/config/imserver/get`, not `/config/get`)	Both editions need an account; professional features (shortcut, `ocr-ex`) are licence-gated per the Phonefarm doc

Verification recipes reused across capability classes

Most of the checks above reduce to a handful of repeatable measurements. Use these instead of ad hoc checks; two of the alternatives below are documented traps.

- **Board and binding:** `/config/usb/get` state 2 means a board is enumerated and a phone is cabled to it; it says nothing about video. `/device/get` `vid` / `pid` / `uid` non-empty means that device record is bound to that board.
- **Mirroring:** `/device/get` `state` non-zero and `air_pid` non-zero together are the live proof; `air_pid` is the PID of the running `AirplayMirroring.exe`.
- **Mouse parameters:** `/device/get` `location_crc` (or `location`) non-empty; empty means every mouse call returns `code` 11.
- **Did an input actually land:** take the mean absolute pixel difference of a raw BMP frame (`/pic/screenshot?binary=true`) in a roughly 40 px box around the target coordinate, before and after the action. An idle phone reads 0.02–0.03; a landed tap or move reads 1.4 and up; a keypress that changes the whole screen reads 20 and up. Never compare frames byte-for-byte or by hash: the AirPlay encoder never reproduces a frame exactly, so a hash comparison reports every idle phone as “reacted”.
- **Stuck taps:** if `/mouse/move` and hardware keys work but `/mouse/click` reads at the idle floor, send `/mouse/reset` to the affected id and retry before concluding the phone is unresponsive. Do this for the whole target set whenever the mirrored group changes, before any tap-critical sequence.
- **Corner accuracy after calibration:** move to three points spanning the frame (e.g. a near corner, the far corner, the centre) and box-diff each; error that grows toward the far corner means the per-device scale is wrong and needs a fresh collection rather than the shared library.
- **Cross-phone actions:** press Home twice before mirroring one coordinate to several phones. The two XRs on this rig have different home screens, so an unnormalised coordinate lands on a different app on each.

API PORT	AIRPLAY PORT	MDNS	MIRROR SIZE
9911	**17000**	**UDP 5353**	**500 × 1080**
HTTP + WebSocket, no auth on the LAN	AirplayMirroring.exe, per-phone socket	Receiver advertisement (Bonjour)	Coordinate space for /mouse and /pic, this rig’s XRs

Diagnostic codes worth memorising

Code	Meaning	Typical cause
2 / 3	Not bound / hardware not connected	Board not bound to that device, or cable/board not seated
5	A device is collecting	Only one phone can run mouse-parameter collection at a time

Code	Meaning	Typical cause
7	Device is online	`/device/del` refused a phone that is still connected
11	No mouse parameters	`location_crc` / `location` empty on that device
15	No coordinates / must be portrait	Auto-cast or auto-restart point not captured, or the phone is landscape
27	Path does not exist	Shortcut file/album target directory was never created on the phone
28	Plug-in not connected	OCR or image-search plug-in missing — the current state of this rig's OCR
29 / 30	Plug-in timeout / call timeout	Plug-in present but slow, or a shortcut hotkey never fired
31 / 32	OCR found nothing / no text found	Screen too small at the current `air_play_ratio`, or genuinely blank
34	Remove devices from group first	`/device/group/del` guard against deleting a non-empty group
404	Interface does not exist	Guessed or Phonefarm-style path sent to this XP kernel
500	List index out of bounds	Phonefarm `{"fun": ...}` body sent to this kernel instead of an XP path

> TIP **Start every session with three calls**
>
> `/config/imserver/get` (kernel alive and configured), `/config/usb/get` (board count and states), `/device/get` (mirroring state and `air_pid` per phone). If IMConsole has not been started since boot, the second and third will read empty or state 0; start it before running anything else.

Also worth knowing

Phone provisioning as an automatable job

Jumping into Settings by URL scheme, then searching inside Settings itself

NEEDS SHORTCUT

WITH CAVEATS

The five phone-side prerequisites this section already calls unreadable and "Not possible" (AssistiveTouch, Full Keyboard Access, tracking speed, Auto-Lock, Control Center rows) all live inside Settings, and today reaching them means tapping the Settings icon by hand on every phone. `/shortcut/exec/url` already opens "any app via its URL scheme or universal link" per its own doc entry; the same call fired at an iOS Settings URL such as `prefs:root=General` or `App-Prefs:root=General` lands directly on a Settings pane without ever touching the home screen. Once inside Settings, its own search field is fixed native chrome, not a user-arranged icon, so it is legal to target with one blind coordinate across every phone on the same iOS build, unlike the home screen this document already flags as non-uniform.

HOW

1. `mouse_reset_pos, send_key` XP `GET /api/mouse/reset?id=<a,b,c> then GET /api/key/sendkey?id=<a,b,c>&fn_key=WIN+h` (twice) — clear stuck clicks and normalize to home page one, same as the existing Spotlight recipe.
2. `shortcut#13 / /shortcut/exec/url` XP `GET /api/shortcut/exec/url?id=<a,b,c>&url=App-Prefs:root=General` — one call, comma-fanned to the whole target set, opens Settings directly on every listed

phone.

3. `click` XP `GET /api/mouse/click?id=<a,b,c>&x=&y=` — tap the visible search field near the top of the Settings root list (a fixed OS-chrome coordinate, not a user-arranged one).
4. `send_key (key)` XP `GET /api/key/sendkey?id=<a,b,c>&key=assistivetouch` — type the pane's name (ASCII only) instead of guessing which nested menu it lives in.
5. `send_key (fn_key)` XP `GET /api/key/sendkey?id=<a,b,c>&fn_key=ENTER` — open the top search hit, then proceed with clicks local to that pane.

NEEDS

- iMouse Shortcut installed, IP-edited, run once, and bound to an AssistiveTouch key (bring-up step 7); verify first with `/shortcut/device/ip?id=<id>&state=0` before designing around this call, exactly as the existing Spotlight card already recommends.
- Full Keyboard Access on, for the typed pane name to register.

WATCH OUT

- Neither vendor doc names `prefs:` or `App-Prefs:` specifically; the doc's grant is the general "any URL scheme" claim on shortcut#13, not a tested Settings deep link. Confirm the exact root anchor and any sub-pane fragment on one phone before fleet-wide use; Apple has narrowed what these schemes can open in past iOS releases without notice.
- A root anchor (`prefs:root=General`) is far more likely to keep working than a deep sub-pane fragment for something like Accessibility > Touch > AssistiveTouch, which is why the in-Settings search step exists: it reaches a specific pane without needing a guessed or brittle deep-link string.
- `/key/sendkey` is ASCII-only in both editions; a pane name is fine, but do not expect this route to help with non-Latin text anywhere else in Settings.
- `/key/sendkey` also documents an `OPENKeyboard` fn_key ("open the on-screen keyboard") in the XP hotkey table; it is not verified here beyond that one-line description, and it does not appear to be a substitute for tapping the search field first, only a possible way to force a keyboard to appear once a field already has focus.
- Without OCR, there is no server-side confirmation the search actually landed on the intended pane rather than a similarly named one; corroborate with the colour test below when the destination is a toggle.

SCALE

`/shortcut/exec/url`, `/mouse/click` and `/key/sendkey` all accept comma-separated id lists, so this whole chain runs once against all 19 boards rather than once per phone, the same fan-out already used elsewhere in this document.

Reading a toggle's on/off state from its pixels

NATIVE API WITH CAVEATS

None of the five accessibility prerequisites expose a queryable state anywhere in either doc; the only way to know AssistiveTouch or Full Keyboard Access is actually on is to look at the switch. With the OCR and image-search plug-ins gated (code 28 on this rig), a colour test on the switch's green fill is the one audit path that does not depend on either plug-in, because the toggle is visible in the mirror stream regardless of plug-in state. This turns setup verification from "assume it's configured" into "read it back," even on this rig today.

HOW

1. `pic/screenshot (rect)` XP `GET /api/pic/screenshot?id=<id>&binary=true&rect=x0,y0,x1,y1` — grab a raw BMP crop tight around the switch's known coordinate; this is the same rect-crop mechanism the setup section already uses for the mean-pixel-difference input check, just decoded for colour instead of difference.

2. `pic/find-multi-color` XP `POST /api {"fun":"/pic/find-multi-color","data":{"id":"<id>","list":[{"first_color":"x|y|34C759-202020","similarity":0.85,"rect":[x0,y0,x1,y1]}],"same":false,"all":false}}` — kernel-side alternative: ask directly whether the anchor pixel matches iOS system green within tolerance, without pulling the frame client-side at all.
3. Compare the sampled or matched colour against a green/gray pair captured once on this rig with the toggle in each known state, rather than trusting a textbook hex value.
4. On a mismatch, `mouse/click` the same coordinate and re-run the read to confirm the flip actually landed, rather than trusting the click alone.

NEEDS

- The toggle already on screen at a known coordinate — this pairs with the URL-scheme-plus-search recipe above (or, until layouts are standardized, a per-phone coordinate) to get there deterministically.
- Fixed brightness and True Tone off, per this section's existing prerequisite table for multi-colour search, so the sampled RGB does not drift between phones or over the course of a day.
- Mirroring only for the read; the board is needed only for the corrective click, not for the check itself.

WATCH OUT

- `/pic/find-multi-color` is not named as needing a plug-in in either doc, unlike OCR and image search, but it has not been exercised on this rig — run one call against a known-on and known-off toggle before trusting it in a loop.
- iOS system green is commonly cited near `#34C759`, but that is a starting anchor, not a measured value from these XRs; calibrate similarity the same empirical way the mean-pixel-difference thresholds elsewhere in this section were calibrated (idle floor vs. landed action), not from the doc's own example similarity of 0.85, which was chosen for a much larger multi-pixel colour table.
- A colour match at one coordinate only proves that coordinate is green; it says nothing about which setting the switch belongs to. It is only meaningful once the on-screen pane is already known, which is why this card is paired with, not a substitute for, deterministic navigation.
- Both `/pic/screenshot` and `/pic/find-multi-color` take one device id per call ("multiple is not allowed"), unlike `/mouse/click` and `/key/sendkey`; a fleet-wide audit is one request per phone, not one request for the fleet.

SCALE

The read itself is one call per phone regardless of fleet size, so a 19-board audit is 19 requests either way; the corrective click and re-check that follow a mismatch are the more expensive part per phone, but they only run for the phones that failed the read, not the whole rack.

Standardizing the home screen so one coordinate is legal fleet-wide

PLUS EXTERNAL TOOLING

WITH CAVEATS

This section already measures the consequence of skipping this step: two identical XRs on this rig have different home screens, so an unnormalized coordinate lands on a different app on each. Restoring every phone from one reference backup, or manually matching layouts by hand, removes that variable at setup time instead of working around it in every later recipe. Neither path runs through the iMouse API; the API's role is limited to confirming and recording which phones have actually been normalized.

HOW

1. (External) Restore each phone from one reference backup, via Finder or Apple Configurator over the same cable already used for board control, or from a single iCloud backup taken on a template phone — so wallpaper, page count, folder structure and icon order match across the fleet. Nothing in either vendor doc exposes this; it is standard Apple device management running alongside, not through, the kernel.
2. (External, only for a handful of phones) Absent a reference backup, manually drag icons into a matching layout once per phone, using the phone's own touch or a board-driven drag (`mouse/down` at the icon, `mouse/move` to the target slot, `mouse/up`) — labor that does not amortize past a few units.
3. `device/set (name)` XP `GET /api/device/set?id=<id>&name=XR-18.7-layoutA` — once a phone's layout is confirmed to match the reference, tag its device record so later fleet-wide blind clicks can filter `get_device_list` on that label before firing. This changes nothing on the phone; it is a queryable marker for "trust one coordinate here," on the same terms this document already uses the device name elsewhere: a convenience label, not a source of truth the kernel checks.

NEEDS

- A reference phone or backup image, and Finder, Apple Configurator, or iCloud — none of which the iMouse API touches.
- The accessibility bring-up in this section's new-phone order (step 3) completed and captured in that same reference backup, not before it.

WATCH OUT

- A restore also resets AssistiveTouch, Full Keyboard Access, tracking speed, Auto-Lock and Control Center layout to whatever the backup captured. If the reference backup predates the accessibility bring-up, restoring it undoes that bring-up, and the phone re-enters exactly the "looks fine, ignores input" state this section warns about, until bring-up is repeated.
- A restore does not require re-collecting mouse parameters, since the shared library keys on model and iOS version rather than a specific phone, but re-check `location_crc` via `/device/get` afterward rather than assuming it survived.
- iCloud or Finder restores need bandwidth and an Apple ID per phone; 19 phones restoring in parallel are bounded by the office uplink, a different ceiling from the 21 req/s screenshot cap or the 0.6 ms/px pointer-walk cost measured elsewhere in this document, and worth planning around separately.
- `/device/set` name is unvalidated free text with no schema; a layout-version convention is a discipline the operator enforces, the kernel does not check it.

SCALE

This converts the home-screen coordinate problem from a per-action risk, remembering which phones are safe to target blindly, into a one-time per-phone setup cost paid once at bring-up and repaid only after a restore, the same amortization this section already applies to board binding and mouse-parameter calibration.

Network prerequisites for the video channel

Opening TCP 9911, TCP 17000 and UDP 5353 on the firewall gets packets through the host; it does not get the phone onto a network where those packets can find the PC. mDNS is link-local multicast, and most of the ways "the receiver does not appear in Screen Mirroring" actually happens live below the firewall, on switch and AP configuration the API cannot see or fix. The table below lists the conditions in the order worth checking, and what `/imserver/regmdns` can and cannot do about each.

Condition	Effect on mirroring	Fix within this API	Note
Phones and the rack PC on the same layer-2 broadcast domain	UDP 5353 mDNS is link-local; it does not cross a router or a VLAN boundary. A phone on a guest VLAN or a separate subnet from the PC will never see `air_play_name` in Screen Mirroring, no matter how long you wait.	None. `/imserver/regmdns` re-sends the advertisement on the interface the kernel is already bound to; it cannot make that advertisement reach a different broadcast domain. This is a switch/AP-configuration fix (same VLAN, same subnet), not an API call.	Same on both editions; mDNS behavior is OS/network-stack level, not kernel-specific.
Access-point client isolation	Client isolation (the default on most guest and many corporate SSIDs) blocks device-to-device traffic between Wi-Fi clients on the same AP, even on the same subnet. The receiver is silently invisible; no error appears anywhere in the API.	None from the kernel side. `/imserver/regmdns` re-broadcasts into a network where isolated clients still cannot receive it. Disable client isolation for the SSID the phones use, or put the rack on a dedicated SSID/VLAN without it.	Same on both editions. Worth checking first since it produces the exact symptom this section is written for and leaves no trace in `/config/imserver/get` or any callback.
Multicast filtering and mDNS proxies on enterprise APs/switches	Enterprise gear commonly drops multicast by default, rate-limits it, or requires IGMP snooping with a working querier; some only forward mDNS across APs/VLANs when an explicit mDNS reflector or Bonjour-gateway service is turned on. Symptoms are intermittent: a phone sees the receiver right after `/imserver/regmdns`, then loses it after roaming to another AP.	None. `allow_ip_list` and `mdns_type` (0 = advertise to everyone, 1 = restrict to `allow_ip_list`) control who the kernel is willing to advertise to, not whether the advertisement is allowed to travel through the network fabric. Re-running `/imserver/regmdns` after an AP-side change is a reasonable retest, not a fix on its own.	Same on both editions; this is infrastructure the rack owner does not control if the network is managed by IT.
2.4 GHz vs 5 GHz airtime for N simultaneous mirrors	Once connected, each AirPlay mirror is a sustained unicast TCP stream on port 17000, so N phones mirroring at once is N concurrent video streams contending for the same radio. 2.4 GHz has less usable bandwidth and far more contention from neighboring networks; this rig already shows a large frame-rate gap between its two XRs (about 29 vs about 4.5 new frames/s under motion, per HARDWARE.md) attributable to per-phone link quality, not the kernel.	None directly; `air_play_ratio`, `air_play_fps` and the per-device `/device/airplay/set` overrides trade image quality for bandwidth, but they cannot fix a contended radio. Put the phones on 5 GHz where the hardware supports it, and keep the rack off a busy shared 2.4 GHz SSID.	Same on both editions; scales worse the more phones mirror at once, independent of any plug-in or licence.

Condition	Effect on mirroring	Fix within this API	Note
DHCP reservation for the rack PC	The iMouse Shortcut on every phone has the kernel's LAN IP typed into it by hand during setup. If the PC's DHCP lease changes, or the PC reboots and is handed a different address, every shortcut-based function (clipboard paste, Unicode text entry, album/file transfer, power/brightness/torch/radio switches, `/shortcut/device/ip` itself) breaks on every phone at once, with no warning from the kernel: the phone simply cannot reach the address baked into its shortcut.	None; there is no API call that pushes a new kernel IP into an already-installed shortcut. Recovery is manual on each phone: open Shortcuts, edit the IP field, run it once, allow the prompts again. Give the rack PC's NIC a DHCP reservation (or a static IP) up front so this never has to happen.	Same on both editions; this is the one item on this list that is a rack-wide manual re-provisioning event rather than a one-time network setting, so it is worth fixing before onboarding more phones, not after.

> TRAP **`/imserver/regmdns` is a re-broadcast, not a network fix**
>
> Four of the five rows above are conditions that `/imserver/regmdns` cannot correct: a wrong VLAN, client isolation, a filtered multicast path and a stale phone-side shortcut IP all sit outside the kernel. Calling it repeatedly against one of these will not help; check the network condition first, then call it once to confirm the advertisement goes out. `/device/airplay/connect/all` is the right recovery call once a batch of phones can see the receiver again, so it belongs after a network fix, not instead of one.

Windows host settings that keep 19 boards enumerated

None of the following is an iMouse function. All five are Windows or filesystem settings on the host PC that sit underneath every capability class above, and none of them is queryable through `/config/usb/get` or `/device/get`, only their absence is: a board that flaps between state 2 and state 1 with no cable touched, or a rack that comes up with an empty board list, is usually one of these settings, not a bad board. Set them once per host, not per phone.

Setting	Where to change it	Why it matters on this rig	How to prove it worked
`USB selective suspend, disabled globally and per hub`	Control Panel > Power Options > Change plan settings > Advanced > USB settings > USB selective suspend setting = Disabled, on every power plan in use; then Device Manager > each Generic USB Hub (VID_1A40&PID_0201, three of them chained, per HARDWARE.md) > Properties > Power Management > uncheck “Allow the computer to turn off this device to save power”. Repeat for every USB Root Hub above the three 7-port hubs.	Windows can independently suspend an idle hub or an idle board to save power. The resume sequence is the exact `usb_change` pattern this rig already produces on purpose from `/device/usb/restart`: code 20 (hardware disconnected), then 19 (connected to computer), then 12 (connected to phone), with the board’s `/config/usb/get` state dropping from 2 to 1 (or briefly absent) and back to 2. A fleet watchdog built to catch `usb_change` flapping cannot tell a power-managed hub from a failing one; on a 19-board, three-hub chain there are 19 boards plus at least 3 hub devices that can each be suspended independently.	Poll `/config/usb/get` every few seconds for an idle period (no calls to any device) and confirm no board changes state and no `usb_change` callback fires. Any flap during a period with zero API traffic is a power-management symptom, not a hardware one.

Setting	Where to change it	Why it matters on this rig	How to prove it worked
`Sleep and hibernate, both set to Never`	Power Options > Screen and sleep > set Sleep to Never (and Hibernate, if enabled, to Never) on both the plugged-in and, if this is a laptop, battery profile.	System sleep suspends the whole USB subsystem at once, not one hub at a time: all 19 boards and both AirPlay sessions drop together. Unlike the console dependency below, a sleep/resume cycle is not something the vendor docs describe recovering from automatically, and this project has not tested it; treat every board and every phone as needing to be re-verified after any resume, the same way they are re-verified after a fresh boot.	`/device/get` for every phone shows a live, non-zero `air_pid` and `/config/usb/get` lists all 19 boards at their prior states, with no manual re-cast or rebind required.
`Fast Startup, disabled`	Control Panel > Power Options > Choose what the power buttons do > Change settings that are currently unavailable > uncheck “Turn on fast startup”.	Fast Startup hibernates the kernel session instead of fully shutting it down, so a “shutdown” does not reinitialize the USB controllers the way a cold boot does. On a chain of three 7-port hubs with 19 of 21 ports populated, that raises the odds of boards re-enumerating with different PIDs or not all coming back, which matters because this rig addresses boards by PID (`vid|pid` in `/device/usb/restart`), not by physical port. This is standard Windows/USB behavior, not something either API doc documents, so verify it on this host rather than assuming it.	After a full shutdown and cold power-on, `/config/usb/get` lists the same 19 `uid` values as before shutdown, at hub/port positions matching the recorded topology.
`IMConsole.exe running at every boot, with a restart-on-exit rule`	Task Scheduler: a task triggered “At log on” (or “At startup” if the host auto-logs in), action = launch `IMConsole.exe`, with an action to relaunch it if the process exits. This is not the same as a Windows service Recovery tab: `IMConsole.exe` is the console GUI, not the `IMService.exe` kernel service, so native service recovery only covers the kernel half.	Directly measured on this rig: with only `IMService.exe` running, `/device/get` read `state` 0 for both phones and `/config/usb/get` returned an empty board list; once `IMConsole.exe` was started and logged in, both phones showed `state` 1 and all 19 boards listed. The API alone does not bring the rack up after a reboot.	`/config/usb/get` returns 19 boards and `/device/get` shows non-zero `state` for every phone that was mirroring before reboot, within a few minutes of the host coming up unattended, with nobody at the keyboard.

> `OPEN` **Headless console launch is untested**
>
> Every observation of the console bringing the rack up on this rig happened with an interactive, logged-in desktop session. Task Scheduler can launch `IMConsole.exe` without anyone logged in (“Run whether user is logged in or not”), but a GUI app started that way may not get a renderable window, and it is not verified here whether the rack still comes up in that mode. Test it once on this host: reboot with no one logged in, wait, then poll `/config/usb/get` and `/device/get` remotely. If the board list stays empty, fall back to auto-login plus an on-screen launch.

Disk budget and rotation for a frame archive

`/pic/screenshot` with `binary=true` returns the raw BMP the decoder already holds: measured 1,620,054 bytes per frame on this rig's 500×1080 stream (500 × 1080 × 3 bytes plus a 54-byte header), call it 1.6 MB. With `save_path` set, the kernel writes that BMP to disk and returns the path instead of the bytes; the documented response is `code`, `id`, `image` (the path) and `message`, nothing about free space, retention or overwriting. A frame archive that polls all 19 boards will fill a disk unless something outside the API manages it.

Capture cadence, per phone	Aggregate rate, 19 phones	Per hour	Per day
1 frame / s (continuous)	~30.4 MB/s	~109 GB	~2.6 TB
1 frame / 10 s	~3.0 MB/s	~11 GB	~263 GB
1 frame / 60 s (health-check only)	~0.5 MB/s	~1.8 GB	~44 GB

The JPEG variant (`binary=true&jpg=true`) shrinks each frame to roughly 96 KB but costs a kernel-side re-encode measured at 40 ms and a hard ceiling around 21 requests per second on this host; that ceiling is a property of the kernel doing the re-encoding, so budget archival JPEG capture across all 19 phones against one shared ceiling, not 21 per phone. Raw BMP has no such cap and is the right default for an archive; reserve JPEG for a slow or remote link.

> TRAP **save_path has no rotation, and the filename is not one either**
>
> Left alone, `save_path` writes one file per call forever; nothing in either API doc deletes, caps or ages out old frames. The filename the kernel generates (for example `1128935437.bmp` in the vendor's own example) is an opaque kernel-assigned number, not a timestamp or sequence you can sort or expire by. Build retention outside the API: a scheduled task that deletes files older than a fixed window, or a watchdog that stops capture below a free-space threshold, keyed off your own capture-time filename rather than the one the kernel hands back.

Clock sync for evidence timestamps

Neither `/pic/screenshot` nor `/config/usb/get` nor `/device/get` nor the `usb_change` callback carries a server-side timestamp field; every one of the response bodies checked in these docs is limited to status, code, id and the payload itself. The only time base for an archived frame or a logged board flap is the host clock at the moment your own code made the call. If that clock drifts, or resets on a sleep/resume or a Fast-Startup hybrid boot, an evidence trail built from this archive cannot be lined up against another system's logs, or even against itself across a restart.

- Enable the Windows Time service (`w32tm`) against a reliable NTP source and confirm it is running, rather than the default best-effort domain/workgroup sync.
- Stamp every archived frame and every logged callback with your own capture-side timestamp at the moment of the call; do not depend on file creation time alone, since a restore, copy or antivirus scan can rewrite it.
- Re-check sync after any change to the sleep/hibernate/Fast Startup settings above: a resume from sleep or hibernate is a common source of a clock jump on hosts that were not already NTP-disciplined.

SECTION 14 · APPENDIX

Reference tables

Six tables to grep rather than read start to end: every `data.code` the kernel returns, the 24 Shortcut function ids with their XP paths, the full Phonefarm-to-XP function map, the hotkey vocabulary `send_key` accepts, the phone models and mirror-pixel geometry this rig actually measured, and the ports and processes running on the PC.

ERROR CODES	SHORTCUT IDS	MIRROR GEOMETRY	API PORT
0–41	**24**	**500 x 1080**	**9911**
data.code table, XP kernel	ids 9, 12, 14 undocumented or withdrawn	imgw x imgh, both XRs on this rig	TCP, HTTP + WS, no authentication

A. Error codes (data.code)

Every XP call answers HTTP `status 200` once the kernel accepts the request; the real result is `data.code` below, with `data.message` carrying a Chinese string that should never be pattern-matched. A bare Phonefarm-style function name (no leading slash, two segments) never reaches this table at all: it fails one level up with HTTP `status 500`, "List index out of bounds," which is a routing rejection, not a code 41.

Code	Meaning	Usual cause or remedy
0	success	—
1	id does not exist	Check the id against `/device/get`; typo, or the device never appeared.
2	device not bound to hardware	Run `/device/set` with vid and pid, or bind it from the vendor console.
3	hardware not connected	Board not seen on the USB bus. Check `/config/usb/get` state, the cable, the hub port.
4	collection failed	Retry mouse-parameter collection; confirm board and phone are both connected.
5	a device is already collecting	Only one device can run `/device/collection/mouse` at a time. Wait or stop it (cmd=2).
6	device is not collecting mouse	Start collection with cmd=1 before sending cmd=2 or a status query.
7	device id does not exist, or device is online	`/device/del` and `/device/group/del` refuse an online device; disconnect it first.
8	scan code opened successfully	Calibration progress notice, not an error.
9	entered the collection page	Calibration progress notice, not an error.
10	hardware id is empty	The `id` parameter is missing from the request.
11	no mouse parameters	Device has no calibration. Set a `location_crc` from the library, or run a collection.

Code	Meaning	Usual cause or remedy
12	`hardware and phone connecting`	Progress notice; also the `usb_change` event code for "phone attached to board."
13	`hardware not connected to phone`	Board is on the PC (state 1) but no phone is cabled to it (not state 2).
14	`start working`	Calibration progress notice, not an error.
15	`no coordinates`	A call that needed a screen position received none.
16	`hardware not working, unknown error`	Board-level fault. Power-cycle the board, reseat the cable, or try `/device/usb/restart`.
17	`retry on failure`	Calibration is retrying automatically.
18	`failure stop`	Calibration gave up after retries.
19	`hardware and computer connecting`	Progress notice; also the `usb_change` event code for "board connected to PC."
20	`hardware disconnected from computer`	Board dropped off the USB bus. Check the hub and cable.
21	`at work`	Calibration in progress.
22	`device online`	The operation needs the device offline first (for example, delete).
23	`hardware unbound device`	The board has no device record bound to it.
24	`hardware not turned on`	Board did not complete its handshake with the kernel.
25	`device is offline`	Not mirroring. Screenshot, OCR and input calls fail until it reconnects.
26	`screenshot failed`	Device dropped mid-capture; retry.
27	`path does not exist`	`save_path`, or a file path inside a Shortcut call, is invalid.
28	`plugin not connected`	OCR or image-search plug-in not installed. This is the code every `/pic/ocr` call returns on this rig.
29	`plugin interface call timed out`	The plug-in is overloaded or hung.
30	`call timeout`	The `outtime` budget elapsed before a phone-side Shortcut replied.
31	`OCR did not recognize the text`	No text in the search region met the similarity threshold.
32	`no text found`	`/pic/find-text` found no match.
33	`fail`	Generic failure with no more specific code.
34	`remove the device from the group first`	`/device/group/del` refuses a non-empty group.
35	`file list empty, or none of the files exist`	A Shortcut file or album transfer was given a bad path or filename.
36	`parameter error`	A required field is missing or malformed.

Code	Meaning	Usual cause or remedy
37	`login failed`	`/config/user/login` credentials rejected.
38	`registration failed`	Account registration rejected.
39	`password reset failed`	Account password reset rejected.
40	`failed to send verification code`	Account verification-code request rejected.
41	`incompatible commands`	The path is not valid for this call or this kernel build. Not the same failure as the HTTP 500 a bare Phonefarm function name gets; that is rejected before it reaches this table.

B. Shortcut function ids 1 to 24

All 24 need the iMouse Shortcut app installed on the phone with the kernel's IP address entered, run once with every permission allowed, and bound to an AssistiveTouch key from the vendor console. Ids 9, 12 and 14 are gaps: 9 is never documented (the vendor table skips 8 to 10), and 12 (send SMS) and 14 (read text aloud) are struck through as withdrawn in the source doc. XP does not gate any of these by edition, unlike the Phonefarm doc's "professional version only" label.

Id	Function	XP path	Parameters	Returns
1	Get photo album list	`/shortcut/album/get`	id (single), album_name (blank = Recents), num (default 10), outtime ms	list[{album_name, name, ext, size, create_time}]
2	Download photos/videos from album	`/shortcut/album/down`	id (single), list[{album_name, name, ext}], zip 0/1, outtime ms	Status only; files land in Shortcut\Media
3	Delete photos/videos from album	`/shortcut/album/del`	id (comma list), list[{album_name, name, ext}], outtime ms	Latest 10 album entries
4	List files in a phone directory	`/shortcut/file/get`	id (single), path (default My iPhone root), outtime ms	list[{name, ext, size, create_time}]; empty ext = folder. iOS 15+
5	Download files from phone	`/shortcut/file/down`	id (single), path, list[{name, ext}], zip 0/1, outtime ms	Status only; files land in Shortcut\File. iOS 15+
6	Delete files on phone	`/shortcut/file/del`	id (comma list), path, list[{name, ext}], outtime ms	Latest 10 directory entries. iOS 15+
7	Upload photos/videos to album	`/shortcut/album/upload`	id (comma list), files [PC paths], album_name (blank = Recents), zip 0/1, outtime ms	Latest 10 album entries
8	Upload files to phone	`/shortcut/file/upload`	id (comma list), path (default root), files [PC paths], zip 0/1, outtime ms	Latest 10 directory entries. iOS 15+
9	Not documented	`none`	The vendor table skips from 8 to 10.	—

Id	Function	XP path	Parameters	Returns
10	Send text to phone clipboard	/shortcut/clipboard/set	id (comma list), text, sleep seconds (delay before returning, some phones paste slowly), outtime ms	Status only
11	Read phone clipboard	/shortcut/clipboard/get	id (single), outtime ms	{text}
12	Send SMS	none	Struck through in the vendor doc; withdrawn.	—
13	Open a URL on the phone	/shortcut/exec/url	id (comma list), url, outtime ms	Status only
14	Read text aloud	none	Struck through in the vendor doc; withdrawn.	—
15	Set screen brightness	/shortcut/switch/bril	id (comma list), state float, 0 < state < 1, outtime ms	Status only
16	Flashlight on	/shortcut/switch/torch	id (comma list), state=1, outtime ms	Status only
17	Flashlight off	/shortcut/switch/torch	id (comma list), state=0, outtime ms	Status only
18	Airplane mode on	/shortcut/switch/flight	id (comma list), state=1, outtime ms	Status only; kills the AirPlay video channel, USB control unaffected
19	Airplane mode off	/shortcut/switch/flight	id (comma list), state=0, outtime ms	Status only
20	Cellular data on	/shortcut/switch/cdpd	id (comma list), state=1, outtime ms	Status only
21	Cellular data off	/shortcut/switch/cdpd	id (comma list), state=0, outtime ms	Status only
22	Wi-Fi on	/shortcut/switch/wlan	id (comma list), state=1, outtime ms	Status only
23	Wi-Fi off	/shortcut/switch/wlan	id (comma list), state=0, outtime ms	Status only; kills AirPlay mirroring, USB control unaffected
24	Get phone IP address	/shortcut/device/ip	id (single), state 0 = LAN / 1 = extranet, outtime ms	{text: ip}. State 0 (LAN) is an XP-only addition; the Phonefarm original only returned the external IP.

XP also exposes two Shortcut paths with no Phonefarm id at all: `/shortcut/album/clear` (empty an album) and `/shortcut/switch/device` (state 0 shuts the phone down, state 1 restarts it, iOS 15+).

C. Phonefarm to XP function mapping

Every Phonefarm function name and its XP path equivalent, grouped by area. "None" means XP has no equivalent call; "partial" means the XP path covers only part of what the Phonefarm function did.

Device and group management

Phonefarm function	XP path	Note
`set_dev`	`/device/set`	id becomes comma-list capable for name/gid; vid+pid binding stays one device per call. XP recommends location_crc over raw location and returns the ids that were set, not the full record.
`del_dev`	`/device/del`	id accepts a comma list. Online devices cannot be deleted (code 7). Broadcasts dev_delete.
`set_group`	`/device/group/set`	id "0" creates a new group. Returns data.list[{id,name}] and broadcasts group_change.
`del_group`	`/device/group/del`	id accepts a comma list; the group must be emptied first (code 34). Broadcasts group_delete.
`get_device_list`	`/device/get`	Optional id filter. Result is a data.list array, not an object keyed by deviceid. Adds imgw/imgh, air_pid, uid, gname, uver and the per-device air_* casting overrides.
`get_group_list`	`/device/group/get`	Optional id filter. data.list[{id,name}]. XP adds /device/group/get-dev to list a group's devices.

Hardware and model discovery

Phonefarm function	XP path	Note
`get_usb_list`	`/config/usb/get`	data.list[{vid,pid,uid,ver,state}]. state distinguishes board-only (1) from board+phone (2), which Phonefarm could not report.
`get_devicemodel_list`	`/config/devicemodel/get`	XP's own doc elides the response as "too many to list." Field shape unconfirmed on this rig.

Screenshot and video

Phonefarm function	XP path	Note
`get_device_screenshot`	`/pic/screenshot`	id single only. isjpg becomes jpg, and the default flips from JPEG to BMP. binary works over plain HTTP on XP (WS-only in Phonefarm) and returns raw bytes. Adds rect crop and save_path; no gzip option.
`loop_device_screenshot`	`none`	XP has no server-push frame stream. Poll /pic/screenshot?binary=true instead (measured 2 ms p50, 84 req/s on loopback).

Mouse

Phonefarm function	XP path	Note
`click`	`/mouse/click`	button "left"/"right" becomes numeric "1".."8". id accepts a comma list, so one call drives every listed phone. Coordinates are imgw x imgh mirror pixels, confirmed by hardware test.
`swipe`	`/mouse/swipe`	length becomes len; for becomes stepping (spelled "steping" in the vendor's own example). Adds step_sleep and brake. id accepts a comma list.
`mouse_up`	`/mouse/up`	button numeric; id accepts a comma list.
`mouse_down`	`/mouse/down`	button numeric; id accepts a comma list. A real drag is down, streamed moves, then up.
`mouse_move`	`/mouse/move`	id accepts a comma list. Absolute mirror-pixel coordinates; latency scales about 0.6 ms per pixel of travel.
`mouse_reset_pos`	`/mouse/reset`	id accepts a comma list. Measured to be the only call that clears a silently stuck click state.
`mouse_wheel`	`/mouse/wheel`	length becomes len (max 127). XP adds left/right in addition to up/down.

Keyboard

Phonefarm function	XP path	Note
`key_release_all`	`/key/upall`	id accepts a comma list; no other parameters.
`key_up`	`/key/up`	Single key only, ASCII or a named key.
`key_down`	`/key/down`	Single key only. XP's parameter table also lists fn_key, though its own examples wrongly show /key/sendkey.
`send_key`	`/key/sendkey`	key types ASCII text; fn_key fires a named hotkey. XP's hotkey vocabulary is much larger than Phonefarm's (table D).

Kernel and AirPlay service

Phonefarm function	XP path	Note
`restart`	`/imserver/restart`	No parameters either side. Drops every mirroring session.
`get_airplaysrvnum`	`none`	XP runs one receiver, not a count. The kernel version is only available from the im_connect WS callback.
`set_airplaysrvnum`	`none`	No multi-receiver concept in XP.

Mouse-parameter calibration

Phonefarm function	XP path	Note
`mouse_collection_open`	`/device/collection/mouse`	Call with cmd=1. XP adds cmd=0 to query status without starting or stopping.
`mouse_collection_close`	`/device/collection/mouse`	Call with cmd=2.

Phonefarm function	XP path	Note
mouse_collection_cfg	/device/collection/mouse	Progress reports through the collection_mouse callback's data.code, not a location string in the reply.
save_dev_location	/device/collection/mouse/save	id single, describe unchanged. XP's own doc example echoes the wrong fun name in its response.
del_dev_location	none	XP has no call to remove a library entry; only adding one is supported.

Text input

Phonefarm function	XP path	Note
send_text	/shortcut/clipboard/set (partial)	No single equivalent. The documented route is clipboard/set followed by /key/sendkey fn_key=WIN+v to paste, with no completion callback (Phonefarm's send_text_ok does not exist in XP).

Shortcuts

Phonefarm function	XP path	Note
shortcut (generic dispatcher, id 1–24)	/shortcut/*	XP has no catch-all call; every id is its own path. devlist becomes a comma-separated id, the parameter JSON string becomes native typed fields, outtime moves from seconds to milliseconds, and retdata (a JSON string) becomes typed data.list or data.text.
shortcut#1 through #24	/shortcut/album/*, /shortcut/file/*, /shortcut/clipboard/*, /shortcut/exec/url, /shortcut/switch/*, /shortcut/device/ip	One-to-one per id. Full parameter and return shape in table B.

Picture recognition

Phonefarm function	XP path	Note
find_image	/pic/find-image	A single base64 img becomes an img_list array (base64 or local file path); the 4-corner rect becomes a flat [left, top, right, bottom]. Result shape changes to a list of {index, centre, rect, code}.
find_image_ex	/pic/find-image	img_list, all, rect and similarity map directly. Phonefarm's repeat (find duplicates) becomes same, but only on the OpenCV variant /pic/find-image-cv.
ocr	/pic/ocr	4-corner rect becomes [left, top, right, bottom]; result fields become {text, similarity, centre, rect}. On this rig the call answers but data.code is 28, plug-in not connected.
ocr_ex	/pic/ocr-ex	Same parameters, heavier engine. Phonefarm gates this to its professional edition; XP does not gate by edition, but the same plug-in dependency applies.

Phonefarm function	XP path	Note
find_multi_color	/pic/find-multi-color	Colour encoding changes from relative "RRGGBB-tol" pairs plus a separate scan region and offset list, to a flat "x|y|RRGGBB-tol" list of absolute points. No scan-direction option in XP.
find_multi_color_ex	/pic/find-multi-color	list of configs and all map directly; XP adds same to also find every identical match.

AirPlay casting and USB reset

Phonefarm function	XP path	Note
auto_connect_screen	/device/airplay/connect	id accepts a comma list. No force flag; call /device/airplay/disconnect first if needed. Progress arrives on the airplay_connect_log callback.
auto_connect_screen_all	/device/airplay/connect/all	No parameters on XP (Phonefarm oddly took a deviceid). Casts every offline device.
discon_airplay	/device/airplay/disconnect	id accepts a comma list.
save_autoscreen_point	none	XP's auto-cast needs no captured coordinate; there is no capture call.
restart_usb	/device/usb/restart	Addressing changes from a deviceid to "VID|PID" of the board, comma-list capable.
set_usb_autoairplay	/config/imserver/set (partial)	Becomes a single global auto_connect flag rather than a per-device setting.
get_usb_autoairplay	/config/imserver/get (partial)	Read the same global flag.
set_airplay_mode	/device/airplay/set (partial)	Phonefarm's three named presets become explicit per-device knobs (air_ratio, air_fps, air_refresh, air_img_fps, air_audio), or the equivalent global fields in /config/imserver/set.
get_airplay_mode	/device/get (partial)	Read the per-device air_* fields from the device record, or the globals from /config/imserver/get.
save_restart_point	none	XP's /device/restart needs no captured coordinate.
restart_device	/device/restart	id accepts a comma list. Progress via the restart_log callback; alternatives are /shortcut/switch/device and the CTRL+ALT+SHIFT+WIN+r hotkey.

WebSocket callbacks

Phonefarm callback	XP callback	Note
dev_connect	dev_connect	Same name. The device record now arrives inside data.list with XP's renamed and added fields.
dev_disconnect	dev_disconnect	Same name. On this rig, means the AirPlay session dropped; the USB control channel may still be up.

Phonefarm callback	XP callback	Note
`dev_rotate`	`dev_rotate`	Same name. imgw/imgh swap meaning when the phone rotates.
`usb_change`	`usb_change`	Same name. Payload changes from {state, vpid} to data.list[{vid,pid,uid,ver,state}] with the event type in data.code (12 phone attached, 19 board connected, 20 board disconnected).
`mouse_collection_cfg_ret`	`collection_mouse`	Renamed. Carries data.code instead of a location string, which must instead be read back from /device/get.
`collection (page opened)`	`collection_mouse`	Folded into the same callback, as codes 8 and 9.
`binary loop-screenshot frames`	`none`	No binary WS push exists in XP.
`send_text_ok`	`none`	No completion signal for the clipboard-and-paste route XP uses instead.
`collection page URL`	`none (undocumented)`	XP's console shows a QR code instead of a documented phone-side URL.

D. Named keys and hotkey combos (send_key)

`/key/sendkey` takes either `key` (ASCII text to type, or a single named key) or `fn_key` (a combination hotkey) in one call, never both; leave the other field blank. Both are ASCII only, no Chinese or other Unicode; for non-ASCII text use `/shortcut/clipboard/set` followed by `fn_key=WIN+v` to paste.

Named single keys (key parameter, and /key/up, /key/down)

- `BACKSPACE`, `SPACE` (a blank single space), `TAB`, `ENTER`, `UpArrow`, `DownArrow`, `LeftArrow`, `RightArrow`, plus any single ASCII letter, digit or punctuation character.

Hotkey combinations (fn_key parameter)

Combo	Effect
`WIN+h`	Home screen
`Home`	Home screen. Fixes an iOS 26.1+ issue; needs kernel ≥ 2026.2.27
`TAB+b`	Back (not supported by every app)
`AppSwitch`	App switcher
`ControlBar`	Control Center
`NoticeBar`	Notification Center
`TAB+l`	Lock screen
`OPENKeyboard`	Open the on-screen keyboard
`shift+win+3`	Screenshot
`shift+win+4`	Screenshot and edit
`CTRL+ALT+SHIFT+WIN+r`	Restart the phone

Combo	Effect
`CTRL+ALT+WIN+p`	Toggle pass-through mode (full keyboard on/off)
`WIN+c`	Copy
`WIN+v`	Paste
`WIN+a`	Select all
`WIN+x`	Cut
`SwitchIme`	Switch input method
`MuteSoundSwitch`	Mute toggle
`VolumeUp`	Volume up
`VolumeDown`	Volume down
`Play/Pause`	Media play/pause

> OPEN **PAGEUP, PAGEDOWN, ESC, FN, F1–F12**
>
> Phonefarm's doc lists these as named keys; the XP hotkey table does not repeat them. This rig runs the XP kernel, not Phonefarm, so treat these as unconfirmed until probed against this build rather than assuming they work.

E. Supported iPhone models and mirror geometry

Confirmed on this rig: two iPhone XR, internal model `iPhone11,8`, iOS 18.7.10, logical width x height 414 x 896 points, mirror `imgw` x `imgh` 500 x 1080 pixels on both phones, `air_pid` 22236, `location_crc` B0DBD800 (a shared-library calibration record, not a per-device one).

MODEL	LOGICAL SIZE	MIRROR SIZE
iPhone11,8	**414 x 896**	**500 x 1080**
iPhone XR, iOS 18.7.10	iOS points, from /device/get width/height	imgw x imgh, the click/move coordinate space

The full model and calibration library is served by `get_devicemodel_list` (Phonefarm) and `/config/devicemodel/get` (XP); the XP doc elides its own response as "too many to list," so the table below is what the reference docs actually show plus the logical sizes their examples span, not a live probe of this kernel's library.

Logical width x height (points)	Doc example device(s)	Note
`320 x 480`	—	Listed in the digest of sizes the model library spans; no worked example in the docs read for this handoff.
`320 x 568`	—	Same as above.
`375 x 667`	iPhone 6s (iPhone8,1)	Doc example device record. Same logical size appeared at two different imgw/imgh mirror sizes in the equipment-related doc's own two examples (see note below).

Logical width x height (points)	Doc example device(s)	Note
`414 x 736`	iPhone 7 Plus (iPhone9,2)	Doc example device record.
`375 x 812`	—	Listed in the digest of sizes the model library spans.
`414 x 896`	iPhone XR (iPhone11,8)	Confirmed live on this rig, both devices.
`390 x 844`	—	Listed in the digest of sizes the model library spans.
`428 x 926`	—	Listed in the digest of sizes the model library spans; the doc's own model table tops out around the iPhone 13 / SE (2022) generation, so anything newer depends on the running kernel's own library.

> `TRAP` **imgw x imgh is not a per-model constant**
>
> The equipment-related XP doc's own two worked examples show the identical iPhone 6s logical size, 375 x 667, captured at `imgw` / `imgh` 750 x 1334 in one record and 460 x 818 in another. The mirror-pixel size is a property of the live AirPlay stream and its cast settings (`air_ratio` , `air_fps` , and so on at connect time), not of the phone model. Always read `imgw` / `imgh` from `/device/get` for the specific device record being addressed, never assume it from the model name or the logical size above. Every `/mouse/click` , `/mouse/move` and `/pic/screenshot` coordinate is in this mirror-pixel space, not the logical points space.

F. Ports and processes on the PC

Port / transport	Process	Purpose	Status on this rig
`9911 TCP`	IMService.exe	HTTP + WebSocket XP API, both under `/api`	Open. The only API surface this rig answers on.
`3362 UDP`	IMService.exe	Internal kernel traffic	Open
`17000 TCP`	AirplayMirroring.exe	AirPlay mirroring session data	Open. 4 established sockets from the 2 phones, measured 10 Sep 2026.
`47010 TCP`	AirplayMirroring.exe	AirPlay mirroring, secondary	Open
`5353 UDP`	mDNSResponder.exe	Bonjour; advertises the PC as AirPlay receiver "iMouseXP-5429"	Open
`5356 UDP`	mDNSResponder.exe	Bonjour	Open
`18082`	mDNSResponder.exe	Bonjour	Open
`9912 TCP`	—	The Phonefarm-edition HTTP API port; not used by the XP kernel installed on this rig	Closed, nothing listening

Processes. `IMService.exe` is the kernel; it must be running for the API to answer at all. `IMConsole.exe` is the vendor console UI; on this rig, with it not running, `/device/get` reported both phones state 0 and `/config/usb/get` returned an empty board list even though `IMService.exe` was up, so the console has to be started for mirroring and board enumeration to come up. `AirplayMirroring.exe` mirrors the AirPlay video; its PID (22236, live on this rig) is what `/device/get`'s `air_pid` field echoes as proof a phone is actually mirroring, not just bound.

> TRAP **no authentication on 9911**
>
> Any host on the LAN can call `http://<pc>:9911/api` with no credential of any kind. Firewall 9911 and 17000 to trusted hosts; do not expose this PC to an untrusted network, and never add a bearer token or API key expecting the kernel to check it.